

ISRO Scientist/Engineer 'SC' (Computer Science) — Master Question Bank

Sequential topic-wise MCQ study material with answers, explanations & rapid revision round

ISRO Scientist/Engineer 'SC' (Computer Science) — Master Question Bank

One sequential, start-to-end study read: every topic compressed to essentials + exam-style MCQs with answers and explanations, followed by a rapid revision round. Built from the compressed KnowledgeGate revision notes (PDSA, OS, DBMS, CN & Cybersecurity, Aptitude) plus newly-added modules (Digital Logic & Computer Organization, Theory of Computation, Compiler Design, Software Engineering) to match real ISRO paper coverage.

How to Use This Document

- First pass (this book, front to back):** Read every module in order. Each chapter is short concept notes (what you actually need, not a textbook) followed by a Questions section. Attempt every MCQ *before* looking at the answer — cover the answer line with your hand/a piece of paper if needed. The explanation ("Why:") is written so the chapter still teaches you the concept even if you get the question wrong.
- TRAP callouts:** Any line starting with **TRAP:** or a question tagged **[Trap]** flags a specific, real way students lose marks on that exact point — a common wrong assumption, a mixed-up pair of terms, an off-by-one, a "looks obvious but isn't" numerical. These are worth re-reading even if you already know the topic.
- Second pass (last few days before the exam):** Use the **Rapid Revision Round** at the very end of this document — it is a dense, answer-key-style sweep of the highest-yield questions across every module, meant to be read in one or two sittings to refresh everything without re-reading full chapters.
- This is Computer Science paper-specific, not generic GATE prep.** Depth and question style are calibrated against the actual 2023 and 2025 ISRO Scientist/Engineer 'SC' (Computer Science) papers (see below) — topic emphasis, numerical-vs-conceptual mix, and trap style all follow what ISRO has actually asked.

Exam Pattern (from the 2023 & 2025 official papers)

	Part A — Discipline Specific	Part B — Aptitude/Ability
Questions	80	15
Marks	80 (1 mark each)	20 (mix of 1- and 2-mark questions)
Total duration	2 hours (120 minutes) for both parts combined	
Negative marking	-0.33 for every wrong answer, across both parts	
Question format	Single-correct MCQ, 4 options (a)–(d)	

- Part A is **~80% of the paper** and covers the full CS core curriculum: Programming/Data Structures/Algorithms, Digital Logic & Computer Organization, Theory of Computation, Compiler Design, Operating Systems, DBMS, Computer Networks & Cybersecurity, and Software Engineering — roughly in proportion to how a 4-year CS degree weights them, with a visible lean toward **Data Structures & Algorithms, DBMS, and Theory of Computation/Automata** in both papers reviewed.
- Part B (Aptitude) mixes quantitative (percentage/ratio/profit-loss-style), data interpretation (worth 2 marks — read the table/graph once, extract all the numbers you'll need in one pass), and verbal/non-verbal reasoning.
- TRAP:** Don't spend your prep time disproportionately on the module you personally find *interesting* — spend it proportional to Part A's real weight (DS/Algo, DBMS, TOC first; Software Engineering and Digital Logic are real but smaller slices).

The Negative-Marking Math (memorize this — it changes how you should guess)

With 4 options, -0.33 for a wrong answer, +1 for a right one:

Your confidence when you attempt	Expected value of attempting	Verdict
Pure blind guess (all 4 options equally likely)	$0.25(+1) + 0.75(-0.33) = +0.0025$	Roughly break-even — attempting doesn't really cost you, but doesn't help either
Eliminated 1 wrong option (down to 3)	$0.33(+1) + 0.67(-0.33) = +0.11$	Worth attempting
Eliminated 2 wrong options (down to 2)	$0.50(+1) + 0.50(-0.33) = +0.335$	Definitely attempt
Genuinely no idea, can't eliminate anything	≈ 0	Skip is fine, but guessing is not actually risky — it is <i>not</i> the disaster many candidates assume

- TRAP:** Many candidates over-correct for negative marking and skip too much — the math shows a **pure random guess is not a losing bet**; it only turns clearly bad if you're guessing based on a *misremembered* fact that biases you toward a wrong option more than 25% of the time (e.g., you're confusing two similar-sounding terms and would consistently pick the wrong one). If you can eliminate even one option, always attempt.
- Practical rule: **skip only when a question is testing a concept you have zero anchor on** (not even elimination). If you recognize the topic at all, attempt it.

Time Budgeting

- 120 minutes ÷ 95 questions ≈ 75 seconds/question on average — but not uniform. Budget roughly:
- Part A 1-mark conceptual/definition MCQs: ~30–40 seconds each (you either know it or you don't; don't stall).
- Part A numerical/trace-through MCQs (recurrence solving, automaton simulation, scheduling Gantt charts, normalization checks): ~90–120 seconds each.
- Part B 2-mark DI questions: budget slightly more since the first one requires reading the whole table/graph (subsequent questions on the same table are then fast).
- Two-pass strategy:** first pass through the whole paper answering only what you know cold or can solve in under ~45 seconds; second pass returns to flagged numerical/trap questions with the remaining time. Never let one stubborn numerical eat 5+ minutes on a first pass — mark it and move on.
- OMR discipline: if using an OMR sheet, bubble in small batches (e.g., every 10 questions) rather than one-by-one or all-at-the-end — one-by-one is slow, all-at-the-end risks running out of time to transfer answers at all.

Cross-Subject Trap Patterns (watch for these in every module below)

- **"NOT" / "EXCEPT" / "all of the following ... except"** — the single most common way a confident candidate loses an easy mark is reading the stem as a positive question. Circle or mentally flag the negation word before reading the options.
 - **Best-case vs. worst-case vs. average-case** — sorting/searching complexity, scheduling turnaround time, hashing lookup time: always check *which* case the question is actually asking about.
 - **Similar-sounding term pairs** that exams deliberately swap: throughput vs. turnaround time; deadlock prevention vs. avoidance vs. detection; 1NF/2NF/3NF/BCNF boundary cases; DFA vs. NFA equivalence claims; compile-time vs. link-time vs. load-time vs. run-time binding; static vs. dynamic scoping; verification ("are we building the product right") vs. validation ("are we building the right product").
 - **Off-by-one / boundary numericals** — number of edges in a complete graph, number of leaf nodes in a full binary tree, number of states in a minimal DFA, number of comparisons in a sort — these are exactly where **"TRAP:"** callouts appear throughout this book; read them even if the surrounding concept feels familiar.
 - **Formula-recall traps** — a question gives you *just enough* of a formula's variables to tempt you into using a memorized special case (e.g., a balanced-tree height formula) when the general-case formula is what's actually asked.
 - **Units and given data mismatches** — MHz vs. GHz, bits vs. bytes, KB (1000) vs. KiB (1024) — computer organization and networking numericals are a common place candidates do the math right but the units wrong.
-

Module Map

This book proceeds in the following sequence — foundational hardware/programming first, theory next, then systems (OS/DBMS/networks), software engineering, and finally the separate Aptitude paper:

1. Digital Logic & Computer Organization
2. Programming, Data Structures & Algorithms (PDSA)
3. Theory of Computation & Automata
4. Compiler Design
5. Operating Systems
6. Database Management Systems (DBMS)
7. Computer Networks & Cybersecurity
8. Software Engineering
9. General Aptitude & Reasoning (Part B of the real exam)

Followed by the **Rapid Revision Round** — a compressed final sweep across all nine modules.

1.1 Number Systems, Codes & Binary Arithmetic

1.1.1 Number System Conversions

- Positional number systems: **Binary (base 2)**, **Octal (base 8)**, **Decimal (base 10)**, **Hexadecimal (base 16)**. Each octal digit = 3 bits; each hex digit = 4 bits — group bits in 3s or 4s from the **binary point outward** (right-to-left for the integer part, left-to-right for the fractional part) to convert quickly.
- Fractional binary → decimal**: $\text{digit} \times 2^{-(\text{position})}$, position starting at 1 after the binary point.
- TRAP**: When grouping bits for octal/hex conversion, pad the *leftmost* group of the integer part and the *rightmost* group of the fractional part with zeros — padding the wrong side silently changes the value.

1.1.2 Signed Number Representations

Representation	Range (n bits)	Zero	Key property
Sign-Magnitude	$-(2^{(n-1)}-1)$ to $+(2^{(n-1)}-1)$	Two zeros (+0, -0)	MSB = sign bit only; simple but arithmetic is awkward
1's Complement	$-(2^{(n-1)}-1)$ to $+(2^{(n-1)}-1)$	Two zeros (+0, -0)	Negate by flipping all bits; needs end-around carry in addition
2's Complement	$-2^{(n-1)}$ to $+(2^{(n-1)}-1)$	One zero	Negate by flipping all bits + 1; used by virtually all real hardware

- TRAP**: 2's complement has **one extra negative number** with no positive counterpart (e.g., 8-bit range is -128 to $+127$) — a classic "what is the range" numerical trap.
- Overflow in 2's complement addition**: occurs when two operands of the **same sign** produce a result of the **opposite sign**. (Adding a positive and a negative number can never overflow.)

1.1.3 Binary Codes

Code	Property	Use
BCD (8421)	Each decimal digit encoded separately in 4 bits	Decimal-facing hardware (calculators, displays)
Excess-3	BCD + 3 (0011); self-complementing (9's complement = bitwise NOT)	Simplifies BCD arithmetic circuits
Gray Code	Only one bit changes between consecutive values	Rotary encoders, Karnaugh maps — avoids glitches from multi-bit transitions
ASCII	7-bit character encoding	Text representation

- Binary → Gray**: MSB unchanged, each subsequent bit = XOR of that bit and the previous *binary* bit.
- TRAP**: Gray code's defining property is **exactly one bit change between adjacent values**, not "fewer bits than binary" — a commonly confused distractor.

Questions

Q1. Convert binary 101110 to its octal equivalent.

A. 56 B. 66 C. 46 D. 65

Answer: A. Why: Group from the right in 3s: 101 110 → 5, 6 → 56_8 .

Q2. The range of values representable in 2's complement using 8 bits is:

A. -127 to $+127$ B. -128 to $+127$ C. -128 to $+128$ D. -127 to $+128$

Answer: B. Why: 2's complement has one zero and one extra negative value; range is $-2^{(n-1)}$ to $+(2^{(n-1)}-1) = -128$ to $+127$ for $n=8$.

Q3. [Trap] In 1's complement representation, negation is performed by:

A. Adding 1 to the number B. Flipping all bits C. Flipping all bits and adding 1 D. Reversing bit order

Answer: B. Why: 1's complement negation = simple bitwise NOT. (Flip-then-add-1 is the 2's complement rule — a commonly swapped pair.)

Q4. Overflow occurs in 2's complement addition when:

A. Adding a positive and a negative number B. Two operands of the same sign yield a result of the opposite sign C. The result is exactly zero D. Any carry out of the MSB occurs

Answer: B. Why: Same-sign operands producing an opposite-sign result signals overflow; adding operands of different signs can never overflow.

Q5. Which code is "self-complementing," meaning the 9's complement of a digit equals its bitwise complement?

A. BCD B. Gray Code C. Excess-3 D. ASCII

Answer: C. Why: Excess-3 (BCD + 0011) has this property, simplifying decimal subtraction circuits.

Q6. The defining property of Gray code is:

A. It uses fewer bits than pure binary B. Exactly one bit changes between any two consecutive values C. It is self-complementing D. It can represent negative numbers directly

Answer: B. Why: Gray code minimizes transition errors in physical encoders/displays by changing only 1 bit per step — not a bit-count reduction.

Q7. [Numerical] The decimal value of the 2's complement number 11101010 (8 bits) is:

A. -10 B. -6 C. 246 D. -118

Answer: A. Why: Invert: 00001001, +1 → 00001010 = 10, so original = -10 .

Q8. How many bits does one hexadecimal digit correspond to?

A. 2 B. 3 C. 4 D. 8

Answer: C. Why: $16 = 2^4$, so each hex digit maps to exactly 4 binary bits (unlike octal, which maps to 3).

Q9. [Trap] Sign-magnitude and 1's complement both share which "problem" that 2's complement avoids?

A. Cannot represent fractions B. Two representations of zero C. Cannot represent negative numbers D. Requires more bits than 2's complement for the same range

Answer: B. Why: Both sign-magnitude and 1's complement have a +0 and a -0; 2's complement has a unique zero representation.

Q10. To convert Gray code back to binary, the rule for each bit (other than the MSB) is:

A. XOR with the previous Gray bit B. XOR with the previously-computed binary bit C. XNOR with the next Gray bit D. Simply copy the Gray bit unchanged

Answer: B. Why: Gray-to-binary conversion is cumulative: each binary bit = XOR of the corresponding Gray bit with the *previously derived binary* bit (MSB copied as-is).

1.2 Boolean Algebra, Logic Gates & K-Map Minimization

1.2.1 Basic Gates & Universal Gates

Gate	Output = 1 when	Note
AND	all inputs = 1	
OR	any input = 1	
NOT	input = 0	
NAND	not all inputs = 1	Universal gate — any circuit can be built from NAND alone
NOR	all inputs = 0	Universal gate — any circuit can be built from NOR alone
XOR	odd number of 1-inputs	Output = 1 when inputs differ (2-input case)
XNOR	even number of 1-inputs	Output = 1 when inputs match (2-input case)

- **TRAP:** AND, OR, and NOT are **not individually universal** — you need at least one from {NAND} or {NOR} alone, or the combination {AND, OR, NOT} together, to realize any Boolean function.

1.2.2 Boolean Algebra Laws

- **De Morgan's Laws:** $(A \cdot B)' = A' + B'$ and $(A + B)' = A' \cdot B'$.
- **Absorption:** $A + AB = A$; $A(A + B) = A$.
- **Consensus:** $AB + A'C + BC = AB + A'C$ (the BC term is redundant when A and A' both appear).
- **Duality principle:** replacing AND $\leftrightarrow$ OR and 0 $\leftrightarrow$ 1 in any valid Boolean identity produces another valid identity.
- **TRAP:** Duality does **not** mean complementing the variables — only the operators and constants swap; variables (A, B, C...) stay exactly as they are.

1.2.3 Karnaugh Maps (K-Maps)

- Used to minimize SOP (Sum of Products) / POS (Product of Sums) expressions graphically. Adjacent cells (including **wrap-around** edges — the map is toroidal) differ in exactly **one bit** — this is why K-map cell ordering uses **Gray code**, not binary order, across rows/columns.
- Grouping rule: group cells in powers of 2 (1, 2, 4, 8...); **larger groups** $\rightarrow$ **fewer literals** in that product term. Every group must contain only 1s (SOP) and be as large as possible.
- **Don't-care (X) conditions:** can be treated as 0 or 1, whichever helps form a larger group — used freely to simplify, never forced into a group that doesn't help.
- **Prime Implicant (PI):** a product term (group) that cannot be combined with another group to form a larger one. **Essential PI (EPI):** a PI that covers at least one minterm not covered by any other PI — an EPI *must* be included in the final minimized expression.
- **TRAP:** A minimized SOP expression must cover every 1-cell using the **fewest, largest** groups — but a "maximal" group is not automatically "essential"; a minterm might be coverable by two different equally-large groups (making both PIs, neither essential for that specific minterm alone) — the final choice is not unique in such cases.

Questions

Q1. Which of these is a "universal" gate, capable of implementing any Boolean function alone?

A. AND B. OR C. NAND D. XOR

Answer: C. **Why:** NAND (and NOR) alone can realize AND, OR, and NOT, hence any Boolean function; AND/OR/XOR individually cannot.

Q2. De Morgan's theorem states that $(A + B)'$ equals:

A. $A' + B'$ B. $A' \cdot B'$ C. $A \cdot B$ D. $A + B$

Answer: B. **Why:** Complement of an OR becomes the AND of the complements: $(A + B)' = A' \cdot B'$.

Q3. [Trap] Applying the duality principle to the identity $A + 1 = 1$ gives:

A. $A' + 1 = 1$ B. $A \cdot 0 = 0$ C. $A \cdot 1 = 1$ D. $A + 0 = A$

Answer: B. **Why:** Duality swaps $+$ $\leftrightarrow$ $\cdot$ and $0 \leftrightarrow 1$ only (variables unchanged): $A + 1 = 1$ dualizes to $A \cdot 0 = 0$.

Q4. In a K-map, adjacent cells are arranged using Gray code ordering because:

A. It makes the map smaller B. Adjacent cells must differ in exactly one bit to allow valid grouping C. It matches decimal ordering D. It is required for don't-care conditions only

Answer: B. **Why:** K-map simplification relies on combining cells that differ in one variable; Gray code ordering guarantees single-bit differences between physically adjacent cells (including wrap-around).

Q5. An Essential Prime Implicant (EPI) is defined as a prime implicant that:

A. Covers the maximum number of minterms overall B. Covers at least one minterm not covered by any other prime implicant C. Uses the fewest literals of all PIs D. Must always be selected first regardless of coverage

Answer: B. **Why:** The defining property of an EPI is that some minterm depends solely on it — no other PI can substitute for it in the final expression.

Q6. [Trap] A don't-care (X) condition in a K-map should be:

A. Always treated as 1 B. Always treated as 0 C. Used as 1 or 0, whichever helps form a larger group; never forced if it doesn't help D. Ignored entirely and left out of every grouping

Answer: C. **Why:** Don't-cares exist purely to help minimization — include one in a group only when doing so yields a larger/simpler group.

Q7. The absorption law states that $A + AB$ simplifies to:

A. A B. B C. AB D. $A + B$

Answer: A. **Why:** $A + AB = A(1 + B) = A \cdot 1 = A$.

Q8. How many gates minimum (of a single universal type) are conceptually needed to build a NOT gate?

A. 0 — impossible with a single gate type B. 1 NAND (or NOR) gate, with both inputs tied together C. 2 D. 3

Answer: B. **Why:** Tying both inputs of a NAND gate together makes it behave as a NOT gate: $\text{NAND}(A, A) = (A \cdot A)' = A'$.

Q9. For a 4-variable K-map with 16 cells, the largest possible single group size (all don't-care aside) that still represents a non-trivial function is:

A. 16 (the whole map — represents constant 1) B. 8 C. 4 D. 2

Answer: A. **Why:** A group covering the entire map (all 16 cells) reduces the expression to the constant "1" — valid but trivial; a meaningful function typically uses smaller groups, but 16 is technically the largest possible group.

Q10. [Trap] The consensus term in $AB + A'C + BC$ that can be eliminated as redundant is:

A. AB B. $A'C$ C. BC D. None — all three are required

Answer: C. **Why:** BC is the "consensus" of AB and $A'C$ (since A and A' together imply BC is already covered when both AB and $A'C$ hold across all cases) and can be dropped without changing the function.

1.3 Combinational Circuits

1.3.1 Adders

- **Half Adder:** adds 2 bits, outputs Sum (XOR) and Carry (AND). No carry-in support.
- **Full Adder:** adds 3 bits (A, B, Carry-in), outputs Sum and Carry-out. Built from 2 half adders + an OR gate.
- **Ripple Carry Adder:** n full adders chained, carry ripples from LSB to MSB. Simple but **slow** — worst-case delay grows linearly with n (each stage waits for the previous carry).
- **Carry Look-Ahead Adder:** computes all carries in parallel using extra gates (generate/propagate logic) — **faster** but more hardware.
- **TRAP:** Ripple carry = simplest hardware but worst propagation delay; carry look-ahead trades more gates for speed — a classic "the faster design isn't magic, it costs area" trade-off question.

1.3.2 Multiplexers & Demultiplexers

- **Multiplexer (MUX):** 2^n data inputs, n select lines, **1 output** — selects one of many inputs to route to the single output ("many-to-one" / data selector).
- **Demultiplexer (DEMUX):** 1 input, n select lines, 2^n outputs — routes the single input to one of many outputs ("one-to-many").
- A $2^n : 1$ MUX needs **n select lines**; e.g., an 8:1 MUX needs 3 select lines.
- **MUX as a universal logic building block:** a $2^n : 1$ MUX can implement any Boolean function of n variables by tying each data input to 0/1 per the truth table (or $n-1$ select lines + the last variable fed into a data input for larger economy).
- **TRAP:** MUX and DEMUX are often confused — MUX **selects/combines** (many→one), DEMUX **distributes** (one→many). Don't mix up which one needs the "select lines to pick an output" vs "select lines to pick an input."

1.3.3 Decoders, Encoders & Comparators

- **Decoder:** n inputs → up to 2^n unique outputs, exactly **one output active** per input combination (e.g., a 3-to-8 decoder). Used for memory address decoding.
- **Encoder:** reverse of a decoder — 2^n inputs → n outputs. **Priority Encoder:** resolves the case where multiple inputs are active simultaneously by outputting the code of the **highest-priority** active input.
- **Magnitude Comparator:** compares two binary numbers, outputs $A > B$, $A = B$, $A < B$.
- **TRAP:** A plain encoder assumes **only one input is ever active at a time** — if multiple inputs can be active, you need a **priority encoder**, otherwise the output is ambiguous/undefined.

Questions

Q1. A Half Adder differs from a Full Adder in that it:

A. Cannot produce a carry output at all B. Does not accept a carry-in from a previous stage C. Only works for 1-bit numbers D. Uses only NAND gates

Answer: B. **Why:** A half adder handles only two input bits (A, B) with no carry-in; a full adder additionally accepts a carry-in, enabling multi-bit chaining.

Q2. [Trap] Compared to Ripple Carry, a Carry Look-Ahead adder is:

A. Slower but uses less hardware B. Faster but requires more hardware/gates C. Identical in both speed and hardware cost D. Slower and uses more hardware

Answer: B. **Why:** Carry look-ahead computes carries in parallel via extra generate/propagate logic, reducing delay at the cost of additional gates — a genuine speed-vs-area trade-off, not a free win.

Q3. How many select lines are required for a 16:1 multiplexer?

A. 2 B. 4 C. 8 D. 16

Answer: B. **Why:** $2^n = 16 \Rightarrow n = 4$ select lines.

Q4. A demultiplexer is best described as a circuit that:

A. Selects one of many inputs and routes it to a single output B. Routes a single input to one of many possible outputs, based on select lines C. Compares two binary numbers D. Detects the highest-priority active input

Answer: B. **Why:** DEMUX = one-to-many distributor; this is the reverse function of a MUX (many-to-one selector).

Q5. A 3-to-8 line decoder, given a valid 3-bit input, activates:

A. All 8 outputs simultaneously B. Exactly one of the 8 outputs C. Exactly 3 outputs D. No outputs; decoders only work with enable signals

Answer: B. **Why:** A decoder's defining behavior is one unique active output line per unique input combination.

Q6. [Trap] A plain (non-priority) encoder produces an undefined/ambiguous output when:

A. No inputs are active B. Exactly one input is active C. More than one input is active simultaneously D. All select lines are grounded

Answer: C. **Why:** A basic encoder assumes only one active input at a time; simultaneous multiple active inputs require a priority encoder to resolve which one's code to output.

Q7. A $2^n : 1$ multiplexer can implement any Boolean function of n variables because:

A. It contains internal AND/OR gates equivalent to a full truth table realization when each data input is tied to the correct constant per the truth table row B. It automatically simplifies the function using K-maps internally C. It has more outputs than a decoder D. It can only implement functions with exactly 2 inputs

Answer: A. **Why:** Since each data input line corresponds to exactly one input combination (minterm), tying each to the function's required output value for that row directly realizes the truth table.

Q8. [Numerical] How many AND gates and OR gates (minimum, standard 2-level design) does a 4-to-16 decoder need at its output stage, ignoring internal input inverters?

A. 16 AND gates, 0 OR gates B. 4 AND gates, 16 OR gates C. 16 OR gates, 4 AND gates D. 8 AND gates, 8 OR gates

Answer: A. **Why:** Each of the 16 outputs is a single AND of all 4 (possibly complemented) inputs — no OR combination is needed since each output corresponds to exactly one unique minterm.

Q9. A magnitude comparator's three possible outputs are:

A. Sum, Carry, Borrow B. $A > B$, $A = B$, $A < B$ C. Enable, Select, Data D. Generate, Propagate, Kill

Answer: B. **Why:** A magnitude comparator's sole job is comparing two binary values and flagging which of the three relational outcomes holds.

Q10. The worst-case propagation delay of an n-bit ripple carry adder is proportional to:

A. $\log_2 n$ B. n C. Constant, independent of n D. n^2

Answer: B. **Why:** Each full-adder stage must wait for the carry from the previous stage, so delay accumulates linearly with the number of bits n.

1.4 Sequential Circuits — Latches, Flip-Flops, Counters & Registers

1.4.1 Latches vs. Flip-Flops

- **Latch:** level-triggered — output changes as long as the enable/clock signal is at the active **level** (e.g., high). Simple SR latch, D latch.
- **Flip-Flop:** edge-triggered — output changes only at a clock **edge transition** (rising or falling), not throughout the level. Used for reliable, synchronized sequential logic.
- **TRAP:** "Latch" and "Flip-flop" are **not interchangeable synonyms** despite casual use — the level-vs-edge triggering distinction is a favorite exam trap.

1.4.2 Flip-Flop Types

Flip-Flop	Inputs	Behavior
SR	S, R	$S=1, R=0 \rightarrow \text{Set } (Q=1)$; $S=0, R=1 \rightarrow \text{Reset } (Q=0)$; $S=R=1 \rightarrow \text{invalid/undefined state}$; $S=R=0 \rightarrow \text{no change}$
D (Data)	D	$Q(\text{next}) = D$ — simply captures the input at the clock edge; eliminates the SR invalid state
JK	J, K	Like SR but $J=K=1 \rightarrow \text{toggles } Q$ (fixes SR's invalid case)
T (Toggle)	T	$T=1 \rightarrow \text{toggles } Q$; $T=0 \rightarrow \text{holds}$. Built from JK with $J=K=T$

- **TRAP:** The SR flip-flop's forbidden/undefined input combination is **S=1, R=1** — JK flip-flops were designed specifically to resolve this by making that combination mean "toggle" instead of "invalid."
- Conversion questions (e.g., "implement a D flip-flop using JK") are common — derive via the excitation table (required J,K inputs to achieve a given $Q \rightarrow Q(\text{next})$ transition).

1.4.3 Registers & Shift Registers

- **Register:** a group of flip-flops storing multiple bits together (e.g., an 8-bit register = 8 D flip-flops).
- **Shift Register types:** SISO (Serial-in Serial-out), SIPO (Serial-in Parallel-out), PISO (Parallel-in Serial-out), PIPO (Parallel-in Parallel-out).
- Used for serial-to-parallel data conversion, delay lines, and as building blocks for counters.

1.4.4 Counters

Counter Type	Property
Asynchronous (Ripple) Counter	Flip-flops not clocked simultaneously — each flip-flop's clock is driven by the previous stage's output; simple but has cumulative propagation delay
Synchronous Counter	All flip-flops clocked simultaneously from the same clock signal; faster, more complex control logic
Mod-N Counter	Counts through N distinct states before repeating; needs $\lceil \log_2 N \rceil$ flip-flops minimum
Ring Counter	Shift register with output fed back to input; only one bit is "1" among N flip-flops, circulating
Johnson (Twisted-Ring) Counter	Ring counter with inverted feedback; N flip-flops give 2N distinct states (more states per flip-flop than a plain ring counter)

- **TRAP:** For an n-flip-flop counter, a plain **ring counter** gives only **n** unique states, while a **Johnson counter** gives **2n** states — this "which gives more states per flip-flop" comparison is frequently tested.

Questions

Q1. [Trap] The key difference between a latch and a flip-flop is:

A. Latches store data, flip-flops don't B. Latches are level-triggered, flip-flops are edge-triggered C. Flip-flops are level-triggered, latches are edge-triggered D. There is no real difference; the terms are fully interchangeable

Answer: B. **Why:** Latches respond throughout the active clock level; flip-flops respond only at a clock edge transition.

Q2. In an SR flip-flop, the input combination $S=1, R=1$ results in:

A. Q toggles B. Q holds its previous value C. An invalid/undefined state D. Q is forced to 0

Answer: C. **Why:** SR flip-flops leave $S=R=1$ undefined; this is precisely the case JK flip-flops were designed to resolve by defining it as "toggle."

Q3. A JK flip-flop with $J=K=1$ at the clock edge will:

A. Hold its previous state B. Toggle its output C. Enter an invalid state, same as SR D. Reset to 0 regardless of current state

Answer: B. **Why:** JK flip-flops specifically define $J=K=1$ as toggle, fixing the SR flip-flop's undefined case.

Q4. A D flip-flop's next state $Q(\text{next})$ is:

A. Always the complement of the current Q B. Equal to the D input at the triggering clock edge C. Undefined unless J and K are also specified D. Always 0 after every clock pulse

Answer: B. **Why:** The D flip-flop simply transfers its input to the output at the clock edge — its main advantage is having no invalid input state.

Q5. [Trap] The main disadvantage of an asynchronous (ripple) counter compared to a synchronous counter is:

A. It requires more flip-flops for the same modulus B. Cumulative propagation delay, since each flip-flop's clock depends on the previous stage's output C. It cannot count in binary D. It can only count up, never down

Answer: B. **Why:** Because each stage triggers the next, delays accumulate down the chain — synchronous counters avoid this by clocking all flip-flops together.

Q6. The minimum number of flip-flops needed to build a Mod-10 (decade) counter is:

A. 3 B. 4 C. 5 D. 10

Answer: B. **Why:** $\lceil \log_2 10 \rceil = 4$ flip-flops (providing 16 possible states, of which 10 are used).

Q7. [Trap] With n flip-flops, a plain ring counter provides how many usable/distinct states, compared to a Johnson counter's states?

A. Ring: n; Johnson: 2n B. Ring: 2n; Johnson: n C. Both provide 2^n states D. Both provide exactly n states

Answer: A. **Why:** A ring counter circulates a single "1" bit among n flip-flops (n states); a Johnson counter's inverted feedback doubles this to 2n states for the same flip-flop count.

Q8. A shift register configured for Serial-in, Parallel-out is abbreviated:

A. SISO B. SIPO C. PISO D. PIPO

Answer: B. Why: SIPO takes data in serially (one bit per clock) and makes all stored bits available simultaneously as parallel output.

Q9. A T (Toggle) flip-flop can be constructed directly from a JK flip-flop by:

A. Tying J and K both to the T input B. Tying J=1, K=0 permanently C. Tying J=0, K=1 permanently D. It cannot be derived from a JK flip-flop

Answer: A. Why: Connecting J=K=T reproduces T flip-flop behavior: T=1 gives J=K=1 (toggle), T=0 gives J=K=0 (hold).

Q10. [Numerical] A synchronous counter is designed to count 0 through 5 (Mod-6) and then repeat. The minimum number of flip-flops required is:

A. 2 B. 3 C. 4 D. 6

Answer: B. Why: $\lceil \log_2 6 \rceil = 3$ flip-flops (giving 8 possible states, 6 of which are used and 2 are unused/invalid states that must be designed out).

1.5 CPU Organization, Instruction Cycle & Addressing Modes

1.5.1 CPU Registers

Register	Role
PC (Program Counter)	Holds address of the next instruction to fetch
IR (Instruction Register)	Holds the currently executing instruction
MAR (Memory Address Register)	Holds the memory address to be accessed
MBR/MDR (Memory Buffer/Data Register)	Holds data being transferred to/from memory
Accumulator (AC)	General register for arithmetic/logic results
PSW / Flags Register	Condition codes: Zero, Carry, Overflow, Sign/Negative

- **TRAP:** PC always points to the next instruction, not the one currently executing — that is IR's job. This is a frequently-flipped pair.

1.5.2 Instruction Cycle

1. **Fetch:** PC → MAR; instruction from memory → MBR → IR; PC incremented.
2. **Decode:** control unit interprets the opcode in IR.
3. **Execute:** operand fetch (if needed) + ALU operation / memory write / branch.
4. **Interrupt check:** at the end of each cycle, CPU checks for pending interrupts before starting the next fetch.
 - **Instruction format:** Opcode field + Operand/Address field(s). Fewer operands per instruction (e.g., single-address) shifts more work onto implicit registers like the accumulator; three-address instructions are more RISC-like/orthogonal but need more instruction bits.

1.5.3 Addressing Modes

Mode	How operand is found	Trade-off
Immediate	Operand is part of the instruction itself	Fastest, but operand value is fixed at compile time
Direct	Instruction holds the operand's memory address directly	Simple; limited by address-field size
Indirect	Instruction holds the address of a location that itself holds the operand's address	Flexible (large address space, pointers) but slower (extra memory access)
Register	Operand is in a CPU register	Fastest memory-based mode (no memory access at all)
Register Indirect	Instruction names a register that holds the operand's memory address	One memory access; combines register speed of addressing with memory-sized operands
Indexed	Effective address = base address (in instruction) + contents of an index register	Efficient for array/loop traversal
Relative	Effective address = PC + offset in the instruction	Used for position-independent branch instructions

- **TRAP:** Indirect addressing requires one **extra memory reference** compared to direct addressing (fetch the pointer, then fetch the actual operand) — a common "how many memory accesses" numerical trap.

1.5.4 RISC vs. CISC

	RISC	CISC
Instruction set	Small, simple, fixed-length	Large, complex, variable-length
Cycles per instruction	Mostly single-cycle (~1 CPI)	Multi-cycle, variable
Addressing modes	Few	Many
Memory access	Only via explicit LOAD/STORE instructions	Many instructions can access memory directly
Design philosophy	More work shifted to the compiler	More work done in hardware/microcode

- **TRAP:** RISC's defining feature is **Load/Store architecture** — ALU operations work only on registers, never directly on memory operands; this is *the* key architectural distinction, not just "fewer instructions."

Questions

Q1. [Trap] Which register holds the address of the *next* instruction to be fetched?

A. Instruction Register (IR) B. Program Counter (PC) C. Memory Address Register (MAR) D. Accumulator

Answer: B. **Why:** PC always points to the next instruction; IR holds the instruction currently being decoded/executed — a commonly swapped pair.

Q2. The correct order of the basic instruction cycle is:

A. Execute → Fetch → Decode B. Fetch → Execute → Decode C. Fetch → Decode → Execute D. Decode → Fetch → Execute

Answer: C. **Why:** The CPU must first fetch the instruction, then decode its opcode/operands, then execute it.

Q3. In immediate addressing mode, the operand is:

A. Found at an address stored in a register B. Part of the instruction itself C. Found via a pointer stored in memory D. Always in the accumulator

Answer: B. **Why:** Immediate mode embeds the literal operand value directly in the instruction — no memory access needed to fetch it.

Q4. [Trap] Compared to direct addressing, indirect addressing requires:

A. Fewer memory accesses B. Exactly one extra memory access to first fetch the pointer C. No memory access at all D. The same number of memory accesses

Answer: B. Why: Indirect addressing fetches a pointer from the address given, then uses that pointer's value as the actual operand's address — one more memory reference than direct addressing.

Q5. The defining architectural feature of a RISC processor is:

A. A very large, variable-length instruction set B. Load/Store architecture — ALU operations only on registers, memory accessed only via explicit load/store C. Multi-cycle execution for every instruction D. Extensive use of microcode for every instruction

Answer: B. Why: RISC's core philosophy restricts memory access to dedicated load/store instructions, keeping ALU operations register-only and simplifying pipelining.

Q6. In indexed addressing mode, the effective address is computed as:

A. PC + offset B. Base address (in instruction) + index register contents C. Contents of a register, directly D. The address stored at another address

Answer: B. Why: Indexed addressing adds an index register's value to a base address in the instruction — ideal for iterating over array elements.

Q7. Relative addressing mode computes its effective address using:

A. The stack pointer B. The program counter (PC) plus an offset C. Two levels of indirection D. A fixed, hardcoded address only

Answer: B. Why: Relative addressing is PC-relative, commonly used for branch/jump instructions to support position-independent code.

Q8. [Numerical] If an instruction using direct addressing requires 1 memory access to fetch the operand, register-indirect addressing (operand in memory, its address in a register) requires:

A. 0 memory accesses B. 1 memory access C. 2 memory accesses D. Cannot be determined

Answer: B. Why: Register-indirect needs only 1 memory access (to fetch the operand itself) since the address is already available in a register — no separate access is needed to fetch that address, unlike full indirect addressing.

Q9. Which of these is true regarding CISC architecture?

A. Instructions are always fixed-length B. Most instructions execute in exactly one clock cycle C. Many instructions can operate directly on memory operands, not just registers D. It has fewer addressing modes than RISC

Answer: C. Why: CISC allows complex instructions to access memory directly (not load/store-restricted), variable instruction length, and many addressing modes — the opposite profile of RISC.

Q10. [Trap] The Memory Buffer/Data Register (MBR/MDR) is used to:

A. Hold the address of the next instruction B. Hold data being transferred between the CPU and memory C. Hold only the currently executing instruction's opcode D. Store CPU condition flags (zero, carry, overflow)

Answer: B. Why: MBR/MDR is the CPU's temporary holding register for data (or instructions) moving to/from memory — distinct from MAR, which holds only the address being accessed.

1.6 Memory Hierarchy & Cache

1.6.1 Memory Hierarchy

- Order (fastest/smallest/most expensive → slowest/largest/cheapest): **CPU Registers → Cache (L1 → L2 → L3) → Main Memory (RAM) → Secondary Storage (SSD/HDD) → Tertiary/Backup Storage.**
- Hierarchy works because of **locality of reference**: **Temporal locality** (recently accessed data likely to be accessed again soon) and **Spatial locality** (data near recently accessed data likely to be accessed soon).

1.6.2 Cache Mapping Techniques

Mapping	How it works	Trade-off
Direct Mapping	Each main memory block maps to exactly one specific cache line (block address mod number of cache lines)	Simple, fast lookup, but high conflict misses if two frequently-used blocks map to the same line
Fully Associative	Any block can go in any cache line	No conflict misses, but needs to search all lines (expensive comparator hardware)
Set-Associative (n-way)	Cache divided into sets; a block maps to one specific set but can go in any of the n lines within that set	Practical middle ground — most real caches use this (e.g., 4-way, 8-way)

- TRAP:** Direct-mapped cache has the **least** flexibility (fastest but most conflict misses); fully associative has the **most** flexibility (slowest lookup, fewest misses of this type) — set-associative is the compromise, not a completely separate third mechanism.

1.6.3 Cache Performance

- Hit ratio (h)** = hits / total accesses. **Average Memory Access Time (AMAT)** = Hit time + Miss rate × Miss penalty.
- Average access time (simplified, 2-level)** = $h \times T_{\text{cache}} + (1-h) \times T_{\text{main}}$ (only when a miss also re-checks cache first, otherwise use full AMAT formula).
- TRAP:** A **higher hit ratio always reduces average access time**, but the relationship is not linear with cache size — diminishing returns as cache grows (a classic "why not just make the cache huge" numerical/conceptual trap: cost and diminishing hit-ratio gains both argue against it).

1.6.4 Cache Write Policies

Policy	Behavior
Write-Through	Every write updates both cache and main memory immediately
Write-Back	Write updates only cache ; marks the line dirty ; main memory updated only when the dirty line is evicted

- TRAP:** Only a **write-back** cache needs a "dirty bit" per line — write-through caches never need one since memory is always current.

1.6.5 Replacement Policies (for associative/set-associative caches)

- LRU (Least Recently Used):** evicts the block unused for the longest time — best locality-based performance but costlier to implement exactly.
- FIFO:** evicts the oldest-loaded block, regardless of recent use.
- Random:** simplest, no tracking overhead, unpredictable performance.
- Optimal (Belady's):** evicts the block that will be used **furthest in the future** — theoretical best-case, cannot be implemented in practice (requires future knowledge), used only as a benchmark.

Questions

Q1. The correct order of the memory hierarchy, from fastest/smallest to slowest/largest, is:

A. Cache → Registers → RAM → Secondary storage B. Registers → Cache → RAM → Secondary storage C. RAM → Cache → Registers → Secondary storage D. Registers → RAM → Cache → Secondary storage

Answer: B. Why: CPU registers are fastest/smallest, then cache, then main memory (RAM), then slower/larger secondary storage.

Q2. [Trap] Spatial locality refers to the principle that:

A. Recently accessed data is likely to be accessed again soon B. Data located near recently accessed data is likely to be accessed soon C. Cache is always faster than main memory D. Programs execute instructions in random order

Answer: B. Why: Spatial locality = nearby addresses tend to be accessed together (e.g., sequential array access); temporal locality is the "same data again soon" principle — a commonly confused pair.

Q3. In direct-mapped cache, each main memory block can be placed:

A. In any cache line whatsoever B. In exactly one specific cache line C. In any line within a designated set D. Nowhere; direct mapping doesn't use cache lines

Answer: B. Why: Direct mapping computes exactly one target line per block (typically via a modulo operation), which is fast but can cause conflict misses.

Q4. [Trap] Which cache mapping scheme has zero conflict misses but the most expensive lookup hardware?

A. Direct mapping B. Fully associative C. Set-associative D. All three are identical in this respect

Answer: B. Why: Fully associative allows a block anywhere, eliminating conflict misses, but requires comparing the tag against every line simultaneously — costly hardware.

Q5. A "dirty bit" in a cache line is meaningful only under which write policy?

A. Write-through B. Write-back C. Both equally D. Neither; dirty bits apply only to main memory

Answer: B. Why: Write-back defers updating main memory until eviction, so it must track which lines have been modified (dirty) since being loaded; write-through keeps memory always current, so no dirty bit is needed.

Q6. Write-through cache policy's main disadvantage compared to write-back is:

A. Risk of data loss on a crash B. Increased memory traffic, since every write goes to main memory immediately C. It cannot support multiple cache levels D. It requires a dirty bit per line

Answer: B. Why: Because every write is immediately propagated to main memory, write-through generates more memory bus traffic than write-back, which batches updates until eviction.

Q7. The Belady's Optimal replacement policy is:

A. Practically implementable in all real systems B. A theoretical best-case benchmark that requires future knowledge, hence not implementable in practice C. Identical to LRU in all cases D. Used only in write-through caches

Answer: B. Why: Optimal replacement evicts the page/block that will be used furthest in the future — impossible to know in real time, so it serves only as a theoretical performance ceiling for comparison.

Q8. [Numerical] A system has a cache hit ratio of 0.9, cache access time 10 ns, and main memory access time 100 ns. Using the simplified average access time formula, the average access time is:

A. 19 ns B. 91 ns C. 55 ns D. 100 ns

Answer: A. Why: $0.9 \times 10 + 0.1 \times 100 = 9 + 10 = 19$ ns.

Q9. In an n-way set-associative cache, a given main memory block:

A. Can go in any line in the entire cache B. Maps to exactly one specific cache set, but can occupy any of the n lines within that set C. Maps to exactly one specific line, with no flexibility D. Must always go into set 0

Answer: B. Why: Set-associative mapping restricts a block to one designated set (like direct mapping does for lines) but allows associative placement (like fully associative) within that set's n lines.

Q10. LRU replacement evicts:

A. The most recently used block B. The block loaded earliest, regardless of use C. The block that has gone unused for the longest time D. A randomly chosen block

Answer: C. Why: LRU tracks recency of use and evicts the block that hasn't been touched in the longest time, exploiting temporal locality — distinct from FIFO, which only tracks load order, not usage.

1.7 I/O Organization, Interrupts & Bus/DMA

1.7.1 I/O Techniques

Technique	Mechanism	Trade-off
Programmed I/O	CPU actively polls the device status register in a loop ("busy-wait") until ready	Simplest, but wastes CPU cycles busy-waiting
Interrupt-Driven I/O	CPU continues other work; device raises an interrupt when ready, CPU handles it via an ISR	No CPU busy-waiting, but interrupt-handling overhead (context save/restore) per transfer
DMA (Direct Memory Access)	A dedicated DMA controller transfers data directly between I/O device and memory without CPU involvement per byte/word	Frees the CPU almost entirely during bulk transfer; CPU only involved at transfer start/end

- **TRAP:** DMA does not eliminate the CPU entirely from I/O — the CPU still **initiates** the DMA transfer (sets up source, destination, count) and is **interrupted once at completion**; it just skips per-word CPU involvement during the transfer itself.

1.7.2 Interrupts

- **Interrupt:** a signal that causes the CPU to suspend its current instruction stream and transition to an **Interrupt Service Routine (ISR)**, saving context (PC, registers/PSW) first.
- **Types:** Hardware interrupts (from I/O devices), Software interrupts (via a trap/system call instruction), **Exceptions** (from erroneous conditions — divide by zero, page fault, illegal instruction).
- **Maskable vs. Non-maskable:** maskable interrupts can be disabled by software (via an interrupt-enable flag); **non-maskable interrupts (NMI)** cannot be ignored — reserved for critical events (e.g., power failure).
- **Vectored interrupts:** the interrupting device itself supplies the address of its ISR (interrupt vector), letting the CPU jump directly without polling every device — faster than **polling-based** interrupt identification, which checks each device in priority order to find the source.
- **Multiple interrupts / priority:** if several interrupts occur, a priority scheme (or daisy-chaining) decides which ISR runs first; a currently-running high-priority ISR can typically preempt (nest inside) a lower-priority one, not vice versa.
- **TRAP:** A system call (software interrupt/trap) is **intentional and synchronous** with the running program; a hardware interrupt is **asynchronous** — it can occur at any point regardless of what the CPU is currently executing.

1.7.3 Bus Organization

- **System bus = Data bus + Address bus + Control bus.**
- **Data bus:** bidirectional, carries actual data being transferred; width determines how many bits moved per cycle.
- **Address bus:** unidirectional (CPU → memory/device), width determines the **maximum addressable memory** (n-bit address bus → 2^n addressable locations).
- **Control bus:** carries control signals (read/write, clock, interrupt requests, bus grant/request for DMA arbitration).
- **TRAP:** **Address bus width determines maximum memory size**, not data bus width — a very common numerical mix-up (e.g., "a 32-bit data bus" tells you nothing about total addressable memory by itself).

Questions

Q1. [Trap] The main disadvantage of Programmed I/O is:

- A. It requires a dedicated DMA controller B. The CPU wastes cycles busy-waiting/polling for device readiness C. It cannot transfer more than one byte at a time D. It requires non-maskable interrupts

Answer: B. **Why:** Programmed I/O has the CPU actively loop-checking a status flag until the device is ready — pure wasted CPU time compared to interrupt-driven or DMA approaches.

Q2. [Trap] Under DMA, the CPU's role during the actual bulk data transfer is:

- A. Fully involved, transferring every word itself B. Not involved per-word; it only sets up the transfer and is notified upon completion C. Completely irrelevant — DMA requires no CPU interaction at all, ever D. Responsible for continuously polling the DMA controller

Answer: B. **Why:** The CPU initializes the DMA controller (source, destination, count) and is interrupted once when the transfer finishes — but does not shepherd each individual word like it would in programmed I/O.

Q3. A non-maskable interrupt (NMI) is characterized by:

- A. Being disabled by clearing the interrupt-enable flag B. Being reserved for critical events and cannot be disabled by software C. Only occurring during system calls D. Having the lowest priority among all interrupts

Answer: B. **Why:** NMIs are reserved for events too critical to ignore (e.g., power failure) and bypass the software interrupt-enable mask entirely.

Q4. Vectored interrupt handling is faster than polling-based interrupt identification because:

- A. It uses a bigger data bus B. The interrupting device itself supplies its ISR address, avoiding a device-by-device check C. It disables all other interrupts permanently D. It requires no ISR at all

Answer: B. **Why:** In vectored interrupts, the CPU jumps directly to the correct ISR using an address supplied by the device; polling instead sequentially checks each device to find which one raised the interrupt, which is slower.

Q5. [Trap] A hardware interrupt differs from a software interrupt (trap/system call) chiefly because it is:

- A. Always higher priority B. Asynchronous — it can occur at any point in program execution, unlike a trap which is a deliberate, synchronous instruction C. Only used for exceptions D. Non-maskable by definition

Answer: B. **Why:** A software interrupt/trap is explicitly invoked by the running program at a known point; a hardware interrupt arrives asynchronously from an external device, independent of what instruction the CPU happens to be executing.

Q6. [Trap] The maximum addressable memory of a system is primarily determined by:

- A. The width of the data bus B. The width of the address bus C. The width of the control bus D. The clock speed of the CPU

Answer: B. **Why:** An n-bit address bus can express 2^n unique addresses — this directly caps maximum addressable memory, independent of data bus width (which affects transfer size/bandwidth per cycle, not address range).

Q7. Which bus is bidirectional, carrying the actual values being transferred between CPU and memory/devices?

- A. Address bus B. Data bus C. Control bus D. None; all buses are unidirectional

Answer: B. **Why:** The data bus moves actual data values in both directions (read from or write to memory/devices); the address bus is one-directional (CPU specifies where).

Q8. An exception, as an interrupt category, is best described as:

A. A signal from an external I/O device B. A deliberate system call made by a program C. An interrupt triggered by an erroneous or unusual condition arising during instruction execution (e.g., divide-by-zero, page fault) D. A type of DMA transfer completion signal

Answer: C. **Why:** Exceptions arise from internal, execution-triggered conditions rather than external device signals (hardware interrupts) or deliberate program requests (software interrupts/traps).

Q9. If a higher-priority interrupt occurs while a lower-priority ISR is executing, standard practice is to:

A. Ignore the higher-priority interrupt until the current ISR finishes B. Allow the higher-priority interrupt to preempt/nest inside the currently running lower-priority ISR C. Immediately halt the CPU D. Convert it into a software interrupt

Answer: B. **Why:** Priority-based interrupt systems typically let a higher-priority interrupt preempt a lower-priority one currently in service — not the reverse.

Q10. [Numerical] A system has a 20-bit address bus. Its maximum addressable memory is:

A. 20 KB B. 1 MB C. 1 GB D. 20 MB

Answer: B. **Why:** $2^{20} = 1,048,576 \approx 1 \text{ MB}$ of addressable locations.

1.8 Pipelining & Instruction-Level Parallelism

1.8.1 Pipelining Basics

- **Pipelining:** overlaps execution of multiple instructions across stages (classic 5-stage RISC pipeline: **IF (Fetch)** → **ID (Decode)** → **EX (Execute)** → **MEM (Memory access)** → **WB (Write-back)**), so a new instruction can enter the pipeline every cycle once it's full — increasing **throughput**, not the latency of any single instruction.
- **Speedup (ideal, k stages, n instructions):** Speedup → k as $n \rightarrow \infty$ (time without pipelining $\approx n \cdot k$ cycles; with pipelining $\approx (k + n - 1)$ cycles).
- **TRAP:** Pipelining improves **throughput (instructions completed per unit time)**, not the **latency** of any individual instruction — a single instruction still takes k cycles to fully pass through k stages; the win is that many instructions overlap.

1.8.2 Pipeline Hazards

Hazard	Cause	Typical fix
Structural Hazard	Two instructions need the same hardware resource simultaneously (e.g., one memory unit for both instruction fetch and data access)	Duplicate hardware (separate instruction/data caches)
Data Hazard	An instruction depends on the result of a prior instruction still in the pipeline (RAW — Read After Write, most common; also WAR, WAW in out-of-order pipelines)	Forwarding/bypassing (feed the result directly to the dependent stage) or stalling
Control Hazard (Branch Hazard)	Pipeline doesn't know which instruction to fetch next until a branch is resolved	Branch prediction , delayed branching, or stalling until the branch resolves

- **TRAP: RAW (Read-After-Write)** is a *true* dependency and the most common real data hazard; **WAR/WAW** are *false* dependencies that only matter in **out-of-order** execution (they don't occur in a simple in-order pipeline where instructions are issued strictly in program order).

1.8.3 Hazard-Handling Techniques

- **Forwarding (bypassing):** routes a result from a later pipeline stage (e.g., EX output) directly to an earlier stage of a dependent instruction, avoiding a stall — cannot always eliminate the hazard alone (e.g., load-use hazard still needs at least one stall cycle since data isn't ready until MEM completes).
- **Stalling (pipeline bubble):** inserts a no-op cycle, delaying the dependent instruction until the hazard clears — guarantees correctness but reduces throughput.
- **Branch prediction:** static (always predict taken/not-taken) or dynamic (based on runtime history, e.g., branch history table). A **misprediction** requires flushing the incorrectly fetched instructions from the pipeline — the deeper the pipeline, the higher the misprediction penalty.
- **TRAP:** Even **perfect forwarding cannot fully eliminate a load-use hazard** (using a value immediately after a LOAD instruction that fetches it) — the loaded value simply isn't available until the MEM stage completes, so at least one stall cycle is unavoidable regardless of forwarding.

Questions

Q1. [Trap] Pipelining primarily improves:

A. The latency of a single instruction B. The throughput — number of instructions completed per unit time C. The clock speed of the CPU D. The size of the instruction set

Answer: B. **Why:** A single instruction still takes the same total number of cycles to traverse all stages; pipelining's benefit comes from overlapping many instructions, boosting overall throughput.

Q2. A structural hazard in a pipeline arises when:

A. An instruction needs a result not yet computed by a prior instruction B. Two instructions require the same hardware resource at the same time C. A branch instruction's target is unknown D. The pipeline has too many stages

Answer: B. **Why:** Structural hazards are resource conflicts (e.g., a single memory port needed by both fetch and memory-access stages simultaneously) — solved by duplicating the contended resource.

Q3. [Trap] Which type of data hazard is a genuine/true dependency that occurs even in simple in-order pipelines?

A. WAR (Write After Read) B. WAW (Write After Write) C. RAW (Read After Write) D. None; all data hazards only occur in out-of-order execution

Answer: C. **Why:** RAW is a true dependency (an instruction needs a value a prior instruction hasn't produced yet) and occurs in ordinary in-order pipelines; WAR/WAW are false dependencies relevant only when instructions can complete out of program order.

Q4. Forwarding (bypassing) as a hazard-resolution technique works by:

A. Delaying the dependent instruction until the hazard clears via a pipeline bubble B. Routing a computed result directly from a later pipeline stage to an earlier stage that needs it, without waiting for it to be written back through the register file C. Predicting the branch outcome D. Duplicating the ALU hardware

Answer: B. **Why:** Forwarding short-circuits the normal register write-then-read path, feeding a result directly to where it's needed as soon as it's computed.

Q5. [Trap] Even with perfect forwarding, a load-use hazard (using a value immediately after loading it from memory) still requires:

A. No stall at all — forwarding fully resolves it B. At least one stall cycle, because the loaded value isn't available until the MEM stage completes C. A complete pipeline flush D. Disabling the pipeline entirely for that instruction

Answer: B. **Why:** The value doesn't exist until the load instruction finishes its memory access (MEM stage); forwarding can route it as soon as it's available, but cannot make it available earlier than that — hence at least one stall is unavoidable.

Q6. A control (branch) hazard arises because:

A. Two instructions compete for the same functional unit B. The pipeline cannot know which instruction to fetch next until the branch outcome is resolved C. A register is written and read too quickly D. The clock frequency is too high

Answer: B. **Why:** Until a branch's condition and target are evaluated, the pipeline doesn't know whether to keep fetching sequentially or jump elsewhere — hence "control" hazard.

Q7. Branch misprediction penalty tends to be worse in:

A. Shorter pipelines B. Deeper pipelines, since more incorrectly-fetched instructions must be flushed C. Pipelines with no branch instructions D. Pipelines using only structural hazard handling

Answer: B. **Why:** A deeper pipeline has more in-flight instructions fetched speculatively past the branch; a misprediction means flushing all of them, at a cost proportional to pipeline depth.

Q8. [Numerical] An ideal 5-stage pipeline executing 100 instructions takes approximately how many cycles (ignoring hazards/stalls)?

A. 500 B. 100 C. 104 D. 20

Answer: C. **Why:** Total cycles $\approx k + (n - 1) = 5 + 99 = 104$, versus $5 \times 100 = 500$ if executed with no overlap at all.

Q9. Static branch prediction, as opposed to dynamic branch prediction, means:

A. The prediction is based on real-time runtime history of that branch B. The prediction rule (e.g., "always predict not-taken") is fixed at design/compile time, independent of runtime behavior C. There is no prediction; the pipeline always stalls at a branch D. Prediction accuracy is always 100%

Answer: B. Why: Static prediction uses a fixed heuristic decided in advance; dynamic prediction adapts using runtime history (e.g., a branch history table), typically achieving better accuracy.

Q10. WAR (Write-After-Read) and WAW (Write-After-Write) hazards are only a concern when:

A. The pipeline strictly issues and completes instructions in program order B. Instructions can complete out of program order (out-of-order execution) C. Only structural hazards are present D. Forwarding is disabled

Answer: B. Why: In a simple in-order pipeline, instructions naturally read and write registers in program order, so WAR/WAW situations cannot actually arise — they only become real risks once out-of-order completion is allowed.

2.1 C Fundamentals — Data Types & Operators

1.1 Character Set, Identifiers & Basic Structure

- **C character set:** letters, digits, special symbols. **Identifier rule:** 1–31 chars, starts with a letter or underscore, only letters/digits/underscore allowed, no keyword can be reused as an identifier.
- **Constant** = value that doesn't change; **Variable** = named memory location whose value can change. Both declared with a type before use.
- Every C statement ends with `;`. C is free-form (no fixed column rules). Comments `/* ... */` — **TRAP: comments cannot be nested** in standard C.
- `main()` is the mandatory single entry point of every C program; forms: `main()`, `main(int argc, char *argv[])` (`argc`=argument count, `argv`=argument vector), `main(int argc, char *argv[], char *env[])`.
- **Uninitialized local variable** → **garbage value** (leftover memory content, NOT assigned by the compiler). **TRAP:** in Java, uninitialized class-level fields default to 0 — do not assume the same "safe default" behavior in C.
- `extern` declares (doesn't define) a variable — no memory allocated until some translation unit actually defines/uses it.

1.2 Data Types & Sizes (typical 32/64-bit system assumption used by GATE/ISRO)

Type	Typical size	Notes
char	1 byte	signed range -128..127, unsigned 0..255
int	4 bytes	signed range $\approx -2.1 \times 10^9$ to 2.1×10^9
float	4 bytes	~6-7 significant digits
double	8 bytes	~15-16 significant digits

- `short` / `long` modify size (short `int` $\leq$ `int` $\leq$ long `int`, sizes are implementation-defined but this ordering always holds).
- **Real (floating) constants:** fractional form (`426.0`) or exponential form (`3.2e-5` , mantissa `e` exponent).
- **Character constant:** single char in single quotes (`'A'`), stored internally as its **ASCII value** (e.g., `'A'`=65, `'a'`=97) — arithmetic on chars operates on these ASCII codes.

1.3 Integer Overflow / Wraparound

Unsigned type wraps modulo 2^n : `unsigned char 255 + 1 = 0`
 Signed overflow is UNDEFINED BEHAVIOR in the C standard, but GATE/exam questions conventionally assume 2's-complement wraparound:
`signed char 127 + 1 = -128 (assumed)`

- **TRAP:** Signed integer overflow is technically undefined behavior (compiler could do anything) — but exam questions expect the 2's-complement wraparound answer unless stated otherwise.

1.4 Type Conversion

`int op int` → `int`
`real op real` → `real`
`int op real` → `int` PROMOTED to real first, result is real
 assignment: RHS result is converted (promoted/demoted) to match LHS variable's type

- **Integer division truncates toward zero; float division keeps the fraction:** $5/2 \rightarrow 2$ (int), but $5.0/2$ or $5/2.0 \rightarrow 2.5$.
- **TRAP:** `k = 9/2` gives `4` (int truncation) even if `k` will later be printed as a float context — the truncation happens on the RHS **before** any assignment-based promotion.
- **Modulus %:** only valid on integers (not float). Result's sign matches the **dividend (numerator)**: `-5 % 2 = -1`, `5 % -2 = -1`.
- No exponentiation operator in C — `a ** b` and `a ^ b` are both invalid for "power" (`^` is bitwise XOR!) — must use `pow()` from `<math.h>`.

1.5 Operators

- **Unary:** `++` / `--` (pre/post), unary `+` / `-`, logical `!`, bitwise `~`, `sizeof`, address-of `&`, dereference `*`, type cast — all **right-to-left** associativity.
- **sizeof:** compile-time only operator; any expression inside it is **NOT executed** at runtime (its side effects never happen).

```
int a = 5;
int x = sizeof(a = 10); // a=10 assignment inside sizeof is NOT executed
printf("%d", a);        // prints 5 (unchanged)
printf("%d", x);        // prints 4 (size of int, unrelated to the expression's value)
```

- **Pre vs Post increment:**

Feature	<code>++x</code> (pre)	<code>x++</code> (post)
When incremented	Before use in the expression	After use in the expression
Value used in expression	New (incremented) value	Original (old) value

- **TRAP:** Expressions like `i = i++ + ++i` invoke **undefined behavior** in C (multiple unsequenced modifications to `i`) — different compilers may give different answers; be aware such questions technically have no single "correct" C answer even though exams often expect one based on common-compiler behavior.
- **Bitwise:** `&`, `|`, `^` (XOR), `~` (one's complement, `~5 = -6` in 2's complement), `<<` (left shift = multiply by 2^n), `>>` (right shift = divide by 2^n for unsigned/logical shift; for signed, commonly sign-extends — implementation-defined but usually assumed in exams).
- **Logical (`&&`, `||`, `!`) — SHORT-CIRCUIT evaluation:**

`&&` stops evaluating as soon as ANY operand is false (0)

`||` stops evaluating as soon as ANY operand is true (nonzero)

- **TRAP:** In `a() && b()`, if `a()` returns 0, `b()` is **NEVER CALLED** — this matters enormously when operands have side effects (e.g., `++x`), since a short-circuited operand's side effect never happens.

1.6 Operator Precedence (high → low, practical GATE-relevant ordering)

1. Postfix <code>++/--</code>	(Left-to-Right)
2. Unary: <code>++/--</code> (pre), <code>+</code> <code>-</code> (unary), <code>!</code> <code>~</code> , <code>sizeof</code> , <code>(type)cast</code> , <code>&</code> <code>*</code>	(Right-to-Left)
3. Multiplicative: <code>*</code> <code>/</code> <code>%</code>	(L-to-R)
4. Additive: <code>+</code> <code>-</code>	(L-to-R)
5. Shift: <code><<</code> <code>>></code>	(L-to-R)
6. Relational: <code><</code> <code><=</code> <code>></code> <code>>=</code>	(L-to-R)
7. Equality: <code>==</code> <code>!=</code>	(L-to-R)
8. Bitwise AND: <code>&</code>	(L-to-R)
9. Bitwise XOR: <code>^</code>	(L-to-R)
10. Bitwise OR: <code> </code>	(L-to-R)
11. Logical AND: <code>&&</code>	(L-to-R)
12. Logical OR: <code> </code>	(L-to-R)
13. Ternary: <code>?:</code>	(Right-to-Left)
14. Assignment: <code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> etc.	(Right-to-Left)
15. Comma: <code>,</code>	(Right-to-Left)

- **TRAP:** Ternary `?:` and assignment are **right-to-left** — a nested ternary `a ? b : c ? d : e` groups as `a ? b : (c ? d : e)`, NOT `(a ? b : c) ? d : e`.

Questions

Q1. [Numerical] What is the output?

```
int a = 2, b;
b = a && !a;
printf("%d", b);
```

A. 0 B. 1 C. 2 D. Compile error

Answer: A. Why: `!a = !2 = 0` (a is nonzero so its logical negation is 0); `2 && 0 = 0` (short-circuits false since second operand is 0).

Q2. [Trap] What is the output?

```
int a = -1, b = 2, c;
c = ++a && ++b;
printf("%d%d%d", a, b, c);
```

A. 020 B. 121 C. 023 D. 122

Answer: A. Why: `++a` makes `a=0` (falsy) — `&&` short-circuits, so `++b` is NEVER evaluated, `b` stays 2; `c = 0`. Output: `a=0,b=2,c=0 → "020"`.

Q3. [Trap] What is the output?

```
int a = 0, b = 2, c;
c = ++a || ++b;
printf("%d%d%d", a, b, c);
```

A. 121 B. 021 C. 122 D. 020

Answer: A. Why: `++a` makes `a=1` (truthy) — `||` short-circuits true, so `++b` is NEVER evaluated, `b` stays 2; `c=1`. Output: "121".

Q4. [Numerical] What is the output?

```
int a = 1, b = 1;
int c = a || --b;
int d = a-- && --b;
printf("a=%d, b=%d, c=%d, d=%d", a, b, c, d);
```

A. `a=0, b=1, c=1, d=0` B. `a=0, b=0, c=1, d=0` C. `a=1, b=1, c=1, d=1` D. `a=0, b=0, c=0, d=0`

Answer: B. Why: `a || --b : a=1` (truthy) → short-circuits, `--b` never runs, `c=1`, `b` still 1. `a-- && --b : a--` evaluates current `a=1` (truthy) THEN decrements `a` to 0; since left is truthy, `--b` runs: `b` becomes 0 (falsy); `d = 1 && 0 = 0`. Final: `a=0, b=0, c=1, d=0`.

Q5. [Numerical] What is the output?

```
int x = 10, y = 5, z = 2;
int result = x > y ? y > z ? 1 : z > x ? 2 : y > z ? 4 : z > x ? 5 : 6;
printf("%d", result);
```

A. 1 B. 2 C. 4 D. 6

Answer: A. Why: Ternary is right-associative: `x>y` is `true(10>5) → evaluate y>z ? 1 : (...)`; `y>z` is `true(5>2) → result = 1`. The rest is never evaluated.

Q6. What is the value of `k` after `k = 9 / 2`; where `k` is declared `int` ?

A. 4.5 B. 4 C. 5 D. 4.0

Answer: B. Why: Both operands are `int`, so integer division truncates toward zero: `9/2 = 4` (the fractional part is simply discarded, not rounded).

Q7. What is `-5 % 2` in C?

A. 1 B. -1 C. -3 D. 3

Answer: B. Why: The sign of the result of `%` always matches the sign of the dividend (numerator): `-5 % 2 = -1`.

Q8. [Trap] Which statement about `sizeof(a = 10)` (where `a` is a previously-declared `int` currently holding 5) is correct?

A. It executes the assignment, so `a` becomes 10 B. It does NOT execute the assignment — `a` remains unchanged; the expression only yields the byte-size of its type C.

It causes a compile error D. It always returns 0

Answer: B. Why: `sizeof` operates at compile time and never executes its operand expression, so any embedded side effect (like an assignment) never actually happens at runtime.

Q9. [Numerical] What does `~5` evaluate to (assuming 2's complement, 32-bit int)?

A. -5 B. -6 C. 6 D. 10

Answer: B. Why: One's complement flips all bits; for 2's-complement representation this equals `-(x+1)`, so `~5 = -6`.

Q10. In the expression `a() && b() || c()`, if `a()` returns a nonzero value, which functions are guaranteed to execute?

A. Only a() B. a() and b() only C. a(), b(), and c() D. Only b() and c()

Answer: B. Why: `&&` has higher precedence than `||`, so this groups as `(a() && b()) || c()`. Since `a()` is nonzero (truthy), `&&` proceeds to evaluate `b()` too. If `a() && b()` is truthy overall, `||` then short-circuits and `c()` is skipped — but `c()`'s execution depends on `b()`'s result, whereas `a()` and `b()` are always both called whenever `a()` is truthy.

Q11. [Numerical] Given `char x = 127;` (signed char, 2's complement assumed), what is the value of `x + 1`?

A. 128 B. -128 C. -1 D. 0

Answer: B. Why: Signed char range is -128..127; 127 is the max value, so incrementing wraps around (under the conventional 2's-complement assumption exam questions use) to -128.

Q12. [Trap] Which of the following correctly computes "3 raised to the power of 2" in standard C?

A. `3 ** 2` B. `3 ^ 2` C. `pow(3, 2)` using `<math.h>` D. Both A and B are valid

Answer: C. Why: C has no built-in exponentiation operator; `**` is not a valid C operator and `^` is bitwise XOR (not power) — you must use the `pow()` library function.

2.2 C Control Flow — Conditionals, Loops & Switch

2.1 if / if-else / Nested if-else

- `if (expr) stmt; — expr` can be ANY valid expression (arithmetic, assignment, even a bare negative number) — C treats **zero as false, any non-zero as true**. So `if(-5)`, `if(a=10)` (an assignment, which evaluates to the assigned value) are all legal and TRUE.
- **TRAP:** A stray semicolon after the condition — `if (i==5); printf("hi");` — makes the `;` an empty statement satisfying the `if`, so `printf` runs **unconditionally**, every time, regardless of the condition. The compiler gives no error; only the logic is broken.
- Multiple statements under one `if / else` MUST be wrapped in `{ }`; without braces, only the single next statement belongs to the `if`.
- **Dangling else:** an `else` always binds to the **nearest unmatched preceding if**, regardless of indentation:

```
if (a > 0)
    if (b > 0)
        printf("both positive");
else
    printf("a not positive"); // actually binds to the INNER if (b>0), NOT the outer one!
```

TRAP: Indentation is just visual — the compiler binds `else` to the closest open `if` lexically, so misleading indentation can hide a logic bug.

2.2 Conditional (Ternary) Operator

```
result = (condition) ? expr_if_true : expr_if_false;
```

- Right-to-left associative when chained: `a ? b : c ? d : e` groups as `a ? b : (c ? d : e)`.
- Equivalent to if-else but is itself an *expression* (has a value), so it can be used inside larger expressions/assignments where if-else cannot.

2.3 Loops: while, do-while, for

Loop	Condition checked	Min. executions of body
<code>while</code>	BEFORE body	0 (may never run)
<code>do-while</code>	AFTER body	1 (always runs at least once)
<code>for</code>	BEFORE body (like while)	0

TRAP: `do-while` is the ONLY loop guaranteed to execute its body at least once — this is the single most tested while-vs-do-while fact.

- `for (init; condition; increment)` — **all three parts are optional**. `for(;;)` is a valid infinite loop. Multiple statements are allowed in init/increment via the **comma operator** (`for(i=1,j=2; j<=10; j++, i++)`), but only ONE expression (possibly combined with `&&` / `||`) is allowed in the condition slot.
- **"Odd loop":** a loop whose exact repeat count isn't known in advance (e.g., "keep asking until user enters 'n'") — typically written as `do-while(condition)` for readability, but can be written with `for(;;cond;)` or `while(cond)` identically.
- A loop counter need not be `int` — it can be `float` / `char` too, BUT: an integral counter that overflows its type's range **wraps around** (Ch.1 §1's cyclic nature) — e.g. `int i=1; while(i<=32767) {...i=i+1;}` on a 16-bit-int system never terminates: after 32767, `i+1` wraps to -32768, which still satisfies `i<=32767`, looping forever.

2.4 break vs continue

- **break:** exits the **nearest enclosing** loop or `switch` immediately — control jumps to the first statement AFTER that loop/switch.
- **continue:** skips the rest of the current iteration's body and jumps to the loop's **update/re-test step** (increments `for`'s counter, then re-checks condition; re-checks `while`'s condition directly).
- **TRAP:** `break` inside a `switch` that itself sits inside a loop only exits the **switch**, not the loop — a very common nested-control trap.
- **TRAP:** `continue` inside a loop does NOT restart the whole loop from scratch — it still runs the `for` loop's increment step before re-testing the condition.

2.5 switch Statement

```
switch (integer_or_char_expression) {
    case CONST1: stmt; break;
    case CONST2: stmt; break;
    default: stmt;
}
```

- Works only on **integral types** (`int/char` and expressions evaluating to them) — **NOT float, NOT string**, in standard C.
- Case labels must be **constant expressions** (e.g. `case 3+7:` is fine; `case a+b:` with variables is a compile error) and each case value must be **unique**.
- **Fall-through:** without a `break`, execution continues into the NEXT case (and the one after, etc.) regardless of whether its value matches — this is the #1 most-tested switch fact.
- `default` can appear **anywhere** in the switch (not necessarily last); if no case matches and there's no `default`, control simply falls through the entire switch with no action.
- Multiple case labels can share one action by stacking labels with no statement between them (e.g. `case 'a': case 'A': printf(...); break;`).
- Any statement written before the first `case` label inside the switch body is **unreachable** (the compiler won't error, but it never executes).
- **switch vs if-else ladder:** switch compiles to a **jump table** (O(1)-ish dispatch) — faster than an if-else ladder when there are MANY cases; but for very few (~2) simple conditions, an if-else ladder is actually faster since the jump-table lookup has its own overhead. switch also cannot test floats or range/complex conditions (only exact-match on an integral constant) — if-else has no such restriction.

2.6 goto

- `goto label;` jumps unconditionally to a `label:` anywhere in the same function.

- Generally discouraged (hurts readability/maintainability), but the one broadly-accepted legitimate use is breaking out of **deeply nested loops** in one step (since `break` only exits one level).

Questions

Q1. [Trap] What does the following print?

```
int i;
for (i = 5; i <= 10; i++);
    printf("%d", i);
```

A. 5 6 7 8 9 10 B. Prints nothing C. 11 (once) D. Infinite loop

Answer: C. Why: The semicolon right after the `for(...)` header makes the loop body an empty statement; the loop just runs to completion (`i` becomes 11) with no output, and THEN the un-braced `printf` (not part of the loop) executes exactly once, printing `11`.

Q2. [Trap] What is the output?

```
int i = 1;
while (i <= 3) {
    printf("%d ", i);
}
```

A. 1 2 3 B. Infinite loop printing "1 1 1 1..." forever C. Prints nothing D. Compile error

Answer: B. Why: `i` is never incremented inside the loop body, so the condition `i <= 3` never becomes false — `i` stays 1 forever, printing "1 " endlessly.

Q3. What does this print?

```
int i = 0;
do {
    printf("%d ", i);
    i++;
} while (i < 0);
```

A. Nothing (condition is false initially) B. 0 C. Infinite loop D. Compile error

Answer: B. Why: `do-while` always executes the body at least once BEFORE checking the condition — so `0` is printed, `i` becomes 1, then the false condition `1 < 0` stops the loop.

Q4. [Trap] Output of:

```
int i;
for (i = 1; i <= 3; i++) {
    if (i == 2) continue;
    printf("%d ", i);
}
```

A. 1 2 3 B. 1 3 C. 1 D. Infinite loop

Answer: B. Why: When `i==2`, `continue` skips the `printf` for that iteration only and jumps straight to `i++` /re-test; `i=1` and `i=3` both print normally.

Q5. [Numerical/Trap — GATE 2015] Consider:

```
int i, j, k = 0;
j = 2 * 3 / 4 + 2.0 / 5 + 8 / 5;
k -= --j;
for (i = 0; i < 5; i++) {
    switch (i + k) {
        case 1:
        case 2: printf("%d", i + k);
        case 3: printf("%d", i + k);
        default: printf("%d", i + k);
    }
}
```

How many times does a `printf` execute in total?

A. 8 B. 9 C. 10 D. 11

Answer: C. Why: $2*3/4 = 1$ (int div), $2.0/5 = 0.4$ (float), $8/5 = 1$ (int div) $\rightarrow j = 1 + 0.4 + 1 = 2.4$, truncated to **int `j=2`**. `--j` pre-decrements to 1, so `k = 0 - 1 = -1`. Now `i+k` for `i=0..4` is `-1,0,1,2,3`. `i=0` $\rightarrow$ no case matches $\rightarrow$ only `default` runs (1 `printf` each = 2). `i=2` (`i+k=1`) $\rightarrow$ falls through `case1` $\rightarrow$ `case2(print)` $\rightarrow$ `case3(print)` $\rightarrow$ `default(print)` = 3 `printfs`. `i=3` (`i+k=2`) $\rightarrow$ `case2(print)` $\rightarrow$ `case3(print)` $\rightarrow$ `default(print)` = 3 `printfs`. `i=4` (`i+k=3`) $\rightarrow$ `case3(print)` $\rightarrow$ `default(print)` = 2 `printfs`. Total = $1+1+3+3+2 = 10$.

Q6. [Trap — GATE 2012] What is the output?

```
char inchar = 'A';
switch (inchar) {
    case 'A': printf("choice A \n");
    case 'B': printf("choice B ");
    case 'C':
    case 'D':
    case 'E': default: printf("No Choice");
}
```

A. No choice B. Choice A C. Choice A Choice B No choice D. Program gives no output (erroneous)

Answer: C. Why: No `break` anywhere — matching `case 'A'` falls through `case 'B'` (prints "choice B "), then through the empty `case 'C' / 'D' / 'E'` labels into `default` (prints "No Choice"). All three `printfs` execute in sequence.

Q7. [Trap] What does `break` do when placed inside a `switch` that itself is inside a `for` loop?

A. Exits the for loop entirely B. Exits only the switch statement; the for loop continues to its next iteration C. Exits both the switch and the loop D. Causes a compile error

Answer: B. Why: `break` only exits the *nearest* enclosing loop or switch — here that's the switch, not the outer for loop.

Q8. What is printed?

```
int i, j;
for (i = 1, j = 2; i <= 2 && j <= 3; i++, j++)
    printf("%d-%d ", i, j);
```

A. 1-2 B. 1-2 2-3 C. 1-2 2-3 3-4 D. Compile error (comma not allowed in for)

Answer: B. Why: Comma operator allows multiple init/increment statements; loop runs while BOTH `i<=2` and `j<=3` hold: (i=1,j=2)→true→print; (i=2,j=3)→true→print; (i=3,j=4)→`i<=2` fails→stop. Prints "1-2 2-3".

Q9. [Trap] Dangling-else: what does this print for `a=5, b=-1` ?

```
if (a > 0)
    if (b > 0)
        printf("both positive");
    else
        printf("a positive, b not");
```

A. Nothing is printed B. "both positive" C. "a positive, b not" D. Compile error

Answer: C. Why: The `else` binds to the nearest unmatched `if` — the INNER `if(b>0)`, not the outer one. Since `a>0` is true, we enter the inner if; since `b>0` is false, its attached `else` fires, printing "a positive, b not".

Q10. A switch statement is generally FASTER than an equivalent if-else ladder mainly because:

A. switch uses less memory always B. The compiler builds a jump table, giving near-constant-time dispatch instead of sequentially evaluating each condition C. switch does not require a break statement D. if-else cannot use integer expressions

Answer: B. Why: A jump table lets execution go directly to the matching case's code; an if-else ladder must evaluate conditions one by one until a match — slower especially with many cases (though for very few simple conditions if-else can actually be faster due to jump-table overhead).

Q11. Which case values are ILLEGAL in a C switch statement?

A. `case 5:` B. `case 'A':` C. `case a+b:` (where a, b are variables) D. `case 3+7:`

Answer: C. Why: Case labels must be constant expressions evaluable at compile time; an expression involving variables is not allowed, while a constant arithmetic expression like `3+7` is fine.

Q12. What does `continue` do inside a `for` loop, precisely?

A. Immediately exits the loop B. Skips remaining body statements for this iteration, then still executes the increment step before re-testing the condition C. Restarts the loop from the initialization step D. Has no effect in a `for` loop

Answer: B. Why: Unlike in a `while` loop (where `continue` jumps straight to the condition test), in a `for` loop `continue` still triggers the increment expression before the condition is re-checked.

2.3 C Functions & Recursion

3.1 Function Basics

- A function is a self-contained block performing a coherent task. Every C program has at least one function; if only one, it must be `main()`. Execution always begins at `main()`.
- Calling function** (suspends, passes control) vs **called function** (runs, then returns control to the exact point after the call).
- A function can be called any number of times, from any other function (even recursively from itself); functions **cannot be defined inside another function** in C (no true nested function definitions).
- Library functions** (`printf`, `scanf` — precompiled, from standard library) vs **User-defined functions**.
- Why use functions:** avoid code repetition, and enable modular/independent design, testing and readability — do not cram all logic into `main()`.

3.2 Arguments, Return Values & Prototypes

- Actual arguments** (in the calling function) vs **formal arguments/parameters** (in the called function) — their type, order, and count must match; names may differ or coincide (compiler treats them as distinct variables regardless, due to separate scope).
- Declaration styles: K&R style `calsum(x,y,z) int x,y,z; {...}` (legacy) vs ANSI style `calsum(int x, int y, int z) {...}` (modern, standard practice).
- `return` serves two purposes: (1) immediately transfers control back to the caller, (2) optionally sends a value back. A function may have multiple `return` statements; `return;` (no value) yields a garbage/unspecified value if the caller expects one.
- A function can return only **one** value directly (`return(a, b);` is invalid — the comma operator would just evaluate both and return the last, not "return two values"). To return multiple values, use **pointers** (call by address) as out-parameters instead.
- Default return type:** a C function with no explicit return type defaults to `int` (pre-C99 behavior, still commonly tested) — **TRAP:** this is why a function meant to return `float` / `double` MUST have both its **definition** and a matching **prototype declaration** state the correct return type, or the value gets silently truncated/misinterpreted as `int`.
- `void` keyword: used when a function does not return any value (and, in a parameter list, `void` means "takes no arguments").
- Function prototype** (declares name, return type, and parameter types before use — lets the compiler catch call/definition mismatches) vs **function definition** (the actual body).

3.3 Parameter Passing — Call by Value (Default in C)

- C's **only direct mechanism is Call by Value:** the *value* of each actual argument is copied into the formal argument. Changes to the formal argument inside the called function **never** affect the caller's original variable.

```
void fun(int b) { b = 60; }
void main() { int a = 30; fun(a); printf("%d", a); } // prints 30 (unchanged) — fun's b=60 is a separate copy
```

- TRAP:** C has **no true "call by reference"** like C++. You can only *simulate* reference-like behavior by explicitly passing a **pointer** (the address) — see §3.4/Ch.4. Passing `&a` is still, technically, "call by value" of an address.
- Scope rule:** every variable's default scope is local to the function where it's declared — a variable in `main()` is invisible to another function unless explicitly passed as an argument (and vice versa for returning a value out).

3.4 Recursion

- A function that calls itself is **recursive**. Every correct recursive function needs a **base case** (stops the recursion) and a **recursive case** (reduces the problem and calls itself).
- Each recursive call gets its **own stack frame** (own copies of parameters and local variables) — pushed on the call stack; frames are popped as calls return. Excessive/infinite recursion (missing/wrong base case) causes **stack overflow**.
- Direct recursion** (function A calls A) vs **Indirect/mutual recursion** (A calls B, B calls A).
- Tail recursion:** the recursive call is the LAST operation performed (nothing happens after it returns) — some compilers can optimize this into an iterative loop internally, though C gives no guarantee of this. **Non-tail (head) recursion:** work happens *after* the recursive call returns (e.g., `f = x * rec(x-1);` — the multiplication happens after `rec(x-1)` returns), so a full stack of pending operations must be kept — this is NOT tail-recursive.

Worked Trace 1 — Factorial (non-tail recursion)

```
int rec(int x) {
    if (x == 1) return 1;
    else return x * rec(x - 1);
}
```

`rec(3) → 3 * rec(2) → 3 * (2 * rec(1)) → 3 * (2 * 1) = 6`. Each call must WAIT for the inner call to return before it can multiply — classic non-tail recursion; stack depth = 3 before unwinding begins.

Worked Trace 2 — Verified recursion (doubling recurrence)

```
int x(int n) {
    if (n < 3) return 1;
    else return x(n-1) + x(n-1) + 1; // = 2*x(n-1) + 1
}
```

Trace for $n=6$: $x(2)=1 \rightarrow x(3)=2(1)+1=3 \rightarrow x(4)=2(3)+1=7 \rightarrow x(5)=2(7)+1=15 \rightarrow x(6)=2(15)+1=31$.

TRAP: this function calls `x(n-1)` TWICE (not stored/reused) — so it recomputes the same sub-values repeatedly; total calls grow **exponentially** ($O(2^n)$) despite the answer having a simple closed form, a classic naive-recursion inefficiency (same root cause as naive Fibonacci recursion).

Questions

Q1. What does a C function return by default if no return type is specified in older/legacy code?

A. void B. float C. int D. Compilation error, always

Answer: C. Why: Pre-C99 (and still commonly tested), an unspecified return type defaults to `int` — this is why returning `float / double` requires explicit declaration + matching prototype.

Q2. [Trap] Which of the following correctly returns two values from a function to its caller?

A. `return (a, b);` B. `return a; return b;` (two statements) C. Pass pointers to `a` and `b` as parameters and modify `*a`, `*b` inside the function D. It's impossible in C

Answer: C. Why: A C function can only directly return ONE value; multiple "return values" require call-by-address (out-parameters via pointers). Option A just evaluates the comma expression and returns `b`'s value alone; option B never executes past the first `return`.

Q3. [Numerical] Trace this code — what is printed?

```
int rec(int x) {
    if (x == 1) return 1;
    else return x * rec(x - 1);
}
int main() { printf("%d", rec(5)); }
```

A. 5 B. 24 C. 120 D. 15

Answer: C. Why: This is $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$.

Q4. [Numerical] Trace this recursion for $n=6$:

```
int x(int n) {
    if (n < 3) return 1;
    else return x(n-1) + x(n-1) + 1;
}
```

A. 15 B. 21 C. 31 D. 63

Answer: C. Why: $x(2)=1$, $x(3)=3$, $x(4)=7$, $x(5)=15$, $x(6)=2(15)+1=31$ (each step doubles the previous result and adds 1).

Q5. [Trap] What is printed by the following program?

```
void junk(int i, int j) { i = i * i; j = j * j; }
int main() {
    int i = 5, j = 2;
    junk(i, j);
    printf("%d %d", i, j);
}
```

A. 25 4 B. 5 2 C. 25 2 D. 5 4

Answer: B. Why: Arguments are passed by VALUE — `junk`'s `i` and `j` are separate local copies; squaring them has no effect on `main`'s `i` and `j`, which remain 5 and 2.

Q6. Which statement about C's parameter-passing mechanism is correct?

A. C supports true call-by-reference like C++, using the `&` symbol in the function signature B. C only has call-by-value; reference-like behavior is simulated by explicitly passing pointers (addresses) C. C automatically passes arrays and structs by reference but scalars by value, with no other option D. Call-by-value and call-by-reference produce identical results in C

Answer: B. Why: Every argument in C is copied (call by value); passing a pointer just means the "value" being copied happens to be an address, letting the callee dereference it to reach the caller's original data.

Q7. A function is called "tail recursive" when:

A. It is the last function defined in the file B. The recursive call is the final action performed, with no pending computation after it returns C. It always uses a `for` loop instead of `if` D. It never has a base case

Answer: B. Why: In tail recursion, nothing happens after the recursive call returns — allowing (but not guaranteeing, in C) optimization into a loop; `return x * rec(x-1);` is NOT tail-recursive since the multiplication happens after the call returns.

Q8. [Trap] What happens when `main()` is called recursively from another function, as in:

```
void display() { printf("Hi"); main(); }
int main() { display(); }
```

A. Compilation error — `main()` cannot be called from another function B. It prints "Hi" once and terminates normally C. It repeats indefinitely, growing the call stack until a stack overflow occurs D. `main()` silently ignores the recursive call

Answer: C. Why: `main()` is a regular function in C and CAN be called recursively (a legal, if unusual, action); with no base case here, this recurses forever, consuming stack frames until the program crashes with a stack overflow.

Q9. Indirect (mutual) recursion refers to:

A. A function calling itself directly in its own body B. Function A calling function B, which in turn calls function A C. A loop disguised as a function call D. Recursion that uses no stack memory

Answer: B. Why: Indirect recursion involves two (or more) functions calling each other in a cycle, rather than a single function calling itself directly.

Q10. Why does naive recursive computation of $x(n) = x(n-1) + x(n-1) + 1$ (Q4) become inefficient for large n ?

A. It uses too much memory for a single stack frame B. It calls $x(n-1)$ twice without reusing the result, causing an exponential ($\sim O(2^n)$) number of total calls C. C cannot handle recursive functions with addition D. It is actually already optimally efficient at $O(n)$

Answer: B. Why: Since the same $x(n-1)$ is computed twice independently (not cached/memoized), the number of function calls roughly doubles at each level, giving exponential growth — the same inefficiency pattern as naive Fibonacci recursion.

2.4 C Arrays & Pointers

4.1 Array Basics

- Arrays store elements in **contiguous memory** (jumping from one element reaches the next directly) and are **homogeneous** (all elements same type). 0-indexed: valid indices are `0` to `size-1`.
- Size must be fixed at declaration UNLESS you initialize at the same time: `int arr[] = {10,20,30};` infers size=3 from the initializer count.
- Partial initialization** → remaining elements auto-set to 0: `int arr[5] = {1,2};` gives `{1,2,0,0,0}`.
- Uninitialized local array elements contain **garbage values** (not automatically zeroed unless explicitly initialized, or the array has static/global storage duration).
- Arrays are the underlying storage mechanism used to implement Stacks (LIFO) and Queues (FIFO) — see later DS chapters.

4.2 Pointer Basics

- A pointer is a variable that stores a memory **address**. `&` = "address-of" operator; `*` = "value-at-address" / **dereference** / indirection operator.
- Declaration `int *j;` means: "the value at the address stored in `j` is an int" — i.e., `j` is a pointer-to-int.
- `int *j = &i;` then `*j` and `i` refer to the exact same memory — reading/writing one is equivalent to the other.
- Pointer to pointer:** `int **k = &j;` — `*k` gives `j`'s value (an address), `**k` dereferences twice to reach `j`'s actual value. Chains can extend arbitrarily (pointer-to-pointer-to-pointer...).
- Pointers themselves always hold whole-number addresses regardless of the type they point to; `float *s` does NOT mean `s` holds a float — it means `s` holds the *address* of a float.

4.3 Arrays, Pointers & Address Arithmetic

- An array name **decays** to a pointer to its first element in most expressions: `arr` $\equiv$ `&arr[0]`.

```
arr[i] is EXACTLY equivalent to *(arr + i)      <- heavily tested equivalence
Loc(arr[i]) = Base_Address + i * sizeof(datatype)
```

- TRAP: Pointer arithmetic is scaled by type size**, not raw bytes — `p + 1` on an `int*` advances by `sizeof(int)` bytes (commonly 4 on a 32/64-bit system), not by 1 byte. E.g., if `int *p` holds address 1000, then `p+2` = address 1000 + 2×4 = **1008**, pointing to `arr[2]`.
- TRAP: Array name is a CONSTANT pointer** — `arr = arr + 1;` is a compile error. A pointer *variable* (`int *p = arr;`) CAN be reassigned: `p = p + 1;` is valid and moves `p` to the next element.
- Pointer subtraction works the same way in reverse: `p = p - 1;` moves backward by one element (scaled by type size).

4.4 Character Pointers & Strings

- A string is conceptually an array of characters terminated by `'\0'` (null character).
- `char str[] = "Namaste";` creates a **mutable copy** of the string in normal (writable) memory.
- `char *p = "Hello";` makes `p` point to a string **literal**, which many systems place in **read-only** memory.
- TRAP:** modifying a string literal through a `char*` (e.g., `p[0] = 'h';` on the literal-pointer version) is **undefined behavior** (often crashes) — but the same modification on the `char str[]` array copy is perfectly legal. This distinction (literal-via-pointer = read-only, array copy = mutable) is a classic C gotcha.

4.5 Call by Address (Pointers as Function Parameters)

- Passing `&variable` lets the called function dereference the pointer and modify the CALLER's actual variable — the closest C gets to "call by reference."
- A function can indirectly return **multiple** values this way (see §3.3) by writing through several pointer parameters.

Worked Trace — Call by Value vs Call by Address (mixed in one call)

```
void junk(int *i, int j) {
    *i = *i * *i;    // modifies caller's i via pointer (call by address)
    j = j * j;       // modifies only the LOCAL copy of j (call by value)
}
int main() {
    int i = 4, j = 2;
    junk(&i, j);
    printf("%d %d", i, j);
}
```

Trace: `*i` dereferences to 4, so `*i = 4*4 = 16` writes through to `main`'s `i`. `j` inside `junk` is a fresh copy (starts at 2, becomes 4), but this NEVER touches `main`'s `j`.

Output: 16 2 — **TRAP:** mixing call-by-value and call-by-address parameters in the SAME function call is a classic exam trick; you must track each parameter independently.

Questions

Q1. [Numerical] `int arr[5] = {10,20,30,40,50};` is stored starting at address 2000 (assume `int` = 4 bytes). What is the address of `arr[3]` ?

A. 2003 B. 2012 C. 2008 D. 2016

Answer: B. Why: `Loc(arr[3]) = Base + 3×4 = 2000 + 12 = 2012.`

Q2. Which expression is exactly equivalent to `arr[i]` ?

A. `&arr + i` B. `*(arr + i)` C. `*arr + i` D. `arr * i`

Answer: B. Why: Array subscripting `arr[i]` is defined in C to mean `*(arr + i)` — pointer arithmetic scaled by the element's type size, then dereferenced.

Q3. [Trap] Which of the following is a compile-time error?

A. `int *p = arr; p = p + 1;` B. `int arr[5]; arr = arr + 1;` C. `int *p; p = &arr[2];` D. `int *p = arr; p++;`

Answer: B. Why: `arr` (the array name) is a constant pointer and cannot be reassigned; a pointer *variable* like `p` can always be reassigned/incremented.

Q4. [Numerical] Given `int *p` holds address 5000 and `sizeof(int) == 4`, what address does `p - 2` refer to?

A. 4998 B. 4992 C. 5008 D. 4996

Answer: B. Why: Pointer arithmetic scales by type size: $5000 - 2 \times 4 = 5000 - 8 = 4992$.

Q5. [Trap] What is the likely behavior of the following code?

```
char *p = "Hello";
p[0] = 'h';
```

A. Always works fine, p[0] becomes 'h' B. Undefined behavior / likely crash — string literals are commonly stored in read-only memory C. Compile-time error, guaranteed D. p automatically becomes a new mutable copy

Answer: B. Why: p points to a string literal, which many implementations place in read-only memory; writing through it is undefined behavior. Using `char p[] = "Hello";` instead would create a safely mutable copy.

Q6. [Numerical/Trap] Trace this program's output:

```
void junk(int *i, int j) {
    *i = *i * *i;
    j = j * j;
}
int main() {
    int i = 4, j = 2;
    junk(&i, j);
    printf("%d %d", i, j);
}
```

A. 16 4 B. 4 2 C. 16 2 D. 4 4

Answer: C. Why: i is passed by address (*i modifies the caller's variable: $4 \times 4 = 16$), but j is passed by value (only the local copy is squared; caller's j stays 2).

Q7. Declaring `int **k;` means:

A. k is an array of 2 integers B. k is a pointer to an integer pointer (pointer-to-pointer) C. k is a double-precision integer D. Invalid syntax in C

Answer: B. Why: Each * adds one level of indirection; `int **k` holds the address of an `int*` variable, requiring two dereferences (`**k`) to reach the underlying int value.

Q8. Why can a C function only directly return one value, yet a function like `areaperi(int r, float *a, float *p)` can effectively produce two results?

A. It cannot — this is impossible in C B. It uses call-by-address: the function writes results into the caller's variables via pointers, rather than using return C. return in C accepts a comma-separated list D. C automatically packs multiple values into a struct behind the scenes

Answer: B. Why: By passing pointers to `area` and `perimeter`, the function dereferences and assigns to `*a / *p`, directly modifying the caller's variables — sidestepping the one-value-per-return limitation.

Q9. In `int arr[4] = {7};`, what are the values of `arr[1]`, `arr[2]`, `arr[3]`?

A. Garbage values B. 7, 7, 7 C. 0, 0, 0 D. Undefined/compile error

Answer: C. Why: Partial initialization sets all remaining, un-listed elements to 0 automatically — only `arr[0]` is explicitly set to 7.

Q10. A pointer variable `p` and an array name `arr` both can appear as `arr` or `p` in expressions referring to "the address of the first element." What is the key functional difference between them?

A. There is no difference whatsoever B. `arr` is a constant pointer (cannot be reassigned); `p` is a regular variable (can be reassigned to point elsewhere) C. `p` can only point to arrays, `arr` can point to any variable D. `arr` uses more memory than `p`

Answer: B. Why: The array name is fixed at compile time to the array's base address; a pointer variable is free to be redirected to any compatible address at runtime.

2.5 C Storage Classes, Structures, Unions & Enums

5.1 Storage Classes

Every C variable has both a **type** and a **storage class**. The storage class determines: where it's stored, its default initial value, its scope, and its lifetime.

Storage Class	Stored In	Default Value	Scope	Lifetime
auto (default)	Memory (stack)	Garbage	Local to block	Only while control is inside the block
register	CPU register (if available)	Garbage	Local to block	Only while control is inside the block
static	Memory	Zero	Local to block (or file, for global statics)	Entire program — value persists across calls
extern	Memory	Zero	Global (across files)	Entire program

- TRAP:** You **cannot take the address (&) of a register variable**, since it may not have a memory address at all (it may live only in a CPU register). Also, not every type fits in a register (e.g., `float` / `double` on a 16-bit-register machine) — the compiler silently falls back to `auto` behavior in that case.
- TRAP:** `static`'s defining exam behavior is that it is initialized **only once**, and **retains its value between function calls** — a `static` local variable used as a counter keeps incrementing across repeated calls, unlike an `auto` local which resets to its initial value every call.

Worked Trace — auto vs static

```
void increment_auto() { int i = 1;      printf("%d ", i); i++; }
void increment_static() { static int i = 1; printf("%d ", i); i++; }
// Called 3 times each:
increment_auto(); increment_auto(); increment_auto(); // prints: 1 1 1
increment_static(); increment_static(); increment_static(); // prints: 1 2 3
```

Scope Rule — Nested Blocks

```
main(){
    auto int i = 1;
    { auto int i = 2;
        { auto int i = 3; printf("%d ", i); } // prints 3 (innermost i)
        printf("%d ", i);                    // prints 2
    }
    printf("%d", i);                          // prints 1
}
// Output: 3 2 1 (each block's i is a completely separate variable)
```

- Local variable **always takes preference** over a global variable of the same name within the block where it's declared (name shadowing) — **TRAP:** this is a very common "what does this print" question.
- `extern` is a **declaration** (no memory reserved) vs a variable **definition** (memory reserved) — a variable can be *declared* many times but *defined* only once.
- A `static` global (file-scope) variable behaves like `extern` (zero default, whole-program lifetime) EXCEPT its scope is limited to the file it's declared in — it is NOT visible to other files, unlike a true `extern` / global.

5.2 Static vs Dynamic Scoping

- Static (Lexical) Scoping** — a variable reference is resolved by the program's **text structure** (nesting of blocks/functions as written), independent of the runtime call sequence. Resolved at **compile time**. **C, C++, Java, Python all use static scoping.**
- Dynamic Scoping** — a variable reference is resolved by the **most recent active call on the call stack** at runtime, not by lexical nesting. Resolved at **runtime**. Rare in modern languages (some Lisp dialects, `local` in Perl).

Worked Trace — proves C is statically scoped

```
int x = 10;
int f(){ return x; } // f is defined at FILE scope -> can only see global x
int g(){ int x = 20; return f(); } // g's local x=20 is NOT visible inside f()
int main(){ printf("%d", g()); }
// Output: 10 (NOT 20 - f() resolves x lexically to the global, ignoring g's local x
// even though g() is the one that called f())
```

TRAP: If this same code pattern ran under **dynamic scoping**, `f()` would instead pick up `g`'s most-recently-active `x = 20` from the call stack, printing **20**. The fact that real C prints **10** is the proof that C is lexically/statically scoped — a favorite conceptual GATE/ISRO question.

Aspect	Static Scoping	Dynamic Scoping
Resolved by	Program text (lexical nesting)	Call stack at runtime
Binding time	Compile time	Runtime
Speed	Faster (known at compile time)	Slower (stack traversal)
Predictability	High — read the source to know	Low — depends on execution path
Supports closures?	Yes	No

5.3 Structures

- A **structure** groups **heterogeneous** (possibly different-type) members under one name — contrast with an array, which groups **homogeneous** (same-type) elements.
- Declaration only defines the **form/type** — it reserves **no memory** until a variable of that struct type is declared.

- Access: **dot** (`.`) operator for a struct variable (`b1.pages`), **arrow** (`->`) operator for a struct pointer (`p->pages` , equivalent to `(*p).pages`).
- Members are stored in **contiguous** memory, in declaration order — BUT the compiler may insert **padding bytes** to align each member on its natural boundary.

Worked Trace — Structure Padding

```
struct Example { char c; int i; }; // naive size = 1 + 4 = 5 bytes
// Actual sizeof(struct Example) is typically 8 bytes on a system with 4-byte int alignment:
// c occupies offset 0 (1 byte), then 3 PADDING bytes (offsets 1-3) so that
// i can start at a 4-byte-aligned offset (offset 4), i occupies offsets 4-7.
```

TRAP: `sizeof(struct)` is often **larger** than the sum of its members' sizes, because of alignment padding — never assume struct size = sum of member sizes.

- Whole-structure **assignment** is allowed directly (`e3 = e2;` copies every member) — no need to copy member-by-member.

- **Array of structures** (`struct book b[100];`) is the standard way to store many records of the same struct type.

5.4 Union vs Structure

- A **union** looks like a struct syntactically, but **all members share the SAME memory location** — only one member holds a valid value at any time; writing to one member overwrites any other member's data.
- `sizeof(union)` = size of its **largest** member (not the sum) — direct contrast with a struct, whose size is roughly the **sum** of its members (plus padding).

Feature	Structure	Union
Memory	Each member gets its own space (sum, plus padding)	All members share one space
Size	~ sum of member sizes (+ padding)	Size of the largest member
Simultaneously valid members	All	Only one at a time
Use case	Represent a record with multiple independent fields	Represent one value that can be interpreted as different types

TRAP: In a union, setting one member and then reading a *different* member gives **garbage/reinterpreted bits**, not the value you set — this is by design, not a bug.

5.5 Enums

- `enum` defines a set of **named integer constants**. By default, the first name = **0**, and each subsequent name = **previous value + 1**.
- Custom values can be assigned to any constant; unassigned ones after a custom value continue incrementing from THAT value (not from 0).

```
enum Number { ONE = 5, TWO, THREE, FOUR = 20, FIVE };
// ONE=5, TWO=6, THREE=7, FOUR=20, FIVE=21
```

- Enum constants are internally just **integers** — `printf("%d", enumVar)` works directly.
- Limitations: fixed at compile time (can't add new constants at runtime), only integer-valued, not suited for large/dynamic value sets.

Questions

Q1. [Trap] What does the following program print?

```
void f() { static int i = 1; printf("%d ", i); i++; }
int main() { f(); f(); f(); return 0; }
```

A. 1 1 1 B. 1 2 3 C. 0 1 2 D. 3 3 3

Answer: B. Why: `static` initializes `i` only once; its value persists across calls, incrementing each time: 1, then 2, then 3.

Q2. Which storage class has a default (unassigned) value of garbage AND cannot have its address taken with `&`?

A. auto B. static C. extern D. register

Answer: D. Why: `register` variables may live in a CPU register (which has no memory address), so `&` on them is illegal; both auto and register default to garbage, but only register has the no-address restriction.

Q3. [Trap] What is the output?

```
int x = 10;
int f(){ return x; }
int g(){ int x = 20; return f(); }
int main(){ printf("%d", g()); return 0; }
```

A. 10 B. 20 C. Compile error D. Undefined behavior

Answer: A. Why: C uses static (lexical) scoping — `f()` is defined at file scope and can only see the global `x=10`; it has no visibility into `g`'s local `x=20` regardless of who called it.

Q4. [Numerical] Given `struct S { char c; int i; };` on a system where `int` requires 4-byte alignment, what is the most likely value of `sizeof(struct S)`?

A. 5 B. 8 C. 4 D. 1

Answer: B. Why: `char c` takes 1 byte, then 3 padding bytes are inserted so `int i` starts at a 4-byte-aligned offset — total 8 bytes, not the naive 1+4=5.

Q5. [Trap] Given `union U { int i; float f; char str[20]; };`, what is `sizeof(union U)`?

A. Sum of all members B. Size of the largest member (20, from `str`) C. Size of `int` only D. 0

Answer: B. Why: A union allocates only enough space for its largest member since all members share the same memory.

Q6. What are the values of RED, GREEN, BLUE in `enum Color { RED, GREEN = 20, BLUE };`?

A. 0, 20, 21 B. 0, 20, 1 C. 1, 20, 21 D. 0, 1, 2

Answer: A. Why: RED defaults to 0 (no prior value); GREEN is explicitly set to 20; BLUE continues incrementing from the previous value, becoming 21.

Q7. Accessing a member through a structure POINTER `p` uses which operator?

A. `.` B. `->` C. `::` D. `&`

Answer: B. Why: The arrow operator `p->member` is shorthand for `(*p).member`, used specifically when you have a pointer to a struct rather than the struct itself.

Q8. [Trap] Which of these correctly describes dynamic scoping (NOT used by standard C)?

A. Variable binding resolved by program text at compile time B. Variable binding resolved by the most recent active call on the runtime call stack C. Identical to static scoping for all practical purposes D. Used by C, C++, Java, and Python

Answer: B. Why: Dynamic scoping resolves a name using the call stack at runtime (so the same code can behave differently depending on who called whom), whereas C's static scoping resolves purely from lexical/text structure at compile time.

Q9. A local variable declared `auto` inside a block loses its value when:

A. The program ends B. Control exits the block in which it was declared C. Another function is called D. It never loses its value

Answer: B. Why: An `auto` variable's lifetime is scoped to its enclosing block — once control leaves that block, the variable (and its value) is gone.

2.6 Dynamic Memory Allocation, Macros & File Handling

6.1 Dynamic Memory Allocation (stdlib.h)

Function	Args	Initializes memory?	Notes
<code>malloc(size)</code>	total bytes	No (garbage)	Single argument; returns <code>void*</code> , or <code>NULL</code> on failure
<code>calloc(n, size)</code>	count, size-per-element	Yes, to 0	Two arguments; slower than <code>malloc</code> due to zeroing
<code>realloc(ptr, newsize)</code>	existing pointer, new size	Preserves old data; new region uninitialized	May move the block — always reassign the returned pointer
<code>free(ptr)</code>	pointer to release	—	Deallocates; does NOT set <code>ptr</code> to <code>NULL</code> automatically

```
int *p1 = (int*) malloc(10 * sizeof(int)); // 10 ints, garbage values
int *p2 = (int*) calloc(10, sizeof(int)); // 10 ints, all zero
```

- **TRAP: Memory leak** = allocated memory that is never `free()`'d — the pointer may go out of scope, making that memory permanently unreachable (and un-freeable) until the program exits.
- **TRAP: Dangling pointer** = using a pointer *after* it has been `free()`'d (or after the memory it pointed to has otherwise expired) — this is undefined behavior (may appear to "work" sometimes, may crash, may silently corrupt data). Always set a pointer to `NULL` after freeing it as a defensive habit.
- Static allocation (ordinary variables, fixed array sizes) is decided at **compile time** on the **stack**; dynamic allocation (`malloc/calloc/realloc`) is decided at **runtime** on the **heap** — heap memory persists until explicitly freed, unlike stack memory which is automatically reclaimed when a block/function exits.

6.2 Static vs Dynamic Scoping

(See Ch.5 §5.2 for the full treatment — C uses **static/lexical** scoping exclusively; a variable reference is resolved by the program's *text* structure, not by the runtime call chain.)

6.3 Macros (`#define`)

- A macro is a **preprocessor** directive: pure **textual substitution** performed *before* compilation — NOT a function call, so there is **no type checking** and **no call overhead**.
- **Object-like macro**: `#define MAX 100` — simple constant substitution.
- **Function-like macro**: `#define SQUARE(x) ((x)*(x))` — takes parameters, but substitutes literally (text-for-text), rather than evaluating like a real function.
- Benefits: faster (no call overhead), but can cause **code bloat** if used heavily (each call site gets its own copy of the expanded code, unlike a function which has one shared body).

TRAP — Unparenthesized Macro Expansion

```
#define abc(x) x*x
int a = 64 / abc(4);
// Expands LITERALLY to: a = 64 / 4*4;
// / and * have EQUAL precedence, left-to-right: (64/4)*4 = 16*4 = 64
// a = 64 -- NOT 4, which is what you'd get if abc(4) were evaluated as a real function first!
```

Always wrap BOTH the macro parameter AND the whole expression in parentheses: `#define SQUARE(x) ((x)*(x))` to avoid precedence-driven surprises like this.

TRAP — Macros Don't Substitute Inside String Literals

```
#define mno pqr
printf("mno"); // Prints: mno (NOT pqr — the preprocessor does not touch text inside quotes)
```

Token-Pasting Operator `##`

```
#define fun(i, j) i##j
int f1 = 10, f12 = 20;
printf("%d", fun(f1, 2)); // fun(f1,2) -> f1##2 -> pastes into identifier "f12"
// f12 = 20, so this prints: 20
```

TRAP — Macro-Redefining a Keyword

```
#define int char
int a = 10, *p = &a;
*p = 20;
printf("%d %d", sizeof(a), sizeof(p));
// After preprocessing, EVERY token "int" becomes "char":
// char a = 10, *p = &a; (p is now a "char *")
// sizeof(a) = 1 (sizeof(char))
// sizeof(p) = size of a pointer (architecture-dependent: commonly 4 on a 32-bit system, 8 on 64-bit)
// Output (32-bit assumption): 1 4
```

This shows macros perform **blind textual replacement** of the exact token — even a language keyword can be "redefined" this way (a favorite ISRO/GATE trick question).

6.4 File Handling in C

- Purpose: persist data beyond program execution (unlike variables, which vanish when the program ends).
- **Text files** (human-readable, line-based) vs **Binary files** (raw bytes, not human-readable, more compact/faster for structured data).

Core Workflow

```
FILE *fp = fopen("data.txt", "r"); // 1. Open – ALWAYS check fp != NULL before use
if (fp == NULL) { /* handle error */ }
// 2. Perform operations (read/write/append)
fclose(fp); // 3. Close – flushes buffers, releases resources
```

fopen() Modes

Mode	Meaning	If file doesn't exist
r	Read only	Returns NULL (fails)
w	Write (truncates /overwrites existing content)	Creates new file
a	Append (writes go to the end)	Creates new file
r+	Read + Write, starts at beginning	Fails
w+	Read + Write, truncates existing content	Creates new file
a+	Read + Append	Creates new file

- Append **b** to any mode (e.g. **rb**, **wb+**) for **binary mode**.
- TRAP:** "w" and "w+" **destroy existing file content immediately upon opening** — a very common accidental-data-loss trap. Use "a" / "a+" if you want to preserve existing content.

Read/Write Function Pairs

Write	Read	Purpose
fprintf()	fscanf()	Formatted (like printf/scanf but to/from a file)
fputs()	fgets()	Whole line
fputc()	fgetc()	Single character
fwrite()	fread()	Raw bytes (binary files)

- feof(fp) — tests for End-Of-File; important to check while reading in a loop to avoid reading past the end (undefined behavior).
- fseek(fp, offset, pos) — moves the file pointer; pos is a reference point (SEEK_SET = start, SEEK_CUR = current, SEEK_END = end), offset can be negative.
- ftell(fp) — returns the current file-pointer position (byte offset from start).
- rewind(fp) — resets the file pointer to the very beginning (equivalent to fseek(fp, 0, SEEK_SET)).

Questions

Q1. [Trap] What is the output?

```
#define abc(x) x*x
int main(){ int a = 64/abc(4); printf("%d", a); return 0; }
```

A. 4 B. 16 C. 64 D. 1

Answer: C. Why: Textual expansion gives `64/4*4`; / and * share precedence and associate left-to-right: $(64/4)4 = 164 = 64$.

Q2. [Trap] What is the output?

```
#define mno pqr
int main(){ printf("mno"); return 0; }
```

A. pqr B. mno C. Compile error D. Empty output

Answer: B. Why: The preprocessor never substitutes tokens that appear inside a string literal — "mno" is printed exactly as written.

Q3. Which dynamic memory function BOTH allocates memory AND initializes it to zero?

A. malloc() B. calloc() C. realloc() D. free()

Answer: B. Why: calloc(n, size) is the only one of the three allocation functions that zero-initializes the memory; malloc leaves it uninitialized (garbage).

Q4. [Trap] Using a pointer after calling free() on it, without setting it to NULL, is called a:

A. Memory leak B. Dangling pointer C. Segmentation table D. Buffer overflow

Answer: B. Why: A dangling pointer refers to freed (deallocated) memory; using it afterward is undefined behavior — distinct from a memory leak, which is forgetting to free memory at all.

Q5. Which fopen() mode will DESTROY the existing contents of a file the moment it is opened?

A. "r" B. "a" C. "w" D. "a+"

Answer: C. Why: Mode "w" truncates (erases) any existing file content immediately on open; "a"/"a+" preserve existing content and only add to the end; "r" doesn't modify content at all (and fails if the file doesn't exist).

Q6. [Numerical] Given `int f1 = 10, f12 = 20;` and `#define fun(i,j) i##j`, what does `fun(f1, 2)` evaluate to?

A. 10 B. 20 C. 12 D. Compile error

Answer: B. Why: The ## operator pastes tokens together: `fun(f1,2)` becomes the single identifier `f12`, whose value is 20.

Q7. Which function moves the file pointer back to the very beginning of a file?

A. fseek() only B. ftell() C. rewind() D. feof()

Answer: C. Why: rewind(fp) resets the position to the start; it's equivalent to fseek(fp, 0, SEEK_SET) but simpler to use for this specific purpose.

Q8. [Trap] What is a key difference between a macro and a real C function?

A. Macros support type checking, functions don't B. Macros are expanded textually at compile time with no call overhead; functions involve an actual call with type checking C. Functions are always faster than macros D. There is no meaningful difference

Answer: B. Why: Macros are pure preprocessor text substitution — no type checking, no call overhead, but can cause code bloat; functions are compiled once and called, with full type checking, at some (usually small) call overhead.

Q9. [Trap] After `#define int char`, what does `int a = 10, *p = &a;` actually declare (post-preprocessing)?

A. a as an int, p as an int pointer B. a as a char, p as a char pointer C. A compile error D. No change — macros can't redefine keywords

Answer: B. Why: The preprocessor performs blind textual replacement of every occurrence of the token "int" with "char" before compilation even begins — this includes keyword tokens, producing `char a = 10, *p = &a; .`

2.7 OOP with C++ — Classes, Objects, Constructors & Overloading

7.1 C vs C++ and the OOP Paradigm

- **C** (Dennis Ritchie, 1972) is procedural/function-oriented. **C++** (Bjarne Stroustrup, 1985) extends C by adding Object-Oriented Programming while keeping all of C's features — so C++ supports BOTH procedural and OOP styles.
- **Middle-level language:** C++ gives low-level features (direct memory access via pointers, fast execution) AND high-level features (functions, portability, readability) in one language.

Feature	Structured (Procedural) Programming	Object-Oriented Programming
Organized around	Functions/procedures	Objects (data + methods bundled)
Approach	Top-down	Bottom-up
Data security	Weak — data is often global/shared	Strong — encapsulation restricts access
Reusability	Harder	Easier (inheritance, classes)
Example languages	C	C++, Java, Python, C#

- Key OOP pillars introduced by C++ (none of these exist in plain C): **Classes & Objects, Inheritance, Polymorphism, Encapsulation, Abstraction, Function/Operator Overloading, Templates.**

7.2 Classes & Objects

- **Class** = user-defined data type / blueprint bundling data members + member functions. **Object** = an instance of a class; memory is allocated only when an object is created, and each object has its own independent copy of the data members (but shares the same member function code).
- Analogy: Class = house blueprint, Object = an actual house built from it.

```
class Student {
    int roll;
public:
    void display() { cout << roll; }
};
Student s1, s2; // two independent objects
```

- **this pointer:** implicitly passed to every non-static member function; holds the address of the CURRENT object. Used to (a) resolve name conflicts between a data member and a same-named parameter, (b) return the current object via `return *this;` (enables chained calls like `obj.set(1).set(2)`).
- **Scope Resolution Operator (::)** lets C++ access a global variable that is hidden ("shadowed") by a local variable of the same name (`::x` reaches the global `x`) — this is a problem C had NO fix for; C++ solved it with `::`.

7.3 struct vs class in C++

Feature	struct	class
Default member access	public	private
Default inheritance mode	public	private
Typical use	Simple data grouping	Full OOP (encapsulation-focused)

- **TRAP:** In C++ (unlike plain C), a `struct` CAN have member functions, constructors, and destructors too — the ONLY hard default difference from `class` is the default access specifier (public vs private).

7.4 Reference Variables

- A reference (`int &ref = x;`) is an **alias** for an existing variable — it does NOT get its own memory; it must be initialized at declaration, and it can **never be re-seated** to refer to a different variable afterward.
- **TRAP:** A pointer CAN be reassigned to point elsewhere later; a reference CANNOT — this is the key pointer-vs-reference distinction.

7.5 Inline Functions

- Requests the compiler to substitute the function's code directly at the call site instead of a real function call → removes call overhead, best for small, frequently-called functions.
- Functions **defined inside a class body are implicitly inline** by default.
- The compiler MAY ignore the inline request (it is only a *request*, never a guarantee) when the function contains loops, a switch, is recursive, contains goto, or is simply too large/complex.

7.6 Static Members

- **Static data member:** belongs to the CLASS, not to any object — only ONE copy exists no matter how many objects are created; all objects share it, and a change by one object is visible to all. Must be defined/initialized OUTSIDE the class using `::` (e.g. `int Student::count = 0;`).
- **Static member function:** can be called via the class name (`ClassName::func()`) without any object; can ONLY access other static members directly (no `this` pointer exists for it) — it CANNOT touch non-static/instance data or call non-static member functions directly.
- **TRAP:** A static member function has no `this` pointer — trying to access an ordinary (non-static) data member inside it is a compile error.

7.7 Friend Functions & Friend Classes

- A **friend function** is a non-member function explicitly granted access to a class's private/protected members (declared with `friend` inside the class, defined outside it). Commonly needed for operator overloading that must touch both operands' private data.

- A **friend class**: ALL member functions of the friend class get access to the granting class's private/protected members.
- Key friendship rules: **NOT mutual** (A befriends B doesn't mean B befriends A), **NOT inherited**, **NOT transitive**.

Friend Function	Friend Class
Only ONE specific function gets access	ALL member functions of that class get access
<code>friend return_type f(...);</code>	<code>friend class ClassName;</code>

7.8 Constructors & Destructors

- **Constructor**: special member function, SAME NAME as the class, auto-invoked on object creation, NO return type (not even void), can be overloaded, CANNOT be called explicitly via `.`, `::`, and CANNOT be virtual.
- If the programmer defines ANY constructor, the compiler does **not** generate its own default (no-argument) constructor — **TRAP**: this means providing only a parameterized constructor and then trying `MyClass obj;` (no args) will fail to compile.
- Types: **Default** (no args) · **Parameterized** (init via arguments) · **Copy constructor** `ClassName(const ClassName &obj)` — invoked automatically when one object is initialized from another (`Temp t2 = t1;`, pass-by-value, return-by-value).
- **Shallow copy vs Deep copy** (the single most tested constructor concept):

	Shallow Copy	Deep Copy
What's copied	The pointer VALUE (address) only	New memory allocated + actual pointed-to DATA copied
Result	Both objects point to the SAME memory	Each object has independent memory
Risk	One object's change affects the other; double-free on destruction	None — fully independent
Who provides it	Compiler's default copy constructor (automatic)	Programmer must write a custom copy constructor

- **TRAP**: The **compiler-generated default copy constructor always performs a shallow copy** — if a class holds a raw pointer to dynamically allocated memory, you MUST write your own deep-copy constructor or risk both objects freeing the same memory.
- **Destructor**: name = `~ClassName`, no arguments (so it can NEVER be overloaded), no return type, auto-invoked when an object is destroyed/goes out of scope, executes in REVERSE order of construction, and (unlike constructors) a destructor **CAN be declared virtual** (essential in inheritance — see Ch.8).

7.9 Function Overloading vs Operator Overloading

- **Function Overloading**: same name, different parameter list (count/type/order) — resolved at **compile time** = "Compile-Time / Static Polymorphism."
- **Operator Overloading**: redefining an existing operator to work with a user-defined (class) type, via a member function or a friend function.

Rules: only EXISTING operators can be overloaded; no new operator can be invented; at least one operand must be a user-defined object; operator precedence/associativity/arity (number of operands) CANNOT be changed; built-in-type-only operations cannot be overloaded.

- Operators that can **NEVER** be overloaded: `.` (member access), `::` (scope resolution), `sizeof`, `?:` (ternary), `.*` (pointer-to-member).
- Operators that **MUST** be overloaded as a member function (friend function NOT allowed): `=` (assignment), `[]` (subscript), `()` (function call), `->` (member access through pointer).
- **TRAP**: Students often assume ANY operator can be overloaded via a friend function — the four operators above are a hard exception and must be true class members.

Questions

Q1. [Trap] What is the ONLY default difference between a `struct` and a `class` in C++?

A. struct cannot have member functions B. struct members default to public, class members default to private C. struct cannot have constructors D. struct cannot be inherited from

Answer: B. Why: In C++, struct and class are functionally identical except for their default access specifier (public vs private) and default inheritance mode.

Q2. A reference variable differs from a pointer in that:

A. A reference can be re-seated to refer to another variable; a pointer cannot B. A reference must be initialized at declaration and can never be re-seated; a pointer can be reassigned freely C. A reference occupies its own separate memory like a pointer does D. There is no difference

Answer: B. Why: References are permanent aliases fixed at initialization; pointers are independent variables that can be redirected to point elsewhere at any time.

Q3. [Trap] Which of these correctly describes a static member function's capability?

A. It can access both static and non-static members directly B. It can access ONLY static members directly, since it has no `this` pointer C. It requires an object to be called D. It is automatically virtual

Answer: B. Why: A static member function is not tied to any specific object (no `this` pointer), so it can only directly touch other static members.

Q4. What does the compiler-generated DEFAULT copy constructor perform when a class contains a raw pointer member?

A. A deep copy automatically B. A shallow copy — copying the pointer's address, not the pointed-to data C. It refuses to compile D. It converts the pointer to a reference

Answer: B. Why: The default copy constructor copies member values as-is; for a pointer member, that means copying the address, leaving both objects pointing at the same memory (shallow copy) — a classic source of double-free bugs.

Q5. [Trap] Which of the following operators CANNOT be overloaded in C++?

A. `+` B. `==` C. `sizeof` D. `[]`

Answer: C. Why: `sizeof`, `.`, `::`, `?:`, and `.*` can never be overloaded; `+`, `==`, and `[]` all can be.

Q6. Which operator MUST be overloaded only as a member function, never as a friend function?

A. `+` B. `-` C. `[]` D. `*`

Answer: C. Why: `=`, `[]`, `()`, and `->` require member-function overloading — this is a hard C++ rule.

Q7. Function Overloading is resolved:

A. At compile time, based on the number/type of arguments B. At runtime, based on the object's actual type C. Only when a virtual keyword is used D. Randomly by the linker

Answer: A. Why: Function overloading is Compile-Time (Static) Polymorphism — the compiler picks the matching overload based on argument signature before the program ever runs.

Q8. [Trap] If a class defines only a parameterized constructor `MyClass(int x)` and no other constructor, what happens with `MyClass obj;` ?

A. It compiles fine using an implicit default constructor B. Compile error — no matching default constructor exists C. `obj` is created with `x=0` D. `obj` is created with

garbage x

Answer: B. Why: Once ANY user-defined constructor exists, the compiler stops auto-generating a default (no-arg) constructor — `MyClass obj;` now has no matching constructor and fails to compile.

Q9. A friend function is best described as:

A. A member function with private access B. A non-member function explicitly granted access to a class's private/protected members C. Automatically inherited by derived classes D. A function that grants access mutually between two classes

Answer: B. Why: Friend functions are NOT members, and friendship is neither mutual, inherited, nor transitive — it's a one-way, explicit grant.

Q10. [Code trace] What does this print?

```
class Counter {
    static int count;
public:
    Counter() { count++; }
    static int getCount() { return count; }
};
int Counter::count = 0;
int main() {
    Counter a, b, c;
    cout << Counter::getCount();
}
```

A. 0 B. 1 C. 3 D. Compile error

Answer: C. Why: `count` is static — shared by all objects; each of the 3 constructor calls (a, b, c) increments the SAME copy, so it reaches 3.

2.8 OOP with C++ — Inheritance, Polymorphism & Templates

8.1 Inheritance Basics & Access-Specifier Interaction

- **Inheritance**: a derived (child/sub) class acquires the members of a base (parent/super) class — models an **IS-A** relationship, promotes reusability.
- Access specifiers: **public** (accessible anywhere), **private** (only inside the same class — default for a class), **protected** (inside the same class AND derived classes, but not from outside via an object).
- How a base member's access level changes under each inheritance mode:

Base Member	Private Inheritance	Protected Inheritance	Public Inheritance
private	Not Inherited (NI)	Not Inherited (NI)	Not Inherited (NI)
protected	becomes Private	stays Protected	stays Protected
public	becomes Private	becomes Protected	stays Public

- **TRAP**: A base class's `private` members are **NEVER directly inherited/accessible** in the derived class under ANY inheritance mode — only `protected` / `public` base members pass through (with visibility possibly narrowed).

8.2 Types of Inheritance

Type	Structure
Single	1 derived class ← 1 base class
Multiple	1 derived class ← 2+ DIFFERENT base classes
Hierarchical	2+ derived classes ← 1 common base class
Multilevel	Chain: A ← B ← C (B is both a derived class of A and a base class of C)
Hybrid	Combination of two or more of the above (e.g. Multiple + Multilevel)

- Note: constructors are **never inherited** — each class must define/inherit-call its own.

8.3 Diamond Problem vs Ambiguity Problem (classic trap pair)

	Ambiguity Problem	Diamond (Hybrid) Problem
Cause	Two UNRELATED base classes happen to have a same-named member	The SAME base class is reached through two different inheritance paths (multiple + hierarchical combined)
Effect	Naming conflict — compiler can't tell which member you mean	Derived class ends up with 2 DUPLICATE copies of the common ancestor's members
Fix	Scope Resolution Operator (<code>Base::member</code>) or override in derived class	Virtual Inheritance (<code>class B : virtual public A</code>) — ensures only ONE shared copy of A

- **TRAP**: Do not conflate these — Ambiguity is about **name conflicts** between unrelated bases (fixed with `::`); Diamond Problem is about **duplicate copies of a shared ancestor** (fixed with `virtual` inheritance). Using `::` does NOT fix the Diamond Problem's duplication.

```
class A {};
class B : virtual public A {};
class C : virtual public A {};
class D : public B, public C {}; // D has exactly ONE A subobject (thanks to `virtual`)
```

8.4 Constructor/Destructor Order in Inheritance

```
Object creation:   Base class constructor -> Derived class constructor
Object destruction: Derived class destructor -> Base class destructor (exact REVERSE order)
```

- If the base class has ONLY a parameterized constructor (no default), the derived class's constructor **must** explicitly forward arguments via an initializer list: `Derived(int y) : Base(y) { }` — omitting this is a compile error ("no matching function for call to Base()").

8.5 Object Slicing

- Assigning a derived-class object to a base-class object **by value** copies ONLY the base-class portion — the derived-specific members are "sliced off" and lost.

```
class Employee { public: string name; };
class Manager : public Employee { public: int teamSize; };
Manager m; Employee e = m; // Object Slicing: e.teamSize doesn't exist, only e.name survives
```

- **TRAP**: Object slicing silently **disables runtime polymorphism** for that copy (since only the base part remains) — this is why polymorphic objects are almost always accessed via base-class **pointers/references**, never assigned by value.

8.6 Polymorphism — Compile-Time vs Runtime

	Compile-Time (Static) Polymorphism	Runtime (Dynamic) Polymorphism
Achieved via	Function Overloading, Operator Overloading	Function Overriding + Virtual Functions
Resolved	At compile time (by the compiler)	At execution time (dynamic dispatch)
Also called	Early Binding	Late Binding
Needs <code>virtual</code> ?	No	Yes (at least one virtual function)
Speed	Faster	Slightly slower (vtable lookup overhead)

- **Function Overloading** (same name, different params, same class, compile-time) vs **Function Overriding** (same name + same params + same return type, Base-Derived relationship, requires the base function to be `virtual`, resolved at runtime) — a frequently confused pair.
- **Function Overriding conditions:** inheritance relationship + identical signature + base function marked `virtual` + accessed via a base-class pointer/reference.

8.7 Virtual Functions

- A `virtual` base-class function lets a **derived class override it**; when called through a **base-class pointer/reference**, the DERIVED class's version actually executes — this is the entire mechanism behind runtime polymorphism (dynamic dispatch via the vtable).

```
class Animal { public: virtual void sound() { cout << "..."; } };
class Dog : public Animal { public: void sound() override { cout << "Bark"; } };
Animal *ptr = new Dog();
ptr->sound(); // prints "Bark" – the DERIVED version, even though ptr's static type is Animal*
```

- Rules: must be a member function (not static, not a constructor); once declared `virtual` in the base, it **stays virtual** in every derived override automatically (no need to repeat the keyword, though `override` is good practice); **destructors CAN be virtual** (and should be, in a polymorphic base class); **friend functions cannot be virtual** (they aren't class members).
- **TRAP: Virtual Destructor** — `Base *ptr = new Derived(); delete ptr;` — if `~Base()` is NOT virtual, only the base destructor runs, leaking any resources the derived part allocated. ALWAYS make a base class destructor virtual if the class will be used polymorphically through base pointers.

8.8 Pure Virtual Functions & Abstract Classes

- **Pure virtual function:** `virtual void show() = 0;` — no implementation in the base class.
- A class with **at least one** pure virtual function becomes an **Abstract Class** — it CANNOT be instantiated directly, and exists only to define a common interface/contract.
- A derived class becomes **Concrete** (instantiable) only once it implements **ALL** inherited pure virtual functions — if even one remains unimplemented, the derived class is ALSO still abstract.
- Constructors can never be virtual (pure or otherwise); a base-class pointer/reference CAN still point to a derived (concrete) object even though the base itself is abstract.

Virtual Function	Pure Virtual Function
Has a body in the base class	No body in base (= 0)
Overriding is optional	Overriding is MANDATORY (else derived stays abstract)
Base class is concrete	Base class becomes Abstract

8.9 Templates & STL

- **Templates** write one generic function/class that works across data types without duplicating code: **Function templates** (`template<class T> T add(T a, T b) { ... }`) and **Class templates**.
- **STL (Standard Template Library):** ready-made generic Containers (vector, list, stack, queue, map, set), Algorithms (sort, search), and Iterators (traverse containers) — the 3 core STL components.
- **vector** = STL's dynamic array (`#include <vector>`) — automatically grows/shrinks, unlike a plain C-style array whose size is fixed at declaration.

Questions

Q1. [Code trace] What is printed?

```
class Base {
public:
    Base() { cout << "B"; }
    ~Base() { cout << "b"; }
};
class Derived : public Base {
public:
    Derived() { cout << "D"; }
    ~Derived() { cout << "d"; }
};
int main() { Derived obj; }
```

A. BDdb B. DBbd C. BDdb D. DBdb

Answer: C. Why: Construction order is Base then Derived ("BD"); destruction is the exact reverse, Derived then Base ("db") — giving "BDdb".

Q2. [Trap] What is the key difference between the Ambiguity Problem and the Diamond Problem in multiple inheritance?

A. They are the same thing B. Ambiguity is a name conflict between unrelated base classes (fixed with `::`); Diamond is duplicate copies of a shared ancestor (fixed with virtual inheritance) C. Diamond is fixed with `::`; Ambiguity with virtual inheritance D. Neither can be fixed in C++

Answer: B. Why: These are distinct problems with distinct fixes — conflating them (or swapping their fixes) is the classic exam trap.

Q3. [Code trace] What does this print?

```

class Animal {
public:
    virtual void sound() { cout << "Generic sound"; }
};
class Cat : public Animal {
public:
    void sound() override { cout << "Meow"; }
};
int main() {
    Animal *ptr = new Cat();
    ptr->sound();
}

```

A. Generic sound B. Meow C. Compile error D. Undefined behavior

Answer: B. Why: `sound()` is virtual, so calling it through a base pointer dispatches to the actual object's (Cat's) override at runtime — the definition of runtime polymorphism.

Q4. [Trap] Why is a virtual destructor essential in a polymorphic base class?

A. It makes the class faster B. Without it, deleting a derived object through a base pointer only calls the base destructor, leaking derived-class resources C. It is required by the C++ standard for all classes D. It prevents object slicing

Answer: B. Why: `Base *ptr = new Derived(); delete ptr;` with a non-virtual `~Base()` only runs `~Base()`, skipping `~Derived()` and any cleanup it performs.

Q5. A class containing at least one pure virtual function is called:

A. A concrete class B. A friend class C. An Abstract Class D. A static class

Answer: C. Why: By definition, ≥ 1 pure virtual function (`= 0`) makes a class abstract — it cannot be instantiated until a derived class implements all such functions.

Q6. [Trap] What happens if a derived class of an abstract base implements only SOME (not all) of the inherited pure virtual functions?

A. It becomes concrete and can be instantiated B. It remains abstract and still cannot be instantiated C. Compile error immediately D. The unimplemented functions default to no-ops

Answer: B. Why: ALL pure virtual functions must be overridden for a derived class to become concrete; partial implementation leaves it abstract too.

Q7. Object Slicing occurs when:

A. A derived object is accessed via a base-class pointer B. A derived-class object is assigned BY VALUE to a base-class object, losing the derived-only members C. A base class has a virtual destructor D. Multiple inheritance is used

Answer: B. Why: Value assignment only copies the base-class portion of the object; the derived part is "sliced off" — polymorphism is also broken in the sliced copy.

Q8. [Code trace] Given:

```

class Base { public: Base(int x) { cout << x; } };
class Derived : public Base {
public:
    Derived() { }
};

```

What happens when you try `Derived d;` ?

A. Compiles fine, prints nothing B. Compile error — no matching `Base()` constructor C. Compiles, prints 0 D. Runtime crash

Answer: B. Why: Base has no default constructor, and Derived's constructor doesn't forward any argument to it — the compiler cannot find a `Base()` to call implicitly.

Q9. Function Overriding requires all of the following EXCEPT:

A. Inheritance relationship B. Same function name and parameter list C. The base function must be declared virtual D. The function must be static

Answer: D. Why: Overriding needs a virtual (non-static) base function; a static function cannot be virtual and cannot be overridden in the polymorphic sense.

Q10. Virtual Inheritance is specifically used to solve:

A. The Ambiguity Problem B. The Diamond Problem — ensuring only one shared copy of a common base C. Object slicing D. Function overloading conflicts

Answer: B. Why: `virtual public Base` in both intermediate classes ensures the final derived class contains only one Base subobject instead of two duplicate copies.

Q11. Which STL component is responsible for traversing a container like `vector` or `list` ?

A. Algorithm B. Iterator C. Template D. Functor

Answer: B. Why: Iterators are STL's mechanism for moving through container elements; Containers store data and Algorithms (like sort) operate on it via iterators.

Q12. [Trap] Compile-time polymorphism differs from runtime polymorphism in that it:

A. Requires at least one virtual function B. Is resolved by the compiler before the program runs, via overloading — no virtual function needed C. Is always slower due to dynamic dispatch D. Only applies to operators, never functions

Answer: B. Why: Compile-time (static) polymorphism = function/operator overloading, resolved purely by signature matching at compile time; runtime polymorphism specifically needs virtual functions and dynamic dispatch.

2.9 Data Structures Basics & Arrays

9.1 Classification of Data Structures

- **Primitive:** system-defined, directly operated on by machine instructions — int, float, char, pointer.
- **Non-Primitive:** built from primitives — **Derived** (Arrays — system-provided but more complex) and **User-Defined** (Linked List, Stack, Queue, Tree, Graph — custom structure + operations).
- **Linear** (single level, elements have exactly one predecessor/successor, traversed in one run): Array, Stack, Queue, Linked List.
- **Non-linear** (hierarchical, multiple levels, cannot be traversed in a single run): Tree, Graph.
- **Homogeneous** (all elements same type — Array) vs **Heterogeneous** (mixed types — Struct, Union).
- **TRAP:** An array is a **derived**, not primitive, data structure — it's built on top of primitive types.

9.2 Arrays — Basics

- Contiguous memory, same data type, accessed via index in **O(1)**.
- Size fixed at declaration (C): $\text{Size} = (\text{Upper Bound} - \text{Lower Bound} + 1) * \text{element size}$.
- **Advantages:** O(1) random access, cache-friendly, simple. **Disadvantages:** fixed size, costly insertion/deletion (O(n) shifting), homogeneous only.
- Indexing conventions: 0-based (C/Java), 1-based, or n-based (arbitrary/negative lower bound — common in GATE-style questions).

9.3 1D Array Addressing

$\text{Address}(a[k]) = B + W * (k - \text{LowerBound})$
 B = base address, W = size of each element, k = index

Worked Example

Array $a[-6..6]$, W=4 bytes, Base=3500. Find address of $a[0]$.
 Address = $3500 + 4(0 - (-6)) = 3500 + 46 = \mathbf{3524}$.

9.4 2D Array Addressing — Row-Major & Column-Major

Row-Major: $\text{Address}(a[i][j]) = B + W * [(U2-L2+1)*(i-L1) + (j-L2)]$
 Column-Major: $\text{Address}(a[i][j]) = B + W * [(U1-L1+1)*(j-L2) + (i-L1)]$
 L1, U1 = row lower/upper bound L2, U2 = column lower/upper bound
 (U2-L2+1) = number of columns (U1-L1+1) = number of rows

TRAP: Row-major uses **number of columns** as the multiplier for the row offset; column-major uses **number of rows** as the multiplier for the column offset — easy to swap under pressure.

Worked Example

Array $\text{VAL}[1..15][1..10]$, W=4 bytes, Base=1500. Find address of $\text{VAL}[12][9]$.
 Row-major: numCols=10; Address = $1500 + 4[10(12-1) + (9-1)] = 1500 + 4[110+8] = 1500 + 472 = 1972$.
 Column-major: numRows=15; Address = $1500 + 4[15(9-1) + (12-1)] = 1500 + 4[120+11] = 1500 + 524 = \mathbf{2024}$.

9.5 3D / N-Dimensional Arrays

- Generalizes row-major addressing: each dimension contributes an offset multiplied by the product of sizes of all *later* dimensions.

$\text{Address}(A[i_1, i_2, \dots, i_n]) = B + (i_1-L_1)*(U_2-L_2+1)*(U_3-L_3+1)*\dots*(U_n-L_n+1)$
 $+ (i_2-L_2)*(U_3-L_3+1)*\dots*(U_n-L_n+1)$
 $+ \dots + (i_n-L_n) \quad [\text{all } \times W]$

9.6 Special Matrix Compact Storage (save memory by storing only non-zero region)

Lower Triangular Matrix (elements only where row $\geq$ col; i.e., $i \geq j$), $n \times n$, 0-indexed, row-major

$\text{Address}(A[i][j]) = B + W * [i*(i+1)/2 + j]$ (valid only for $j \leq i$)

Upper Triangular Matrix ($i \leq j$), $n \times n$, 0-indexed, row-major

elements before row $i = i*n - i*(i-1)/2$
 $\text{Address}(A[i][j]) = B + W * [i*n - i*(i-1)/2 + (j - i)]$ (valid only for $j \geq i$)

Worked Examples

Lower triangular, $n=6$, Base=1000, W=4. Find address of $A[4][2]$.
 index = $45/2 + 2 = 10+2 = 12 \rightarrow \text{Address} = 1000 + 412 = \mathbf{1048}$.

Upper triangular, $n=6$, Base=2000, W=2. Find address of $A[2][4]$.
 elements before row 2 = $26 - 21/2 = 12-1 = 11$; offset = $4-2 = 2$; index = 13 $\rightarrow$ Address = $2000 + 213 = 2026^*$.

9.7 Space Needed for Special $n \times n$ Matrices (classic GATE space-complexity questions)

Matrix type	Condition	Non-zero elements to store
Lower/Upper Triangular	$i \geq j$ (or $i \leq j$)	$n(n+1)/2$
Symmetric ($A[i][j] = A[j][i]$)	mirror about diagonal	$n(n+1)/2$ (only need one triangle incl. diagonal)
Tridiagonal ($A[i][j] = 0$ if $ i-j > 1$)	only diagonal + immediate neighbors	$3n - 2$
Toeplitz / diagonal-constant ($A[i][j] = A[i-1][j-1]$)	constant along each diagonal	$2n - 1$ (one value per diagonal)

- **TRAP:** A **symmetric** matrix needs the SAME space as a triangular matrix ($n(n+1)/2$) — since the other half is redundant, not because it's "smaller by nature."

Questions

Q1. [Numerical] Array `a[-6..6]`, base address 3500, W=4 bytes. What is the address of `a[3]` ?

A. 3512 B. 3536 C. 3524 D. 3548

Answer: B. **Why:** Address = $3500 + 4(3 - (-6)) = 3500 + 49 = 3536$.

Q2. [Numerical] `VAL[1..15][1..10]`, Base=1500, W=4, stored row-major. What is the address of `VAL[12][9]` ?

A. 1972 B. 2024 C. 1968 D. 1980

Answer: A. **Why:** Row-major: $1500 + 4(10(12-1) + (9-1)) = 1500 + 4*118 = 1972$.

Q3. [Trap] For the same array as Q2 but stored column-major, the address of `VAL[12][9]` is:

A. 1972 B. 2024 C. 1996 D. 2000

Answer: B. **Why:** Column-major: $1500 + 4(15(9-1) + (12-1)) = 1500 + 4*131 = 2024$ — a different value from row-major for the same element, since the traversal order differs.

Q4. How much storage (in terms of n) does an $n \times n$ tridiagonal matrix need, storing only non-zero elements?

A. n^2 B. $n(n+1)/2$ C. $3n - 2$ D. $2n - 1$

Answer: C. **Why:** Only the main diagonal (n) plus the super- and sub-diagonals ($n-1$ each) are non-zero: $n + (n-1) + (n-1) = 3n - 2$.

Q5. A Toeplitz matrix ($A[i][j] = A[i-1][j-1]$ for valid i, j) of size $n \times n$ requires how much storage?

A. n B. $2n - 1$ C. $n(n+1)/2$ D. n^2

Answer: B. **Why:** Every element on the same diagonal is identical, and an $n \times n$ matrix has exactly $2n - 1$ diagonals, so only $2n - 1$ distinct values need storing.

Q6. [Numerical] Lower triangular matrix, $n=8$, 0-indexed, row-major, Base=1000, W=4. Find the address of `A[5][3]`.

A. 1072 B. 1084 C. 1088 D. 1076

Answer: A. **Why:** index = $i(i+1)/2 + j = 5(5+1)/2 + 3 = 15 + 3 = 18 \rightarrow$ Address = $1000 + 4*18 = 1072$.

Q7. [Conceptual] Why does a symmetric matrix need the same storage as a triangular matrix, not less?

A. Because symmetric matrices are always smaller B. Because only one triangle (including the diagonal) determines the entire matrix — the mirrored half is redundant C. Because symmetric matrices have more non-zero entries D. There is no such relationship

Answer: B. **Why:** $A[j][i] = A[i][j]$, so storing one triangular half ($n(n+1)/2$ elements, including the diagonal) is sufficient to reconstruct the full matrix.

Q8. Which data structure below is classified as non-linear?

A. Stack B. Queue C. Tree D. Linked List

Answer: C. **Why:** Trees (and Graphs) have hierarchical, multi-level relationships between elements — unlike linear structures where each element has a single predecessor/successor.

2.10 Stacks — Expression Evaluation & Recursion Applications

10.1 Stack Basics

- **LIFO** (Last In First Out). Operations: **push**, **pop**, **peek/top**, **isEmpty**, **isFull** — all **O(1)**.
- **Static (array) implementation**: fixed capacity, simple, risk of overflow. **Dynamic (linked-list) implementation**: flexible size, extra pointer memory per node.
- Overflow: push when $TOP == N-1$ (array full). Underflow: pop when $TOP == -1$ (empty).

10.2 Valid Stack Permutations (Pop Sequences)

- Given a fixed **push order**, not every possible ordering of pop outputs is achievable — simulate mentally with a stack to check.

Worked Example

Push order 1,2,3,4,5. Which of these is **NOT** a valid pop sequence?

A. 3,5,4,2,1 B. 2,4,3,5,1 C. 4,3,5,2,1 D. 5,4,3,1,2

Answer: D. Trace: push 1,2,3,4,5 → pop 5,4,3 leaves stack [1,2] (2 on top). To output "1" next, 2 would have to be popped first (it's on top) — but the target wants 1 *before* 2, which is impossible once the stack is [1,2]. Options A, B, C are all achievable by direct simulation.

10.3 Expression Notations

Notation	Form	Example (a+b)
Infix	operator between operands	a + b
Prefix (Polish)	operator before operands	+ a b
Postfix (Reverse Polish)	operator after operands	a b +

- Postfix/Prefix need **no parentheses or precedence rules** — directly stack-evaluable, which is why compilers/ALUs prefer them internally.

Infix → Postfix Conversion Algorithm (operator stack)

```

Scan left to right:
Operand      -> append to output
'('          -> push
')'          -> pop and append until '(' is popped (discard both parens)
Operator op  -> while stack top is an operator with GREATER OR EQUAL precedence
               (respecting left-associativity), pop and append it;
               then push op
End: pop all remaining operators to output

```

Worked Example — Convert $A + B * C - D$ to postfix

Scan: $A \rightarrow \text{out}(A)$; $+$ → push(+); $B \rightarrow \text{out}(A B)$; $\rightarrow \text{push}()$ [since + has lower precedence, don't pop] (stack: +);

$C \rightarrow \text{out}(A B C)$; $\rightarrow \text{pop} *$ (out: A B C), pop + (out: A B C * +) [both $\geq$ precedence of -], push - (stack: -);

End → pop - (out: A B C * + D - ... wait D must be appended first)

Corrected scan: $A(\text{out}:A) + B(\text{out}:AB) (\text{out}:ABC \text{ after } C) - (\text{out}:ABC+) D(\text{out}:ABC+D) \rightarrow \text{pop remaining} - \rightarrow \text{Postfix} = A B C * + D - *$

Postfix Evaluation Algorithm

```

Scan left to right: operand -> push; operator -> pop 2 (top=second operand), apply, push result.

```

Worked Example — Evaluate $6\ 2\ 3\ * + 5\ -$

```

push 6; push 2; push 3 → stack[6,2,3]
* → pop 3,2 → 2*3=6 → push 6 → stack[6,6]
+ → pop 6,6 → 6+6=12 → push 12 → stack[12]
push 5 → stack[12,5]
- → pop 5,12 → 12-5=7 → push 7 → stack[7]
Result = 7*

```

10.4 Recursion & the Call Stack

- Recursion needs a **base case** (stops recursion) + a **recursive case**. Each call gets its own **stack frame** (activation record) with its own copies of parameters/locals — this is *why* recursion has memory overhead and can **stack overflow** with excessive depth.
- **Direct** recursion (A calls A) vs **Indirect** recursion (A calls B calls A).
- **Tail recursion**: the recursive call is the LAST operation (some compilers optimize this into a loop). **Head/non-tail recursion**: work happens AFTER the recursive call returns (e.g., printing after the recursive call) — this is why reversing print-order (print-before-call vs print-after-call) is a classic trace question (see recall pattern below).
- **Recognition pattern**: print-BEFORE-recursive-call → prints in original order; print-AFTER-recursive-call → prints in REVERSED order (since printing happens during the unwind phase).

Worked Trace

```

void fun(int x) { if (x>0) { fun(x-1); printf("%d",x); } }
fun(4) call chain: fun(4)->fun(3)->fun(2)->fun(1)->fun(0)[base, x=0 not>0, returns]
Unwind (prints happen AFTER the recursive call, so in REVERSE call order): 1 2 3 4
Output: 1234

```

10.5 Tower of Hanoi (classic stack/recursion application)

$T(n) = 2 \cdot T(n-1) + 1$, $T(0) = 0 \Rightarrow T(n) = 2^n - 1$ (minimum moves for n disks)
 Total recursive function calls = $2^{n+1} - 1$
 Number of invocations before the FIRST disk moves = n

- Rules: move 1 disk at a time; never place a larger disk on a smaller one.

10.6 Fibonacci & Ackermann (brief)

- Fibonacci recurrence: $f(n) = f(n-1) + f(n-2)$, $f(0)=0$, $f(1)=1$ — naive recursive computation is exponential time (many repeated subcalls) since it doesn't cache results; number of invocations to compute $f(n) = 2 \cdot f(n+1) - 1$.
- Ackermann function $A(m,n)$: a classic example of a **total computable function that is NOT primitive recursive** — grows dramatically faster than any exponential/tower function (e.g., $A(4,2)$ is already astronomically large). [VERIFY exact growth values if needed for a specific question — the key exam fact is just "not primitive recursive, grows faster than any primitive recursive function."]

Questions

Q1. [Trap] Push order is 1,2,3,4,5. Which of the following is NOT achievable as a pop sequence?

A. 3,5,4,2,1 B. 2,4,3,5,1 C. 4,3,5,2,1 D. 5,4,3,1,2

Answer: D. **Why:** After popping 5,4,3, the stack holds [1,2] with 2 on top — 1 cannot be popped before 2, so "...1,2" at the end is impossible.

Q2. [Numerical] Convert infix $A + B * C - D$ to postfix.

A. $A B C * + D -$ B. $A B + C * D -$ C. $A B C + * D -$ D. $A B * C + D -$

Answer: A. **Why:** $*$ binds tighter than $+$ / $-$; BC is grouped first ($\rightarrow BC$), then added to A ($\rightarrow ABC+$), then D is subtracted ($\rightarrow ABC+D-$).

Q3. [Numerical] Evaluate postfix expression $6\ 2\ 3\ * + 5 -$.

A. 5 B. 7 C. 13 D. 19

Answer: B. **Why:** $2*3=6$; $6+5=11$; $11-6=5$.

Q4. [Numerical] Tower of Hanoi with 5 disks — minimum number of moves required?

A. 10 B. 25 C. 31 D. 32

Answer: C. **Why:** $T(n) = 2^n - 1 = 2^5 - 1 = 31$.

Q5. [Trap] `void fun(int x){ if(x>0){ printf("%d",x); fun(x-1); } }` called as `fun(3)`. What is printed?

A. 123 B. 321 C. 3210 D. 0123

Answer: B. **Why:** Print happens BEFORE the recursive call, so output follows call order directly: 3, then 2, then 1 $\rightarrow$ "321".

Q6. [Trap] `void fun(int x){ if(x>0){ fun(x-1); printf("%d",x); } }` called as `fun(3)`. What is printed?

A. 123 B. 321 C. 3210 D. 0123

Answer: A. **Why:** Print happens AFTER the recursive call (during unwind), so output is in reverse call order: 1, 2, 3 $\rightarrow$ "123".

Q7. [Numerical] `int fun(int x,int y){ if(y==0) return 0; return x+fun(x,y-1); }` — what does `fun(x,y)` compute in general, and what is `fun(4,3)`?

A. $x+y = 7$ B. $xy = 12$ C. $x^y = 64$ D. $y-x = -1$

Answer: B. **Why:** The function adds x to itself y times ($x+x+x+\dots+0$), i.e., computes xy ; $\text{fun}(4,3) = 4+4+4+0 = 12$.

Q8. A recursive function's recursive call is the very last statement executed in the function body. This is called:

A. Head recursion B. Tail recursion C. Indirect recursion D. Mutual recursion

Answer: B. **Why:** Tail recursion places the recursive call last, with no pending work after it returns — some compilers can optimize this into an iterative loop.

Q9. Why can excessive (or infinite) recursion cause a program crash?

A. The CPU cache overflows B. Each recursive call consumes stack frame memory for its own parameters/locals, and unbounded calls exhaust the call stack (stack overflow) C. Recursive functions cannot use local variables D. It causes a compile-time error

Answer: B. **Why:** Every call pushes a new activation record onto the call stack; without a reachable base case, this grows until the stack's memory limit is exceeded.

Q10. [Numerical] `int fun(int x,int y){ if(x==0) return y; return fun(x-1, x+y); }` — evaluate `fun(4,3)`.

A. 7 B. 10 C. 13 D. 12

Answer: C. **Why:** $\text{fun}(4,3) \rightarrow \text{fun}(3,7) \rightarrow \text{fun}(2,10) \rightarrow \text{fun}(1,12) \rightarrow \text{fun}(0,13) \rightarrow$ returns 13.

2.11 Queues & Linked Lists

11.1 Queue Basics

- **FIFO** (First In First Out): insert (**enqueue**) at REAR, remove (**dequeue**) at FRONT — both **O(1)**.
- Naive linear-array queue: REAR = REAR+1 on enqueue, FRONT = FRONT+1 on dequeue — but this **wastes space at the front** after repeated dequeues (front slots become unusable), motivating the Circular Queue.
- Applications: shared-resource scheduling, asynchronous data transfer between processes (producer/consumer buffering), load balancing, printer spooling.

11.2 Circular Queue

- Wraps front/rear via modulo: $R = (R+1) \% N$ on enqueue, $F = (F+1) \% N$ on dequeue.
- **Overflow condition**: $(F==0 \ \&\& \ R==N-1) \text{ OR } (F==R+1)$ — the second clause detects the wrapped-around-full case.
- **Underflow condition**: $F == -1$.
- **TRAP**: Distinguishing **full** from **empty** is subtle — a plain $F==R$ could mean either "one element left" or "empty," so implementations use a sentinel ($F=R=-1$ for empty), an explicit counter, or sacrifice one slot to disambiguate.

11.3 Priority Queue

- Each element has a **priority**; dequeue always removes the **highest-priority** element (not strict arrival order — ties broken by arrival order/FIFO among equal priorities).
- Typically implemented via a **Heap** for $O(\log n)$ insert/extract (see Trees chapter) — a plain sorted array/list would need $O(n)$ insertion or $O(n)$ extraction.
- Reversing a queue using only a stack: dequeue everything into a stack (reverses order), then dequeue-from-stack (pop) back into the queue — restores original FIFO order after this double-reversal... **TRAP**: actually dequeuing into a stack and pushing each popped item back into the queue via enqueue **reverses the queue's order once** (net single reversal, not restoration) — verify the exact intended behavior against the specific pseudocode given, since "push into stack then push back into queue" is a classic "reverse a queue using one stack" utility.

11.4 Linked List Basics

- Node = {data, next pointer} . **head** pointer references the first node; last node's next = NULL .
- **vs Array**: dynamic size (no fixed pre-allocation), $O(1)$ insert/delete GIVEN a pointer to the location — but **no random access** ($O(n)$ to reach the i -th node), and extra memory overhead per node for the pointer.
- **TRAP**: Linked lists give up **cache locality/random access** (array's strength) in exchange for dynamic sizing and cheap insert/delete.

11.5 Linked List Variants

Variant	Key property
Singly Linked List (SLL)	one next pointer; forward traversal only
Circular SLL	last node's next points back to the first node (no NULL) — needs an explicit stopping condition (e.g., "stop when back at start") to avoid infinite traversal
Header Linked List	a dummy header node (no real data) always present, even when list is "empty" — simplifies insert/delete at the head (no special-casing an empty list) and can store metadata (e.g., length)
Header Circular LL	header node + circular structure combined
Doubly Linked List (DLL)	each node has both next and prev — enables $O(1)$ backward traversal and easier deletion (no need to track the previous node separately), at the cost of extra memory per node
Header Circular Doubly LL	header + circular + bidirectional — most flexible, most memory overhead

11.6 Core Operations (verified logic)

- **Insert at head**: `new->next = *head; *head = new;` — $O(1)$.
- **Delete at head**: `temp = *head; *head = (*head)->next; free(temp);` — $O(1)$.
- **Iterative reversal**: walk with three pointers (`prev=NULL`, `curr=head`, `next`), at each step save `next=curr->next`, reverse the link `curr->next=prev`, then advance `prev=curr; curr=next`. At the end, ***head_ref = prev** (prev ends up as the new head, i.e. the former last node) — **TRAP**: a common "fill in the blank" question asks for exactly this missing final statement.
- **Recursive reversal**: `reverse(cn, pn) : if cn==NULL return pn ; else save nn=cn->next , set cn->next=pn , recurse reverse(nn, cn) .`

11.7 Comparison Table

Feature	Array	Singly LL	Doubly LL	Circular LL
Random access	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Insert/delete at front	$O(n)$ (shift)	$O(1)$	$O(1)$	$O(1)$
Insert/delete at end (no tail ptr)	$O(1)$ (amortized)	$O(n)$	$O(n)$ ($O(1)$ with tail ptr)	$O(n)$
Backward traversal	$O(1)$ (indexable)	Not possible	$O(1)$	Not possible (SLL variant)
Extra memory/node	None	1 pointer	2 pointers	1 pointer

Questions

Q1. [Trap] Distinguishing a "full" circular queue from an "empty" one using only front/rear pointers is tricky because:

A. Circular queues cannot become full B. `front == rear` can represent BOTH the empty and the "one element left" states, requiring an extra counter or sentinel to disambiguate C. Modulo arithmetic is undefined for queues D. There is no such ambiguity

Answer: B. **Why:** Without an explicit counter or sacrificed slot, `front==rear` is structurally ambiguous between "0 elements" and "N elements" in some implementations — careful sentinel handling (like `F=R=-1` for empty) is required.

Q2. Why is a naive (non-circular) array-based queue considered wasteful over time?

A. It cannot support enqueue B. Dequeued front slots become permanently unusable, wasting array space even though logically free C. It requires $O(n)$ enqueue D. It cannot store more than 2 elements

Answer: B. **Why:** As `FRONT` increments with each dequeue, the slots before it are never reused in a simple linear array queue — motivating the circular (modulo-wraparound) design.

Q3. A Priority Queue is most efficiently implemented using which data structure for $O(\log n)$ insert/extract?

A. Sorted array B. Unsorted linked list C. Heap D. Stack

Answer: C. **Why:** A heap supports $O(\log n)$ insertion and $O(\log n)$ extraction of the max/min element, unlike a sorted array ($O(n)$ insert) or unsorted list ($O(n)$ extraction).

Q4. In a Doubly Linked List, what is the main cost paid for gaining backward traversal capability?

A. $O(n)$ random access (same as SLL, no change) B. An extra pointer (`prev`) per node, increasing memory overhead C. Loss of the ability to insert at the head D. It becomes non-linear

Answer: B. **Why:** Each DLL node stores both `next` and `prev`, doubling the pointer overhead compared to a Singly Linked List.

Q5. [Trap] What is the key advantage of a Header Linked List over a plain Singly Linked List?

A. It uses less memory overall B. It eliminates special-case handling for insertion/deletion at an empty list or at the head, since a dummy header node is always present C. It allows $O(1)$ random access D. It removes the need for a next pointer

Answer: B. **Why:** Because the header node always exists (even for a logically empty list), insert/delete code doesn't need special "is the list empty?" branches.

Q6. [Numerical/Trace] `reverse(struct node** head_ref)` iteratively swaps `next` pointers using `prev`, `current`, `next` as described in the notes. What single statement correctly completes the function after the loop?

A. `*head_ref = current;` B. `*head_ref = next;` C. `*head_ref = prev;` D. `*head_ref = NULL;`

Answer: C. **Why:** After the loop, `current` is `NULL` and `prev` holds the former last node — now the new head — so the head pointer must be updated to `prev`.

Q7. [Trace] For linked list `1->2->3->4->5->6`, what does this print? `void fun(node* s){ if(s==NULL) return; printf("%d ",s->data); if(s->next!=NULL) fun(s->next->next); printf("%d ",s->data); }` called with the head.

A. 1 4 6 6 4 1 B. 1 3 5 1 3 5 C. 1 2 3 5 D. 1 3 5 5 3 1

Answer: D. **Why:** `fun(1)` prints 1, calls `fun(3)`; `fun(3)` prints 3, calls `fun(5)`; `fun(5)` prints 5, calls `fun(NULL)` [node 6's `next->next`, no-op]; unwinding prints 5, 3, 1 again → "1 3 5 5 3 1".

Q8. [Trap] `void traverse(Node *head){ while(head->next!=NULL){ printf("%d",head->data); head=head->next; } }`. Which statement about this function is FALSE?

A. It may crash if the list is empty B. It fails to print the last node's data even when the list is non-empty C. It is "incorrect" specifically because it reassigns the local `head` parameter D. None of the above

Answer: C. **Why:** Reassigning the local copy of `head` (pass-by-value) has no effect on the caller's pointer and causes no bug — so labeling this as the cause of incorrectness is a false claim. (A and B are both true statements about the function's real behavior.)

Q9. Which linked list variant requires an explicit stopping condition (rather than a `NULL` check) to avoid an infinite traversal loop?

A. Singly Linked List B. Doubly Linked List C. Circular Linked List D. None — all use `NULL` termination

Answer: C. **Why:** A circular list's last node points back to the first node instead of `NULL`, so traversal must track the starting node or use a counter to know when to stop.

2.12 Trees — BST, AVL, Heaps & K-ary Trees

12.1 Tree Terminology & Binary Tree Formulas

- Root, leaf (no children), internal node (≥ 1 child), degree (# children), level/depth (root=level 0 or 1 depending on convention — state which convention a question uses), edges = $N-1$ for N nodes.

Max nodes at level L (root=level 0) = 2^L
 Max nodes in a tree of height h = $2^{(h+1)} - 1$
 Min height for n nodes (i.e. most balanced) = $\text{ceil}(\log_2(n+1)) - 1$ [0-indexed height]

12.2 Traversals

Traversal	Order	Note
Preorder	Root, Left, Right	used to serialize/clone a tree
Inorder	Left, Root, Right	produces SORTED order for a BST — key fact
Postorder	Left, Right, Root	used to safely delete a tree (children before parent)
Level-order	top-to-bottom, left-to-right per level	BFS-style, uses a queue

- Common to all three DFS traversals: **left subtree is always visited before right subtree** — only the ROOT's position changes.

Worked Example

```

      4
     / \
    2   6
   /\  /\
  1 3 5 7
Preorder: 4 2 1 3 6 5 7
Inorder:   1 2 3 4 5 6 7 (sorted — this is a BST)
Postorder: 1 3 2 5 7 6 4
Level-order: 4 2 6 1 3 5 7
  
```

12.3 Reconstructing a Tree from Traversals

- Inorder + Preorder** → unique tree (root = first of preorder, split inorder around it).
- Inorder + Postorder** → unique tree (root = last of postorder).
- TRAP: Preorder + Postorder alone is NOT sufficient** in general (only works if the tree is guaranteed full, i.e., every node has 0 or 2 children) — Inorder is essential to locate the split point otherwise.

Worked Example (reconstruct from Preorder=4 2 1 3 6 5 7, Inorder=1 2 3 4 5 6 7)

Root = 4 (first of preorder). In inorder, 4 splits into left={1,2,3}, right={5,6,7}.
 Left preorder(next 3) = 2,1,3 → root=2, splits into left={1}, right={3}.
 Right preorder(next 3) = 6,5,7 → root=6, splits into left={5}, right={7}.
 Reconstructs exactly the tree shown above. ✓

12.4 Binary Search Tree (BST)

- Left subtree < node < right subtree (both subtrees also BSTs).
- Insertion:** traverse like a search, insert at the NULL spot found. **Deletion** 3 cases: leaf (just remove), one child (splice child up to parent), two children (replace with **inorder predecessor** [max of left subtree] or **inorder successor** [min of right subtree], then delete that node from its original position).
- TRAP:** the inorder successor used in two-child deletion is always either a **leaf, or a node with an empty LEFT child** (it's the leftmost node of the right subtree, so it can only have a right child, never a left one).

Operation	Average	Worst case
Search / Insert / Delete	$O(\log n)$	$O(n)$ (degenerates to a linked list if inserted in sorted order)

- Number of distinct BSTs possible with n distinct keys = Catalan number $C_n = \frac{(2n)!}{[(n+1)! n!]}$ → $C_3=5$, $C_4=14$ (both classic GATE numbers, verify by direct formula).**
- Expected/average comparisons for a random search** = (sum over all nodes of [node's level] $\times$ 1) / (number of nodes), where root is level 1.

Worked Example — Expected comparisons

```

Tree:
      10 (L1)
     /  \
    5    20 (L2)
   /  \  /  \
  4   15 30 (L3)
   /
  11 (L4)
Levels: 10=1, 5=2, 20=2, 4=3, 15=3, 30=3, 11=4 (7 nodes)
Expected comparisons = (1*1 + 2*2 + 3*3 + 4*1) / 7 = (1+4+9+4)/7 = 18/7 ≈ 2.57
  
```

12.5 AVL Tree (self-balancing BST)

- **Balance Factor(N) = height(left) – height(right)**, must be in **{–1, 0, +1}** for every node.
- Guarantees **O(log n) worst case** for search/insert/delete (unlike plain BST's O(n) worst case) — this IS the entire point of AVL.

Rotation needed | Fix
 LL (left-left) → single Right rotation
 RR (right-right) → single Left rotation
 LR (left-right) → Left rotation then Right rotation (double)
 RL (right-left) → Right rotation then Left rotation (double)

- After any single insertion, **at most 2 rotations** are ever needed to restore balance.

Structure	Search/Insert/Delete (avg)	Worst case
Plain BST	$O(\log n)$	$O(n)$
AVL Tree	$O(\log n)$	$O(\log n)$

12.6 Complete Binary Tree & Heap

- **Complete Binary Tree:** all levels full except possibly the last, which fills **strictly left to right** — this property is exactly what allows array-based storage without wasted gaps.
- For a node at (0-indexed) array position **i**: **parent(i) = (i-1)/2** (integer division), **left(i) = 2i+1**, **right(i) = 2i+2**.
- **Max-Heap:** parent $\geq$ both children. **Min-Heap:** parent $\leq$ both children. A heap is ALWAYS a complete binary tree, but NOT necessarily a BST (no left<root<right ordering across subtrees, only parent-child ordering).
- **TRAP:** A heap guarantees order only between parent and child, NOT between siblings or across subtrees — don't assume heap order = sorted order.

Worked Example — Valid Max-Heap Check (GATE 2006 style)

Array: 23,17,14,7,13,10,1,5,6,12 (0-indexed). Check every parent(i) $\geq$ child:

idx1=17 $\leq$ 23 $\checkmark$, idx2=14 $\leq$ 23 $\checkmark$, idx3=7 $\leq$ 17 $\checkmark$, idx4=13 $\leq$ 17 $\checkmark$, idx5=10 $\leq$ 14 $\checkmark$, idx6=1 $\leq$ 14 $\checkmark$, idx7=5 $\leq$ 7 $\checkmark$, idx8=6 $\leq$ 7 $\checkmark$, idx9=12 $\leq$ 13 $\checkmark$ → **Valid max-heap.**

Worked Example — Two Delete-Max Operations

Starting heap array: 25,14,16,13,10,8,12.

Delete 1: remove 25; move last (12) to root → [12,14,16,13,10,8]; sift down: 12<16(larger child)→swap→ [16,14,12,13,10,8]; 12 vs child 8 → 12 $\geq$ 8, stop. Result: 16,14,12,13,10,8.

Delete 2: remove 16; move last (8) to root → [8,14,12,13,10]; sift down: 8<14→swap→ [14,8,12,13,10]; 8 vs children 13,10 → 8<13(larger)→swap→ [14,13,12,8,10]; no more children. Result: **14,13,12,8,10**.

12.7 Threaded Binary Tree

- Replaces NULL pointers with **threads** to the inorder predecessor/successor — enables traversal **without recursion or an explicit stack**. Single-threaded (only left OR right NULLs replaced) vs Double-threaded (both).

12.8 K-ary Trees (Full / Strict k-ary tree: every node has 0 or exactly k children)

Leaf nodes $L = I \cdot (k-1) + 1$
 Total nodes $N = I + L = I \cdot k + 1$

(I = internal nodes). **Full** k-ary (node-based: 0 or k children) is distinct from **Complete** k-ary (level-based: all levels full except possibly last, filled left-to-right).

Worked Examples (all independently verified)

Full 3-ary tree with 41 leaves → find total nodes: $41 = I \cdot 2 + 1 \rightarrow I = 20 \rightarrow N = 20 + 41 = 61$.

Full 4-ary tree with 85 total nodes → find internal nodes: $85 = I \cdot 4 + 1 \rightarrow I = 21$.

Full 5-ary tree with 26 internal nodes → find leaves: $L = 26 \cdot 4 + 1 = 105$.

Full k-ary tree, 121 total nodes, 30 internal nodes → find k: $121 = 30k + 1 \rightarrow k = 4$.

Full k-ary tree, 67 leaves, 33 internal nodes → find k: $67 = 33(k-1) + 1 \rightarrow k-1 = 2 \rightarrow k = 3$.

Questions

Q1. Which traversal of a Binary Search Tree always produces output in sorted order?

A. Preorder B. Inorder C. Postorder D. Level-order

Answer: B. Why: Inorder visits Left-Root-Right, which for a BST always yields ascending sorted order.

Q2. [Trap] Which pair of traversals is generally NOT sufficient to uniquely reconstruct a binary tree?

A. Preorder and Inorder B. Inorder and Postorder C. Preorder and Postorder D. Level-order and Inorder

Answer: C. Why: Without Inorder, Preorder+Postorder cannot locate the split between left/right subtrees uniquely (unless the tree is known to be full).

Q3. [Numerical] How many structurally distinct BSTs can be formed using 4 distinct keys?

A. 4 B. 14 C. 24 D. 42

Answer: B. Why: Catalan number $C_4 = (2 \times 4)! / ((5)! \times 4!) = 14$.

Q4. What is the worst-case time complexity of search in a plain (unbalanced) BST?

A. $O(1)$ B. $O(\log n)$ C. $O(n)$ D. $O(n \log n)$

Answer: C. Why: Inserting sorted data degenerates a BST into a linked-list shape, making search $O(n)$ in the worst case — this is exactly what AVL trees fix.

Q5. In an AVL tree, a node's Balance Factor is defined as:

A. height(right) – height(left) B. height(left) – height(right) C. height(left) + height(right) D. depth(node) – height(node)

Answer: B. Why: $BF(N) = \text{height(LeftSubtree)} - \text{height(RightSubtree)}$; a valid AVL node must have $BF \in \{-1, 0, +1\}$.

Q6. An imbalance classified as "RL" (right-left) is fixed by:

A. A single left rotation B. A single right rotation C. Right rotation then left rotation D. Left rotation then right rotation

Answer: C. Why: RL imbalance requires a right rotation on the right child first, then a left rotation on the node itself (double rotation).

Q7. [Numerical] For the 0-indexed array `23, 17, 14, 7, 13, 10, 1, 5, 6, 12`, which of these correctly identifies it as:

A. An invalid max-heap because $\text{idx}3(7) < \text{idx}7(5)$'s parent check fails B. A valid max-heap — every parent is $\geq$ both its children C. A valid min-heap D. Not a complete binary tree

Answer: B. **Why:** Checking every parent-child pair ($\text{idx}0=23$ down to $\text{idx}9=12$) confirms parent $\geq$ child throughout — a valid max-heap.

Q8. [Numerical] Starting from max-heap array `25, 14, 16, 13, 10, 8, 12`, what is the array after 2 delete-max operations?

A. 14, 13, 12, 10, 8 B. 14, 12, 13, 8, 10 C. 14, 13, 8, 12, 10 D. 14, 13, 12, 8, 10

Answer: D. **Why:** First delete-max: 25 removed, 12 moved to root, sifts down to give `16, 14, 12, 13, 10, 8`. Second delete-max: 16 removed, 8 moved to root, sifts down to give `14, 13, 12, 8, 10`.

Q9. [Numerical] A full 3-ary tree has 41 leaf nodes. How many total nodes does it have?

A. 20 B. 41 C. 60 D. 61

Answer: D. **Why:** $L = I(k-1) + 1 \rightarrow 41 = 2I + 1 \rightarrow I = 20$; Total = $I + L = 20 + 41 = 61$.

Q10. [Numerical] A full k-ary tree has 121 total nodes and 30 internal nodes. What is k?

A. 3 B. 4 C. 5 D. 6

Answer: B. **Why:** $N = I(k+1) \rightarrow 121 = 30k + 1 \rightarrow k = 4$.

Q11. [Trap] For a complete binary tree stored in a 0-indexed array, the parent of the node at index i is at index:

A. $i/2$ B. $(i-1)/2$ (integer division) C. $2i$ D. $(i+1)/2$

Answer: B. **Why:** With children at $2i+1$ and $2i+2$, solving for parent gives $(i-1)/2$ — a common off-by-one trap if you assume 1-indexing instead.

Q12. A Threaded Binary Tree replaces NULL pointers with threads mainly to:

A. Save memory by removing all pointers B. Enable traversal without recursion or an explicit stack C. Convert the tree into a BST D. Allow more than 2 children per node

Answer: B. **Why:** Threads point to the inorder predecessor/successor, letting traversal algorithms walk the tree iteratively without needing a call stack.

2.13 Graphs — Representation, DFS, BFS & Topological Sort

13.1 Graph Terminology & Representation

- **Graph** = Vertices (nodes) + Edges (ordered pairs (u,v) for directed, unordered for undirected). Weighted graph: edges carry a cost/value.
- **Degree** (undirected) = number of incident edges. Directed graph: **in-degree** (edges pointing in) and **out-degree** (edges pointing out), tracked separately.
- **Adjacency Matrix vs Adjacency List:**

Feature	Adjacency Matrix	Adjacency List
Space	$O(V^2)$ — same regardless of edge count	$O(V+E)$ — efficient for sparse graphs
Edge lookup (u,v)?	$O(1)$	$O(\text{degree}(u))$
Add vertex	$O(V^2)$ (resize)	$O(1)$
Best for	Dense graphs, frequent edge queries	Sparse graphs (most real-world graphs)

- **TRAP:** Adjacency Matrix wastes $O(V^2)$ space even on a sparse graph (few edges) — Adjacency List is the standard default choice since most graphs encountered in practice ARE sparse.
- Undirected graph's adjacency matrix is always **symmetric**.

13.2 DFS (Depth-First Search)

- Uses a **stack** (explicit, or implicitly via recursion). Goes as deep as possible along one branch before backtracking.
- Needs a **visited[] array** to avoid infinite loops on cycles (unlike tree traversal).

```
DFS(v):
    visited[v] = true
    for each x adjacent to v:
        if not visited[x]: DFS(x)
```

- **Time complexity:** $O(V+E)$ with adjacency list, $O(V^2)$ with adjacency matrix.
- Applications: cycle detection, topological sort, connected components, maze/path-finding.
- **Edge classification in directed-graph DFS** (based on discovery/finish times):

Edge type	Meaning
Tree edge	Leads to a not-yet-visited vertex — forms the DFS tree
Back edge	Points to an ancestor in the DFS tree — indicates a cycle
Forward edge	Points to a descendant (already visited, not a tree edge)
Cross edge	Connects unrelated branches (neither ancestor nor descendant)

- **TRAP:** A directed graph has a cycle **if and only if** DFS finds at least one **back edge**.

13.3 BFS (Breadth-First Search)

- Uses a **queue**. Explores level by level (all neighbors of current vertex before going further).

```
BFS(v):
    visited[v] = true; enqueue(v)
    while queue not empty:
        u = dequeue()
        for each x adjacent to u:
            if not visited[x]: visited[x]=true; enqueue(x)
```

- **Time complexity:** $O(V+E)$ with adjacency list (same as DFS).
- **TRAP:** BFS finds the shortest path in an **UNWEIGHTED graph** (fewest edges) — DFS does **not** guarantee shortest path at all; it just finds *a* path.
- BFS edges (directed context) simplify to just Tree edges and Cross edges — no back/forward-edge distinction since exploration is by distance/level, not depth.

Worked Traversal Example

Graph (undirected): edges 1-2, 1-3, 2-4, 2-5, 3-6, 3-7. Start at vertex 1. Convention: visit neighbors in increasing vertex-number order.

DFS order: 1, 2, 4, 5, 3, 6, 7 (go deep via 2→4, backtrack to 2→5, backtrack to 1→3→6→7).

BFS order: 1, 2, 3, 4, 5, 6, 7 (level 0: {1}; level 1: {2,3}; level 2: {4,5,6,7}).

13.4 Topological Sort

- A linear ordering of vertices such that for every directed edge $u \rightarrow v$, **u comes before v**.
- **TRAP:** Topological sort is defined **only for a DAG (Directed Acyclic Graph)** — a graph with a cycle has **no valid topological order**.
- Two standard methods:
 1. **DFS-based:** run DFS, record each vertex's finish time; topological order = vertices in **decreasing order of finish time** (reverse of finish order).
 2. **Kahn's Algorithm (BFS-based):** repeatedly remove a vertex with **in-degree 0**, decrement its neighbors' in-degrees, repeat — order of removal is a valid topological sort. If vertices remain but none has in-degree 0, the graph has a cycle (no topological order exists).

13.5 Strongly Connected Components (SCC)

- An SCC is a **maximal set of vertices** where every vertex can reach every other vertex — meaningful **only for directed graphs**.

- Found via **Kosaraju's Algorithm** (2 DFS passes: one on the graph, one on its transpose) or **Tarjan's Algorithm** (single DFS pass using discovery times/low-link values).

Questions

Q1. Which data structure does BFS use to explore a graph, and which does DFS use?

A. Both use a stack B. BFS uses a queue, DFS uses a stack (or recursion) C. BFS uses a stack, DFS uses a queue D. Both use a priority queue

Answer: B. **Why:** BFS's queue enforces level-by-level (FIFO) exploration; DFS's stack (or the call stack via recursion) enforces depth-first (LIFO) backtracking.

Q2. [Trap] Which traversal guarantees the shortest path (fewest edges) from the source in an unweighted graph?

A. DFS B. BFS C. Both equally D. Neither

Answer: B. **Why:** BFS explores level-by-level, so the first time it reaches a vertex is guaranteed to be via the minimum number of edges; DFS may reach a vertex via a much longer path first.

Q3. A directed graph's DFS reveals at least one back edge. What can you conclude?

A. The graph is a tree B. The graph is bipartite C. The graph contains a cycle D. The graph is disconnected

Answer: C. **Why:** A back edge (pointing to an ancestor in the DFS tree) is the defining signature of a cycle in directed-graph DFS.

Q4. [Trap] Topological sorting can be performed on:

A. Any directed graph B. Only a Directed Acyclic Graph (DAG) C. Only an undirected graph D. Only a tree

Answer: B. **Why:** A cycle makes it impossible to linearly order vertices consistent with all edge directions — topological sort is undefined for cyclic graphs.

Q5. In Kahn's Algorithm for topological sort, a vertex becomes eligible for removal when:

A. Its out-degree becomes 0 B. Its in-degree becomes 0 C. It has the smallest vertex number D. All its neighbors are visited

Answer: B. **Why:** In-degree 0 means no remaining prerequisite edges point to it, so it can safely be placed next in the order.

Q6. For a SPARSE graph ($E \ll V^2$), which representation is more space-efficient?

A. Adjacency Matrix B. Adjacency List C. Both are equal D. Incidence Matrix is always best

Answer: B. **Why:** Adjacency List uses $O(V+E)$ space, scaling with actual edges; Adjacency Matrix always uses $O(V^2)$ regardless of how sparse the graph is.

Q7. Strongly Connected Components (SCCs) are a meaningful concept for:

A. Only undirected graphs B. Only directed graphs C. Both equally D. Only trees

Answer: B. **Why:** SCCs are defined by mutual reachability between vertices via directed paths — in an undirected connected graph, the whole component is trivially "strongly connected," so the concept is only interesting for directed graphs.

Q8. [Numerical/Trap] Given nodes discovered via DFS with timestamps, which edge type indicates NO cycle risk and simply connects unrelated subtrees?

A. Back edge B. Tree edge C. Cross edge D. Forward edge

Answer: C. **Why:** A cross edge connects vertices in different branches of the DFS forest (neither ancestor nor descendant) — it does not create a cycle, unlike a back edge.

Q9. Time complexity of both BFS and DFS using an adjacency list representation is:

A. $O(V^2)$ B. $O(V+E)$ C. $O(E \log V)$ D. $O(V \log V)$

Answer: B. **Why:** Both algorithms visit every vertex once and examine every edge once via the adjacency list, giving $O(V+E)$; using an adjacency matrix instead would cost $O(V^2)$.

2.14 Hashing

14.1 Why Hashing

- Search time comparison motivates hashing:

Structure	Search time
Unsorted array / Linked list	$O(n)$
Sorted array (binary search)	$O(\log n)$
BST	$O(n)$ worst case
AVL Tree	$O(\log n)$
Hash Table	$O(1)$ average case — independent of n

- Hash function** $H: K \rightarrow L$ maps a large key space K to a small set of table indices L .

14.2 Hash Functions

Division method: $H(K) = K \bmod m$

- Choosing table size **m as a PRIME number** reduces clustering/collisions (a common design guideline).
- Good hash function properties: easy/fast to compute, uniform distribution across slots, low collision rate, minimal clustering.

14.3 Collision Resolution

- Collision:** two different keys map to the same slot — inevitable regardless of hash function quality; must be handled.

(A) Open Addressing / Closed Hashing — all elements stored WITHIN the table itself

Technique	Formula	Key issue
Linear Probing	$h(k,i) = (h'(k) + i) \bmod m$	Primary clustering — collisions form long contiguous runs, growing worse over time
Quadratic Probing	$h(k,i) = (h'(k) + c_1 \cdot i + c_2 \cdot i^2) \bmod m$	Secondary clustering — keys hashing to the same slot follow the same probe sequence; better than linear but not immune
Double Hashing	$h(k,i) = (h_1(k) + i \cdot h_2(k)) \bmod m$	Best distribution among open-addressing schemes — uses a 2nd hash function for the probe step

- Load factor $\alpha = n/m$ (number of keys / table size); for open addressing α must stay < 1 (table can become full).
- Expected search/insert/delete time under open addressing $\approx 1/(1-\alpha)$.
- Deletion in open addressing is tricky: a slot cannot simply be marked empty (would break the probe chain for later searches) — must be marked **"deleted"** (tombstone): insert may reuse it, but search must continue probing past it.

(B) Chaining / Open Hashing — each slot holds a LINKED LIST

- All keys hashing to a slot are appended to that slot's list. Load factor $\alpha = n/m$ can **exceed 1**.
- TRAP on terminology (notoriously counter-intuitive): **"Open Addressing" = Closed Hashing** (no external structures, all in-table); **"Chaining" = Open Hashing** (uses external linked lists). The names are essentially swapped from what you'd guess.

Comparison

Feature	Chaining	Open Addressing
Implementation	Simpler	More complex (probing logic)
Table can fill up?	Never (list grows)	Yes — bounded by table size
Sensitivity to load factor	Lower	Higher — needs care to avoid clustering
Best when	Number of keys unknown/variable	Number/frequency of keys is known in advance
Cache performance	Worse (linked list, scattered memory)	Better (all data in one contiguous table)
Space	Some wastage (unused slots) + extra list-node memory	Slot can be used by any key regardless of its "home" — better bit-for-bit utilization

Questions

Q1. [Numerical] A hash table with 9 slots uses $h(k) = k \bmod 9$, collisions resolved by chaining. Keys inserted: 5, 28, 19, 15, 20, 33, 12, 17, 10. What are the max, min, and average chain lengths?

A. 3, 0, and 1 B. 3, 3, and 3 C. 4, 0, and 1 D. 3, 0, and 2

Answer: A. Why: Slot occupancy by $k \bmod 9$: slot1={28,19,10}{3}, slot2={20}{1}, slot3={12}{1}, slot5={5}{1}, slot6={15,33}{2}, slot8={17}{1}, slots 0,4,7 empty(0). Max=3, Min=0, Average=9 keys/9 slots=1. (This is a real GATE 2014 question.)

Q2. [Trap] "Open Addressing" in hashing terminology refers to:

A. Storing colliding keys in external linked lists B. Storing all elements within the hash table itself via probing C. A hash function that changes dynamically D. A table with no size limit

Answer: B. Why: Despite the name sounding like it might mean "open/external" storage, Open Addressing = Closed Hashing = all keys stored in-table via probing sequences — the reverse of what the name suggests.

Q3. Which open-addressing technique uses a second independent hash function to determine the probe interval?

A. Linear Probing B. Quadratic Probing C. Double Hashing D. Chaining

Answer: C. Why: Double hashing computes $h(k,i) = (h_1(k) + i \cdot h_2(k)) \bmod m$, using h_2 to vary the probe step per key, minimizing clustering.

Q4. Primary clustering (long runs of occupied consecutive slots) is a defining weakness of:

A. Chaining B. Linear Probing C. Double Hashing D. AVL Trees

Answer: B. Why: Linear probing's fixed +1 step causes collisions to merge into ever-growing contiguous blocks, degrading performance as the table fills.

Q5. Why is it recommended to choose the hash table size m as a prime number in the division method?

A. Prime numbers use less memory B. It reduces systematic clustering/collisions caused by common factors between keys and m C. It is required by the C language D. It makes chaining unnecessary

Answer: B. Why: If m shares factors with many keys' patterns (e.g., m is a power of 2 and keys are often even), collisions cluster into a subset of slots; a prime m avoids this systematic bias.

Q6. [Trap] Under open addressing, why can't a deleted key's slot simply be marked "empty"?

A. It would violate the hash function B. It could break the probe sequence for other keys, causing later searches to incorrectly stop early / report "not found" C. It saves memory D. Deletion is not allowed in open addressing

Answer: B. Why: Search relies on continuing to probe until an empty slot is hit; if a mid-sequence slot were marked truly empty after deletion, searches for keys further along that chain would wrongly terminate early. A "deleted" tombstone marker is used instead.

Q7. In chaining, unlike open addressing, the load factor $\alpha = n/m$ can be:

A. At most 0.5 B. Greater than 1 C. Never more than 1 D. Always exactly 1

Answer: B. Why: Since each slot holds a linked list of unbounded length, chaining never "fills up" — the number of keys n can exceed the number of slots m .

Q8. Which collision resolution technique gives the BEST cache/locality-of-reference performance, at the cost of being most prone to clustering?

A. Chaining B. Double Hashing C. Linear Probing D. Quadratic Probing

Answer: C. Why: Linear probing accesses sequential memory slots, maximizing cache locality, but this same sequential structure is exactly what causes primary clustering.

2.15 Algorithm Analysis — Asymptotic Notation & Recurrences

(High-yield in GATE/ISRO: 5-6 questions typically, mostly numerical.)

15.1 Types of Analysis

Aspect	Experimental (A Posteriori)	Asymptotic (Apriori)
When	After implementation/execution	Before implementation, purely theoretical
Measures	Actual time/space used	Estimated complexity
Depends on hardware?	Yes	No — machine-independent
Use case	Real-world comparison	Efficiency for large inputs

- **Worst-case analysis** is the most commonly used (guarantees reliability); average-case requires knowing the input distribution and is rarely tractable in practice.
- **Deterministic algorithm**: always same output/steps for a given input. **Non-deterministic**: may vary (e.g., involves randomness/choice).

15.2 Asymptotic Notations (precise definitions)

Big-O (upper bound): $O(g(n)) = \{ f(n) : 0 \leq f(n) \leq C \cdot g(n), \text{ for all } n \geq n_0 \}$
 Big-Omega (lower bound): $\Omega(g(n)) = \{ f(n) : 0 \leq C \cdot g(n) \leq f(n), \text{ for all } n \geq n_0 \}$
 Big-Theta (tight bound): $\Theta(g(n)) = \{ f(n) : C_1 \cdot g(n) \leq f(n) \leq C_2 \cdot g(n), \text{ for all } n \geq n_0 \}$
 little-o / little-omega: strict versions ($< / >$, not $\leq / \geq$) of O / Ω

- Analogy to real numbers: $O \sim "<="$, $\Omega \sim ">="$, $\Theta \sim "="$, little-o $\sim "<"$, little-omega $\sim ">"$.
- **Properties**: Reflexive ($f=O(f)$), Symmetric only for Theta ($f=\Theta(g)$ iff $g=\Theta(f)$), Transitive (all 5 notations), **Transpose symmetry**: $f=O(g)$ iff $g=\Omega(f)$.
- Sum rule: if $f=O(g)$ and $p=O(q)$, then $f+p = O(\max(g,q))$. Product rule: $f \cdot p = O(g \cdot q)$.

15.3 Complexity Class Ordering (increasing)

$O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$

15.4 Analyzing Iterative Loops — Recognition Patterns

Loop pattern	Complexity	Why
$i=1; i \leq n; i++$ (step $\pm$ constant)	$O(n)$	Linear scan
$i=1; i \leq n; i=i \cdot c$ (multiply by constant $c > 1$)	$O(\log n)$	Halves/divides the remaining range each step
$i=n; i > 1; i=i/c$ (divide by constant)	$O(\log n)$	Same — logarithmic shrink
$i=2; i \leq n; i=i^2$ (i squared each time)	$O(\log \log n)$	Exponent itself doubles each step
$i=n; i > 2; i=\sqrt{i}$	$O(\log \log n)$	Reverse of squaring — exponent halves each step
$i=1; i \leq n; i++$ (condition on i^2)	$O(\sqrt{n})$	Loop runs while $i \leq \sqrt{n}$
$i=1; i \leq n; i=i+(n/2)$ (step proportional to n)	$O(1)$	Step size scales with n itself — only ~ 2 -3 iterations regardless of n

- **TRAP**: A loop with step size $i = i + n/2$ looks like it depends on n but actually runs a CONSTANT number of times (~ 2 -3) since the step itself grows with n — don't assume "involves n " automatically means "not $O(1)$."
- **Nested independent loops**: total iterations = product of each loop's individual iteration count (e.g., outer $O(n) \times$ inner $O(\log n) = O(n \log n)$).

15.5 Solving Recurrences — 4 Methods

1. **Substitution**: guess the closed form, verify by mathematical induction.
2. **Recursion Tree**: draw out the recursion levels, sum work per level.
3. **Master Method**: for $T(n) = a \cdot T(n/b) + f(n)$ (see below).
4. **Change of Variable**: substitute $n = b^m$ to convert a non-standard recurrence (e.g., involving $\sqrt{n}$) into a standard one solvable by the Master Method.

Worked Recursion Examples (verified by hand)

```

rec(n): if n==1 return 1; else return rec(n-1) + n;
-> ONE recursive call -> T(n) = T(n-1) + O(1) => T(n) = O(n), Space = O(n) [recursion depth]

rec(n): if n==1 return 1; else return 2*rec(n-1) + n;
-> Still only ONE recursive call (the "2*" just scales the return value)
-> T(n) = T(n-1) + O(1) = O(n), Space = O(n)
TRAP: "2*rec(n-1)" looks like 2 calls but is actually 1 call with its result doubled – don't confuse
a multiplied return value with a second recursive invocation.

rec(n): if n==1 return 1; else return rec(n-1) + rec(n-1) + n;
-> TWO separate recursive calls, each to rec(n-1)
-> T(n) = 2*T(n-1) + O(1) => T(n) = O(2^n), Space = O(n) [max stack depth along one branch]

rec(n): if n==1 return 1; else { for(i=1;i<=n;i++) print(...); rec(n-1); }
-> T(n) = T(n-1) + O(n) => T(n) = O(n^2), Space = O(n)

rec(n): if n==1 return 1; else return 2*rec(n/2) + n;
-> ONE call (again, "2*" scales the result) -> T(n) = T(n/2) + O(n) = O(n) [Master Case 3], Space=O(log n)

rec(n): if n==1 return 1; else return rec(n/3)+rec(n/3)+rec(n/3)+n;
-> THREE calls to rec(n/3) -> T(n) = 3T(n/3) + O(n) = O(n log n) [Master Case 2], Space=O(log n)

rec(n): if n==1 return 1; else { rec(n/2); rec(n/2); for(i=1;i<=n;i++) print(...); }
-> TWO calls to rec(n/2) -> T(n) = 2T(n/2) + O(n) = O(n log n) [Master Case 2], Space=O(log n)

```

- **TRAP: Space complexity of recursion = maximum stack depth at any single point in time**, NOT the total number of calls made. Even $T(n)=2T(n-1)+O(1)$ ($O(2^n)$ time) only needs $O(n)$ space, since calls execute sequentially — only one root-to-leaf path exists on the stack at once.

15.6 Master Theorem — for $T(n) = a \cdot T(n/b) + f(n)$

Let $p = \log_b(a)$ [the "watershed" exponent]

Case 1: $f(n) = O(n^{p-\epsilon})$ for some $\epsilon > 0$ $\Rightarrow T(n) = \Theta(n^p)$ [recursive work dominates]
Case 2: $f(n) = \Theta(n^p)$ $\Rightarrow T(n) = \Theta(n^p \cdot \log n)$ [balanced – extra log factor]
Case 3: $f(n) = \Omega(n^{p+\epsilon})$, regularity holds $\Rightarrow T(n) = \Theta(f(n))$ [combine step dominates]

Worked Examples (all verified)

Recurrence	a, b	$p = \log_b(a)$	$f(n)$ vs n^p	Case	Result
$T(n) = 4T(n/2) + n$	4, 2	2	$n = O(n^{(2-1)})$	1	$\Theta(n^2)$
$T(n) = 9T(n/3) + n$	9, 3	2	$n = O(n^{(2-1)})$	1	$\Theta(n^2)$
$T(n) = 7T(n/2) + n^2$	7, 2	$\log_2 7 \approx 2.807$	$n^2 = O(n^{(2.807-\epsilon)})$	1	$\Theta(n^{2.807})$
$T(n) = 8T(n/2) + n^2$	8, 2	3	$n^2 = O(n^{(3-1)})$	1	$\Theta(n^3)$
$T(n) = 2T(n/2) + n$	2, 2	1	$n = \Theta(n^{(1)})$	2	$\Theta(n \log n)$
$T(n) = T(2n/3) + 1$	1, 1.5	0	$1 = \Theta(n^{(0)})$	2	$\Theta(\log n)$
$T(n) = T(n/3) + n$	1, 3	0	$n = \Omega(n^{(0+1)})$	3	$\Theta(n)$

15.7 Change of Variable Method

- Used when the recurrence shrinks by $\sqrt{n}$ rather than n/b (Master Theorem doesn't directly apply).
- Substitute $n = 2^m$ (so $m = \log_2 n$), define $S(m) = T(2^m)$, transforming $T(\sqrt{n})$ into $S(m/2)$ — now solvable by Master Theorem, then substitute back $m = \log n$.

Example: $T(n) = T(\sqrt{n}) + 1$
Let $n = 2^m \Rightarrow S(m) = S(m/2) + 1$ [Master: $a=1, b=2, p=0$, $f(m)=1=\Theta(m^0) \Rightarrow$ Case 2 $\Rightarrow S(m)=\Theta(\log m)$]
Since $m = \log n$: $T(n) = \Theta(\log \log n)$

Questions

Q1. [Numerical] What is the complexity of: `for(i=1; i<=n; i=i*3) print(...);`

A. $O(n)$ B. $O(\log n)$ C. $O(\sqrt{n})$ D. $O(n \log n)$

Answer: B. **Why:** i multiplies by a constant (3) each iteration, so the loop runs $O(\log_3 n) = O(\log n)$ times.

Q2. [Numerical] What is the complexity of: `for(i=2; i<=n; i=i*i) print(...);`

A. $O(\log n)$ B. $O(\sqrt{n})$ C. $O(\log \log n)$ D. $O(n)$

Answer: C. **Why:** Squaring i each time means the EXPONENT of i doubles every iteration — this reaches n in $O(\log \log n)$ steps, a classic doubling-exponent pattern.

Q3. [Trap] What is the complexity of: `for(i=1; i<n; i=i+(n/2)) print(...);`

A. $O(n)$ B. $O(\log n)$ C. $O(1)$ D. $O(\sqrt{n})$

Answer: C. **Why:** The step size $(n/2)$ itself scales with n , so the loop always runs about 2-3 times regardless of n 's value — a constant number of iterations, $O(1)$.

Q4. [Numerical] `rec(n): if(n==1) return 1; else return rec(n-1)+rec(n-1)+n;` — what are the time and space complexity?

A. $O(n)$, $O(n)$ B. $O(2^n)$, $O(n)$ C. $O(2^n)$, $O(2^n)$ D. $O(n^2)$, $O(n)$

Answer: B. **Why:** $T(n)=2T(n-1)+O(1)$ gives $O(2^n)$ time, but since calls execute sequentially (not all simultaneously live), max stack depth is only $O(n)$.

Q5. [Trap] `rec(n): if(n==1) return 1; else return 2*rec(n-1)+n;` — how many recursive calls does this make per invocation, and what is $T(n)$?

A. Two calls; $T(n)=O(2^n)$ B. One call; $T(n)=O(n)$ C. One call; $T(n)=O(n^2)$ D. Two calls; $T(n)=O(n^2)$

Answer: B. **Why:** "2*rec(n-1)" is a single call whose returned value is scaled by 2 — not two separate calls. $T(n)=T(n-1)+O(1)=O(n)$.

Q6. [Numerical] Using the Master Theorem, solve $T(n) = 8T(n/2) + n^2$.

A. $\Theta(n^2)$ B. $\Theta(n^2 \log n)$ C. $\Theta(n^3)$ D. $\Theta(n \log n)$

Answer: C. **Why:** $p = \log_2 8 = 3$; $f(n) = n^2 = O(n^{(3-1)}) \rightarrow$ Case 1 $\rightarrow T(n) = \Theta(n^p) = \Theta(n^3)$.

Q7. [Numerical] Solve $T(n) = T(n/3) + n$ using the Master Theorem.

A. $\Theta(n)$ B. $\Theta(n \log n)$ C. $\Theta(\log n)$ D. $\Theta(n^{\log_3 1})$

Answer: A. **Why:** $p = \log_3 1 = 0$; $f(n) = n = \Omega(n^{(0+1)})$, and the regularity condition holds $\rightarrow$ Case 3 $\rightarrow T(n) = \Theta(f(n)) = \Theta(n)$.

Q8. [Numerical] Solve $T(n) = T(\sqrt{n}) + 1$ using change of variable.

A. $\Theta(\log n)$ B. $\Theta(\log \log n)$ C. $\Theta(\sqrt{n})$ D. $\Theta(1)$

Answer: B. **Why:** Substituting $n = 2^m$ gives $S(m) = S(m/2) + 1 = \Theta(\log m)$ by Master Case 2, and $m = \log n$, so $T(n) = \Theta(\log \log n)$.

Q9. Which asymptotic notation gives a TIGHT bound (both upper and lower)?

A. Big-O B. Big-Omega C. Big-Theta D. little-o

Answer: C. **Why:** Theta sandwiches $f(n)$ between $C_1 \cdot g(n)$ and $C_2 \cdot g(n)$ — both bounds hold, unlike O (upper only) or Omega (lower only).

Q10. [Conceptual] Why is worst-case analysis preferred over average-case analysis in most algorithm design?

A. Worst-case is always faster to compute B. Average-case requires knowing the input probability distribution, which is often unknown/intractable, while worst-case gives a reliable guarantee C. Worst-case and average-case are always identical D. Average-case analysis is not mathematically valid

Answer: B. **Why:** Average-case analysis needs a model of "typical" input distribution, which is rarely known precisely; worst-case guarantees reliability regardless of input, making it the most practically useful and commonly reported metric.

2.16 Sorting Algorithms

16.1 Key Vocabulary

- **In-place/Internal**: needs no extra memory beyond the input (e.g., Heap Sort). **External**: needs additional memory (e.g., Merge Sort's $O(n)$ buffer).
- **Stable**: preserves the relative order of EQUAL elements. **Unstable**: does not guarantee this.

16.2 Algorithm-by-Algorithm Reference

- **Selection Sort**: repeatedly find the min of the unsorted part, swap it to the front. $O(n^2)$ in **every** case (best=worst=average) — it always scans the full remaining unsorted part regardless of input order. **TRAP**: unlike Bubble/Insertion, Selection Sort has **no best-case speedup** even on already-sorted input. Unstable (naive swap can reorder equal elements). Key advantage: makes at most **$n-1$ swaps total** — the fewest of any comparison sort — useful when swap cost is high.
- **Bubble Sort**: repeatedly swap adjacent out-of-order pairs, "bubbling" larger elements to the end. $O(n^2)$ average/worst. **TRAP**: $O(n)$ best case **only if an early-exit flag** detects a pass with zero swaps (already sorted) — without the flag, it's $O(n^2)$ even on sorted input. Stable.
- **Insertion Sort**: builds the sorted portion by inserting each new element into its correct position via adjacent comparisons+shifts. $O(n)$ best case (already sorted — comparison fails immediately, no shifting), $O(n^2)$ average/worst. Stable. **Online** (can sort a stream as elements arrive) and best for small/nearly-sorted datasets.
- **Merge Sort**: divide-and-conquer — split in half, recursively sort, then **merge**. $O(n \log n)$ in **every** case (best=average=worst — its key selling point over Quick Sort). $O(n)$ **extra** space for the merge buffer (External). Stable. Recurrence: $T(n) = 2T(n/2) + n$.
- **Quick Sort**: divide-and-conquer via pivot + partition (elements < pivot go left, > pivot go right). $O(n \log n)$ average/best, but **$O(n^2)$ worst case** — occurs when the pivot is always the smallest/largest element (e.g., already-sorted input with naive first/last-element pivot choice). **TRAP**: **randomized pivot selection** mitigates this worst case in practice, which is why production Quick Sort implementations randomize. $O(\log n)$ space (recursion stack, average case). **Unstable** (in-place partition swaps break relative order).
- **Heap Sort**: build a Max-Heap, then repeatedly extract the max to the end. $O(n \log n)$ in **every** case. $O(1)$ **extra** space (fully in-place) — **TRAP**: this $O(1)$ -space + guaranteed- $O(n \log n)$ -worst-case combination is exactly why Heap Sort is preferred over Quick Sort when worst-case guarantees matter, and over Merge Sort when memory is constrained. Unstable.

Master Comparison Table

Algorithm	Best	Average	Worst	Extra Space	Stable?
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Bubble	$O(n)^*$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No

(Bubble's $O(n)$ best case requires the early-termination flag.)

16.3 The Comparison-Sort Lower Bound

Any comparison-based sorting algorithm needs at least $\Omega(n \log n)$ comparisons in the worst case.

- **Why**: there are $n!$ possible orderings of n elements; each comparison yields at most 1 bit of information (branches a decision tree in 2); so at least $\log_2(n!) = \Omega(n \log n)$ comparisons are needed to distinguish all $n!$ outcomes (decision-tree argument).
- **TRAP**: This bound applies **only to comparison-based sorts**. Non-comparison sorts can beat it under special assumptions: **Counting Sort** — $O(n+k)$ where k = range of input values, stable, needs $O(k)$ extra space; works only for a bounded/small range of integer keys, not general comparable data.

Questions

Q1. [Trap] What is the best-case time complexity of Bubble Sort?

A. $O(n \log n)$ B. $O(n)$ C. $O(n^2)$ D. $O(\log n)$

Answer: B. Why: With the early-exit flag optimization, an already-sorted array triggers zero swaps in the first pass, so the algorithm terminates after one $O(n)$ pass.

Q2. [Trap] What is the worst-case time complexity of Insertion Sort when the insertion POSITION is found via binary search instead of linear scan?

A. $O(n)$ B. $O(n \log n)$ C. $O(n^2)$ D. $O(n(\log n)^2)$

Answer: C. Why: Binary search reduces the number of COMPARISONS to $O(\log n)$ per insertion, but the actual SHIFTING of elements to make room is still $O(n)$ per insertion in an array — so the overall worst case remains $O(n^2)$; this is a classic trap (comparisons $\neq$ total work).

Q3. Which of these sorting algorithms has the lowest WORST-CASE time complexity?

A. Quick Sort B. Bubble Sort C. Merge Sort D. Selection Sort

Answer: C. Why: Merge Sort guarantees $O(n \log n)$ even in the worst case; Quick Sort degrades to $O(n^2)$, and Bubble/Selection are always $O(n^2)$.

Q4. [Numerical/GATE] After the FIRST partitioning step of Quicksort on an 8-element array, it looks like: 2 5 1 7 9 12 11 10. Which statement is correct about the pivot?

A. The pivot could be either the 7 or the 9 B. The pivot could be the 7, but not the 9 C. The pivot is not the 7, but could be the 9 D. Neither 7 nor 9 is the pivot

Answer: A. Why: Checking the partition property (all elements left $\leq$ value $\leq$ all elements right): for 7 — left {2,5,1} all ≤ 7 , right {9,12,11,10} all > 7 ✓. For 9 — left {2,5,1,7} all ≤ 9 , right {12,11,10} all > 9 ✓. Both independently satisfy the partitioned property, so either could have been the pivot (classic GATE question).

Q5. What is the worst-case recurrence for Quicksort, and its resulting time complexity?

A. $T(n)=T(n-2)+O(n)$, $O(n^2)$ B. $T(n)=T(n-1)+O(n)$, $O(n^2)$ C. $T(n)=2T(n/2)+O(n)$, $O(n \log n)$ D. $T(n)=T(n/10)+T(9n/10)+O(n)$, $O(n \log n)$

Answer: B. Why: Worst case occurs when the pivot always splits the array into sizes 0 and $n-1$ (e.g., already-sorted input with last-element pivot); $T(n)=T(n-1)+O(n)$ solves to $O(n^2)$.

Q6. [Trap] Which sorting algorithm is generally NOT stable in its typical implementation?

A. Insertion Sort B. Merge Sort C. Quick Sort D. Bubble Sort

Answer: C. Why: Standard in-place partitioning in Quick Sort can swap equal elements past each other, breaking their original relative order.

Q7. Which algorithm gives the best performance on an array that is already sorted or has just 1-2 misplaced elements?

A. Quick Sort B. Heap Sort C. Merge Sort D. Insertion Sort

Answer: D. Why: Insertion Sort's comparisons fail almost immediately when data is already in order, giving near- $O(n)$ performance — the best adaptation to nearly-sorted input among these options.

Q8. [Trap] If swap operations are very costly, which sorting algorithm should be preferred to minimize the total number of swaps?

A. Heap Sort B. Selection Sort C. Insertion Sort D. Merge Sort

Answer: B. Why: Selection Sort performs at most $n-1$ swaps total (one swap per outer iteration), the fewest of any comparison sort, even though its comparison count is still $O(n^2)$.

Q9. [Numerical] When all elements of the input array are IDENTICAL, which of these (Insertion/Heap/Merge/Selection Sort) takes the LEAST time in a typical implementation?

A. Insertion Sort B. Heap Sort C. Merge Sort D. Selection Sort

Answer: A. Why: With all-equal elements, Insertion Sort's inner-loop condition ($A[i] > \text{key}$) is false immediately for every element — no shifting ever happens, giving $O(n)$ total; Heap/Merge Sort still do $O(n \log n)$ regardless of content, and Selection Sort always does $O(n^2)$ comparisons.

Q10. Why can Counting Sort achieve $O(n+k)$ time, beating the $\Omega(n \log n)$ comparison-sort lower bound?

A. It secretly still compares elements pairwise B. It is not a comparison-based algorithm — it uses key values directly as array indices, so the decision-tree lower bound doesn't apply C. The lower bound is only theoretical and never actually holds D. It only works on already-sorted arrays

Answer: B. Why: The $\Omega(n \log n)$ bound applies specifically to algorithms that sort purely via pairwise comparisons; Counting Sort instead uses the key values directly to place elements, sidestepping the comparison-based decision-tree argument entirely (at the cost of needing a bounded key range k).

Q11. Merge Sort's space complexity is $O(n)$ extra, while Heap Sort's is $O(1)$ extra. Both achieve $O(n \log n)$ worst case. What is the main practical tradeoff?

A. There is no tradeoff — they are identical in every way B. Heap Sort is preferred under memory constraints; Merge Sort is preferred for stability and works better on sequential/external storage C. Merge Sort is always slower D. Heap Sort is always unstable and therefore never used

Answer: B. Why: Merge Sort's stability and sequential-access pattern suit external/linked data; Heap Sort's in-place $O(1)$ space suits memory-constrained internal sorting, at the cost of being unstable.

2.17 Searching Algorithms & Min-Max

17.1 Linear vs Binary Search

Feature	Linear Search	Binary Search
Precondition	None — works on unsorted data	Requires sorted array
Best case	$O(1)$	$O(1)$
Average/Worst	$O(n)$	$O(\log n)$
Space	$O(1)$	Iterative $O(1)$, Recursive $O(\log n)$ (recursion stack)

TRAP: Binary search's $O(\log n)$ speed is worthless if you forget it **requires sorted input** — sorting an unsorted array just to binary-search it once costs $O(n \log n)$, worse than a single $O(n)$ linear scan. Binary search pays off only when you search the SAME sorted data many times.

Binary Search — mechanism & recurrence

Each comparison eliminates half the remaining search space:

$$T(n) = T(n/2) + O(1) \Rightarrow T(n) = \Theta(\log n)$$

Worked trace — search for 47 in [3,9,15,22,31,40,47,58,66,72] (indices 0-9)

```
low=0 high=9 mid=4 -> A[4]=31 < 47 -> low=5
low=5 high=9 mid=7 -> A[7]=58 > 47 -> high=6
low=5 high=6 mid=5 -> A[5]=40 < 47 -> low=6
low=6 high=6 mid=6 -> A[6]=47 == 47 -> FOUND at index 6 (3 comparisons)
```

- Binary search may be **iterative** ($O(1)$ space) or **recursive** ($O(\log n)$ stack space, since each call needs its own low/high/mid frame).
- Not efficient on a **linked list** (no $O(1)$ random access to the middle element — must walk the list, defeating the purpose).

Recognition Pattern

If a query is issued **once**, plain linear scan (or a single sort+scan) may be cheaper overall. If the SAME dataset is searched **repeatedly**, sort once (amortize the $O(n \log n)$ cost) then binary-search each query in $O(\log n)$ — this trade-off is a common conceptual MCQ.

17.2 Min-Max Algorithm (find both min AND max together)

Straight (linear) method — compare each element to running min AND max

```
Best case (array increasing): n-1 comparisons (each element only beats "max")
Worst case (array decreasing): 2(n-1) comparisons (each element tested against BOTH max and min)
Average case: (3n/2) - 1.5 comparisons
```

Divide & Conquer method — split into halves, recurse, merge (compare max-vs-max, min-vs-min)

```
Recurrence: T(n) = 2T(n/2) + 2
Comparisons (n a power of 2): ceil(3n/2) - 2
```

TRAP: D&C Min-Max needs only **~1.5n** comparisons vs. the straight method's **up to 2n** — a rare case where restructuring comparisons (not the asymptotic complexity, both are $O(n)$) still meaningfully reduces the constant factor, a classic "efficient algorithm" numerical question.

Worked Example

Q. Comparisons needed by an efficient (D&C) Min-Max algorithm for $n = 250$ elements?
 $\text{ceil}(3 \times 250 / 2) - 2 = \text{ceil}(375) - 2 = \mathbf{373}$.

Questions

Q1. Binary search on an array of 1024 sorted elements takes at most how many comparisons in the worst case?

A. 1024 B. 512 C. 10 D. 100

Answer: C. Why: $\log_2(1024) = 10$ — each comparison halves the search space.

Q2. [Trap] Which statement about binary search is FALSE?

A. It requires sorted data B. Its worst-case complexity is $O(\log n)$ C. It is efficient for singly linked lists D. Recursive implementation uses $O(\log n)$ stack space

Answer: C. Why: Binary search needs $O(1)$ random access to the middle element; a linked list only allows $O(n)$ sequential access, defeating binary search's advantage.

Q3. [Numerical] Using the straight Min-Max method on a strictly decreasing array of 20 elements, how many comparisons are performed?

A. 19 B. 20 C. 38 D. 40

Answer: C. Why: Worst case = $2(n-1) = 2(19) = 38$ (every new element beats the running min, forcing both comparisons).

Q4. [Numerical] How many comparisons does the Divide-and-Conquer Min-Max algorithm need for $n = 64$ elements?

A. 63 B. 94 C. 126 D. 96

Answer: B. Why: $\text{ceil}(3 \times 64 / 2) - 2 = 96 - 2 = 94$.

Q5. Why is it inefficient to binary-search an array that will only ever be queried once?

A. Binary search never works on unsorted data and there's no way around it B. Sorting first costs $O(n \log n)$, which dominates the $O(\log n)$ search benefit for a single query — a linear scan ($O(n)$) would be cheaper overall C. Binary search always requires more memory than linear search D. There is no inefficiency; binary search is always preferred

Answer: B. Why: The one-time cost of sorting ($O(n \log n)$) exceeds a single linear scan's $O(n)$, so binary search only pays off when the sort cost is amortized across many repeated queries.

Q6. In the recursive binary search implementation, what causes the recursion to terminate with "not found"?

A. mid equals key B. low exceeds high (search space exhausted) C. The array is empty from the start D. A stack overflow occurs

Answer: B. Why: The search space shrinks by half each call; once $\text{low} > \text{high}$, no elements remain to check, so the key is confirmed absent.

2.18 Greedy Algorithms

18.1 Greedy Paradigm

- Makes the **locally optimal** choice at each step, hoping it leads to a global optimum; decisions are **never revisited** (contrast with DP, which may reconsider via subproblem tables — see Ch.19).
- Works only when the problem has: **Greedy-choice property** (a locally optimal choice leads to a globally optimal solution) + **Optimal substructure** (an optimal solution contains optimal solutions to subproblems).
- **TRAP:** Greedy does **NOT** always yield the optimal answer — classic counterexample: **0/1 Knapsack** (greedy by any single criterion can fail; needs DP — Ch.19), while **Fractional Knapsack** IS correctly solved by greedy (see §18.3).

18.2 Huffman Coding

- Builds an **optimal prefix-free** (no code is a prefix of another) variable-length binary code by repeatedly merging the **two lowest-frequency** nodes into a new node (weight = sum) until one tree remains.
- More frequent symbols get **shorter** codes. Time complexity: **$O(n \log n)$** (using a min-heap: n extract-min + insert operations, each $O(\log n)$).
- Built **bottom-up** (contrast: Shannon-Fano builds top-down and is generally sub-optimal vs Huffman).

Worked Example — 5 symbols, frequencies $M_1=12, M_2=4, M_3=45, M_4=17, M_5=23$ (total=101)

```
Merge steps (always combine two smallest):
M2(4)+M1(12) = 16 [N1]
N1(16)+M4(17) = 33 [N2]
M5(23)+N2(33) = 56 [N3]
M3(45)+N3(56) = 101 [root]

Resulting codes (one valid left/right assignment):
M3 = 0      (1 bit)
M5 = 10     (2 bits)
M4 = 111    (3 bits)
M1 = 1101   (4 bits)
M2 = 1100   (4 bits)

Total bits = 12(4) + 4(4) + 45(1) + 17(3) + 23(2)
           = 48 + 16 + 45 + 51 + 46 = 206 bits
Fixed-length alternative:  $\lceil \log_2 5 \rceil = 3$  bits/symbol  $\times 101 = 303$  bits
Bits saved = 303 - 206 = 97 bits
```

TRAP: Huffman codes are **not unique** — swapping the 0/1 assignment at any node (or breaking merge ties differently) gives an equally-valid, equally-optimal code with the SAME bit lengths. A question may list two different-looking bit strings for the same word and both can be correct (see Q3 below).

Worked Example 2 — frequencies $a=5, b=9, c=12, d=13, e=16, f=45$ (total=100)

```
Merges: a5+b9=14[N1]; c12+d13=25[N2]; N1(14)+e16=30[N3]; N2(25)+N3(30)=55[N4]; f45+N4(55)=100[root]
Codes:  f=0, c=100, d=101, a=1100, b=1101, e=111
Total Huffman bits = 45(1)+12(3)+13(3)+16(3)+5(4)+9(4) = 45+36+39+48+20+36 = 224
Fixed-length:  $\lceil \log_2 6 \rceil = 3$  bits  $\times 100 = 300$  bits
Bits saved = 300 - 224 = 76 bits
Code for "dead" = d+e+a+d = 101+111+1100+101 = 1011111100101 (13 bits)
```

18.3 Optimal Merge Pattern

- Problem: merge multiple sorted files into one; merging files of size m and n costs **$m+n$** record movements.
- Greedy rule: always merge the **two smallest** files first — this is exactly Huffman's merge logic applied to file sizes (minimizes total weighted movement).

Worked Example — 8 files, page counts: 18,3,15,12,10,11,7,9

```
Sorted: 3,7,9,10,11,12,15,18
Merge 3+7=10 (running total 10)
Merge 9+10=19 (total 29)
Merge 10+11=21 (total 50)
Merge 12+15=27 (total 77)
Merge 18+19=37 (total 114)
Merge 21+27=48 (total 162)
Merge 37+48=85 (total 247)
Minimum total record movements = 247
```

18.4 Knapsack — Fractional (Greedy WORKS) vs 0/1 (Greedy FAILS)

- **Fractional Knapsack:** items are divisible. Sort by **value/weight ratio descending**, take as much of the best-ratio item as fits, then move to the next. This greedy strategy is **provably optimal**.
- **0/1 Knapsack:** items are indivisible (all-or-nothing). Greedy by ratio (or profit, or weight alone) can be **sub-optimal** — must use Dynamic Programming (Ch.19).
- **TRAP:** Greedy by **profit alone** or **weight alone** is WRONG even for the fractional version — only the **ratio (profit/weight)** ordering guarantees optimality.

Worked Example — capacity $W=20$; $O_1(\text{profit}25, \text{weight}18)$, $O_2(\text{profit}24, \text{weight}15)$, $O_3(\text{profit}15, \text{weight}10)$

Ratios: $O1=1.39$, $O2=1.60$, $O3=1.50$ → optimal order: $O2$, $O3$, $O1$

Greedy by RATIO (optimal): take $O2$ fully ($w15, p24$), capacity left=5.
Take $5/10$ of $O3$: profit += $15 \times (5/10) = 7.5$. TOTAL = 31.5 <- OPTIMAL

Greedy by PROFIT alone (wrong order): $O1$ first ($w18, p25$), capacity left=2.
Take $2/15$ of $O2$: profit += $24 \times (2/15) = 3.2$. TOTAL = 28.2 (sub-optimal)

Greedy by WEIGHT alone (wrong order): $O3$ first ($w10, p15$), capacity left=10.
Take $10/15$ of $O2$: profit += $24 \times (10/15) = 16$. TOTAL = 31 (sub-optimal, close but not best)

18.5 Job Sequencing with Deadlines

- n jobs, each with a deadline and profit; each job takes 1 time unit; **one job per time slot**; maximize total profit.
- Algorithm: (1) Sort jobs by profit **descending**. (2) For each job, place it in the **latest free slot** at or before its deadline. (3) If no such slot exists, discard the job.

Worked Example — $T1(P=100, D=2)$, $T2(P=10, D=1)$, $T3(P=27, D=2)$, $T4(P=15, D=1)$

Sort by profit: $T1(100, d2)$, $T3(27, d2)$, $T4(15, d1)$, $T2(10, d1)$
 $T1(d2)$: slot 2 free → assign. Profit=100
 $T3(d2)$: slot 2 taken → try slot 1 → free → assign. Profit=127
 $T4(d1)$: slot 1 taken → no earlier slot → REJECT
 $T2(d1)$: slot 1 taken → REJECT
 Maximum profit = 127 (jobs $T1$, $T3$ scheduled)

Questions

Q1. [Trap] For which problem does a straightforward greedy strategy generally FAIL to find the optimal solution?

A. Fractional Knapsack B. Huffman Coding C. 0/1 Knapsack D. Optimal Merge Pattern

Answer: C. **Why:** 0/1 Knapsack's indivisibility breaks the greedy-choice property; only Dynamic Programming guarantees the optimal solution here.

Q2. [Numerical] Using the frequencies $M1=12$, $M2=4$, $M3=45$, $M4=17$, $M5=23$, what is the total number of bits needed to Huffman-encode a message with these character counts?

A. 303 B. 206 C. 224 D. 101

Answer: B. **Why:** Sum of (frequency × code length): $12(4)+4(4)+45(1)+17(3)+23(2) = 206$ bits (see worked example above).

Q3. [Trap] For the frequency set $a=5, b=9, c=12, d=13, e=16, f=45$, the word "dead" can be validly encoded as EITHER 101111100101 OR its bit-complement 0100000011010. Why are both correct?

A. Only one can be correct; the other is an error B. The 0/1 labeling of left/right children at each merge is an arbitrary convention — swapping it everywhere yields an equally valid, equally optimal code with identical lengths C. Huffman coding is non-deterministic and gives random results D. Because "dead" has repeated letters

Answer: B. **Why:** Huffman optimality depends only on code LENGTHS (determined by tree structure/depth), not which branch is labeled 0 vs 1 — both labelings are valid, prefix-free, and equally optimal.

Q4. What is the time complexity of building a Huffman code for n symbols using a min-heap?

A. $O(n)$ B. $O(n \log n)$ C. $O(n^2)$ D. $O(n (\log n)^2)$

Answer: B. **Why:** $n-1$ merge steps, each requiring $O(\log n)$ heap extract-min and insert operations.

Q5. [Numerical] 8 files of sizes 18, 3, 15, 12, 10, 11, 7, 9 pages are merged two at a time (optimal merge pattern). What is the minimum total number of record movements?

A. 162 B. 200 C. 247 D. 300

Answer: C. **Why:** Repeatedly merging the two smallest values (Huffman-style) gives cumulative cost $10+19+21+27+37+48+85 = 247$.

Q6. [Numerical] Knapsack capacity=20; items $O1(\text{profit}25, w18)$, $O2(\text{profit}24, w15)$, $O3(\text{profit}15, w10)$. Using the CORRECT (ratio-based) greedy strategy for Fractional Knapsack, what is the maximum profit?

A. 28.2 B. 31 C. 31.5 D. 34

Answer: C. **Why:** Sort by ratio ($O2=1.6$, $O3=1.5$, $O1=1.39$): take $O2$ fully (profit 24), then $5/10$ of $O3$ (profit 7.5) = 31.5 total — the true optimum (profit-only or weight-only ordering give 28.2 or 31, both sub-optimal).

Q7. [Numerical] Jobs $T1(\text{profit}100, \text{deadline}2)$, $T2(\text{profit}10, \text{deadline}1)$, $T3(\text{profit}27, \text{deadline}2)$, $T4(\text{profit}15, \text{deadline}1)$ — single machine, 1 unit per job. What is the maximum achievable profit?

A. 100 B. 115 C. 127 D. 152

Answer: C. **Why:** Scheduling $T1$ in slot 2 and $T3$ in slot 1 (both meet their deadlines) yields $100+27=127$; $T2$ and $T4$ cannot be scheduled since both slots by their deadline (1) are already used.

Q8. Why does the Fractional Knapsack greedy strategy require sorting by value/weight RATIO rather than by value or weight alone?

A. Ratio sorting is just a convention with no mathematical necessity B. Only ratio ordering guarantees the maximum value gained per unit of capacity consumed at every step, which is what optimality requires C. Weight-based sorting is always equivalent to ratio-based sorting D. Profit-based sorting always gives the same result as ratio-based sorting

Answer: B. **Why:** The greedy-choice property for Fractional Knapsack specifically requires maximizing value-per-unit-weight at each greedy step; sorting by profit or weight alone ignores this ratio and can pick a worse combination (as shown in the worked example: 28.2 or 31 vs the true optimum 31.5).

2.19 Dynamic Programming

19.1 DP Paradigm

- Like Divide & Conquer, DP combines solutions to subproblems — but DP applies when subproblems are **NOT independent**, i.e., subproblems **overlap** (share subproblems). Each distinct subproblem is solved **once** and its answer is **cached/tabulated**, avoiding recomputation.
- TRAP:** This overlapping-subproblems property is exactly what **distinguishes DP from plain Divide & Conquer** — Merge Sort's subproblems never overlap, so memoization gives it no benefit; DP problems (LCS, MCM, Knapsack) have massively overlapping subproblems, so memoization gives an exponential-to-polynomial speedup.
- Two implementation styles: **Top-down/Memoization** (recursive + cache — same asymptotic complexity, but risks stack depth for large n) vs **Bottom-up/Tabulation** (iterative, build the table from the smallest subproblems up — no recursion overhead).
- 4 standard steps: (1) Characterize the structure of an optimal solution. (2) Recursively define its value. (3) Compute it bottom-up. (4) Reconstruct the actual solution from the table.

19.2 Longest Common Subsequence (LCS)

```

C[i,j] = 0                      if i=0 or j=0
C[i,j] = C[i-1,j-1] + 1        if x_i = y_j
C[i,j] = max( C[i-1,j], C[i,j-1] )  if x_i != y_j

```

- Time & space: **O(m x n)** for the table; reconstructing the actual LCS string takes an additional O(m+n).

Worked Example — X = "ABCBADAB" (m=7), Y = "BDCABA" (n=6)

Filling the C[i,j] table per the recurrence above gives **LCS length = 4**, e.g. the subsequence **"BCBA"** (or equivalently "BDAB") — both are valid longest common subsequences of the same length.

19.3 Matrix Chain Multiplication (MCM)

- Goal: choose the **order of multiplication** (parenthesization) of a chain of matrices to minimize total scalar multiplications — the matrices themselves are NOT actually multiplied, only the optimal split point is found.
- If matrix A_i has dimensions P_(i-1) x P_(i):

```

m[i,i] = 0
m[i,j] = min over k in [i,j-1] of ( m[i,k] + m[k+1,j] + P(i-1)*P(k)*P(j) )

```

- Time complexity: **O(n^3)** (O(n^2) subproblems, O(n) choices of k each).

Worked Example 1 — A₁(30x35), A₂(35x15), A₃(15x5), A₄(5x10), A₅(10x20)

Minimum scalar multiplications = **15125** (classic result; optimal split groups as ((A₁(A₂ A₃))(A₄ A₅))).

Worked Example 2 — M₁(10x100), M₂(100x20), M₃(20x5), M₄(5x80) [P = 10,100,20,5,80]

```

m[1,2]=10*100*20=20000      m[2,3]=100*20*5=10000      m[3,4]=20*5*80=8000
m[1,3]=min(k=1: 0+10000+10*100*5=15000, k=2: 20000+0+10*20*5=21000) = 15000
m[2,4]=min(k=2: 0+8000+100*20*80=168000, k=3: 10000+0+100*5*80=50000) = 50000
m[1,4]=min(
  k=1: 0+50000+10*100*80=130000,
  k=2: 20000+8000+10*20*80=44000,
  k=3: 15000+0+10*5*80=19000    <- MINIMUM
)
Minimum scalar multiplications = 19000
(optimal grouping: M1 * ((M2*M3)*M4) )

```

TRAP: Two "obvious" parenthesizations of this same chain give 44000 and 25000 multiplications — NEITHER is optimal. The true minimum (19000) requires checking **all** split points via the DP recurrence, not just eyeballing a couple of groupings.

19.4 Floyd-Warshall (All-Pairs Shortest Path)

```

dist[i][j] = min( dist[i][j], dist[i][k] + dist[k][j] )   for each intermediate vertex k = 1..n

```

- O(n^3) time; works with **negative edge weights** but NOT with a **negative-weight cycle** (result becomes meaningless/undefined along that cycle).

Worked Example — 3 vertices, direct edges: 1->2=5, 2->3=2, 3->1=1 (others = infinity)

Initial:	After k=1:	After k=2:	After k=3 (final):
1 2 3	1 2 3	1 2 3	1 2 3
1 0 5 INF	1 0 5 INF	1 0 5 7	1 0 5 7
2 INF 0 2	2 INF 0 2	2 INF 0 2	2 3 0 2
3 1 INF 0	3 1 6 0	3 1 6 0	3 1 6 0

Final shortest distances: d(1,3)=7 (via 1->2->3), d(2,1)=3 (via 2->3->1), d(3,2)=6 (via 3->1->2)

19.5 Subset Sum Problem

```

dp[i][s] = dp[i-1][s] OR dp[i-1][s - arr[i]]   (True if a subset of first i elements sums to s)

```

Worked Example — S = {2, 3, 7, 8, 10}, target sum = 14

Checking every one of the 31 non-empty subsets systematically (singles, pairs, triples, quadruples, and the full set) — **no subset sums to exactly 14** (closest are 13 and 15). **Answer: No such subset exists.**

TRAP: Don't assume a subset always exists just because the target is "in range" ($2 \leq 14 \leq 30 = \text{total sum}$) — the DP table must be checked exactly; existence is not guaranteed.

Questions

Q1. [Trap] What is the key property that makes Dynamic Programming applicable but NOT plain Divide & Conquer?

- A. The problem must be recursive B. Subproblems must overlap (shared sub-subproblems), unlike D&C's independent subproblems C. The problem must involve sorting D. DP only works on graphs

Answer: B. Why: DP's advantage (caching) only pays off when subproblems recur; D&C problems like Merge Sort have non-overlapping subproblems, so there's nothing to cache.

Q2. [Numerical] For $X = \text{"ABCBDAB"}$ and $Y = \text{"BDCABA"}$, what is the length of the Longest Common Subsequence?

- A. 3 B. 4 C. 5 D. 6

Answer: B. Why: Applying the LCS recurrence yields length 4 (e.g., "BCBA").

Q3. [Numerical] Matrices $A_1(30 \times 35)$, $A_2(35 \times 15)$, $A_3(15 \times 5)$, $A_4(5 \times 10)$, $A_5(10 \times 20)$ — minimum scalar multiplications to compute $A_1A_2A_3A_4A_5$?

- A. 30000 B. 15125 C. 7875 D. 45500

Answer: B. Why: This is the classic MCM DP result — 15125, achieved via the specific split $((A_1(A_2A_3))(A_4A_5))$.

Q4. [Numerical/Trap] For $M_1(10 \times 100)$, $M_2(100 \times 20)$, $M_3(20 \times 5)$, $M_4(5 \times 80)$, which grouping gives the TRUE minimum multiplication count?

- A. $((M_1M_2)(M_3M_4))$ with 44000 B. $((((M_1M_2)M_3)M_4))$ with 25000 C. $(M_1((M_2M_3)M_4))$ with 19000 D. All groupings give the same cost

Answer: C. Why: Full DP evaluation of all split points shows grouping as $M_1((M_2M_3)M_4)$ costs only 19000 — lower than either of the two "obvious" groupings (44000, 25000).

Q5. What is the time complexity of the Matrix Chain Multiplication DP algorithm for n matrices?

- A. $O(n)$ B. $O(n \log n)$ C. $O(n^2)$ D. $O(n^3)$

Answer: D. Why: $O(n^2)$ subproblems (i, j pairs), each requiring an $O(n)$ scan over split points k .

Q6. Floyd-Warshall's all-pairs shortest path algorithm fails to give meaningful results when the graph contains:

- A. Negative edge weights B. A negative-weight cycle C. Disconnected components D. Self-loops with weight 0

Answer: B. Why: Floyd-Warshall correctly handles individual negative edges, but a negative cycle allows unbounded decreasing path weight, making shortest-path distances undefined along it.

Q7. [Numerical/Trap] Does the set $S = \{2, 3, 7, 8, 10\}$ contain a subset summing to exactly 14?

- A. Yes, $\{2, 3, \dots\}$ sums to 14 B. Yes, $\{7, \dots\}$ sums to 14 C. No such subset exists D. Cannot be determined without more information

Answer: C. Why: Systematically checking all 31 non-empty subsets shows none sum to exactly 14 (13 and 15 are the closest achievable sums).

Q8. Top-down (memoized) and Bottom-up (tabulated) DP implementations differ mainly in:

- A. Asymptotic time complexity — tabulation is always faster B. Tabulation avoids recursion-call/stack overhead by building the table iteratively from smallest subproblems, while memoization uses recursion with caching C. Only memoization gives the correct answer D. Tabulation cannot reconstruct the actual optimal solution, only its value

Answer: B. Why: Both achieve the same asymptotic complexity; tabulation's iterative bottom-up build avoids the function-call and stack-depth overhead that recursive memoization carries.

2.20 Graph Algorithms — MST & Shortest Path

20.1 Minimum Spanning Tree (MST)

- A **spanning tree** of a connected graph G is a subgraph that is a tree AND includes every vertex of G . An MST is the spanning tree with **minimum total edge weight**.
- A graph with n vertices has an MST with **exactly $n-1$ edges** (this fact is heavily tested).
- MST is defined only for **connected** graphs; a disconnected graph instead has a **minimum spanning forest** (union of each component's MST).
- Multiple MSTs can exist if edge weights tie, but the **total MST weight is always unique**.

20.2 Kruskal's Algorithm (edge-centric, Greedy)

```

1. Sort all edges by weight, ascending.
2. For each edge (u,v) in that order:
   if Find_Set(u) != Find_Set(v):      # doesn't create a cycle
       add edge to MST; Union(u,v)
3. Stop once n-1 edges are added.
```

- Cycle detection via **Union-Find (Disjoint Set)**. Time: **$O(E \log E)$** (dominated by the sort).
- Best suited for **sparse** graphs (edge-list based).

20.3 Prim's Algorithm (vertex-centric, Greedy)

```

1. Start from any vertex; tree = {start}.
2. Repeatedly add the MINIMUM weight edge connecting the tree to a vertex OUTSIDE the tree.
3. Stop when all vertices are included.
```

- Time: **$O(E \log V)$** with a min-heap (or $O(V^2)$ with a simple array — better for **dense** graphs).
- **TRAP:** Kruskal's picks edges **globally** by weight (needs Union-Find to avoid cycles); Prim's grows a **single connected tree** greedily from a chosen source — different mechanisms, but both are provably correct and give the **same total MST weight**.

Worked Example — vertices A,B,C,D,E; edges A-B=2, A-C=3, B-C=1, B-D=4, C-D=5, C-E=6, D-E=2

```

KRUSKAL (sort: BC=1, AB=2, DE=2, AC=3, BD=4, CD=5, CE=6):
BC(1): add      {B,C}
AB(2): add      {A,B,C}
DE(2): add      {D,E}
AC(3): SKIP (A,C already same component -> would cycle)
BD(4): add -> connects {A,B,C} and {D,E}    ALL CONNECTED (4 edges)
Total weight = 1+2+2+4 = 9

PRIM (start at A):
Add B via A-B=2      (tree wt=2)
Add C via B-C=1      (tree wt=3)
Add D via B-D=4      (tree wt=7)
Add E via D-E=2      (tree wt=9)
Total weight = 9    <- MATCHES Kruskal's total
```

20.4 Dijkstra's Algorithm (Single-Source Shortest Path)

```

1. dist[source]=0, all others=infinity.
2. Repeatedly extract the UNVISITED vertex with smallest known dist; mark it visited (finalized).
3. RELAX every neighbor v of u: if dist[v] > dist[u] + w(u,v): dist[v] = dist[u] + w(u,v)
```

- Time: **$O(E \log V)$** with a min-heap. Requires **non-negative** edge weights.
- **TRAP:** Dijkstra **fails with negative edge weights** — once a vertex is "finalized" (visited), its distance is assumed final, but a later negative edge could still improve it. Use **Bellman-Ford** for graphs with negative weights.

Worked Example — Dijkstra from source A on the same graph as §20.3

```

Visit A(0): relax B->2, C->3
Visit B(2): relax D-> 2+4=6      (C via B: 2+1=3, no improvement, tie)
Visit C(3): relax E-> 3+6=9      (D via C: 3+5=8, no improvement, 6 already better)
Visit D(6): relax E-> 6+2=8      (improves 9 -> 8)
Visit E(8): done

Final shortest distances from A:  A=0, B=2, C=3, D=6, E=8
```

20.5 Bellman-Ford Algorithm

```

1. dist[source]=0, all others=infinity.
2. Relax ALL edges, repeated (V-1) times.
3. One more pass: if any edge can STILL be relaxed -> a negative-weight cycle exists (report it).
```

- Time: **$O(V \times E)$** — slower than Dijkstra, but works with **negative edges** and can **detect negative cycles** (a capability Dijkstra has no equivalent for at all).

20.6 Comparison Table

Algorithm	Purpose	Handles negative weights?	Complexity
Kruskal	MST	N/A (undirected weights, can be negative too — still works)	$O(E \log E)$
Prim	MST	N/A	$O(E \log V)$
Dijkstra	Single-source shortest path	No	$O(E \log V)$
Bellman-Ford	Single-source shortest path	Yes , and detects negative cycles	$O(V \cdot E)$

20.7 Classic Conceptual Traps (verified True/False)

- **"If all edge weights in a graph are unique (no ties), the shortest path between any two vertices is always unique."** → **TRUE**. Distinct weights eliminate the possibility of two different paths tying for minimum total weight.
- **"With all-positive weights, Dijkstra and Bellman-Ford may report different actual PATHS, but the path WEIGHT is always the same."** → **TRUE**. Both algorithms correctly compute the shortest-path *weight*; only the specific path chosen among ties can differ.
- **"If every edge weight is doubled, the shortest path (which edges are used) stays the same — only the total weight doubles."** → **TRUE**. Multiplying all weights by a positive constant preserves the relative ordering of all path lengths.
- **TRAP — "If every edge weight is increased by the SAME constant (e.g., +1), the shortest path always remains the same."** → **FALSE**. Adding a constant per edge disproportionately penalizes paths with MORE edges (hops). Counter-example: direct edge $A \rightarrow B = 10$ (1 edge) vs. a 3-edge path totaling 9 (3 edges of weight 3 each) — the 3-edge path is shorter ($9 < 10$). Add 1 to every edge: direct becomes 11 (1 edge), the 3-edge path becomes $3 \times 4 = 12$ (3 edges) — now the DIRECT edge is shorter ($11 < 12$). The shortest path **flipped**.
- **"Which of these is NOT a Greedy algorithm: Dijkstra / Prim / Kruskal / Huffman / Bellman-Ford?"** → **Bellman-Ford** (it systematically relaxes all edges $V-1$ times rather than making a locally-greedy choice; it's relaxation/DP-flavored, not greedy).
- **"Which of these is NOT Dynamic-Programming based: Bellman-Ford / Floyd-Warshall / 0-1 Knapsack / Prim's MST?"** → **Prim's MST** (it's Greedy; the other three are classic DP algorithms).

Questions

Q1. A connected graph has 12 vertices. How many edges does its Minimum Spanning Tree contain?

A. 12 B. 11 C. 13 D. Depends on the graph's total edges

Answer: B. Why: An MST always has exactly $(n-1)$ edges for n vertices — here $12-1=11$.

Q2. [Numerical] Using the graph $A-B=2$, $A-C=3$, $B-C=1$, $B-D=4$, $C-D=5$, $C-E=6$, $D-E=2$, what is the total MST weight?

A. 7 B. 9 C. 11 D. 13

Answer: B. Why: Both Kruskal's (edges BC, AB, DE, BD) and Prim's independently arrive at total weight $1+2+2+4=9$.

Q3. [Trap] Dijkstra's algorithm is unsuitable for a graph with:

A. Many vertices B. Cycles C. Negative edge weights D. Disconnected components

Answer: C. Why: Dijkstra assumes a finalized vertex's distance can never improve — a later negative edge can violate this assumption, producing incorrect results.

Q4. [Numerical] Using Dijkstra from source A on $A-B=2$, $A-C=3$, $B-C=1$, $B-D=4$, $C-D=5$, $C-E=6$, $D-E=2$, what is the shortest distance from A to E?

A. 6 B. 8 C. 9 D. 11

Answer: B. Why: Shortest path $A \rightarrow B \rightarrow D \rightarrow E = 2+4+2 = 8$, beating $A \rightarrow C \rightarrow E$ ($3+6=9$).

Q5. Which algorithm can both compute shortest paths with negative edges AND detect a negative-weight cycle?

A. Dijkstra's B. Prim's C. Kruskal's D. Bellman-Ford

Answer: D. Why: Bellman-Ford's extra (V -th) relaxation pass reveals a negative cycle if any edge can still be relaxed — a capability unique to it among these four.

Q6. [Trap] "If every edge's weight is increased by the same constant, the shortest path never changes." This statement is:

A. True — additive constants never affect relative path comparisons B. False — paths with more edges are penalized more than paths with fewer edges, which can flip which path is shortest C. True only for undirected graphs D. True only when the constant is negative

Answer: B. Why: A constant added per edge scales with the number of edges in each path, so a longer (more-hop) path that was originally shorter can become longer than a shorter-hop path after the addition (see the worked 1-edge-vs-3-edge counter-example above).

Q7. Which of the following is NOT a Greedy algorithm?

A. Prim's MST B. Kruskal's MST C. Huffman Coding D. Bellman-Ford Shortest Path

Answer: D. Why: Bellman-Ford performs systematic edge relaxation over $V-1$ rounds rather than a locally greedy selection; the other three make a clear greedy choice at each step.

Q8. Which of the following is NOT Dynamic-Programming based?

A. Bellman-Ford B. Floyd-Warshall C. 0-1 Knapsack D. Prim's MST

Answer: D. Why: Prim's is a Greedy algorithm; the other three rely on DP recurrences over subproblems.

Q9. Why is Kruskal's algorithm generally preferred for SPARSE graphs, while Prim's (array-based) is preferred for DENSE graphs?

A. Kruskal's cannot run on dense graphs at all B. Kruskal's cost is dominated by sorting E edges (efficient when E is small relative to V^2), while a simple Prim's implementation scans all vertices each step regardless of edge count, which pays off only when E is large C. Prim's cannot handle sparse graphs D. There is no such distinction; both always perform identically

Answer: B. Why: Kruskal's $O(E \log E)$ scales with edge count, favoring sparse graphs; a simple $O(V^2)$ Prim's implementation performs consistently regardless of E , so it becomes relatively more efficient as graphs get denser (E close to V^2).

2.21 String Matching Algorithms

21.1 The Problem

- Find all occurrences of a **Pattern (P, length m)** within a **Text (T, length n)**.
- 4 standard approaches: Naive, Rabin-Karp, Finite Automata-based, and Knuth-Morris-Pratt (KMP).

21.2 Naive (Brute-Force) String Matching

```
for i = 1 to (n-m+1):
    compare P[1..m] against T[i..i+m-1] character by character
    if all match: report a hit at position i
```

- Tries **every** possible alignment ($n-m+1$ positions), comparing character by character.
- Complexity: **Best case $O(n)$** (mismatches found immediately at most positions), **Average/Worst case $O(n-m)$** (e.g., text of all A's, pattern of all A's-then-B — many near-full matches before the final mismatch). Space: **$O(1)$** .

Worked Example — $T = \text{"AABAACAADAABAABA"} (n=16), P = \text{"AABA"} (m=4)$

Checking every alignment position by position: **matches found at positions 1, 10, and 13** (1-based indexing).

21.3 Rabin-Karp Algorithm

```
1. Compute hash(P).
2. Compute hash of the first length-m window of T.
3. If hash(window) == hash(P): verify with a full character comparison (rule out hash collision).
4. Slide the window by 1; update its hash in  $O(1)$  via a ROLLING HASH (remove outgoing char's contribution, shift, add incoming char) — NOT recompute
5. Repeat until the text is exhausted.
```

- Hash Collision:** two different substrings can hash to the same value — character verification is mandatory whenever hashes match, to rule out a false positive.
- Complexity: **Best/Average case $O(n+m)$** (rolling hash update is $O(1)$ per shift, verification is rare); **Worst case $O(n-m)$** if hash collisions are frequent (e.g., a poorly chosen hash function). Space: **$O(1)$** .
- Advantages: faster than Naive on average; efficient when searching for **multiple patterns simultaneously** (precompute multiple pattern hashes, scan text once).
- Disadvantages: performance degrades with hash collisions; needs a good hash function.

Worked Example — $P = \text{"31415"}, T = \text{"2359023141526739921"}$

Rolling-hash comparison finds the hash match (then verifies) at **position 7** of T : $T[7..11] = \text{"31415"}$ — an exact match.

21.4 Knuth-Morris-Pratt (KMP) — brief contrast

- Precomputes a **failure function / prefix table** (longest proper prefix of P that is also a suffix, for every prefix of P) so that on a mismatch, the pattern is shifted intelligently — **never re-examining a text character already matched**.
- Achieves a guaranteed **$O(n+m)$ worst case** — unlike Rabin-Karp, which can degrade to $O(n-m)$ under hash collisions, and unlike Naive's $O(n-m)$ worst case. [VERIFY: exact prefix-table construction steps if a numeric KMP trace question appears — the source material for this chapter did not cover KMP's mechanics in depth, only its existence as a standard companion algorithm.]

21.5 Summary Comparison

Algorithm	Best Case	Average Case	Worst Case	Space	Key Idea
Naive	$O(n)$	$O(n-m)$	$O(n-m)$	$O(1)$	Brute-force compare at every position
Rabin-Karp	$O(n+m)$	$O(n+m)$	$O(n-m)$	$O(1)$	Rolling hash, verify on hash match
KMP	$O(n+m)$	$O(n+m)$	$O(n+m)$ guaranteed	$O(m)$ (prefix table)	Precomputed failure function, no re-scanning

TRAP: Rabin-Karp's *average-case* speed is the same order as KMP's *guaranteed worst case* — but Rabin-Karp can still degrade to $O(n-m)$ under adversarial input with many hash collisions, while KMP **never** does. When a worst-case guarantee matters, KMP is the stronger choice despite similar typical-case performance.

Questions

Q1. [Numerical] For $T = \text{"AABAACAADAABAABA"}$ and $P = \text{"AABA"}$, at which positions (1-indexed) does the Naive algorithm report a match?

A. 1, 5, 9 B. 1, 10, 13 C. 1, 4, 10 D. 10, 13, 16

Answer: B. Why: Direct character-by-character comparison at each alignment confirms exact matches only at positions 1, 10, and 13.

Q2. What is the worst-case time complexity of the Naive string matching algorithm?

A. $O(n+m)$ B. $O(n \log m)$ C. $O(n-m)$ D. $O(\log n)$

Answer: C. Why: In the worst case (e.g., highly repetitive text), nearly every alignment requires close to m comparisons before a mismatch, across $n-m+1$ positions.

Q3. In Rabin-Karp, why is a character-by-character comparison still needed even after a hash match is found?

A. It's not needed; hash match alone confirms a true match B. To rule out a hash collision, where two different substrings share the same hash value C. To recompute the hash from scratch D. Only needed if the pattern length exceeds 10

Answer: B. Why: A hash match is necessary but not sufficient for a true match — distinct strings can collide to the same hash value, so verification is required.

Q4. [Numerical] Searching for pattern "31415" in text "2359023141526739921" using Rabin-Karp, at what position does the true match occur?

A. Position 3 B. Position 5 C. Position 7 D. No match exists

Answer: C. Why: $T[7..11] = \text{"31415"}$, an exact character match confirming the hash hit at position 7.

Q5. [Trap] Which string matching algorithm guarantees $O(n+m)$ even in the WORST case, unlike Rabin-Karp which can degrade under hash collisions?

A. Naive B. Rabin-Karp C. KMP D. None can guarantee this

Answer: C. Why: KMP's precomputed failure function ensures text characters are never re-examined, giving a true worst-case $O(n+m)$ bound that Rabin-Karp cannot guarantee under adversarial hash collisions.

Q6. What technique allows Rabin-Karp to update its window hash in $O(1)$ time as it slides across the text?

A. Recomputing the hash from scratch each time B. Rolling Hash (remove the outgoing character's contribution, shift, add the incoming character) C. Sorting the characters in the window D. Using a separate thread for each window

Answer: B. Why: The rolling hash technique incrementally updates the hash value in constant time instead of rehashing the entire window, which is what gives Rabin-Karp its average-case speed advantage.

3.1 Finite Automata — DFA & NFA

3.1.1 Deterministic Finite Automata (DFA)

- A DFA is formally a 5-tuple $(Q, \Sigma, \delta, q_0, F)$: states, alphabet, transition function, start state, final/accepting states.
- Deterministic**: for every state and every input symbol, there is **exactly one** defined transition (no missing transitions, no choices). A DFA can be made "complete" by adding a **dead/trap state** to absorb undefined transitions.
- A string is **accepted** if, starting from q_0 and reading all symbols, the automaton ends in a state $\in F$.
- TRAP**: A DFA has no ϵ -(epsilon) transitions and no ambiguity — "determinism" means both **no missing** and **no multiple** transitions per (state, symbol) pair; a transition table with any blank cell is not yet a valid complete DFA.

3.1.2 Nondeterministic Finite Automata (NFA)

- An NFA relaxes the transition function to allow: **zero, one, or multiple** transitions per (state, symbol), and **ϵ -transitions** (moving states without consuming input).
- A string is accepted if **at least one** computation path (among all nondeterministic choices) ends in an accepting state.
- NFA-to-DFA conversion (subset construction)**: each DFA state = a set of NFA states reachable so far. Worst case, an NFA with n states converts to a DFA with up to 2^n states (though often far fewer in practice).
- TRAP**: **Every NFA has an equivalent DFA** — NFAs do **not** recognize a strictly larger class of languages than DFAs; nondeterminism only affects the *convenience of construction* and potential state-count blowup, not the expressive power. Both recognize exactly the **regular languages**.

3.1.3 Minimization of DFAs

- Minimal DFA**: the DFA with the fewest states recognizing a given regular language; unique up to renaming of states (via Myhill-Nerode equivalence classes).
- Method**: partition states into equivalence classes — start with {final states} and {non-final states}, then iteratively split any class whose members transition to *different* classes on some input symbol, until no further split occurs (fixed point).
- Unreachable states** (unreachable from q_0) must be removed *before* minimization; they don't affect language recognition but bloat the state count.
- TRAP**: Two states can only be merged in minimization if they are **equivalent for ALL possible future input** (not just the inputs seen so far) — a common shortcut error is merging states that happen to look identical on a few test strings but actually diverge later.

Questions

Q1. In a DFA, for a given state and input symbol, the number of possible next-states is:

A. Zero or one, but never exactly one is required B. Exactly one, always C. Any number, including multiple D. Always equal to the alphabet size

Answer: B. Why: Determinism specifically requires exactly one transition per (state, symbol) pair — this is the defining constraint distinguishing a DFA from an NFA.

Q2. [Trap] Compared to a DFA, an NFA:

A. Can recognize a strictly larger class of languages B. Recognizes exactly the same class of languages (regular languages), just with more flexible construction C. Cannot have epsilon-transitions D. Always requires fewer states to represent any regular language

Answer: B. Why: NFAs and DFAs are equivalent in expressive power — every NFA can be converted to an equivalent DFA (subset construction); nondeterminism is a convenience for design, not extra power.

Q3. [Numerical] Using subset construction, an NFA with n states converts to a DFA with, in the absolute worst case, how many states?

A. n B. $2n$ C. 2^n D. n^2

Answer: C. Why: Each DFA state corresponds to a subset of the NFA's state set, giving up to 2^n possible DFA states in the worst case (though many fewer are often reachable in practice).

Q4. An epsilon (ϵ) transition in an NFA allows:

A. Consuming exactly one input symbol while changing state B. Changing state without consuming any input symbol C. Rejecting the input immediately D. Looping on the same state forever

Answer: B. Why: ϵ -transitions let the automaton move between states "for free," without reading a symbol from the input — a purely NFA-only construct not permitted in DFAs.

Q5. [Trap] When minimizing a DFA, two states can be merged only if they are:

A. Both final states, regardless of transitions B. Equivalent for all possible future input strings, not just strings tested so far C. Reachable from the start state within the same number of steps D. Labeled with the same name

Answer: B. Why: True state equivalence requires identical behavior (accept/reject) for every possible suffix string — a state pair that merely matches on a few sample strings but diverges later must remain distinct.

Q6. Before running DFA minimization, which states should first be removed?

A. All final states B. All states with self-loops C. States unreachable from the start state D. The start state itself

Answer: C. Why: Unreachable states never affect the language recognized and must be pruned first; leaving them in only adds unnecessary states (they wouldn't be merged with reachable ones in a meaningful way).

Q7. A DFA accepts a given input string if, after processing every symbol starting at q_0 , the automaton is in:

A. Any state at all B. A state that is a member of the set of final/accepting states F C. The start state q_0 again D. A dead/trap state

Answer: B. Why: Acceptance is defined purely by whether the automaton's final resting state, after consuming the whole string, belongs to F.

Q8. A "dead" or "trap" state is added to a DFA to:

A. Represent an accepting condition B. Absorb all otherwise-undefined transitions, keeping the DFA total/complete C. Serve as the new start state D. Reduce the total number of states

Answer: B. Why: A complete DFA needs a transition defined for every (state, symbol) pair; a non-accepting dead state catches all "no valid path forward" cases so the automaton remains formally total.

Q9. The minimal DFA for a given regular language is unique:

A. Only if the language is finite B. Up to renaming/relabeling of states C. Never — there can be many equally minimal DFAs with genuinely different structure D. Only for languages over a single-symbol alphabet

Answer: B. Why: Myhill-Nerode theory guarantees the minimal DFA is unique in structure; different minimal DFAs for the same language differ only in how states happen to be labeled.

Q10. [Trap] The primary practical benefit of using an NFA instead of a DFA when designing an automaton is:

A. It can accept languages a DFA cannot B. It is often much easier/more compact to construct directly from a problem description or regular expression, even though it may later be converted to a DFA C. It executes faster at runtime than a DFA D. It requires no start state

Answer: B. Why: NFAs are a design convenience — smaller, more natural constructions (e.g., directly from a regex via Thompson's construction) — with the theoretical guarantee they're convertible to an equivalent DFA when a deterministic implementation is needed.

3.2 Regular Expressions & Closure Properties of Regular Languages

3.2.1 Regular Expression Basics

- Operators: **Union (+ or |)**, **Concatenation**, **Kleene star (*, zero or more repetitions)**. **Kleene plus (+, one or more)** is a shorthand derivable as $R^+ = R \cdot R^*$.
- Every regular expression has an equivalent NFA (Thompson's construction) and hence an equivalent DFA — regular expressions, NFAs, and DFAs are **three equally powerful notations for exactly the regular languages**.
- **Arden's Theorem**: for equations of the form $R = Q + RP$ (P must not contain ϵ), the solution is $R = QP^*$. Used to convert a finite automaton's transition equations back into a regular expression.
- **TRAP**: Arden's theorem requires **P must not derive ϵ** (not contain the empty string) — if it does, the equation may have multiple solutions and the theorem doesn't directly apply.

3.2.2 Closure Properties of Regular Languages

Regular languages are **closed** under:

- Union, Concatenation, Kleene star, Complementation, Intersection, Difference, Reversal, Homomorphism (and inverse homomorphism).
- "Closed under X" means: if L_1, L_2 are regular, then applying operation X to them produces **another regular language** (guaranteed, provably).
- **Complement** closure works because DFAs are total/complete — simply swap final and non-final states to get a DFA for the complement.
- **Intersection** closure: via De Morgan's law using complement + union, **or** directly via a **product construction** (build a combined DFA whose states are pairs (state in M_1 , state in M_2), running both automata in lockstep).
- **TRAP**: Regular languages ARE closed under intersection and complement — a common wrong belief (mixing them up with **context-free** languages, which are closed under union/concatenation/star but NOT closed under intersection or complement in general).

3.2.3 Proving a Language is NOT Regular (Preview)

- Closure properties are also used to prove non-regularity indirectly: if L is regular and you can express some *known non-regular* language as a closure operation on L (e.g., an intersection with a simple regular set), but that result is provably non-regular, then L itself cannot have been regular. (The direct/primary tool for this, the **Pumping Lemma**, is covered in the next chapter.)

Questions

Q1. The Kleene star operator applied to a language L (L represents:

A. Exactly one occurrence of a string from L B. Zero or more concatenations of strings from L C. Exactly one or more concatenations of strings from L D. The complement of L
Answer: B. Why: L includes the empty string (zero repetitions) plus any finite concatenation of one or more strings from L — this "zero or more" inclusiveness is Kleene star's defining feature.

Q2. [Trap] Regular languages are closed under intersection because:

A. Intersection is not actually a valid regular language operation B. A product-construction DFA (running two automata in lockstep, states as pairs) can be built to accept exactly the intersection, or equivalently via De Morgan's with complement and union C. Only NFAs support intersection, not DFAs D. Intersection always produces the empty language for regular languages

Answer: B. Why: Product construction directly builds a combined automaton for $L_1 \cap L_2$; alternatively, closure under complement and union gives intersection via De Morgan's law ($L_1 \cap L_2 = (L_1^c \cup L_2^c)^c$).

Q3. Arden's Theorem states that the equation $R = Q + RP$ has the solution:

A. $R = Q + P$ B. $R = QP$ C. $R = QP^*$ D. $R = (Q+P)^*$

Answer: B. Why: This is Arden's theorem's standard form solution, valid provided P does not contain (derive) the empty string ϵ .

Q4. [Trap] Which statement correctly distinguishes regular languages from context-free languages regarding closure?

A. Both are closed under intersection and complement B. Regular languages are closed under intersection and complement; context-free languages are NOT closed under these in general C. Context-free languages are closed under intersection and complement, but regular languages are not D. Neither class is closed under any operation

Answer: B. Why: This is a frequently tested contrast — regular languages have strong closure properties (including intersection/complement) that context-free languages lack in general.

Q5. Complementation closure for a regular language L (recognized by DFA M) is achieved by:

A. Reversing all transitions in M B. Swapping the final and non-final states in the (complete/total) DFA M C. Converting M to an NFA first D. It cannot be achieved; regular languages are not closed under complement

Answer: B. Why: Since a complete DFA has exactly one path per string, simply flipping which states count as "accepting" directly yields a DFA for the complement language.

Q6. R^+ (Kleene plus) is defined in terms of Kleene star as:

A. $R^+ = R$ alone, they are identical B. $R^+ = R \cdot R$ C. $R^+ = R \cdot R^*$ D. R^+ is unrelated to R

Answer: B. Why: Plus requires at least one occurrence, achieved by concatenating one mandatory copy of R with zero-or-more additional copies (R).

Q7. Regular expressions, NFAs, and DFAs are related as:

A. Three notations of strictly increasing expressive power, in that order B. Three equally powerful notations, each convertible to the others, all describing exactly the regular languages C. Regular expressions can express strictly more languages than DFAs D. NFAs can express languages that no DFA can express

Answer: B. Why: This equivalence ($\text{regex} \Leftrightarrow \text{NFA} \Leftrightarrow \text{DFA}$, all describing regular languages) is one of the foundational theorems of automata theory.

Q8. [Trap] The condition required for Arden's Theorem to apply validly to $R = Q + RP$ is:

A. Q must not contain ϵ B. P must not contain (derive) ϵ C. Both Q and P must be single characters D. R must already be known

Answer: B. Why: If P can derive the empty string, the equation $R = Q + RP$ can have more than one valid solution, so the clean $R = QP^*$ formula no longer uniquely applies.

Q9. Homomorphism closure for regular languages means:

A. Regular languages become non-regular under any symbol substitution B. Applying a symbol-to-string substitution mapping to every string of a regular language yields another regular language C. Only applies to finite languages D. Homomorphism is not a valid regular-language operation

Answer: B. Why: A homomorphism (systematic substitution of each alphabet symbol by some string) applied to a regular language's strings preserves regularity of the result.

Q10. Which of the following is NOT typically listed as a closure property of regular languages?

A. Union B. Kleene star C. Context-free grammar generation (i.e., regular languages are closed under producing arbitrary CFGs) D. Complementation

Answer: C. Why: "Closure" properties describe operations that, applied to regular languages, yield another *regular* language — "generating a CFG" isn't a closure operation in this sense; union, star, and complement are standard closure properties, but this option describes something else entirely (a category-confusion distractor).

3.3 Pumping Lemma for Regular Languages

3.3.1 The Pumping Lemma (Statement)

- If L is regular, then there exists a **pumping length** p such that **every** string $w \in L$ with $|w| \geq p$ can be split as $w = xyz$ where:
 - $|xy| \leq p$
 - $|y| > 0$ (y is non-empty)
 - For **every** $i \geq 0$, $xy^iz \in L$ (pumping y up or down, including removing it entirely at $i=0$, must keep the string in L)
- TRAP:** The pumping lemma is used to **prove a language is NOT regular** by *contradiction* — you assume L IS regular, then show that **no valid split exists** satisfying all three conditions simultaneously for some specific string. It can never be used to *prove* a language IS regular (satisfying the lemma is necessary but not sufficient for regularity).

3.3.2 How to Apply It (Proof Technique)

- Assume** L is regular; let p be the pumping length guaranteed by the lemma.
- Choose** a specific string $w \in L$ with $|w| \geq p$ (your choice — pick one that makes the argument work, classically something like $a^p b^p$).
- The **adversary** (lemma) splits $w = xyz$ satisfying conditions 1 and 2 above — you must show your argument holds for **every possible way** the adversary could split it, not just one.
- Derive a contradiction:** show that for some i (usually $i=0$ or $i=2$), $xy^iz \notin L$, violating condition 3 for every possible valid split — contradicting the assumption that L is regular.
- Conclude: L is **not regular**.
 - TRAP:** In step 3, since $|xy| \leq p$, y necessarily consists only of symbols from the **first p characters** of w — a very common mistake is picking a split of y that ignores this constraint (e.g., assuming y could contain characters far into the string).

3.3.3 Classic Non-Regular Languages (know these by heart)

Language	Why not regular (intuition)
$L = \{a^n b^n : n \geq 0\}$	A DFA would need to "count" and match an unbounded n — impossible with finite states
$L = \{ww : w \in \{a,b\}^*\}$	Requires comparing two arbitrarily long, unbounded halves for equality
$L = \{w w^R : w \in \{a,b\}^*\}$ (unbounded-length palindromes)	Same unbounded-memory issue — must match symbols from the start against symbols from the end
$L = \{a^p : p \text{ is prime}\}$	No finite-state machine can track "is this count prime" for unbounded counts

- TRAP:** $L = \{a^n b^m : n, m \geq 0\}$ (any number of a 's followed by any number of b 's, with **no relation required between n and m**) **IS regular** (regex: $a^* b^*$) — students often over-apply "a's followed by b's looks suspicious" intuition; it's the *equality constraint* ($n=m$) that breaks regularity, not the general shape.

Questions

Q1. [Trap] The pumping lemma for regular languages can be used to:

A. Prove that a language IS regular B. Prove that a language is NOT regular, via contradiction C. Construct the minimal DFA for any language D. Convert an NFA to a DFA

Answer: B. **Why:** Satisfying the pumping lemma's conditions is necessary but not sufficient for regularity — it is only useful as a proof-by-contradiction tool to disprove regularity, never to confirm it.

Q2. In the pumping lemma statement $w = xyz$, which condition must hold?

A. $|y| = 0$ is allowed B. $|xy| \leq p$ and $|y| > 0$ C. $|yz| \leq p$ D. z must be empty

Answer: B. **Why:** The split constrains y to fall within the first p characters ($|xy| \leq p$) and to be non-empty ($|y| > 0$) — both must hold simultaneously.

Q3. [Trap] Which of the following languages IS actually regular, despite superficially resembling a classic non-regular pattern?

A. $\{a^n b^n : n \geq 0\}$ B. $\{a^n b^m : n, m \geq 0\}$ (no relation between n and m required) C. $\{ww : w \in \{a,b\}^*\}$ D. $\{a^p : p \text{ is prime}\}$

Answer: B. **Why:** Without an equality/counting constraint linking n and m , this language is simply $a^* b^*$, expressible by a straightforward regular expression — the "a's then b's" shape alone doesn't make a language non-regular.

Q4. When applying the pumping lemma proof technique, who chooses how to split $w = xyz$?

A. You (the prover) choose the split freely B. An adversary chooses the split, subject to the lemma's constraints; you must show a contradiction for every possible such split C. The split is always uniquely determined by w D. Splitting is irrelevant to the proof

Answer: B. **Why:** Since the lemma guarantees *some* split exists satisfying the conditions, your proof must handle all possible adversarial splits — not just a convenient one.

Q5. [Trap] A common pumping-lemma proof mistake is:

A. Choosing a string w with $|w| < p$ B. Assuming y can span characters beyond the first p symbols of w , violating $|xy| \leq p$ C. Using $i = 1$ instead of $i = 0$ D. Forgetting to assume L is regular first

Answer: B. **Why:** Since $|xy| \leq p$ is guaranteed, y must lie entirely within the first p characters — proofs that implicitly let y "reach into" later parts of the string (e.g., into the b 's of $a^n b^n$ when p is chosen as n) are invalid.

Q6. For $L = \{a^n b^n : n \geq 0\}$, the standard pumping lemma proof pumps the string:

A. $b^p b^p$ B. $a^p b^p$ C. $a^{p+1} b^p$ D. $a^p b^{p+1}$

Answer: B. **Why:** Choosing $w = a^p b^p$ forces y to consist only of a 's (since $|xy| \leq p$ keeps it within the a -block), so pumping y (e.g., at $i=2$) unbalances the a -count vs. b -count, breaking the $a^n b^n$ pattern.

Q7. In the pumping lemma, the value p ("pumping length") represents:

A. The length of the specific string chosen B. A constant, guaranteed by the lemma, that depends on the language's DFA (roughly its number of states) C. Always equal to 1 D. The number of times y must be pumped

Answer: B. **Why:** p is tied to the structure of a hypothetical DFA for L (intuitively, related to the number of states, via pigeonhole on repeated states along any path of length $\geq p$) — it's a fixed constant for the language, not something chosen per-string.

Q8. Satisfying all pumping lemma conditions for a language L means:

A. L is definitely regular B. L might be regular — it is a necessary condition, not sufficient C. L is definitely not regular D. L must be finite

Answer: B. **Why:** The lemma gives a necessary property of all regular languages; some non-regular languages can coincidentally satisfy the pumping conditions too, so satisfying them alone never proves regularity.

Q9. [Trap] Why is $L = \{ww : w \in \{a,b\}^*\}$ not regular?

A. It contains only even-length strings, which no DFA can recognize B. Recognizing it requires comparing two arbitrarily long, unbounded halves of the string for exact

equality — impossible with the finite, fixed memory of any DFA C. It's actually regular; this is a trick option D. It contains the empty string, which DFAs cannot accept

Answer: B. Why:* A DFA has only finitely many states, so it cannot "remember" an unboundedly long first half to compare against the second half — the pumping lemma formalizes this intuition into a rigorous proof.

Q10. The negation of "for every $i \geq 0$, $xy^iz \in L$ " (used to derive the contradiction) means you must exhibit:

A. Some specific $i \geq 0$ for which $xy^iz \notin L$ B. Every possible i for which $xy^iz \notin L$ C. A string entirely unrelated to w D. A different pumping length p

Answer: A. Why: To break the "for all i " universal claim, it suffices to find just one counterexample value of i (commonly $i=0$, removing y , or $i=2$, duplicating it) where the pumped string leaves L .

3.4 Context-Free Grammars & Pushdown Automata

3.4.1 Context-Free Grammars (CFG)

- A CFG is a 4-tuple (V, Σ, R, S) : variables/non-terminals, terminals (alphabet), production rules, start symbol.
- Each rule has the form $A \rightarrow \alpha$ where A is a **single non-terminal** and α is any string of terminals/non-terminals — this "single non-terminal on the left" restriction is exactly what makes it "context-free" (the rule can be applied regardless of A 's surrounding context).
- Derivation**: repeatedly replacing a non-terminal using a rule, starting from S , until only terminals remain. **Leftmost derivation** always expands the leftmost non-terminal; **rightmost derivation** expands the rightmost.
- Parse tree**: a tree representation of a derivation; **the same parse tree can correspond to multiple different derivation orders** (leftmost vs. rightmost), but represents one underlying structural interpretation.
- Ambiguous grammar**: a grammar where **some string** has two or more distinct parse trees (not just two derivation orders for the same tree). Classic example: unparenthesized $E \rightarrow E + E \mid E * E \mid id$ is ambiguous because " $id + id * id$ " can be parsed with either $+$ or $*$ grouped first.
- TRAP**: **Ambiguity is a property of the grammar, not (necessarily) of the language** — many inherently ambiguous languages exist, but for most practical ambiguous grammars, an **equivalent unambiguous grammar** can be constructed for the same language (e.g., by encoding operator precedence/associativity directly into the grammar's structure).

3.4.2 Chomsky Normal Form (CNF) & Greibach Normal Form (GNF)

- CNF**: every production is either $A \rightarrow BC$ (two non-terminals) or $A \rightarrow a$ (single terminal) — plus $S \rightarrow \epsilon$ if ϵ is in the language (only for the start symbol, and only if S doesn't appear on the right side of any rule).
- GNF**: every production is $A \rightarrow a\alpha$ (a terminal followed by zero or more non-terminals) — useful for top-down parsing and proving PDA equivalence.
- Every CFG (with the ϵ -exception above) can be converted to an equivalent CNF/GNF grammar.
- TRAP**: A CNF derivation of a string of length n ($n \geq 1$) takes exactly $2n - 1$ derivation steps — a specific, testable numerical fact (each terminal-producing step is a single non-terminal $\rightarrow$ single terminal, each other step doubles a non-terminal into two).

3.4.3 Pushdown Automata (PDA)

- A PDA = an NFA + **an unbounded stack** for auxiliary memory — this stack is exactly what lets a PDA count/match unbounded nested structure (like balanced parentheses or $a^n b^n$) that a finite automaton cannot.
- Acceptance**: by **final state**, or by **empty stack** — these two acceptance modes are provably equivalent in power (any PDA of one type has an equivalent PDA of the other type).
- Deterministic PDA (DPDA)** vs. general (nondeterministic) PDA: unlike the DFA/NFA case, **DPDAs are strictly weaker** than general PDAs — some context-free languages (e.g., even-length palindromes $\{ww^R\}$) can only be recognized by a nondeterministic PDA, not any deterministic one.
- TRAP**: The DFA $\rightarrow$ NFA equivalence-in-power result does **NOT** carry over to PDAs — $DPDA \subsetneq PDA$ in expressive power, a frequently mis-generalized fact.

Questions

Q1. In a context-free grammar, every production rule must have on its left-hand side:

A. Any string of terminals and non-terminals B. Exactly one single non-terminal C. At least two non-terminals D. Only terminals

Answer: B. **Why:** This "single non-terminal, context-independent" restriction on the left-hand side is precisely what defines a grammar as context-free.

Q2. [Trap] A grammar is called ambiguous if:

A. It has more than one derivation order for some string B. Some string in the language has two or more distinct parse trees C. It contains recursive productions D. It cannot be converted to Chomsky Normal Form

Answer: B. **Why:** Multiple derivation *orders* (leftmost vs. rightmost) for the *same* parse tree is normal and doesn't imply ambiguity; true ambiguity requires genuinely different tree structures for the same string.

Q3. In Chomsky Normal Form, every production must be of the form:

A. $A \rightarrow BC$ or $A \rightarrow a$ B. $A \rightarrow a\alpha$ (terminal followed by non-terminals) C. $A \rightarrow \alpha\beta\gamma$ for any strings α, β, γ D. $A \rightarrow \epsilon$ only

Answer: A. **Why:** CNF restricts productions to exactly two non-terminals or exactly one terminal (with a limited exception allowing $S \rightarrow \epsilon$).

Q4. [Numerical] A CNF derivation of a terminal string of length n ($n \geq 1$) requires exactly how many derivation steps?

A. n B. $n - 1$ C. $2n - 1$ D. $2n$

Answer: C. **Why:** Each " $A \rightarrow BC$ " step effectively adds one non-terminal to the frontier and each " $A \rightarrow a$ " step converts one non-terminal to a terminal; the total works out to $2n - 1$ steps for a string of length n .

Q5. A Pushdown Automaton differs from an NFA by having:

A. Multiple start states B. An additional unbounded stack for auxiliary memory C. No accepting states D. A finite tape instead of an input string

Answer: B. **Why:** The stack is exactly what gives a PDA the extra memory needed to recognize context-free (but not regular) languages like matched parentheses or $a^n b^n$.

Q6. A PDA can accept an input by:

A. Final state only B. Empty stack only C. Either final state or empty stack — the two modes are provably equivalent in expressive power D. Neither; PDAs don't have an "acceptance" concept

Answer: C. **Why:** Both acceptance conditions define exactly the context-free languages; any PDA using one mode can be converted to an equivalent PDA using the other.

Q7. [Trap] Regarding deterministic vs. nondeterministic pushdown automata:

A. DPDAs and PDAs are exactly equal in expressive power, just like DFAs and NFAs B. DPDAs are strictly weaker than general PDAs — some context-free languages require nondeterminism C. DPDAs are strictly more powerful than general PDAs D. Neither DPDAs nor PDAs can recognize any context-free language

Answer: B. **Why:** Unlike the finite-automaton case, determinism genuinely restricts a PDA's power; languages like even-length palindromes $\{ww^R\}$ are context-free but not recognizable by any deterministic PDA.

Q8. The classic example of an inherently ambiguous-looking (with a naive grammar) expression grammar is:

A. $A \rightarrow aA \mid \epsilon$ B. $E \rightarrow E + E \mid E * E \mid id$ C. $S \rightarrow aSb \mid \epsilon$ D. $A \rightarrow AB \mid a$

Answer: B. **Why:** Without explicit precedence/associativity rules, " $id + id * id$ " can be parsed with either operator grouped first, giving two distinct parse trees — the textbook example of grammar ambiguity.

Q9. Greibach Normal Form (GNF) requires every production to begin with:

A. A non-terminal, followed by anything B. A terminal symbol, followed by zero or more non-terminals C. Exactly two terminals D. The start symbol S

Answer: B. **Why:** GNF's defining shape ($A \rightarrow a\alpha$, where α is zero or more non-terminals) is useful for top-down parsing constructions and PDA equivalence proofs.

Q10. [Trap] If a CFG is ambiguous, then in general:

A. The language it generates is always inherently ambiguous (no unambiguous grammar can exist for it) B. In most practical cases, an equivalent unambiguous grammar

generating the same language can be constructed, though some languages are genuinely inherently ambiguous C. It cannot be converted to CNF D. It is not actually context-free

Answer: B. Why: Ambiguity is fundamentally a grammar property; most familiar ambiguous grammars (like unparenthesized arithmetic expressions) have well-known unambiguous equivalents — though a smaller class of "inherently ambiguous" languages does exist where no unambiguous CFG can generate them.

3.5 Pumping Lemma for CFLs & Closure Properties

3.5.1 Pumping Lemma for Context-Free Languages

- If L is context-free, there exists a pumping length p such that every string $w \in L$ with $|w| \geq p$ can be split as $w = uvxyz$ where:
 - $|vxy| \leq p$
 - $|vy| > 0$ (at least one of v, y is non-empty — unlike the regular case, it's the *combined* pumped pieces that must be non-empty)
 - For every $i \geq 0$, $u v^i x y^i z \in L$ (v and y are pumped **together, the same number of times each**)
- TRAP:** In the CFL pumping lemma, **v and y must be pumped the same number of times simultaneously** — you cannot pump v alone while leaving y fixed (or vice versa); this "linked pumping" is the key structural difference from the regular pumping lemma (which pumps a single piece y).

3.5.2 Classic Non-Context-Free Languages

Language	Why not context-free (intuition)
$L = \{a^n b^n c^n : n \geq 0\}$	A PDA's single stack can enforce <i>one</i> equality (matching two counts) but not two simultaneous equalities across three counts
$L = \{ww : w \in \{a,b\}^*\}$	Requires comparing two <i>arbitrary, unbounded</i> copies for exact equality — a stack can reverse-match but not verify an unreversed duplicate
$L = \{a^i b^j c^k : i < j < k\}$	Requires comparing three unbounded counts with strict ordering

- TRAP:** $\{a^n b^n : n \geq 0\}$ IS context-free (matched by a simple PDA/CFG: $S \rightarrow aSb \mid \epsilon$) — the trap is over-generalizing "counting" fears to a language that is actually a textbook CFL; it's specifically the **third simultaneous count** ($a^n b^n c^n$) that breaks context-freeness, not counting/matching itself.

3.5.3 Closure Properties of Context-Free Languages

CFLs ARE closed under: **Union, Concatenation, Kleene star, Reversal, Homomorphism.**

CFLs are **NOT** closed (in general) under: **Intersection, Complementation, Difference.**

- **Exception:** the intersection of a context-free language with a **regular** language IS always context-free (a useful special case, provable via a product construction combining a PDA and a DFA).

- **TRAP:** "CFL $\cap$ CFL" is generally NOT context-free (e.g., $\{a^n b^n c^n\} \cap \{a^n b^n c^n\} = \{a^n b^n c^n\}$, non-CFL, from two CFLs) — but "CFL $\cap$ Regular" always IS context-free. Mixing up these two very similar-looking closure claims is a frequent exam trap.

Questions

Q1. [Trap] In the CFL pumping lemma's $w = uvxyz$ split, the pumped pieces v and y must satisfy:

A. Only v needs to be non-empty; y can always be empty B. v and y are pumped together, the same number of times, and at least one of them is non-empty C. Only y needs to be non-empty; v can always be empty D. Both v and y must always be non-empty simultaneously

Answer: B. Why: The lemma's condition $|vy| > 0$ requires at least one of the two pieces to be non-empty (not necessarily both), and both are pumped by the same count i in $u v^i x y^i z$.

Q2. [Trap] Which of these languages is actually context-free, despite involving "counting"?

A. $\{a^n b^n c^n : n \geq 0\}$ B. $\{a^n b^n : n \geq 0\}$ C. $\{ww : w \in \{a,b\}^*\}$ D. $\{a^i b^j c^k : i < j < k\}$

Answer: B. Why: $\{a^n b^n\}$ is matched by a simple single-stack PDA/CFG; it's specifically a third* simultaneous count (as in $a^n b^n c^n$) that exceeds a single stack's ability to enforce, not counting in general.

Q3. Context-free languages are closed under which of the following?

A. Intersection (of two arbitrary CFLs) B. Complementation C. Union, concatenation, and Kleene star D. Difference of two arbitrary CFLs

Answer: C. Why: Union, concatenation, and star are standard CFL closure properties; general intersection, complementation, and difference are not guaranteed to stay context-free.

Q4. [Trap] The intersection of a context-free language with a regular language is:

A. Never context-free B. Always context-free C. Only context-free if the regular language is finite D. Always regular

Answer: B. Why: This special-case closure (CFL $\cap$ Regular = CFL) holds via a product construction combining a PDA with a DFA — distinct from the general CFL $\cap$ CFL case, which is not guaranteed to be context-free.

Q5. [Numerical/conceptual] $\{a^n b^n c^m : n, m \geq 0\}$ intersected with $\{a^m b^n c^n : n, m \geq 0\}$ produces a language equal to:

A. $\{a^n b^n c^n : n \geq 0\}$, which is NOT context-free — demonstrating CFLs are not closed under general intersection B. The empty language C. $\{abc\}$, which is regular D. All strings over $\{a,b,c\}$

Answer: A. Why: This is the standard textbook proof that CFL $\cap$ CFL can escape context-freeness — both operands are individually context-free, but their intersection forces all three counts equal, which is provably not context-free.

Q6. Why is $\{a^i b^j c^k : i < j < k\}$ not context-free?

A. It's actually regular B. It requires simultaneously verifying a strict ordering across three unbounded counts, beyond what a single stack can track C. It contains only finite strings D. It's ambiguous, not non-context-free

Answer: B. Why: Like $a^n b^n c^n$, this requires coordinating relationships among three unbounded quantities — a single-stack PDA cannot maintain that much simultaneous unbounded state.

Q7. The pumping length p in the CFL pumping lemma is most closely tied to:

A. The length of the alphabet B. The size (number of variables) of an equivalent Chomsky Normal Form grammar for L C. The number of terminals only D. It has no real structural basis, purely arbitrary

Answer: B. Why: The CFL pumping lemma proof relies on parse-tree height and a pigeonhole argument over repeated non-terminals along a path in a CNF-form grammar's parse tree, tying p to the grammar's variable count.

Q8. [Trap] Just like with regular languages, satisfying the CFL pumping lemma's conditions for a language:

A. Proves the language is context-free B. Is only a necessary condition for context-freeness, not sufficient — it can only be used to disprove context-freeness by contradiction C. Proves the language is regular D. Has no logical use at all

Answer: B. Why: As with the regular pumping lemma, this is a necessary-but-not-sufficient property; its only valid use is proof-by-contradiction to show a language is NOT context-free.

Q9. CFLs are closed under reversal, meaning:

A. If L is context-free, the language of all reversed strings of L is also context-free B. Reversal always turns a CFL into a regular language C. Only regular languages support reversal D. Reversal is undefined for context-free languages

Answer: A. Why: Reversal is one of the standard closure properties for CFLs, provable by reversing the right-hand sides of all productions in an equivalent grammar.

Q10. A PDA fails to recognize $\{a^n b^n c^n\}$ fundamentally because:

A. It has too few states B. Its single stack can enforce at most one count-matching relationship at a time, insufficient for coordinating three counts simultaneously C. It cannot process the symbol 'c' at all D. The alphabet has too many symbols

Answer: B. Why: A stack naturally supports matching pairs (push for a's, pop for b's, say) but has no second independent memory structure to also verify the c-count matches — this single-stack limitation is the core reason $a^n b^n c^n$ escapes context-freeness.

3.6 Turing Machines & Decidability

3.6.1 Turing Machine (TM) Basics

- A TM = a finite-state control + an **infinite tape** (unbounded in at least one direction) + a **read/write head** that can move **Left or Right** — the tape is TM's key extra power over a PDA's stack: random re-reading/rewriting anywhere already visited, not just LIFO access.
- A TM computation either: **Accepts** (reaches an accept state), **Rejects** (reaches a reject state or gets stuck with no valid transition), or **Loops forever** (never halts) — this third possibility is what makes some problems fundamentally **undecidable**.
- Multi-tape, multi-head, non-deterministic TMs** are all provably **equivalent in power** to a standard single-tape deterministic TM (they can simulate each other, at some cost in time/space, but recognize exactly the same class of languages) — this equivalence is the basis of the **Church-Turing Thesis** (any "effectively computable" function can be computed by some TM).

3.6.2 Recursive vs. Recursively Enumerable Languages

Class	Definition	Key property
Recursive (Decidable)	A TM exists that halts on every input — accepting strings in L, rejecting strings not in L	Membership is always <i>decidable</i> in finite time
Recursively Enumerable / RE (Semi-decidable / Turing-recognizable)	A TM exists that accepts every string in L but may loop forever on strings not in L	Membership can be confirmed if true, but "no" answers may never be determined
Non-RE	No TM can even enumerate/accept exactly this language	E.g., the complement of the Halting Problem

- TRAP:** Every recursive language is also RE, but **not every RE language is recursive** — RE is the strictly larger class. A language whose **complement is also RE** is guaranteed to be recursive (this "L and L' both RE $\Rightarrow$ L recursive" fact is a classic exam point).

3.6.3 The Halting Problem

- Halting Problem:** "Given a TM M and input w, does M halt on w?" — **Proven undecidable** (Alan Turing, via diagonalization) — no TM/algorithm can correctly answer this for *all* possible (M, w) pairs.
- The Halting Problem is **RE** (semi-decidable): you can simulate M on w; if it halts, you'll know (say "yes"); but if it doesn't halt, your simulation runs forever, never producing "no."
- The **complement** of the Halting Problem is **not RE** — combined with the fact above (L and L' both RE $\Rightarrow$ recursive), this reconfirms the Halting Problem itself is not recursive (if it were, its complement would be RE too, but it isn't).
- TRAP:** "Undecidable" does **not** mean "no answer exists" or "unknowable in every specific instance" — for many *specific* (M, w) pairs you absolutely can determine the answer; it means **no single general algorithm works correctly for ALL possible inputs**.

3.6.4 Reduction & Other Undecidable Problems

- Reduction-based undecidability proofs:** to show problem B is undecidable, show that if B were decidable, you could use it to decide the (already known undecidable) Halting Problem — i.e., **reduce** the Halting Problem to B.
- Other classic undecidable problems: whether a given CFG is ambiguous, whether two CFGs generate the same language, whether a TM's language is empty, the **Post Correspondence Problem (PCP)**.
- Rice's Theorem:** any **non-trivial property of the language recognized by a TM** (a property true for some RE languages and false for others) is undecidable — this single theorem covers most "is this property of a TM's behavior decidable?" questions in one shot.

Questions

Q1. The key structural advantage a Turing Machine's tape has over a PDA's stack is:

A. It can only be read once, left to right B. It allows random access — the head can move both left and right, re-reading/rewriting previously visited cells, unlike a stack's strict LIFO access C. It has a fixed, bounded size D. It cannot be written to, only read

Answer: B. Why: This unrestricted read/write/move-both-directions capability is exactly what gives a TM more computational power than a PDA's single, LIFO-only stack.

Q2. [Trap] A recursively enumerable (RE) language is one for which a TM exists that:

A. Halts on every input, accepting or rejecting correctly B. Accepts every string in the language, but may loop forever on strings not in the language C. Rejects every string not in the language, in finite time, but need not accept strings that are in the language D. Never halts on any input

Answer: B. Why: RE (semi-decidable) means "yes" instances are always eventually confirmed, but "no" instances might never be confirmed (the TM could run forever) — this asymmetry is RE's defining feature, distinct from the stronger "recursive/decidable" guarantee.

Q3. The relationship between recursive and RE languages is:

A. They are exactly the same class B. Every recursive language is RE, but not every RE language is recursive C. Every RE language is recursive, but not vice versa D. The two classes are disjoint, with no overlap

Answer: B. Why: Recursive (decidable) is a strict subset of RE (semi-decidable) — a decidable language trivially has a TM that halts on everything (hence also qualifies as RE), but RE alone doesn't guarantee halting on rejecting inputs.

Q4. [Trap] If a language L and its complement L' are both recursively enumerable, then:

A. L must be undecidable B. L must be recursive (decidable) C. This situation is impossible and can never occur D. Only L is RE; L' cannot also be RE

Answer: B. Why: This is a key decidability theorem — being able to semi-decide both "yes" and "no" instances (by running both TMs in parallel and seeing which halts first) gives you a full decision procedure, hence recursive.

Q5. The Halting Problem is:

A. Decidable for all (M, w) pairs using a sufficiently clever algorithm B. Undecidable — no single algorithm correctly determines halting behavior for every possible (M, w) pair C. Not even recursively enumerable D. Only undecidable for non-deterministic Turing Machines

Answer: B. Why: Turing's classic diagonalization proof shows no general algorithm can correctly decide halting for all machine/input pairs — one of the foundational results of computability theory.

Q6. [Trap] "The Halting Problem is undecidable" means:

A. For every specific (M, w) pair, it is fundamentally impossible to ever determine whether M halts on w B. No single general-purpose algorithm exists that correctly answers the halting question for ALL possible (M, w) pairs — many individual instances remain perfectly determinable C. Turing Machines cannot halt at all D. The problem has no mathematical definition

Answer: B. Why: Undecidability is about the non-existence of a universal algorithm, not about every individual case being unknowable — many specific machines can obviously be shown to halt or not.

Q7. Rice's Theorem states that:

A. All properties of Turing Machines are decidable B. Any non-trivial property of the language a TM recognizes is undecidable C. Only properties related to the number of states are undecidable D. It applies only to deterministic TMs, not non-deterministic ones

Answer: B. Why: Rice's Theorem is a sweeping generalization covering essentially all "semantic" questions about what language a TM recognizes (as opposed to purely syntactic questions about the TM's description) — nearly all such questions are undecidable.

Q8. To prove a new problem B is undecidable via reduction, the standard technique is to show:

A. B can be solved using a known decidable algorithm B. If B were decidable, that decision procedure could be used to solve the (already known undecidable) Halting Problem — a contradiction C. B has infinitely many solutions D. B is a subset of a decidable language

Answer: B. Why: Reduction-based undecidability proofs work by contradiction: assuming a hypothetical decider for B, then showing it could be repurposed to decide something already proven undecidable.

Q9. According to the Church-Turing Thesis:

A. Every function is computable by some Turing Machine B. Any function that is "effectively/intuitively computable" by any reasonable computational model can be computed by some Turing Machine C. Only recursive languages are computable D. Non-deterministic TMs can solve problems deterministic TMs cannot

Answer: B. Why: This is a thesis (not a formally provable theorem, since "effectively computable" isn't itself a rigorous mathematical term) asserting TMs capture the full intuitive notion of computability — supported by the equivalence of many alternative computational models to standard TMs.

Q10. A Turing Machine computation on a given input can result in three possible outcomes: accept, reject, or:

A. Return an error code B. Loop forever without halting C. Automatically switch to a different machine D. Produce a probabilistic result

Answer: B. Why: This "may never halt" possibility is exactly what distinguishes a TM (and undecidable/RE-but-not-recursive problems) from simpler automata models that always eventually stop.

3.7 Chomsky Hierarchy & Complexity Classes (P, NP, NP-Complete)

3.7.1 Chomsky Hierarchy

Type	Grammar name	Recognizer	Restriction on productions
Type 0	Unrestricted / Recursively Enumerable	Turing Machine	No restrictions ($\alpha \rightarrow \beta$, any strings)
Type 1	Context-Sensitive	Linear Bounded Automaton (LBA)	$ \alpha \leq \beta $ (non-contracting; length cannot decrease)
Type 2	Context-Free	Pushdown Automaton (PDA)	$A \rightarrow \beta$, single non-terminal on left
Type 3	Regular	Finite Automaton (DFA/NFA)	$A \rightarrow aB$, $A \rightarrow a$, or $A \rightarrow Ba$ (linear, one non-terminal at one end)

- Each level is a **strict superset** of the level below: **Regular** $\subset$ **Context-Free** $\subset$ **Context-Sensitive** $\subset$ **Recursively Enumerable**.
- TRAP:** The hierarchy's numbering is **backwards from intuition** — Type 0 is the **most general/powerful** (fewest restrictions), Type 3 is the **most restricted/weakest** (regular) — a very common "higher number = more powerful" mix-up.
- Recursive languages** sit strictly between Context-Sensitive and Recursively Enumerable in the power ordering (every context-sensitive language is recursive; every recursive language is RE) but recursive languages are **not** one of the four official numbered "Chomsky types" — they're an additional named class overlaid on the hierarchy.

3.7.2 Complexity Classes

- P (Polynomial time):** problems solvable by a **deterministic** TM in time polynomial in input size — considered "efficiently solvable."
- NP (Nondeterministic Polynomial time):** problems where a **proposed solution/certificate can be VERIFIED** in polynomial time by a deterministic TM (equivalently: solvable in polynomial time by a nondeterministic TM). NP is about **fast verification**, not necessarily fast solving.
- $P \subseteq NP$** is proven (any problem solvable quickly can trivially have its solution verified quickly). Whether **$P = NP$** is the single most famous open problem in computer science — **still unsolved**.
- TRAP:** NP does **not** stand for "Not Polynomial" — a very common misreading. It stands for **Nondeterministic Polynomial**, and NP problems are NOT known to be unsolvable in polynomial time; they're simply not *proven* solvable in polynomial time (some might turn out to be, if $P=NP$ is ever proven).

3.7.3 NP-Complete & NP-Hard

- NP-Complete (NPC):** a problem that is (a) in NP, AND (b) **every** problem in NP can be **polynomial-time reduced** to it. NPC problems are the "hardest" problems within NP.
- NP-Hard:** satisfies condition (b) above (every NP problem reduces to it) but **not necessarily** condition (a) — an NP-Hard problem need not itself be in NP (it could be even harder, e.g., undecidable).
- If **any single** NP-Complete problem is ever shown solvable in polynomial time, then **$P = NP$** (since every NP problem reduces to it) — this is why NP-Complete problems are the focus of the P vs NP question.
- Cook-Levin Theorem:** SAT (Boolean satisfiability) was the **first problem proven NP-Complete**; other NP-Complete proofs typically reduce a known NPC problem (often SAT) to the new problem.
- Classic NP-Complete problems: SAT, 3-SAT, Vertex Cover, Hamiltonian Cycle, Traveling Salesman (decision version), Subset Sum, Graph Coloring, Clique.
- TRAP: NP-Hard is not automatically "in NP"** — e.g., the Halting Problem is NP-Hard (in a loose/general sense used for undecidable problems) but is **not** in NP (it's not even decidable, let alone verifiable in polynomial time). Every NP-Complete problem IS NP-Hard, but not every NP-Hard problem is NP-Complete.

Questions

Q1. [Trap] In the Chomsky hierarchy, which type is the MOST general/powerful (fewest restrictions on productions)?

A. Type 0 B. Type 1 C. Type 2 D. Type 3

Answer: A. Why: Type 0 (unrestricted/recursively enumerable) allows any production rule; the type number decreases in power as it increases, opposite to naive intuition — Type 3 (regular) is the most restricted.

Q2. A Type 1 (context-sensitive) grammar's productions must satisfy which restriction?

A. Left side must be a single non-terminal, no restriction on right side B. $|\alpha| \leq |\beta|$ — the right-hand side cannot be shorter than the left-hand side C. Every production must produce exactly one terminal D. No restriction whatsoever

Answer: B. Why: Context-sensitive grammars are "non-contracting" — string length can stay the same or grow, but never shrink, across a derivation step.

Q3. Which automaton model recognizes exactly the context-sensitive languages?

A. DFA B. PDA C. Linear Bounded Automaton (LBA) D. Standard unrestricted Turing Machine

Answer: C. Why: An LBA is a Turing Machine restricted to using only the tape space occupied by the input itself — exactly matching the power of Type 1 grammars.

Q4. [Trap] NP stands for:

A. "Not Polynomial," meaning these problems provably cannot be solved in polynomial time B. "Nondeterministic Polynomial" — problems whose proposed solutions can be verified in polynomial time C. "New Problem," a category with no formal definition D. "Non-Provable"

Answer: B. Why: NP problems are defined by fast *verifiability*, not proven unsolvability — whether they can also be *solved* quickly ($P=NP$) remains the most famous open question in computer science.

Q5. The relationship between P and NP that has been rigorously proven is:

A. $P = NP$ B. $P \subseteq NP$ C. P and NP are provably disjoint D. $NP \subseteq P$

Answer: B. Why: Any problem solvable in polynomial time trivially has its solution verifiable in polynomial time too (just re-solve it), giving $P \subseteq NP$; whether the reverse also holds (making them equal) is unsolved.

Q6. [Trap] An NP-Hard problem:

A. Must always also be a member of NP B. Need not be in NP at all — it only needs every NP problem to reduce to it in polynomial time C. Is, by definition, always solvable in polynomial time D. Is a strictly smaller class than NP-Complete

Answer: B. Why: NP-Hard only requires the "every NP problem reduces to this" property; it says nothing about whether the problem itself is even decidable/verifiable — NP-Complete is the stricter subset that additionally requires membership in NP.

Q7. The Cook-Levin Theorem's historical significance is that it:

A. Proved $P = NP$ B. Proved $P \neq NP$ C. Established SAT as the first problem proven NP-Complete, providing a base case for future reductions D. Proved all NP problems are solvable in polynomial time

Answer: C. Why: Cook-Levin gave the foundational NP-Completeness proof for SAT; nearly every subsequent NP-Completeness proof works by reducing SAT (or another already-proven NPC problem) to the new target problem.

Q8. If a polynomial-time algorithm were discovered for any single NP-Complete problem, the consequence would be:

A. Nothing; NP-Complete problems are independent of each other B. $P = NP$, since every NP problem can be reduced to that one NP-Complete problem in polynomial time C. That problem would no longer be in NP D. All NP-Hard problems would become undecidable

Answer: B. **Why:** Because every NP problem polynomial-time-reduces to any given NP-Complete problem, a fast algorithm for one NPC problem would give a fast algorithm for all of NP — collapsing P and NP.

Q9. Where do recursive (decidable) languages sit in relation to the four Chomsky types?

A. They are exactly Type 1 B. They are exactly Type 0 C. They sit strictly between context-sensitive (Type 1) and recursively enumerable (Type 0), but are not one of the four officially numbered Chomsky types themselves D. They don't relate to the Chomsky hierarchy at all

Answer: C. **Why:** Recursive languages form an additional class (every context-sensitive language is recursive, every recursive language is RE) layered onto the classical four-type hierarchy, rather than being one of the four types itself.

Q10. Which of the following is a classic NP-Complete problem?

A. Sorting a list of numbers B. Finding the shortest path between two nodes in a graph C. Boolean satisfiability (SAT) D. Binary search in a sorted array

Answer: C. **Why:** SAT is the canonical, historically first NP-Complete problem; sorting, single-source shortest path, and binary search all have known polynomial-time algorithms and are in P, not NP-Complete.

4.1 Phases of a Compiler & Lexical Analysis

4.1.1 Compiler Phases (in order)

1. **Lexical Analysis** — source code → **tokens** (via a scanner/lexer).
2. **Syntax Analysis (Parsing)** — tokens → **parse tree / syntax tree**, checking grammatical structure.
3. **Semantic Analysis** — type checking, scope resolution, checks meaning (e.g., "add int to string" flagged here, not earlier).
4. **Intermediate Code Generation** — syntax tree → machine-independent intermediate representation (e.g., three-address code).
5. **Code Optimization** — improves the intermediate code (speed/space) without changing its meaning.
6. **Code Generation** — intermediate code → target machine code.
 - **Symbol Table Management** and **Error Handling** run **throughout all phases**, not as a standalone phase in this sequence.
 - **TRAP**: Lexical analysis catches only errors in **token formation** (e.g., an illegal character); it does **not** catch grammar errors (that's syntax analysis) or "undeclared variable" errors (that's semantic analysis) — a frequently tested "which phase catches this error" distinction.

4.1.2 Analysis vs. Synthesis Phases

- **Analysis (Front End)**: Lexical + Syntax + Semantic + Intermediate code generation — breaks the source down and understands it; largely **machine-independent**.
- **Synthesis (Back End)**: Code optimization + Code generation — builds the target program; **machine-dependent**.

4.1.3 Lexical Analysis Details

- **Token**: a category (e.g., IDENTIFIER, KEYWORD, NUMBER, OPERATOR). **Lexeme**: the actual matched character sequence (e.g., "total"). **Pattern**: the rule (typically a regular expression) describing what lexemes belong to a token type.
- Lexical analyzers are implemented using **Finite Automata** (built from regular expressions describing each token pattern) — this is precisely why lexical analysis is a **regular-language-level** problem (Type 3 in the Chomsky hierarchy), while parsing needs the more powerful context-free level.
- **Maximal munch (longest match) rule**: when multiple prefixes of the input could match different valid tokens, the lexer always chooses the **longest** matching lexeme (e.g., ">=" is read as one operator, not ">" followed by "=").
- **TRAP**: Lexical analysis operates strictly on **regular languages/expressions**; it **cannot** by itself handle recursive/nested structures (like matching balanced parentheses) — that requires the context-free power of the parser.

Questions

Q1. The correct order of the first three compiler phases is:

- A. Syntax Analysis → Lexical Analysis → Semantic Analysis B. Lexical Analysis → Syntax Analysis → Semantic Analysis C. Semantic Analysis → Lexical Analysis → Syntax Analysis D. Lexical Analysis → Semantic Analysis → Syntax Analysis

Answer: B. Why: The lexer first tokenizes the raw source, the parser then checks grammatical structure of those tokens, and only then does semantic analysis check meaning (types, scope).

Q2. [Trap] An "undeclared variable used in an expression" error is typically caught during:

- A. Lexical analysis B. Syntax analysis C. Semantic analysis D. Code generation

Answer: C. Why: This requires understanding meaning/scope (a semantic property), not just token formation (lexical) or grammatical structure (syntax) — a classic "which phase" trap.

Q3. The Analysis (front-end) phases of a compiler are generally:

- A. Machine-dependent B. Machine-independent C. Only relevant for interpreted languages D. Skipped entirely for optimizing compilers

Answer: B. Why: Lexical, syntax, semantic analysis, and intermediate code generation understand and represent the source program's meaning independent of any specific target machine; machine-dependence enters only in the back-end (optimization tuned to a target, and code generation).

Q4. In lexical analysis terminology, the actual sequence of characters matched (e.g., "total") is called the:

- A. Token B. Lexeme C. Pattern D. Attribute

Answer: B. Why: "Token" is the abstract category (e.g., IDENTIFIER); "lexeme" is the concrete matched text; "pattern" is the rule (regex) describing the lexeme's shape.

Q5. Lexical analyzers are typically implemented using:

- A. Pushdown automata B. Turing machines C. Finite automata, built from regular expressions for each token type D. Context-sensitive grammars

Answer: C. Why: Since token patterns are regular languages, finite automata are the natural, efficient recognizer — this is also why lexical analysis alone cannot handle nested/recursive structures.

Q6. The "maximal munch" (longest match) rule in lexical analysis means:

- A. The shortest valid token match is always chosen B. When multiple token matches are possible at a point, the longest matching lexeme is chosen C. Only single-character tokens are allowed D. The lexer ignores whitespace-separated tokens

Answer: B. Why: This rule resolves ambiguity like reading ">=" as one token rather than ">" then "=" — always preferring the longest valid match starting at the current position.

Q7. [Trap] Which of these errors can lexical analysis NOT catch on its own?

- A. An illegal character in the source (e.g., "@" where not allowed) B. A malformed numeric literal C. A missing closing parenthesis that breaks the nesting structure of an expression D. An unterminated string literal

Answer: C. Why: Balanced/nested constructs (like matching parentheses) require context-free grammar power (the parser), since regular languages/finite automata (which lexical analysis uses) cannot count/match unbounded nesting.

Q8. Symbol table management, in the standard compiler phase model, is best described as:

- A. A single standalone phase that runs strictly after code generation B. An activity that runs throughout all phases, recording and looking up identifier information as needed C. Only relevant to the lexical analysis phase D. Not needed in modern compilers

Answer: B. Why: Every phase — from lexical analysis (recording newly seen identifiers) through code generation (looking up storage locations) — reads or updates the symbol table.

Q9. The back-end (synthesis) phases of a compiler consist of:

- A. Lexical and syntax analysis B. Semantic analysis and intermediate code generation C. Code optimization and code generation D. Only code generation, nothing else

Answer: C. Why: These are the phases that "build" the target-specific output, in contrast to the machine-independent front-end analysis phases.

Q10. A regular expression for identifiers (letter followed by letters/digits) is used by the lexer because:

- A. Identifiers require unbounded nested structure to define B. Identifier patterns are simple enough to be fully described within the regular language class, matching what finite automata can recognize efficiently C. Regular expressions are only used for syntax analysis, not lexical analysis D. Identifiers cannot be tokenized using automata at all

Answer: B. Why: Token patterns like identifiers, numbers, and operators are regular languages by design — this keeps lexical analysis fast (linear-time finite automaton simulation) and cleanly separated from the heavier context-free parsing stage.

4.2 Top-Down Parsing — LL(1) & Recursive Descent

4.2.1 Top-Down Parsing Basics

- Builds the parse tree from the **root (start symbol) downward**, trying to match the input by predicting which production to expand next.
- Recursive Descent Parser**: a set of mutually recursive procedures, one per non-terminal — simple to hand-write, but requires the grammar to have **no left recursion** (a left-recursive procedure would call itself immediately, infinite-looping without consuming input).
- Backtracking parsers** (general top-down, trying alternatives and backtracking on failure) are possible but **inefficient** (exponential worst case) — practical parsers avoid backtracking via lookahead (LL parsing).

4.2.2 Left Recursion & Left Factoring

- Left recursion**: $A \rightarrow \alpha A \mid \beta$. Must be **eliminated** before top-down parsing: rewrite as $A \rightarrow \beta A', A' \rightarrow \alpha A' \mid \epsilon$.
- TRAP**: Eliminating left recursion changes a **left-recursive** rule into a **right-recursive** one — the language generated is unchanged, but this transformation is mandatory for LL parsing (top-down parsers cannot handle left recursion directly; it would cause infinite recursion with no input consumed).
- Left factoring**: when two productions for the same non-terminal share a common prefix ($A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$), factor out the common part so the parser can decide which alternative to take using only 1 symbol of lookahead: $A \rightarrow \alpha A', A' \rightarrow \beta_1 \mid \beta_2$.

4.2.3 LL(1) Parsing

- "LL(1)" = Left-to-right scan of input, Leftmost derivation, 1 symbol of lookahead.
- A grammar is LL(1) if, for every non-terminal, its alternative productions' **FIRST sets are disjoint** (no overlap) — this guarantees the parser can always choose the correct production using just the next input symbol.
- FIRST(α)**: the set of terminals that can begin any string derived from α (includes ϵ if α can derive ϵ).
- FOLLOW(A)**: the set of terminals that can immediately follow A in some derivation from the start symbol (includes $\$$ — end-of-input marker — if A can be the last symbol of a derivation).
- FOLLOW is used when a production can derive ϵ** — if $A \rightarrow \epsilon$ is a choice, the parser uses FOLLOW(A) to decide when to apply it.
- TRAP**: A grammar that is **left-recursive** or **ambiguous** can **never** be LL(1) — both properties directly violate the disjoint-FIRST-sets requirement (left recursion in particular breaks parsing before FIRST/FOLLOW analysis even helps).
- LL(1) Parsing Table**: built from FIRST/FOLLOW sets; a **conflict** (more than one production in a single table cell) means the grammar is **not LL(1)**.

Questions

Q1. [Trap] A recursive descent parser cannot directly handle a grammar that is:

A. Right-recursive B. Left-recursive — it would recurse infinitely without consuming input C. Ambiguous only in rare edge cases D. Written with more than 10 productions

Answer: B. **Why:** A left-recursive procedure ($A \rightarrow \alpha A$) calls itself as the very first step, before consuming any input token, causing infinite recursion.

Q2. The standard transformation to eliminate direct left recursion ($A \rightarrow \alpha A \mid \beta$) produces:

A. $A \rightarrow \beta A', A' \rightarrow \alpha A' \mid \epsilon$ B. $A \rightarrow \alpha A', A' \rightarrow \beta A' \mid \epsilon$ C. $A \rightarrow \alpha\beta \mid \alpha$ D. It cannot be eliminated; the grammar must be rejected

Answer: A. **Why:** This standard rewrite preserves the generated language while converting left recursion into right recursion, which recursive-descent/LL parsers can handle safely.

Q3. Left factoring is applied when:

A. A grammar has left recursion B. Two or more productions for the same non-terminal share a common prefix, causing ambiguity in the parser's choice C. The grammar has no terminals D. FOLLOW sets overlap

Answer: B. **Why:** A shared prefix means one token of lookahead isn't enough to choose between alternatives; factoring the common part out defers the decision until enough input has been seen.

Q4. In "LL(1)," the two L's stand for:

A. Left-to-right scan of input, and Leftmost derivation B. Left recursion, and Left factoring C. Lower, and Lexical D. Left-to-right scan, and Lookahead

Answer: A. **Why:** LL(1) = Left-to-right input scan + Leftmost derivation + 1 symbol of lookahead — this exact naming breakdown is a common direct-recall question.

Q5. [Trap] A grammar is guaranteed to NOT be LL(1) if it is:

A. Right-recursive B. Left-recursive, or ambiguous C. Written entirely in Chomsky Normal Form D. Missing a start symbol

Answer: B. **Why:** Both left recursion and ambiguity inherently violate the disjoint-FIRST-sets condition required for LL(1) parsing — these grammars must be transformed (or are fundamentally unparseable by LL(1)) before top-down parsing can work.

Q6. FOLLOW(A) is used by the LL(1) parser construction specifically when:

A. A never appears on the right side of any production B. A production $A \rightarrow \epsilon$ exists, to decide when applying it is appropriate based on the next input symbol C. A is the start symbol only D. FOLLOW sets are never actually used in LL(1) table construction

Answer: B. **Why:** Since $A \rightarrow \epsilon$ doesn't consume any terminal to help decide via FIRST, the parser instead checks whether the current lookahead symbol is in FOLLOW(A) to justify choosing the ϵ -production.

Q7. A conflict in an LL(1) parsing table (more than one production entry in a single cell) indicates:

A. A syntax error in the input string being parsed B. The grammar is not LL(1) C. The grammar has no start symbol D. The lexer has failed

Answer: B. **Why:** Table conflicts arise precisely when FIRST (and/or FOLLOW, for ϵ -productions) sets overlap between alternative productions — meaning 1-symbol lookahead is insufficient to choose deterministically.

Q8. FIRST(α) is defined as:

A. The set of terminals that can immediately follow α in the input B. The set of terminals that can begin some string derivable from α C. Only the very first character of the grammar's alphabet D. The set of all non-terminals in the grammar

Answer: B. **Why:** FIRST captures every possible terminal that could start a derivation from α , which is exactly what a 1-token-lookahead parser needs to decide which production to try.

Q9. Backtracking-based general top-down parsers are avoided in practice mainly because:

A. They cannot parse context-free grammars at all B. They can have exponential worst-case time complexity trying different alternatives C. They require no lookahead whatsoever D. They only work for regular languages

Answer: B. **Why:** Trying each alternative production and backtracking on failure can, in the worst case, explore an exponential number of paths — LL(k) parsing avoids this by using bounded lookahead to choose deterministically.

Q10. [Numerical/conceptual] If two alternative productions for a non-terminal A both have overlapping FIRST sets and neither derives ϵ , the grammar:

A. Is still LL(1), since FOLLOW sets can always resolve it B. Is not LL(1) — the overlap alone, regardless of FOLLOW, disqualifies it C. Becomes LL(1) automatically after left factoring is applied elsewhere D. Must be left-recursive

Answer: B. Why: FOLLOW sets only come into play to resolve an ϵ -production's placement; if FIRST sets of two non- ϵ alternatives already overlap, 1-symbol lookahead is fundamentally insufficient regardless of FOLLOW.

4.3 Bottom-Up Parsing — LR(o), SLR, LALR, CLR

4.3.1 Bottom-Up Parsing Basics

- Builds the parse tree from the **leaves upward** to the root, via **Shift-Reduce** parsing: **Shift** (push next input symbol onto a stack) and **Reduce** (replace a string matching a production's right-hand side, on top of the stack, with its left-hand-side non-terminal).
- Bottom-up parsing traces a **rightmost derivation in reverse** — a frequently tested contrast with top-down's leftmost derivation.
- Handle**: a substring matching a production's RHS, at a position where reducing it is a correct step toward eventually deriving the start symbol.

4.3.2 LR Parser Family — Power Ordering

LR(0) $\subset$ SLR(1) $\subset$ LALR(1) $\subset$ CLR(1)/LR(1) (each is strictly more powerful — can parse a strictly larger class of grammars — than the previous, in this order).

| Parser | Lookahead used for reduce decisions | Table size |

| LR(0) | None — decides purely from the stack's state | Smallest |

| SLR(1) | FOLLOW set of the non-terminal being reduced to | Small |

| LALR(1) | Merges states with identical "core" items from CLR(1), using more precise lookaheads per item | Medium (same number of states as SLR, but more accurate) |

| CLR(1) / Canonical LR(1) | Full, item-specific lookahead computed per state | Largest (most states) |

- TRAP**: LALR(1) has the **same number of states** as SLR(1) (it merges CLR(1) states with identical cores) but is **more powerful** than SLR(1) — a common wrong assumption is that "fewer/same states = less powerful," but LALR's per-item lookahead precision (inherited from before merging) gives it SLR-beating accuracy at SLR-level table size. This is exactly why **most real-world parser generators (like YACC/Bison) default to LALR(1)** — near-CLR(1) power at much smaller table size than full CLR(1).

4.3.3 Conflicts in Shift-Reduce Parsing

- Shift-Reduce Conflict**: the parser cannot decide whether to shift the next symbol or reduce by some rule (classic example: the **dangling else** problem in if-then-else grammars).
- Reduce-Reduce Conflict**: the parser cannot decide **which** of two or more possible productions to reduce by.
- A grammar with any such unresolved conflict is **not** a valid grammar for that specific LR parser type (though a stronger parser type, e.g. CLR(1) instead of SLR(1), may resolve it).
- TRAP**: Not all shift-reduce conflicts indicate a genuinely ambiguous grammar — some (like dangling-else) are resolved by a **fixed convention** (e.g., "always prefer shift," which correctly binds an else to the nearest unmatched if) built into most parser generators by default.

Questions

Q1. Bottom-up parsing constructs a derivation that corresponds to:

- A. A leftmost derivation, read forward B. A rightmost derivation, read in reverse (from the end of the derivation back to the start symbol) C. Neither leftmost nor rightmost; it is derivation-order-independent D. A leftmost derivation, read in reverse

Answer: B. Why: Bottom-up (shift-reduce) parsers effectively reconstruct a rightmost derivation but in reverse order, reducing handles from the input back up to the start symbol.

Q2. In shift-reduce parsing, a "reduce" action means:

- A. Pushing the next input symbol onto the stack B. Replacing a handle (a string matching a production's RHS) on top of the stack with the corresponding non-terminal C. Discarding the entire stack and restarting D. Backtracking to a previous parser state

Answer: B. Why: Reduction is the core bottom-up construction step — recognizing that the top of the stack matches some production's right side and collapsing it to the left side non-terminal.

Q3. [Trap] Regarding parser power, which ordering is correct?

- A. $LR(0) \subset SLR(1) \subset LALR(1) \subset CLR(1)$ B. $CLR(1) \subset LALR(1) \subset SLR(1) \subset LR(0)$ C. All four are exactly equal in parsing power D. $SLR(1) \subset LR(0) \subset CLR(1) \subset LALR(1)$

Answer: A. Why: Each successive parser type in this order can handle a strictly larger class of grammars, with CLR(1)/canonical LR(1) being the most powerful of the four.

Q4. [Trap] Compared to SLR(1), an LALR(1) parser has:

- A. Significantly more states, but is less powerful B. The same number of states, but is more powerful (can parse a larger class of grammars) due to more precise per-item lookaheads C. Fewer states and is less powerful D. Exactly the same power and same number of states — they are identical

Answer: B. Why: LALR(1) is built by merging CLR(1) states with identical cores, which keeps state count equal to SLR(1)'s, but its lookaheads (inherited before merging) are precise enough to resolve some conflicts SLR(1) cannot — a classic "table size doesn't determine power" trap.

Q5. The "dangling else" ambiguity in if-then-else grammars is a classic example of a:

- A. Reduce-reduce conflict B. Shift-reduce conflict C. Lexical error D. Left-recursion problem

Answer: B. Why: At the point of encountering "else," the parser must choose between shifting it (attaching to the nearest if) or reducing the current if-then construct first — this exact ambiguity is the textbook shift-reduce conflict example.

Q6. A reduce-reduce conflict occurs when:

- A. The parser must choose between shifting and reducing B. Two or more different productions could both validly apply for a reduction at the same parser state C. The input string has a lexical error D. The grammar contains no non-terminals

Answer: B. Why: This is a distinct conflict type from shift-reduce — here the parser has already committed to reducing, but doesn't know which of multiple matching productions to use.

Q7. [Trap] The standard default resolution most parser generators apply to the dangling-else shift-reduce conflict is:

- A. Always reduce B. Always shift, which correctly binds each "else" to its nearest unmatched "if" C. Reject the input as a syntax error D. Randomly choose shift or reduce

Answer: B. Why: "Prefer shift" is the conventional fixed resolution, correctly implementing the natural rule that else binds to the closest unresolved if — this is why the conflict, though present, doesn't need to be treated as a grammar rejection.

Q8. Which LR parser variant uses NO lookahead at all to decide on a reduce action?

- A. CLR(1) B. LALR(1) C. SLR(1) D. LR(0)

Answer: D. Why: LR(0) parsers decide purely from the current stack/state, without consulting any lookahead symbol — the weakest (and simplest/smallest-table) member of the LR family.

Q9. SLR(1) parsers use which set to help decide when to reduce by a given production?

- A. FIRST set of the production's right-hand side B. FOLLOW set of the non-terminal being reduced to C. The entire grammar's terminal set, unconditionally D. No set at all; it behaves identically to LR(0)

Answer: B. Why: SLR(1) upgrades LR(0) by consulting FOLLOW(A) before reducing by $A \rightarrow \alpha$ — this is exactly the "Simple" refinement that distinguishes SLR(1) from plain LR(0).

Q10. Why do most real-world parser generator tools (e.g., YACC/Bison) default to LALR(1) rather than full CLR(1)?

A. LALR(1) is strictly more powerful than CLR(1) B. LALR(1) achieves nearly the same power as CLR(1) with a much smaller (SLR(1)-sized) parsing table, a practical engineering trade-off C. CLR(1) cannot handle any real programming language grammar D. LALR(1) requires no lookahead at all, making it fastest

Answer: B. Why: The huge table-size savings (same state count as SLR(1)) for only a small loss in parsing power compared to canonical LR(1) makes LALR(1) the practical sweet spot most tools adopt.

4.4 Syntax-Directed Translation & Intermediate Code

4.4.1 Syntax-Directed Definitions (SDD) & Attributes

- **Synthesized attribute:** computed at a node from its **children's** attribute values (flows **bottom-up** the parse tree).
- **Inherited attribute:** computed at a node from its **parent's and/or siblings'** attribute values (flows **top-down / left-to-right** across the tree).
- **S-attributed grammar:** uses **only synthesized** attributes — evaluable via simple bottom-up (LR) parsing, attaching attribute computation to each reduction.
- **L-attributed grammar:** allows inherited attributes too, but restricted so each attribute only depends on attributes of nodes **to its left** (parent, or earlier siblings) — evaluable via depth-first, left-to-right traversal (compatible with both top-down and bottom-up parsing, unlike a fully general SDD).
- **TRAP:** Every S-attributed grammar is also L-attributed (synthesized-only is a special, more restrictive case), but **not every L-attributed grammar is S-attributed** — a very common "which is the more general category" mix-up.

4.4.2 Intermediate Code Representations

Form	Description
Three-Address Code (TAC)	Each statement has at most one operator on the right + at most 3 addresses/operands total (e.g., $t1 = b + c$) — makes optimization and code generation easier by exposing every intermediate computation explicitly
Quadruples	TAC represented as 4 fields: (operator, arg1, arg2, result)
Triples	3 fields: (operator, arg1, arg2) — results are referenced by the position/index of the triple itself, not stored explicitly; makes code reordering harder (positions shift)
Postfix (Reverse Polish) Notation	Operators follow their operands, no parentheses needed
Abstract Syntax Tree (AST)	Compact tree omitting redundant nodes present in a full parse tree (e.g., no separate nodes for grammar-only symbols)

- **TRAP: Triples are harder to rearrange/optimize than quadruples** — because triples reference results by their statement position, moving/reordering a triple silently breaks every reference to it; quadruples' explicit named results avoid this problem, making them more optimization-friendly (at the cost of slightly more storage).

4.4.3 Three-Address Code Generation Examples

- $a = b + c * d \rightarrow t1 = c * d; t2 = b + t1; a = t2$ (respecting operator precedence: multiplication computed first).
- **Backpatching:** technique for generating code for boolean expressions/control statements (if, while) in a **single pass**, where jump target addresses aren't known yet at generation time — the target is left blank and filled in ("patched") later once the target instruction's address becomes known.
- **TRAP:** Backpatching exists specifically to avoid needing **multiple passes** over the syntax tree just to resolve forward jump targets — a common wrong belief is that it's only relevant for optimization, when it is actually a single-pass **code-generation** technique.

Questions

Q1. A synthesized attribute at a parse tree node is computed using:

A. Only its parent's attribute values B. Only its children's attribute values C. Only global variables D. Values from unrelated subtrees

Answer: B. Why: Synthesized attributes flow information upward from children to parent — the defining "bottom-up" attribute-passing direction.

Q2. [Trap] Which statement correctly relates S-attributed and L-attributed grammars?

A. They are exactly the same class of grammars B. Every S-attributed grammar is also L-attributed, but not every L-attributed grammar is S-attributed C. Every L-attributed grammar is S-attributed, but not vice versa D. Neither class can use synthesized attributes

Answer: B. Why: S-attributed (synthesized-only) is a strict subset of L-attributed (which also allows left-dependent inherited attributes) — S-attributed grammars trivially satisfy L-attribution's "depends only on the left" rule since they don't use inherited attributes at all.

Q3. An inherited attribute's value at a node can depend on:

A. Only that node's own children B. That node's parent and/or its left siblings C. Nothing; inherited attributes are always constants D. Only nodes to its right in the tree

Answer: B. Why: Inherited attributes flow information downward/sideways (parent → child, or left sibling → right sibling), the opposite direction from synthesized attributes.

Q4. [Trap] Compared to quadruples, triples for intermediate code representation are:

A. Easier to rearrange during optimization B. Harder to rearrange, since results are referenced implicitly by statement position rather than by an explicit named result C. Identical in every respect; the terms are interchangeable D. Only used for lexical analysis, not code generation

Answer: B. Why: A triple's "result" is just "the value produced by the triple at position i" — reordering statements shifts positions and breaks every such reference, unlike quadruples' explicit result field.

Q5. Three-address code restricts each statement to have:

A. Exactly three operators B. At most one operator on the right-hand side, with at most three total addresses/operands C. No operators at all, only assignments D. Exactly three variables, never constants

Answer: B. Why: TAC's defining simplicity is one operation per statement (e.g., $t1 = b + c$) — this granularity is exactly what later optimization and code-generation phases exploit.

Q6. Backpatching is a technique used to:

A. Fix bugs found after compilation, in the source code B. Generate code for control structures (if/while) in a single pass, by leaving jump targets blank and filling them in once known C. Optimize register allocation only D. Convert quadruples into triples

Answer: B. Why: Backpatching avoids needing to know a forward jump's target address at the moment that jump instruction is generated — it defers filling in ("patches") the address until it becomes available later in the same pass.

Q7. Given $a = b + c * d$, correct three-address code generation (respecting standard operator precedence) produces:

A. $t1 = b + c; t2 = t1 * d; a = t2$ B. $t1 = c * d; t2 = b + t1; a = t2$ C. $a = b + c * d$ directly, with no temporaries D. $t1 = b * c; t2 = t1 + d; a = t2$

Answer: B. Why: Multiplication has higher precedence than addition, so $c*d$ must be computed first into a temporary, then added to b .

Q8. An Abstract Syntax Tree (AST) differs from a full parse tree in that it:

A. Contains more nodes for clarity B. Omits nodes/details that are only artifacts of the grammar (not meaningful structure), giving a more compact representation C. Cannot represent expressions with operators D. Is only used in lexical analysis

Answer: B. Why: An AST strips away grammar-specific scaffolding (like nodes for redundant non-terminals used purely to enforce precedence) while preserving the program's essential structure.

Q9. [Trap] The main motivation for using backpatching, rather than making a second pass over the syntax tree once all labels are known, is:

A. To reduce memory usage during lexical analysis B. To allow single-pass code generation, which is more efficient than requiring a separate resolution pass C. It has no real benefit; it's purely a stylistic choice D. To enable dynamic typing

Answer: B. Why: Backpatching specifically targets the efficiency/simplicity gain of generating code in one pass, rather than needing to revisit the entire intermediate representation again just to fill in jump targets.

Q10. Postfix (Reverse Polish) notation places operators:

A. Before both operands B. Between the two operands, requiring parentheses for precedence C. After their operands, eliminating the need for parentheses to express precedence unambiguously D. Only at the very end of the entire expression, once

Answer: C. Why: In postfix notation (e.g., "b c d * +" for $b + c * d$), operand order combined with operator placement after them fully encodes evaluation order without ambiguity, so no parentheses are ever needed.

4.5 Code Optimization

4.5.1 Goals & Scope Levels

- Optimization must **preserve program meaning/output** exactly — never change what the program computes, only how efficiently.
- **Local optimization**: within a single **basic block** (straight-line code, no jumps in or out except at the very start/end).
- **Global optimization**: across basic blocks within a function/procedure (needs control-flow graph analysis: which blocks can precede/follow which).
- **Machine-independent** optimizations (apply to intermediate code, portable across targets) vs. **machine-dependent** optimizations (register allocation, instruction selection tuned to a specific target architecture).

4.5.2 Common Local/Machine-Independent Optimizations

Technique	What it does
Constant Folding	Evaluates constant expressions at compile time (e.g., $x = 2 + 3 \rightarrow x = 5$)
Constant Propagation	Replaces a variable known to hold a constant value with that literal value, wherever used
Common Subexpression Elimination (CSE)	Computes a repeated identical expression once , reuses the stored result instead of recomputing
Dead Code Elimination	Removes code whose result is never used (e.g., an assignment to a variable never read afterward)
Copy Propagation	Replaces <code>y = x; ... use y ...</code> with direct uses of <code>x</code> , potentially exposing more dead code afterward
Strength Reduction	Replaces an expensive operation with a cheaper equivalent (e.g., $x * 2 \rightarrow x << 1$, or replacing repeated multiplication in a loop with repeated addition)
Loop-Invariant Code Motion	Moves a computation that doesn't change across loop iterations outside the loop, so it executes once instead of every iteration
Induction Variable Elimination	Removes/simplifies loop variables that just track a linear function of the loop counter

- **TRAP**: **Common Subexpression Elimination** and **Constant Propagation** are often confused with each other — CSE reuses a **repeated computed expression's result**; constant propagation substitutes a **known compile-time literal value** for a variable — these solve different problems and are usually applied together, not interchangeably.

4.5.3 Basic Blocks & Control Flow Graphs (CFG)

- **Basic Block**: a maximal sequence of instructions with **one entry point (first instruction)** and **one exit point (last instruction)** — control enters only at the top and leaves only at the bottom, no branches into the middle and no branches out except at the end.
- **Leader instructions** (identify the start of each basic block): the first instruction of the program; any instruction that is the **target** of a jump/branch; any instruction **immediately following** a jump/branch/conditional instruction.
- **Control Flow Graph (CFG)**: nodes = basic blocks, edges = possible control transfers between them — the foundation for all global (inter-block) optimizations like loop-invariant code motion.
- **TRAP**: A basic block can never contain a jump/branch instruction **except as its very last instruction** — a common mistake is including a conditional jump in the middle of a proposed "basic block," which by definition splits it into two separate blocks.

4.5.4 Peephole Optimization

- A **local, machine-dependent** technique that examines a small, sliding "peephole" (window) of a few consecutive instructions in the final generated code, replacing inefficient patterns with better equivalents.
- Examples: eliminating redundant loads/stores (`store x; load x` → `store x`, keep value in register), eliminating jumps-to-jumps, simplifying algebraic identities in generated code (`x = x + 0` removed), replacing multiply-by-power-of-2 with a shift.
- **TRAP**: Peephole optimization operates on the **final generated (target) code**, not on the intermediate representation — a common confusion with the machine-independent optimizations listed above, which act earlier, on three-address code/intermediate representation.

Questions

Q1. Constant folding refers to:

- A. Replacing a variable with its known runtime value B. Evaluating expressions involving only constants at compile time, rather than at runtime C. Removing unused variables D. Merging two basic blocks into one

Answer: B. Why: $x = 2+3$ becomes $x = 5$ directly during compilation — no runtime computation needed for that expression at all.

Q2. [Trap] The key difference between Common Subexpression Elimination and Constant Propagation is:

- A. They are exactly the same technique with different names B. CSE reuses the result of a repeated computed expression; constant propagation substitutes a known compile-time constant value for a variable C. CSE only works on loops; constant propagation only works on basic blocks D. Constant propagation only applies to floating-point values

Answer: B. Why: These solve distinct problems — recomputation avoidance (CSE) vs. compile-time constant substitution (constant propagation) — though both are standard optimizations often applied in the same optimization pass sequence.

Q3. A basic block is defined as a code sequence with:

- A. Multiple entry points and multiple exit points B. Exactly one entry point (first instruction) and one exit point (last instruction), with no branches into the middle C. At least one loop inside it D. No instructions at all, only labels

Answer: B. Why: This "single entry, single exit" property is the defining rule for basic blocks — control can only enter at the top and leave at the bottom.

Q4. [Trap] Which of these correctly identifies a "leader" instruction, marking the start of a new basic block?

- A. Only the very first instruction of the program B. The first instruction of the program; any jump/branch target; any instruction immediately following a jump or conditional branch C. Only instructions containing arithmetic operators D. Only instructions that are jump targets, nothing else

Answer: B. Why: All three conditions independently create a new basic block boundary — missing any one of them (e.g., forgetting "instruction right after a branch" is also a leader) leads to incorrectly-formed basic blocks.

Q5. Loop-invariant code motion optimizes a loop by:

- A. Unrolling the loop body multiple times B. Moving a computation whose value doesn't change across iterations to outside the loop, so it runs once instead of every

iteration C. Reversing the loop's iteration order D. Replacing the loop with recursion

Answer: B. Why: If an expression's operands never change within the loop, recomputing it every iteration is wasted work — hoisting it before the loop preserves correctness while saving repeated computation.

Q6. Strength reduction is best exemplified by:

A. Replacing $x * 2$ with $x \ll 1$ (a bit shift) B. Removing an assignment whose result is never used C. Combining two basic blocks into one D. Replacing a variable with a constant

Answer: A. Why: Strength reduction swaps a costlier operation (multiplication) for a functionally equivalent but cheaper one (a shift), a classic instance of this optimization category.

Q7. [Trap] Peephole optimization is applied to:

A. The abstract syntax tree, before intermediate code generation B. The three-address intermediate code only C. The final generated target machine code, examining a small sliding window of instructions D. The source code text directly, before lexical analysis

Answer: C. Why: Peephole optimization is specifically a late-stage, machine-dependent pass over the actual generated instructions — distinct from the earlier machine-independent optimizations on intermediate code.

Q8. Dead code elimination removes:

A. Code that contains syntax errors B. Code whose computed result is never subsequently used C. All comments in the source program D. Loop bodies that execute zero times at runtime, detected dynamically

Answer: B. Why: If an assignment's value is never read afterward (and has no side effects), it contributes nothing to the program's observable output and can be safely removed.

Q9. A Control Flow Graph's nodes and edges represent:

A. Nodes = variables, edges = their dependencies B. Nodes = basic blocks, edges = possible control transfers between them C. Nodes = individual instructions only, never grouped D. Nodes = functions, edges = call relationships only

Answer: B. Why: The CFG is built at the basic-block granularity specifically to enable global (inter-block) data-flow analysis needed for optimizations like loop-invariant code motion.

Q10. [Trap] Copy propagation (replacing $y = x; \dots \text{use } y \dots$ with direct uses of x) is valuable mainly because:

A. It directly improves runtime speed on its own, with no further benefit B. It can expose additional dead code — once y is no longer used anywhere, its original assignment becomes eligible for dead code elimination C. It eliminates the need for constant folding D. It only works inside loops

Answer: B. Why: Optimizations are often applied iteratively/in combination — copy propagation frequently doesn't help by itself, but it enables a subsequent dead-code-elimination pass to remove the now-unused original assignment.

4.6 Symbol Tables, Runtime Environment & Code Generation

4.6.1 Symbol Table

- A data structure storing, for each identifier: name, type, scope, storage location/offset, and other attributes (e.g., whether it's a function and its parameter list).
- Typically implemented as a **hash table** for fast average-case lookup, often organized as **nested scopes** (one table per block/function, linked to its enclosing scope) to correctly resolve which declaration of a name applies where.
- **TRAP:** A single flat symbol table cannot correctly handle **nested scopes with shadowing** (an inner block redeclaring an outer variable name) — a scoped/nested table structure (or scope-stack) is required so each lookup finds the *innermost* applicable declaration.

4.6.2 Activation Records (Stack Frames)

Each procedure call creates an **activation record** on the runtime stack, typically containing:

- **Return address** (where to resume the caller after this call returns)
- **Control link (dynamic link)** — pointer to the caller's activation record
- **Access link (static link)** — pointer to the activation record of the **lexically enclosing** scope (needed for nested functions to access outer variables) — this is used specifically for **static scoping**
- **Saved machine status** (registers, etc. to restore on return)
- **Local variables, Parameters, Return value storage**
- **TRAP:** **Control link** points to the **caller** (whoever actually called this function, at runtime); **Access link** points to the **lexically/statically enclosing** function (based on where this function is defined in the source, not who called it) — for a non-nested/non-recursive call these often coincide, but they diverge precisely in cases like recursion or a function called from an unrelated part of the program, which is exactly where exam questions target this distinction.

4.6.3 Static vs. Dynamic Scoping

	Static (Lexical) Scoping	Dynamic Scoping
Resolved by	Where a variable is textually/lexically declared in the source	Which function called the current one, at runtime
Determined at	Compile time	Runtime
Used by	Most modern languages (C, Java, Python, etc.)	Older languages (e.g., original Lisp), some shell scripting contexts

- **TRAP:** Static scoping is resolved using the **access/static link** chain; dynamic scoping would instead need the **control/dynamic link** chain — mixing up which link supports which scoping discipline is a frequent error.

4.6.4 Storage Allocation Strategies

Strategy	Applies to	Property
Static Allocation	Global variables, static locals	Address fixed at compile time; no recursion support possible for such storage
Stack Allocation	Local variables, parameters	Allocated/deallocated automatically per call via activation records; supports recursion naturally
Heap Allocation	Dynamically-sized/lifetime data (<code>malloc</code> / <code>new</code>)	Explicit or garbage-collected management; lifetime not tied to any particular call

- **TRAP:** **Static allocation cannot support recursive procedures** — since each recursive call would need its own separate copy of local variables, but static allocation gives every call the *same fixed* memory address, causing all recursive calls to clobber each other's locals. This is precisely why languages supporting recursion require stack (or heap) allocation for locals.

4.6.5 Code Generation Considerations

- **Register allocation:** deciding which values live in (scarce) CPU registers vs. memory at each point — a well-known NP-hard problem in general, commonly approximated via **graph coloring** (build an interference graph of which variables are "live" simultaneously, then color it with available register colors).
- **Instruction selection:** choosing target machine instructions to implement each intermediate-code operation, often via tree-pattern matching against instruction templates.

Questions

Q1. A symbol table typically stores, for each identifier:

A. Only its name, nothing else B. Name, type, scope, and storage location/offset information C. Only its memory address, not its type D. The entire source code containing that identifier

Answer: B. Why: The compiler needs all of this information at various later phases (semantic checking, code generation) to correctly type-check and allocate storage for each identifier.

Q2. [Trap] To correctly resolve variable shadowing across nested scopes, a compiler's symbol table structure should be:

A. A single flat table shared across the entire program B. A nested/scoped structure (e.g., one table per scope, linked to enclosing scopes), so lookups find the innermost applicable declaration C. Sorted alphabetically with no scope information at all D. Rebuilt from scratch for every single statement

Answer: B. Why: A flat table cannot distinguish an inner redeclaration from an outer one with the same name; nested/scoped tables (or an explicit scope stack) let lookup correctly favor the innermost matching declaration.

Q3. [Trap] The control (dynamic) link in an activation record points to:

A. The activation record of the lexically enclosing function in the source code B. The activation record of whichever function actually called this one at runtime C. The next instruction to execute after the function returns D. The heap memory allocated by this function

Answer: B. Why: Control link tracks the runtime call chain (who called whom); this is distinct from the access/static link, which instead tracks lexical nesting in the source — these two coincide only when a function is called from its own lexically enclosing scope.

Q4. The access (static) link in an activation record is used primarily to support:

A. Dynamic scoping B. Static (lexical) scoping — allowing nested functions to correctly access variables of their enclosing function C. Register allocation D. Constant folding

Answer: B. Why: The access link chain lets a nested function walk up to the activation record of the function it is lexically defined within, correctly resolving outer-variable references under static scoping rules.

Q5. Static scoping resolves a variable reference based on:

A. Which function called the current function at runtime B. Where the variable is textually/lexically declared in the source program, determined at compile time C. The order functions are defined in a separate configuration file D. The current values of all global variables

Answer: B. Why: Static (lexical) scoping is a compile-time, source-structure-based rule — the vast majority of modern languages use this, in contrast to runtime-call-chain-based dynamic scoping.

Q6. [Trap] Why can't static storage allocation support recursive procedures?

A. Static allocation is too slow for recursive calls B. Every recursive call would share the exact same fixed memory address for its locals, causing different active calls to overwrite each other's data C. Static allocation only works for global variables, never for any procedure D. Recursive procedures don't have local variables

Answer: B. Why: Recursion needs a fresh, independent copy of locals per active call — static allocation's single fixed address per variable cannot provide this, hence stack (or heap) allocation is required instead.

Q7. Stack-based allocation for local variables and parameters works well because:

A. It provides a fixed, unchanging address for each variable across all calls B. Activation records are automatically pushed on call and popped on return, naturally giving each call its own separate storage and correctly supporting recursion C. It eliminates the need for a return address D. It requires garbage collection

Answer: B. Why: The stack discipline (LIFO push/pop matching call/return) is exactly what gives each active invocation — including each level of recursion — its own independent set of locals.

Q8. Register allocation via graph coloring works by:

A. Sorting variables alphabetically and assigning registers in that order B. Building an interference graph of variables that are simultaneously live, then assigning registers such that no two interfering (simultaneously live) variables share the same register C. Assigning every variable its own dedicated register regardless of availability D. Only applying to global variables

Answer: B. Why: Two variables that are both "live" (needed) at the same program point cannot share a register without corrupting one's value — graph coloring formalizes this as a standard, well-studied (NP-hard in general) constraint satisfaction problem.

Q9. Heap allocation, compared to stack allocation, is characterized by:

A. Storage lifetime strictly tied to a single function call's duration B. Storage lifetime not tied to any particular function call — explicitly managed or garbage-collected, useful for dynamically-sized or longer-lived data C. Fixed compile-time addresses, like static allocation D. Automatic deallocation the instant the allocating function returns

Answer: B. Why: Heap-allocated memory (via malloc/new) persists independently of the call stack, exactly enabling data structures whose size or lifetime can't be determined/bounded at compile time or tied to one call's scope.

Q10. [Numerical/conceptual] In a deeply recursive function call (the function calling itself), the control link and access link of the newest activation record will:

A. Always point to the exact same activation record B. Point to the immediately previous recursive call's activation record for BOTH links, since the caller and the lexically enclosing function are the same function in simple self-recursion C. Always point to completely unrelated, different activation records D. Be undefined, since recursion breaks activation record structure

Answer: B. Why: In simple self-recursion, the function that called this invocation (control link target) and the function this invocation is lexically nested within (access link target) are literally the same function/previous call — so both links coincide here, unlike in cases involving nested (non-recursive) functions called from elsewhere.

5.1 Introduction to OS — Types, Structure & System Calls

1.1 What is an OS?

- **OS** = system software acting as an intermediary between user and hardware; a **resource manager/allocator** (CPU time, memory, I/O devices, buses) and an **extended machine** (hides hardware complexity, provides a convenient platform for applications).
- **Goals:** Primary = **Convenience** (user-friendly). Secondary = **Efficiency, Reliability, Maintainability**. Functions are the means; goals are the ends.

1.2 Evolution & Types of OS

Type	Core Idea	Key Trade-off
Batch OS	Similar jobs grouped and run as a batch with no user interaction; a Batch Monitor sequences them	Improves turnaround vs manual operation, but no interactivity; CPU still idles during I/O
Spooling	(Simultaneous Peripheral Operations On-Line) Data queued in a buffer/disk for a slow device (e.g., printer); overlaps I/O of one job with CPU of another	Lets CPU keep working instead of waiting on slow I/O devices
Multiprogramming	Several jobs held in main memory at once; when the running job blocks (e.g., on I/O), CPU switches to another ready job	Keeps CPU busy as long as ANY job is ready — but needs more complex memory/scheduling management
Multitasking / Time-sharing	Multiprogramming + very fast context switching so each user feels they have the whole machine to themselves	Enables interactivity; overhead = frequent context switches
Multiprocessing (tightly coupled)	Two or more CPUs share one main memory	Higher throughput/reliability, but added complexity and contention overhead
Real-Time OS (RTOS)	Deterministic, deadline-driven scheduling	Hard RT vs Soft RT (see below)
Distributed OS	Independent, networked, loosely-coupled nodes (no shared memory/clock) presented to the user as ONE single system (transparency / single-system-image)	Enables resource sharing, speedup, reliability, communication — at the cost of network dependency

- **TRAP: Multitasking and Time-sharing are essentially the same concept** (both = multiprogramming + fast switching for interactive use) — do not treat them as unrelated OS categories.
- **Multiprocessing types:**
- **Symmetric (SMP):** all CPUs are **peers**, each runs an identical copy of the OS, no boss-worker relation. Most modern systems (Windows/Linux/macOS).
- **Asymmetric: master-slave** — one master CPU schedules/controls; others (slaves) only execute assigned work. Simpler to design but less efficient (master can bottleneck).
- **Multiprocessing advantages:** increased throughput (speed-up with N processors is **less than N** due to coordination overhead — **TRAP:** don't assume linear speedup), economy of scale (shared peripherals/storage), increased reliability/fault tolerance (losing 1 of 10 CPUs slows the system ~10%, doesn't crash it), less power/heat per unit of work.
- **RTOS:** Hard RT = missing a deadline is a **total failure** (airbags, anti-lock brakes — reacts at t=0). Soft RT = missing a deadline **degrades quality** but is tolerable (digital cameras, streaming — reacts at t=0+).

1.3 User Interfaces

Interface	Style	Typical User
CLI (Command-Line)	Text commands via a shell/interpreter	Sysadmins, power users — efficient once learned
GUI	Mouse, windows, icons, menus	General users — intuitive, visually guided
Touchscreen	Gesture-based	Mobile/tablet users

1.4 OS Structure

Approach	Idea	Trade-off
Simple/Monolithic	No well-defined structure; everything runs in kernel space (e.g., original MS-DOS)	Fast (no IPC overhead) but hard to maintain/extend, poor fault isolation
Layered	OS broken into layers 0 (hardware) → N (user interface); each layer only uses services of the layer directly below it	Modular, easier to debug/verify (information hiding); but added layer-crossing overhead
Microkernel	Kernel keeps only bare essentials (e.g., IPC, basic scheduling); most services (file system, drivers) run as user-space servers	Smaller kernel → more reliable & easier to extend (new services need no kernel changes) but slower due to frequent user-kernel IPC (e.g., Mach, MINIX)

- **TRAP:** Microkernel = **more** reliable/extensible but **slower**; Monolithic = **faster** but less modular/reliable — students often flip this trade-off.

1.5 System Calls & Dual Mode

- **System call:** the interface through which a user program requests an OS service (implemented via an **API** — e.g., POSIX API for UNIX/Linux/macOS, Windows API, Java API). The caller only needs to know the API contract, not the internal implementation.
- **System call categories:** Process control, **File management**, **Device management**, **Information maintenance** (get/set time, system data, process/file attributes), **Communication** (create/delete connections, send/receive messages).
- **Dual Mode & Mode Bit:** hardware includes a **mode bit** — **0 = Kernel/Supervisor mode**, **1 = User mode**. Purpose: protect the OS and hardware from buggy or malicious user programs by restricting which instructions user code may execute directly (e.g., I/O instructions are privileged).
- **Mechanism:** user program executes a system call → **trap (software interrupt)** switches CPU from user mode → kernel mode → OS services the request → control returns to user mode.
- **TRAP:** A **normal function call** does NOT change the mode bit; only a **system call (via trap/interrupt)** switches user mode ↔ kernel mode.

Questions

Q1. Which of the following best defines an Operating System's primary goal?

A. Maximum efficiency at all costs B. Convenience / user-friendliness, with efficiency as a secondary goal C. Minimum memory footprint D. Maximum throughput only

Answer: B. Why: Convenience (usability) is the primary goal; efficiency, reliability, and maintainability are secondary goals that support it.

Q2. [Trap] "Multitasking" and "Time-sharing" as OS concepts are:

A. Completely unrelated B. Essentially the same concept — multiprogramming plus fast context switching for interactive use C. Time-sharing is older and unrelated to multiprogramming D. Multitasking only applies to single-CPU systems and time-sharing only to multi-CPU systems

Answer: B. Why: Both describe multiple users/programs sharing the CPU via frequent switching, giving each the illusion of dedicated access.

Q3. In Symmetric Multiprocessing (SMP), the relationship between processors is:

A. Boss-worker (master-slave) B. Peers — no processor controls another, each runs an identical OS copy C. Only one processor may ever execute kernel code D.

Processors do not communicate

Answer: B. Why: SMP treats all CPUs as equals; Asymmetric multiprocessing is the one with a master-slave relationship.

Q4. A pacemaker's control software, where missing a deadline can cause catastrophic failure, requires:

A. Soft Real-Time OS B. Hard Real-Time OS C. Batch OS D. Time-sharing OS

Answer: B. Why: Hard RT systems treat any missed deadline as total failure — appropriate for life-critical embedded control.

Q5. [Numerical/Conceptual] With N processors cooperating on a task, the actual speed-up is:

A. Always exactly N B. Always greater than N C. Less than N, due to coordination overhead and resource contention D. Always exactly 1

Answer: C. Why: Overhead from synchronization and shared-resource contention means real speed-up with N processors is sub-linear (less than N).

Q6. Which OS structuring approach moves most services out of the kernel into user-space servers, shrinking the kernel itself?

A. Monolithic B. Layered C. Microkernel D. Simple structure

Answer: C. Why: The microkernel approach keeps only essential functions in the kernel, improving reliability/extensibility at some IPC performance cost.

Q7. [Trap] What actually happens when a user program executes a system call?

A. Nothing changes; it behaves like a normal function call B. A trap (software interrupt) switches the CPU from user mode to kernel mode to service the request, then returns to user mode C. The mode bit is permanently set to kernel mode D. The program is terminated

Answer: B. Why: A system call triggers a trap that temporarily elevates execution to kernel mode for the OS to perform the privileged operation, then control returns to user mode — unlike an ordinary function call, which never changes the mode bit.

Q8. Which system call category includes "get/set system time" and "get process/file attributes"?

A. Process control B. Device management C. Information maintenance D. Communication

Answer: C. Why: Information maintenance calls handle querying/setting system data, time, and process/file/device attributes.

Answer: C. Why: The PCB stores runtime state (PC, registers, memory-management info, accounting, I/O status) — not unrelated details like which compiler produced the executable.

Q3. [Numerical/Trap] On a single-CPU system with n total processes, what is the maximum number of processes that can be in the Running state at once?

A. n B. $n-1$ C. 1 D. 0

Answer: C. Why: A single CPU can execute only one process at a time, regardless of how many total processes (n) exist.

Q4. Which scheduler is responsible for controlling the degree of multiprogramming?

A. Short-term scheduler B. Long-term scheduler C. Medium-term scheduler D. Dispatcher

Answer: B. Why: The long-term (job) scheduler decides which jobs are admitted from the disk pool into memory, directly setting how many processes reside in memory at once.

Q5. Why must the short-term (CPU) scheduler be much faster than the long-term scheduler?

A. It handles fewer processes B. It is invoked far more frequently (every ~ 100 ms or less) so any delay directly wastes CPU time C. It runs only once per session D. Speed doesn't matter for the short-term scheduler

Answer: B. Why: Since the CPU scheduler runs on the order of every 100 ms, any inefficiency in scheduler execution itself becomes significant overhead; the long-term scheduler runs far less often and can afford to be slower/more deliberate.

Q6. [Trap] What kind of process mix should the Long-Term Scheduler aim for?

A. All CPU-bound processes for maximum computation B. All I/O-bound processes for maximum device usage C. A balanced mix of CPU-bound and I/O-bound processes D. It doesn't matter what mix is chosen

Answer: C. Why: An all-I/O-bound mix starves the CPU scheduler (empty Ready queue); an all-CPU-bound mix leaves I/O devices idle — a balanced mix keeps both CPU and I/O subsystems utilized.

Q7. Context switch time is best described as:

A. Time during which useful process computation occurs B. Pure overhead — no useful work is done while switching C. Time spent only on I/O operations D. Time spent compiling the next process

Answer: B. Why: A context switch only saves/restores state; the CPU performs no productive computation for either process during that interval.

Q8. The module that performs the actual context switch and hands control to the process selected by the short-term scheduler is the:

A. Long-term scheduler B. Medium-term scheduler C. Dispatcher D. Job pool manager

Answer: C. Why: The dispatcher executes the mechanics of switching to the chosen process (context switch, mode switch, jump to resume point); the time this takes is the dispatch latency.

5.3 CPU Scheduling Algorithms

3.1 Preemptive vs Non-Preemptive

- **Non-preemptive:** once CPU is allocated, process keeps it until it terminates or voluntarily blocks for I/O.
- **Preemptive:** CPU can be forcibly taken away — on process termination, on voluntary I/O wait, when a higher-priority process becomes ready, or when the time quantum expires.

3.2 Scheduling Criteria & Core Formulas

Turnaround Time (TAT) = Completion Time (CT) - Arrival Time (AT)
 Waiting Time (WT) = TAT - Burst Time (BT)
 Response Time = Time of FIRST CPU allocation - Arrival Time

- Goals: maximize CPU utilization & throughput; minimize TAT, WT, response time.
- **TRAP:** The scheduling algorithm does NOT change how long a process spends doing I/O — it only affects time spent **waiting in the ready queue**.

3.3 FCFS (First Come First Serve)

- Always **non-preemptive**; FIFO queue order by arrival time.
- **TRAP:** Suffers from **Convoy Effect** — short processes queue up behind one long process, inflating average waiting time.

Worked Example (reused across 3.3–3.6, 3.8 for comparison)

Processes: P1(AT=0,BT=5), P2(AT=1,BT=3), P3(AT=2,BT=8), P4(AT=3,BT=6)

FCFS order: P1, P2, P3, P4
 P1: 0-5 CT=5 TAT=5 WT=0
 P2: 5-8 CT=8 TAT=7 WT=4
 P3: 8-16 CT=16 TAT=14 WT=6
 P4: 16-22 CT=22 TAT=19 WT=13
 Avg TAT = (5+7+14+19)/4 = 11.25 Avg WT = (0+4+6+13)/4 = 5.75

3.4 SJF (non-preemptive) & SRTF (preemptive)

- **SJF:** among AVAILABLE processes, run the one with smallest total burst time; cannot be preempted once started, even if a shorter job arrives.
- **SRTF (Shortest Remaining Time First):** the preemptive version — re-evaluates on every new arrival; if the new arrival's burst < running process's REMAINING time, preempt.
- SRTF is **provably optimal** (minimum average waiting time of any scheduling algorithm) — but is NOT implementable in practice because burst time is never known in advance (only estimated, e.g. via exponential averaging $\tau(n+1) = \alpha \tau_n + (1-\alpha) \cdot \tau_n$).
- **TRAP:** SRTF/SJF risk **starvation** of long processes if short processes keep arriving.

Worked Example (same process set)

SJF (non-preemptive):
 P1: 0-5 (only one available) CT=5 TAT=5 WT=0
 P2: 5-8 (shortest of P2,P3,P4) CT=8 TAT=7 WT=4
 P4: 8-14 (shortest of P3,P4) CT=14 TAT=11 WT=5
 P3: 14-22 CT=22 TAT=20 WT=12
 Avg TAT = 43/4 = 10.75 Avg WT = 21/4 = 5.25

SRTF (preemptive):
 t=0: run P1(rem5)
 t=1: P2 arrives(rem3) < P1 rem(4) -> preempt, run P2
 t=2,3: P3,P4 arrive but P2's remaining (2, then 1) stays smallest -> P2 continues
 t=4: P2 completes (CT=4) TAT=3 WT=0
 t=4: shortest remaining = P1(4) -> run P1 to completion
 t=8: P1 completes (CT=8) TAT=8 WT=3
 t=8: shortest of P3(8),P4(6) = P4 -> run to completion
 t=14: P4 completes (CT=14) TAT=11 WT=5
 t=14-22: P3 runs alone CT=22 TAT=20 WT=12
 Avg TAT = 42/4 = 10.5 Avg WT = 20/4 = 5.0 <- lowest of all algorithms (optimal)

3.5 Priority Scheduling

- CPU given to highest-priority available process (convention for "highest" — lower or higher number — varies by system, always check the question's convention). Non-preemptive or preemptive variants exist, exactly like SJF/SRTF.
- **TRAP:** Risk of **starvation** for low-priority processes — solved by **Aging** (gradually increase the priority of a process the longer it waits, e.g., +1 priority level every 10 minutes).

Worked Example (non-preemptive, LOWER number = higher priority)

P1(AT0,BT4,Pr2), P2(AT1,BT3,Pr1), P3(AT2,BT1,Pr3), P4(AT3,BT5,Pr4)
 P1: 0-4 (only one available) CT=4 TAT=4 WT=0
 P2: 4-7 (highest priority among P2,P3,P4) CT=7 TAT=6 WT=3
 P3: 7-8 (higher priority than P4) CT=8 TAT=6 WT=5
 P4: 8-13 CT=13 TAT=10 WT=5
 Avg TAT = 26/4 = 6.5 Avg WT = 13/4 = 3.25

3.6 Round Robin (RR)

- Preemptive; ready queue is circular (FIFO); each process gets the CPU for at most **1 time quantum (TQ)**, then moves to the tail of the queue if not finished.
- If n processes and quantum q : each process waits **at most $(n-1) \times q$** time units between turns.

Time-quantum trade-off:

TQ too LARGE → behaves like FCFS (worse response time)

TQ too SMALL → approaches ideal "processor sharing," but context-switch overhead dominates → lower CPU utilization

- **TRAP:** Round Robin is designed to optimize **response time** for time-sharing systems — it does NOT generally minimize average waiting time (can even be worse than FCFS on some inputs, as the worked example shows).

Worked Example (same process set, quantum = 4)

```
t=0-4: P1 runs (rem 5-4=1); P2,P3,P4 arrive during this → queue: [P2,P3,P4,P1(rem1)]
t=4-7: P2 runs to completion (rem3, finishes early) CT=7
t=7-11: P3 runs (rem 8-4=4) queue: [P4,P1(rem1),P3(rem4)]
t=11-15: P4 runs (rem 6-4=2) queue: [P1(rem1),P3(rem4),P4(rem2)]
t=15-16: P1 runs to completion (rem1) CT=16
t=16-20: P3 runs to completion (rem4) CT=20
t=20-22: P4 runs to completion (rem2) CT=22
TAT: P1=16,P2=6,P3=18,P4=19 Avg TAT=59/4=14.75
WT: P1=11,P2=3,P3=10,P4=13 Avg WT=37/4=9.25 (worse than FCFS here!)
```

3.7 LJF / LRTF and HRRN

- **LJF (Longest Job First) / LRTF (Longest Remaining Time First, preemptive)** — opposite of SJF/SRTF; included mainly for contrast, rarely optimal, generally poor average waiting time (favors long jobs, starves short ones).
- **HRRN (Highest Response Ratio Next)** — non-preemptive; solves SJF's starvation problem by factoring in wait time:

Response Ratio = (Waiting Time + Burst Time) / Burst Time

Highest response ratio runs next — a process's ratio grows the longer it waits, guaranteeing it eventually gets scheduled (no starvation), while still favoring short jobs early on.

Worked Example (same process set)

```
t=5: candidates P2(RR=(4+3)/3=2.33), P3(RR=(3+8)/8=1.375), P4(RR=(2+6)/6=1.333) → run P2
t=8: candidates P3(RR=(6+8)/8=1.75), P4(RR=(5+6)/6=1.833) → run P4
t=14: only P3 left → run P3
Result identical to SJF-NP for this data set: Avg TAT=10.75, Avg WT=5.25
```

3.8 Multilevel Queue vs Multilevel Feedback Queue

Feature	Multilevel Queue	Multilevel Feedback Queue
Queue assignment	Permanent (fixed at entry, e.g. by process type)	Dynamic — process can move between queues
Flexibility	Low (simple, low overhead)	High (most general/configurable CPU scheduling algorithm)
Handles aging?	No	Yes — long-waiting process promoted to a higher-priority queue
Inter-queue scheduling	Fixed priority (higher queue starves lower) or time-sliced % of CPU	Defined by scheduler's demotion/promotion rules

- Multilevel Feedback Queue is defined by: **(1)** number of queues, **(2)** scheduling algorithm per queue, **(3)** rule to upgrade a process, **(4)** rule to demote a process, **(5)** rule for which queue a new process enters.
- **TRAP:** Multilevel Queue = fixed/rigid queue membership; Multilevel FEEDBACK Queue = processes migrate — do not confuse the two.

3.9 Summary Comparison

Algorithm	Preemptive?	Starvation risk?	Optimal avg WT?
FCFS	No	No (but Convoy Effect)	No
SJF	No	Yes (long jobs)	Best among non-preemptive
SRTF	Yes	Yes (long jobs)	Yes — optimal overall
Priority	Either	Yes (low priority) — fixed by Aging	No
Round Robin	Yes	No	No (optimizes response time instead)
HRRN	No	No (response ratio prevents it)	Close to SJF, no starvation
Multilevel Feedback Queue	Yes	No (aging built in)	Configurable / most general

Questions

Q1. [Numerical] For P1(AT=0,BT=5), P2(AT=1,BT=3), P3(AT=2,BT=8), P4(AT=3,BT=6) under FCFS, what is the average waiting time?

A. 4.25 B. 5.0 C. 5.75 D. 6.5

Answer: C. **Why:** WTs are 0,4,6,13 → average = $23/4 = 5.75$ (see worked example, §3.3).

Q2. [Trap] Which statement about FCFS is correct?

A. It is always preemptive B. It never suffers from convoy effect C. It is always non-preemptive and can suffer from the convoy effect D. It requires knowing burst time in

advance

Answer: C. Why: FCFS is purely arrival-order based and non-preemptive; a long process at the front delays all shorter ones behind it (convoy effect).

Q3. Why is SRTF considered optimal but practically unusable?

A. It has high context-switch overhead only B. It provably minimizes average waiting time but requires knowing future burst times, which is generally impossible C. It always causes deadlock D. It cannot be preempted

Answer: B. Why: SRTF/SJF need the exact CPU burst length in advance; real systems can only estimate it, so pure SRTF cannot actually be implemented.

Q4. [Numerical] Using the same process set as the worked examples, what is the average turnaround time under SRTF?

A. 10.5 B. 10.75 C. 11.25 D. 14.75

Answer: A. Why: TATs are 8,3,20,11 → average = $42/4 = 10.5$ (lowest among all algorithms compared, confirming SRTF's optimality).

Q5. [Trap] A low-priority process never gets the CPU because higher-priority processes keep arriving. Which technique fixes this?

A. Round Robin quantum reduction B. Aging — gradually increasing the waiting process's priority over time C. Switching to FCFS D. Increasing the time quantum

Answer: B. Why: Aging directly counters starvation by boosting priority the longer a process waits, ensuring it eventually becomes highest priority.

Q6. [Numerical] In Round Robin with $n=5$ ready processes and time quantum $q=3$, what is the maximum time any process waits between two of its turns?

A. 3 B. 12 C. 15 D. 5

Answer: B. Why: Max wait = $(n-1) \times q = 4 \times 3 = 12$.

Q7. [Trap] What happens to Round Robin's behavior if the time quantum is set extremely large?

A. It behaves like SRTF B. It behaves like FCFS C. It becomes Priority Scheduling D. It causes immediate deadlock

Answer: B. Why: If the quantum exceeds every process's burst time, each process finishes in its first turn — identical to FCFS ordering.

Q8. HRRN's Response Ratio formula is:

A. Burst Time / Waiting Time B. (Waiting Time + Burst Time) / Burst Time C. Waiting Time – Burst Time D. Burst Time / (Waiting Time + Burst Time)

Answer: B. Why: $RR = (W+S)/S$ — this rises over time for any waiting process (even a long one), preventing indefinite starvation while still favoring short jobs.

Q9. [Trap] What is the key structural difference between Multilevel Queue and Multilevel Feedback Queue scheduling?

A. There is no difference B. Multilevel Queue allows movement between queues; Feedback does not C. Multilevel Feedback Queue allows a process to move between queues (e.g. via aging); plain Multilevel Queue assigns processes permanently D. Multilevel Queue is always preemptive, Feedback is not

Answer: C. Why: The defining feature of the "feedback" variant is dynamic reassignment between queues based on observed behavior/waiting time; plain multilevel queue is static once a process is classified.

Q10. [Numerical] For $P1(AT0,BT4,Pr2)$, $P2(AT1,BT3,Pr1)$, $P3(AT2,BT1,Pr3)$, $P4(AT3,BT5,Pr4)$ under non-preemptive priority scheduling (lower number = higher priority), what is $P3$'s waiting time?

A. 1 B. 5 C. 6 D. 3

Answer: B. Why: $P3$ runs from $t=7$ to $t=8$ ($CT=8$); $TAT = 8-2 = 6$; $WT = TAT - BT = 6 - 1 = 5$.

Q11. Which CPU scheduling algorithm is described as "the most general," configurable via number of queues, per-queue algorithm, and promotion/demotion rules?

A. Priority Scheduling B. Round Robin C. Multilevel Feedback Queue Scheduling D. HRRN

Answer: C. Why: Multilevel Feedback Queue can be tuned via 5 defining parameters, making it flexible enough to emulate many other scheduling behaviors — at the cost of high design complexity.

5.4 Process Synchronization — Critical Section & Semaphores

4.1 Race Condition & Critical Section

- **Race condition:** final output depends on the (unpredictable) relative execution order/interleaving of concurrent processes accessing shared data.
- **General process structure:** Entry Section (request permission) → **Critical Section** (accesses shared resource) → Exit Section (release) → Remainder Section.

4.2 The 3 Critical-Section Requirements

1. **Mutual Exclusion:** at most one process may be inside its critical section at any instant.
2. **Progress:** if no process is in its CS and some wish to enter, only processes NOT in their remainder section may participate in deciding who enters next — this decision cannot be postponed indefinitely (no deadlock in the decision itself).
3. **Bounded Waiting:** there is a fixed limit on how many times other processes may enter their CS after a process has requested entry, before that process is granted entry (no indefinite postponement / no starvation).
 - **TRAP: Mutual Exclusion and Progress are mandatory; Bounded Waiting is often treated as the optional/stronger criterion** — a valid solution can satisfy ME+Progress but still risk starvation if Bounded Waiting isn't met.

4.3 Two-Process Software Solutions

Approach	Mutual Exclusion?	Progress?	Issue
Boolean <code>turn</code> variable (strict alternation)	Yes	No	Forces strict turn-by-turn access even if the other process doesn't want to enter — violates Progress
Boolean <code>flag[]</code> array (intent flags)	Yes	No	Both processes can set their flag simultaneously and wait forever on each other → deadlock
Peterson's Solution (<code>turn + flag[]</code> combined)	Yes	Yes	Satisfies all 3 properties, but only for exactly 2 processes ; not guaranteed on modern multiprocessor architectures (relies on memory-ordering assumptions)

- **TRAP:** Peterson's Solution is a classic *software-only* 2-process solution — it does NOT generalize directly to N processes (that's what semaphores are for).

4.4 Semaphores

- A semaphore S is an integer accessed only via two **atomic** operations:

```
wait(S) / P(S):  while (S <= 0) ;  S--;
signal(S) / V(S): S++;
```

- Semaphore-based CS solution: initialize `S=1`, then `wait(S)` before CS, `signal(S)` after CS — generalizes to **N processes** (unlike Peterson's).
- Semaphores guarantee **Mutual Exclusion and Progress**, but the basic busy-wait version does **NOT guarantee Bounded Waiting** by itself.
- **Binary semaphore:** value restricted to {0,1} — behaves like a mutex lock.
- **Counting semaphore:** unrestricted integer range — models a pool of N identical resource instances; initialized to N, each `wait()` claims one instance, each `signal()` releases one.
- **TRAP:** Ordering multiple `wait()` calls on different semaphores **inconsistently across processes** (e.g., P: `wait(S1)` then `wait(S2)`; Q: `wait(S2)` then `wait(S1)`) risks **deadlock** — always acquire multiple semaphores in the same global order across all processes.

4.5 Classical Synchronization Problems

Problem	Core requirement	Key semaphores/variables
Producer-Consumer (bounded buffer, size n)	Producer must not overflow; consumer must not underflow; buffer access is mutually exclusive	<code>S=1</code> (mutex for buffer), <code>E=n</code> (count of empty slots), <code>F=0</code> (count of filled slots) — producer: <code>wait(E), wait(S), ..., signal(S), signal(F)</code> ; consumer: <code>wait(F), wait(S), ..., signal(S), signal(E)</code>
Readers-Writers (readers-priority version)	Multiple readers may read concurrently; a writer needs exclusive access; no reader/writer runs while a writer is writing	<code>mutex=1</code> (protects <code>readcount</code>), <code>wrt=1</code> (writer's exclusive lock); first reader does <code>wait(wrt)</code> , last reader does <code>signal(wrt)</code> — <code>readcount</code> tracks active readers
Dining Philosophers (5 philosophers, 5 chopsticks)	Each philosopher needs both adjacent chopsticks to eat; naive "pick left then right" for everyone → deadlock (all pick up left simultaneously)	Semaphore array <code>chopstick[5]</code> ; deadlock fixes: (a) allow at most N-1 (4) philosophers to sit at once, (b) pick up both chopsticks only inside a critical section (atomically), (c) asymmetric solution — odd-numbered philosopher picks left-then-right, even-numbered picks right-then-left

- **TRAP:** The naive Dining Philosophers solution (everyone picks up left chopstick first) is NOT deadlock-free — this is the whole point of the problem; a valid fix (limit to N-1 diners, or asymmetric pickup order) must be stated explicitly.

4.6 Semaphore Implementation: Busy-Waiting vs Block/Wakeup

- **Busy-wait implementation** (the simple `while(S<=0);` loop) wastes CPU cycles while waiting — bad on a single-processor system.
- **Block-and-wakeup implementation:** if `wait()` finds `S<=0`, the process calls `block()` — it's added to the semaphore's waiting list and suspended (Ready→Waiting), and the CPU is given to another process. `signal()` calls `wakeup(P)` on a waiting process, moving it back to Ready.
- **TRAP:** In the block/wakeup implementation, the semaphore's integer **value can go negative** — its magnitude then equals the number of processes currently waiting on it (this differs from the classical busy-wait definition where S never goes below 0).

4.7 Hardware Synchronization Primitives

- **Disable Interrupts:** simplest single-processor solution (disable interrupts in the entry section, re-enable in exit section) — prevents any context switch during the CS.
- **TRAP:** does **not scale to multiprocessor systems** (must message every processor to disable interrupts — slow, and doesn't stop a process running on ANOTHER CPU from entering its own CS simultaneously).

- **Test-and-Set (TAS):** an atomic hardware instruction that reads and sets a boolean in one indivisible step:

```
Boolean test_and_set(Boolean *target) {
    Boolean rv = *target;
    *target = true;
    return rv;
}
```

Used as a spinlock: `while(test_and_set(&lock));` then CS, then `lock=false;` . Works correctly on **multiprocessor** systems (unlike disabling interrupts) since the atomicity is a hardware guarantee on the memory word itself, not on interrupt delivery.

Questions

Q1. Which of the 3 critical-section requirements is generally considered optional, whose absence risks starvation but not necessarily an invalid solution?

A. Mutual Exclusion B. Progress C. Bounded Waiting D. Atomicity

Answer: C. **Why:** Mutual Exclusion and Progress are mandatory; Bounded Waiting's absence specifically risks (but doesn't guarantee) starvation of some process.

Q2. [Trap] The two-process solution using a single Boolean `turn` variable with strict alternation fails to satisfy:

A. Mutual Exclusion B. Progress (forces alternation even if one process doesn't want to enter) C. Both ME and Progress D. It satisfies all three properties

Answer: B. **Why:** It achieves ME but fails Progress since a process must wait its turn even when the other process has no interest in entering its CS.

Q3. Peterson's Solution is valid for:

A. Any number of processes B. Exactly 2 processes C. Exactly 3 processes D. Only single-processor hardware with disabled interrupts

Answer: B. **Why:** Peterson's Solution is specifically a 2-process software algorithm; semaphores are needed to generalize to N processes.

Q4. A counting semaphore initialized to 5 models:

A. A binary lock B. A resource pool with 5 identical instances available C. A queue of exactly 5 waiting processes D. 5 separate critical sections

Answer: B. **Why:** Counting semaphores track available instances of a resource type; each `wait()` claims one (decrementing), each `signal()` returns one (incrementing).

Q5. In the Readers-Writers (readers-priority) solution, which reader is responsible for acquiring the `wrt` lock?

A. Every reader, individually B. Only the FIRST reader to arrive when `readcount` goes from 0 to 1 C. Only the last reader D. No reader ever touches `wrt`

Answer: B. **Why:** Only the first reader (transitioning `readcount` 0→1) needs to lock out writers via `wait(wrt)`; subsequent readers just increment `readcount` under mutex. The LAST reader (`readcount`→0) calls `signal(wrt)`.

Q6. [Trap] Why is the naive Dining Philosophers solution (every philosopher picks up left chopstick, then right) invalid?

A. It violates mutual exclusion on each chopstick B. All 5 philosophers can simultaneously pick up their left chopstick, then all wait forever for their right — deadlock C. It runs out of memory D. It doesn't use semaphores at all

Answer: B. **Why:** Symmetric pickup order allows a circular-wait deadlock; fixes include limiting concurrent diners to N−1 or using an asymmetric (odd/even) pickup order.

Q7. [Trap] In the block-and-wakeup implementation of a semaphore, a semaphore value of −3 means:

A. An implementation error occurred B. 3 processes are currently blocked/waiting on this semaphore C. The semaphore has 3 available resources D. 3 processes have exited

Answer: B. **Why:** Under block/wakeup, value can go negative; its absolute value equals the number of processes queued waiting on that semaphore.

Q8. [Trap] Why doesn't disabling interrupts work as a general critical-section solution on multiprocessor systems?

A. Interrupts cannot be disabled on any modern CPU B. It only blocks interrupts on the local processor, so a process on another CPU can still enter the same critical section; and multiprocessor-wide interrupt disabling is slow C. It causes immediate system crash D. It works perfectly fine on multiprocessors

Answer: B. **Why:** Disabling interrupts is inherently a single-processor mutual-exclusion technique; message-passing to disable interrupts across all CPUs is costly and doesn't provide atomicity on shared memory.

Q9. The Test-and-Set instruction's key guarantee that makes it useful for synchronization is:

A. It is always fast B. It reads and modifies a memory location as one atomic (indivisible) hardware operation C. It disables all interrupts automatically D. It only works with binary semaphores

Answer: B. **Why:** Atomicity means even if two CPUs execute test-and-set on the same word "simultaneously," the hardware serializes them — one gets the old (false) value and proceeds, the other gets true and spins.

Q10. Which classical problem's semaphore solution uses the pair of semaphores E (count of empty slots) and F (count of filled slots)?

A. Readers-Writers B. Dining Philosophers C. Producer-Consumer (bounded buffer) D. Peterson's Solution

Answer: C. **Why:** Producer waits on E (space available) and signals F (item added); Consumer waits on F (item available) and signals E (space freed) — plus a mutex S for the buffer itself.

5.5 Threads & Multithreading Models

5.1 Process vs Thread

- **Process** = heavyweight, has its own independent memory space (code, data, heap, stack, PCB, resources).
- **Thread** = "lightweight process" — a basic unit of CPU utilization within a process. Has its own program counter, stack, register set, and thread ID, but **shares** code, data segment, and OS resources (open files, etc.) with sibling threads in the same process.
- **TRAP:** Threads within one process share the heap/global data — this is exactly why thread synchronization (mutexes/semaphores) is needed even though process-level protection (separate address spaces) already isolates *processes* from each other.

5.2 fork() System Call

- `fork()` creates a **child process** as a near-exact copy (duplicate image) of the calling (parent) process.
- Return value convention (classic exam fact):

```
fork() returns 0          -> in the CHILD process
fork() returns child's PID -> in the PARENT process (a positive integer)
fork() returns negative  -> on failure (no child created)
```

- Because both parent and child continue executing from the point right after `fork()`, programmer code typically branches on the return value to give parent and child different behavior.
- **Numerical rule:** `n` calls to `fork()` (e.g., in a loop or sequential calls, with no conditional branching) create $2^n - 1$ new processes in total (2^n processes exist overall including the original).
- **TRAP:** Don't confuse this with "n forks create n processes" — each fork doubles the process count from that point onward, giving exponential (2^n) growth, not linear.

5.3 Why Multithreading

- **Responsiveness:** one thread can keep the UI/interaction responsive while another blocks (e.g., disk I/O).
- **Resource sharing:** threads share the parent's memory/resources — cheaper than duplicating them per process.
- **Economy:** creating/context-switching a thread is much cheaper than a full process (no new address space needed).
- **Scalability:** threads can run in parallel on multicore/multiprocessor systems.

5.4 Multithreading Models

Threads exist at two levels: **User-level threads** (managed entirely in user space, no kernel awareness) and **Kernel-level threads** (managed/scheduled by the OS kernel directly).

Model	Mapping	Key Strength	Key Weakness
Many-to-One	Many user threads → 1 kernel thread	Fast, cheap thread management (no kernel calls)	One blocking system call blocks the entire process; no true parallelism (only 1 kernel thread = only 1 CPU used ever)
One-to-One	Each user thread → its own kernel thread	True parallelism across CPUs; a blocking call in one thread doesn't block others	Higher overhead — creating a kernel thread per user thread is costly; most OSes cap the number allowed
Many-to-Many	Many user threads → smaller/equal number of kernel threads	Combines both benefits: unlimited user threads + no full-process blocking + true parallelism	Most complex to implement

- **TRAP:** **Many-to-One** is the model where a single blocking syscall stalls every thread in the process — this is its single biggest exam-tested weakness. **One-to-One** (used by modern Linux and Windows) avoids that but at higher per-thread overhead.

Questions

Q1. [Trap] In a Unix-like system, what does `fork()` return inside the newly created child process?

A. The child's own PID B. 0 C. The parent's PID D. -1

Answer: B. **Why:** `fork()` returns 0 in the child; it returns the child's PID to the parent, and a negative value only on failure.

Q2. [Numerical] A program calls `fork()` three times in sequence (no conditional logic, so every resulting process also executes the remaining `fork()` calls). How many processes exist in total after all forks complete?

A. 3 B. 6 C. 7 D. 8

Answer: D. **Why:** `n` forks give 2^n total processes = $2^3 = 8$ (i.e., $2^n - 1 = 7$ new processes plus the original 1).

Q3. Which multithreading model suffers from the problem that a single blocking system call halts every thread in the process?

A. One-to-One B. Many-to-Many C. Many-to-One D. None of these

Answer: C. **Why:** With Many-to-One, all user threads map onto a single kernel thread, so if that one kernel thread blocks (e.g., on I/O), the entire process — all its threads — is blocked.

Q4. Which multithreading model is used by modern Linux and Windows, giving true parallel execution across CPUs at the cost of per-thread kernel overhead?

A. Many-to-One B. One-to-One C. Many-to-Many D. Two-to-One

Answer: B. **Why:** One-to-One maps each user thread directly to its own kernel thread, enabling genuine multi-CPU parallelism, though creating many kernel threads has more overhead than the other models.

Q5. Which of the following is shared between threads of the same process, but NOT between separate processes?

A. Program counter B. Stack C. Code and data segment D. Register set

Answer: C. **Why:** Threads of one process share the code/data segment and other resources; each thread still keeps its own PC, stack, and registers, which is what makes it independently schedulable.

Q6. [Conceptual] Why is creating a new thread generally cheaper than creating a new process?

A. Threads don't need a program counter B. A thread reuses the parent process's address space/resources instead of duplicating them C. Threads cannot be scheduled by the OS D. Threads always run on a separate CPU

Answer: B. **Why:** Since threads share the code, data, and resources of the owning process, thread creation avoids the overhead of setting up an entirely new address space, unlike process creation.

5.6 Deadlock — Prevention, Avoidance, Detection

6.1 Basics

- **Deadlock:** every process in a set is waiting for an event (resource release) that can only be caused by another process in that same set — none can ever proceed.
- **TRAP: Starvation = long/indefinite waiting** (may eventually get served); **Deadlock = infinite waiting** (can never be served without external intervention).
Starvation does not imply deadlock, but deadlock does imply (permanent) starvation of the involved processes.
- Normal resource usage cycle: **Request** → **Use** → **Release**.

6.2 The 4 Necessary Conditions (ALL must hold simultaneously)

1. **Mutual Exclusion** — at least one resource is held in non-sharable mode (only one process can use it at a time).
2. **Hold and Wait** — a process holds at least one resource while waiting to acquire additional resources held by others.
3. **No Preemption** — a resource can only be released voluntarily by the process holding it (never forcibly taken).
4. **Circular Wait** — a cycle of processes $P_0 \rightarrow P_1 \rightarrow P_2 \rightarrow \dots \rightarrow P_0$ exists, each waiting for a resource held by the next.

TRAP: Breaking even **ONE** of these four conditions prevents deadlock — this single fact is the entire basis of every deadlock-prevention strategy below.

6.3 Deadlock Handling Approaches — Overview

Approach	Idea
Prevention	Structurally deny one of the 4 conditions so deadlock is impossible
Avoidance	Allow all 4 conditions to be possible, but dynamically check every resource request to avoid entering an unsafe state (Banker's Algorithm)
Detection & Recovery	Let deadlock happen, periodically detect it, then recover
Ignorance (Ostrich Algorithm)	Pretend deadlocks don't happen — used by most general-purpose OSes (Linux/Windows) since deadlocks are rare in practice and prevention/avoidance costs (reduced concurrency/throughput) aren't worth it

6.4 Deadlock Prevention (deny one condition)

- **Mutual Exclusion:** generally CANNOT be denied — some resources (printer, etc.) are intrinsically non-sharable by hardware nature.
- **Hold and Wait:** deny by requiring a process to request ALL its resources upfront before running (conservative), or require releasing everything held before requesting more.
- **No Preemption:** deny by allowing forcible preemption — if a waiting process requests unavailable resources, preempt resources already held by OTHER waiting processes instead.
- **Circular Wait:** deny by imposing a **total ordering** on resource types — a process may only request resources in strictly increasing order (must release higher-numbered resources before requesting a lower-numbered one).
- Drawback of prevention overall: low resource utilization and reduced system throughput/concurrency (overly restrictive).

6.5 Deadlock Avoidance — Banker's Algorithm

Requires advance knowledge of each process's **maximum** possible resource demand.

```
Data structures (n = #processes, m = #resource types):
Available[m]      - available instances of each resource type
Max[n][m]         - max demand of each process
Allocation[n][m]  - currently allocated to each process
Need[n][m] = Max[n][m] - Allocation[n][m]  (remaining demand)
```

Safety Algorithm (is the current state safe?):

```
1. Work = Available; Finish[i] = false for all i
2. Find i where Finish[i]==false AND Need[i] <= Work
   - if none exists -> go to step 4
3. Work = Work + Allocation[i]; Finish[i] = true; go to step 2
4. If Finish[i]==true for all i -> state is SAFE (the order found = safe sequence)
```

Resource-Request Algorithm (grant a request only if it keeps the system safe):

```
1. If Request[i] > Need[i] -> error (exceeds max claim)
2. If Request[i] > Available -> Pi must wait (not enough resources right now)
3. Pretend to allocate: Available -= Request; Allocation[i] += Request; Need[i] -= Request
   - Run Safety Algorithm on this new state.
   - If SAFE -> grant the request for real.
   - If UNSAFE -> rollback the pretend-allocation; Pi must wait.
```

Worked Example (verified)

```
Resources: E, F, G      Available = (3, 3, 0)
Max
P0 (4, 3, 1)           Allocation      Need = Max - Allocation
P1 (2, 1, 4)           (1, 0, 1)       (3, 3, 0)
P2 (1, 3, 3)           (1, 1, 2)       (1, 0, 2)
P3 (5, 4, 1)           (1, 0, 3)       (0, 3, 0)
                       (2, 0, 0)       (3, 4, 1)
```

Run safety algorithm: $Work = (3, 3, 0)$.

- P0: $Need(3, 3, 0) \leq Work(3, 3, 0) \checkmark \rightarrow Work += Alloc(P0) = (4, 3, 1)$

- P2: $\text{Need}(0,3,0) \leq (4,3,1) \checkmark \rightarrow \text{Work} += \text{Alloc}(P2) = (5,3,4)$
 - P1: $\text{Need}(1,0,2) \leq (5,3,4) \checkmark \rightarrow \text{Work} += \text{Alloc}(P1) = (6,4,6)$
 - P3: $\text{Need}(3,4,1) \leq (6,4,6) \checkmark \rightarrow \text{Work} += \text{Alloc}(P3) = (8,4,6)$
 All Finish=true $\rightarrow$ **Safe sequence = P0, P2, P1, P3** $\rightarrow$ system is in a safe state.

6.5b Resource Allocation Graph (RAG)

- Nodes: **processes** (circles) + **resource types** (rectangles, one dot per instance).
- Request edge** $P_i \rightarrow R_j$ (waiting for an instance). **Assignment edge** $R_j \rightarrow P_i$ (already holds an instance).
- TRAP:** A cycle in the RAG is **necessary but NOT sufficient** for deadlock when a resource type has **multiple instances** (the cycle might resolve if another instance frees up). A cycle is **necessary AND sufficient** for deadlock **only when every resource type has a single instance**.

6.6 Deadlock Detection & Recovery

- Detection algorithm = essentially the Safety Algorithm run periodically, but using only **Allocation** and **Request** (no Max/Need needed, since we're not avoiding — just checking if the CURRENT state is already deadlocked).
- Invocation strategy: run periodically (e.g., hourly) or reactively (e.g., when CPU utilization drops suggesting processes are stuck waiting).
- Recovery — Process Termination:** Abort ALL deadlocked processes at once (guaranteed but wastes all their computation) vs abort ONE at a time + re-run detection after each (more careful, but repeated detection overhead).
- Recovery — Resource Preemption:** pick a victim, preempt its resources, roll it back to a safe state. Victim selection weighs cost factors: priority, computation time invested, resources held, resources still needed. Must track rollback counts per process to avoid **starving the same victim repeatedly**.

Questions

Q1. Which of the following is NOT one of the four necessary conditions for deadlock?

A. Mutual Exclusion B. Hold and Wait C. Starvation D. Circular Wait

Answer: C. **Why:** Starvation is a separate phenomenon (indefinite but not necessarily infinite waiting); the four deadlock conditions are Mutual Exclusion, Hold-and-Wait, No Preemption, and Circular Wait.

Q2. [Trap] Deadlock prevention typically cannot deny which condition, since it is often an intrinsic hardware property of the resource?

A. Hold and Wait B. Mutual Exclusion C. Circular Wait D. No Preemption

Answer: B. **Why:** Many resources (e.g., a printer) are inherently non-sharable, so Mutual Exclusion generally cannot be eliminated as a prevention strategy.

Q3. [Numerical] Using the Banker's Algorithm example: Available=(3,3,0); Need(P0)=(3,3,0), Need(P1)=(1,0,2), Need(P2)=(0,3,0), Need(P3)=(3,4,1). Which process(es) CANNOT be satisfied by the initial Available=(3,3,0) alone (i.e., must wait for others to release resources first)?

A. Only P3 B. P1 and P3 C. Only P1 D. None — all can be satisfied immediately

Answer: B. **Why:** P1's Need (1,0,2) needs 2 units of G but Available has 0 of G; P3's Need (3,4,1) needs 4 units of F but Available has only 3 — both must wait for P0 or P2 to finish first and release resources, which the full safe sequence (P0, P2, P1, P3) confirms is possible.

Q4. A cycle exists in a Resource Allocation Graph where every resource type has exactly one instance. What can be concluded?

A. The system is definitely NOT deadlocked B. The system is definitely deadlocked C. Nothing can be concluded D. More information about process priority is needed

Answer: B. **Why:** With single-instance resource types, a cycle in the RAG is both necessary AND sufficient for deadlock.

Q5. [Trap] A cycle exists in a Resource Allocation Graph where at least one resource type has multiple instances. What can be concluded?

A. The system is definitely deadlocked B. The system is definitely safe C. A cycle is necessary but not sufficient — deadlock is only possible, not guaranteed D. RAGs cannot be drawn for multi-instance resources

Answer: C. **Why:** With multi-instance resource types, another instance might free up and break the apparent cycle's hold, so a cycle alone doesn't guarantee deadlock.

Q6. Which deadlock-handling approach is used by most general-purpose operating systems like Linux and Windows?

A. Prevention B. Avoidance (Banker's Algorithm) C. Detection and Recovery D. Ignorance (Ostrich Algorithm)

Answer: D. **Why:** Since deadlocks are rare in practice and prevention/avoidance impose significant performance/concurrency costs, most general-purpose OSes simply ignore the possibility and rely on manual/general recovery if it ever occurs.

Q7. In deadlock recovery via resource preemption, why must the "number of rollbacks" be tracked per process?

A. To calculate CPU burst time B. To prevent the same process from being repeatedly chosen as a victim (starvation) C. To determine the process's priority D. It is not necessary to track this

Answer: B. **Why:** Without tracking rollback counts, a low-cost victim might be picked every single time, indefinitely starving that one process even though the overall deadlock keeps recurring for others.

5.7 Memory Management — Contiguous Allocation, Paging, Segmentation

7.1 Memory Hierarchy & Locality of Reference

- Hierarchy (fastest/smallest/costliest → slowest/largest/cheapest): **Registers** → **Cache (L1/L2/L3)** → **Main Memory (RAM)** → **Secondary/Auxiliary Memory (disk)**.
- **Locality of Reference** is why caching/paging work at all — predicts what to prefetch.
- **Temporal locality**: a recently used item will likely be used again soon (basis of **LRU**).
- **Spatial locality**: items near a used address will likely be used soon too (basis of block/page prefetching).
- OS's memory duties: logical→physical address translation, deciding what to swap in/out, tracking free/used memory.

7.2 Contiguous Allocation

- Process must be loaded **entirely** and **contiguously** into main memory.
- **MMU (Memory Management Unit)**: hardware that translates logical→physical address using a **relocation register** (base address, added to every logical address) and a **limit register** (max allowed logical address — if logical address $\geq$ limit, illegal access is trapped by the OS).

Physical Address = Logical Address + Relocation Register (valid only if Logical Address < Limit Register)

- **Partitioning**: Fixed-size partitioning (simple, but internal fragmentation) vs Variable-size partitioning (exact fit per request, but external fragmentation accumulates).

Space Allocation Strategies

Strategy	Rule	Best for	Weakness
First Fit	First partition big enough	Speed	Poor packing over time
Best Fit	Smallest partition big enough	Minimizes leftover per allocation	Slow (full scan); leaves tiny unusable slivers
Worst Fit	Largest available partition	Leaves large reusable leftover in variable partitioning	Wastes large amounts in fixed partitioning
Next Fit	Like First Fit, but resumes scan from last allocation point	Avoids re-scanning from start	Similar issues to First Fit

- **External fragmentation**: enough TOTAL free memory exists, but it's scattered in pieces too small individually — fixed by **compaction** (costly, requires stopping/relocating processes) or moving to **non-contiguous allocation**.
- **Internal fragmentation**: allocated partition $\geq$ request size; the leftover INSIDE the allocated block is wasted (inherent to fixed-size schemes).
- **TRAP**: External fragmentation $\Rightarrow$ variable-size/contiguous schemes; Internal fragmentation $\Rightarrow$ fixed-size schemes (including paging, Ch 7.3). Don't swap these.

7.3 Paging (Non-Contiguous Allocation)

- Physical memory → fixed-size **frames**; logical/process memory → same-size **pages**. Page size = Frame size. Eliminates external fragmentation (any free frame fits any page) but keeps internal fragmentation (last page may be partly empty).
- **Address translation**:

Logical Address = (Page Number p , Offset d)
 - if Page size = 2^d bytes and Logical Address Space = 2^m , then p occupies $(m - d)$ bits
 Physical Address = (Frame Number f , Offset d) [offset d is IDENTICAL on both sides]
 Page Table: indexed by $p \rightarrow$ gives f

Worked Example

Page size = 1 KB (2^{10}), Logical Address = 5000.
 Page number = $\text{floor}(5000 / 1024) = 4$ (page 4 covers bytes 4096–5119); Offset = $5000 - 4096 = 904$.
 If page table says page 4 → frame 7: Physical Address = $7 \times 1024 + 904 = 8072$.

- **Page table** is a data structure (not hardware) stored in main memory; one per process; entries = number of pages; each entry stores a frame number.
- Disadvantage: naive paging needs **2 memory accesses per reference** (1 for page table, 1 for actual data) — solved by the **TLB**.

7.4 TLB (Translation Lookaside Buffer)

- Small, fast, **associative** (parallel-search) hardware cache of recent (page→frame) mappings.
- **TLB hit** → 1 memory access needed. **TLB miss** → page table lookup + memory access (2 accesses total, or 3 if you count the TLB check itself, depending on convention — use the standard 2-vs-1 split for EAT).

Effective Access Time (EAT) = $\text{HitRatio} \times (\text{TLB time} + \text{Memory time}) + \text{MissRatio} \times (\text{TLB time} + \text{Memory time} + \text{Memory time})$

Worked Example

TLB access = 20 ns, Memory access = 100 ns, Hit ratio = 80%.
 $\text{EAT} = 0.8 \times (20 + 100) + 0.2 \times (20 + 100 + 100) = 0.8 \times 120 + 0.2 \times 220 = 96 + 44 = 140 \text{ ns}$.
 - TLB holds entries for ONE process context at a time in the simplest design → frequent context switches force **TLB flush** (costly); mitigated by wiring down kernel entries or tagging entries with a process/address-space ID.

7.5 Multilevel (Hierarchical) Paging

- Modern address spaces (32–64 bit) make a single-level page table huge (millions of entries, too big for one frame).
- Fix: **page the page table itself** — a two-level (or more) scheme with an outer page table pointing to inner page table pages. Trade-off: MORE memory accesses per translation (one per level + 1 for data), but manageable table size.
- Page-size trade-off: bigger pages → smaller page table but MORE internal fragmentation; smaller pages → less internal fragmentation but LARGER page table. Optimal page size balances both costs.

- **Inverted Page Table:** one entry PER PHYSICAL FRAME (not per logical page) storing which process+page occupies it → table size = number of frames (not pages), saving huge amounts of memory when address spaces are large, at the cost of slower lookups (needs associative/hashed search).

7.6 Segmentation

- Supports the **user's logical view** of a program (code segment, stack segment, data segment — variable-sized, meaningful units) — contrast directly with paging's **physical**, fixed-size, programmer-invisible division.
- **TRAP: Segmentation = logical/variable-size (user view). Paging = physical/fixed-size (system view).** This exact contrast is a classic recall question.
- Logical address = (**segment number, offset**). Segment table entry = (base, limit) per segment; offset must be $< \text{limit}$ or it traps to the OS.

Physical Address = Segment Base + Offset (valid only if $0 \leq \text{Offset} < \text{Segment Limit}$)

- Segmentation suffers **external fragmentation** (segments are variable-sized, like variable contiguous allocation).

Worked Example (Segment/Limit Check)

Limit/Relocation table: P0(limit=500, reloc=1200), P1(limit=275, reloc=550), P2(limit=212, reloc=880), P3(limit=420, reloc=1400), P4(limit=118, reloc=200).
Requests: P0 offset 450 → $450 < 500$ valid → PA = $1200 + 450 = 1650$.

P1 offset 300 → $300 \geq 275$ → **illegal (trap)**.

P2 offset 210 → $210 < 212$ valid → PA = $880 + 210 = 1090$.

P3 offset 450 → $450 \geq 420$ → **illegal (trap)**.

P4 offset 80 → $80 < 118$ valid → PA = $200 + 80 = 280$.

7.7 Segmentation with Paging (Hybrid)

- Each segment is itself divided into pages — combines segmentation's logical/user-view benefit with paging's no-external-fragmentation benefit.
- Similar in spirit to multilevel paging, but segments (unlike the fixed outer-table chunks of multilevel paging) can vary in size according to logical program structure.

Questions

Q1. [Trap] Internal fragmentation is characteristic of which allocation scheme, and external fragmentation of which?

A. Internal→variable partitioning, External→fixed partitioning B. Internal→fixed-size schemes (e.g. paging), External→variable-size contiguous schemes C. Both always occur together D. Neither occurs in modern OSes

Answer: B. Why: Fixed-size blocks (paging, fixed partitions) waste space inside an allocated block (internal); variable-size contiguous allocation leaves scattered unusable gaps between blocks (external).

Q2. [Numerical] A system has logical address space of 2^{20} bytes and page size 2^{12} bytes. How many bits are used for the page number?

A. 8 B. 12 C. 20 D. 32

Answer: A. Why: Page number bits = $m - d = 20 - 12 = 8$ bits (offset uses the remaining 12 bits).

Q3. [Numerical] TLB access = 10 ns, memory access = 90 ns, hit ratio = 90%. Find the Effective Access Time.

A. 90 ns B. 100 ns C. 109 ns D. 190 ns

Answer: C. Why: $EAT = 0.9 \times (10 + 90) + 0.1 \times (10 + 90 + 90) = 0.9 \times 100 + 0.1 \times 190 = 90 + 19 = 109$ ns.

Q4. [Trap] Which statement correctly distinguishes segmentation from paging?

A. Segmentation is physical and fixed-size; paging is logical and variable-size B. Segmentation reflects the user's logical view with variable-size units; paging is a physical, fixed-size, system-level division C. Both are always identical in modern OS design D. Paging suffers external fragmentation; segmentation does not

Answer: B. Why: Segmentation exposes the program's natural logical structure (segments of varying size); paging is a physical bookkeeping mechanism using fixed-size frames invisible to the programmer.

Q5. [Numerical] Using the worked segmentation example (P2: limit=212, relocation=880), is a request for offset 212 valid, and why?

A. Valid, physical address = 1092 B. Invalid — offset must be strictly less than the limit C. Valid, physical address = 880 D. Invalid — relocation register is too large

Answer: B. Why: The check requires $\text{offset} < \text{limit}$; $\text{offset} = 212$ equals the limit exactly, which fails the strict inequality, so it traps as illegal.

Q6. Why does multilevel paging use more memory accesses per address translation than single-level paging?

A. It doesn't; multilevel paging is always faster B. Each additional page table level requires its own memory access before reaching the actual data C. Multilevel paging removes the need for a page table entirely D. It only affects TLB size, not access count

Answer: B. Why: Address translation must walk each level (outer table → inner table → data), adding one memory access per level compared to single-level paging.

Q7. An Inverted Page Table's size is primarily determined by:

A. The number of logical pages in the largest process B. The number of physical frames in the system C. The number of processes running D. The TLB size

Answer: B. Why: It has exactly one entry per physical frame (recording which process/page occupies it), so its size scales with physical memory, not with per-process logical address space.

Q8. [Numerical] Using Weighted/Proportional frame allocation: 3 processes of sizes 20K, 30K, 50K share 40 available frames. How many frames does the 30K process get?

A. 8 B. 10 C. 12 D. 15

Answer: C. Why: $a(i) = (s(i)/S) \times m = (30/100) \times 40 = 12$.

Q9. [Trap] A page fault occurs when:

A. A page is accessed and found in the TLB B. A process accesses a page marked invalid (not currently in main memory) C. The CPU cache misses D. The relocation register overflows

Answer: B. Why: The valid-invalid bit scheme marks pages not currently resident in memory as invalid; accessing one triggers a page fault, causing the OS to fetch it from disk.

5.8 Virtual Memory & Page Replacement

8.1 Virtual Memory & Demand Paging

- **Virtual memory** lets a process execute even if it (or the sum of all running processes) is larger than physical memory — only needed pages are resident.
- **Pure Demand Paging:** never bring a page into memory until it is actually referenced (process starts with zero pages loaded, faults its way to a working set).
- Advantages: processes can exceed physical memory size; more processes fit in memory simultaneously (higher multiprogramming); enables easy memory sharing.
- Disadvantages: complex to implement; can cause severe performance loss if mismanaged (**Thrashing**, see §8.5).
- **Valid-invalid bit** (per page-table entry): valid = page is legally in the process's address space AND currently in memory; invalid = either not part of the address space at all, or valid but currently only on disk.

Page Fault Handling Steps

1. Trap to OS (invalid page referenced)
2. Check if reference is valid (illegal → abort) or just not-yet-loaded
3. Find a free frame (or select a victim to evict, if none free)
4. Schedule a disk read to bring the required page into that frame
5. Update the page table (mark valid, set frame number)
6. Restart the instruction that caused the fault

Effective Access Time (Demand Paging)

$$EAT = (1 - p) \times ma + p \times \text{page_fault_service_time} \quad [p = \text{page fault probability, } ma = \text{memory access time}]$$

Worked Example

$ma = 100 \text{ ns}$, page fault service time = $8 \text{ ms} = 8,000,000 \text{ ns}$, $p = 0.001$ (0.1%).

$EAT = (0.999 \times 100) + (0.001 \times 8,000,000) = 99.9 + 8000 = \approx \mathbf{8100 \text{ ns } (\sim 8.1 \mu\text{s})}$.

- **TRAP:** Even a TINY page-fault probability (0.1%) dominates EAT because the fault service time is $\sim 100,000\times$ the normal memory access time — this is why minimizing page faults matters so much.

8.2 Page Replacement — Why & The Dirty/Modify Bit

- When no free frame exists, the OS must evict ("replace") a victim page — if it has been modified since loading (**dirty/modify bit set**), it must be written back to disk first (costly); if unmodified, it can simply be discarded (already matches the disk copy).

8.3 Page Replacement Algorithms — Compared on ONE Reference String

Reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2 with **3 frames** (hand-traced below; each algorithm operates on the identical string so results are directly comparable):

```
FIFO -> 10 page faults  (evict OLDEST-loaded page)
LRU  -> 9 page faults  (evict LEAST-RECENTLY-used page)
OPT  -> 7 page faults  (evict page used FARTHEST in the future / never again)
```

- **FIFO:** simple (just a queue of load order), but can suffer **Belady's Anomaly** — increasing the number of frames can, for some reference strings, *increase* the fault count (a uniquely FIFO-family pathology).
- **Optimal (OPT):** replace the page not needed for the longest future time — provably the **LOWEST** possible fault rate for a given frame count, but **impossible to implement** (needs future knowledge); used only as a theoretical benchmark for comparison.
- **LRU:** replace the page unused for the longest PAST time — a practical backward-looking approximation of OPT. Implemented via per-page **counters** (timestamp on each reference, evict smallest) or a **stack** (move referenced page to top; LRU page is always at the bottom).
- LRU and OPT both belong to the class of **stack algorithms**: the set of resident pages for n frames is always a subset of the set for $(n+1)$ frames — this property is exactly why stack algorithms **never** suffer Belady's Anomaly (a guarantee FIFO does not have).
- **TRAP:** Belady's Anomaly (more frames $\Rightarrow$ possibly more faults) is a FIFO-specific risk — LRU and OPT are immune to it by construction (stack property).

8.4 Counting-Based Page Replacement

Algorithm	Rule	Weakness
LFU (Least Frequently Used)	Evict the page with the smallest reference count	A page used heavily early then abandoned keeps a high count and wrongly stays resident
MFU (Most Frequently Used)	Evict the page with the HIGHEST count (argues a low-count page was just brought in and will be used soon)	Counter-intuitive; rarely used in practice

- Neither LFU nor MFU is common in real systems — both are expensive to implement and approximate OPT poorly compared to LRU.

8.5 Frame Allocation, Working Set & Thrashing

- **Equal allocation:** every process gets $\text{Total_Frames} / N$, regardless of size — simple but ignores actual need.
- **Proportional/Weighted allocation:** $a(i) = (s(i) / S) \times m$ where $s(i)$ =process i 's size, S =sum of all sizes, m =total frames (see Ch.7 §7 worked example — same formula).
- **Working Set Model:** the set of pages a process actively references within its most recent Δ (window of references) — approximates the process's current "locality." Too small $\Delta \rightarrow$ misses the true locality; too large $\Delta \rightarrow$ overlaps multiple localities and over-allocates.
- **Thrashing:** system spends more time servicing page faults than executing useful work. Root cause: too many processes / too little memory per process (multiprogramming pushed too high) $\rightarrow$ each process's allocated frames fall below its working-set size $\rightarrow$ constant faulting $\rightarrow$ CPU utilization drops $\rightarrow$ OS (misreading low CPU utilization as "room for more work") admits MORE processes $\rightarrow$ makes it worse (a vicious cycle).

- **TRAP:** The fix for thrashing is to **DECREASE** the degree of multiprogramming (suspend/swap out some processes), NOT increase it — a classic exam trap since low CPU utilization intuitively (but wrongly) suggests "add more processes."

8.6 Local vs Global Replacement

Feature	Local Replacement	Global Replacement
Victim pool	Only the faulting process's OWN frames	ANY frame in the system, even another process's
Isolation	Process performance depends only on its own paging behavior	One process can be affected by another's paging behavior
Frame count per process	Fixed	Can grow/shrink dynamically
Common in practice?	Less common	More common — generally better system-wide throughput

Questions

Q1. [Numerical] For the reference string 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2 with 3 frames, how many page faults does FIFO produce?

A. 7 B. 9 C. 10 D. 13

Answer: C. **Why:** Hand-tracing FIFO (evict oldest-loaded on each miss) over all 13 references with 3 frames yields exactly 10 faults (verified step by step).

Q2. [Numerical] For the same reference string and frame count, how many faults does the Optimal algorithm produce?

A. 5 B. 7 C. 9 D. 10

Answer: B. **Why:** OPT always evicts the page needed farthest in the future; tracing this string yields 7 faults — the lowest possible for 3 frames, confirming $\text{OPT} \leq \text{LRU} \leq \text{FIFO}$ here ($7 \leq 9 \leq 10$).

Q3. [Trap] Which page replacement algorithm(s) can exhibit Belady's Anomaly (more frames → possibly more faults)?

A. LRU only B. OPT only C. FIFO D. Both LRU and OPT

Answer: C. **Why:** FIFO lacks the "stack property"; LRU and OPT are stack algorithms and are provably immune to Belady's Anomaly.

Q4. [Numerical] $m_a = 200$ ns, page fault service time = 10 ms, page fault rate $p = 0.0005$. Find EAT (approx).

A. 200 ns B. 5200 ns C. 5000 ns D. 10000 ns

Answer: B. **Why:** $\text{EAT} = (1 - 0.0005) \times 200 + 0.0005 \times 10,000,000 \approx 199.9 + 5000 = 5199.9 \approx 5200$ ns.

Q5. [Trap] A system shows falling CPU utilization as more processes are admitted, so the scheduler admits even MORE processes to compensate. This is:

A. The correct fix — always add more processes when CPU utilization drops B. Thrashing — the correct fix is to REDUCE the degree of multiprogramming, not increase it C. A sign the disk is too fast D. Impossible; CPU utilization always rises with more processes

Answer: B. **Why:** This describes classic thrashing — the intuitive "add more work" response worsens it; the real fix is to lower multiprogramming so each process regains enough frames for its working set.

Q6. Which two page-replacement algorithms belong to the "stack algorithm" class and can never suffer Belady's Anomaly?

A. FIFO and LFU B. LRU and OPT C. MFU and FIFO D. LFU and MFU

Answer: B. **Why:** LRU and OPT both satisfy the stack property (frames for n is a subset of frames for $n+1$), which structurally rules out Belady's Anomaly.

Q7. Global page replacement is generally preferred over local replacement in practice because:

A. It fully isolates processes from each other B. It generally yields greater overall system throughput, despite reducing a process's control over its own fault rate C. It uses less memory D. It eliminates thrashing entirely

Answer: B. **Why:** Global replacement lets frames flow to processes that need them most, usually improving system-wide throughput, at the cost of one process's performance being influenced by others' paging.

Q8. [Trap] The Least Frequently Used (LFU) algorithm can wrongly retain a page in memory when:

A. The page was never referenced at all B. The page was used heavily very early on but has since become unused, keeping a high count that resists eviction C. The frame table is empty D. The reference string is very short

Answer: B. **Why:** LFU relies purely on cumulative counts, so a page with an old burst of heavy use keeps a large count long after it stops being relevant — a known weakness addressed only by extra mechanisms like count-decay.

Q9. In the Working Set model, choosing too LARGE a value of Δ (the window size) causes:

A. The working set to miss part of the true locality B. The working set to overlap multiple distinct localities, reducing its accuracy C. Guaranteed thrashing D. No effect on accuracy

Answer: B. **Why:** A window that's too wide captures pages from more than one locality, making the "working set" a poor approximation of what the process is actually using right now.

5.9 Disk Scheduling Algorithms

9.1 Why Disk Scheduling Matters

- Seek time (moving the read/write head to the target track/cylinder) dominates total disk access time. Minimizing total head movement across a batch of pending requests improves throughput and response time.
- When the request queue has only one outstanding request, ALL algorithms behave identically (only one place to go) — they only differ meaningfully under load.

9.2 The Algorithms

Algorithm	Rule	Starvation risk
FCFS	Service requests strictly in arrival order	None, but poor seek-time optimization
SSTF (Shortest Seek Time First)	Always jump to the closest pending request	Yes — far-away requests can wait indefinitely (like SJF scheduling)
SCAN ("Elevator")	Sweep to one END of the disk servicing all requests along the way, then reverse	None, but requests just missed wait almost a full sweep
C-SCAN (Circular SCAN)	Sweep to one end servicing requests, then jump back to the START with no servicing on the return, then sweep again	None; more <i>uniform</i> wait times than SCAN
LOOK	Like SCAN but reverses at the last request in that direction, not the physical disk end	None
C-LOOK	Like C-SCAN but jumps back only to the lowest pending request , not the physical disk start	None

- **TRAP:** SSTF is essentially "SJF for disk scheduling" — same greedy-optimal-locally-but-starves-far-requests trade-off.
- **TRAP:** SCAN/C-SCAN go all the way to the **physical end/start of the disk** even if no request is waiting there (wasted movement); LOOK/C-LOOK fix this by only going as far as the last real request.

9.3 Worked Example (classic, verify by hand)

Disk: cylinders 0-199. Head starts at 53, initially moving toward higher numbers.
Pending requests: 98, 183, 37, 122, 14, 124, 65, 67

FCFS (service in given order: 98,183,37,122,14,124,65,67):

$$|98-53| + |183-98| + |37-183| + |122-37| + |14-122| + |124-14| + |65-124| + |67-65|$$

$$= 45 + 85 + 146 + 85 + 108 + 110 + 59 + 2 = 640$$

SSTF (always nearest next): 53→65→67→37→14→98→122→124→183

$$12 + 2 + 30 + 23 + 84 + 24 + 2 + 59 = 236 \quad (\text{better than FCFS, but NOT optimal — the theoretical best reordering for this exact set totals 208})$$

TRAP: SSTF is a good greedy heuristic but is **not guaranteed optimal** — it can still leave a shorter overall path on the table (as this example shows: 236 vs the true optimum 208).

SCAN (sweep up to disk end 199, then down to far end 0):

$$(199-53) + (199-14) = 146 + 185 = 331$$

C-SCAN (sweep up to 199, jump to 0 with no service, continue up to 37):

$$(199-53) + 199 + (37-0) = 146 + 199 + 37 = 382$$

LOOK (sweep up only to last request 183, reverse to last request 14):

$$(183-53) + (183-14) = 130 + 169 = 299$$

C-LOOK (sweep up to last request 183, jump directly to lowest request 14, continue up to 37):

$$(183-53) + (183-14) + (37-14) = 130 + 169 + 23 = 322$$

Algorithm	Total head movement (this example)
FCFS	640
SSTF	236
SCAN	331
C-SCAN	382
LOOK	299
C-LOOK	322

9.4 Choosing an Algorithm

- SSTF is common and a natural improvement over FCFS.

- SCAN/C-SCAN suit **heavy-load** systems better (less starvation risk under constant new requests) even though their raw totals may exceed LOOK/C-LOOK in a single static snapshot.
- LOOK/C-LOOK are generally more efficient than their SCAN/C-SCAN counterparts (avoid unnecessary travel to the physical disk ends) but have overhead in tracking the current furthest request.

Questions

Q1. [Numerical] Head at 50 (range 0-199), pending requests {20, 90, 60, 150}, servicing FCFS in that order. Total head movement is:

A. 180 B. 220 C. 260 D. 300

Answer: B. **Why:** $|20-50|+|90-20|+|60-90|+|150-60| = 30+70+30+90 = 220$.

Q2. Which disk scheduling algorithm is described as behaving like the "SJF of disk scheduling," greedily minimizing the NEXT seek but risking starvation of distant requests?

A. FCFS B. SSTF C. SCAN D. C-LOOK

Answer: B. **Why:** SSTF always services the geometrically closest pending request next — the same greedy, starvation-prone logic as Shortest Job First in CPU scheduling.

Q3. [Trap] Which algorithm(s) travel all the way to the physical ends of the disk (0 and max cylinder) even when no request is waiting there?

A. LOOK and C-LOOK only B. SCAN and C-SCAN C. FCFS only D. SSTF only

Answer: B. **Why:** SCAN/C-SCAN always sweep to the physical disk boundary before reversing/jumping; LOOK/C-LOOK instead reverse/jump at the last actual pending request, saving that wasted travel.

Q4. Why does C-SCAN generally provide more *uniform* wait times than plain SCAN?

A. It services fewer requests B. It never reverses direction, always sweeping the same way and jumping back without servicing, so no cylinder waits through two consecutive sweep-passes C. It always starts from cylinder 0 D. It is identical to FCFS

Answer: B. **Why:** Because C-SCAN treats the disk as circular (only ever sweeping in one direction and jumping back without servicing), a request just missed waits roughly one full cycle rather than the uneven wait pattern SCAN produces near the reversal point.

Q5. For the classic example (head=53, requests {98,183,37,122,14,124,65,67}, disk 0-199, initial direction increasing), which algorithm yields the SMALLEST total head movement?

A. FCFS B. SSTF C. SCAN D. LOOK

Answer: B. **Why:** SSTF totals 236 in this example, less than FCFS (640), SCAN (331), and LOOK (299) — though note SSTF is still not the true theoretical optimum (208) for this specific set.

Q6. [Conceptual] Under which condition do FCFS, SSTF, SCAN, and all other disk scheduling algorithms behave identically?

A. When the disk is defragmented B. When there is only one outstanding request in the queue C. When the head starts at cylinder 0 D. Never — they always differ

Answer: B. **Why:** With only one pending request, every algorithm has just one possible next move, so they all degenerate to the same behavior as FCFS in that instant.

5.10 File Management — Allocation, Free Space, Directories & RAID

10.1 File Allocation Methods

Method	Access supported	Advantage	Disadvantage
Contiguous	Sequential + Direct	Simple, fast, minimal seek between blocks	Severe external fragmentation ; hard to grow a file in place
Linked	Sequential only (efficient)	No external fragmentation; file grows freely as long as free blocks exist	Direct access is slow (must traverse pointer chain from the start); extra space per block for pointers
Indexed	Sequential + Direct	Direct access without external fragmentation (index block gathers all pointers in one place)	Wastes space for small files (index block overhead); needs linked/multilevel index schemes for very large files

- **TRAP:** Linked allocation's single point of failure — a single corrupted/lost pointer breaks the rest of the chain. Partial fixes: doubly linked lists, or storing filename+relative-block-number in each block (both add overhead).
- **Indexed allocation variants** for large files: Linked index blocks (chain of index blocks), Multilevel index (first-level index points to second-level index blocks, which point to data — extendable to more levels), Combined scheme (classic UNIX inode: ~12 direct pointers + single/double/triple indirect pointers — small files need no indirection at all, keeping common-case access fast).

10.2 Free Space Management

Technique	Mechanism	Tradeoff
Bit Vector (Bitmap)	1 bit per block: 1=free, 0=allocated	Simple, efficient at finding n consecutive free blocks, but must be entirely memory-resident to be fast — large disks need a large bitmap
Linked List	Free blocks chained via pointers; only head pointer cached in memory	No bitmap-storage overhead, but traversing the list requires a disk read per block — slow (rarely needed though, since usually only the FIRST free block is needed)

Bit vector size (bytes) = $\text{ceil}(\text{total_blocks} / 8)$
 Example: 1.3 GB disk, 512-byte blocks → ~2.7M blocks → bitmap needs > 332 KB
 Example: 1 TB disk, 4 KB blocks → ~256M blocks → bitmap needs 256 MB

10.3 File Access Methods

Method	How it works	Best for
Sequential	Read word-by-word from a base pointer that auto-advances	Simple streaming (tape-era design)
Direct / Relative	Jump straight to a block via its block number (fixed-length records)	Random-access needs; faster avg access, but complex to implement, raises security concerns (arbitrary block access)
Indexed Sequential	An index (like a book's index) on top of sequential storage — primary index → secondary index → data	Databases: fast direct lookup AND ordered sequential scans together

10.4 Directory Structures

Structure	Key idea	Limitation
Single-Level	All files in one directory	Name collisions once files/users grow
Two-Level	One User File Directory (UFD) per user, indexed by a Master File Directory (MFD)	Solves collisions, but users can't create their own sub-directories
Tree-Structured	Hierarchical folders; absolute or relative path names; 1 bit per entry marks file(0)/subdirectory(1)	Most common today
Acyclic-Graph	Allows sharing via links (shared files/subdirectories) but disallows true cycles	More flexible than a tree, but deletion needs care — reference counting or a link/entry-count so the actual file is deleted only when count = 0
General Graph	Links allowed even where they create cycles	TRAP: cycles can cause infinite loops during traversal — requires cycle detection (checked on every new link, computationally expensive) or garbage collection , or simply restrict links to files only (never subdirectories) to guarantee no cycle can form

- Path name = sequence of directory names from a starting point to the target file. **Absolute** = from root. **Relative** = from current directory.

10.5 File System Mounting & Sharing

- **Mounting:** attach an unmounted file system at a **mount point** (typically an empty directory) so it becomes part of the directory tree.
- **File Sharing — Access Rights:** a graded list from none up to deletion rights (None, Knowledge, Execution, Reading, Appending, Updating, Changing Protection, Deletion).
- **Simultaneous Access:** concurrent writers risk corruption; brute-force fix = whole-file locking when updating — but this raises **mutual exclusion and deadlock** concerns, same as process synchronization.

10.6 File System Implementation

- **On-disk structures:** Boot Control Block, Volume Control Block / **superblock** (volume-wide metadata), per-file **File Control Block (FCB)** / **inode** (owner, size, permissions, block locations).
- **In-memory structures:** mount table, system-wide + per-process open-file tables, directory-structure cache.

- **Layered implementation stack:** I/O Control (device drivers, lowest level) -> Basic File System (issues generic read/write-physical-block commands, manages buffers/cache) -> File Organization Module (maps logical blocks 0..N to physical blocks, tracks free space) -> Logical File System (manages metadata/FCB, everything about a file except its actual content).
- **Directory implementation:** Singly Linked List (simple, but O(n) linear search per operation — slow) vs **Hash Table** (key = hash of filename -> fast average-case O(1) lookup; used by most modern file systems).

10.7 Mass Storage & RAID

- Magnetic disks = bulk secondary storage (rotate ~7000 rpm); SSDs = no moving parts so no seek/rotational latency, faster but pricier per MB, erasable only a limited number of cycles (~100,000); Magnetic tape = large capacity, cheapest per byte, but slow/sequential-only — used for backup/archival, not random access.
- Disk attachment: Host-attached (direct I/O bus, e.g. SCSI/Fibre Channel), Network-Attached Storage/NAS (remote file-level access over TCP/IP, e.g. NFS/CIFS/iSCSI), Storage Area Network/SAN (private high-speed block-level network connecting servers+storage).

```
RAID 0: striping, NO redundancy      -> Read/Write: N (fastest), but ANY 1 disk failure loses all data
RAID 1: mirroring (full duplicate)   -> Read: N, Write: slowest disk speed; 50% storage efficiency
RAID 2: bit-level striping + Hamming ECC -> rarely used in practice
RAID 4: block striping + 1 dedicated parity disk -> Read: N-1, Write: bottlenecked at the single parity disk
RAID 5: block striping + parity DISTRIBUTED across all disks -> fixes RAID 4's bottleneck; tolerates 1 disk failure
RAID 6: RAID 5 + a SECOND independent parity block -> tolerates 2 simultaneous disk failures
```

- **TRAP: RAID 0 has ZERO fault tolerance** despite the "RAID" name — it exists purely for performance/capacity via striping, not redundancy; losing any one disk loses everything.
- RAID only detects/recovers from **disk failure** — it does NOT detect silent data corruption. Filesystems like ZFS add end-to-end checksums (collocated with data pointers) for that, and allocate disks into shared **pools** rather than fixed volumes/partitions.

Questions

Q1. Which file allocation method allows direct access to any block without suffering external fragmentation?

A. Contiguous B. Linked C. Indexed D. None of these

Answer: C. Why: Indexed allocation gathers all block pointers in one index block, allowing direct lookup via the i-th entry — any free disk block can satisfy new space requests, avoiding external fragmentation (unlike contiguous allocation).

Q2. [Trap] Why is direct access to the i-th block inefficient under linked allocation?

A. Linked allocation doesn't support direct access syntax B. The system must traverse pointers sequentially from the first block to reach the i-th block, each requiring a disk read C. Linked allocation has no index structure at all, making i undefined D. It is actually just as fast as contiguous allocation

Answer: B. Why: Each block only stores a pointer to the next one, so reaching block i means reading i blocks in sequence — there's no way to "jump" directly.

Q3. [Numerical] A disk has 800,000 blocks. How many bytes are needed for a bit-vector free-space map?

A. 8,000 bytes B. 80,000 bytes C. 100,000 bytes D. 800,000 bytes

Answer: C. Why: Bitmap size = $\text{ceil}(\text{total_blocks} / 8) = 800,000 / 8 = 100,000$ bytes.

Q4. Which directory structure solves the file-name-collision problem across multiple users but still does not allow users to create their own sub-directories?

A. Single-Level B. Two-Level C. Tree-Structured D. Acyclic-Graph

Answer: B. Why: Two-Level directories give each user a private UFD indexed via the MFD, eliminating cross-user name collisions, but each UFD is still flat (no nested sub-directories).

Q5. [Trap] What critical structural difference must a General Graph directory handle that an Acyclic-Graph directory does not?

A. Linked Graph directories cannot support links at all B. General Graph directories may contain cycles, requiring cycle detection or garbage collection to avoid infinite traversal loops C. General Graph directories are always faster to search D. There is no real difference between them

Answer: B. Why: Acyclic-Graph structures allow shared links but are constructed to disallow cycles; General Graph structures permit cycles, which can cause infinite loops during traversal unless actively detected/collected.

Q6. In directory implementation, which approach gives faster average-case file lookup — Singly Linked List or Hash Table — and why?

A. Singly Linked List, because it's simpler B. Hash Table, because it computes a key from the filename for near O(1) average lookup instead of a full linear scan C. Both are equally fast D. Neither supports fast lookup

Answer: B. Why: A hash table maps a filename's hash directly to its entry, avoiding the O(n) traversal a linked list requires for every operation.

Q7. [Trap] Which RAID level provides ZERO fault tolerance despite improving performance through striping?

A. RAID 1 B. RAID 0 C. RAID 5 D. RAID 6

Answer: B. Why: RAID 0 stripes data across disks with no parity or mirroring — it boosts speed/capacity but the failure of even one disk loses all data; "RAID" here is purely about performance, not redundancy.

Q8. Which RAID level uses block-level striping with parity distributed across ALL disks (rather than a single dedicated parity disk), avoiding a parity-disk write bottleneck?

A. RAID 2 B. RAID 4 C. RAID 5 D. RAID 1

Answer: C. Why: RAID 5 distributes parity blocks across all disks in the array, unlike RAID 4's single dedicated parity disk which becomes a write bottleneck; RAID 5 tolerates exactly 1 disk failure.

Q9. How many simultaneous disk failures can RAID 6 tolerate, and why?

A. 0, same as RAID 0 B. 1, same as RAID 5 C. 2, because it maintains two independent parity blocks D. Unlimited, since it uses full mirroring

Answer: C. Why: RAID 6 extends RAID 5 with a second, independently computed parity block, allowing reconstruction even if two disks fail simultaneously.

Q10. What is the main disadvantage of Indexed Allocation compared to Linked Allocation?

A. It cannot support direct access B. It suffers from external fragmentation C. It wastes more space on pointer/index overhead, especially for small files D. It cannot be used for large files at all

Answer: C. Why: Every file needs its own index block regardless of size, so small files pay a fixed overhead cost that linked allocation avoids (linked allocation only stores one pointer per data block already in use).

Q11. Why does RAID (by itself) fail to protect against silent data corruption, and what is the typical fix?

A. RAID always includes corruption checksums B. RAID only handles whole-disk failures; filesystems like ZFS add end-to-end checksums collocated with data pointers to detect/correct corruption C. RAID corruption protection depends on RAID 0 specifically D. There is no known fix for this problem

Answer: B. Why: RAID's redundancy is designed around recovering from a disk failing entirely — it doesn't verify that data itself hasn't silently degraded, which is why modern filesystems add their own checksum layer.

5.11 Unix OS — Kernel, Commands & Shell Scripting

11.1 Unix Overview & Architecture

- Developed at **AT&T Bell Labs**; key features: **Multiuser**, **Multitasking**, **Portable** (written in C), **Hierarchical file system** (root `/`), permission-based security (**rw**x for owner/group/other), powerful **Shell** for scripting/automation.
- Layered architecture (4 layers, innermost to outermost):

```

Layer 1: Hardware      (CPU, memory, storage, I/O devices)
Layer 2: Kernel        (core – bridges hardware & software; manages memory/process/file/device)
Layer 3: System Programs & Utilities (Shell, compilers, editors, grep/wc/date/who)
Layer 4: Application Layer (user/third-party apps – browsers, DBs, business apps)

```

- Kernel's 3-level internal view: **User Level** (programs request OS services) → **System Call & Library Interface** (bridge; C programs use library functions, Assembly can invoke syscalls directly) → **Kernel Level** (File Subsystem with `open()/read()/write()/close()/chmod()/chown()`, Buffering & Block I/O, Device Drivers, Process Control Subsystem) → **Hardware Level** (generates interrupts that the kernel handles).
- TRAP**: The kernel level is reached via **system calls/traps**, not direct hardware access by user programs — this indirection is what makes Unix secure/stable.

11.2 Unix File System & File Types

- "Everything is a file"** — even hardware devices are represented as special files (usually under `/dev`).
- Every file has a unique **inode** (metadata: size, permissions, disk location) and a unique **pathname**.
- File types and their `ls -l` first-character indicator:

Type	<code>ls -l</code> symbol	Description
Ordinary/Regular	-	User data, programs, text — cannot contain other files
Directory	d	Organizes files/subdirectories; branching point of the tree
Character Special	c	Transfers data 1 byte at a time (keyboard, terminal)
Block Special	b	Transfers data in fixed-size blocks (disks, SSDs)
Pipe (FIFO)	p	One-way data flow connecting two commands
Socket	s	IPC / client-server communication
Symbolic Link	l	Shortcut storing a path to another file; broken if target moves/deleted

- TRAP**: Deleting a **symbolic link** does NOT affect the original file; but if the **original** is deleted/moved, the symlink becomes a **broken link** — the dependency is one-directional.

11.3 Essential Unix Commands (Reference)

```

Navigation:      ls, cd, pwd, mkdir, rmdir
File Manipulation: cp, mv, rm, touch, cat
Text Processing: grep, sed, awk, sort, cut, wc
Text Editors:    vi/vim, nano
Permissions:     chmod, chown, ls -l (permission string rwxrwxrwx = owner|group|other)
Process Mgmt:    ps, kill, top, nice, bg, fg, jobs
Networking:      ping, netstat, ssh, scp
Sys Admin:       who, df, du, uname

```

- Permission string example: `-rwxr-xr--` → owner=read/write/execute, group=read/execute, other=read-only.

11.4 Shell & Shell Scripting Basics

- Shell** = command-line interpreter; accepts commands, interprets them, requests kernel services, displays results. Also manages scripts, I/O redirection, environment variables.
- A **shell script** = text file of commands executed automatically by the shell; starts with a **shebang** `#!/bin/bash`.
- Variables**: `name="Alice"` (no spaces around `=`); accessed with `$name`; case-sensitive; types = user-defined, environment, read-only, local.
- Special variables**: `$0` =script name, `$1,$2,...` =positional args, `$#` =argument count, `$*` =all args as one string, `$@` =all args as separate values, `$?` =exit status of last command (0=success, non-zero=failure).
- Reading input**: `read varname` (add `-s` for silent/password input).

Operators

```

Arithmetic: + - * / %          (via `expr $a + $b` OR modern =$((a + b)))
Relational: -eq -ne -gt -lt -ge -le (numeric comparison)
Logical:    && (AND) || (OR) ! (NOT)
String:     = (equal) != (not equal) -z (empty) -n (not empty)
File test:  -f (regular file) -d (directory) -r/-w/-x (permissions) -e (exists) -s (not empty)

```

- TRAP**: In `expr`, spaces around operators are REQUIRED and `*` must be escaped (`\*`) since `*` is a shell wildcard; the modern `=$((...))` syntax needs neither.

Control Flow

```

if [ cond ]; then ... fi
if [ cond ]; then ... else ... fi
if [ cond ]; then ... elif [ cond2 ]; then ... else ... fi
case $var in
    pattern1) commands ;;
    *) default ;;
esac

while [ cond ]; do ... done      # runs WHILE condition is true
until [ cond ]; do ... done     # runs UNTIL condition becomes true (opposite of while)
for var in list; do ... done    # fixed number of iterations
break      # exits the loop entirely
continue   # skips to next iteration

```

- **TRAP:** `while` loops **while true**; `until` loops **while false** (i.e., until it becomes true) — easy to mix up.

Functions

```

function_name() {
    commands
    return status_code    # 0-255 ONLY – an exit status, NOT a real return value
}
result=$(function_name) # capture "returned" data via echo + command substitution

```

- **TRAP:** Shell functions **cannot return actual values** via `return` (unlike C/C++) — `return` only yields an exit status (0–255). To get a computed value back, the function must `echo` it and the caller captures it with `$(...)`.

I/O Redirection

```

>  overwrite output to file
>> append output to file
<  read input from file

|  pipe output of one command as input to another

```

Questions

Q1. In `ls -l` output, a line starts with `d`. This entry is a:

A. Regular file B. Directory C. Symbolic link D. Socket

Answer: B. **Why:** `d` is the first-character indicator for a directory in `ls -l` output.

Q2. [Trap] If the original file that a symbolic link points to is deleted, what happens to the symbolic link?

A. It automatically deletes itself B. It becomes a broken/invalid link C. It becomes a regular file D. Nothing changes, it still works

Answer: B. **Why:** A symbolic link only stores a path reference; if the target is gone, the link is broken (dangling), while deleting the LINK itself never affects the original file.

Q3. Which shell loop construct executes its body repeatedly UNTIL a condition becomes true (i.e., while it is false)?

A. for B. while C. until D. case

Answer: C. **Why:** `until` is the logical opposite of `while` — it keeps looping as long as the condition evaluates false.

Q4. [Trap] A shell function uses `return 25` to try to send back the value 25 to the caller. What actually happens?

A. The caller receives 25 as a normal value B. This fails to compile C. It sets the exit status to 25, retrievable only via `$?` , not as a usable return value D. Shell functions cannot use `return` at all

Answer: C. **Why:** `return` in shell scripting only sets an exit status (0-255); to pass back computed data, the function must `echo` it and the caller captures it with command substitution `$(...)`.

Q5. Which file-test operator checks whether a file exists AND is a regular file (not a directory)?

A. `-e` B. `-d` C. `-f` D. `-s`

Answer: C. **Why:** `-f` specifically tests for a regular file; `-e` only checks existence (any type), `-d` tests for directory, `-s` tests non-empty.

Q6. The permission string `-rw-r--r--` on a file means:

A. Owner: read/write, Group: read-only, Other: read-only B. Owner: read-only, Group: read/write, Other: none C. It is a directory D. Everyone can execute it

Answer: A. **Why:** The 9 characters split into 3 groups of 3 (owner/group/other): `rw-` = read+write, `r--` = read-only, `r--` = read-only; leading `-` confirms it's a regular file.

Q7. What does the pipe operator `|` do in a shell command?

A. Redirects output to a file, overwriting it B. Appends output to a file C. Passes the output of one command as input to the next command D. Reads input from a file

Answer: C. **Why:** `|` chains commands together, feeding the first command's stdout into the second command's stdin.

5.12 Linux OS — Kernel, Job Scheduling & Memory Management

12.1 Linux Overview & Architecture

- **Linux** = free/open-source, Unix-inspired kernel (Linus Torvalds). A **distribution (distro)** = Linux kernel + bundled software/tools (Ubuntu, Fedora, Debian, CentOS/RHEL).
- Architecture (bottom to top): **Hardware** → **Kernel** → **System Libraries (glibc etc.)** → **Shell (Bash/Zsh/ksh/csh)** → **System Utilities/Applications**.
- Kernel responsibilities: process/memory/file-system/device management, security & access control, system calls, networking (TCP/IP stack).

Kernel Types (general OS classification, not just Linux)

Type	Design	Trade-off
Monolithic (Linux uses this)	All OS services run in kernel space	High performance (direct calls), but large/complex to maintain
Microkernel	Only essential services in kernel; rest in user space	More secure/modular, but IPC overhead reduces performance
Exokernel	Minimal abstraction; apps get near-direct hardware access	Max flexibility/performance, but much more application complexity
Hybrid	Combines monolithic + microkernel traits	Balances performance, security, modularity — used by many modern OSes

- **TRAP:** Linux's kernel is **Monolithic** (not microkernel) — despite this, it's modular via loadable kernel modules, which sometimes causes confusion.

12.2 Essential Linux Commands (Reference)

```

Directory Listing: ls, ls -l, ls -a
File Management:  cp, mv, rm, touch, mkdir, rmdir
Searching:        find, grep, locate
Permissions:      chmod, chown, ls -l
User Management:  useradd, passwd, usermod, who, whoami
Disk & Storage:   df, du, mount, umount, fdisk
Process Control:  kill, killall, systemctl
Process Monitoring: top, htop, ps aux
Sys Monitoring:  vmstat, dmesg, free -h
Networking:      ping, ifconfig/ip, netstat, ssh, scp
Compression:     tar, gzip, gunzip, zip, unzip

```

12.3 Job Scheduling — cron, at, batch, anacron

- **Job types:** One-time (runs once) vs Recurring (runs repeatedly at fixed intervals).
- **cron:** time-based scheduler for **recurring** jobs. **crond:** the background daemon that continuously checks schedules and actually executes them (cron = the mechanism/table; crond = the running service that reads it).
- **crontab** (cron table) — commands: `crontab -e` (edit), `crontab -l` (list), `crontab -r` (remove all), `crontab -u user -e/-l` (another user's jobs).

```

Crontab format: minute hour day-of-month month day-of-week  command
Example: 0 2 * * * /home/user/backup.sh -> runs daily at 2:00 AM

```

- **Special characters:** `*` = every value, `,` = list of values (e.g., `9,18` = 9 AM and 6 PM), `-` = range (e.g., `9-17` = every hour 9AM-5PM), `/` = step interval (e.g., `*/*5` = every 5 minutes).
- **at / atq / atrm:** for **ONE-TIME** future jobs. `at [time]` schedules it (submit with Ctrl+D); `atq` lists pending at-jobs (shows job IDs); `atrm job_id` cancels one.
- **batch:** one-time job, but runs whenever **system load** permits (load-based, not time-based) — contrast directly with `at` (time-based).
- **anacron:** designed for machines **not always powered on** (e.g., laptops) — if a scheduled job's time was missed (machine was off), anacron runs it once the system is back up; tracks last-execution time. Config file: `/etc/anacrontab`, format `period delay job-identifier command`.
- **TRAP:** **at = one-time, time-based** | **cron = recurring, time-based** | **batch = one-time, load-based** | **anacron = recurring, tolerant of downtime** — all four are commonly confused pairwise; the key differentiators are (one-time vs recurring) × (time-based vs load-based vs downtime-tolerant).

12.4 Linux Memory Management

- Responsibilities: track free/used memory, allocate/deallocate per process, manage virtual memory + paging + swapping, memory protection (one process cannot access another's memory — violation → **Segmentation Fault**).
- **Process memory layout** (regions): **Text/Code** (instructions) → **Data** (initialized globals/statics) → **BSS** (uninitialized/zero globals) → **Heap** (dynamic allocation, grows up) → **Memory Mapping** (shared libs) → **Stack** (function calls/locals, grows down).
- **Address translation:** CPU generates **Logical/Virtual Address** → **MMU** (Memory Management Unit) → **Physical Address** in RAM.
- **Page Table Entry (PTE) fields:** Frame Number, Present/Valid bit, Protection bits (R/W/X), Accessed bit, Dirty bit (modified).
- **Single-level page table:** simple, but can become huge for large address spaces. **Multi-level page table:** splits into levels — saves page-table memory (only needed portions allocated) at the cost of more lookups per translation.
- **TLB:** caches recent page-table lookups. TLB hit = fast (no page-table walk); TLB miss = slower (must consult page table).
- **Demand Paging & Page Fault:** load a page only when referenced; if not present → **Page Fault**. Handling steps: CPU references memory → page not in RAM → hardware raises page fault → OS validates the reference → OS finds/selects a free frame → loads page from disk → updates page table → restarts the interrupted instruction.

Page Replacement Algorithms (shared reference string: 7,0,1,2,0,3,0,4,2,3,0,3,2 — 4 frames)

Algorithm	Rule	Belady's Anomaly?
FIFO	Evict the oldest-loaded page	Yes — more frames can mean MORE faults
Optimal (OPT)	Evict the page used farthest in the future (benchmark only, not implementable)	No
LRU	Evict the Least Recently Used page (based on the past)	No
NRU	Uses Reference(R) + Modified(M) bits; 4 classes (00,01,10,11); evict from lowest-numbered non-empty class	No
Clock/Second-Chance	Circular pointer + reference bit; R=0 → evict; R=1 → give second chance (set R=0), advance pointer	Generally avoids it

- **TRAP: Belady's Anomaly** (more frames → more page faults) is a **FIFO-specific** pathology — LRU and Optimal belong to the "stack algorithm" class and never exhibit it.
- **Swappiness** (Linux kernel tunable, `/proc/sys/vm/swappiness`, current docs allow range **0–200**, default **60**): higher value = more eager to swap out pages vs reclaiming cache; it's a tuning weight, NOT a direct percentage of RAM to swap.
- **Swap Partition** (dedicated, fixed size, slightly faster, needs partition-level setup) vs **Swap File** (regular file inside a filesystem, resizable, easier to manage; need not be contiguous on disk).

12.5 Linux Kernel Memory Allocators

- **Buddy Allocator**: manages physical memory in power-of-2-sized page blocks; splits a larger block when a smaller one is needed; merges adjacent same-size free "buddy" blocks on free — reduces external fragmentation for physically-contiguous allocations.
- **SLAB Allocator**: caches pre-allocated kernel objects of the same type/size (a cache = multiple "slabs" of objects) — very fast alloc/dealloc by reusing freed objects, but has higher memory overhead (per-CPU caches).
- **SLOB Allocator**: simple first-fit free-list allocator for very small/embedded systems — low overhead but slow (linear search) and prone to external fragmentation; **removed from the kernel in Linux 6.8** [VERIFY exact version].
- **SLUB Allocator**: simpler, more scalable modern replacement for SLAB — **the default slab allocator in current Linux kernels**; less metadata, better multi-core scalability, supports slab merging.
- **TRAP**: Don't assume SLAB is still the default — modern Linux defaults to **SLUB**; SLOB has been removed entirely.

Questions

Q1. Linux's kernel architecture is classified as:

A. Microkernel B. Monolithic C. Exokernel D. Hybrid only

Answer: B. **Why:** All core OS services (process, memory, file, device management) run in kernel space in Linux — the defining trait of a monolithic kernel (though it supports loadable modules for modularity).

Q2. [Trap] A backup job must run exactly once, at 11 PM tonight only. Which Linux tool is appropriate?

A. cron B. at C. anacron D. crond

Answer: B. **Why:** `at` schedules one-time jobs for a specific future time; `cron` is for recurring jobs.

Q3. A laptop is frequently powered off overnight and misses its scheduled daily cron job. Which tool is designed to still run the missed job once the system restarts?

A. cron B. at C. anacron D. batch

Answer: C. **Why:** anacron tracks the last execution time and runs missed periodic jobs once the machine becomes available again — cron simply skips a missed window.

Q4. The crontab entry `* /15 * * * * /script.sh` runs the script:

A. Once every 15 hours B. Once every 15 minutes C. At 3:15 PM daily D. Only on the 15th day of the month

Answer: B. **Why:** `* /15` in the minute field means "every 15 minutes" (step interval), with all other fields as "every value."

Q5. [Trap] Which page replacement algorithm is specifically known for suffering from Belady's Anomaly (more frames can cause MORE page faults)?

A. LRU B. Optimal C. FIFO D. Clock

Answer: C. **Why:** FIFO uniquely exhibits Belady's Anomaly; LRU, Optimal, and Clock (as LRU/FIFO-improved approximations) do not.

Q6. In the Clock (Second-Chance) page replacement algorithm, when the pointer reaches a page with Reference bit R=1, what happens?

A. The page is immediately evicted B. The R bit is cleared to 0, the page is given a second chance, and the pointer moves on C. The page is moved to secondary storage immediately D. The algorithm terminates

Answer: B. **Why:** R=1 means recently used — the algorithm spares it, resets R to 0, and continues scanning, rather than evicting a recently-referenced page.

Q7. Which Linux kernel memory allocator is the current default for managing frequently-used kernel objects, having replaced both SLAB and SLOB?

A. Buddy Allocator B. SLAB C. SLOB D. SLUB

Answer: D. **Why:** SLUB is simpler and more scalable than SLAB and is the modern default; SLOB has been removed from the kernel.

Q8. A Linux system has `vm.swappiness` set to a very low value. This means the kernel will:

A. Swap aggressively even under mild memory pressure B. Prefer to avoid swapping, favoring cache reclaim instead C. Disable swap entirely, no exceptions D. Swap exactly that percentage of RAM at all times

Answer: B. **Why:** A lower swappiness value biases the kernel toward reclaiming page cache over swapping out process memory; it's a tuning weight, not a fixed percentage or hard on/off switch.

5.13 Distributed Systems

13.1 Basics

- **Distributed System:** independent computers, each with its own processor/local memory/**local clock** (no global clock), connected via a network, communicating **only via message passing** (no shared memory) — yet appearing to users as one single coherent system.
- **Distributed File System (DFS):** provides transparent access to files stored across multiple networked computers through one common interface.
- Process **events:** **Internal** (within a process), **Send** (message to another process), **Receive** (message from another process) — the building blocks used to reason about ordering/causality in distributed algorithms.

13.2 Distributed Mutual Exclusion

- Same 3 requirements as single-site critical sections: **Mutual Exclusion** (at most 1 process in CS), **Progress** (a requesting process eventually gets in), **Bounded Waiting/Fairness** (no indefinite wait) — but now must be achieved with NO shared memory, only messages.

Algorithm	Main Idea	Central Coordinator?	Key Feature
Centralized	A coordinator process grants/queues CS requests	Yes	Simple, but single point of failure + bottleneck
Ricart-Agrawala	Fully distributed; broadcast timestamped request to ALL processes, enter only after receiving replies from everyone	No	Request/Reply, timestamp-based
Suzuki-Kasami	A unique token circulates; holding it = permission to enter CS; uses sequence numbers to detect outdated requests	No	Token-based
Token-Based (general)	Token = the sole permission object	No	Must guard against token loss/duplication

- **Message complexity (Suzuki-Kasami):** if the requesting process already holds the token → **0 messages**; if the token must be requested → **~N messages** (broadcast request + token transfer). This "0 or N" framing is the standard exam-level fact.
- Token-based problems: **Token Loss** (processes may wait indefinitely — needs a regeneration mechanism), **Token Duplication** (breaks mutual exclusion if it happens), **Starvation** (possible with poor token-passing policy).
- **TRAP:** Centralized approach has the FEWEST messages per request but the WORST fault-tolerance (single point of failure) — token/Ricart-Agrawala trade more messages for no central dependency.

13.3 Distributed Deadlock

- **Distributed Deadlock:** processes at *different sites* wait indefinitely for resources held by others, forming a cycle that spans multiple machines.
- **Wait-For Graph (WFG):** represents which process is waiting for which other process, extended across sites (a global WFG must be assembled from local views).
- **Phantom Deadlock:** a **FALSE positive** — the system detects a deadlock that no longer actually exists, because network/message delays caused it to observe a **stale, inconsistent snapshot** of global state that *looks* like a cycle even though (e.g.) one of the transactions already released its resource.
- **TRAP:** Phantom deadlock is a problem **unique to distributed** deadlock detection — single-site deadlock detection (Ch.6) works on a single consistent, instantaneous state and never suffers this false-positive issue.

13.4 Agreement Problems

- **Consensus Problem:** all non-faulty processes must agree on a single common value, despite some processes possibly failing (crashing).
- **Byzantine Generals / Byzantine Agreement Problem:** agreement must be reached even when some processes send **arbitrary, contradictory, or malicious** messages (not just silently crashing) — a strictly harder problem than simple crash-fault consensus. [VERIFY] The classical result requires more than $3f$ processes (specifically $\geq 3f+1$) to tolerate f Byzantine-faulty processes.
- **TRAP:** Simple **Consensus** only needs to tolerate crash failures; **Byzantine Agreement** must additionally tolerate processes that actively lie/send conflicting messages — don't treat the two as identical difficulty.

13.5 Distributed Transactions & Two-Phase Commit (2PC)

- **Distributed Transaction:** a transaction that accesses data/resources at **multiple sites**; ACID properties (Ch.12) must still hold across all of them.
- **Two-Phase Commit (2PC):**

Phase 1 (Voting): Coordinator asks every participant "can you commit?"
Each participant replies YES or NO (and locks its part if YES).
Phase 2 (Commit/Abort): If ALL replied YES → coordinator broadcasts COMMIT.
If ANY replied NO → coordinator broadcasts ABORT.

- **TRAP:** 2PC is **blocking** — if the coordinator crashes AFTER Phase 1 but before sending the Phase-2 decision, participants who already voted YES **must wait** (they cannot unilaterally decide to commit or abort on their own) — this is 2PC's key weakness, which Three-Phase Commit (3PC) attempts to fix by adding a further non-blocking phase.
- Failure handling in distributed systems generally: **Checkpointing** (periodically save consistent state), **Rollback** (revert to a checkpoint after failure), **Message Logging** (replay messages to reconstruct lost state) — used together for failure recovery. **Replication** (keeping copies of data at multiple sites) improves availability but adds the complexity of keeping replicas consistent during distributed transactions.

Questions

Q1. Which distributed mutual exclusion approach has a single point of failure by design?

A. Ricart-Agrawala B. Suzuki-Kasami C. Centralized D. Token-based (general)

Answer: C. Why: The centralized approach relies on one coordinator process; if it fails, no process can be granted access to the critical section.

Q2. [Numerical/Conceptual] In the Suzuki-Kasami algorithm, if the requesting process already holds the token, how many messages are needed to enter the critical section?

A. N B. N-1 C. 0 D. 2N

Answer: C. Why: If the process already has the token, no request/reply messages are needed at all — it can enter immediately.

Q3. [Trap] A distributed deadlock detection algorithm reports a cycle in the global Wait-For Graph, but by the time this is detected, the deadlock no longer actually exists. This is called:

A. Livelock B. Phantom Deadlock C. Starvation D. Byzantine Failure

Answer: B. Why: Phantom deadlock is a false-positive caused by network delay creating a stale, inconsistent global snapshot — a problem specific to distributed (not single-site) deadlock detection.

Q4. The Byzantine Generals Problem is harder than simple crash-fault Consensus because:

A. It involves more processes always B. Faulty processes may send arbitrary/contradictory/malicious messages, not just crash silently C. It never requires agreement D. It only applies to single-site systems

Answer: B. Why: Byzantine faults include active, arbitrary misbehavior (not just stopping), which is strictly harder to tolerate than simple process crashes.

Q5. [Trap] In Two-Phase Commit, the coordinator crashes right after Phase 1 (Voting), where all participants voted YES. What is the participants' situation?

A. Each participant can independently decide to commit B. Each participant can independently decide to abort C. Participants must wait/block since they cannot unilaterally decide D. The transaction is automatically aborted

Answer: C. Why: This is 2PC's defining weakness — a coordinator failure after voting leaves "yes" voters blocked, unable to safely commit or abort on their own, since they don't know what other participants decided.

Q6. Which distributed mutual exclusion algorithm is based on broadcasting timestamped requests to all other processes and waiting for all replies before entering the critical section?

A. Centralized B. Ricart-Agrawala C. Suzuki-Kasami D. Token ring only

Answer: B. Why: Ricart-Agrawala is the classic fully-distributed, timestamp-based request/reply algorithm requiring replies from every other process.

Q7. Why is a global clock unavailable in a typical distributed system model?

A. Each computer has its own local clock and processor, with no shared memory or synchronized global time source B. Distributed systems don't use clocks at all C. All computers share one master CPU D. Message passing eliminates the need for time

Answer: A. Why: By definition, distributed system nodes are independent, each with a local clock; the absence of a global clock is precisely why event ordering (e.g., via logical clocks) becomes a core distributed-systems challenge.

6.1 DBMS Basics, Architecture & Languages

1.1 Data, Information & Database

- **Data** = raw facts/figures (qualitative or quantitative). **Information** = processed data placed in context. **Database** = an organized, electronically stored/accessed collection of data.
- **DBMS** = software that lets users/applications define, create, maintain, and control access to a database — solving the problems of raw "flat file" systems.

1.2 Problems with Traditional File Systems (why DBMS exists)

- **Data redundancy & inconsistency** — same fact stored in multiple files, updated in one but not another.
- **Difficulty in accessing data** — no query capability; needs a new program for every new access pattern.
- **Data isolation** — data scattered across incompatible file formats, hard to combine.
- **Integrity problems** — no built-in way to enforce rules (e.g., marks must be 0–100).
- **Atomicity problems** — a crash mid-operation leaves data in a half-updated (inconsistent) state.
- **Concurrent-access anomalies** — simultaneous edits by multiple users overwrite each other.
- **Security problems** — no fine-grained access control per user/role.
- **TRAP:** These 6 problems (redundancy, access difficulty, isolation, integrity, atomicity, concurrency + security) are a classic "list all problems of file-based systems" recall question — don't merge atomicity and concurrency, they are distinct failure modes (crash-related vs simultaneous-access-related).

1.3 Three-Schema Architecture (View of Data)

Level	Describes	Who works here
Physical/Internal	How data is actually stored on disk (lowest level)	DBA only
Logical/Conceptual	What data is stored + relationships (structure, entities, types)	Database designers
View/External	What a specific user sees (subset relevant to them)	End users

- **Data Independence:**
- **Logical Data Independence** — change the conceptual schema without changing external schemas/application programs.
- **Physical Data Independence** — change the internal schema (e.g., add an index, reorganize files) without changing the conceptual schema.
- **TRAP:** Physical data independence is *easier* to achieve than logical data independence, since application programs are typically tied more closely to the conceptual/logical structure than to physical storage details.

1.4 Schema vs Instance

- **Schema** = the database's fixed design/structure (like a variable's declared type) — defined once, rarely changes.
- **Instance** = the actual data present at a given moment (like a variable's current value) — changes constantly.

1.5 DBA Functions

Schema definition, storage & access-method design, schema/physical modification, granting authorization, routine maintenance (backups, monitoring disk space/performance).

1.6 OLTP vs OLAP

Feature	OLTP (Online Transaction Processing)	OLAP (Online Analytical Processing)
Goal	Day-to-day operations	Planning/decision-making
DB size	MB–GB	TB–PB
Data source	Live/current transactions	Historical, consolidated (Data Warehouse)
Query	Short, simple, fast	Complex, long-running
Updates	Continuous (insert/delete)	Periodic batch refresh
Design	Normalized (many small tables)	De-normalized (fewer, larger tables)
Example	Booking a flight, bank balance check	5-year sales trend analysis

1.7 Data Models (Types of Databases)

- **Hierarchical:** tree structure, each child has exactly one parent — rigid.
- **Network:** child can have multiple parents — graph-like, more flexible than hierarchical.
- **Relational:** data in tables (rows=tuples/records, columns=attributes) — most widely used; every value has a unique identity via its record.
- **Object-Oriented:** data stored as objects (attributes + methods) — suited to complex data (multimedia, large files).
- **TRAP:** Network model **generalizes** the hierarchical model by allowing multiple parents per child — don't describe them as unrelated models.

1.8 DBMS Languages

DDL (Data Definition Language)	-> CREATE, ALTER, DROP, TRUNCATE, RENAME	[defines schema]
DML (Data Manipulation Language)	-> SELECT, INSERT, UPDATE, DELETE	[manipulates data]
DCL (Data Control Language)	-> GRANT, REVOKE	[access control/permissions]
TCL (Transaction Control Lang.)	-> COMMIT, ROLLBACK, SAVEPOINT	[manages transactions]

- **DML sub-types:** High-level/non-procedural (SQL — states *what* data, not how) vs Low-level/procedural (must be embedded in a host language like C/Java, states *how* to get it record-by-record).
- **DBMS Interfaces:** stand-alone query interface (e.g., SQL*Plus), programmer interface (embedded SQL, SQLJ), procedure-call APIs (JDBC for Java, ODBC for others), database programming languages (PL/SQL), scripting (PHP/Python/JS for web apps), user-friendly menu/forms/graphics-based interfaces, mobile interfaces.
- **TRAP:** DDL statements (like CREATE TABLE) cause an **implicit commit** in most RDBMSs — they cannot be rolled back like DML statements.

1.9 Centralized vs Client-Server Architecture

- **Centralized DBMS:** DBMS software, hardware, application programs, and UI processing all combined in a single system; remote terminals may connect, but ALL processing happens at the central site.
- **Client-Server (2-tier):** Client (application/UI) directly talks to the Database server; simple, but business logic lives on the client.
- **Client-Server (3-tier):** Client → **Application/Business-logic server** → Database server. Separates presentation, logic, and data layers — better scalability, security, and maintainability; standard for web applications.
- **TRAP:** In 2-tier architecture, the application logic resides on the *client* side; in 3-tier, it's moved to a *middle application-server tier* — this shift is the key exam distinction.

Questions

Q1. Which of the following is NOT typically listed as a problem of traditional file-based systems?

- A. Data redundancy and inconsistency B. Difficulty in accessing data C. Built-in concurrency control D. Data isolation

Answer: C. **Why:** Traditional file systems *lack* built-in concurrency control — this is exactly the problem DBMS solves, not a feature file systems have.

Q2. Adding an index to speed up search, without altering the conceptual schema or any application program, is an example of:

- A. Logical data independence B. Physical data independence C. View-level modification D. Schema-instance separation

Answer: B. **Why:** Changing internal storage structures (like adding an index) while the conceptual schema stays fixed is precisely physical data independence.

Q3. [Trap] The "instance" of a database refers to:

- A. The fixed structural design of the database B. The actual data stored at a specific point in time C. The DBA's access rights D. The query language used

Answer: B. **Why:** Schema = fixed structure/design; Instance = the actual data content, which changes over time while the schema stays constant.

Q4. Which DBMS language category includes GRANT and REVOKE?

- A. DDL B. DML C. DCL D. TCL

Answer: C. **Why:** DCL (Data Control Language) manages permissions/access control via GRANT and REVOKE.

Q5. OLAP systems are best characterized by:

- A. Small database size, simple fast queries, continuous updates B. Large historical data, complex long-running queries, periodic batch updates C. Real-time transaction processing for daily operations D. Fully normalized schema design for speed

Answer: B. **Why:** OLAP serves analytical/decision-making needs over large historical datasets with complex queries and de-normalized, batch-refreshed designs — the opposite profile of OLTP.

Q6. In the Network data model compared to the Hierarchical model:

- A. A child record can have only one parent, same as hierarchical B. A child record can be linked to multiple parent records, unlike hierarchical C. There is no concept of parent-child relationships D. It only supports object-oriented data

Answer: B. **Why:** The Network model generalizes the hierarchical (strict tree) model by allowing multiple-parent linkages, forming a graph-like structure.

Q7. [Trap] In a 3-tier client-server architecture, business/application logic resides primarily in:

- A. The client tier B. The database tier C. A separate middle application-server tier D. It is split evenly and arbitrarily across all tiers

Answer: C. **Why:** 3-tier architecture's defining feature is separating presentation (client), business logic (application server), and data (database server) into distinct tiers, unlike 2-tier where logic sits with the client.

Q8. Which of the following operations, if executed, typically causes an implicit commit and cannot be rolled back in most RDBMSs?

- A. UPDATE B. DELETE C. CREATE TABLE (DDL) D. INSERT

Answer: C. **Why:** DDL commands like CREATE TABLE auto-commit in most database systems, unlike DML commands (INSERT/UPDATE/DELETE) which can be rolled back within a transaction.

6.2 ER Modeling

2.1 Core Concepts

- **ER Model** (Peter Chen, 1976): conceptual/high-level design tool bridging real-world requirements and technical schema, using **Entities, Attributes, Relationships**.
- **Entity**: a real-world object distinguishable from others (concrete: person, book; abstract: course, flight reservation).
- **Entity Set**: collection of entities sharing the same attributes → rectangle in ER diagram → **table** in relational model. (An individual *entity* = a row/tuple, not shown directly in an ER diagram, since ER models schema, not instances.)

2.2 Attribute Types

Type	Meaning	Example
Simple	Cannot be subdivided	Age
Composite	Divisible into sub-parts (→ its own separate column(s) in relational model)	Name → First+Last
Single-valued	One value per entity	Aadhar No
Multivalued	Multiple values possible (double ellipse; → separate table in relational model, with main table's PK as FK)	Phone numbers, Email
Stored	Physically stored	Date of Birth
Derived	Computed from other attributes	Age (from DOB)
Descriptive	Belongs to a <i>relationship</i> , not an entity	e.g., "date of enrollment" on a Student–Course relationship

- Null value meanings: **not applicable** (value doesn't exist) vs **unknown** (missing — exists but not recorded, or genuinely not known if it exists).
- **TRAP**: A multivalued attribute is NOT stored as a comma-separated list in one column — relational conversion always needs a **separate child table** (PK of parent + the multivalued attribute, combined, as the child's PK).

2.3 Relationships & Degree

- **Relationship** = association between ≥ 2 entity sets; diamond shape in ER diagram.

Degree = number of entity sets participating in a relationship
 Unary (1 entity set, self-referential, e.g. "supervises")
 Binary (2 entity sets) <- most common
 Ternary (3 entity sets)
 N-ary (n entity sets)

2.4 Cardinality Ratios & Participation

Ratio	Meaning
1:1	≤ 1 entity in A $\leftrightarrow$ ≤ 1 entity in B
1:N	1 entity in A $\leftrightarrow$ many in B; but each B $\leftrightarrow$ ≤ 1 A
N:1	mirror of 1:N
M:N	many $\leftrightarrow$ many, both sides unrestricted

- **Participation constraint**: does existence of an entity *depend on* the relationship?
- **Total participation** (min cardinality ≥ 1): EVERY entity in the set must participate.
- **Partial participation** (min cardinality = 0): some entities may not participate at all.
- **TRAP**: Total participation $\Leftrightarrow$ **minimum cardinality ≥ 1** , not maximum. A common wrong-answer trap swaps min and max cardinality when defining total/partial participation.

2.5 Strong vs Weak Entity Sets

- **Strong entity set**: has its own primary key; tuples fully distinguishable.
- **Weak entity set**: lacks enough attributes for its own PK; has a **discriminator/partial key** (distinguishes weak entities only *within* one owner). Double rectangle in ER diagram.
- Must be linked to a **strong (owner/identifying) entity set** via an **identifying relationship** (double diamond).
- Identifying relationship is always **many-to-one** (weak → owner) and the weak entity's participation is always **total**.
- **PK of weak entity = PK of owner entity $\cup$ discriminator (partial key) of the weak entity.**
- Why weak entities exist: reflect true logical dependency, auto-delete when owner is deleted (cascading), avoid redundancy/inconsistency that would occur if data were duplicated into the owner table.

2.6 ER-to-Relational Conversion Rules

Strong entity set	-> its own table
Weak entity set	-> its own table; PK = owner's PK + discriminator
Unary relationship	-> add FK column referencing own table's PK (no new table)
Binary 1:1	-> no new table; put PK of one side as FK on the other (prefer placing FK on the side with TOTAL participation)
Binary 1:N / N:1	-> no new table; put PK of the "1" side as FK on the "N" side
Binary M:N	-> NEW table; PK = combination of both sides' PKs (as FKs)
Ternary / N-ary	-> NEW table; PK = combination of all participating entities' PKs
Multivalued attribute	-> NEW table; PK = (owner's PK + the multivalued attribute)
Composite attribute	-> a separate column per simple sub-attribute (no new table)

- **TRAP:** Only **M:N (and ternary+)** relationships require a brand-new table. 1:1 and 1:N/N:1 relationships are folded into existing tables via a foreign key — a very common "how many tables are needed" numerical question hinges on this rule.

2.7 ER Traps

Trap	Cause	Fix
Fan Trap	Two or more 1:N relationships fan out from a single entity , making it impossible to determine which instance on one "many" side relates to which on the other	Restructure the model to represent the correct direct association
Chasm Trap	Two entities are connected only via a third entity with partial participation , so the model implies a path/relationship that doesn't actually exist for every occurrence	Add a direct relationship between the two entities

- **TRAP:** Fan Trap arises from **1:N branching out of one entity** (ambiguous many-side matching); Chasm Trap arises from **partial participation along an indirect path** (implied but missing connectivity) — these have different root causes and different fixes; don't conflate them.

2.8 ER Diagram — Pros/Cons

- **Advantages:** easy conversion to relational tables; simple, intuitive, minimal training needed.
- **Disadvantages:** loss of information content (can't express all constraints), limited constraint representation, overly complex for very small projects.

Questions

Q1. A weak entity set's primary key consists of:

- A. Only its own discriminator attribute B. Only the owner entity's primary key C. The owner entity's primary key combined with the weak entity's discriminator (partial key)
D. No primary key is needed for weak entities

Answer: C. Why: A discriminator alone only distinguishes weak entities within one owner; combining it with the owner's PK makes each weak-entity tuple globally unique.

Q2. [Trap] Total participation of an entity set in a relationship means:

- A. Maximum cardinality is one B. Minimum cardinality is zero C. Minimum cardinality is at least one — every entity must participate D. The relationship must be many-to-many

Answer: C. Why: Total participation requires min cardinality ≥ 1 (every entity occurrence participates); partial participation is min cardinality = 0.

Q3. Converting a Many-to-Many (M:N) binary relationship into the relational model requires:

- A. No new table; add FK to either side B. A new table whose PK is the combination of both entities' PKs C. Modifying only the "many" side with a foreign key D. Merging both entity tables into one

Answer: B. Why: M:N relationships cannot be represented via a simple FK on one side (would violate the many-to-many nature); they need a bridge/junction table keyed by the combined PKs.

Q4. [Trap] A single company site has many departments and employs many staff, but the ER diagram cannot express which specific staff work in which specific department because two separate 1:N relationships fan out from the "Site" entity. This is a:

- A. Chasm Trap B. Fan Trap C. Normalization anomaly D. Weak entity problem

Answer: B. Why: Multiple 1:N relationships emerging from one entity, making many-side associations ambiguous, is the defining signature of a Fan Trap.

Q5. A chasm trap is fixed by:

- A. Adding a discriminator attribute B. Creating a direct relationship between the two entities that need to be connected C. Converting the weak entity to a strong entity D. Removing the third entity entirely

Answer: B. Why: Since the implied path through the third (partially-participating) entity doesn't always exist, a direct relationship restores the missing connectivity.

Q6. A multivalued attribute like "Phone Numbers" for a Customer entity is converted to the relational model as:

- A. A single column storing comma-separated values B. A separate table with (Customer's PK + Phone Number) as its composite PK C. It is dropped since it cannot be represented D. Multiple separate tables, one per phone number

Answer: B. Why: Standard conversion rule: multivalued attributes become their own child table, PK = owner's PK combined with the attribute value.

Q7. For a binary 1:1 relationship where one side has total participation and the other has partial participation, best practice is to:

- A. Always place the FK arbitrarily on either side B. Place the FK on the side with total participation C. Create a new bridging table regardless D. It cannot be converted to relational model

Answer: B. Why: Placing the FK on the total-participation side avoids NULL foreign keys, since every entity on that side is guaranteed to have a matching related entity.

Q8. A ternary relationship (3 entity sets) is converted into the relational model by:

- A. Splitting it into three separate binary relationships always B. Creating one new table whose PK is the combination of all three participating entities' PKs C. Ignoring the third entity D. Making it a weak entity set

Answer: B. Why: Ternary (and higher, N-ary) relationships always require their own bridging table combining all participating entities' keys — same principle as M:N binary relationships, generalized.

Q9. [Conceptual] Why can't an ER diagram represent an individual entity directly?

- A. Because entities don't exist in ER diagrams at all B. Because an ER diagram models the schema (entity SETS/types), not instances/data, and an individual entity is an instance C. Because entities are only shown in the relational model D. Because entities require a discriminator

Answer: B. Why: ER diagrams are schema-level (conceptual) design tools; a specific entity (a particular row of data) is an instance, which belongs to the database instance, not the schema diagram.

6.3 Relational Model & Functional Dependencies

3.1 Relational Model Terminology

- **RDBMS** based on Edgar F. Codd's 1970 relational model.

Term	Meaning
Relation / Table	Set of tuples
Tuple / Row / Record	One entry in a relation
Attribute / Column / Field	A named property
Domain	Set of permissible atomic (indivisible) values for an attribute
Degree / Arity	Number of attributes (columns)
Cardinality	Number of tuples (rows)

3.2 Properties of a Relation

Atomic cell values; all values in a column same type; every row unique; unique table names in a schema; unique column names; **row order insignificant**; **column order insignificant**.

- **TRAP**: "Order of rows/columns doesn't matter" is a defining relational property — a query result presented in a different row order is still the same relation.

3.3 Update Anomalies (motivation for Normalization)

Anomaly	Description
Insertion	Can't record one fact without also inserting unrelated/unknown data (e.g., can't add a new branch until it has an employee)
Deletion	Deleting one fact accidentally destroys other, unrelated information (e.g., deleting the last employee of a branch deletes the branch's HOD info too)
Modification (Update)	The same fact is duplicated in many rows, so updating it means editing multiple rows (miss one → inconsistency)

- These anomalies arise from mixing multiple "ideas"/themes in one table — the fix is **normalization**, using **Functional Dependencies** as the analytical tool.

3.4 Functional Dependency (FD)

alpha $\rightarrow$ beta (read: "alpha functionally determines beta")
 Formal definition: for every pair of tuples t1, t2 in relation R,
 if t1[alpha] = t2[alpha] then t1[beta] = t2[beta]

- **alpha = determinant, beta = dependent.**
- **Trivial FD**: beta $\subseteq$ alpha (always holds automatically, e.g., AB $\rightarrow$ A).
- If **K $\rightarrow$ R** (K determines every attribute of R), K is a **super key** of R.
- FD is a property of the **schema** (a design-time rule), not of one particular instance — **TRAP**: you can only *disprove* an FD from a sample instance (find 2 tuples with matching α but different β); you can never *fully prove* an FD is a true schema-level rule just by checking one instance (the instance might just accidentally satisfy it).
- Quick instance-check shortcut: FD $\alpha \rightarrow \beta$ is **violated** iff you find two tuples with equal α but different β values.

3.5 Armstrong's Axioms (sound & complete inference rules)

Reflexivity: if Y subset-of X, then X $\rightarrow$ Y (trivial FDs)
 Augmentation: if X $\rightarrow$ Y, then XZ $\rightarrow$ YZ
 Transitivity: if X $\rightarrow$ Y and Y $\rightarrow$ Z, then X $\rightarrow$ Z

Derived (secondary) rules:

Union: if X $\rightarrow$ Y and X $\rightarrow$ Z, then X $\rightarrow$ YZ
 Decomposition: if X $\rightarrow$ YZ, then X $\rightarrow$ Y and X $\rightarrow$ Z
 Pseudotransitivity: if X $\rightarrow$ Y and WY $\rightarrow$ Z, then WX $\rightarrow$ Z
 Composition: if X $\rightarrow$ Y and Z $\rightarrow$ W, then XZ $\rightarrow$ YW

- **Sound**: every FD inferred via these axioms genuinely holds in any relation state satisfying F. **Complete**: repeated application eventually derives ALL FDs implied by F (the full closure F+).
- **TRAP**: Reflexivity, Augmentation, Transitivity are the **primary/basic** axioms; Union/Decomposition/Pseudotransitivity/Composition are **derived** from them — a common "which are the 3 basic Armstrong axioms" recall question.

3.6 Attribute Closure (X+)

X+ = set of ALL attributes functionally determined by X (directly or by inference)
 Algorithm:
 result = X
 repeat: for each FD (a $\rightarrow$ b) in F, if a subset-of result, then result = result UNION b
 until no change

- **Use 1 — Candidate key check**: X is a **super key** iff X+ = all attributes of R. X is a **candidate key** if X+ = R AND no proper subset of X also satisfies X+ = R (minimality).
- **Use 2 — Check if an FD X $\rightarrow$ Y is implied by F**: compute X+; if Y $\subseteq$ X+, the FD holds (is implied).

Worked Example

$R(A,B,C,D)$, $F = \{A \rightarrow B, B \rightarrow C, AB \rightarrow D\}$. Find A^+ .

$A^+ = \{A\} \rightarrow$ apply $A \rightarrow B$: $\{A,B\} \rightarrow$ apply $B \rightarrow C$: $\{A,B,C\} \rightarrow$ apply $AB \rightarrow D$ ($AB \subseteq$ current set): $\{A,B,C,D\}$.

$A^+ = \{A,B,C,D\} =$ all of $R \Rightarrow A$ is a super key; since no smaller subset of $\{A\}$ exists, A is a candidate key.

3.7 Equivalence of Two FD Sets

F_1 and F_2 are equivalent $\Leftrightarrow F_1^+ = F_2^+$ (i.e., $F_1 \models F_2$ AND $F_2 \models F_1$)

Check by computing, for every FD in F_2 , whether its LHS-closure (under F_1) contains its RHS — and vice versa.

3.8 Minimal Cover / Canonical Cover

- Minimal cover** = an equivalent FD set with **no redundancy**: (1) every RHS is a single attribute, (2) no extraneous attribute on any LHS, (3) no entire FD is redundant (derivable from the others).

Procedure:

- Decompose: split every FD's RHS into single attributes (via Decomposition rule).
- Remove extraneous LHS attributes: for FD $(AC \rightarrow B)$, test if A alone already determines B (i.e., is B in $(A)^+$ computed using the current FD set?); if yes, C was extraneous — drop it.
- Remove redundant FDs: for each FD, remove it and check if its RHS is still derivable from LHS using the REMAINING FDs; if yes, the FD itself was redundant — drop it permanently.

- TRAP:** A minimal cover is **not always unique** — different valid orders of eliminating redundancy can yield different (but equally minimal, same-size) covers.

Worked Example (fully verified)

$F = \{AC \rightarrow BD, A \rightarrow C, B \rightarrow C, D \rightarrow C\}$ on $R(A,B,C,D)$.

Step 1 (split RHS): $AC \rightarrow B, AC \rightarrow D, A \rightarrow C, B \rightarrow C, D \rightarrow C$.

Step 2 (extraneous LHS): Test C in " $AC \rightarrow B$ ": compute A^+ using the full set $\rightarrow A \rightarrow C$ gives C , then $AC \rightarrow B$ (LHS now satisfied) gives B , then $AC \rightarrow D$ gives $D \Rightarrow A^+ = ABCD$. Since $B \in A^+$, C was extraneous $\Rightarrow$ replace $AC \rightarrow B$ with **$A \rightarrow B$** . Same logic makes $AC \rightarrow D$ become **$A \rightarrow D$** .

Set now: $\{A \rightarrow B, A \rightarrow D, A \rightarrow C, B \rightarrow C, D \rightarrow C\}$.

Step 3 (redundant whole FDs): Test $A \rightarrow C$: compute A^+ using the OTHER four FDs $\{A \rightarrow B, A \rightarrow D, B \rightarrow C, D \rightarrow C\} \rightarrow A \rightarrow B$ gives B , $A \rightarrow D$ gives D , $B \rightarrow C$ gives $C \Rightarrow A^+ = ABCD$ already contains $C \Rightarrow$ **$A \rightarrow C$ is redundant, remove it**. Testing $A \rightarrow B, A \rightarrow D, B \rightarrow C, D \rightarrow C$ similarly shows none of them are redundant.

Final minimal cover = **$\{A \rightarrow B, A \rightarrow D, B \rightarrow C, D \rightarrow C\}$** .

Questions

Q1. In a relation $R(A,B,C,D,E)$, the number of columns (5) is called its:

A. Cardinality B. Domain C. Degree/Arity D. Instance

Answer: C. **Why:** Degree/Arity = number of attributes (columns); Cardinality = number of tuples (rows).

Q2. [Trap] Deleting the only employee of a branch also erases the branch's name and HOD info from an unnormalized combined table. This is an example of:

A. Insertion anomaly B. Deletion anomaly C. Modification anomaly D. Referential integrity violation

Answer: B. **Why:** Losing unrelated information (branch details) as an unintended side-effect of deleting a specific record (the employee) is precisely a deletion anomaly.

Q3. [Numerical] Given $R(A,B,C,D)$, $F = \{A \rightarrow B, B \rightarrow C, AB \rightarrow D\}$. What is A^+ ?

A. $\{A,B\}$ B. $\{A,B,C\}$ C. $\{A,B,C,D\}$ D. $\{A\}$

Answer: C. **Why:** $A \rightarrow B$ gives B ; $B \rightarrow C$ gives C ; $AB \rightarrow D$ (AB now available) gives $D \Rightarrow A^+ = ABCD$.

Q4. [Conceptual] Which three are Armstrong's PRIMARY (basic) axioms?

A. Union, Decomposition, Composition B. Reflexivity, Augmentation, Transitivity C. Pseudotransitivity, Union, Reflexivity D. Augmentation, Composition, Decomposition

Answer: B. **Why:** Reflexivity, Augmentation, and Transitivity are the three basic Armstrong axioms; Union, Decomposition, Pseudotransitivity, and Composition are all derivable secondary rules.

Q5. [Trap] A functional dependency $X \rightarrow Y$ that appears to hold on a specific sample instance of a relation:

A. Is proven to be a true schema-level FD B. May simply be a coincidence of that particular instance's data, not a guaranteed schema rule C. Can never be a valid FD D. Automatically becomes a candidate key

Answer: B. **Why:** An FD is a schema-level constraint; a sample instance can only *disprove* an FD (by finding matching- α /different- β tuples) — satisfying it in one instance doesn't prove it holds for all legal instances.

Q6. [Numerical] Find the minimal cover for $F = \{AC \rightarrow BD, A \rightarrow C, B \rightarrow C, D \rightarrow C\}$ on $R(A,B,C,D)$.

A. $\{A \rightarrow B, A \rightarrow C, B \rightarrow C, A \rightarrow D\}$ B. $\{A \rightarrow B, A \rightarrow D, A \rightarrow C, B \rightarrow C\}$ C. $\{A \rightarrow B, A \rightarrow D, A \rightarrow C, B \rightarrow C\}$ D. $\{A \rightarrow B, A \rightarrow D, D \rightarrow C, B \rightarrow C\}$

Answer: D. **Why:** After decomposing RHS and removing the extraneous C from $AC \rightarrow B/AC \rightarrow D$ (both become $A \rightarrow B, A \rightarrow D$), the redundant FD $A \rightarrow C$ is eliminated since A already reaches C via $A \rightarrow B \rightarrow C$, leaving the minimal set $\{A \rightarrow B, A \rightarrow D, D \rightarrow C, B \rightarrow C\}$.

Q7. If X^+ (closure of X) equals the full attribute set of R , and no proper subset of X also has this property, then X is:

A. A foreign key B. A candidate key C. A trivial dependency D. A weak entity's discriminator

Answer: B. **Why:** $X^+ = R$ makes X a super key; minimality (no smaller subset also works) upgrades it specifically to a candidate key.

Q8. Two FD sets F_1 and F_2 are said to be equivalent when:

A. They have the same number of FDs B. $F_1^+ = F_2^+$ (their closures are identical) C. F_1 is a subset of F_2 D. They apply to different relations

Answer: B. **Why:** Equivalence is defined purely by identical closures, regardless of how many or which individual FDs each set explicitly lists.

6.4 Keys & Integrity Constraints

4.1 Types of Keys

- **Super Key:** any set of attributes X such that X^+ (closure) = all attributes of R . Uniquely identifies each tuple, but may contain extra/redundant attributes.
- **Candidate Key:** a *minimal* super key (no proper subset is also a super key). Also called minimal super key.
- **Prime attribute:** any attribute that is part of AT LEAST ONE candidate key. **Non-prime:** part of none.
- **Primary Key:** one candidate key chosen by the DBA to uniquely identify tuples; cannot be NULL; at most one PK per table.
- **Alternate Key:** candidate key(s) NOT chosen as primary key.
- **Composite Key:** a key made of more than one attribute (concatenated key).
- **Secondary Key:** an attribute (not necessarily unique) used to speed up search/retrieval — unlike PK, values need not be unique.

For a relation with n attributes, if EVERY attribute participates in some minimal super key in the "all attributes are prime" edge case, max possible super keys = $2^n - 1$ (any non-empty subset works only when the FULL attribute set is the sole candidate key, i.e., no smaller subset determines all attributes).

TRAP: "Super key" and "Candidate key" are often confused — every candidate key IS a super key, but not every super key is minimal (hence not every super key is a candidate key). The primary key is always a candidate key, never just any super key.

4.2 Finding All Candidate Keys (given a set of FDs)

1. Identify attributes that **never appear on the RHS of any FD** — these are "source-only" attributes and MUST belong to every candidate key (nothing else can ever derive them).
2. Compute the closure of this mandatory set. If it already equals all attributes of R , it IS the (unique) candidate key.
3. If not, try adding one attribute at a time (preferring attributes not already covered) and recompute closure until it covers all attributes, checking minimality by removing each added attribute and confirming closure shrinks.

Worked Example

$R(W,X,Y,Z)$, FDs: $X \rightarrow W$, $WZ \rightarrow XY$, $Y \rightarrow WXZ$

Attributes never on RHS of any FD: none obviously excluded — but check Y : $Y \rightarrow WXZ$, so Y alone gives $Y^+ = \{W,X,Y,Z\}$ = all attributes $\rightarrow$ **Y is a candidate key** by itself.

Check X : $X^+ = \{X,W\}$ (from $X \rightarrow W$ only) — incomplete, not a key alone.

So here Y is one candidate key; checking further combinations is needed for completeness, but Y alone already qualifies as minimal (single attribute, cannot be smaller).

4.3 Foreign Key & Referential Integrity

- **Foreign Key (FK):** an attribute (or set) in one table that refers to the candidate key (usually PK) of the same or another table, representing a relationship.
- **Referential Integrity constraint:** every non-null FK value must match an existing candidate-key value in the referenced table — no "dangling" references allowed.
- **Entity Integrity constraint:** Primary Key attributes can NEVER be NULL (distinct from referential integrity, which governs FK values).
- **On-delete / On-update actions** when the referenced row changes/is removed:

Action	Behavior
CASCADE	Automatically delete/update the dependent (child) rows too
SET NULL	Set the FK value in child rows to NULL
RESTRICT / NO ACTION	Reject the delete/update if any child row still references it

Worked Example — ON DELETE CASCADE

Table $T(A, B)$ where B is a self-referencing FK to candidate key A (e.g., an "employee $\rightarrow$ manager" style hierarchy), ON DELETE CASCADE.

Rows: (1,2), (2,3), (3,5), (5,9), (6,2). Delete the row where $A=3$.

Step 1: Delete (3,5) directly ($A=3$ matched).

Step 2 (cascade): Any row whose $B=3$ must also cascade-delete $\rightarrow$ (2,3) is removed.

Step 3 (cascade continues): Now $A=2$ is gone; any row whose $B=2$ cascades $\rightarrow$ (1,2) and (6,2) are removed.

Step 4: Now $A=1$ gone; any row with $B=1$? None. Cascade stops.

Total removed: (3,5), (2,3), (1,2), (6,2) = 4 rows. Remaining: (5,9) only, i.e., 1 row out of the original 5.

TRAP: Cascade deletes can ripple recursively through self-referencing FKs — always check every newly-deleted key value for further dependents, not just the first level.

Questions

Q1. Which statement correctly distinguishes a super key from a candidate key?

A. They are always identical B. Every candidate key is a super key, but a super key need not be minimal (candidate key) C. Every super key is a candidate key D. A table can have at most one super key

Answer: B. **Why:** Candidate key = minimal super key; a super key may contain extra, non-essential attributes and still uniquely identify tuples.

Q2. [Trap] A table has PK = $\{A\}$ and a separate UNIQUE, NOT NULL column B that isn't chosen as PK. B is called:

A. A foreign key B. An alternate key C. A secondary key D. A composite key

Answer: B. **Why:** Any candidate key not selected as the primary key is an alternate key.

Q3. Entity integrity constraint specifically requires that:

A. Foreign keys always reference an existing row B. Primary key attributes can never be NULL C. All attributes must be atomic D. Every table must have a foreign key

Answer: B. **Why:** Entity integrity is about the primary key never being NULL; referential integrity (a separate rule) governs foreign key validity.

Q4. [Numerical] $R(W,X,Y,Z)$ with FDs $X \rightarrow W$, $WZ \rightarrow XY$, $Y \rightarrow WXZ$. Which of the following is a candidate key?

A. $\{X\}$ B. $\{W\}$ C. $\{Y\}$ D. $\{Z\}$

Answer: C. **Why:** $Y^+ = \{W,X,Y,Z\}$ (via $Y \rightarrow WXZ$) = all attributes, so $\{Y\}$ alone is a (minimal) candidate key; X^+ only reaches $\{X,W\}$, incomplete.

Q5. A foreign key configured with ON DELETE SET NULL means that when the referenced row is deleted:

A. The delete is rejected if children exist B. Dependent rows are deleted too C. The FK column in dependent rows is set to NULL, and the rows themselves remain D. Nothing happens at all

Answer: C. Why: SET NULL preserves the dependent rows but nullifies the now-invalid reference, unlike CASCADE (which removes the rows) or RESTRICT (which blocks the operation).

Q6. [Trap] RESTRICT and NO ACTION on a foreign key both:

A. Allow the parent row deletion unconditionally B. Prevent deleting/updating a referenced row while dependent child rows still exist C. Set child FK values to NULL D. Cascade the delete automatically

Answer: B. Why: Both block the operation to protect referential integrity when dependents exist; they differ mainly in *when* the check fires (immediate vs. deferred to end of statement/transaction), not in the end effect for simple cases.

Q7. Which of these is true about prime attributes?

A. A prime attribute must be part of the primary key only B. A prime attribute is any attribute belonging to at least one candidate key C. Prime attributes can never be part of a foreign key D. A relation can have at most one prime attribute

Answer: B. Why: "Prime" is defined relative to ALL candidate keys, not just the chosen primary key — an attribute in any alternate key is also prime.

6.5 Normalization (1NF - BCNF)

5.1 Purpose & Progression

- **Normalization** = decomposing a relation using FDs/candidate keys to eliminate redundancy and the anomalies it causes: **Insertion, Update, Deletion anomalies**.
- Progression: **1NF** → **2NF** → **3NF** → **BCNF** (each stricter than the last). Plain FD-based normalization stops at BCNF; MVD/JD are needed for 4NF/5NF (Ch.6).
- **Prime attribute**: part of some candidate key. **Non-prime**: part of none. **Partial dependency**: a non-prime attribute depends on a proper subset of a (composite) candidate key. **Total dependency**: depends on the whole key. **Transitive dependency**: a non-prime attribute functionally depends on another non-prime attribute (via a chain, not directly on a key).

5.2 Definitions

Form	Requirement
1NF	All attribute values atomic (no multivalued/composite attributes); unique rows; a PK exists.
2NF	In 1NF + no partial dependency of any non-prime attribute on a candidate key. (Automatically satisfied if every candidate key is a single attribute — partial dependency needs a composite key to even exist.)
3NF	In 2NF + no transitive dependency . Equivalent "direct" definition (exam-preferred): for every FD $X \rightarrow Y$ in F, either X is a super key, OR Y is a prime attribute .
BCNF	For every FD $X \rightarrow Y$ in F, X must be a super key — no prime-attribute exception (strictly stronger than 3NF).

TRAP: The 3NF "OR Y is prime" escape clause is exactly what BCNF removes — a relation can be in 3NF (satisfying that OR) yet fail BCNF (X still not a super key).

5.3 Useful Shortcuts (memorize for quick MCQ elimination)

- A relation with **only 2 attributes** is always in BCNF.
- If R has **no non-trivial FD** at all, R is automatically in BCNF.
- If R's candidate key is a **single attribute (simple key)**, R is always at least in **2NF** (no partial dependency possible) — but may still fail 3NF/BCNF.
- If **every attribute of R is prime**, R is always at least in **3NF** (the "Y is prime" clause always rescues it) — but may still fail BCNF.
- If R is in **3NF** and has **only simple (single-attribute) candidate keys**, R is guaranteed to also be in **BCNF**.

5.4 Worked Examples (highest normal form given FDs) — verified by hand

Example 1 — stops at 1NF (classic GATE pattern)

$R(A,B,C,D,E,F,G,H)$, $F = \{CH \rightarrow G, A \rightarrow BC, B \rightarrow CFH, E \rightarrow A, F \rightarrow EG\}$

D never appears anywhere in F (no FD produces or uses D) → D must be in every candidate key.

$\{A,D\}^+ : A \rightarrow BC$ adds B,C; $B \rightarrow CFH$ adds F,H; $F \rightarrow EG$ adds E,G → closure = all 8 attributes. $\{A\}$ alone only reaches $\{A,B,C,E,F,G,H\}$ (misses D). So **candidate key = $\{A,D\}$** (unique, composite).

Prime = $\{A,D\}$. Non-prime = $\{B,C,E,F,G,H\}$.

Check 2NF: $A \rightarrow BC$ is a dependency of non-prime B,C on just $\{A\}$, a proper subset of the key $\{A,D\}$ → **partial dependency exists** → 2NF fails.

Result: 1NF only (not even 2NF).

Example 2 — 2NF but not 3NF

$R(A,B,C,D)$, $F = \{A \rightarrow BC, C \rightarrow D\}$

$A^+ = \{A,B,C,D\}$ ($A \rightarrow BC$, then $C \rightarrow D$) = all attributes → **$\{A\}$ is the candidate key** (single attribute).

Since the key is a single attribute, no partial dependency is possible → **2NF holds automatically**.

Check 3NF: for $C \rightarrow D$ — is C a super key? $C^+ = \{C,D\}$ only → No. Is D prime? No (only A is prime). → violates 3NF.

Result: 2NF, not 3NF.

Example 3 — 3NF but not BCNF (the classic textbook case)

$R(A,B,C)$, $F = \{AB \rightarrow C, C \rightarrow A\}$

$\{AB\}^+ = \{A,B,C\}$ → superkey, minimal (A alone, B alone insufficient) → **candidate key $\{A,B\}$** .

Also check $\{BC\}^+ = \{B,C,A\}$ (via $C \rightarrow A$) = all → **$\{B,C\}$ is also a candidate key**.

Prime attributes = $\{A,B,C\}$ (all attributes are prime — each is in some candidate key!). Non-prime = none.

Check 3NF: $AB \rightarrow C$ (AB is a superkey, fine). $C \rightarrow A$: is C a superkey? No ($\{C\}^+ = \{C,A\}$, missing B). Is A prime? **Yes** → satisfies the 3NF "Y is prime" clause.

Result: 3NF holds. Check BCNF: $C \rightarrow A$ has $X=C$ which is NOT a superkey, and BCNF allows no exception → **BCNF fails**.

Result: 3NF, not BCNF.

Example 4 — full BCNF

$R(A,B,C)$, $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow A\}$

Every single attribute's closure reaches all three ($A^+ = B^+ = C^+ = \{A,B,C\}$) → **$\{A\}$, $\{B\}$, $\{C\}$ are all individually candidate keys**.

Every FD's LHS (A, B, or C) is therefore always a superkey by itself.

Result: BCNF holds (every determinant is a key — the strictest requirement is met).

Questions

Q1. A relation with FD set having only a single-attribute candidate key can never violate:

A. 3NF B. BCNF C. 2NF D. 1NF

Answer: C. Why: Partial dependency (the only 2NF violation) requires a *composite* candidate key; a single-attribute key makes 2NF automatic.

Q2. [Numerical] $R(A,B,C,D)$ with $F = \{A \rightarrow BC, C \rightarrow D\}$. What is the highest normal form satisfied?

A. 1NF B. 2NF C. 3NF D. BCNF

Answer: B. Why: $\{A\}$ is the sole candidate key (single attribute $\Rightarrow$ automatic 2NF); but $C \rightarrow D$ violates 3NF since C is not a superkey and D is not prime.

Q3. [Trap] $R(A,B,C)$ with $F=\{AB\rightarrow C, C\rightarrow A\}$. This relation is:

A. In BCNF B. In 3NF but not BCNF C. In 2NF but not 3NF D. Not even in 1NF

Answer: B. Why: Candidate keys are $\{A,B\}$ and $\{B,C\}$; all attributes are prime, so $C\rightarrow A$ survives 3NF via the "Y is prime" clause, but fails BCNF since C is not a superkey.

Q4. Which condition, by itself, guarantees that a 3NF relation is automatically also in BCNF?

A. The relation has more than 3 attributes B. Every candidate key of the relation is a single (simple) attribute C. The relation has no foreign keys D. The relation is fully normalized to 5NF

Answer: B. Why: With only simple candidate keys, the 3NF "prime attribute" exception can never rescue a genuine BCNF violation the way it can with composite keys — 3NF + simple keys $\Rightarrow$ BCNF.

Q5. A table $R(A, B)$ with exactly two attributes and any FD between them is always in:

A. 1NF only B. 2NF only C. 3NF only D. BCNF

Answer: D. Why: A 2-attribute relation is always in BCNF — the determinant of any FD $A\rightarrow B$ or $B\rightarrow A$ must be a candidate key by elimination.

Q6. [Numerical] $R(A,B,C,D,E,F,G,H)$ with $F = \{CH\rightarrow G, A\rightarrow BC, B\rightarrow CFH, E\rightarrow A, F\rightarrow EG\}$. Highest normal form?

A. 1NF, not 2NF B. 2NF, not 3NF C. 3NF, not BCNF D. BCNF

Answer: A. Why: D appears in no FD, so the sole candidate key is $\{A,D\}$; $A\rightarrow BC$ is a partial dependency (B,C non-prime, depend only on $A \subset \{A,D\}$) — fails 2NF, so R sits only in 1NF.

Q7. A relation R has no non-trivial functional dependency at all. R is:

A. Not necessarily in any normal form B. Automatically in BCNF C. Automatically only in 1NF D. Automatically in 2NF but never BCNF

Answer: B. Why: With no non-trivial FDs, there is nothing to violate any of the normal-form rules (all conditions are vacuously true) — R is trivially in BCNF.

Q8. Which of the following is the correct "direct" (LHS/RHS) definition of 3NF used for fast exam elimination?

A. No composite candidate keys allowed B. For every FD $X\rightarrow Y$, X must be a superkey C. For every FD $X\rightarrow Y$, either X is a superkey or Y is a prime attribute D. Every attribute must be a candidate key

Answer: C. Why: This is the standard 3NF test; BCNF (option B) drops the "Y is prime" exception, making it strictly stronger.

6.6 Decomposition, Multivalued Dependency & 5NF

6.1 Why Decompose? Two Desired Properties

When normalization splits R into R1, R2, ..., Rn, two properties are checked:

1. **Lossless (Non-Additive) Join Decomposition** — mandatory, non-negotiable.
2. **Dependency-Preserving Decomposition** — desirable, but sometimes sacrificed.

6.2 Lossless Join Decomposition

Decomposition of R into R1, R2 is LOSSY if: $R1 \Join R2 \supset R$ (produces spurious/extra tuples)
 Decomposition of R into R1, R2 is LOSSLESS if: $R1 \Join R2 = R$ (reconstructs R exactly)

Test using FDs — for R decomposed into R1, R2, ALL of the following must hold:

1. $Att(R1) \cup Att(R2) = Att(R)$ (every attribute lands in at least one piece).
2. $Att(R1) \cap Att(R2) \neq \emptyset$ (they must share at least one common attribute).
3. The common attribute set must be a **key for at least one** of R1 or R2:

$Att(R1) \cap Att(R2) \rightarrow Att(R1)$ OR $Att(R1) \cap Att(R2) \rightarrow Att(R2)$

TRAP: Sharing a common attribute is NOT enough by itself — that common attribute (or attribute set) must *functionally determine* one entire side of the decomposition, or the join will produce spurious tuples.

Worked Example

$R(A,B,C)$, $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow A\}$, decomposed into $R1(A,B)$ and $R2(B,C)$.

Common attribute = $\{B\}$. Does $B \rightarrow A$ hold? Via $B \rightarrow C \rightarrow A$ (transitivity) — yes. So B determines all of R1's attributes $(A,B) \rightarrow$ **condition 3 satisfied** → **lossless**.

6.3 Dependency-Preserving Decomposition

Decomposition into $R1 \dots Rn$ (with projected FD sets $F1 \dots Fn$) is dependency-preserving iff:
 $\{F1 \cup F2 \cup \dots \cup Fn\}^+ = F^+$ (the union of projected FDs, closed, equals the original closure)

- **BCNF decomposition is ALWAYS lossless, but is NOT always dependency-preserving.**
- **TRAP:** This is one of the most commonly tested DBMS facts — students assume "properly normalized $\Rightarrow$ both properties guaranteed." BCNF only guarantees losslessness; 3NF decomposition (via the synthesis algorithm) guarantees BOTH lossless AND dependency-preserving, which is *why* 3NF is sometimes deliberately preferred over BCNF in practice.

Worked Example (same relation as above)

$R(A,B,C)$, $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow A\} \rightarrow R1(A,B), R2(B,C)$.

$F1$ (projected onto A,B): $A \rightarrow B$ holds; $B \rightarrow A$ also holds (derivable, since $B \rightarrow C \rightarrow A$ stays within... wait, C isn't in R1, but $B \rightarrow A$ is still a valid consequence of F^+ expressed using only R1's attributes) $\rightarrow F1 = \{A \rightarrow B, B \rightarrow A\}$.

$F2$ (projected onto B,C): $B \rightarrow C$ holds; $C \rightarrow B$ holds (via $C \rightarrow A \rightarrow B$) $\rightarrow F2 = \{B \rightarrow C, C \rightarrow B\}$.

Union $F1 \cup F2 = \{A \rightarrow B, B \rightarrow A, B \rightarrow C, C \rightarrow B\}$. Does this imply $C \rightarrow A$ (needed to match F^+)? $C \rightarrow B, B \rightarrow A \Rightarrow C \rightarrow A$ by transitivity. **Yes, preserved.**

This decomposition is both lossless AND dependency-preserving.

6.4 Multivalued Dependency (MVD) & 4NF

- **MVD notation:** $X \twoheadrightarrow Y$ — for every value of X, there exists a set of values of Y, *independent* of the other attributes in R.
- **Trivial MVD:** $X \twoheadrightarrow Y$ is trivial if $Y \subseteq X$, or $X \cup Y =$ all attributes of R.
- If a normal FD $X \rightarrow Y$ holds, then the MVD $X \twoheadrightarrow Y$ also trivially holds ($FD \Rightarrow MVD$, not the reverse).
- **Why MVDs matter:** a relation can be in BCNF (no non-trivial FDs at all) yet still have massive redundancy caused purely by two *independent* multivalued facts about the same key — this is exactly what 4NF fixes.

Classic Example

$Student(S_name, Club_name, P_no)$ where a student can be in multiple clubs AND have multiple phone numbers — completely independently of each other.

MVDs: $S_name \twoheadrightarrow Club_name$, and $S_name \twoheadrightarrow P_no$.

No FD holds at all (so the relation is vacuously in BCNF) — yet every (club, phone) combination must be repeated for consistency, causing redundancy.

Fix: decompose into $(S_name, Club_name)$ and (S_name, P_no) — each MVD becomes trivial in its own smaller table.

4NF definition: R is in 4NF iff it is in BCNF AND has **no non-trivial MVD**.

TRAP: A relation can satisfy BCNF perfectly (zero FD violations) and still fail 4NF purely due to independent multivalued attributes — "BCNF" does NOT imply "4NF."

6.5 Fifth Normal Form (5NF / PJNF)

- **5NF:** R is in 4NF AND every non-trivial **join dependency (JD)** is implied by the candidate keys (i.e., R cannot be losslessly decomposed into smaller pieces except via its keys).
- Targets redundancy from **three-way (or higher) circular relationships** that a 2-table decomposition cannot fix, but a 3-table decomposition can.

Classic Example — Supplier–Part–Project

A relation SPJ(Supplier, Part, Project) where the business rule is: "if S supplies P, S works with J, and J uses P, the combination is valid" — this is a genuine 3-way constraint, not fully captured by any 2 of the 3 pairwise relationships alone.

5NF decomposition: split into three binary relations — $SP(Supplier, Part)$, $SJ(Supplier, Project)$, $PJ(Part, Project)$.

Rejoining $SP \bowtie SJ \bowtie PJ$ reconstructs the valid combinations losslessly — this 3-way reconstruction requirement is exactly a **non-trivial join dependency**, and 5NF says: decompose until no such non-trivial JD remains unless it's implied by the keys.

Questions

Q1. For a decomposition of R into R1 and R2 to be lossless, the common attribute set $\text{Att}(R1) \cap \text{Att}(R2)$ must:

A. Be non-empty only B. Be a candidate key of R C. Functionally determine all attributes of R1 or all attributes of R2 D. Contain the primary key of R1 only

Answer: C. Why: The common attributes must act as a determinant (key) for at least one of the two decomposed relations — this is what guarantees no spurious tuples on rejoining.

Q2. [Trap] "A BCNF decomposition is always both lossless and dependency-preserving." This statement is:

A. True B. False — BCNF decomposition is always lossless, but not always dependency-preserving C. False — BCNF decomposition is dependency preserving but not always lossless D. False on both counts, always

Answer: B. Why: Losslessness is guaranteed by the BCNF decomposition algorithm; dependency preservation is not, which is a classic real-world trade-off (3NF synthesis is used instead when preservation matters more).

Q3. A Multivalued Dependency $X \twoheadrightarrow Y$ is called trivial when:

A. X and Y are both empty B. $Y \subseteq X$, or $X \cup Y$ equals all attributes of R C. X is a candidate key D. Y is a prime attribute

Answer: B. Why: These are exactly the two cases where the MVD carries no extra constraint beyond what's structurally guaranteed.

Q4. [Trap] A relation is confirmed to be in BCNF (no FD violations whatsoever). Can it still violate 4NF?

A. No, BCNF always implies 4NF B. Yes — independent multivalued attributes can cause redundancy even with zero FDs present C. No, 4NF is weaker than BCNF so it's automatically satisfied D. Only if the relation has a composite key

Answer: B. Why: MVD-based redundancy is a separate phenomenon from FD-based redundancy; a relation can be a "perfect" BCNF design yet still redundantly repeat combinations of two independent multivalued facts — exactly what 4NF targets.

Q5. The Supplier-Part-Project example is used to illustrate the need for:

A. 2NF B. BCNF C. 4NF D. 5NF

Answer: D. Why: It demonstrates a genuine three-way (non-trivial) join dependency that requires decomposition into three binary relations — the defining case for 5NF.

Q6. [Numerical] $R(A,B,C)$, $F=\{A \rightarrow B, B \rightarrow C, C \rightarrow A\}$, decomposed into $R1(A,B)$, $R2(B,C)$. Is this decomposition lossless?

A. Yes, since $B \rightarrow A$ is derivable, making B a key of R1 B. No, because C is missing from R1 C. No, since the intersection is empty D. Cannot be determined

Answer: A. Why: $\text{Att}(R1) \cap \text{Att}(R2) = \{B\}$; $B \rightarrow A$ holds (via $B \rightarrow C \rightarrow A$ transitively), so B determines all of R1 — satisfying the lossless-join condition.

Q7. Why is dependency preservation described as "desirable but sometimes sacrificed"?

A. It is always impossible to achieve B. Enforcing certain FDs after BCNF decomposition may require an expensive join to check, so it's occasionally traded off for a lossless-only decomposition C. It only matters for 1NF relations D. Because MVDs override FDs

Answer: B. Why: Some FDs may span multiple decomposed relations after BCNF normalization, making them impossible to check without rejoining — hence BCNF sometimes trades away dependency preservation for guaranteed losslessness.

6.7 File Organization, Indexing & B-Trees

7.1 File Organization

- **Ordered (sorted) file:** supports **binary search** (only on the field the file is sorted by); insertion/deletion is costly (needs reorganization).
- **Unordered (heap) file:** new records appended at the end; only **linear search** possible; insertion/deletion is cheap (no reorganization).
- **TRAP:** Binary search on an ordered file works only when searching on the field it is physically sorted by — searching any other field still degrades to linear search.

7.2 Indexing Basics

- **Index** = auxiliary access structure providing a secondary access path without touching the physical placement of main-file records.
- Index file record has just 2 columns: **key** + **block pointer** — far smaller than the main file, so it is always kept **sorted** (regardless of whether the main file is sorted), enabling binary search on the index itself.
- Index can be built on **any** attribute (key or non-key); multiple index files can exist for one main file.

```
Blocking factor (bfr) = floor(Block size / Record size)
Number of blocks (b) = ceil(Number of records / bfr)
Block accesses (unordered file) = O(n) [n = number of blocks; linear search]
Block accesses (ordered file / single-level index, binary search) = ceil(log2(n)) + 1 [+1 for the final data-block access]
```

- **Dense index:** one entry per *distinct search-key value* (not per record) in the main file.
- **Sparse index:** entries for only *some* records — requires the main file to be ordered on that key.

7.3 Types of Indexing

Type	Main file order	Index density	Entries in index file
Primary	Ordered on primary key	Sparse	= number of blocks in main file (1 per block, via anchor/first record)
Clustering	Ordered on a non-key attribute	Sparse or dense	= number of distinct values of that attribute
Secondary	Main file order irrelevant	Dense	= number of records in main file (1 per record)
Multilevel	—	—	Recursively indexes the index itself until one block fits in memory

- **TRAP:** Primary indexing works **only on the ordering key** and is sparse; Secondary indexing can be on any (key/non-key) attribute and is always **dense** — mixing these two up is a classic exam trap.

Worked Example (Primary vs Secondary Indexing + Multilevel Benefit)

Given: $r = 30,000$ records, Block size $B = 1024$ B, Record size $R = 100$ B (unspanned),
key field $V = 9$ B, block pointer $P = 6$ B.

Main file:

```
bfr = floor(1024/100) = 10 records/block
b   = ceil(30000/10) = 3,000 blocks
```

PRIMARY INDEX (sparse, 1 entry per block of main file):

```
Index entry size = V+P = 15 B => bfri = floor(1024/15) = 68 entries/block
Index entries = 3,000 (= number of main-file blocks)
Index blocks (level 1) = ceil(3000/68) = 45
-- Single-level binary search: ceil(log2(45)) + 1 = 6 + 1 = 7 block accesses
-- Add level 2: ceil(45/68) = 1 block (fits in memory)
-- Multilevel (2-level) direct access: levels + 1 = 2 + 1 = 3 block accesses
```

SECONDARY INDEX (dense, 1 entry per record):

```
Index entries = 30,000 (= number of records)
Index blocks (level 1) = ceil(30000/68) = 442
-- Single-level binary search: ceil(log2(442)) + 1 = 9 + 1 = 10 block accesses
-- Level 2: ceil(442/68) = 7 blocks; Level 3: ceil(7/68) = 1 block (fits in memory)
-- Multilevel (3-level) direct access: levels + 1 = 3 + 1 = 4 block accesses
```

TRAP: Multilevel indexing turns an $O(\log_2 n)$ binary-search cost into an $O(\text{\#levels})$ direct-access cost — always dramatically fewer block accesses once the index itself grows large (7→3 and 10→4 above).

7.4 B-Trees & B+ Trees

- **Why not a plain BST/AVL tree for disk indexes?** Disk I/O (block access) is the bottleneck, not comparisons — B/B+ trees maximize **fan-out** (children per node) so the tree height (and thus disk accesses) stays very small even for millions of records.
- **B-tree of order m** (m-way search tree):

Node	Max children	Min children	Max keys	Min keys
Root	m	0 (or 2 if it has any children)	$m-1$	1
Internal (non-root)	m	$\lceil m/2 \rceil$	$m-1$	$\lceil m/2 \rceil - 1$
Leaf	0	0	$m-1$	$\lceil m/2 \rceil - 1$

- All leaves are on the **same level** (perfectly height-balanced) — this guarantees worst-case search time stays $O(\log n)$.

B-Tree vs B+ Tree

Feature	B-Tree	B+ Tree
Data pointers	At every level (internal + leaf)	Only at leaf level
Internal nodes	Store keys + data pointers	Store only keys (used purely for routing)
Leaf nodes	Normal leaves	Linked together (sequence set) for fast ordered range scans
Range queries	Poor — no way to traverse keys sequentially	Excellent — just follow leaf-level linked list
Redundant keys	No	Yes (a key may appear in both an internal node and a leaf)
Preferred for	—	Actual DB indexing (used far more in practice)

- **TRAP:** Because B+ tree internal nodes hold no data pointers, they fit **more keys per node** than a B-tree of the same order — meaning B+ trees usually achieve a **smaller height** for the same data, on top of their range-query advantage. This is why virtually all real DB indexes (and file systems) use B+ trees, not plain B-trees.

Insertion / Deletion Rules (conceptual)

- **Insertion (both):** insert in sorted position; if a node **overflows** (exceeds $m-1$ keys), split it — median key moves up to the parent (in B+ tree, the median is *copied* up, not removed from the leaf level); splitting can cascade up to the root, adding a new level.
- **Deletion (both):** if removal causes a node to **underflow** (fall below $\lceil m/2 \rceil - 1$ keys), first try to **borrow** a key from an adjacent sibling (via the parent); if siblings are also at minimum, **merge** with a sibling — this can cascade up and even reduce the tree's height.
- Empirically, after many random insert/delete operations, B-tree/B+ tree nodes stabilize at about **69% full**.

Questions

Q1. Which type of indexing must always be dense (one entry per record), regardless of whether it is built on a key or non-key attribute?

A. Primary B. Clustering C. Secondary D. Multilevel

Answer: C. Why: Secondary indexing always has as many entries as there are records in the main file (dense); primary indexing is sparse (1 entry per block) and requires the main file to be sorted on that key.

Q2. [Numerical] A file has 2,000 blocks in its main file, and a single-level sparse primary index is built on it. Using binary search on the index followed by one data-block access, how many total block accesses are needed in the worst case?

A. 10 B. 11 C. 12 D. 2000

Answer: C. Why: $\text{ceil}(\log_2(2000)) + 1 = 11 + 1 = 12$.

Q3. [Trap] Why is a B+ tree generally preferred over a plain B-tree for actual database indexing?

A. B+ trees use less memory per node B. B+ trees are not balanced, so they are faster C. B+ tree leaves are linked, enabling efficient range queries, and internal nodes (holding only keys) achieve higher fan-out D. B-trees cannot store more than 3 levels

Answer: C. Why: B+ tree internal nodes carry no data pointers (only routing keys), allowing more keys per node (lower height) and, crucially, their linked leaf level supports fast sequential range scans — something a B-tree cannot do efficiently.

Q4. In a B-tree of order $m=5$, what is the minimum number of keys allowed in a non-root internal node?

A. 1 B. 2 C. 3 D. 4

Answer: B. Why: Min keys = $\lceil m/2 \rceil - 1 = \lceil 2.5 \rceil - 1 = 3 - 1 = 2$.

Q5. In a B-tree of order $m=5$, what is the maximum number of keys a node (any node) can hold?

A. 3 B. 4 C. 5 D. 6

Answer: B. Why: Max keys = $m - 1 = 5 - 1 = 4$ for every node type (root, internal, leaf).

Q6. When a B+ tree leaf node overflows during insertion, the median key is:

A. Deleted B. Moved up to the parent and removed from the leaf C. Copied up to the parent while also remaining in the (new) leaf D. Ignored

Answer: C. Why: In a B+ tree, since only leaves hold actual data pointers, the median key must remain at the leaf level (retained/duplicated) even as its copy is pushed up to route searches at the internal level — unlike a B-tree, where the median is fully moved up.

Q7. [Trap] A junior developer claims: "B-trees are better than B+ trees for database indexes because B-trees can locate data at internal nodes without reaching a leaf." What is the flaw?

A. This is entirely correct B. B-trees don't have internal nodes C. While technically true for point lookups, this comes at the cost of no ordered leaf-linkage, making range queries (a very common DB operation) far slower than in B+ trees — hence B+ trees are still preferred in practice D. Only B-trees are balanced

Answer: C. Why: The point-lookup argument is real but minor compared to the huge practical cost of poor range-query support, which is why virtually all production databases use B+ trees for indexing.

Q8. What happens when deletion causes a B+ tree node to underflow and no sibling has a key to spare?

A. Nothing, underflow is ignored B. The node is merged with a sibling, potentially propagating up and reducing tree height C. The tree becomes unbalanced permanently D. A new root is always created

Answer: B. Why: Merging propagates upward if the parent then also underflows, and can reduce the number of levels in the tree — the mirror-image process of insertion-triggered splits.

6.8 Relational Algebra

8.1 Query Languages Overview

Feature	Relational Algebra (RA)	Relational Calculus (RC)
Type	Procedural (what + how)	Non-procedural / declarative (what only)
Basis	Set theory, operators on relations	Mathematical logic (predicates)
Executed directly?	No — theoretical foundation for SQL	No — theoretical foundation for SQL

- SQL is practical and blends both procedural and declarative elements; RA/RC are the theoretical basis SQL is built on (never run directly on a real DBMS).
- Every RA operator takes 1–2 relations as input and returns exactly **one unnamed relation** as output — this closure property lets operators be nested/composed.
- TRAP:** RA is **set-based**, so it does **not** consider duplicates by default — the same query in SQL (which uses bag/multiset semantics) can return a different number of rows.

8.2 Fundamental (Basic) Operators

Operator	Symbol	Arity	Eliminates duplicates?
Select	σ (sigma)	Unary	No — only removes rows via a condition
Project	Π (pi)	Unary	Yes — by default
Union	$\cup$	Binary	Yes (set union)
Set Difference	$-$	Binary	Yes
Cartesian Product	$\times$	Binary	No
Rename	ρ (rho)	Unary	—

- Select (σ):** $\sigma \text{ condition}(R)$ — horizontal filter (chooses rows). Degree of result = degree of R; $0 \leq \text{cardinality}(\text{result}) \leq \text{cardinality}(R)$. σ is **commutative**: $\sigma_1(\sigma_2(R)) = \sigma_2(\sigma_1(R))$.
- Project (Π):** $\Pi \text{ col1, col2}(R)$ — vertical filter (chooses columns), **eliminates duplicate rows** by default. **Not commutative** with itself in the sense that column order in the subscript matters for the output schema.
- TRAP:** Select removes **rows only**, never eliminates duplicates; Project removes **columns AND** eliminates duplicate rows — a very frequently tested distinction.

8.3 Set Operators & Cartesian Product

Union-compatibility requirement for $r \cup s$, $r - s$, $r \cap s$:
Same degree (arity) AND matching attribute domains (same order).

Union: $r \cup s$; $\text{Deg}(r \cup s) = \text{Deg}(r)$; $\max(|r|, |s|) \leq |r \cup s| \leq |r| + |s|$
 Set diff: $r - s$; $0 \leq |r - s| \leq |r|$
 Intersection: $r \cap s = r - (r - s)$ [derived, not fundamental] $0 \leq |r \cap s| \leq \min(|r|, |s|)$
 Cartesian: $r \times s$; $|r \times s| = |r| * |s|$; $\text{Deg}(r \times s) = \text{Deg}(r) + \text{Deg}(s)$

- TRAP:** Union/Difference/Intersection all require **union-compatible** relations (same degree + matching domains); Cartesian Product has **no such restriction** — it pairs every tuple of r with every tuple of s regardless of schema.
- Intersection is a **derived** operator (expressible via difference) — it adds no new power to RA, just convenience.

8.4 Joins

Join hierarchy:
Join

```
-- Inner Join
|  |-- Theta Join (any comparison operator: =, <, <=, !=, >, >=)
|  |  |-- Equi Join (theta join restricted to "=" only)
|  |  |-- Natural Join (equi-join on ALL common attribute names, duplicate column dropped)
|-- Outer Join
|  |-- Left Outer Join (all of R1, unmatched R2 side padded with NULL)
|  |-- Right Outer Join (all of R2, unmatched R1 side padded with NULL)
|  |-- Full Outer Join (all of both, unmatched sides padded with NULL)
```

- Natural Join ($\bowtie$):** implicit equality condition on all commonly-named attributes; the shared attribute appears **only once** in the result (unlike a plain Cartesian product + selection, which would keep both copies).
- Theta Join ($R \bowtie_{\theta} S$):** general condition $A_i \theta B_j$; result has $n+m$ attributes (both copies of any shared-name columns kept, distinguished via relation-name prefix).
- Equi Join:** theta join using **only "="** as the comparison operator.
- TRAP:** Natural Join is really "Equi Join + drop duplicate column"; if you only apply an equi-join condition without dropping the duplicate attribute, that's technically just a theta/equi join, NOT a natural join.
- Outer Joins** preserve unmatched tuples (pad with NULL) instead of discarding them like an inner/natural join would.

8.5 Rename Operator

```
rho_X(E)           -- rename relation resulting from expression E to X
rho_X(A1,A2,...,An)(E) -- rename relation to X AND rename its attributes to A1..An
```

- Does **not** modify the original table in the database — it creates a renamed reference/copy of the query result, used for self-joins or giving a query result a usable name for further composition.

8.6 Division Operator ($\div$)

```
R(Z) / S(X)   where X subset-of Z,  Y = Z - X
Formula:  R / S = Pi_Y(R) - Pi_Y[ (Pi_Y(R) x S) - Pi_Y,X(R) ]
```

- Meaning:** returns values of Y in R that are associated with **every** value of X in S ("Y-values related to ALL X-values").
- Recognition Pattern: if a query says "**find X who are related to ALL Y**" (e.g., "find students enrolled in ALL courses," "find suppliers who supply ALL parts") → think **Division**.
- Division is a **derived** operator (built from Project, Cartesian Product, and Set Difference) — not fundamental, but essential for "for-all" style queries that would otherwise need deeply nested RA expressions.

Questions

Q1. [Trap] Which relational algebra operator eliminates duplicate rows in its result by default?

A. Select (σ) B. Project (Π) C. Cartesian Product ($\times$) D. Theta Join

Answer: B. Why: Projection removes columns and, being set-based, automatically drops any resulting duplicate rows; Selection only filters rows and does not deduplicate.

Q2. [Numerical] Relation R has 5 tuples and degree 3; Relation S has 4 tuples and degree 2. What is the degree and cardinality of $R \times S$?

A. Degree 5, Cardinality 9 B. Degree 6, Cardinality 9 C. Degree 5, Cardinality 20 D. Degree 6, Cardinality 20

Answer: C. Why: Degree($R \times S$) = Degree(R) + Degree(S) = 3 + 2 = 5; Cardinality = $|R| \times |S| = 5 \times 4 = 20$.

Q3. For $R \cup S$ to be a valid relational algebra operation, R and S must:

A. Have the same number of tuples B. Be union-compatible (same degree and matching attribute domains) C. Share at least one common attribute name D. Have identical primary keys

Answer: B. Why: Union (like Set-Difference and Intersection) requires union-compatibility; unlike Cartesian Product or Join, matching attribute names is not itself the requirement — matching arity and domains is.

Q4. Which join preserves all tuples from BOTH relations, padding unmatched sides with NULL?

A. Natural Join B. Left Outer Join C. Right Outer Join D. Full Outer Join

Answer: D. Why: Full Outer Join = Left Outer Join + Right Outer Join combined — every tuple from both relations appears, matched or not.

Q5. A query asks: "Find the names of suppliers who supply ALL parts in the Parts table." Which RA operator is the natural fit?

A. Natural Join B. Division C. Set Difference D. Cartesian Product

Answer: B. Why: "Related to ALL values" is the textbook signal for the Division operator.

Q6. [Trap] A student writes a query joining Employee and Department on `Employee.dept_id = Department.dept_id` but keeps both `dept_id` columns in the output. This is best classified as:

A. A Natural Join B. An Equi Join (not a natural join, since the duplicate column wasn't dropped) C. A Left Outer Join D. Invalid RA syntax

Answer: B. Why: Natural Join specifically drops the duplicate shared-name column; keeping both copies makes this only an Equi Join (a type of Theta Join), not a Natural Join.

Q7. What is the correct relationship between Intersection and the fundamental RA operators?

A. Intersection is fundamental and cannot be derived B. Intersection = $R - (R - S)$, a derived operator adding no new expressive power C. Intersection requires a Cartesian Product D. Intersection is only valid for numeric attributes

Answer: B. Why: Set-Intersection is expressible purely via Set-Difference, so it's classified as a derived (not fundamental) operator.

Q8. [Numerical] R has attributes (A,B,C) with 100 tuples; S has attributes (B,C) with 30 distinct (B,C) combinations appearing across all of R's groups. After computing $R \div S$, which attribute(s) remain in the result?

A. A, B, C B. B, C only C. A only D. None

Answer: C. Why: For $R(Z) \div S(X)$ where $X \subseteq Z$, the result contains only the attributes $Y = Z - X$ — here $Z = \{A,B,C\}$, $X = \{B,C\}$, so $Y = \{A\}$.

Q9. Which of the following is NOT a valid form of outer join?

A. Left Outer Join B. Right Outer Join C. Full Outer Join D. Cartesian Outer Join

Answer: D. Why: The three valid outer join forms are Left ($\bowtie$), Right ($\bowtie$), and Full ($\bowtie$) — "Cartesian Outer Join" is not a defined RA operation.

6.9 SQL — Basics, Select/Where, DDL/DML/DCL/TCL

9.1 What SQL Is

- SQL = declarative language for RDBMS (originated as "Sequel" in IBM's System R, 1970s; standardized as SQL-86 by ANSI/ISO, evolved through SQL-92, SQL:1999, ... SQL:2023).
- Built on **Relational Algebra** (procedural) and **Tuple Relational Calculus** (non-procedural) — SQL itself is declarative (says WHAT, not HOW).
- Sub-languages: **DDL** (structure), **DML** (data), **DCL** (permissions), **TCL** (transactions), **DQL** (SELECT, sometimes classed under DML).

9.2 Basic Query Structure

```
SELECT A1, A2, ..., An
FROM r1, r2, ..., rm
WHERE P;
```

- A query takes 1+ tables as input, always produces exactly **one unnamed (anonymous) output table**.
- Only **SELECT** and **FROM** are **mandatory**; omitted WHERE = condition treated as true for all rows.
- Logical order of execution: FROM → WHERE → SELECT** (not the written top-to-bottom order!) — this ordering matters a lot once GROUP BY/HAVING enter the picture (Ch.10).
- SQL keywords are **not case-sensitive** (SELECT = select); string literal comparisons ARE case-sensitive per the SQL standard (though MySQL/SQL Server often ignore case by default).
- Default duplicate handling = **ALL** (keeps duplicates); use **DISTINCT** to remove them.

9.3 SELECT, WHERE, DISTINCT

- SELECT** = vertical filtering (choose columns) → maps to relational algebra **Projection** (π).
- WHERE** = horizontal filtering (choose rows) → maps to relational algebra **Selection** (σ).
- Comparison operators: =, <, >, <=, >=, <> (not equal). Logical connectives: AND, OR, NOT.
- BETWEEN low AND high** = shorthand for >= low AND <= high (inclusive both ends).
- DISTINCT** removes duplicate rows (based on the full selected column combination), not per-column.
- TRAP:** Arithmetic expressions in SELECT (e.g., balance * 1.06) only change **displayed** output — they never modify stored data.

9.4 SQL Data Types

Category	Types	Notes
Numeric	INT, DECIMAL(p,s), FLOAT	DECIMAL = exact (currency); FLOAT = approximate
String	CHAR(n), VARCHAR(n), TEXT	CHAR = fixed-length (padded); VARCHAR = variable-length; TEXT = long-form
Date/Time	DATE, DATETIME/TIME	DATE = YYYY-MM-DD only
Boolean	BOOLEAN	TRUE/FALSE

- TRAP:** **CHAR(n)** is **fixed-length** (pads with spaces to n) — **VARCHAR(n)** is **variable-length** (stores only actual content, up to n). CHAR wastes space for short values but is marginally faster for fixed-format data (e.g., state codes).

9.5 Operators, Precedence & NULL

- Categories: Arithmetic (+ - * / %), Comparison (= != < > <= >= BETWEEN LIKE IN IS NULL), Logical (AND OR NOT), Set/Concatenation (|| UNION INTERSECT EXCEPT).
- NULL** = missing/unknown/not-applicable — **NOT the same as 0, empty string, or spaces**.
- TRAP:** WHERE age = NULL **always fails** (NULL isn't "equal" to anything, even itself) — you MUST use **IS NULL** / **IS NOT NULL** to test for it.

9.6 SQL Command Categories

Category	Full Name	Commands	Auto-commit?	Rollback?
DDL	Data Definition	CREATE, ALTER, DROP, TRUNCATE	Yes	No
DML	Data Manipulation	SELECT, INSERT, UPDATE, DELETE	No	Yes
DCL	Data Control	GRANT, REVOKE	Yes	No
TCL	Transaction Control	COMMIT, ROLLBACK, SAVEPOINT	—	—

DELETE vs TRUNCATE vs DROP (classic trap table)

Feature	DELETE	TRUNCATE	DROP
Type	DML	DDL	DDL
Action	Removes rows (supports WHERE)	Wipes ALL data, keeps structure	Removes table + structure entirely
Rollback?	Yes	No (auto-committed)	No
Fires triggers?	Yes	No	No
Speed	Slower (row-by-row + logging)	Faster (deallocates pages)	—

TRAP: This DELETE/TRUNCATE/DROP trio is one of the most frequently tested DBMS facts — remember DELETE is the only one of the three that is undoable (rollback-capable) and the only one that can be filtered with WHERE.

9.7 ALTER Command

```
ALTER TABLE Student ADD email VARCHAR(100);           -- add column
ALTER TABLE Student DROP COLUMN email;               -- remove column
ALTER TABLE Student MODIFY name VARCHAR(100);        -- change datatype/size
ALTER TABLE Student RENAME COLUMN marks TO score;    -- rename column
ALTER TABLE Student ADD CONSTRAINT pk_id PRIMARY KEY(id);
ALTER TABLE Student ADD CONSTRAINT fk_dept FOREIGN KEY(dept_id) REFERENCES Dept(id);
```

- **CASCADE** on a DROP removes the column/constraint AND everything depending on it (vs **RESTRICT**, which blocks the drop if dependencies exist).

9.8 DCL — GRANT / REVOKE

```
GRANT SELECT, UPDATE ON Student TO 'Teacher_User';
REVOKE UPDATE ON Student FROM 'Teacher_User';
```

- Common privileges: SELECT (read), INSERT, UPDATE, DELETE.

9.9 TCL — COMMIT / ROLLBACK / SAVEPOINT

- **COMMIT**: permanently saves all changes since the last COMMIT.
- **ROLLBACK**: undoes changes since the last COMMIT (or SAVEPOINT).
- **SAVEPOINT**: a named checkpoint within a transaction, so you can roll back partially rather than the whole transaction.

9.10 DESCRIBE / SHOW TABLES

Command	Shows
DESC / DESCRIBE table_name	Table blueprint: column names, types, NULL-ability, key info, defaults
SHOW TABLES (MySQL)	List of table names — Oracle instead uses <code>SELECT table_name FROM user_tables;</code>
SELECT * FROM table	The actual stored rows

9.11 ORDER BY & Aliases (Rename)

- **Alias (AS)**: temporary name for a column/table, exists only for the query's duration — doesn't alter stored schema. Usable in both SELECT and FROM clauses.
- **ORDER BY**: sorts final result rows; default ascending (ASC), use DESC for descending.

Questions

Q1. [Trap] Which of DELETE, TRUNCATE, and DROP can be rolled back?

A. All three B. Only DELETE C. Only TRUNCATE D. None of them

Answer: B. **Why:** DELETE is a DML command (transaction-logged, rollback-capable); TRUNCATE and DROP are DDL and auto-commit immediately.

Q2. [Trap] WHERE salary = NULL in a SQL query will:

A. Return rows where salary is unknown B. Return rows where salary is exactly 0 C. Return no rows, because NULL cannot be compared with = D. Cause a syntax error

Answer: C. **Why:** NULL represents an unknown value; standard comparison operators always evaluate to unknown/false against NULL — IS NULL must be used instead.

Q3. What is the correct logical order of execution for SELECT ... FROM ... WHERE ... ?

A. SELECT → FROM → WHERE B. WHERE → FROM → SELECT C. FROM → WHERE → SELECT D. FROM → SELECT → WHERE

Answer: C. **Why:** The engine first identifies source tables (FROM), filters rows (WHERE), then picks columns to display (SELECT) — even though it's *written* SELECT-first.

Q4. A column declared as CHAR(10) storing the value 'Cat' will actually occupy:

A. 3 characters, like VARCHAR B. 10 characters (padded with trailing spaces) C. 0 characters until updated D. An error, since 'Cat' is too short

Answer: B. **Why:** CHAR(n) is fixed-length — shorter values are space-padded to the full declared length, unlike VARCHAR(n) which stores only the actual content.

Q5. Which command is used to remove specific rows from a table using a WHERE clause, in a way that CAN be rolled back?

A. TRUNCATE B. DROP C. DELETE D. ALTER

Answer: C. **Why:** DELETE supports WHERE-based row filtering and is a DML command, so its effect is transaction-logged and reversible before COMMIT.

Q6. SELECT DISTINCT name, city FROM Students; will return:

A. Only unique names B. Only unique cities C. Unique (name, city) combinations, treating each pair as a unit D. All rows including duplicates

Answer: C. **Why:** DISTINCT with multiple columns removes duplicates based on the *combination* of all selected columns, not any single column alone.

Q7. Which SQL command category includes GRANT and REVOKE?

A. DDL B. DML C. DCL D. TCL

Answer: C. **Why:** DCL (Data Control Language) manages access permissions; GRANT gives privileges, REVOKE removes them.

Q8. [Trap] Which statement about DDL commands is correct?

A. DDL commands can always be rolled back like DML B. DDL commands (CREATE, ALTER, DROP, TRUNCATE) are auto-committed and generally cannot be rolled back C. DDL only affects data, never structure D. DDL requires a COMMIT statement to take effect

Answer: B. **Why:** DDL changes the schema and is auto-committed immediately — unlike DML, which stays reversible until COMMIT.

Q9. An alias created with AS in a query:

A. Permanently renames the column in the database B. Exists only for the duration of that single query C. Requires an ALTER TABLE statement D. Can only be used on tables, never columns

Answer: B. **Why:** Aliases are temporary display names scoped to the query — the underlying schema/column names remain unchanged.

Q10. A DECIMAL(p,s) data type is preferred over FLOAT for storing currency because:

A. DECIMAL uses less storage B. DECIMAL stores exact values, avoiding floating-point rounding errors C. FLOAT cannot store numbers at all D. DECIMAL is faster for all arithmetic

Answer: B. **Why:** DECIMAL(p,s) stores an exact fixed-point value (important for money), while FLOAT is an approximate binary representation prone to rounding errors.

6.10 SQL — Aggregates, Joins, Set Operations, Subqueries, Views

10.1 Aggregate Functions

- Five built-ins: **AVG, MIN, MAX, SUM, COUNT**.
- SUM / AVG require numeric input; MIN / MAX / COUNT also work on non-numeric (strings, dates).
- TRAP:** COUNT(*) counts ALL rows including NULLs; COUNT(column) skips NULL values in that column. All other aggregate functions (SUM, AVG, MIN, MAX) ignore NULLs automatically.
- AVG(balance) is NOT always equal to SUM(balance)/COUNT(balance) when NULLs are involved in a way that changes denominators — but when using COUNT(column) (which also skips NULLs) both forms match; COUNT(*) in the denominator would NOT match if NULLs exist (it'd divide by too many rows).

10.2 String Pattern Matching (LIKE) & Common String Functions

```
LIKE '%Comp%'    -> matches any string containing "Comp" anywhere
LIKE '___'       -> matches exactly 3 characters (each _ = exactly 1 char)
LIKE '___%'      -> matches strings of AT LEAST 3 characters
NOT LIKE         -> mismatches
ESCAPE 'char'    -> lets % or _ be treated literally, e.g. LIKE 'ab\%cd%' ESCAPE '\'
```

Function	Purpose
UPPER / LOWER	Case conversion
LENGTH vs CHAR_LENGTH	LENGTH = bytes; CHAR_LENGTH = characters (differ for multi-byte chars)
CONCAT	Join strings
SUBSTRING(col, start, len)	Extract substring
REPLACE(col, old, new)	Replace text
TRIM / LTRIM / RTRIM	Remove spaces (both/left/right)
INSTR / POSITION	Find position of a substring

- TRAP:** LENGTH() measures **bytes**, CHAR_LENGTH() measures **characters** — identical for plain ASCII but diverge for multi-byte symbols (e.g., '©' → 1 character but 2 bytes).

10.3 Set Operations

```
UNION          -> combines rows, REMOVES duplicates (like mathematical union)
UNION ALL      -> combines rows, KEEPS duplicates
INTERSECT     -> common rows only, removes duplicates (INTERSECT ALL keeps dups)
EXCEPT/MINUS -> rows in first query but not second, removes duplicates (EXCEPT ALL keeps dups)
```

- Requires **union-compatible** relations: same number of columns, matching/compatible domains.
- Cartesian Product:** a bare FROM r1, r2 (no join condition) = every tuple of r1 paired with every tuple of r2; commutative; size = |r1| × |r2|.

10.4 Joins — SQL Syntax

Join type	Behavior
INNER JOIN (or JOIN ... USING(col) / natural join)	Only matching rows from both tables
NATURAL JOIN	Auto-matches on ALL identically-named common columns; shows the common column only once
JOIN ... USING(col)	You explicitly pick which common column(s) must match — other same-named columns are ignored
LEFT OUTER JOIN	All left-table rows + matched right rows (NULL if no match)
RIGHT OUTER JOIN	All right-table rows + matched left rows (NULL if no match)
FULL OUTER JOIN	All rows from both sides, NULL-padded where unmatched

- TRAP:** Natural Join can silently misbehave if two tables share a common column name that ISN'T meant to be a join key — JOIN ... USING(col) avoids this by letting you pick exactly which column(s) to match on.
- Inner joins **lose unmatched tuples** from both sides — this "loss of information" is exactly what Outer Joins are designed to prevent (via NULL-padding).

10.5 GROUP BY & HAVING

```
Logical execution order with aggregation:
FROM -> WHERE (filters ROWS, before grouping) -> GROUP BY (forms groups)
      -> HAVING (filters GROUPS, after aggregation) -> SELECT
```

- Rule:** any SELECT column NOT in GROUP BY must appear **only inside an aggregate function** — otherwise the query is invalid (which row's value would it show for a group of many?).
- Same rule applies to HAVING: a non-aggregated attribute in HAVING must appear in GROUP BY.
- TRAP:** **WHERE filters individual rows BEFORE grouping; HAVING filters entire GROUPS AFTER aggregation.** You cannot use an aggregate function (e.g., AVG(balance)) inside WHERE — it must go in HAVING, since the aggregate doesn't exist yet at the WHERE stage.

10.6 Subqueries — Simple vs Correlated

- **Simple (non-correlated) subquery:** inner query executes ONCE, independently, before the outer query uses its result.
- **Correlated subquery:** inner query REFERENCES a column from the outer query, so it must be **re-evaluated once per outer row** — much slower than a simple subquery.

```
-- Simple subquery: employees earning the max salary in the whole company
SELECT ename FROM Emp WHERE sal = (SELECT MAX(sal) FROM Emp);

-- Correlated subquery: employees earning above their OWN department's average
SELECT e.ename FROM Emp e
WHERE e.sal > (SELECT AVG(sal) FROM Emp WHERE dno = e.dno);
```

- **TRAP:** A correlated subquery cannot simply be "run first" — its result *depends on* the current outer row, so conceptually it re-runs per row (though real optimizers may rewrite it as a join for performance).

10.7 Views

- A **View** = a virtual table defined by a stored SELECT query; stores no data of its own, always reflects the underlying table(s) live.
- Uses: simplify complex/repeated queries, restrict access to sensitive columns/rows, present customized data.

```
CREATE VIEW student_result AS
SELECT name, marks FROM Student;
```

- **TRAP:** A view is generally **NOT updatable** (INSERT/UPDATE/DELETE through it will fail or be restricted) if its defining query contains: aggregate functions, GROUP BY, DISTINCT, or a JOIN across multiple base tables — because there's no unambiguous way to map the change back to base-table rows.

10.8 PL/SQL Basics (Oracle extension)

- PL/SQL = SQL + procedural constructs (loops, conditionals, variables) — used for procedures, functions, triggers.

```
DECLARE -- optional: variable declarations
BEGIN -- executable statements
EXCEPTION -- optional: error handling
END;
```

Component	Returns a value?	Called explicitly?
Procedure	No	Yes (EXEC name;)
Function	Yes	Yes (usable inside a SELECT)
Trigger	No	No — fires automatically on INSERT/UPDATE/DELETE

- **Trigger timing:** BEFORE (runs before the DML event) vs AFTER (runs after) INSERT/UPDATE/DELETE.

Questions

Q1. [Trap] For a `balance` column containing some NULLs, which pair of expressions always produce identical results?

A. COUNT(*) and COUNT(balance) B. COUNT(balance) and SUM(balance)/AVG(balance) C. AVG(balance) and SUM(balance)/COUNT(balance) D. COUNT(*) and SUM(balance)

Answer: C. **Why:** Both AVG and the manual SUM/COUNT(column) form skip NULL rows identically; COUNT(*) would count NULL rows too, making it diverge from COUNT(column).

Q2. [Trap] What is the key functional difference between WHERE and HAVING?

A. They are interchangeable B. WHERE filters rows before grouping; HAVING filters groups after aggregation C. HAVING is used only with SELECT * D. WHERE can use aggregate functions but HAVING cannot

Answer: B. **Why:** WHERE operates on raw rows prior to GROUP BY; HAVING operates on the aggregated groups afterward — hence only HAVING can filter on an aggregate condition like `AVG(balance) > 1500`.

Q3. A query pattern like `LIKE ' _ _ %'` (no spaces, 3 underscores then %) matches:

A. Exactly 3 characters B. At least 3 characters C. At most 3 characters D. Exactly 4 characters

Answer: B. **Why:** Each underscore matches exactly one character (3 mandatory chars), and % matches zero or more additional characters — so the total is "at least 3."

Q4. Which set operation keeps duplicate rows instead of removing them?

A. UNION B. INTERSECT C. UNION ALL D. EXCEPT

Answer: C. **Why:** Plain UNION/INTERSECT/EXCEPT all remove duplicates by default; appending ALL (e.g., UNION ALL) preserves duplicate rows.

Q5. [Trap] A correlated subquery is generally slower than a simple subquery because:

A. It uses more disk space B. It must conceptually re-evaluate once for every row of the outer query, since it references an outer column C. It cannot use WHERE clauses D. It always returns more rows

Answer: B. **Why:** Because a correlated subquery's result depends on the current outer row's value, it is (conceptually) re-executed per outer row, unlike an independent simple subquery executed once.

Q6. A view defined as `CREATE VIEW v AS SELECT dept, AVG(sal) FROM Emp GROUP BY dept;` is:

A. Fully updatable via INSERT/UPDATE B. Not updatable, because it contains an aggregate function and GROUP BY C. Updatable only for DELETE D. Identical to a base table

Answer: B. **Why:** Views containing aggregates/GROUP BY/DISTINCT/multi-table joins generally cannot be updated, since changes can't be unambiguously mapped back to base rows.

Q7. Which join type ensures NO row from either table is lost, padding unmatched sides with NULL?

A. INNER JOIN B. NATURAL JOIN C. FULL OUTER JOIN D. Cartesian Product

Answer: C. **Why:** FULL OUTER JOIN preserves unmatched rows from both the left and right relations, unlike inner/natural joins which only keep matching rows.

Q8. Which PL/SQL component MUST return a value and can be used directly inside a SELECT statement?

A. Procedure B. Trigger C. Function D. Block

Answer: C. **Why:** A Function always returns a value (e.g., `SELECT get_bonus FROM dual;`); a Procedure performs an action but returns nothing, and a Trigger fires automatically rather than being called explicitly.

Q9. [Numerical/Conceptual] `SELECT name FROM Emp WHERE name LIKE 'A%' AND name LIKE '%n';` returns names that:

A. Start with 'A' OR end with 'n' B. Start with 'A' AND end with 'n' C. Contain 'A' and 'n' anywhere D. Are exactly "An"

Answer: B. Why: The AND combines both LIKE conditions — the name must simultaneously start with 'A' and end with 'n' (e.g., "Arjun").

Q10. A trigger declared as `BEFORE INSERT ON Student` will:

A. Run after the row is inserted B. Run automatically before the INSERT completes, and must be called manually otherwise it won't fire C. Run automatically before the INSERT completes; it is never called explicitly D. Only run if explicitly invoked with EXEC

Answer: C. Why: Triggers fire automatically in response to the specified DML event (here, before an INSERT) — unlike procedures/functions, they are never called explicitly.

6.11 Tuple & Domain Relational Calculus

11.1 Declarative vs Procedural Query Languages

Language	Nature	Specifies
Relational Algebra (RA)	Procedural	HOW to get the result (explicit sequence of operations)
Tuple Relational Calculus (TRC)	Declarative	WHAT result is wanted (tuple-variable based)
Domain Relational Calculus (DRC)	Declarative	WHAT result is wanted (domain-variable based)

- SQL itself is declarative, and is formally grounded in both RA and TRC — this is why the "underlying model" question (RA=procedural, calculus=non-procedural) is a favorite exam fact.

11.2 Tuple Relational Calculus (TRC)

General form: $\{ t \mid P(t) \}$

- t is a **tuple variable** ranging over an entire relation; result = set of all tuples t for which predicate $P(t)$ is true.
- $t[A]$ or $t.A$ = value of tuple t on attribute A . $t \in r$ (or $r(t)$) = "t is a tuple of relation r ."
- Free vs Bound variable**: a tuple variable is **free** unless quantified by $\exists$ or $\forall$; quantified variables are **bound**.
- Quantifiers: $\exists t \in r (Q(t))$ = "there EXISTS a tuple t in r such that $Q(t)$ holds." $\forall t \in r (Q(t))$ = "Q(t) holds for ALL tuples t in r ."
- Logical connectives allowed in $P(t)$: $\wedge$ (**AND**), $\vee$ (**OR**), $\neg$ (**NOT**), $\Rightarrow$ (**implies**).
- Key equivalences:

```

P1 AND P2           ≡ NOT(NOT(P1) OR NOT(P2))
FOR ALL t in r (P1(t)) ≡ NOT EXISTS t in r (NOT P1(t))    <- universal via negated existential
P1 => P2             ≡ NOT(P1) OR P2

```

Worked TRC Examples — Student(RollNo, Name, Branch)

Find all CSE students:
 $\{ t \mid \text{Student}(t) \text{ AND } t.\text{branch} = \text{'CSE'} \}$

Find Roll No only of CSE students (projection within calculus):
 $\{ t.\text{RollNo} \mid \text{Student}(t) \text{ AND } t.\text{branch} = \text{'CSE'} \}$

Worked TRC Example — banking schema (loan, borrower, depositor)

Loans over 1200:
 $\{ t \mid t \in \text{loan AND } t[\text{amount}] > 1200 \}$

Customers with a loan AND an account (both):
 $\{ t \mid \exists s \in \text{borrower } (t[\text{cname}] = s[\text{cname}]) \text{ AND } \exists u \in \text{depositor } (t[\text{cname}] = u[\text{cname}]) \}$

Customers with a loan but NO account (existential + negation):
 $\{ t \mid \exists s \in \text{borrower } (t[\text{cname}] = s[\text{cname}]) \text{ AND NOT } \exists u \in \text{depositor } (t[\text{cname}] = u[\text{cname}]) \}$

- TRAP**: "Has X or Y" uses $\vee$ (**OR**) between two $\exists$ clauses; "has X but NOT Y" uses $\exists \dots \text{AND NOT } \exists \dots$ — mixing these up is the single most common TRC mistake.

11.3 Domain Relational Calculus (DRC)

General form: $\{ \langle x_1, x_2, \dots, x_n \rangle \mid P(x_1, x_2, \dots, x_n) \}$

- Variables ($x_1 \dots x_n$) range over **individual domain values** (one variable per attribute), NOT whole tuples — this is the key structural difference from TRC.
- To produce a degree- n result relation, you need exactly n domain variables (one per output attribute).
- Atom forms: $\langle x_1, \dots, x_n \rangle \in r$ (membership), $x \text{ op } y$ (compare two domain variables), $x \text{ op } c$ (compare a domain variable to a constant).
- DRC is the theoretical basis of the **QBE (Query-By-Example)** language.

Worked DRC Examples

Student(RollNo, Name, Branch) — find CSE students' details:
 $\{ \langle \text{RollNo}, \text{Name}, \text{Branch} \rangle \mid \text{Student}(\text{RollNo}, \text{Name}, \text{Branch}) \text{ AND } \text{Branch} = \text{'CSE'} \}$

Instructor(id, name, dept, salary) — salary > 80000:
 $\{ \langle i, n, d, s \rangle \mid \langle i, n, d, s \rangle \in \text{Instructor AND } s > 80000 \}$

loan(loan_no, branch, amount) — branch/loan_no/amount for loans over 1200:
 $\{ \langle l, b, a \rangle \mid \langle l, b, a \rangle \in \text{loan AND } a > 1200 \}$

11.4 Safety of Expressions

- A calculus expression must be **safe** — guaranteed to yield a **finite** result.
- Unsafe example**: $\{ t \mid \text{NOT } (t \in r) \}$ — this could denote an infinite set (every tuple NOT in r , over an unbounded domain), which is meaningless for a real database.
- TRAP**: Negation and universal quantification ($\forall$) are the usual sources of unsafe expressions — always check that a negated/universal condition is still bounded by an explicit relation membership check.

11.5 RA vs TRC vs DRC — Summary

Feature	RA	TRC	DRC
Style	Procedural	Declarative	Declarative
Variable ranges over	(no variables — operator chain)	Whole tuples	Single domain values
Basis for	—	Part of SQL's theoretical foundation	QBE language
"For all" queries	Division operator	$\forall$ quantifier	$\forall$ quantifier

Questions

Q1. Which of the following best distinguishes Relational Algebra from Relational Calculus?

- A. RA is declarative, Calculus is procedural B. RA is procedural (specifies how), Calculus is declarative (specifies what) C. They are functionally identical with no difference
D. Calculus cannot express joins

Answer: B. Why: RA is a step-by-step procedural language; both TRC and DRC are declarative, specifying only the desired result condition.

Q2. In TRC, a tuple variable that is NOT quantified by $\exists$ or $\forall$ is called:

- A. Bound B. Free C. Universal D. Undefined

Answer: B. Why: By definition, a tuple variable is free unless explicitly quantified (bound) by an existential or universal quantifier.

Q3. [Trap] Which TRC expression correctly finds customers who have a loan but NO account?

- A. $\{t \mid \exists s \in \text{borrower}(t[\text{cname}] = s[\text{cname}]) \text{ OR } \exists u \in \text{depositor}(t[\text{cname}] = u[\text{cname}])\}$ B. $\{t \mid \exists s \in \text{borrower}(t[\text{cname}] = s[\text{cname}]) \text{ AND } \exists u \in \text{depositor}(t[\text{cname}] = u[\text{cname}])\}$ C. $\{t \mid \exists s \in \text{borrower}(t[\text{cname}] = s[\text{cname}]) \text{ AND NOT } \exists u \in \text{depositor}(t[\text{cname}] = u[\text{cname}])\}$ D. $\{t \mid \text{NOT } \exists s \in \text{borrower}(t[\text{cname}] = s[\text{cname}])\}$

Answer: C. Why: "Has a loan" needs an existential match in borrower; "but no account" needs the negation of an existential match in depositor — combined with AND.

Q4. In Domain Relational Calculus, a query producing a degree-3 result relation requires:

- A. 1 domain variable B. Exactly 3 domain variables (one per output attribute) C. As many variables as base relations used D. No variables, only constants

Answer: B. Why: DRC requires one domain variable per output attribute — a degree-3 result needs 3 domain variables, e.g., $\langle x_1, x_2, x_3 \rangle$.

Q5. [Trap] Why is $\{t \mid \text{NOT}(t \in r)\}$ considered an unsafe expression?

- A. It uses invalid syntax B. It could denote an infinite set of tuples, since NOT($t \in r$) is unbounded over the whole domain C. NOT is not allowed in relational calculus at all
D. It only works for empty relations

Answer: B. Why: Without a bounding relation membership, the negation could range over an infinite domain of possible tuples — violating safety (finiteness) requirements.

Q6. The universal quantifier $\forall$ in relational calculus is most analogous to which relational algebra operator?

- A. Selection (σ) B. Projection (π) C. Division ($\div$) D. Cartesian Product ($\times$)

Answer: C. Why: Both $\forall$ ("for all") and Division express "related to ALL values of some set" — e.g., "find X related to every Y."

Q7. Domain Relational Calculus is described as the theoretical basis for which practical query language?

- A. SQL B. QBE (Query-By-Example) C. Relational Algebra D. PL/SQL

Answer: B. Why: DRC's domain-variable-per-attribute structure directly underlies QBE's grid-based query interface.

6.12 Transactions, ACID Properties & Serializability

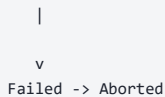
(Serializability testing via precedence graph is one of the most numerically-tested DBMS topics in GATE/ISRO.)

12.1 Transaction Basics & ACID

- **Transaction** = a set of logically related instructions forming one logical unit of work; either executes fully or not at all (removes the "partial execution" problem that plain OS-level instruction atomicity can't prevent).
- Two DB-level operations: **Read(X)** (disk → memory), **Write(X)** (memory → disk).
- **ACID:**
- **Atomicity** — all-or-nothing; enforced by the recovery manager.
- **Consistency** — DB moves from one consistent state to another; enforced by programmers/integrity constraints.
- **Isolation** — concurrent transactions don't interfere, as if run in isolation; enforced by the concurrency control manager.
- **Durability** — committed changes survive failures; enforced by the recovery manager.
- **TRAP:** Isolation is the concurrency-control manager's job; Atomicity & Durability are the recovery manager's job; Consistency is the programmer's/constraint's job — a commonly confused mapping.

12.2 Transaction States

Active → Partially Committed → Committed



- **Active:** executing. **Partially committed:** final statement executed, but output may still be in memory (not yet safely on disk) — can still be forced to abort. **Failed:** cannot proceed further (must roll back). **Aborted:** rolled back, DB restored to pre-transaction state. **Committed:** successfully finished, changes durable.

12.3 Why Concurrency — and Its Anomalies

- Concurrent execution improves throughput, resource utilization, and response/waiting time — but naive interleaving can violate consistency even if each transaction individually respects ACID.

Anomaly	What happens
Lost Update (write-write)	Two transactions write the same item with no read in between; the 2nd write silently overwrites the 1st.
Dirty Read (read-write)	A transaction reads a value written by an uncommitted transaction, which later aborts — the reader now holds a value that never really existed.
Unrepeatable Read	A transaction reads the same item twice and gets different values because another transaction updated it in between.
Phantom Read	A transaction re-executes a query and finds new/vanished rows because another transaction inserted/deleted matching rows in between.

- **TRAP:** Dirty Read is about reading an **uncommitted** write; Unrepeatable Read is about a value **changing** between two reads of the same item (both transactions may be committed) — don't conflate the two.

12.4 Schedules

- **Schedule:** an interleaving of operations from multiple transactions; each transaction's own operations must keep their original relative order.
- **Serial schedule:** one transaction runs fully, then the next (no interleaving). For n transactions, there are $n!$ serial schedules. Every serial schedule is trivially consistent, but throughput is poor.
- **Non-serial schedule:** operations interleaved across transactions (real concurrent execution).

Total possible schedules (given instruction counts $n_1, n_2, \dots, n_k$ for k transactions)
 $= (n_1 + n_2 + \dots + n_k)! / (n_1! \times n_2! \times \dots \times n_k!)$
 Non-serial schedule count = Total - $k!$ ($k!$ = the serial ones)

Worked Example

T1 has 2 instructions, T2 has 3 instructions.
 Total schedules = $(2+3)! / (2! \times 3!) = 120/12 = 10$.
 Serial schedules = $2! = 2$ (T1-then-T2, or T2-then-T1).
 Non-serial schedules = $10 - 2 = 8$.

12.5 Conflict Serializability

- Two operations **conflict** iff: (1) different transactions, (2) same data item, (3) at least one is a WRITE.

Read-Read → Never conflicts
 Read-Write / Write-Read / Write-Write → Conflicts (if different transactions, same item)

- **Conflict equivalent:** schedule S can be transformed into S' purely by swapping adjacent *non-conflicting* operations.
- **Conflict serializable:** S is conflict-equivalent to some serial schedule.

Precedence Graph Method (the standard test)

1. One node per transaction.
2. For every pair of conflicting operations, add edge $T_i \rightarrow T_j$ if T_i 's operation occurs BEFORE T_j 's operation in S .
3. Schedule is conflict serializable $\Leftrightarrow$ the graph has NO CYCLE.
4. If acyclic, any topological order of the graph is a valid equivalent serial schedule.

- **TRAP:** You must check EVERY conflicting pair across the whole schedule, not just "adjacent" operations — a single missed conflicting pair can hide a cycle.

Worked Example

$S: R1(A) R2(B) W1(A) R3(A) W2(B) W3(A)$

Conflicts on A: $R1(A) \rightarrow W3(A)$ gives **$T1 \rightarrow T3$** ; $W1(A) \rightarrow R3(A)$ gives **$T1 \rightarrow T3$** (same edge, no new info); $R1(A)$ vs $R3(A)$ don't conflict (both reads).

Conflicts on B: only $T2$ touches B ($R2(B), W2(B)$) $\rightarrow$ same transaction, no edge.

Graph edges: **$T1 \rightarrow T3$** only. $T2$ is unconstrained.

Acyclic $\Rightarrow$ conflict serializable. Valid equivalent serial orders: **$T1, T2, T3$** or **$T2, T1, T3$** or **$T1, T3, T2$** (any order respecting $T1$ before $T3$).

12.6 View Serializability (weaker/broader than conflict serializability)

- **View equivalent** (S and S'): (1) same "reads-from" relationships — if T_i reads the initial value of Q in S , it must also read the initial value of Q in S' ; if T_i reads Q as written by T_j in S , same in S' ; (2) same **final writer** of each data item in both schedules.
- **View serializable:** S is view-equivalent to some serial schedule. Checking this is NP-complete in general (compare against all $n!$ serial orders).
- **Key containment fact:** every **conflict serializable** schedule is **view serializable**, but NOT every view serializable schedule is conflict serializable.
- **TRAP:** Direction matters — "conflict serializable $\Rightarrow$ view serializable" is TRUE; the reverse ("view serializable $\Rightarrow$ conflict serializable") is FALSE. A classic exam trap reverses this.
- **When they can differ:** only when the schedule has a **blind write** (a write with no prior read of that item by the same transaction, in that schedule). With no blind writes, "not conflict serializable" $\Rightarrow$ "not view serializable" too.

Worked Example (the classic conflict-vs-view-serializable gap)

$S: R1(B) R3(C) R1(A) W2(A) W1(A) W2(B) W3(A) W1(B) W3(B) W3(C)$

Precedence graph: on A $\rightarrow T1 \rightarrow T2$ ($R1(A)$ before $W2(A)$) **and** $T2 \rightarrow T1$ ($W2(A)$ before $W1(A)$) — a **2-cycle** $\Rightarrow$ **NOT conflict serializable**.

But $T2$'s writes $W2(A), W2(B)$ are **blind writes** ($T2$ never reads). Checking view-equivalence: $T1$ reads initial A and B (so $T1$ must precede any writer of A/B in a serial order); $T3$ is the final writer of A, B, C in S (so $T3$ must be last). Checking **$T1, T2, T3$** : reads-from and final-writer conditions both match S exactly.

$\Rightarrow S$ is **view serializable** (equivalent to serial $T1, T2, T3$) **despite not being conflict serializable** — precisely because the "conflicting" writes by $T2$ were blind and never actually observed by anyone.

Questions

Q1. [Numerical] $T1$ has 3 instructions and $T2$ has 2 instructions. How many non-serial schedules are possible?

A. 10 B. 8 C. 120 D. 6

Answer: B. Why: Total = $5!/(3!2!) = 10$; serial = $2! = 2$; non-serial = $10 - 2 = 8$.

Q2. Two operations belonging to different transactions on the same data item, where both are READs, are:

A. Always conflicting B. Never conflicting C. Conflicting only if isolation level is low D. Conflicting only in strict schedules

Answer: B. Why: Conflict requires at least one WRITE; two reads never conflict regardless of transaction or isolation level.

Q3. [Application] Schedule $S: R1(x) R1(y) R2(x) R2(y) W2(y) W1(x)$ — is S conflict serializable?

A. Yes, equivalent to $T1, T2$ B. Yes, equivalent to $T2, T1$ C. No — the precedence graph has a cycle D. Cannot be determined

Answer: C. Why: On x : $R1(x)@1$ before $R2(x)@3$ (no conflict, both reads only relevant if later write — actually the conflict is $R2(x)$ then $W1(x)$: $T2 \rightarrow T1$). On y : $R1(y)@2$ then $W2(y)@5$ gives $T1 \rightarrow T2$. Combined edges $T1 \rightarrow T2$ (from y) and $T2 \rightarrow T1$ (from x) form a 2-cycle $\Rightarrow$ not conflict serializable.

Q4. [Trap] Which statement about conflict and view serializability is TRUE?

A. Every view serializable schedule is conflict serializable B. Every conflict serializable schedule is view serializable C. The two are always identical D. Neither implies the other

Answer: B. Why: Conflict serializability is the stronger, narrower condition; it always implies view serializability. The reverse implication fails whenever blind writes are present.

Q5. A schedule has a cycle in its precedence graph but contains blind writes. It is:

A. Definitely not view serializable either B. Possibly still view serializable — must check view-equivalence conditions directly C. Automatically view serializable D. Automatically strict

Answer: B. Why: A precedence-graph cycle only rules out conflict serializability. With blind writes present, the schedule might still satisfy the (weaker) view-equivalence conditions — it must be checked separately, not assumed either way.

Q6. In a serial schedule of n transactions, how many distinct serial schedules exist regardless of each transaction's instruction count?

A. n B. $n!$ C. 2^n D. n^2

Answer: B. Why: A serial schedule is just an ordering of the n transactions as whole blocks — $n!$ such orderings exist, independent of how many instructions each transaction has.

Q7. [Trap] A transaction $T2$ reads a value written by transaction $T1$, which later ABORTS. This is an example of:

A. Lost update B. Dirty read C. Unrepeatable read D. Phantom read

Answer: B. Why: Reading data written by an uncommitted (and here, ultimately aborted) transaction is precisely the dirty-read anomaly.

Q8. [Application] Schedule $S: R2(x) W2(x) R2(y) W2(y) R1(y) W1(x)$ — checking conflicts between $T1$ and $T2$ only, which edge appears in the precedence graph?

A. $T1 \rightarrow T2$ only B. $T2 \rightarrow T1$ only C. Both $T1 \rightarrow T2$ and $T2 \rightarrow T1$ (cycle) D. No edge at all

Answer: B. Why: On y : $W2(y)$ precedes $R1(y) \Rightarrow T2 \rightarrow T1$. On x : $W2(x)$ precedes $W1(x) \Rightarrow T2 \rightarrow T1$. Both conflicts point the same direction ($T2 \rightarrow T1$), so no cycle — this schedule is conflict serializable (equivalent to $T2, T1$).

Q9. Which of the following is/are TRUE?

A. All view serializable schedules are also conflict serializable B. All conflict serializable schedules are also view serializable, and if a schedule is not view serializable then it is not conflict serializable C. Conflict and view serializability are unrelated concepts D. View serializability is always easier to compute than conflict serializability

Answer: B. Why: "Conflict serializable $\Rightarrow$ view serializable" and its logical contrapositive "not view serializable $\Rightarrow$ not conflict serializable" are both true; the converse statements (A) are false, and checking view serializability is NP-complete — harder, not easier, than the polynomial-time precedence-graph check for conflict serializability.

6.13 Recoverability & Concurrency Control Protocols

13.1 Recoverability Classification

- **Recoverable schedule:** if Tj reads a data item previously written by Ti, then Ti must **commit before** Tj commits.
- **Cascadeless schedule:** if Tj reads a data item previously written by Ti, then Ti must **commit before** Tj even **reads** it (i.e., no dirty reads at all). Avoids **cascading rollback** (one abort forcing a chain of other rollbacks).
- **Strict schedule:** if Tj reads OR writes a data item previously written by Ti, Ti must commit before Tj's read/write. Strongest — allows trivial UNDO via before-images since no one ever touched an uncommitted value.

Containment hierarchy (strict nesting):
 Strict $\subset$ Cascadeless $\subset$ Recoverable $\subset$ All Schedules

- **TRAP:** This is a strict one-way nesting — every Strict schedule is Cascadeless, every Cascadeless schedule is Recoverable, but NOT vice versa. A schedule can be Recoverable without being Cascadeless, and Cascadeless without being Strict.

Quick Classification Procedure

1. Scan for a DIRTY READ (Ti reads an item written by an uncommitted Tj).
2. No dirty read anywhere $\Rightarrow$ schedule is BOTH Recoverable AND Cascadeless.
3. Dirty read present $\Rightarrow$ NOT Cascadeless. Then check commit order:
 - If every writer commits before its dirty-reader commits $\Rightarrow$ Recoverable (but not cascadeless).
 - If any dirty-reader commits before its writer commits $\Rightarrow$ NOT Recoverable.

Worked Examples (verified)

- | | |
|--------------------------|---|
| A) W1(X); R2(X); C1; C2; | -> T2 dirty-reads X from uncommitted T1 (not cascadeless),
but T1 commits before T2 commits -> RECOVERABLE, not cascadeless. |
| B) W1(X); W2(X); C1; C2; | -> T2 overwrites X before T1 commits (allowed - cascadeless only
restricts READS of uncommitted data, not writes); nobody reads X
-> RECOVERABLE and CASCADELESS, but NOT STRICT (a write touched
X before T1's commit). |
| C) W1(X); C1; W2(X); C2; | -> T2 only touches X after T1 already committed -> STRICT
(and therefore also cascadeless and recoverable). |

- **TRAP:** Cascadelessness only cares about dirty **reads**; a transaction is free to **write** over another's uncommitted data and still be cascadeless. Only **strictness** also blocks that early write.

13.2 Concurrency Control — Overview

- Goal: guarantee (conflict) serializability while maximizing concurrency, with acceptable overhead.
- Three main approaches: **Lock-based** (2PL family), **Timestamp-ordering**, **Validation/optimistic** (for mostly-read workloads with low conflict rates).

13.3 Timestamp Ordering Protocol (TO)

- Each transaction gets a unique, fixed **TS(Ti)** on entry (never repeats, no reset needed); serialization order = timestamp order.
- Each data item Q tracks **W-timestamp(Q)** (latest committed-write's TS) and **R-timestamp(Q)** (latest successful read's TS).

Read(Q) by Ti:
 if TS(Ti) < W-timestamp(Q): REJECT, roll back Ti (Ti would read an already-overwritten value)
 else: allow; R-timestamp(Q) = max(R-timestamp(Q), TS(Ti))

Write(Q) by Ti:
 if TS(Ti) < R-timestamp(Q): REJECT, roll back Ti (a later txn already needed a newer value)
 elif TS(Ti) < W-timestamp(Q): REJECT, roll back Ti (Ti's write is already obsolete)
 else: allow; W-timestamp(Q) = max(W-timestamp(Q), TS(Ti))

- A rolled-back transaction is restarted with a **new, later** timestamp.
- **TRAP:** TO guarantees conflict serializability (conflicting ops are always processed in TS order, so the precedence graph can never cycle) and hence view serializability too — but it does **NOT** guarantee recoverability or cascadelessness (no rule ties commit order to anything), and it IS deadlock-free (a transaction is immediately rejected/rolled back rather than made to wait).

13.4 Thomas's Write Rule (TO optimization)

- Modifies ONLY the write rule: if TS(Ti) < W-timestamp(Q), instead of rejecting, **simply ignore** that write (it's an obsolete write that no future reader will ever see, so skipping it is harmless).
- **TRAP:** The **read rule is completely unchanged** from basic TO — Thomas's rule improves concurrency for **writes only**. A read that fails the TO read-check (TS(Ti) < W-timestamp(Q)) is still rejected exactly as in basic TO.
- Net effect: allows some **view-serializable** (but not conflict-serializable) schedules that basic TO would have rejected — trades strict conflict-serializability for higher concurrency.

13.5 Lock-Based Protocols & Two-Phase Locking (2PL)

- **Shared (S) lock:** read-only, multiple transactions may hold S on the same item simultaneously. **Exclusive (X) lock:** read+write, no other transaction may hold any lock on that item. Compatibility: S-S = compatible; S-X, X-X = incompatible.
- Plain locking (lock/unlock freely) does NOT guarantee serializability by itself — need a locking discipline.

- **Two-Phase Locking (2PL):** every transaction (not the whole schedule) splits into a **Growing phase** (only acquire locks, never release) and a **Shrinking phase** (only release locks, never acquire). Once the first unlock happens, no more locks may be taken.
- **2PL guarantees conflict serializability** (and hence view serializability) — the equivalent serial order follows the transactions' **lock-point** order. It does **NOT** guarantee deadlock-freedom, recoverability, or cascadelessness by itself.
- **TRAP:** "If a schedule follows 2PL, it is conflict serializable" is TRUE, but the converse is FALSE — a schedule can be conflict serializable without ever being produced by 2PL.

13.6 2PL Variants — Complete Comparison

Conservative 2PL: acquire ALL needed locks BEFORE starting execution (or wait entirely) - no true growing phase overlap with execution. Needs advance knowledge of data needs.
 Basic 2PL: standard growing/shrinking split, locks released as soon as no longer needed.
 Rigorous 2PL: hold ALL locks (both S and X) until commit - no real shrinking phase at all.
 Strict 2PL: hold only X-locks until commit (S-locks may be released earlier) - simplified Rigorous 2PL; most widely used in real DBMSs.

Protocol	Conflict Serializable	View Serializable	Recoverable	Cascadeless	Deadlock-free
Timestamp Ordering	YES	YES	NO	NO	YES
Thomas's Write Rule	NO	YES	NO	NO	YES
Basic 2PL	YES	YES	NO	NO	NO
Conservative 2PL	YES	YES	NO	NO	YES
Rigorous 2PL	YES	YES	YES	YES	NO
Strict 2PL	YES	YES	YES	YES	NO

- **TRAP:** Conservative 2PL is the ONLY 2PL variant that is deadlock-free (since it never lets a transaction hold some locks while waiting for others — no hold-and-wait). Rigorous/Strict 2PL give recoverability+cascadelessness (via strictness) but still risk deadlock.
- **TRAP:** Thomas's Write Rule is the only protocol in this table that sacrifices conflict serializability (NO) while keeping view serializability (YES) — a direct consequence of it deliberately allowing certain blind-write reorderings.

Questions

Q1. [Application] In schedule W1(X); R2(X); C1; C2; — classify it.

A. Not recoverable B. Recoverable but not cascadeless C. Cascadeless but not recoverable D. Strict

Answer: B. Why: T2 dirty-reads X from uncommitted T1 (violates cascadelessness), but T1 commits before T2 commits, satisfying recoverability.

Q2. [Trap] Which is the correct containment relationship among schedule classes?

A. Recoverable $\subset$ Cascadeless $\subset$ Strict B. Strict $\subset$ Cascadeless $\subset$ Recoverable C. All three are equivalent D. Cascadeless $\subset$ Strict $\subset$ Recoverable

Answer: B. Why: Strict is the most restrictive (implies Cascadeless), Cascadeless is more restrictive than Recoverable — the nesting narrows as you add restrictions, so Strict is the smallest, most-restrictive set.

Q3. Under the Timestamp Ordering protocol, a Read(Q) by Ti is rejected when:

A. $TS(Ti) < R\text{-timestamp}(Q)$ B. $TS(Ti) < W\text{-timestamp}(Q)$ C. $TS(Ti) > W\text{-timestamp}(Q)$ D. $TS(Ti) = W\text{-timestamp}(Q)$

Answer: B. Why: If Ti's timestamp is older than the latest committed write's timestamp, Ti would be reading an already-overwritten (obsolete) value, so it is rejected and rolled back.

Q4. [Trap] Thomas's Write Rule improves on basic Timestamp Ordering by:

A. Relaxing both the read and write rejection rules B. Relaxing ONLY the write rule (ignoring obsolete blind writes instead of rejecting them); the read rule is unchanged C. Eliminating rollbacks entirely D. Making the protocol deadlock-prone

Answer: B. Why: Thomas's rule specifically allows an obsolete write to be silently ignored rather than triggering a rollback — but reads still follow the exact same rejection rule as basic TO.

Q5. Which 2PL variant is guaranteed to be deadlock-free?

A. Basic 2PL B. Strict 2PL C. Conservative 2PL D. Rigorous 2PL

Answer: C. Why: Conservative 2PL acquires all required locks before execution begins (or waits entirely), eliminating the hold-and-wait condition necessary for deadlock.

Q6. [Trap] Basic 2PL guarantees conflict serializability. Does it also guarantee recoverability?

A. Yes, always B. No — a schedule can follow 2PL and still not be recoverable C. Only if all locks are exclusive D. Only for read-only transactions

Answer: B. Why: 2PL constrains only lock acquisition/release ordering relative to conflicts; it says nothing about commit ordering, so a non-recoverable schedule can still legally arise under Basic 2PL.

Q7. The key structural difference between Strict 2PL and Rigorous 2PL is:

A. Strict 2PL holds only X-locks until commit; Rigorous 2PL holds both S and X locks until commit B. Rigorous 2PL has no growing phase C. Strict 2PL is not two-phase D. They are identical in every respect

Answer: A. Why: Rigorous 2PL is the more conservative variant, retaining shared locks too until commit, whereas Strict 2PL allows shared locks to be released earlier — both still guarantee recoverability/cascadelessness via strictness on writes.

Q8. [Application] Which protocol allows a schedule to be view serializable but explicitly NOT conflict serializable?

A. Strict 2PL B. Conservative 2PL C. Thomas's Write Rule D. Basic Timestamp Ordering

Answer: C. Why: Thomas's Write Rule intentionally permits ignoring certain obsolete (blind) writes, which can produce schedules that are view serializable without being conflict serializable — the only protocol in this set with that property.

Q9. A schedule with no dirty reads anywhere is automatically:

A. Only recoverable, not cascadeless B. Both recoverable and cascadeless C. Neither, more checks needed D. Automatically strict as well

Answer: B. Why: With zero dirty reads, both the cascadeless condition (no reads of uncommitted data) and recoverability (trivially, since no problematic reads-from relationships exist) are automatically satisfied. Strictness is a separate, stronger condition about writes too and isn't automatically implied.

6.14 Database Recovery

14.1 Storage Types & Buffer Management Policies

- **Volatile storage** (RAM/cache) — lost on crash/power-off. **Non-volatile storage** (disk, tape, SSD) — survives. **Stable storage** — an idealized concept (survives ALL failures); approximated via multiple copies on different non-volatile devices.
- Buffer management policy pairs (determine WHEN dirty pages reach disk):
- **Steal** — an uncommitted transaction's page CAN be written to disk before commit. **No-steal** — cannot.
- **Force** — all pages of a transaction are flushed to disk immediately at commit. **No-force** — they aren't (flushed later, lazily).
- **TRAP: Steal + No-force** is what real systems use (best performance) — but this combination is exactly why **both UNDO and REDO are needed** during recovery (steal → uncommitted work may be on disk needing undo; no-force → committed work may NOT be on disk needing redo).

14.2 Log-Based Recovery & WAL

- **Log record types:** `<Ti start>`, `<Ti, Xj, V1, V2>` (update: old value V1, new value V2), `<Ti commit>`, `<Ti abort>`.
- **Write-Ahead Logging (WAL)** — the single most important rule in this chapter:

Atomicity Rule: the log entry for an insert/update/delete MUST reach stable storage BEFORE the actual data change is written to the database on disk.
Durability Rule: ALL log entries for a transaction must reach disk BEFORE that transaction's commit record is written to disk.

- **TRAP:** WAL means **log-first, data-second** — students often reverse this. Without WAL, a crash between the data-write and log-write would leave an update with no way to undo it.
- Each log record gets a unique **LSN (Log Sequence Number)**; **undo** restores the old value, **redo** re-applies the new value — both operations must be **idempotent** (repeating them has the same effect as doing them once, since recovery may re-scan the same log records).

14.3 Deferred vs Immediate Update

Feature	Deferred Update (No-Undo/Redo)	Immediate Update (Undo/Redo)
DB updated on disk...	...only AFTER commit	...possibly BEFORE commit
On crash, need UNDO?	Never (DB never touched by uncommitted txn)	Yes , for uncommitted transactions
On crash, need REDO?	Yes, for committed transactions	Yes, for committed transactions
Redo condition	Both <code><Ti start></code> and <code><Ti commit></code> present in log	<code><Ti commit></code> present (redo); absent (undo)

- **TRAP:** "No-Undo/Redo" (Deferred) is named for what recovery must do, NOT for what happens during normal execution — a transaction still executes normally, it just doesn't touch the actual DB on disk until commit.

14.4 Checkpoints

- Periodically: (1) flush all in-memory log records to stable storage, (2) flush all modified buffer pages to disk, (3) write a `<checkpoint L>` log record (L = list of transactions active at checkpoint time). Updates are paused during this process.
- **Purpose:** bounds how far back the recovery log must be scanned — transactions that committed *before* the checkpoint need no redo, cutting recovery time drastically on a long-running log.

14.5 Shadow Paging

- Alternative to logging: maintain **current page table** + **shadow page table** (identical at start). First write to any page → copy-on-write to a new page; current table repoints to the copy; shadow table is untouched.
- **Commit** = atomically flip a single on-disk pointer so the current table becomes the new shadow table.
- Advantages: **no log needed, no undo/redo, trivial/instant recovery** (crash before commit → old shadow table is still valid, nothing to undo).
- Disadvantages: data fragmentation (related pages scattered), needs garbage collection of orphaned pages, expensive to copy the whole page table each transaction (mitigated with a B+-tree-structured page table so only the path to the updated leaf is copied), poor support for concurrent transactions. Rarely used in modern practice — **TRAP:** don't assume shadow paging is "the standard" method; log-based (WAL/ARIES) recovery is what real systems use.

14.6 ARIES Recovery Algorithm

(ARIES = Algorithms for Recovery and Isolation Exploiting Semantics — the industry-standard log-based recovery algorithm.)

3 passes, always in this order:

1. ANALYSIS — scan FORWARD from the last checkpoint.
Rebuilds: Transaction Table (active txns) + Dirty Page Table (pages modified but maybe not on disk).
Determines: RedoLSN = where the Redo pass must start.
2. REDO — scan FORWARD from RedoLSN.
Re-applies EVERY logged update from that point on -- even for transactions that will later be undone ("repeating history"). Uses PageLSN vs log-record LSN to skip redos already reflected on disk (idempotence).
3. UNDO — scan BACKWARD from the end of the log.
Rolls back all "loser" transactions (active/uncommitted at crash time), writing a Compensation Log Record (CLR) for each undone action so undo is never repeated.

- Key data structures: **LSN** (sequentially increasing log position), **PageLSN** (LSN of the last update reflected on that page — prevents redoing an already-applied update), **Dirty Page Table** (RecLSN = earliest log record that dirtied this page — lets Redo skip pages/records that are provably already safe on disk), **CLR** (redo-only log record written during undo, so undo actions are never re-undone).

- **TRAP:** ARIES deliberately **redoes even the actions of transactions that will be undone next** ("repeating history") — this looks wasteful but massively simplifies the algorithm (Redo never needs to check whether a transaction eventually committed; Undo cleans up afterward).
- **TRAP:** The pass order is always **Analysis → Redo → Undo**, never Undo-before-Redo.

Questions

Q1. What is the correct order of ARIES recovery phases?

A. Redo, Undo, Analysis B. Analysis, Undo, Redo C. Analysis, Redo, Undo D. Undo, Analysis, Redo

Answer: C. Why: Analysis first rebuilds the transaction/dirty-page tables and finds where Redo should start; Redo re-applies all logged updates; Undo then rolls back uncommitted ("loser") transactions.

Q2. [Trap] The Write-Ahead Logging rule requires that:

A. Data is written to disk before its log record B. The log record for a change must reach stable storage before the data change itself is written to disk C. Only committed transactions are logged D. Checkpoints replace the need for logging

Answer: B. Why: WAL guarantees the log always has enough information to undo/redo a change, which only works if the log entry is durable before the actual data page is overwritten.

Q3. Under the Deferred Update (No-Undo/Redo) technique, why is UNDO never required during recovery?

A. Because logging is disabled B. Because the database on disk is never modified by a transaction until after it commits C. Because checkpoints eliminate the need D. Because only redo-only CLR's are used

Answer: B. Why: Since updates are deferred and only written to the DB after commit, an uncommitted transaction can never have left any trace on disk to undo.

Q4. A transaction's log shows `<Ti start>` but no `<Ti commit>` at the time of a crash under the Immediate Update scheme. What recovery action is needed?

A. Redo Ti B. Undo Ti C. No action needed D. Redo then Undo Ti

Answer: B. Why: Immediate update may have written some of Ti's changes to disk before the crash; since Ti never committed, those changes must be undone using the log's old values.

Q5. What is the main advantage of Shadow Paging over log-based recovery?

A. Better concurrency support B. No log records are needed; recovery is nearly instantaneous since commit is a single atomic pointer switch C. It uses less disk space than logging overall D. It avoids garbage collection

Answer: B. Why: Shadow paging's commit is just repointing to the current page table, so a crash before commit leaves the old shadow table fully valid — no undo/redo machinery needed. Its downsides are fragmentation, garbage collection, and poor concurrency.

Q6. [Trap] Why does ARIES redo the actions of transactions that will later be undone?

A. It is a bug in the algorithm B. "Repeating history" simplifies the Redo pass by not requiring it to know a transaction's eventual fate; Undo cleans up afterward C. Because Undo cannot run without a prior Redo pass touching those pages D. Because CLR's require it

Answer: B. Why: ARIES intentionally decouples Redo from transaction outcome — Redo restores the exact pre-crash state, and Undo then removes the effects of any transaction that hadn't committed.

Q7. What information does ARIES's PageLSN serve to prevent?

A. Duplicate log entries B. Redoing an update that is already reflected in the on-disk page C. Undoing committed transactions D. Checkpoint corruption

Answer: B. Why: By comparing a log record's LSN to the page's stored PageLSN, the Redo pass can skip updates already durably applied — ensuring idempotent recovery.

Q8. In the Steal/No-Force buffer policy combination used by most real DBMSs, which recovery operations become necessary?

A. Neither undo nor redo B. Only redo C. Only undo D. Both undo and redo

Answer: D. Why: "Steal" means uncommitted data can reach disk (needs undo on crash); "No-force" means committed data might NOT yet be on disk (needs redo on crash) — together they require both.

Q9. A Compensation Log Record (CLR) written during the ARIES Undo pass is best described as:

A. A record that itself will need to be undone later B. A redo-only record logging an undo action, which is never undone again C. A duplicate of the original update record D. Only used during the Analysis pass

Answer: B. Why: CLR's record the undo action taken during recovery; they are redo-only, ensuring that if a second crash occurs mid-recovery, the already-completed undo isn't repeated.

6.15 Data Warehousing, Mining, Distributed DB & NoSQL

15.1 Data Warehouse — Definition & Architecture

- **W.H. Inmon's definition (classic exam recall):** a data warehouse is **Subject-Oriented, Integrated, Time-Variant, and Non-Volatile** — a collection of data supporting management decision-making.
- **Subject-oriented:** organized around subjects (customer, sales), not daily transactions.
- **Integrated:** merges multiple heterogeneous sources with consistent naming/encoding.
- **Time-variant:** stores long historical horizons (years), unlike operational DBs (current values only).
- **Non-volatile:** no in-place updates — only initial load + read access (no transaction processing/recovery/concurrency-control needed, unlike OLTP).
- **ETL = Extract** (pull from heterogeneous sources) → **Transform** (clean, convert, aggregate) → **Load** (into the warehouse).
- **Architecture tiers:** Single-tier (rare) → Two-tier (adds a staging area before the warehouse) → **Three-tier (most common):** bottom = warehouse DB, middle = OLAP/application server (abstracted analysis view), top = user-facing tools.
- **Schemas:** **Star schema** (1 central fact table + surrounding dimension tables, denormalized, simplest/fastest). **Snowflake schema** (dimension tables further normalized into sub-tables — saves space, more joins needed). **Fact constellation / Galaxy schema** (multiple fact tables sharing dimension tables).
- **TRAP:** Snowflake = *normalized* dimensions (more joins, less redundancy); Star = *denormalized* dimensions (fewer joins, faster queries, some redundancy) — commonly reversed.

15.2 Data Cube & OLAP Operations

- **Data cube:** multidimensional view of data — **Dimension tables** (e.g., time, item, location) describe context; **Fact table** holds measures (e.g., dollars_sold) + foreign keys to dimensions.

```
Roll-up    : summarize / climb UP a concept hierarchy (or reduce a dimension)  -- e.g. city -> state -> country
Drill-down: opposite of roll-up; go to finer detail (more specific)             -- e.g. year -> quarter -> month
Slice     : fix ONE dimension to a single value (cube -> 2D)
Dice      : filter on TWO OR MORE dimensions (a sub-cube)
Pivot (rotate): reorient the cube axes for a different cross-tab view
```

- **TRAP:** Roll-up and Drill-down are exact opposites (aggregate-up vs detail-down); Slice (one dimension fixed) vs Dice (multiple dimensions filtered) is the pair most often confused.

15.3 Data Mining & KDD Process

- **Data Mining** = extracting **knowledge** (task-relevant information) from large data volumes; it is ONE step within the broader **KDD (Knowledge Discovery in Databases)** process.

```
KDD steps (in order): Data Cleaning -> Data Integration -> Data Selection ->
                      Data Transformation -> DATA MINING -> Pattern Evaluation -> Knowledge Presentation
```

- **Data preprocessing tasks:** Cleaning (fix missing/noisy/inconsistent data), Integration (merge sources), Transformation (normalize/aggregate), Reduction (smaller volume, same analytical value).
- **Major data mining tasks:** Classification & Prediction (assign records to predefined classes), Clustering (group by similarity — maximize intra-cluster, minimize inter-cluster similarity — NO predefined classes, unlike classification), Association Rule Mining (find frequent co-occurring itemsets), Outlier/Anomaly Analysis (find dissimilar objects), Evolution Analysis (trends over time).
- **TRAP:** Classification needs predefined class labels (supervised); Clustering discovers groups with no predefined labels (unsupervised) — the single most common mix-up in this topic.
- **Clustering approaches:** Partitioning (k-means, k-medoids — needs k specified upfront), Hierarchical (no k needed, builds a dendrogram), Density-based, Grid-based, Model-based.

15.4 Association Rule Mining — Apriori Algorithm

```
Support(X)      = (transactions containing X) / (total transactions)
Confidence(X->Y) = Support(X union Y) / Support(X)
Apriori (downward-closure) property: every subset of a frequent itemset must ALSO be frequent.
=> if a (k-1)-itemset is NOT frequent, no k-itemset containing it can be frequent (prune it).
Algorithm: L1 = frequent 1-itemsets
for k = 1, 2, ... while Lk is non-empty:
    Ck+1 = candidates generated by joining Lk with itself, then PRUNE using downward closure
    scan DB, count support of each candidate in Ck+1
    Lk+1 = candidates in Ck+1 meeting min_support
return union of all Lk
```

- A rule $X \rightarrow Y$ is output only if **both** $\text{Support}(X \cup Y) \geq \text{min_support}$ **and** $\text{Confidence}(X \rightarrow Y) \geq \text{min_confidence}$.
- **TRAP:** Apriori's efficiency comes entirely from the **downward-closure/anti-monotone property** — pruning candidates whose subsets aren't frequent avoids checking the full power set.
- Other frequent-pattern methods (name recognition only): **FP-Growth** (avoids candidate generation, uses a compact tree structure), **ECLAT** (vertical data format).

Worked Example

Itemset {Bread, PeanutButter} has Support = 60% (appears in 60% of transactions). If $\text{Support}(\text{Bread}) = 80\%$, then $\text{Confidence}(\text{Bread} \rightarrow \text{PeanutButter}) = 60/80 = 75\%$.

15.5 Distributed Databases

- Sites are loosely coupled, share no physical hardware. **Homogeneous** (identical software/schema everywhere, cooperate fully, appears as one system to users) vs **Heterogeneous** (different schemas/software per site — harder query/transaction processing).
- **Replication:** multiple copies of data at different sites (faster reads, fault tolerance); **full replication** = every site has the entire DB.

- **Fragmentation:** splitting one relation across sites — **Horizontal** (row-wise split, each fragment has all columns, subset of rows) vs **Vertical** (column-wise split, each fragment needs the key attribute repeated to allow lossless rejoining).
- **Data transparency types:** Fragmentation transparency, Replication transparency, Location transparency — user need not know HOW/WHERE data is actually split/copied/stored.

15.6 NoSQL

- **"Not Only SQL"** — non-relational, schema-less/flexible-schema databases built for Big Data (Volume/Variety/Velocity) that rigid relational schemas handle poorly at scale.
- Trades strict **ACID** guarantees for **BASE**: **B**asically **A**vailable, **S**oft state, **E**ventual consistency.

CAP Theorem: a distributed system can guarantee at most 2 of the following 3 simultaneously:

- C - Consistency (every read gets the most recent write)
- A - Availability (every request gets a non-error response)
- P - Partition tolerance (system keeps working despite network partition/link failure)

- **TRAP:** CAP theorem is THE defining fact of distributed/NoSQL systems — since network partitions in a distributed system are essentially unavoidable (P is not optional), the real-world trade-off in practice is between **C and A** (e.g., choose CP or AP), not "pick any 2 freely."
- **NoSQL types** (4 categories):

Type	Structure	Examples
Key-Value	Hash table of key→value (JSON/BLOB/string)	Redis, DynamoDB, Riak, Memcached
Document	Key→value, but value is a structured document (JSON/XML)	MongoDB, CouchDB
Column-family	Data grouped/stored by column, not row (fast aggregates)	Cassandra, HBase
Graph	Nodes + edges storing entities and their relationships	Neo4j, OrientDB

- **TRAP:** Column-family stores are optimized for aggregate queries (SUM/COUNT/AVG) since a whole column is stored contiguously — the opposite strength profile of row-oriented RDBMS tables.

Questions

Q1. Which of the following is NOT one of the four defining properties of a data warehouse (per Inmon's definition)?

- A. Subject-oriented B. Volatile C. Integrated D. Time-variant

Answer: B. Why: A data warehouse is explicitly **Non-volatile** (no in-place updates) — "Volatile" is the opposite of a defining property.

Q2. [Trap] A Snowflake schema differs from a Star schema in that:

- A. Snowflake has no fact table B. Snowflake normalizes dimension tables into sub-tables, requiring more joins than the denormalized Star schema C. Star schema is always slower D. They are identical

Answer: B. Why: Snowflake schema trades query simplicity for reduced redundancy by further normalizing dimension hierarchies; Star schema keeps dimensions denormalized for simpler/faster queries.

Q3. Viewing sales data at the "country" level after previously viewing it at the "city" level is an example of:

- A. Drill-down B. Roll-up C. Dice D. Pivot

Answer: B. Why: Moving from a more detailed level (city) to a more summarized level (country) is Roll-up (climbing the concept hierarchy); the reverse would be Drill-down.

Q4. [Trap] The key distinguishing feature between Classification and Clustering is:

- A. Classification is unsupervised, Clustering is supervised B. Classification assigns records to predefined classes (supervised); Clustering groups by similarity with no predefined labels (unsupervised) C. They are the same technique D. Clustering requires labeled training data

Answer: B. Why: Classification learns from labeled examples to predict a known class; Clustering discovers natural groupings without any predefined class labels.

Q5. [Numerical] If $\text{Support}(\{A,B\}) = 40\%$ and $\text{Support}(\{A\}) = 50\%$, what is $\text{Confidence}(A \rightarrow B)$?

- A. 40% B. 50% C. 80% D. 90%

Answer: C. Why: $\text{Confidence}(A \rightarrow B) = \text{Support}(A \cup B) / \text{Support}(A) = 40/50 = 80\%$.

Q6. The Apriori algorithm's efficiency relies fundamentally on which property?

- A. Every superset of a frequent itemset is frequent B. Every subset of a frequent itemset must also be frequent (downward closure), letting infrequent candidates be pruned early C. Random sampling of transactions D. Sorting items alphabetically

Answer: B. Why: The anti-monotone/downward-closure property lets Apriori discard any candidate itemset that has an infrequent subset, without ever counting its actual support.

Q7. A relation is split by putting all rows for "North region" customers at Site 1 and "South region" customers at Site 2, with each fragment retaining all columns. This is:

- A. Vertical fragmentation B. Horizontal fragmentation C. Full replication D. Star schema

Answer: B. Why: Splitting by ROWS while keeping all columns in each fragment is horizontal fragmentation; vertical fragmentation splits by columns instead.

Q8. [Trap] According to the CAP theorem, in the presence of an unavoidable network partition, a distributed NoSQL system must primarily trade off between:

- A. Consistency and Partition tolerance B. Availability and Partition tolerance C. Consistency and Availability D. None of the above; all 3 are always achievable together

Answer: C. Why: Since partition tolerance is essentially mandatory for any real distributed system, the practical CAP trade-off in system design is between Consistency and Availability.

Q9. Which NoSQL database type is best suited for storing and querying entities along with their interconnections (e.g., social network friend relationships)?

- A. Key-Value store B. Document store C. Column-family store D. Graph database

Answer: D. Why: Graph databases (e.g., Neo4j) natively store nodes and edges, making relationship-heavy queries (friend-of-a-friend, shortest path) efficient — unlike key-value or column stores.

7.1 Introduction to Computer Networks

1.1 Basics, Goals & Applications

- **Network** = autonomous computers interconnected by one technology to share resources. Internet = "network of networks."
- **Goals:** Resource sharing, Communication, Reliability (alternate paths), Cost reduction (centralize hardware), Scalability, Remote access.
- **Applications:** Resource sharing, Interpersonal communication (email/VoIP/video conf), Remote info access (WWW), Distributed/Cloud computing, E-commerce, Interactive entertainment (VoD, gaming).
- **Data communication = 5 components:** Message, Sender, Receiver, Transmission medium, Protocol.
- **4 characteristics of effective data communication:** Delivery, Accuracy, Timeliness, **Jitter** (variation in packet arrival time — **TRAP:** jitter ≠ delay, it's the *variation* in delay).
- **Network criteria:** Performance, Reliability, Security.

1.2 Evolution of Networking

Standalone computers → ARPANET (1969) → TCP/IP adoption (1983) → NSFNET (1986) → Commercial Internet + WWW (1990s) → Modern Internet

- **ARPANET** (1969, DARPA/US DoD): first operational **packet-switching** network; goal = survive partial failure (nuclear attack resilience); first link UCLA→SRI.
- **1983:** ARPANET migrates from NCP to **TCP/IP** — this is considered the true birth of "the Internet."
- **NSFNET** (1986): high-speed academic backbone connecting 5 US supercomputing centers; replaced ARPANET; decommissioned **1995** once commercial ISPs took over.
- **WWW** (1991, Tim Berners-Lee): added HTTP/HTML, made Internet browsable.
- **TRAP:** ARPANET ≠ Internet. ARPANET is the *ancestor*; "Internet" formally begins with TCP/IP standardization (1983).

1.3 Transmission Modes & Connection Types

Mode	Direction	Example
Simplex	One-way only	Radio, keyboard→CPU
Half-Duplex	Both ways, not simultaneous	Walkie-talkie
Full-Duplex	Both ways, simultaneous (= 2 half-duplex)	Telephone

- **Point-to-point:** dedicated link between exactly 2 devices.
- **Multipoint (multidrop):** single link shared by more than 2 devices.

1.4 Network Topologies

Topology	Links Needed	Key Advantage	Key Disadvantage
Mesh	$n(n-1)/2$ dedicated links	No traffic issues, robust, private, easy fault ID	Expensive, hard to install/wire
Star	n links to central hub	Easy install/reconfig/fault-ID, robust to single-link failure	Hub is single point of failure
Bus	1 backbone cable	Least cabling, easy install	Break in cable halts all; hard fault isolation
Ring	n links (closed loop)	Simple install, easy fault isolation	One device failure can break the ring
Tree	Star-of-Stars (Bus+Star hybrid)	Scalable, fault isolated per branch	Backbone failure kills whole network
Hybrid	Combination of ≥ 2 topologies	Flexible, fault-tolerant across sub-nets	Complex & expensive to design

TRAP: Mesh topology needs $n(n-1)/2$ links (undirected/duplex) — a common numerical question. For $n=5 \rightarrow 10$ links; each node needs $(n-1)$ ports.

1.5 Types of Networks (by scale)

Feature	PAN	LAN	MAN	WAN
Range	~10 m	few km (building/campus)	5–50 km (city)	Unlimited (global)
Standard	Bluetooth (802.15.1)	Ethernet (802.3) / Wi-Fi (802.11)	Fiber/Microwave	Satellite, undersea cable, leased lines
Ownership	Personal	Private (single org)	Public/Private (ISP/Govt)	Mixed, no single owner
Speed	Moderate	Very high (100 Mbps–10 Gbps)	Moderate–high	Low (vs LAN) — higher latency
Example	Bluetooth headset	Office/School LAN	Cable TV network, city Wi-Fi	Internet, MNC WAN, 4G/5G

1.6 Switching Techniques

Aspect	Circuit Switching	Packet Switching
Path	Dedicated, reserved before transfer	No fixed path; dynamic per packet
Connection	Connection-oriented	Connectionless (per packet)
Bandwidth	Reserved (wasted when idle)	Shared dynamically — better utilization
Delay	Setup delay, then constant	Variable (queuing/congestion); no setup delay
Order	In-order (same path)	May arrive out-of-order — needs reassembly
Example	Telephone network	Internet

- **Message Switching:** whole message (not packets) stored-and-forwarded hop-by-hop; high delay/memory; not real-time (legacy: telegram, early email).

- Circuit switching phases: **Connection establishment** → **Data transfer** → **Connection release**.
- **TRAP:** Packet switching ⇒ possible packet loss/out-of-order delivery; circuit switching ⇒ guaranteed bandwidth but wastes it when idle.

1.7 X-Series Standards (ITU-T, for Public Data Networks)

- **X.21** — Physical layer interface for **circuit-switched** networks (DTE→DCE signalling).
- **X.25** — classic **packet switching**, connection-oriented (virtual circuit); maps to OSI L1–L3: **X.21 (Physical)** → **LAPB (Data Link)** → **PLP (Network)**. Hop-by-hop error checking ⇒ reliable but slow; superseded by Frame Relay / TCP/IP.

1.8 Transmission Media

Guided (wired): Twisted pair (telephone), Coaxial (cable TV), Fiber optic (light, total internal reflection, up to ~1600 Gbps, immune to EM noise, unidirectional light).

Unguided (wireless):

Feature	Radio Waves	Microwaves	Infrared
Frequency	3 kHz–1 GHz	1 GHz–300 GHz	300 GHz–400 THz
Directionality	Omnidirectional	Unidirectional (Line-of-Sight)	Line-of-Sight
Penetrates walls?	Yes	No	No (→ secure, indoor use)
Use	AM/FM radio	Satellite, point-to-point towers, cellular backhaul	TV remote, wireless mouse

TRAP: Microwave requires precise antenna alignment (LOS) and is blocked by obstacles — often confused with radio waves, which penetrate buildings.

1.9 OSI Model — 7 Layers

Layer	Core Job	Adds
7. Application	User-facing network services (mail, FTM, directory)	—
6. Presentation	Translation, Encryption , Compression	—
5. Session	Dialog control, synchronization checkpoints	—
4. Transport	Process-to-process, segmentation, flow/error control	Port (Service-point) address
3. Network	End-to-end packet delivery across networks, routing	Logical (IP) address
2. Data Link	Node-to-node reliable transfer, framing, MAC access control, flow/error control	Physical (MAC) address
1. Physical	Raw bit transmission, signaling, topology, transmission mode	—

- Mnemonic: **All People Seem To Need Data Processing** (L7→L1).
- **TRAP:** Encryption/compression is a **Presentation layer** job, not Application layer. Error control appears at BOTH Data Link (node-to-node) and Transport (end-to-end) — a classic "which layer does X" trap.
- OSI is a **reference model**, not a protocol itself (contrast with TCP/IP, which is an actual protocol suite).

Questions

Q1. How many dedicated links are required in a mesh topology with 8 nodes?

A. 8 B. 16 C. 28 D. 56

Answer: C. Why: $n(n-1)/2 = 8 \times 7/2 = 28$.

Q2. Which topology's total failure risk is concentrated entirely in one central device?

A. Bus B. Ring C. Star D. Mesh

Answer: C. Why: Star depends entirely on the central hub; hub failure disables the whole network.

Q3. Packet switching is preferred over circuit switching mainly because it offers:

A. Fixed delay B. Guaranteed in-order delivery C. Better bandwidth utilization via dynamic sharing D. No processing overhead per packet

Answer: C. Why: Packet switching shares bandwidth dynamically among users instead of reserving it, unlike circuit switching. It trades this for possible variable delay/out-of-order delivery.

Q4. X.25 maps its Data Link layer function to which protocol?

A. PLP B. LAPB C. X.21 D. TCP

Answer: B. Why: X.25 layering: Physical=X.21, Data Link=LAPB, Network=PLP.

Q5. Which transmission medium is most vulnerable to being blocked by a building between sender and receiver?

A. Radio waves B. Microwaves C. Infrared and Microwaves (both, being line-of-sight) D. None, all penetrate walls

Answer: C. Why: Both microwaves and infrared require line-of-sight and cannot penetrate solid obstacles; radio waves can.

Q6. Encryption of data for secure transmission is a standard responsibility of which OSI layer?

A. Application B. Presentation C. Session D. Transport

Answer: B. Why: Presentation layer handles translation, encryption/decryption, and compression — a frequently tested trap since students assume "Application layer" handles security.

Q7. ARPANET's shift that is considered the true technical birth of "the Internet" was:

A. First message sent between UCLA and SRI (1969) B. Adoption of TCP/IP in 1983 C. Launch of NSFNET in 1986 D. Introduction of WWW in 1991

Answer: B. Why: TCP/IP standardization in 1983 enabled internetworking of heterogeneous networks — the defining feature of "the Internet" as a network of networks.

Q8. A network spanning an entire city, owned by an ISP/government consortium and typically using fiber/microwave links, is classified as:

A. LAN B. MAN C. WAN D. PAN

Answer: B. Why: MAN covers ~5–50 km (city-scale) and is typically owned by a consortium/ISP, unlike a privately-owned LAN.

Q9. [Trap] A network engineer says "packet switching guarantees in-order packet delivery just like circuit switching." This is:

A. True, both guarantee order B. False — packets may take different paths and arrive out of order, requiring reassembly C. True only for the Internet D. False, because circuit switching also allows out-of-order delivery

Answer: B. Why: Only circuit switching guarantees order (fixed path); packet switching may deliver packets out of order since each packet is routed independently.

Q10. Which layer of the OSI model is responsible for adding logical (IP) addressing and routing decisions?

A. Data Link B. Network C. Transport D. Physical

Answer: B. Why: The Network layer adds logical addressing and performs routing for end-to-end delivery across multiple networks; Data Link only adds physical (MAC) addressing for node-to-node hops.

7.2 Data Communication Fundamentals

2.1 Analog vs Digital Signals

Feature	Analog	Digital
Nature	Continuous, infinite values	Discrete, fixed levels
Waveform	Sine wave	Square wave
Noise immunity	Low — degrades with distance	High — regenerable
Example	Voice, thermometer	Computers, CD/DVD, USB

2.2 Core Formulas (High-Yield)

Period (T) = $1 / \text{Frequency (f)}$ [T in seconds, f in Hz — inversely related]
 Bit Rate = Baud Rate $\times$ n [n = bits encoded per signal symbol]
 Throughput = actual bits successfully delivered / time taken ($\leq$ Bandwidth, always)
 Transmission Delay = Packet Size (bits) / Bandwidth (bps)
 Propagation Delay = Distance / Propagation Speed
 Total Latency = Transmission Delay + Propagation Delay (+ Queuing + Processing delay)

- Bandwidth** = theoretical max capacity of the link (Hz analog / bps digital). **Throughput** = actual delivered rate — always $\leq$ bandwidth (reduced by noise/congestion/errors).
- TRAP:** Bandwidth and Throughput are NOT the same — a question giving "link bandwidth 10 Mbps but average delivered X Mbps" is testing this gap.

Worked Example

Link bandwidth = 10 Mbps. Transmits 12,000 frames/min, 10,000 bits/frame.
 Frames/sec = $12000/60 = 200$. Throughput = $200 \times 10,000 = 2,000,000$ bps = **2 Mbps**.
 Efficiency = Throughput/Bandwidth = $2/10 = 20\%$.

2.3 Encoding Schemes — Bits per Symbol (n)

Scheme	n (bits/symbol)	Note
Manchester	0.5	Least efficient; 2 transitions/bit but self-synchronizing; legacy 10 Mbps Ethernet, still used in RFID/NFC
NRZ	1	1 pulse = 1 bit; simple but loses sync on long 0/1 runs; used in USB/UART, not long-distance
16-QAM	4	Combines amplitude + phase; old WiFi 4
4096-QAM	12	Max efficiency; needs very clean signal (short range); WiFi 7 / 5G

- TRAP:** Higher n (bits/symbol) = more efficient BUT more vulnerable to noise (needs cleaner channel).

2.4 Channel Capacity Theorems

Nyquist (NOISELESS channel): BitRate = $2 \times B \times \log_2(L)$ [B=Bandwidth(Hz), L=signal levels]
 Shannon (NOISY channel): $C = B \times \log_2(1 + \text{SNR})$ [SNR = Signal Power / Noise Power]

Feature	Nyquist	Shannon
Channel	Noiseless (ideal)	Noisy (real-world)
Depends on	Bandwidth, signal levels (L)	Bandwidth, SNR
Use	Theoretical upper bound, limited practical use	Universal real-world limit

- TRAP:** Nyquist alone is used to find max bit rate ONLY when a channel is stated as noiseless; if noise/SNR is given, use Shannon. In practice, the **actual channel capacity is the minimum of the two** if both are computed for the same channel.
- Increasing signal levels (L) increases Nyquist bit rate, but real noisy channels can't have unlimited L (error probability rises) — this is why Shannon's limit governs real channels.

2.5 Encoding / Modulation Types

- QAM** = ASK + PSK combined (varies amplitude AND phase together) → higher data rate per symbol.
- Digital-to-Digital:** Line coding (NRZ, Manchester) → Block coding (mB/nB, adds redundancy bits for sync/error-detection, $n > m$) → Scrambling (avoids long 0-runs without adding bits, keeps clock sync).
- Analog-to-Digital (PCM):** Sampling → Quantization → Encoding (binary).
- Base64:** 3 bytes (24 bits) → four 6-bit chunks → 4 printable characters (used to send binary data over text protocols like email/JSON).
- Fixed-length coding:** ASCII/EBDIC/Unicode (every symbol = same # bits). **Variable-length coding:** Huffman (compression; frequent symbols get fewer bits).

Questions

Q1. A signal has a frequency of 500 Hz. What is its period?

A. 500 s B. 2 ms C. 0.5 ms D. 5 ms

Answer: B. Why: $T = 1/f = 1/500 = 0.002$ s = 2 ms.

Q2. A modulation scheme encodes 4 bits per signal symbol using a baud rate of 2400 baud. Find the bit rate.

A. 600 bps B. 2400 bps C. 4800 bps D. 9600 bps

Answer: D. Why: Bit Rate = Baud Rate $\times$ n = 2400 $\times$ 4 = 9600 bps.

Q3. [Trap] Bandwidth of a link is 100 Mbps but actual measured data delivery is 40 Mbps. This 40 Mbps figure is the link's:

A. Bandwidth B. Baud rate C. Throughput D. Channel capacity

Answer: C. Why: Throughput is the actually achieved delivery rate; it is always $\leq$ bandwidth due to noise, congestion, or overhead.

Q4. A noiseless channel with bandwidth 3000 Hz uses 2 signal levels. Maximum bit rate (Nyquist) is:

A. 3000 bps B. 6000 bps C. 1500 bps D. 9000 bps

Answer: B. Why: BitRate = $2 \times B \times \log_2(L) = 2 \times 3000 \times \log_2(2) = 6000 \times 1 = 6000$ bps.

Q5. A channel has bandwidth 3 kHz and SNR = 7. Its Shannon capacity is approximately:

A. 3000 bps B. 6000 bps C. 9000 bps D. 12000 bps

Answer: C. Why: $C = B \times \log_2(1 + \text{SNR}) = 3000 \times \log_2(8) = 3000 \times 3 = 9000$ bps.

Q6. Which statement correctly distinguishes Nyquist from Shannon's theorem?

A. Nyquist applies to noisy channels, Shannon to noiseless B. Nyquist depends on SNR, Shannon on signal levels C. Nyquist gives the limit for noiseless channels using signal levels; Shannon gives the limit for noisy channels using SNR D. They give identical formulas

Answer: C. Why: Nyquist: BitRate = $2B \cdot \log_2(L)$ for a noiseless channel; Shannon: $C = B \cdot \log_2(1 + \text{SNR})$ accounts for real-world noise.

Q7. Manchester encoding is described as "least efficient" (n = 0.5) mainly because:

A. It cannot detect errors B. It requires 2 signal transitions to encode 1 bit, trading bandwidth for guaranteed synchronization C. It uses more signal levels than QAM D. It is only used for analog signals

Answer: B. Why: Manchester sacrifices bandwidth efficiency (2 transitions/bit) specifically to guarantee the receiver stays synchronized (signal never "flat-lines").

Q8. Base64 encoding converts every 3 bytes of binary data into how many output characters?

A. 2 B. 3 C. 4 D. 6

Answer: C. Why: 24 bits $\rightarrow$ four 6-bit groups $\rightarrow$ 4 printable characters (the "24-to-32-bit" rule).

7.3 Data Link Layer — Multiple Access Control

3.1 DLL Sublayers & Access Categories

- DLL splits into **LLC** (top: flow/error control, part of framing) and **MAC** (bottom: access method, e.g. CSMA/CD for Ethernet). Framing is shared by both.
- Random Access (contention)**: no station has priority; stations compete → collisions possible. (ALOHA, CSMA, CSMA/CD, CSMA/CA)
- Controlled Access**: stations coordinate; a station transmits only when authorized. (Reservation, Polling, Token Passing)
- Channelization**: bandwidth split among stations by frequency/time/code. (FDMA, TDMA, CDMA)

3.2 ALOHA

Propagation delay T_p = Distance / Propagation speed
 Transmission delay T_t (= T_{fr}) = Frame length / Bandwidth
 Pure ALOHA vulnerable time = $2 \times T_{fr}$
 Slotted ALOHA vulnerable time = $1 \times T_{fr}$ (half of pure ALOHA)
 Pure ALOHA max throughput ~ 18.4% (at $G = 0.5$)
 Slotted ALOHA max throughput ~ 36.8% (at $G = 1$)

- Pure ALOHA**: station sends whenever it has data; collision if any other station transmits within $\pm T_{fr}$ of it → vulnerable window = $2 \times T_{fr}$.
- Slotted ALOHA**: time divided into slots of size T_{fr} ; stations may only start at slot boundaries → collision window shrinks to just T_{fr} (halves vulnerability, doubles max throughput vs pure ALOHA).
- On collision: station waits **random back-off time T_B** , retransmits; gives up after **K_{max}** attempts.
- TRAP**: Slotted ALOHA's efficiency is exactly **double** pure ALOHA's — a favorite numerical/conceptual GATE trap.

3.3 CSMA and Persistence Strategies

- CSMA** = "sense before transmit." Reduces but does NOT eliminate collisions (propagation delay means a station may sense idle before another's signal arrives).
- Vulnerable time (CSMA) = T_p** (propagation time) — much smaller than ALOHA's vulnerable window since carrier sensing is used.

Method	Behavior when line found IDLE	Behavior when BUSY	Collision Risk
1-persistent	Sends immediately (prob=1)	Waits, keeps sensing	Highest
Non-persistent	Sends immediately	Waits random time, senses again	Lowest, but wastes idle time (low efficiency)
p-persistent (slotted)	Sends with probability p; else waits 1 slot & re-checks	Backs off as if collided	Balanced (best of both)

3.4 CSMA/CD (Wired Ethernet)

Minimum frame transmission time: $T_{fr} \geq 2 \times T_p$

- A station must still be transmitting when a collision signal could return, so it can detect the collision and abort. Worst case: two max-distance-apart stations — signal takes T_p to reach the far end, and the collision effect takes another T_p to come back ⇒ **$T_{fr} \geq 2T_p$** .
- Channel energy levels: **Zero** (idle) / **Normal** (one station transmitting) / **Abnormal** (collision, $\sim 2 \times$ normal energy).
- TRAP**: This $2T_p$ rule is why Ethernet enforces a **minimum frame size** (64 bytes) — a frame shorter than this on a given cable length+speed cannot guarantee collision detection before it finishes sending.

3.5 CSMA/CA (Wireless — Wi-Fi)

- Collisions can't be *detected* in wireless (received signal energy too weak to distinguish from a collision) → must be *avoided*.
- Three mechanisms:
 - IFS (Interframe Space)**: wait even if channel is idle, to let far signals arrive.
 - Contention Window**: random back-off slots; doubles after each collision (**binary exponential back-off**); if channel goes busy mid-wait, timer **pauses** (not restarts) — gives priority to the longest-waiting station.
 - ACK**: explicit acknowledgment required since collisions can't be sensed directly.
- TRAP**: CSMA/CD = collision **detection** (wired); CSMA/CA = collision **avoidance** (wireless) — do not swap these.

3.6 Controlled Access

Method	Mechanism
Reservation	Time divided into Reservation Interval (M slots, 1 per station, mark intent to send) + Data period (only reserved stations send, in order).
Polling	Primary/secondary hierarchy. Select : primary → secondary ("I have data for you," waits for ready-ACK). Poll : primary asks each secondary in turn ("anything to send?" → NAK if nothing, data frame if something).
Token Passing	Logical ring; only the station holding the token may transmit; token passed to successor after use; token-holding time is capped to prevent monopolization; lost token must be regenerated.

3.7 Channelization

Scheme	Divides bandwidth by	Key idea
FDMA	Frequency	Each station gets a permanent frequency band (all the time).
TDMA	Time	Each station gets the full bandwidth but only during its time slot.
CDMA	Code (not time/frequency)	All stations transmit simultaneously over the full bandwidth, separated by unique orthogonal codes (like different "languages" in one room).

- **TRAP:** CDMA is the odd one out — no time-sharing (unlike TDMA) and no frequency partition (unlike FDMA); separation is purely by code.

Questions

Q1. In Pure ALOHA, if the frame transmission time is T_{fr} , the vulnerable time during which a collision can occur is:

- A. T_{fr} B. $2 \times T_{fr}$ C. $0.5 \times T_{fr}$ D. $4 \times T_{fr}$

Answer: B. Why: A frame starting up to T_{fr} before or after another's frame will overlap it, giving a vulnerable window of $2 \times T_{fr}$.

Q2. [Numerical] Maximum throughput of Slotted ALOHA compared to Pure ALOHA is:

- A. Same B. Half C. Double ($\sim 36.8\%$ vs $\sim 18.4\%$) D. Four times

Answer: C. Why: Slotted ALOHA's max throughput ($1/e \approx 36.8\%$) is exactly double Pure ALOHA's ($1/2e \approx 18.4\%$) because slotting halves the vulnerable window.

Q3. [Trap] A student claims "CSMA guarantees no collisions since stations sense the medium before transmitting." This is:

- A. True B. False — propagation delay still allows two stations to sense idle simultaneously and collide C. True only for wireless D. True only for slotted CSMA

Answer: B. Why: CSMA reduces collision probability but cannot eliminate it, because a station's signal takes finite time (T_p) to reach other stations, who may sense "idle" in that window.

Q4. Why does CSMA/CD require a minimum frame transmission time of $2 \times T_p$?

- A. To reduce bandwidth usage B. So the sender is still transmitting when a collision signal could return from the farthest station C. To allow encryption of the frame D. To synchronize clocks

Answer: B. Why: Worst case, a collision at the far end takes T_p to occur and another T_p for its effect to propagate back — the sender must still be transmitting ($T_{fr} \geq 2T_p$) to detect it and abort.

Q5. Which multiple access method is specifically designed because collisions cannot be reliably *detected* (only avoided)?

- A. CSMA/CD B. CSMA/CA C. Pure ALOHA D. TDMA

Answer: B. Why: In wireless networks, received signal energy is too weak for collision detection, so CSMA/CA avoids collisions via IFS, contention window, and ACKs instead.

Q6. In IEEE 802 controlled access via Polling, a secondary station with no data to send responds to a poll with:

- A. SEL frame B. ACK frame C. NAK frame D. Token frame

Answer: C. Why: The primary "polls" each secondary; a secondary with nothing to send replies NAK, prompting the primary to poll the next one.

Q7. Which channelization method allows all stations to transmit simultaneously over the entire bandwidth, distinguished only by unique codes?

- A. FDMA B. TDMA C. CDMA D. Polling

Answer: C. Why: CDMA uses orthogonal codes for simultaneous full-bandwidth transmission — unlike FDMA (frequency split) or TDMA (time split).

Q8. [Trap] In CSMA/CA's contention window, if a station's back-off timer is running and the channel becomes busy, the station:

- A. Restarts the timer from the maximum value B. Pauses the timer and resumes when the channel is idle again C. Immediately transmits anyway D. Discards the frame

Answer: B. Why: The timer pauses (doesn't reset) on a busy channel — this preserves relative priority for the station that has already waited longest.

7.4 Data Link Layer — Flow Control (ARQ Protocols)

(Highest-yield numerical topic in CN — sliding window efficiency appears almost every year.)

4.1 Flow Control vs Error Control

- **Flow control:** prevents overwhelming a slow receiver (buffer-based feedback to sender).
- **Error control (ARQ = Automatic Repeat Request):** detects lost/corrupted/duplicate frames and triggers retransmission.

4.2 Core Delay & Efficiency Formulas

T_t (transmission delay) = Frame size / Bandwidth
 T_p (propagation delay) = Distance / Propagation speed
 RTT (Round Trip Time) = $2 \times T_p$ (ACK size/time treated as negligible)
 $\text{Total cycle time (Stop-and-Wait)} = T_t + 2 \times T_p$
 $a = T_p / T_t$

 Efficiency (Stop-and-Wait): $\eta = 1 / (1 + 2a)$
 Throughput = $\eta \times \text{Bandwidth}$

 Optimum window size to reach 100% efficiency: $W = 1 + 2a$

- If $W \geq (1+2a)$ the link is fully utilized ($\eta=1$); if $W < (1+2a)$, $\eta = W / (1+2a)$.

4.3 Stop-and-Wait ARQ

- Sender sends **1 frame**, waits for ACK before sending next → very simple, but wastes the "pipe" on high bandwidth-delay-product links.
- Adds: frame **sequence numbers** (just 0/1 — 1 bit is enough since only 1 frame is ever outstanding) to detect duplicates/lost frames; **timer** for retransmission on ACK timeout; corrupted frames are **silently discarded** (receiver "silence" is itself the error signal).
- **Bandwidth-delay product** = Bandwidth $\times$ RTT = amount of data (bits) that can be "in flight." Stop-and-Wait cannot fill this pipe on long-fat networks (large a) — hence poor efficiency for high a .

4.4 Go-Back-N ARQ (GBN)

- Sender may have **multiple frames outstanding** (sliding window) without waiting for each ACK individually → fixes Stop-and-Wait's efficiency problem.
- **Receiver window size = 1** — only accepts frames in exact order; an out-of-order/damaged frame is discarded (receiver stays silent), and ALL subsequent frames already sent get discarded by receiver too.
- **Cumulative ACK:** one ACK confirms all frames up to that number.
- On timeout for the oldest outstanding frame, sender resends **all outstanding frames from that point on** (hence "Go-Back-N" — wasteful on noisy links).
- Single timer suffices (only for the oldest unacked frame — it always expires first).

4.5 Selective Repeat ARQ (SR)

- Fixes GBN's waste: **only the specific damaged/lost frame is retransmitted**, not everything after it.
- Receiver **buffers out-of-order frames** and delivers to network layer only once a contiguous run is complete.
- Needs **individual timer per frame** and can send **NAK** for a specific damaged frame.
- Higher receiver buffer/complexity cost than GBN, but far more bandwidth-efficient on noisy/lossy links.

4.6 Sequence-Number & Window-Size Rules (m = bits used for seq. numbers, so 2^m distinct numbers)

Protocol	Sender window max	Receiver window
Stop-and-Wait	1	1
Go-Back-N	$2^m - 1$	1
Selective Repeat	2^{m-1}	2^{m-1}

- **TRAP:** Selective Repeat's window is capped at **half** of Go-Back-N's (2^{m-1} vs $2^m - 1$). Reason: if SR's window were larger, the receiver could mistake a retransmitted *old* frame for a *new* one (ambiguity in distinguishing old vs new sequence numbers when both windows can hold out-of-order frames).
- Efficiency for windowed protocols (GBN/SR) with window size N : $\eta = N/(1+2a)$ if $N < (1+2a)$, else $\eta = 1$.

4.7 Comparison

Feature	Stop-and-Wait	Go-Back-N	Selective Repeat
Frames in flight	1	Multiple (window)	Multiple (window)
Receiver window	1	1	> 1 (up to 2^{m-1})
On error	Resend that 1 frame	Resend ALL from error onward	Resend ONLY the erroneous frame
Efficiency (large a)	Poor	Good	Best (esp. on noisy links)
Complexity/Buffer	Lowest	Low	Highest (per-frame timers + buffer)

Questions

Q1. [Numerical] $T_t = 1$ ms, $T_p = 4$ ms. What is the efficiency of Stop-and-Wait ARQ?
 A. 20% B. 11.1% C. 25% D. 50%

Answer: B. Why: $a = T_p/T_t = 4$; $\eta = 1/(1+2a) = 1/(1+8) = 1/9 \approx 11.1\%$.

Q2. [Numerical] For the link in Q1, what window size would achieve 100% utilization under Go-Back-N?

A. 4 B. 8 C. 9 D. 5

Answer: C. Why: Optimum $W = 1 + 2a = 1 + 8 = 9$.

Q3. A 3-bit sequence number field ($m=3$) is used. What is the maximum sender window size under Go-Back-N vs Selective Repeat respectively?

A. 8 and 4 B. 7 and 4 C. 8 and 8 D. 7 and 8

Answer: B. Why: GBN max = $2^m - 1 = 7$; Selective Repeat max = $2^{m-1} = 4$.

Q4. [Trap] Why can't Selective Repeat use the same window size as Go-Back-N ($2^m - 1$)?

A. It would need more timers than allowed B. The receiver could then misinterpret a retransmitted old frame as a new one due to sequence number ambiguity C. It would violate flow control rules D. Selective Repeat doesn't use sequence numbers at all

Answer: B. Why: With overlapping large windows on both ends, an old retransmitted frame's sequence number could fall within the receiver's new expected range, causing ambiguity — hence the window is capped at half.

Q5. In Go-Back-N, if frames 4, 5, 6, 7 are sent and frame 4's timer expires (others undelivered/unacknowledged), the sender resends:

A. Only frame 4 B. Frames 4, 5, 6, 7 C. Only frame 7 D. Nothing, it waits for a NAK

Answer: B. Why: Go-Back-N resends the timed-out frame AND all subsequent outstanding frames — the defining (and wasteful) behavior that Selective Repeat improves upon.

Q6. Which ARQ protocol requires the receiver to maintain a reordering buffer for out-of-order frames?

A. Stop-and-Wait B. Go-Back-N C. Selective Repeat D. All of the above

Answer: C. Why: Only Selective Repeat has a receiver window > 1 , so it must buffer out-of-order frames until a contiguous sequence can be delivered upward.

Q7. [Conceptual] Why does Stop-and-Wait ARQ need only 1 bit for sequence numbers?

A. Because errors never occur B. Because at most one frame is ever outstanding — only "current" and "next" need distinguishing (0/1 suffices) C. Because ACKs are numbered instead D. Because the window size is $2^m - 1$

Answer: B. Why: Since the sender waits for each ACK before sending the next frame, only two states (current frame, next frame) ever need distinguishing — hence sequence numbers cycle 0,1,0,1...

7.5 Data Link Layer — Error Control (Hamming, CRC, Checksum)

5.1 Error Types & Redundancy

- **Single-bit error**: exactly 1 bit flipped. **Burst error**: ≥ 2 bits flipped; length = span from first to last corrupted bit (bits in between need not all be wrong).
- **Error Detection** (know *that* an error occurred) is simpler than **Error Correction** (know *where*, to fix it).
- **FEC (Forward Error Correction)**: receiver reconstructs data itself using redundant bits (no retransmission needed) — used when retransmission is costly/impossible (e.g., live streaming).
- **Retransmission (ARQ)**: receiver asks sender to resend — see Ch.4.

5.2 Block Coding & Hamming Distance

- Dataword k bits $\rightarrow$ Codeword n bits ($n=k+r$, r =redundant bits). Of 2^n possible codewords, only 2^k are valid; $2^n - 2^k$ are **invalid/unused** — an error is detected if the received word lands on an invalid codeword.
- **Hamming distance $d(x,y)$** = number of differing bit positions = $\text{popcount}(x \oplus y)$.
- **$d_{\min}$** = minimum Hamming distance over all pairs of valid codewords.

5.3 The Governing Rules (Very High-Yield)

To DETECT up to s errors: $d_{\min} \geq s + 1$
 To CORRECT up to t errors: $d_{\min} \geq 2t + 1$
 To DETECT s errors AND CORRECT t errors ($s \geq t$): $d_{\min} \geq s + t + 1$

- **TRAP**: " $d_{\min} = 3$ " is commonly misread. $d_{\min}=3$ allows you to EITHER detect up to 2 errors OR correct up to 1 error — not both simultaneously at those magnitudes.
- Coding scheme notation: **$C(n, k)$** with a stated $d_{\min}$, e.g. $C(7,4)$, $d_{\min}=3$.

5.4 Parity Check Codes

Code	Redundant bits	Detects	$d_{\min}$
Simple parity	1 bit ($n=k+1$)	Odd number of bit errors only	2
2D (row+column) parity	1 row per row + 1 column per col	Up to 3 errors anywhere in the table; 4-bit errors may go undetected	—

- **TRAP**: Simple parity CANNOT detect an even number of flipped bits (they cancel out and parity looks fine).

5.5 Hamming Code (single-bit error correction)

r redundant bits $\Rightarrow n = 2^r - 1$, $k = n - r = 2^r - r - 1$
 Parity bits placed at positions that are powers of 2: 1, 2, 4, 8, ...

- Classic exam case: **$r=3 \rightarrow C(7,4)$, $d_{\min}=3$** $\rightarrow$ corrects any single-bit error.
- Parity bit **P1** checks positions with bit-0 set (1,3,5,7); **P2** checks positions with bit-1 set (2,3,6,7); **P4** checks positions with bit-2 set (4,5,6,7). At the receiver, the syndrome (P1P2P4 read as a binary number) directly gives the **position** of the error bit — this "self-locating" property is the key exam insight.
- **TRAP**: Hamming code positions are 1-indexed and use POWERS OF 2 for the parity/check bits, not the first r positions.

5.6 Checksum

- Sender adds up all data words $\rightarrow$ sends the **sum** (or its complement) along with the data.
- Receiver re-adds everything (including the checksum); if using ordinary sum, compare to sent value; if using **1's-complement sum**, the total should be all 1s (or, equivalently, adding the complement makes the total = 0).
- Uses **1's complement arithmetic** (end-around carry) — a wraparound-add scheme, not 2's complement.
- **TRAP**: Checksum is a *weak* error-detection method (used at Transport/IP layers) — it can miss certain compensating errors that CRC would catch. CRC is stronger and used at the Data Link layer/hardware level.

5.7 Cyclic Redundancy Check (CRC)

- **Cyclic code property**: any cyclic shift of a valid codeword is also a valid codeword.
- Procedure: append $(n-k)$ zeros to the k -bit dataword $\rightarrow$ divide by a fixed generator (degree $n-k$, so $n-k+1$ bits) using **modulo-2 (XOR) division** $\rightarrow$ the remainder ($n-k$ bits) replaces the appended zeros to form the final n -bit codeword.
- Receiver divides the received codeword by the same generator; **remainder = 0 $\rightarrow$ no error detected**; nonzero remainder ("syndrome") $\rightarrow$ error.
- Polynomial view: bit string $\leftrightarrow$ polynomial (bit position = power of x); a 7-bit string has degree ≤ 6 . Left-shift by n bits $\equiv$ multiply by x^n .

Worked CRC Example (classic)

Dataword = **1001** ($k=4$), Generator = **1011** ($= x^3+x+1$, so $n-k=3$).
 Augmented dataword = 1001 000 (append 3 zeros) $= x^6+x^3$.
 Divide (mod-2) by $x^3+x+1 \rightarrow$ **remainder = 110**.
 Final codeword transmitted = **1001 110**.
 (Receiver divides 1001110 by 1011; remainder 000 $\Rightarrow$ accepted as error-free.)

Questions

Q1. [Numerical] Find the Hamming distance between 10110 and 11100.
 A. 1 B. 2 C. 3 D. 4

Answer: B. **Why:** XOR: $10110 \oplus 11100 = 01010 \rightarrow$ two 1s $\rightarrow d=2$.

Q2. A coding scheme has $d_{\min} = 5$. What is the maximum number of errors it can guarantee to CORRECT?

A. 5 B. 4 C. 2 D. 3

Answer: C. Why: $d_{\min} \geq 2t+1 \Rightarrow 5 \geq 2t+1 \Rightarrow t \leq 2 \rightarrow$ corrects up to 2 errors.

Q3. [Trap] The same code from Q2 ($d_{\min}=5$) — what is the max number of errors it can guarantee to DETECT (not correct)?

A. 2 B. 3 C. 4 D. 5

Answer: C. Why: $d_{\min} \geq s+1 \Rightarrow 5 \geq s+1 \Rightarrow s \leq 4 \rightarrow$ detects up to 4 errors (detection capacity is always $\geq$ correction capacity for the same $d_{\min}$).

Q4. A Hamming code is designed with $r=4$ redundant bits. What are n and k ?

A. $n=15, k=11$ B. $n=16, k=12$ C. $n=15, k=10$ D. $n=14, k=10$

Answer: A. Why: $n = 2^r - 1 = 15$; $k = n - r = 15 - 4 = 11 \rightarrow C(15,11)$.

Q5. In a 7-bit Hamming code ($C(7,4)$), which bit positions hold the parity bits?

A. 1, 2, 3 B. 1, 2, 4 C. 4, 5, 6 D. 1, 4, 7

Answer: B. Why: Parity bits occupy positions that are powers of 2: 1, 2, 4 (P_1, P_2, P_4); data bits occupy the rest (3,5,6,7).

Q6. [Trap] Simple parity check fails to detect an error when:

A. Exactly 1 bit flips B. Exactly 3 bits flip C. Exactly 2 bits flip (or any even number) D. No bits flip

Answer: C. Why: Simple parity only preserves odd/even 1-count; an even number of flips restores the original parity, hiding the error — a textbook trap on "always detects errors" claims.

Q7. [Numerical] Dataword 1101 is sent with CRC generator 1011 ($n-k=3$). What must the remainder be for the receiver to accept the frame as error-free?

A. Equal to the generator B. Equal to the original dataword C. All zeros (000) D. Equal to 1011

Answer: C. Why: The receiver divides the received codeword by the same generator; a zero remainder (syndrome) means no detected error.

Q8. Which error-detection technique is generally considered stronger and hardware-implemented at the Data Link layer, compared to the simpler checksum used at Transport/IP layers?

A. Parity bit B. Checksum C. CRC D. Hamming code

Answer: C. Why: CRC (via polynomial division) catches a much wider class of error patterns than the additive checksum, which is why CRC is standard in Ethernet frames while checksum suffices for IP/TCP/UDP headers.

7.6 Data Link Layer — Framing & Ethernet

6.1 Framing Types

- **Fixed-size framing:** frame length itself is the delimiter (no explicit boundary markers needed).
- **Variable-size framing:** needs explicit boundary markers —
- **Character-oriented:** uses a flag byte + **byte stuffing** (insert ESC before any data byte matching the flag/ESC pattern).
- **Bit-oriented:** uses flag bit pattern **01111110** + **bit stuffing** (insert a 0 after every 5 consecutive 1s in the data, to prevent the flag pattern appearing inside data; receiver removes the stuffed 0).
- **TRAP:** Byte stuffing inserts an extra **byte** (ESC); bit stuffing inserts a single extra **bit** (0) — don't conflate the two mechanisms.

6.2 Ethernet (IEEE 802.3)

- Standardized **1983**; originally **10BASE5** (coaxial, bus topology, **Manchester encoding**); connectionless & **unreliable** (no built-in ACK — corrupted frames are silently dropped; retransmission is left to higher layers, e.g., TCP).
- Uses **48-bit MAC addresses**.

Ethernet Frame Format

| Preamble (7B) | SFD (1B) | Dest MAC (6B) | Src MAC (6B) | Length (2B) | Data (46-1500B) | CRC (4B) |

- **Preamble (7B):** alternating 0/1 pattern for bit synchronization — added at Physical layer.
- **SFD (1B) = 10101011:** trailing "11" signals start of the actual frame.
- **Data field range: 46–1500 bytes.** If actual data < 46B → padded with 0s. If > 1500B → must be fragmented across multiple frames.
- **CRC (4B):** computed over the frame, placed last since it must be calculated after the rest of the frame is assembled.
- **Minimum frame size = 64 bytes** ($6+6+2+46+4 = 64$, excluding preamble+SFD) — this exists specifically to satisfy the **CSMA/CD rule $T_{fr} \geq 2T_p$** (Ch.3 §3.4), ensuring collision detection works before the frame finishes transmitting.
- Maximum length cap exists for 2 historical reasons: (1) memory was expensive (buffer size), (2) prevents one station from monopolizing the shared medium.

MAC Address Types (via LSB of the first byte)

Type	Rule	Meaning
Unicast	LSB of first byte = 0	One specific destination (source address is ALWAYS unicast)
Multicast	LSB of first byte = 1	A group of stations
Broadcast	All 48 bits = 1 (FF:FF:FF:FF:FF:FF)	Every station on the LAN

6.3 Manchester Encoding

- Each bit period is split in half: signal is High-then-Low (or vice versa) — the **mid-bit transition itself provides clock synchronization** (self-clocking).
- Used in classic 10 Mbps Ethernet. (See Ch.2 §2.3 for its $n=0.5$ bits/symbol efficiency cost.)

6.4 PPP (Point-to-Point Protocol) — quick reference

- Establishes a direct link between two nodes over serial/telephone lines; encapsulates Network-layer packets; supports authentication (**PAP, CHAP**); used in DSL/dial-up; replaced the older **SLIP**.

Questions

Q1. Why is bit stuffing needed in bit-oriented framing?

A. To compress the data B. To prevent the flag pattern 01111110 from appearing accidentally inside the data stream C. To add error correction D. To encrypt the frame

Answer: B. Why: A 0 is inserted after every 5 consecutive 1s in the data so the receiver never mistakes real data for the frame-delimiting flag.

Q2. [Numerical] What is the minimum valid Ethernet frame size (excluding preamble & SFD)?

A. 46 bytes B. 60 bytes C. 64 bytes D. 1500 bytes

Answer: C. Why: $6(\text{Dest})+6(\text{Src})+2(\text{Length})+46(\text{min data})+4(\text{CRC}) = 64$ bytes — required to satisfy the CSMA/CD $2T_p$ collision-detection window.

Q3. A MAC address has its first byte's LSB = 1. This address is:

A. Unicast B. Multicast C. Broadcast D. Invalid

Answer: B. Why: LSB=1 in the first byte marks a multicast (group) address; LSB=0 marks unicast; all-1s (FF...FF) is broadcast.

Q4. [Trap] Since Ethernet has no built-in acknowledgment mechanism, reliability (retransmission of lost data) is handled by:

A. The Physical layer B. The Ethernet CRC field automatically C. Higher layer protocols (e.g., TCP) D. The Preamble

Answer: C. Why: Ethernet is connectionless and unreliable by design — corrupted frames are silently dropped; any needed retransmission is the job of a higher-layer protocol like TCP.

Q5. Why must the CRC field be placed at the end of an Ethernet frame rather than the beginning?

A. Historical convention only B. Because CRC must be calculated after the rest of the frame's bits are known/assembled C. Because CRC is part of the Preamble D. Because MAC addresses must come first for switching

Answer: B. Why: CRC is a checksum over the frame contents, so it can only be computed once the full frame (addresses + length + data) is ready.

Q6. Manchester encoding embeds clock synchronization by:

A. Sending a separate clock signal B. A guaranteed voltage transition at the mid-point of every bit C. Using only high voltage for all bits D. Encrypting the bit stream

Answer: B. Why: The mandatory mid-bit transition means the signal never "flat-lines," letting the receiver continuously resynchronize — at the cost of needing 2 transitions per bit ($n=0.5$ efficiency).

7.7 Network Layer — IPv4 Protocol

7.1 Network Layer Role

- **Services:** Logical addressing, Routing (best path via routers), Packetizing (encapsulation with header), Error control (header checksum + ICMP), Congestion control.
- **TRAP:** Network layer does **not** provide flow control directly (that's DLL/Transport's job) — a frequently misattributed function.
- **IPv4** = unreliable, connectionless, **best-effort**, datagram (packet-switched) protocol. "Best-effort" ⇒ packets may be corrupted, lost, duplicated, delayed, or arrive **out of order**. Reliability must be added by a higher layer (TCP).

7.2 IPv4 Datagram Header

Header length = 20 to 60 bytes (variable, in units of 4-byte words)
 HLEN field (4 bits) stores header length / 4 ⇒ actual header length = HLEN × 4
 Range of HLEN field value = [5, 15] (5×4=20 min, 15×4=60 max)
 Total Length field (16 bits) = header + data, max = 65535 bytes
 Data length = Total Length - (HLEN × 4)

Field	Size	Purpose
Version	4 bits	IPv4 = value 4
HLEN	4 bits	Header length ÷ 4 (see above)
Type of Service/Precedence	8 bits	QoS priority; low-precedence datagrams dropped first under congestion
Total Length	16 bits	Header + data, in bytes
Identification	16 bits	Unique per original datagram; copied unchanged to every fragment for reassembly
Flags	3 bits	bit0 reserved, D (Don't Fragment) , M (More Fragments)
Fragmentation Offset	13 bits	Position of fragment's data in original datagram, in units of 8 bytes
TTL	8 bits	Hop limit
Protocol	8 bits	Which upper-layer protocol (TCP/UDP/...) gets the payload
Header Checksum	16 bits	Covers header only , not payload
Source/Dest Address	32 bits each	IPv4 addresses
Options	0–40 bytes	Optional; testing/debugging/routing control

- Minimum total datagram length = 20 bytes (header only, no data).

7.3 Fragmentation & MTU

- **MTU** = max payload a link-layer frame can carry (e.g., typical LAN Ethernet MTU = 1500 bytes). If a datagram > next-hop MTU → **fragmented**.
- Fragmentation can occur at the **source or any intermediate router**; **only the final destination reassembles** (not intermediate routers).
- A datagram may be fragmented multiple times across different MTU links en route.
- **D bit = 1** ⇒ must NOT fragment (if too big, it's dropped + ICMP error). **M bit = 1** ⇒ more fragments follow; **M=0** ⇒ last (or only) fragment.
- **Fragmentation offset** = (starting byte position of this fragment's data in original datagram) / 8.

Worked Fragmentation Example

Original datagram data = 4000 bytes (numbered 0–3999); MTU allows 1400-byte data chunks.

Fragment 1: bytes 0–1399 → offset = 0/8 = **0**.

Fragment 2: bytes 1400–2799 → offset = 1400/8 = **175**.

Fragment 3: bytes 2800–3999 → offset = 2800/8 = **350**.

(Each fragment's own Total Length = its data size + its own header size; all fragments share the same Identification value; only the last fragment has M=0.)

7.4 TTL, Protocol & Checksum Fields

- **TTL (Time To Live):** decremented by 1 at every router hop; **reaches 0** → **datagram discarded** (router sends ICMP Time Exceeded). Prevents infinite routing loops. Setting TTL=1 intentionally confines a packet to the local network (used by `traceroute`).
- **Protocol field:** tells the destination which upper-layer protocol (TCP/UDP/ICMP/IGMP) should receive the payload.
- **Header Checksum:** covers **ONLY the header**, not the payload — because (a) TTL (and other header fields) change at every hop, forcing recalculation each time, and (b) upper layers (TCP/UDP) already checksum the full payload, so re-checking payload at IP level would be redundant and slow.

7.5 IP Options (variable part, up to 40 bytes, optional)

Option	Function
End of Option	Padding/terminator
Record Route	Logs up to 9 router addresses traversed (debugging)
Strict Source Route	Sender dictates the EXACT route; deviation ⇒ datagram discarded
Loose Source Route	Must visit listed routers, but additional routers are also allowed
Timestamp	Records processing time at each router (clock sync not guaranteed)

7.6 Companion Network-Layer Protocols

Protocol	Maps	Direction
ARP	IP address → MAC (link-layer) address	Forward — "I know your IP, what's your MAC?" (broadcast request, unicast reply)
RARP	MAC address → IP address	Reverse — used by diskless machines at boot to learn their own IP
ICMP	—	Error reporting + diagnostic queries for IP (IP itself has no error-reporting)
IGMP	—	Manages multicast group membership (one-to-many delivery)

- **TRAP:** ARP and RARP are exact **inverses** of each other — ARP: IP→MAC; RARP: MAC→IP. Easy to swap under exam pressure.
- **ICMP reports errors, it does NOT correct them** — correction/retransmission is TCP's job.
- **ICMP error messages are suppressed for:** datagrams carrying another ICMP error message, non-first fragments, multicast destinations, and special addresses (127.0.0.0, 0.0.0.0) — prevents infinite error-loops.
- **ICMP message types:** Error-reporting (Destination Unreachable, Time Exceeded, Parameter Problem, Redirect) vs Query pairs (Echo Request/Reply = ping ; Router Solicitation/Advertisement; Address-Mask Request/Reply; Timestamp Request/Reply for RTT/clock sync).

Questions

Q1. [Numerical] An IPv4 header's HLEN field contains binary value 0110. What is the actual header length?

A. 6 bytes B. 20 bytes C. 24 bytes D. 30 bytes

Answer: C. **Why:** 0110 = 6; header length = $6 \times 4 = 24$ bytes.

Q2. [Numerical] A datagram has Total Length = 1500 bytes and HLEN = 5. How much actual data (payload) does it carry?

A. 1500 bytes B. 1480 bytes C. 1475 bytes D. 1420 bytes

Answer: B. **Why:** Header = $5 \times 4 = 20$ bytes; Data = Total Length – Header = $1500 - 20 = 1480$ bytes.

Q3. [Numerical] A 3000-byte payload must be fragmented for a link with MTU allowing 1000 bytes of data per fragment (multiple of 8). What is the fragmentation offset of the 3rd fragment?

A. 100 B. 200 C. 250 D. 1000

Answer: C. **Why:** 3rd fragment starts at byte 2000; offset = $2000/8 = 250$.

Q4. [Trap] If the "Don't Fragment" (D) bit is set to 1 and the datagram exceeds the outgoing link's MTU, what happens?

A. It is fragmented anyway B. It is dropped, and an ICMP error is sent to the sender C. It is queued until the link is upgraded D. The M bit is automatically set to 1

Answer: B. **Why:** D=1 forbids fragmentation; a router that cannot forward the oversized datagram intact must discard it and report the problem via ICMP.

Q5. Why is the IPv4 header checksum recalculated at every router hop?

A. Because the payload changes at each hop B. Because fields like TTL change at every hop, altering the header C. Because IP addresses change at each hop D. It is never recalculated

Answer: B. **Why:** TTL decrements at each hop, changing header content, so the header checksum (which covers header only) must be recomputed each time.

Q6. Which protocol allows a diskless machine to discover its own IP address using only its known MAC address?

A. ARP B. RARP C. ICMP D. IGMP

Answer: B. **Why:** RARP performs the reverse mapping (MAC→IP), historically used for diskless workstation booting.

Q7. Which ICMP message is generated when a router's decremented TTL field reaches zero?

A. Destination Unreachable B. Redirect C. Time Exceeded D. Parameter Problem

Answer: C. **Why:** TTL reaching 0 causes the router to discard the datagram and notify the source with a Time Exceeded message.

Q8. [Trap] Which of the following will NOT trigger an ICMP error message?

A. A normal unicast datagram whose TTL expires B. A datagram that is itself carrying an ICMP error message C. A datagram with a bad header checksum D. A datagram whose destination port is unreachable

Answer: B. **Why:** ICMP explicitly suppresses generating error messages about datagrams that already carry ICMP error messages, to prevent infinite error-reporting loops.

Q9. IGMP exists primarily to support which type of communication?

A. Unicast B. Broadcast C. Multicast D. Anycast

Answer: C. **Why:** IGMP manages group membership so routers know which hosts want a given multicast stream, enabling one-to-many delivery.

7.8 Network Layer — IP Addressing

(Highest-yield numerical topic in the whole CN syllabus — subnetting/CIDR/VLSM appear almost every year.)

8.1 IPv4 Basics

- IPv4 address = **32 bits** → max 2^{32} (~4.29 billion) addresses. IPv6 = 128 bits (~ 3.4×10^{38}).
- Structure: **Prefix (Network ID) + Suffix (Host ID)**. All hosts in one network share the same Network ID; different networks may reuse the same Host ID.
- Notation: Binary (32 bits, 4 octets) or Dotted-Decimal (e.g., 117.149.29.2).
- Only network-layer devices (hosts, routers) have IP addresses — **switches, hubs, repeaters do NOT** (they operate below the network layer).

8.2 Classful Addressing

Class	1st bit(s)	1st octet range	NetID bits	HostID bits	#Networks	#Hosts/network
A	0	0–127	8	24	$2^7 - 2 = 126$	$2^{24} - 2 = 16,777,214$
B	10	128–191	16	16	$2^{14} = 16,384$	$2^{16} - 2 = 65,534$
C	110	192–223	24	8	2^{21}	$2^8 - 2 = 254$
D	1110	224–239	— (multicast)	—	—	—
E	1111	240–255	— (reserved/experimental)	—	—	—

Formula: Hosts per network = $2^{(\text{host bits})} - 2$ [-2 for Network ID (all-0 host bits) and Broadcast (all-1 host bits)]
 Networks in class = $2^{(\text{netid bits used, excluding class-marker bits})}$ [minus reserved, for Class A]

- Class A loses 2 networks to reserved addresses: **0.0.0.0** (broadcast/unspecified) and **127.0.0.0/8** (loopback, e.g. 127.0.0.1 — used for local software testing) → usable range [1,126].
- **TRAP:** The “-2” applies twice for different reasons — once for reserved network numbers (Class A only) and always for host addresses in every network (network address + broadcast address).

8.3 Casting Types

- **Unicast:** 1-to-1. **Multicast:** 1-to-many (specific group, Class D). **Broadcast:** 1-to-all.
- **Limited broadcast:** all 32 bits = 1 → **255.255.255.255** (same network only).
- **Direct broadcast:** Network ID fixed + Host ID all 1s → reaches all hosts of a *specific other* network (e.g., 200.1.2.255 for network 200.1.2.0/24).

8.4 Subnetting — Why & FLSM

- **Why subnet:** improves security (isolate departments), simplifies management of huge Class A/B blocks.
- **Disadvantages:** loses 2 addresses per subnet (net ID + broadcast), increases routing table size, adds complexity.
- **FLSM (Fixed-Length Subnet Mask):** every subnet is the SAME size (same mask, same host count) — simple but wastes addresses if actual host needs vary.
- **VLSM (Variable-Length Subnet Mask):** subnets sized to actual need — no wastage, but harder to design (must allocate **largest subnets first**, so start addresses stay divisible by block size).
- Subnet mask = 32-bit number, 1s = Network+Subnet ID portion, 0s = Host ID portion. Default masks: Class A=255.0.0.0, B=255.255.0.0, C=255.255.255.0.

Worked FLSM Example — 200.1.2.0/24 split into 4 equal subnets

Need 4 subnets => borrow 2 bits => each subnet size = $256/4 = 64$ addresses => /26 mask (255.255.255.192)

Subnet 1: 200.1.2.0 - 200.1.2.63 | hosts [200.1.2.1 - 200.1.2.62] | broadcast 200.1.2.63
 Subnet 2: 200.1.2.64 - 200.1.2.127 | hosts [200.1.2.65 - 200.1.2.126] | broadcast 200.1.2.127
 Subnet 3: 200.1.2.128 - 200.1.2.191 | hosts [200.1.2.129 - 200.1.2.190] | broadcast 200.1.2.191
 Subnet 4: 200.1.2.192 - 200.1.2.255 | hosts [200.1.2.193 - 200.1.2.254] | broadcast 200.1.2.255
 (each subnet: 64 addresses, 62 usable hosts = $64 - 2$)

Worked VLSM Example — 14.24.74.0/24 needs subnets of 120, 60, 10 hosts

Step 1: round each requirement up to nearest (power of 2 - 2):
 120 hosts -> need 122+ addresses -> 128 ($=2^7$) -> /25 mask
 60 hosts -> need 62+ addresses -> 64 ($=2^6$) -> /26 mask
 10 hosts -> need 12+ addresses -> 16 ($=2^4$) -> /28 mask

Step 2: allocate LARGEST block first so start addresses stay aligned:

120-block: 14.24.74.0/25 -> range .0-.127 | hosts .1-.126 | broadcast .127
 60-block: 14.24.74.128/26 -> range .128-.191 | hosts .129-.190 | broadcast .191
 10-block: 14.24.74.192/28 -> range .192-.207 | hosts .193-.206 | broadcast .207
 (Unused/reserved for future growth: .208-.255)

- **TRAP:** If you allocate smallest-first in VLSM, later larger blocks may not start at an address divisible by their own size — always allocate **largest requirement first**.

8.5 CIDR (Classless Inter-Domain Routing)

- Solves classful addressing's rigidity (Class A/B too big, C too small, E wasted) — notation **a.b.c.d/n** (n = prefix/network bits).

Number of addresses in block N = $2^{(32-n)}$
 First address = (any address in block) AND (mask)
 Last address = (any address in block) OR (NOT mask)

Rules for a valid CIDR block

1. All addresses must be **contiguous**.
2. Block size must be a **power of 2**.
3. The first address must be **divisible by the block size**.

Worked Example — 167.199.170.82/27

```
/27 => mask = 255.255.255.224 => host bits = 5 => block size = 2^5 = 32 addresses
First address = address AND mask = 167.199.170.64
Last address = address OR (NOT mask) = 167.199.170.95
=> Block range: 167.199.170.64 - 167.199.170.95 (32 addresses)
```

Quick validity checks (apply the 3 rules)

- Range 100.1.2.32–100.1.2.47: size=16 ($=2^4$), 32 is divisible by 16 → **Valid CIDR block**, representation = 100.1.2.32/28.
- Range 150.10.20.64–150.10.20.127: size=64 ($=2^6$), 64 divisible by 64 → **Valid**, = 150.10.20.64/26.

8.6 Supernetting (CIDR route aggregation)

- Combines multiple contiguous same-size blocks into one larger block (reduces routing table size).
- Rules: blocks must be **contiguous**, and the **first network ID must be divisible by the combined block's size**.
- Example: 128.56.24.0/24, 128.56.25.0/24, 128.56.26.0/24, 128.56.27.0/24 → 4 contiguous /24s ($4 \times 256 = 1024 = 2^{10}$ addresses) → first ID 24 is divisible by 4 → valid supernet = **128.56.24.0/22**.
- **TRAP**: Aggregation FAILS if blocks are non-contiguous or the first ID isn't divisible by the combined size — e.g., 100.1.2.0/25 + 100.1.2.128/26 + 100.1.3.192/26 cannot form one supernet (there's a gap between 100.1.2.192 and 100.1.3.191 that isn't covered).

8.7 Special / Reserved Addresses

- **127.0.0.0/8** — Loopback (self-test, e.g. 127.0.0.1).
- **0.0.0.0** — unspecified/this-host.
- **255.255.255.255** — limited broadcast.
- **Private IP ranges (not routable on public Internet):**

```
10.0.0.0 - 10.255.255.255 (10.0.0.0/8)
172.16.0.0 - 172.31.255.255 (172.16.0.0/12)
192.168.0.0 - 192.168.255.255 (192.168.0.0/16)
```

- An ISP typically provides: **IP address, Default Gateway (DGW), Subnet Mask (SM), DNS server**.

Questions

Q1. [Numerical] How many usable host addresses are available in a Class B network?

A. 65,536 B. 65,534 C. 16,384 D. 254

Answer: B. **Why:** Class B has 16 host bits → $2^{16} - 2 = 65,534$ (subtracting network ID and broadcast addresses).

Q2. [Numerical] A network 192.168.10.0/24 is divided into 8 equal subnets. What is the new subnet mask and how many usable hosts per subnet?

A. /26, 62 hosts B. /27, 30 hosts C. /28, 14 hosts D. /27, 32 hosts

Answer: B. **Why:** 8 subnets need 3 borrowed bits → /27 (255.255.255.224); each subnet has $2^5 = 32$ addresses → $32 - 2 = 30$ usable hosts.

Q3. [Numerical] Using VLSM on 14.24.74.0/24 for subnets needing 120, 60, and 10 hosts (allocated largest-first), what is the broadcast address of the 60-host subnet?

A. 14.24.74.127 B. 14.24.74.191 C. 14.24.74.207 D. 14.24.74.255

Answer: B. **Why:** 120-host block takes /25 (.0–.127); the 60-host block starts right after at .128 and needs /26 (64 addresses) → range .128–.191 → broadcast = .191.

Q4. [Trap] Why must VLSM allocate the largest subnet requirement FIRST?

A. It doesn't matter, order is irrelevant B. So each block's starting address stays properly aligned/divisible by its own size C. To save router memory D. Because smaller blocks must always get lower addresses

Answer: B. **Why:** A CIDR/VLSM block's start address must be divisible by its block size; assigning large blocks first keeps every subsequent block's start address correctly aligned.

Q5. [Numerical] Find the number of addresses, first address, and last address for 167.199.170.82/27.

A. 16 addresses, .64–.79 B. 32 addresses, .64–.95 C. 64 addresses, .0–.63 D. 32 addresses, .82–.113

Answer: B. **Why:** /27 → 5 host bits → $2^5 = 32$ addresses; mask=255.255.255.224; First = address AND mask = .64; Last = address OR NOT(mask) = .95.

Q6. [Numerical/Trap] Is the range 100.1.2.20–100.1.2.35 a valid CIDR block?

A. Yes, it's /28 B. Yes, it's /27 C. No — the block size (16) is a power of 2, but the first address (20) is not divisible by 16 D. No, because the range isn't contiguous

Answer: C. **Why:** Size = $35 - 20 + 1 = 16 = 2^4$ (power of 2, rule 2 OK), but $20 \div 16$ is not an integer, so rule 3 (first address divisible by block size) fails — not a valid CIDR block.

Q7. Four contiguous blocks 128.56.24.0/24 through 128.56.27.0/24 are aggregated. What is the resulting supernet?

A. 128.56.24.0/23 B. 128.56.24.0/22 C. 128.56.0.0/16 D. 128.56.24.0/25

Answer: B. **Why:** $4 \times 256 = 1024 = 2^{10}$ addresses, and 24 is divisible by 4 (the block count) → valid aggregation into a /22 ($32 - 10 = 22$).

Q8. Which of these is a valid private IP address range?

A. 11.0.0.0 – 11.255.255.255 B. 172.16.0.0 – 172.31.255.255 C. 192.169.0.0 – 192.169.255.255 D. 127.0.0.0 – 127.255.255.255

Answer: B. **Why:** 172.16.0.0/12 is one of the three reserved private ranges (along with 10.0.0.0/8 and 192.168.0.0/16); 127.x is loopback, not private-use.

Q9. [Conceptual] What is the fundamental disadvantage of FLSM compared to VLSM?

A. FLSM cannot use CIDR notation B. FLSM wastes addresses when actual subnet host-count needs differ, since every subnet must be the same size C. FLSM cannot be used with Class C D. FLSM requires more routing entries than VLSM

Answer: B. **Why:** FLSM forces all subnets to the same fixed size (sized for the largest requirement), wasting addresses in smaller subnets — VLSM sizes each subnet to its actual need.

Q10. [Trap] A router receives a packet for 127.0.0.1. This address is:

A. A valid public destination on the Internet B. A loopback address used for local self-testing, never routed onto a real network C. The default gateway address D. A broadcast address

Answer: B. **Why:** The entire 127.0.0.0/8 block is reserved for loopback/local testing — it never actually leaves the host.

7.9 Network Layer — Routing Protocols

9.1 Routing Basics

- **Routing table:** tells a router which interface to use to forward a packet toward a destination. **Routing** = building/updating this table; **switching** = actually forwarding a packet along the chosen path.
- **Static routing table:** manually configured; impractical for large/changing networks like the Internet.
- **Dynamic routing table:** auto-updates as topology/traffic changes — required for the Internet's scale.
- **Flooding** (routing-table-free alternative): forward an incoming packet out on every interface except the one it arrived on. Guarantees shortest-path delivery and high reliability, but causes massive **duplicate packets** and traffic — needs hop-count/sequence-number limits to avoid infinite propagation.
- **Autonomous System (AS):** a group of networks/routers under one administrative domain. **Intra-domain routing** = within one AS (e.g., RIP, OSPF). **Inter-domain routing** = between ASes (uses one standard protocol, e.g., BGP).

9.2 Distance Vector Routing (DVR)

- Each router keeps a table of (destination, cost, next-hop) and shares **only destination+cost** (not next-hop) with its **immediate neighbors only** — based on **Bellman-Ford** algorithm.
- Updates: **Periodic** (e.g., every 30s) and **Triggered** (on detecting a change/link failure).
- **Count-to-Infinity / Two-Node Loop Instability:** if a link to destination X fails, a neighbor that still has stale "I can reach X" info keeps advertising it; the two nodes then loop, each believing the other has a path, slowly incrementing cost toward infinity instead of converging immediately.
- **Split Horizon:** a router never advertises a route back to the neighbor it learned that route from (since that neighbor already knows it, and it prevents feeding back stale info) — reduces but does not fully eliminate count-to-infinity.
- **TRAP:** DVR shares information with **neighbors only**, not the whole network — this is the opposite of Link State Routing (see below); a very common mix-up.

9.3 RIP (Routing Information Protocol) — a real DVR implementation

- Metric = **hop count**; same cost per hop regardless of link speed.
- **Max hop count = 15; 16 = unreachable (infinity).**
- Runs over **UDP port 520** at the application layer.
- Timers: Periodic update timer; **Expiration timer (180s)** — no update ⇒ route marked expired; **Garbage collection timer (120s after expiry)** — route then removed.
- Fast convergence *because* of the small hop-count ceiling, but this small ceiling also limits RIP to small networks.

9.4 Link State Routing (LSR)

- Every router builds a **complete map of the entire network topology** (nodes, links, costs, up/down state) by **flooding Link State Packets (LSPs) to ALL routers** — not just neighbors.
- Uses **Dijkstra's algorithm** to compute the shortest-path tree with itself as root → derives its own routing table (each router's tree/table is unique even though the underlying topology data — the Link-State Database, LSDB — is identical everywhere).
- No count-to-infinity problem (each router computes routes independently from full, consistent topology data) — converges faster, but at higher bandwidth/CPU/memory cost.
- **TRAP:** LSR floods to the *entire network* about a router's *own direct neighbors only*; DVR tells *neighbors only* about the *entire network* (as the router understands it) — these are inverse in scope.

9.5 OSPF (Open Shortest Path First) — a real LSR implementation

- **Intra-domain** protocol; metric = flexible **cost** (can reflect bandwidth/throughput/RTT/reliability — administrator-configurable), unlike RIP's fixed hop count.
- Divides a large AS into **Areas** connected via a central **Backbone Area**, to limit flooding scope.
- Runs **directly over IP** (not TCP/UDP).
- Faster convergence and more robust than RIP (a single router's failure has less network-wide impact), at the cost of more complex/heavier update traffic.

9.6 DVR vs LSR vs RIP vs OSPF

Feature	DVR (RIP)	LSR (OSPF)
Shares	Own routing table (dest+cost)	Own direct-link states (LSPs)
Shared with	Neighbors only	All routers (flooded)
Knowledge basis	Local	Global (full topology)
Algorithm	Bellman-Ford	Dijkstra
Convergence	Slow	Fast
Bandwidth/traffic use	Low	High
Count-to-infinity?	Yes (mitigated by split horizon)	No
Era popularized	~1980s	~1990s
Metric	Hop count (max 15)	Configurable cost

9.7 Routing in Subnets (Longest Prefix Match)

- Router does **bitwise AND** of destination IP with each routing-table subnet mask, compares to table entries:
- **Single match** → forward via that interface.
- **Multiple matches** → use the entry with the **longest subnet mask** (longest prefix match).
- **No match** → use the **default route**.

- **Fixed-Length Subnetting:** one mask for all subnets, AND done once. **Variable-Length Subnetting:** different masks per subnet, AND done against each until a match is found.

Questions

Q1. [Trap] In Distance Vector Routing, a router shares its routing information with:

A. All routers in the network B. Only its immediate neighbors C. Only the backbone router D. No one; it is passive

Answer: B. Why: DVR is based on local knowledge — each router exchanges its distance vector only with directly connected neighbors, unlike Link State Routing which floods to everyone.

Q2. The "count-to-infinity" problem arises in which routing approach, and is reduced by which technique?

A. Link State Routing; reduced by Dijkstra's algorithm B. Distance Vector Routing; reduced by Split Horizon C. OSPF; reduced by flooding D. Flooding; reduced by hop count

Answer: B. Why: Count-to-infinity is a DVR-specific instability from stale neighbor information; Split Horizon (not re-advertising a route back to the neighbor it came from) mitigates it.

Q3. In RIP, a destination reported with hop count 16 is interpreted as:

A. 16 hops away, still reachable B. Unreachable (infinity) C. The default route D. A routing loop

Answer: B. Why: RIP caps usable hop counts at 15; a metric of 16 is defined as "unreachable," limiting RIP to small/medium networks.

Q4. Which algorithm underlies Link State Routing's shortest-path computation, and which underlies Distance Vector Routing?

A. Dijkstra for DVR, Bellman-Ford for LSR B. Bellman-Ford for both C. Dijkstra for LSR, Bellman-Ford for DVR D. Neither uses a named algorithm

Answer: C. Why: LSR builds a shortest-path tree via Dijkstra's algorithm using full topology knowledge; DVR is rooted in the Bellman-Ford relaxation approach using only neighbor-exchanged distances.

Q5. Why does OSPF generally converge faster and prove more robust than RIP for large networks?

A. OSPF uses hop count exclusively B. OSPF floods full topology (link-state) data so each router independently computes consistent shortest paths, avoiding DVR-style instabilities C. OSPF has a lower maximum hop count D. OSPF does not use any routing algorithm

Answer: B. Why: Because every OSPF router works from a complete, consistent view of the topology (via LSPs + Dijkstra), it avoids the slow, locally-inconsistent convergence that plagues distance-vector protocols like RIP.

Q6. A router's routing table has two matching entries for a destination IP after the bitwise-AND check, with subnet masks /24 and /28. Which entry is used?

A. /24, since it was likely added first B. /28, the longest prefix match C. Both are used via load balancing D. Neither; the default route is used

Answer: B. Why: When multiple entries match, the router selects the one with the longest subnet mask (most specific match) — the standard longest-prefix-match rule.

Q7. OSPF's metric differs from RIP's in that OSPF:

A. Always equals hop count like RIP B. Can be configured based on factors like bandwidth/throughput/reliability, not just a fixed per-hop cost C. Is limited to a maximum of 15 D. Requires UDP port 520 like RIP

Answer: B. Why: RIP assigns a uniform "1" per hop; OSPF allows administrator-defined cost reflecting actual link characteristics, giving more realistic path selection.

Q8. [Trap] "Link State Routing tells each router what it knows about the entire network." Correct this statement:

A. It is correct as stated B. Link State Routing has each router tell the WHOLE network what it knows about its OWN direct links only C. LSR never shares any information D. LSR only works within a single subnet

Answer: B. Why: The scope is inverted from DVR: in LSR, a router broadcasts (floods) information about only its own immediate links, but this reaches every router in the network — the opposite pattern from DVR (local broadcast of global-ish knowledge).

7.10 Transport Layer — Services & TCP

10.1 Transport Layer Role & Protocol Types

- Network layer = **host-to-host** delivery. Transport layer = **process-to-process** delivery (the missing link to the actual application).
- **Connectionless (UDP)**: each segment independent, no setup/teardown, unreliable, fast.
- **Connection-oriented (TCP, SCTP)**: connection established before transfer, torn down after; reliable (flow + error control done **end-to-end**, not link-by-link like DLL).
- **TCP**: connection-oriented, reliable, full-duplex, byte-stream. **SCTP**: connection-oriented, reliable, combines TCP+UDP features (message-oriented + multi-streaming).

10.2 Ports & Socket Addresses

- Port number = **16-bit** (0–65535), identifies a process on a host.

Well-Known Ports:	0 – 1023	(IANA-assigned; e.g. HTTP 80, HTTPS 443, FTP 20/21, SSH 22, Telnet 23, SMTP 25, DNS 53, DHCP 67/68, POP3 110)
Registered Ports:	1024 – 49151	(registerable, not IANA-controlled)
Dynamic/Ephemeral:	49152 – 65535	(temporary, client-side, randomly chosen)

- **Socket address** = **IP address** + **Port number**. A TCP connection is uniquely identified by the pair of socket addresses (client socket, server socket).
- **Encapsulation**: App message + (src,dst) socket addresses → Transport layer adds header → **segment**. **Decapsulation** reverses this at the receiver.

10.3 TCP Header (20–60 bytes)

```
| Src Port(16b) | Dst Port(16b) |
| Sequence Number (32b) |
| Acknowledgment Number (32b) |
| HdrLen(4b) | Reserved(6b) | Flags(6b: URG,ACK,PSH,RST,SYN,FIN) | Window Size(16b) |
| Checksum(16b) | Urgent Pointer(16b) |
| Options (0-40 bytes) |
```

- **Header Length field (4 bits)** = number of 4-byte words → value ranges **5 to 15** → header size = value × 4 bytes → **20 bytes (no options) to 60 bytes (max options, 40B)**.
- **Checksum** covers the whole TCP segment + **a pseudo-header from IP** (pseudo-header's Protocol field = **6** for TCP).
- **Window field** = receiver's advertised buffer space, max **65,535 bytes** (16 bits) — can be extended via a **Window Scale option** (up to ~1 GB, effectively 30 bits) since 64KB is tiny for modern high-bandwidth links.
- **Flags**: **PSH** = push data to application immediately, skip buffering (used for interactive/chat traffic). **RST** = abort an invalid/erroneous connection. **URG** + Urgent Pointer = marks urgent data.
- **TRAP**: TCP counts **bytes**, not packets (unlike IP, which counts packets) or bits (unlike DLL/HDLC, which counts bits).

10.4 Sequence & Acknowledgment Numbers

- **ISN (Initial Sequence Number)** is chosen **randomly** at connection setup (not 0) — prevents old-duplicate-segment confusion and improves security.
- Next segment's sequence number = previous segment's sequence number + bytes it carried.
- **Acknowledgment number** is **cumulative**: ACK = (last byte received safely) + 1.

Worked Example

5000-byte file sent as five 1000-byte segments; first byte numbered **10001**.
Segment sequence numbers: **10001, 11001, 12001, 13001, 14001**.

Wrap-Around

Wrap-Around Time (WAT) = $2^{32} / \text{Bandwidth (bytes/sec)}$

- If **WAT > segment Lifetime (LT, ~180s / 3 min)** → safe (old sequence numbers fully retired before reuse). If **WAT < LT** → risk of duplicate/confusing sequence numbers on very high-bandwidth links.
- **TRAP**: Higher bandwidth → **smaller** WAT (sequence space exhausts faster) → *more* risk of wrap-around conflicts, not less.

10.5 Connection Establishment & Termination

- **3-Way Handshake (establishment)**: Client --SYN--> Server --SYN+ACK--> Client --ACK--> Server. SYN (and FIN) segments **consume one sequence number** each even with no data payload.
- **Termination**: initiator sends **FIN**; other side replies **FIN+ACK** (or separately ACK then its own independent FIN) → final **ACK**. **TRAP**: Textbooks often show termination as a 4-way exchange (each side sends its own independent FIN, since TCP is full-duplex and each direction must close separately) — the "3-segment" version shown here just combines the responder's ACK and FIN into one segment; do not assume termination is always exactly symmetric with the 3-way SYN handshake.
- **SYN Flooding (DoS)**: attacker sends many spoofed-source SYNs; server allocates resources (TCB) for each and waits for the final ACK that never comes → resource exhaustion, denial of service to real clients.

10.6 Retransmission Triggers

Trigger	Confidence of loss
Timeout timer expiry	Strong signal of congestion — nothing came back at all
3 duplicate ACKs ("Fast Retransmit")	Weaker signal — later segments <i>are</i> getting through, only one was likely dropped

- TCP effectively blends **Go-Back-N** (cumulative ACKs, simplicity) with **Selective Repeat** (accepts/buffers out-of-order segments) behavior.

Questions

Q1. [Numerical] A TCP segment's header length field value is 10. What is the actual header size?

A. 10 bytes B. 20 bytes C. 40 bytes D. 60 bytes

Answer: C. **Why:** Header length = field value $\times$ 4 bytes = $10 \times 4 = 40$ bytes.

Q2. Why does TCP choose a random Initial Sequence Number (ISN) instead of starting at 0?

A. To save memory B. To prevent confusion with old duplicate segments from a previous connection and improve security C. Because 0 is reserved D. To simplify checksum calculation

Answer: B. **Why:** A predictable/reused starting point could let old delayed segments be mistaken for new ones, or allow sequence-prediction attacks.

Q3. [Trap] Which flag should be set for a chat application needing immediate delivery without buffering delay?

A. RST B. URG C. PSH D. FIN

Answer: C. **Why:** PSH forces the transport layer to send immediately and tells the receiver to hand data to the application without waiting to fill a full segment.

Q4. [Numerical] Network bandwidth = 2^{32} bytes/sec (hypothetical). What is the sequence number wrap-around time?

A. 1 second B. 2 seconds C. 4 seconds D. It never wraps around

Answer: A. **Why:** $WAT = 2^{32} / \text{Bandwidth} = 2^{32} / 2^{32} = 1$ second.

Q5. [Trap] A sender detects segment loss via 3 duplicate ACKs rather than a timeout. Compared to timeout-triggered loss detection, this indicates:

A. A stronger, more certain sign of network congestion B. A weaker sign of congestion, since later segments are still getting through C. That the connection has failed completely D. No difference at all

Answer: B. **Why:** Duplicate ACKs mean subsequent segments ARE arriving (just one was dropped/reordered) — a much milder signal than a full timeout where nothing arrived at all.

Q6. In the TCP pseudo-header used for checksum calculation, the Protocol field value for TCP is:

A. 17 B. 6 C. 1 D. 89

Answer: B. **Why:** Protocol number 6 identifies TCP in the IP pseudo-header (17 = UDP, 1 = ICMP, 89 = OSPF).

Q7. What is the maximum receiver window size TCP can advertise using only the standard 16-bit Window field (without the Window Scale option)?

A. 4095 bytes B. 65,535 bytes C. 1 GB D. Unlimited

Answer: B. **Why:** 16 bits $\rightarrow$ max value 65,535 bytes; larger windows need the Window Scale option (extends effectively to ~ 30 bits / $\sim 1\text{GB}$).

Q8. Which TCP behavior most resembles Go-Back-N, and which resembles Selective Repeat?

A. Cumulative ACKs resemble GBN; buffering/accepting out-of-order segments resembles SR B. TCP uses only Stop-and-Wait C. TCP uses only Selective Repeat, never GBN D. TCP uses only Go-Back-N, never SR

Answer: A. **Why:** TCP's cumulative acknowledgment scheme is GBN-like, while its ability to buffer and accept out-of-order segments (rather than discarding them) is SR-like — a hybrid design.

7.11 Transport Layer — Congestion Control & UDP

11.1 Sender Window = min(Receiver Window, Congestion Window)

Sender Window Size = MIN(Receiver Window Size, Congestion Window Size)

- **Receiver window (flow control):** advertised by receiver via TCP header; caps data in flight so the receiver isn't overwhelmed.
- **Congestion window (cwnd):** sender-side-only estimate (never transmitted on the wire) of what the *network* can currently handle without congesting.
- TCP has 4 windows total for bidirectional communication (send+receive window, ×2 directions).

11.2 TCP Congestion Control — 3 Phases

Phase 1 - Slow Start (EXPONENTIAL growth):

cwnd starts at 1 MSS

After each RTT: cwnd = cwnd × 2 (1, 2, 4, 8, 16 MSS ... after RTT 1,2,3,4...)

Continues until cwnd reaches ssthresh (slow-start threshold)

$ssthresh = (\text{Receiver Window Size} / \text{MSS}) / 2$

Phase 2 - Congestion Avoidance (LINEAR / additive growth):

On each ACK: cwnd = cwnd + 1 MSS (roughly +1 MSS per RTT)

Continues until cwnd = Receiver Window Size, or loss detected

Phase 3 - Congestion Detection (on loss):

If detected via TIMEOUT (strong congestion signal):

ssthresh = cwnd / 2 ; cwnd = 1 MSS ; restart Slow Start

If detected via 3 DUPLICATE ACKs (weak signal, "Fast Retransmit/Recovery"):

ssthresh = cwnd / 2 ; cwnd = ssthresh ; resume Congestion Avoidance (NOT slow start)

- **TRAP:** "Slow Start" is a **misnomer** — its growth is exponential (fastest phase), not slow; it's only "slow" relative to just blasting the full window immediately.
- **TRAP:** Timeout and Triple-Duplicate-ACK loss detection do **NOT** react the same way — timeout drops all the way back to cwnd=1 MSS (full Slow Start restart); 3-dup-ACK only drops to ssthresh and resumes linear growth (Fast Recovery) — because 3-dup-ACK is weaker evidence of congestion (later packets are still arriving).
- Overall behavior = **AIMD** (Additive Increase, Multiplicative Decrease).

11.3 TCP Timers

Timer	Purpose	Behavior
Time-Wait	Avoid conflicts with late/duplicate packets after connection close	Waits 2 × segment Lifetime (2×LT) before fully releasing the port
Keep-Alive	Detect & close idle/dead connections	Sends up to 10 probes at 75-second intervals ; closes if no reply
Persistent	Avoid deadlock when receiver advertises window size = 0	Sender periodically sends a 1-byte probe to check if window has reopened
Acknowledgment (delayed ACK)	Efficient cumulative ACKs via piggybacking	Starts on segment arrival; ACKs everything received within the window in one shot

11.4 RTT / Timeout-Timer (TOT) Estimation Algorithms

- Setting TOT is harder at Transport layer than DLL (DLL only deals with one adjacent hop; Transport spans multi-hop, variable paths).
- **Rule of thumb:** High traffic → RTT rising → increase TOT (else false retransmissions worsen congestion). Low traffic → RTT falling → decrease TOT (else sender waits too long).

Basic Algorithm: $IRTT(n+1) = \alpha \times IRTT(n) + (1-\alpha) \times ARTT(n)$; $TOT = 2 \times IRTT$

Jacobson's Algorithm adds deviation:

$ID(n+1) = \alpha \times ID(n) + (1-\alpha) \times AD(n)$ where $AD(n) = |IRTT(n) - ARTT(n)|$

$TOT = 4 \times ID + IRTT$ (accounts for RTT *variability*, not just a fixed ×2 multiplier)

Karn's Modification: ignore RTT samples from RETRANSMITTED segments (their true RTT is ambiguous); instead simply DOUBLE the timeout timer on each retransmission.

- **TRAP:** The Basic Algorithm's flat "×2" multiplier has no adaptive justification — Jacobson's fixes this by explicitly modeling RTT deviation, giving a more responsive, accurate timeout.

11.5 Silly Window Syndrome (SWS)

- Problem: sender sends tiny segments repeatedly, or receiver advertises tiny window updates repeatedly → terrible efficiency (huge header-to-data overhead ratio).
- **Nagle's Algorithm** (sender-side fix): send the first small segment immediately, then **buffer** further data until the first is ACKed; then send buffered data as one segment. (Bad for latency-sensitive apps — can be disabled, e.g. TCP_NODELAY.)
- **Clark's Solution** (receiver-side fix): don't advertise a window update for a tiny amount of newly-free space — wait until a **substantial** amount (e.g., half the buffer) is free.
- **TRAP:** Nagle's Algorithm and Clark's Solution attack **opposite ends** of the same problem (sender-side batching vs receiver-side window-update batching) — they are complementary, not alternatives.

11.6 UDP (User Datagram Protocol)

UDP Header (fixed 8 bytes):

| Source Port (16b) | Destination Port (16b) |
| Total Length (16b) | Checksum (16b) |

Feature	TCP	UDP
Connection	Connection-oriented	Connectionless
Reliability	Reliable (ACK, retransmission)	Unreliable (best-effort)
Ordering	Ordered (sequence numbers)	No ordering guarantee
Header size	20–60 bytes	Fixed 8 bytes
Flow/Congestion control	Yes	None
Overhead	Higher	Very low
Typical use	File transfer, web (HTTP), email	DNS, streaming/VoIP, DHCP, RIP, SNMP

- UDP is preferred where **speed/low latency matters more than perfect delivery** (real-time media tolerates minor loss better than delay), or where the application implements its own reliability (e.g., TFTP).

Questions

Q1. [Numerical] cwnd starts at 1 MSS. What is cwnd after 4 RTTs of Slow Start (no loss)?

A. 4 MSS B. 8 MSS C. 16 MSS D. 32 MSS

Answer: C. **Why:** Exponential growth: $\text{cwnd} = 2^{\text{RTT}} = 2^4 = 16$ MSS.

Q2. [Trap] TCP detects a segment loss via a timeout (not 3 duplicate ACKs). What happens to cwnd and ssthresh?

A. ssthresh = cwnd/2, cwnd = ssthresh, resume Congestion Avoidance B. ssthresh = cwnd/2, cwnd = 1 MSS, restart Slow Start C. Both cwnd and ssthresh double D. No change, TCP ignores timeouts

Answer: B. **Why:** Timeout is a strong congestion signal → full reset: cwnd drops to 1 MSS and Slow Start restarts (unlike the milder 3-dup-ACK case, which only halves and resumes linear growth).

Q3. Which timer prevents a "zero window" deadlock where the receiver's window has silently reopened but the sender never finds out?

A. Keep-Alive Timer B. Time-Wait Timer C. Persistent Timer D. Acknowledgment Timer

Answer: C. **Why:** The Persistent Timer triggers periodic 1-byte probes specifically to detect a reopened window after a zero-window advertisement.

Q4. [Trap] What is the actual growth pattern during TCP's "Slow Start" phase?

A. Linear (+1 MSS per RTT) B. Exponential (doubles per RTT) C. Logarithmic D. Constant (no growth)

Answer: B. **Why:** Despite the name, Slow Start grows exponentially (cwnd doubles each RTT) until ssthresh is reached — a classic naming trap.

Q5. Why does Karn's Modification ignore RTT measurements taken from retransmitted segments?

A. Retransmitted segments are always corrupted B. It's ambiguous whether the received ACK corresponds to the original or the retransmitted copy, making the measured RTT unreliable C. Retransmissions never receive ACKs D. To save computation

Answer: B. **Why:** If a segment is retransmitted, an ACK could be for either copy — using this ambiguous timing to update RTT estimates would corrupt future timeout calculations.

Q6. [Numerical] What is the fixed size of a UDP header?

A. 4 bytes B. 8 bytes C. 20 bytes D. 16 bytes

Answer: B. **Why:** UDP header = Source Port + Dest Port + Length + Checksum, each 16 bits = 8 bytes total (much smaller than TCP's 20–60 byte header).

Q7. Which mechanism specifically prevents a sender from repeatedly transmitting tiny, inefficient segments?

A. Clark's Solution B. Nagle's Algorithm C. Keep-Alive Timer D. Karn's Modification

Answer: B. **Why:** Nagle's Algorithm buffers outgoing small segments until the previous one is acknowledged, then sends accumulated data as one larger segment.

Q8. Why is UDP preferred over TCP for DNS queries and live video/audio streaming?

A. UDP guarantees delivery, TCP does not B. UDP's lower overhead and lack of retransmission delay suit latency-sensitive, loss-tolerant traffic better than TCP's reliability guarantees C. UDP has a larger header for more metadata D. TCP cannot be used for these applications at all

Answer: B. **Why:** Real-time applications tolerate occasional packet loss far better than the added latency TCP's acknowledgment/retransmission machinery would introduce.

Q9. [Numerical] Receiver window size = 8000 bytes, MSS = 1000 bytes. What is the Slow Start threshold (ssthresh)?

A. 2 B. 4 C. 8 D. 16

Answer: B. **Why:** $\text{ssthresh} = (\text{Receiver Window} / \text{MSS}) / 2 = (8000/1000)/2 = 8/2 = 4$.

7.12 Application Layer Protocols

12.0 Consolidated Port/Protocol Reference (memorize this table)

Protocol	Port(s)	Transport	Purpose
SMTP	25	TCP	Sending/pushing mail (server→server, client→server)
POP3	110	TCP	Retrieving mail — downloads then typically deletes from server
IMAP4	143	TCP	Retrieving mail — keeps mail on server, multi-device sync, folders
HTTP	80	TCP	Web page transfer (stateless)
HTTPS	443	TCP (+SSL/TLS)	Encrypted web transfer
FTP	21 (control) + 20 (data)	TCP	File transfer — TWO separate connections
Telnet	23	TCP	Remote login, plaintext (insecure)
SSH	22	TCP	Secure remote login — replaced Telnet
DNS	53	UDP (queries) / TCP (zone transfer)	Domain name ↔ IP resolution
DHCP	67 (server) / 68 (client)	UDP	Dynamic IP address assignment

- **TRAP:** FTP is the classic "two ports" exam question — control (21) stays open for commands, data (20) opens per-transfer. Don't answer "FTP uses port 21" alone.
- **TRAP:** SMTP only **sends/pushes** mail; POP3/IMAP4 are **pull** protocols used only for retrieval — SMTP is never used to download mail.

12.1 Email — SMTP, POP3, IMAP4

- Email is **store-and-forward**: sender/receiver need not be online simultaneously.
- **MTA (Message Transfer Agent)**: handles transfer using SMTP, used **twice** — sender→sender's mail server, then sender's server→receiver's server.
- **POP3**: simple; **Delete mode** (mail removed from server after download — for a permanent PC) vs **Keep mode** (mail stays on server — for accessing from multiple/temporary devices). Cannot organize mail in folders on the server or partially download.
- **IMAP4**: more powerful — supports server-side folders/hierarchies, partial download, header preview before download, search before download.
- **MIME**: extends email beyond plain 7-bit ASCII to support non-English scripts, binary/audio/video attachments (converts non-ASCII → ASCII for transport, then back).
- **Webmail (Hotmail/Gmail-style)**: browser→mail server uses **HTTP** (not POP/IMAP) for both sending and retrieval; server→server transfer between mail servers still uses **SMTP**.
- Email address rule: exactly one "@"; no spaces; case-insensitive.
- **Bcc rule**: Bcc recipients are invisible to To/Cc and to each other; a Bcc'd person CAN see the To/Cc list.

12.2 WWW & Cookies

- WWW (Tim Berners-Lee, 1989, CERN) = client-server system of linked web pages.
- **URL** has 4 parts: **Protocol : Host : Port : Path** (e.g., http://host:port/path).
- WWW/HTTP is fundamentally **stateless** — each request/response is independent, server doesn't remember prior visits.
- **Cookies** solve statelessness: server sends a cookie in its response → browser stores it (keyed by domain) → browser automatically re-sends it on future requests to that domain → server recognizes the returning client.
- **TRAP:** A cookie's contents are set and read only by the **server** — the browser stores but never interprets/discloses cookie contents to the user ("made by the server, eaten by the server").

12.3 HTTP / HTTPS

- HTTP = connection-oriented over **TCP port 80**; defines request/response rules between browser and server.
- **Non-persistent connection**: new TCP connection per request/response (overhead for pages with many embedded objects).
- **Persistent connection (default since HTTP/1.1)**: one TCP connection reused for multiple requests — saves setup time/RTTs.
- **Proxy server**: caches responses to reduce load/latency for repeated requests.
- **HTTPS** = HTTP + SSL/TLS, **port 443** — adds Confidentiality (encryption), Integrity, Authentication (via digital certificates). HTTP itself has none of these.
- **TRAP:** HTTP uses a **single** TCP connection for a request/response cycle — unlike FTP's separate control+data connections.

12.4 FTP, DNS, Telnet, DHCP

- **FTP**: two TCP connections — **Control (port 21)** for commands, **Data (port 20)** for the actual file transfer. Sends credentials/data in **plaintext** (insecure) unless upgraded to SSL-FTP.
- **DNS**: translates domain names → IP addresses; **hierarchical & distributed**: Root servers → TLD servers (.com, .in, .org) → Authoritative servers (final name→IP mapping). Domain name space = inverted tree, max 128 levels, each label ≤ 63 characters, root = null label.
- **Generic domains**: .com, .edu, .mil, .org, .net. **Country domains**: .in, .uk, .us. **Inverse domain**: IP → name (reverse lookup).
- **Telnet**: port 23, remote virtual-terminal login; **plaintext** (major security flaw) → replaced by **SSH (port 22)** for production use; still used for raw port/connectivity testing.
- **DHCP**: Application-layer protocol using **UDP**; automates IP assignment via the **DORA** process: **Discover** (client broadcasts) → **Offer** (server proposes an IP) → **Request** (client accepts) → **Acknowledge** (server confirms). Server = port 67, Client = port 68.
- **0.0.0.0** = "I have no IP yet" (placeholder during Discover). **127.0.0.1** = loopback/localhost (tests own NIC, unrelated to DHCP). **169.254.x.x (APIPA)** = self-assigned address when DHCP server is unreachable.

12.5 VoIP & Mobile Communication Protocols

- **VoIP**: converts voice → digital IP packets (packet-switched) instead of the old PSTN's dedicated circuit per call.
- **SIP (Session Initiation Protocol)**: the "manager" — sets up/rings/tears down the call (signaling).

- **RTP (Real-time Transport Protocol):** the "courier" — carries the actual voice/video media stream.
- **PPP (Point-to-Point Protocol):** Data Link layer; direct link between 2 nodes (dial-up/DSL/PPPoE); supports authentication via **PAP** and **CHAP**; carries multiple network-layer protocols (multiprotocol encapsulation) — unlike Ethernet, which needs no login.
- **Mobile generations:** 1G (analog voice) → **2G/GSM** (digital voice+SMS, uses **TDMA**, introduced the **SIM card**) → **2.5G/GPRS** (packet-switched data bridge to 3G, "always-on," ~114 kbps) → 3G (mobile internet, video calls) → 4G/LTE (all-IP, HD streaming) → 5G (ultra-low latency, massive IoT).
- **WLL (Wireless Local Loop):** replaces the physical "last-mile" copper wire with a fixed wireless radio link to the local exchange — used in rural/hard-to-cable areas.
- **TRAP:** GSM uses **circuit switching** for voice (via TDMA time-slots); GPRS was the shift to **packet switching** for data — don't assume all mobile-gen protocols are packet-switched from the start.

12.6 Web Services (Search, HTML/XML, Hosting)

- **3 pillars of the Web:** URL (addressing) + HTTP (transfer protocol) + HTML (page structure/display).

Feature	HTML	XML
Purpose	Document display/formatting	Data transport/storage (structure & meaning)
Tags	Fixed, predefined (e.g. <code><h1></code> , <code></code>)	Custom, user-defined (e.g. <code><price></code>)
Concerned with	How content looks	What the data means

- **Static website:** same content for every visitor (pure HTML/CSS). **Dynamic website:** content generated per-user via server-side scripting + database (e.g., PHP/Python).
- **Web server:** software (Apache, Nginx) that stores site files and serves them on request. **Web hosting:** the rented service/infrastructure that keeps a web server online.
- Search engine (Google, Bing) ≠ Web browser (Chrome, Safari): browser is the client app; search engine is a website you visit through the browser.

Questions

Q1. [Trap] FTP is best described as using:

- A. A single TCP connection on port 21 for everything B. Two separate connections — control (port 21) and data (port 20) C. UDP on port 21 D. A single UDP connection on port 20

Answer: B. **Why:** FTP uniquely separates command/control traffic (port 21) from the actual file data transfer (port 20).

Q2. Which protocol is used only to send/push email, never to retrieve it?

- A. POP3 B. IMAP4 C. SMTP D. HTTP

Answer: C. **Why:** SMTP is a push protocol for sending mail between clients and servers; retrieval requires a pull protocol like POP3 or IMAP4.

Q3. A user wants to read email from multiple devices with all messages and folders staying synced on the server. Which protocol should they use?

- A. POP3 in delete mode B. POP3 in keep mode C. IMAP4 D. SMTP

Answer: C. **Why:** IMAP4 keeps mail organized in server-side folders and supports multi-device sync; POP3 (especially delete mode) is designed for single-device, download-then-remove access.

Q4. [Trap] The World Wide Web (HTTP) is inherently stateless. Which mechanism allows a server to recognize a returning client across requests?

- A. TCP sequence numbers B. Cookies C. The URL path D. DNS caching

Answer: B. **Why:** Cookies are stored client-side and automatically resent to the originating domain, letting an otherwise stateless server infer session continuity.

Q5. HTTPS differs from HTTP mainly by adding which capability, via SSL/TLS on port 443?

- A. Faster page loading only B. Confidentiality, Integrity, and Authentication C. A second TCP connection like FTP D. Persistent connections (HTTP lacks these)

Answer: B. **Why:** HTTPS wraps HTTP traffic in an encrypted SSL/TLS tunnel, adding the three security properties HTTP itself lacks; both HTTP/1.1 and HTTPS support persistent connections.

Q6. In DNS's hierarchical structure, which server holds the FINAL mapping from a specific domain name to its IP address?

- A. Root server B. TLD server C. Authoritative name server D. Local resolver cache only

Answer: C. **Why:** Root servers redirect to TLD servers (.com, .in, etc.), which in turn point to the Authoritative server that actually stores the definitive name→IP record.

Q7. Telnet has been largely replaced in production use by which more secure protocol?

- A. FTP B. SSH C. HTTP D. SNMP

Answer: B. **Why:** Telnet transmits credentials/data in plaintext; SSH (port 22) provides the same remote-terminal function with encryption.

Q8. [Numerical/Conceptual] In the DHCP DORA process, what is the correct order of messages?

- A. Offer → Discover → Request → Acknowledge B. Discover → Offer → Request → Acknowledge C. Discover → Request → Offer → Acknowledge D. Request → Discover → Acknowledge → Offer

Answer: B. **Why:** Client broadcasts Discover → server replies with an Offer → client sends Request to accept it → server sends Acknowledge to finalize.

Q9. Which IP address indicates "DHCP server unreachable, self-assigned fallback address"?

- A. 0.0.0.0 B. 127.0.0.1 C. 169.254.x.x (APIPA) D. 255.255.255.255

Answer: C. **Why:** APIPA (169.254.x.x) is auto-assigned by Windows when no DHCP server responds; 0.0.0.0 means "no IP yet," and 127.0.0.1 is the loopback address (unrelated to DHCP failure).

Q10. GSM's core multiple-access technology, allowing several users to share one frequency channel by taking turns in time, is:

- A. CDMA B. FDMA C. TDMA D. OFDM

Answer: C. **Why:** GSM (2G) uses TDMA, slicing time into slots so up to 8 users share one radio channel.

Q11. [Trap] GPRS's key innovation over plain GSM was:

- A. Introducing the SIM card B. Switching from circuit-switched voice to packet-switched, "always-on" data C. Introducing video calling D. Introducing digital voice for the first time

Answer: B. **Why:** GSM handled only circuit-switched voice; GPRS ("2.5G") added packet-switched, always-on data — the bridge to 3G.

Q12. The key structural difference between HTML and XML is that:

- A. HTML is for data storage, XML is for display B. HTML uses fixed display tags; XML uses custom, user-defined tags to describe data meaning C. XML cannot be used on the web D. They are functionally identical

Answer: B. **Why:** HTML tags (`<h1>` , `<img>`) control visual presentation; XML tags are custom-defined to describe the structure/meaning of data for exchange between systems.

Q13. Which VoIP protocol is responsible for setting up, ringing, and tearing down a call, as opposed to carrying the actual voice data?

- A. RTP B. SIP C. TCP D. HTTP

Answer: B. **Why:** SIP handles call signaling (setup/teardown); RTP carries the real-time media stream itself.

Q14. PPP's key advantage over a plain Ethernet connection for a dial-up/DSL link is:

A. Higher raw bandwidth B. Built-in authentication (PAP/CHAP) before granting network access C. Use of MAC addresses D. Native encryption of all data

Answer: B. **Why:** PPP requires the far end to authenticate (PAP or the stronger CHAP) before a connection is established — Ethernet has no such built-in login step.

7.13 Networking Hardware Devices

13.1 Layer-by-Layer Device Summary

Device	OSI Layer	Uses which address?	Filters/forwards intelligently?	Collision domain	Broadcast domain
Repeater	Physical (L1)	None	No — regenerates signal only	Not separated	Not separated
Hub	Physical (L1)	None (multiport repeater)	No — broadcasts to all ports	One single domain for all ports	Not separated
Bridge	Data Link (L2)	MAC address	Yes — has a MAC address table	Separates collision domains per segment	NOT separated (broadcasts pass through)
Switch	Data Link (L2)	MAC address	Yes — multiport bridge, per-port MAC table	Each port = own collision domain	NOT separated (unless VLANs used)
Router	Network (L3) (also L1/L2)	IP address	Yes — routing table, no collisions	Separates	Separates broadcast domains
Gateway	Any layer (L1–L7)	Protocol-dependent	Yes — protocol conversion	—	—

13.2 Key Facts

- **Repeater:** regenerates (not merely amplifies) a weakened signal to extend cable distance; blind to frames/addresses.
- **Hub = multiport repeater:** all ports share ONE collision domain (half-duplex); no MAC table, no filtering — data sent to one port is repeated to ALL ports.
- **Bridge:** connects LAN segments of the *same* technology; builds a MAC address table to filter frames onto only the correct segment — reduces collisions, but still forwards broadcasts everywhere.
- **Switch:** essentially a fast, multiport bridge — every port is its own collision domain, but the whole switched LAN is still one broadcast domain (until VLANs subdivide it).
- **Router:** operates using IP addresses; reads the routing table to forward packets between different networks; does routing/filtering/forwarding — **routers separate broadcast domains** (this is the key differentiator from switches).
- **Gateway:** connects networks using **different protocols** (protocol conversion); can operate at any OSI layer; broader role than router — e.g., protocol translation, proxy, NAT, firewall, deep packet inspection.
- **TRAP:** "Bridge/Switch separate collision domains but NOT broadcast domains; only Routers separate broadcast domains" — this is one of the most frequently tested hardware facts.
- **TRAP:** A **Hub** does NOT separate even collision domains (all ports = 1 domain) — weaker than a Bridge/Switch, which separates collision domains per port/segment.

Questions

Q1. Which device operates purely at the Physical layer and has no concept of MAC addresses?

A. Bridge B. Switch C. Hub D. Router

Answer: C. **Why:** A Hub (multiport repeater) only regenerates and repeats electrical signals; it has no MAC address table and floods all ports.

Q2. [Trap] Which of these devices separates collision domains but NOT broadcast domains?

A. Router B. Hub C. Switch D. Repeater

Answer: C. **Why:** A Switch gives each port its own collision domain but still forwards broadcast frames to every port — only a Router separates broadcast domains.

Q3. A device that connects two networks running entirely different protocols (e.g., converting between two different network architectures) is best described as a:

A. Repeater B. Hub C. Gateway D. Bridge

Answer: C. **Why:** Gateways specifically handle protocol conversion between dissimilar networks and can function at any OSI layer, unlike routers/bridges which typically operate within a single protocol family.

Q4. Why is a switch generally preferred over a hub in modern LANs?

A. A switch is cheaper B. A switch gives every port its own collision domain via MAC-address-based forwarding, reducing collisions C. A switch works only at the Physical layer D. A switch cannot operate in full-duplex

Answer: B. **Why:** Unlike a hub's shared collision domain, a switch's per-port forwarding (via its MAC table) means devices on different ports don't collide with each other.

Q5. Which device makes its forwarding decisions using the destination IP address and a routing table?

A. Switch B. Bridge C. Router D. Repeater

Answer: C. **Why:** Routers operate at the Network layer, using IP addresses and routing tables/policies to select the next hop — unlike Bridges/Switches, which use MAC addresses at Layer 2.

7.14 Network Security Fundamentals & Cryptography

14.1 Security Goals (CIA Triad +2) — OSI Security Architecture (ITU-T X.800)

- **Confidentiality** — only authorized users read data. Tools: Encryption, Access Control, VPN, SSL/TLS.
- **Integrity** — data unaltered/detectable if changed. Tools: Hash, MAC, Digital Signature.
- **Availability** — resources accessible when needed. Threats: DoS/DDoS, hardware/power failure. Tools: redundancy, load balancing, disaster recovery.
- **Authentication** ("who are you?") — Peer-entity (session start) vs Data-origin (per-message sender verification).
- **Non-Repudiation** — sender/receiver can't deny an action. **NRO** = proof of origin, **NRR** = proof of receipt.
- **TRAP**: Cryptography alone gives Confidentiality, Integrity, Authentication, Non-Repudiation — **NOT Availability** (needs redundancy/DoS defenses, not crypto).

14.2 Security Attacks

Feature	Passive Attack	Active Attack
Data modified?	No	Yes
Detection	Difficult	Easier
Prevention	Easier (encryption)	Harder
Target	Confidentiality	Integrity & Availability
Types	Release of message contents, Traffic Analysis	Masquerade, Replay, Modification, DoS

- **Traffic Analysis**: even encrypted traffic leaks metadata (who/when/how much) — encryption does NOT stop this.
- **Replay**: resend a captured *valid* message later. **Masquerade**: impersonate using stolen credentials. **Modification**: alter message content in transit.

14.3 Security Services (X.800)

Authentication, Access Control ("what are you allowed to do," comes after authentication), Data Confidentiality, Data Integrity, Non-Repudiation.

Guarantee ladder: Hash Function -> Integrity only
 MAC -> Integrity + Authentication
 Digital Signature -> Integrity + Authentication + Non-Repudiation

14.4 Cryptography Basics

- **Plaintext** → (**Encryption+Key**) → **Ciphertext** → (**Decryption+Key**) → **Plaintext**. **Cipher** = the algorithm (AES, DES, RSA...). **Key** = the secret value.
- **Kerckhoffs's Principle**: system must stay secure even if the algorithm is public — security depends on the **key**, not algorithm secrecy.
- **Cryptanalysis** = breaking a cipher without the key. **Cryptology** = **Cryptography** + **Cryptanalysis**.
- Cryptography gives Confidentiality/Integrity/Authentication/Non-Repudiation — **not Availability**.

14.5 Symmetric vs Asymmetric Cryptography

Feature	Symmetric	Asymmetric
Keys	1 shared secret key	2 (Public + Private)
Speed	Fast	Slow
Key distribution	Hard (must securely share)	Easy (public key can be open)
Main use	Bulk data encryption	Key exchange, Digital Signatures
Examples	DES, 3DES, AES, Blowfish	RSA, ElGamal, ECC, Diffie-Hellman

Number of unique symmetric keys needed for n users (full pairwise) = $n(n-1)/2$

- Symmetric disadvantages: key-distribution problem, poor scalability (formula above), **no non-repudiation** (either party could've created the message since both share the key).
- Asymmetric: a message encrypted with one key (of a pair) can only be decrypted with the *other* key of that same pair. Solves key distribution; enables digital signatures; but slower & unsuitable for large data.
- **Hash function**: fixed-length one-way output, NO key needed for basic integrity check (contrast with MAC which needs a key).

14.6 Encryption Algorithms

- **Block Cipher** (fixed-size blocks, e.g., DES/AES; needs padding) **vs Stream Cipher** (bit/byte-by-byte via XOR with a keystream, no padding, faster for real-time; keystream must NEVER repeat with same key). Examples stream: RC4, Salsa20, ChaCha20.

Feature	DES	3DES	AES
Block size	64 bits	64 bits	128 bits
Key size	56 bits (effective)	112/168 bits	128/192/256 bits
Security	Low (obsolete)	Medium (deprecated)	High (current standard)
Structure	16-round Feistel network	3x DES (Encrypt-Decrypt-Encrypt)	Substitution-Permutation

- **DES**: IBM, 1977 standard; 64-bit blocks, 56-bit key (64-bit key minus 8 parity bits); 16 Feistel rounds; broken by brute force → replaced by AES.
- **AES**: NIST selected 2001; 128-bit blocks; 128/192/256-bit keys; faster AND more secure than DES/3DES; current standard.

- **Modes of operation** (how a block cipher handles data > 1 block): **ECB, CBC, CFB, OFB, CTR**.
- **RSA**: asymmetric; security rests on difficulty of **factoring large primes**; used for encryption, digital signatures, key exchange, SSL/TLS.
- **Diffie-Hellman (DHKE)**: asymmetric — NOT an encryption algorithm, only a **key-exchange** method; shared secret is never transmitted; each side computes the same key independently; vulnerable to **Man-in-the-Middle** unless combined with authentication.

Feature	RSA	Diffie-Hellman
Purpose	Encryption & Digital Signature	Key Exchange only
Encrypts data?	Yes	No
Produces shared secret?	No	Yes
Provides authentication (by itself)?	Yes	No

Questions

Q1. Which security goal is NOT guaranteed by cryptography alone?

A. Confidentiality B. Integrity C. Availability D. Authentication

Answer: C. Why: Availability requires redundancy/anti-DoS infrastructure — encryption cannot stop resource exhaustion attacks.

Q2. An attacker resends a previously captured valid login message to gain access. This is a:

A. Masquerade Attack B. Replay Attack C. Traffic Analysis D. Modification Attack

Answer: B. Why: Reuse of a genuine, previously captured message defines Replay — the credentials/message are real, unlike Masquerade (stolen identity) or Modification (altered content).

Q3. [Numerical] In a symmetric-key system, 10 users need pairwise unique keys. How many keys are required?

A. 10 B. 45 C. 90 D. 100

Answer: B. Why: $n(n-1)/2 = 10 \times 9/2 = 45$.

Q4. Which mechanism provides Integrity + Authentication + Non-Repudiation, unlike a plain hash (integrity only) or MAC (integrity+authentication only)?

A. Symmetric encryption B. Digital Signature C. Simple checksum D. VPN

Answer: B. Why: Digital Signature uses asymmetric crypto (private key signs), so only the owner could have produced it — giving non-repudiation, which a shared-key MAC cannot provide.

Q5. [Trap] Traffic Analysis is classified as passive even though it yields useful intelligence, because:

A. It always uses unencrypted data B. Passive attacks are defined by absence of data modification, not by the value of information gained C. It targets availability D. It modifies packet headers

Answer: B. Why: Passive = no data altered; traffic analysis only observes metadata patterns (timing, size, frequency) without touching content.

Q6. Kerckhoffs's Principle states that a cryptosystem should remain secure even if:

A. The key is publicly known B. The algorithm is publicly known, provided the key stays secret C. Both algorithm and key are public D. No key is used at all

Answer: B. Why: Security must rest on the secrecy of the key alone, not on hiding the algorithm — a core assumption behind modern public cipher design (AES, RSA, etc.).

Q7. DES uses a 64-bit key, but its effective security strength comes from only:

A. 64 bits B. 56 bits C. 48 bits D. 32 bits

Answer: B. Why: 8 of the 64 bits are parity bits, leaving 56 effective key bits — this small size is why DES is now considered insecure (brute-forceable).

Q8. [Trap] Diffie-Hellman Key Exchange is best described as:

A. A symmetric block cipher B. An algorithm to encrypt bulk data C. A method to establish a shared secret key, not an encryption algorithm itself D. A digital signature scheme

Answer: C. Why: DHKE only lets two parties agree on a shared secret over an insecure channel; it does not encrypt data and provides no authentication by itself (vulnerable to MITM without added authentication).

Q9. Which cipher mode/type is preferred for real-time streaming data because it avoids padding and encrypts continuously?

A. Block cipher (ECB mode) B. Stream cipher C. AES with CBC D. RSA

Answer: B. Why: Stream ciphers XOR data with a keystream bit/byte-by-byte, needing no padding and suiting continuous, low-latency transmission — unlike block ciphers, which need full blocks (and possibly padding).

7.15 Hash/MAC/Digital Signature, Authentication & Security Tools

15.1 Hash Functions

- Converts any-length input → **fixed-length** digest; **one-way** (irreversible); Hash = H(M).
- Good hash properties:** fixed-length output, deterministic, one-way, fast computation, **avalanche effect** (tiny input change → completely different hash), **collision resistant**.
- Used for integrity checking (NOT confidentiality — hashing isn't encryption, there's no decryption step).

Algorithm	Output size	Status
MD5	128 bits	Insecure — vulnerable to collision attacks; legacy only
SHA-1	160 bits	Deprecated (collision attacks found)
SHA-256	256 bits	Recommended, widely used
SHA-384 / SHA-512	384 / 512 bits	Stronger variants
SHA-3	Variable	Modern, recommended

- TRAP:** MD5's 128-bit output and known collision vulnerabilities make it unsuitable for security-critical integrity checks today — a very common "which is insecure" question.

15.2 MAC vs HMAC vs Hash

Hash: Hash = H(M) → Integrity only, no key
 MAC: MAC = F(K, M) → Integrity + Authentication (shared secret key K)
 HMAC: Hash function + secret key → stronger, tamper-resistant MAC (commonly built on SHA-256/SHA-3)

Feature	Hash	MAC	HMAC
Uses a key?	No	Yes (shared secret)	Yes (shared secret + hash function)
Provides	Integrity only	Integrity + Authentication	Integrity + Authentication (more tamper-resistant)
Non-repudiation?	No	No (shared key ⇒ either side could forge)	No

15.3 Digital Signature

- Uses **asymmetric** crypto: sign with sender's **Private Key**, verify with sender's **Public Key** (opposite direction of normal encryption-for-confidentiality use of key pairs).
- Working:** Message → Hash → **Encrypt digest with sender's private key** = signature → send (message + signature) → receiver decrypts signature with sender's public key → receiver independently hashes the received message → **compare the two digests**; match ⇒ valid.
- Achieves: **Authentication, Integrity, Non-Repudiation**. Does **NOT by itself provide Confidentiality** (message itself isn't encrypted, only the digest is signed) — TRAP.

15.4 Digital Certificate & PKI

- Digital Certificate:** binds an identity to a Public Key; issued by a **Certificate Authority (CA)**; contains Subject, Public Key, Issuer, Serial Number, Validity Period, CA's own Digital Signature.
- CA:** trusted 3rd party that issues/verifies/manages certificates.
- PKI (Public Key Infrastructure):** the whole framework — CA, Registration Authority (RA), Certificates, Public/Private keys — that makes public-key crypto usable at scale (e.g., HTTPS).

15.5 Authentication Methods & Kerberos

- Password** → **OTP** (one-time, single-use) → **2FA** (exactly 2 factors, e.g., password+OTP) → **MFA** (2 or more factors).
- Kerberos:** network authentication protocol using **secret-key (symmetric) cryptography** + a trusted **KDC (Key Distribution Center)**; provides **mutual authentication**, avoids sending passwords over the network, enables **Single Sign-On (SSO)**. Used in Windows Active Directory.
- TRAP:** Kerberos relies on **symmetric** (secret-key) cryptography via a trusted third party — NOT on public-key/asymmetric crypto or digital certificates like PKI/CA.

15.6 Security Tools

Tool	Function	Blocks traffic?
Firewall	Filters traffic by rules (IP/port/protocol/application) at the network perimeter	Yes (by rule)
IDS	Detects and alerts on suspicious activity	No — passive, monitor-only
IPS	Detects AND automatically blocks malicious traffic in real time	Yes
VPN	Encrypted tunnel over a public network (Remote-Access or Site-to-Site)	N/A (confidentiality, not filtering)
SSL/TLS	Encrypted communication over the Internet (TLS = modern, more secure successor to SSL)	N/A

Firewall types: Packet Filtering (IP/port/protocol) → Stateful Inspection (tracks connection state) → Circuit-Level Gateway (verifies TCP handshake) → Application-Level Gateway/Proxy (inspects HTTP/FTP/SMTP payload) → Next-Gen Firewall (adds Deep Packet Inspection + Intrusion Prevention).

- **TRAP:** **IDS only alerts; it does NOT block traffic.** IPS is the active, blocking counterpart — this IDS-vs-IPS distinction is one of the most frequently tested facts in this topic.

Questions

Q1. Which hash algorithm is now considered insecure due to demonstrated collision attacks?

A. SHA-256 B. SHA-3 C. MD5 D. SHA-512

Answer: C. **Why:** MD5 (128-bit digest) has known practical collision attacks and is no longer recommended for security-critical use.

Q2. [Trap] What is the key difference between a MAC and a plain Hash function?

A. MAC produces a variable-length output B. MAC requires a shared secret key; plain hash needs no key C. Hash is reversible, MAC is not D. There is no difference

Answer: B. **Why:** MAC = $F(K, M)$ uses a secret key for authentication+integrity; a plain hash $H(M)$ needs no key and only verifies integrity.

Q3. A Digital Signature is created by encrypting the message digest with the sender's:

A. Public Key B. Private Key C. Shared secret key D. Session key

Answer: B. **Why:** Signing uses the sender's Private Key; anyone can then verify it using the corresponding Public Key — the reverse role of keys compared to confidentiality-focused asymmetric encryption.

Q4. [Trap] Does a Digital Signature by itself provide Confidentiality?

A. Yes, always B. No — only the digest is encrypted/signed, not the whole message C. Yes, but only for text messages D. No signature scheme provides Integrity either

Answer: B. **Why:** Digital signatures target Authentication, Integrity, and Non-Repudiation; the message body itself is sent as-is unless separately encrypted.

Q5. Which entity issues and digitally signs Digital Certificates to bind identities to public keys?

A. Registration Authority only B. Certificate Authority (CA) C. KDC D. IDS

Answer: B. **Why:** The CA is the trusted third party that issues, signs, and manages digital certificates within a PKI.

Q6. Kerberos achieves secure authentication primarily through:

A. Public-key certificates issued by a CA B. A trusted KDC using symmetric (secret-key) cryptography C. Digital signatures D. Firewalls

Answer: B. **Why:** Kerberos is built on secret-key cryptography and a trusted Key Distribution Center — distinct from PKI/CA-based asymmetric certificate systems.

Q7. [Trap] Which security tool only detects and alerts on suspicious activity but does NOT block it?

A. Firewall B. IPS C. IDS D. VPN

Answer: C. **Why:** IDS = Intrusion *Detection* System — passive monitoring/alerting only; IPS additionally blocks malicious traffic automatically.

Q8. A firewall that inspects application-layer payloads such as HTTP or FTP commands is called a:

A. Packet Filtering Firewall B. Circuit-Level Gateway C. Application-Level Gateway (Proxy Firewall) D. Stateful Inspection Firewall

Answer: C. **Why:** Only Application-Level Gateways (proxy firewalls) inspect payload content at the application layer; the others work at IP/port or connection-state level.

Q9. SSL and TLS are best described as:

A. Firewall types B. Encryption algorithms for disk storage only C. Protocols providing encrypted communication over the Internet, with TLS as the modern successor to SSL D. Authentication factors like OTP

Answer: C. **Why:** SSL/TLS secure data in transit (e.g., HTTPS); TLS supersedes the older, weaker SSL while serving the same purpose.

8.1 SDLC Process Models

8.1.1 Waterfall Model

- Strictly **sequential** phases: Requirements → Design → Implementation → Testing → Deployment → Maintenance — each phase must **fully complete** before the next begins.
- Advantage:** simple, easy to manage/document, works well for well-understood, stable requirements.
- Disadvantage: no working software until late;** very costly to go back and fix a requirements error discovered during testing (the further downstream a defect is caught, the more expensive it is to fix — this cost-of-fixing-late-defects curve is itself a common exam point).
- TRAP:** Pure waterfall has **no feedback loop back to earlier phases** built in by design — real projects using it informally add "reviews," but the textbook model itself is strictly one-directional.

8.1.2 Incremental & Iterative Models

- Incremental Model:** requirements divided into **builds/increments**; each increment adds working functionality, delivered progressively — later increments build on earlier ones.
- Iterative Model:** software developed through repeated cycles (iterations), refining/expanding it each time, often re-visiting earlier phases.
- Both reduce risk versus waterfall by delivering **working software earlier and more often**.

8.1.3 Spiral Model

- Combines iterative development with **explicit risk analysis** at each cycle — each "loop" of the spiral passes through 4 quadrants: **(1) Determine objectives, (2) Identify & resolve risks, (3) Develop & test, (4) Plan next iteration**.
- Best suited for **large, high-risk projects** where risk assessment is critical.
- TRAP:** The Spiral Model's *defining, unique* feature (vs. plain iterative models) is its **explicit, formal risk-analysis quadrant repeated every cycle** — a generic "it's iterative" answer misses the actual distinguishing point tested.

8.1.4 Agile & Scrum

- Agile Manifesto values:** individuals & interactions over processes/tools; working software over comprehensive documentation; customer collaboration over contract negotiation; responding to change over following a plan.
- Scrum roles:** **Product Owner** (defines/prioritizes the backlog — represents stakeholder/customer interests), **Scrum Master** (facilitates the process, removes blockers — NOT a traditional manager giving orders), **Development Team** (builds the increment).
- Sprint:** a fixed, short time-box (commonly 2–4 weeks) producing a potentially shippable increment. **Sprint Backlog:** items selected from the **Product Backlog** for the current sprint.
- Daily Standup/Scrum:** short daily sync meeting — what was done, what's next, any blockers.
- TRAP:** The Scrum Master is a **facilitator/coach**, not a project manager who assigns tasks — a frequently mis-tested role definition.

8.1.5 Prototyping Model

- A working (often incomplete/throwaway) model is built early to help **clarify unclear or evolving requirements** with the customer, then discarded or evolved into the final system.
- Best when requirements are **not well understood upfront** — the opposite scenario from where waterfall thrives.

Questions

Q1. [Trap] A defining characteristic of the pure Waterfall model is:

A. It allows unlimited iteration back to earlier phases B. Each phase must fully complete before the next begins, with no built-in feedback loop to revisit earlier phases C. It is best suited for projects with poorly understood, changing requirements D. It delivers working software incrementally from the very first phase

Answer: B. Why: Waterfall's strict sequential, one-directional phase structure is exactly why late-discovered requirement errors are so costly to fix — there's no formal mechanism to revisit earlier phases.

Q2. The general principle regarding defect cost across SDLC phases is:

A. Defects cost the same to fix regardless of when they're discovered B. Defects discovered later in the lifecycle (e.g., during testing or after deployment) are significantly more costly to fix than those caught early (e.g., during requirements/design) C. Defects are cheapest to fix after deployment D. Only design-phase defects have any cost impact

Answer: B. Why: This escalating cost-of-late-defect-discovery is a foundational software engineering economics principle, motivating early testing/review practices.

Q3. [Trap] The Spiral Model's key distinguishing feature, compared to a generic iterative model, is:

A. It never involves testing until the final iteration B. Explicit, formal risk analysis performed in every cycle, before committing to development C. It has no phases at all D. It requires all requirements to be frozen upfront, like waterfall

Answer: B. Why: While many models are iterative, the Spiral Model specifically formalizes a risk-identification-and-resolution quadrant repeated every loop — this is its defining innovation, aimed at large/high-risk projects.

Q4. In Scrum, the role responsible for defining and prioritizing the Product Backlog is:

A. Scrum Master B. Product Owner C. Development Team D. Project Manager (a role Scrum does not formally define)

Answer: B. Why: The Product Owner represents stakeholder/business interests, deciding what gets built and in what priority order.

Q5. [Trap] The Scrum Master's role is best described as:

A. A traditional manager who assigns daily tasks to the development team B. A facilitator/coach who removes obstacles and ensures the Scrum process is followed, without directing task assignment C. The person solely responsible for writing all code D. Identical to the Product Owner role

Answer: B. Why: Scrum deliberately avoids a command-and-control manager role — the Scrum Master's job is process facilitation and blocker removal, while the team self-organizes its own task assignment.

Q6. A "Sprint" in Scrum refers to:

A. An unlimited-length development phase B. A fixed, short time-boxed period (commonly 2–4 weeks) that produces a potentially shippable product increment C. The final release of the entire product D. A single daily standup meeting

Answer: B. Why: Sprints are Scrum's core iteration unit — fixed-length, ending with a working increment reviewed by stakeholders.

Q7. The Prototyping model is most appropriate when:

A. Requirements are fully understood and stable from the start B. Requirements are unclear or evolving, and a working mock-up will help clarify them with the customer

before full-scale development C. The project has zero budget for early development work D. Only used for maintenance projects, never new development

Answer: B. Why: Prototyping directly targets requirement ambiguity — building something tangible early surfaces misunderstandings far cheaper than discovering them after full implementation.

Q8. The Incremental model differs from a single-shot Waterfall approach by:

A. Delivering the entire system only once, at the very end B. Dividing requirements into builds/increments, each delivering working functionality progressively C. Eliminating the need for a requirements phase entirely D. Requiring formal risk analysis every cycle, like the Spiral model

Answer: B. Why: Incremental development's core idea is progressive delivery of working subsets of functionality, reducing the "big bang, nothing works until the very end" risk of pure waterfall.

Q9. Which of these is one of the four core values explicitly stated in the Agile Manifesto?

A. Comprehensive documentation over working software B. Following a plan over responding to change C. Working software over comprehensive documentation D. Contract negotiation over customer collaboration

Answer: C. Why: This is one of the four explicitly stated Agile Manifesto value pairs — Agile prioritizes delivering working software over exhaustive documentation (while not eliminating documentation entirely).

Q10. A Daily Standup (Daily Scrum) meeting is meant to be:

A. A lengthy, multi-hour planning session held once a month B. A short daily sync where team members share what they did, what's next, and any blockers C. A formal performance review of individual team members D. Held only at the very end of each sprint

Answer: B. Why: The daily standup's purpose is brief, frequent synchronization and blocker visibility — not detailed planning or evaluation, which happen in separate Scrum ceremonies (sprint planning, retrospective).

8.2 Requirement Engineering & Software Metrics

8.2.1 Requirements Engineering Process

- **Requirement Elicitation** (gathering, from interviews/surveys/observation) → **Analysis** (resolving conflicts/ambiguity) → **Specification** (documenting, e.g., in an **SRS** — **Software Requirements Specification**) → **Validation** (confirming with stakeholders it's correct/complete) → **Management** (tracking changes over the project's life).
- **Functional Requirements:** *what* the system should do (specific features/behaviors).
- **Non-Functional Requirements:** *how well* the system performs — quality attributes like performance, reliability, usability, security, scalability, maintainability.
- **TRAP:** A requirement like "the system must respond within 2 seconds" is **non-functional** (a performance quality attribute), not functional — a very commonly mis-tested classification, since it sounds like a concrete/specific requirement.

8.2.2 Verification vs. Validation

	Verification	Validation
Question asked	"Are we building the product right?"	"Are we building the right product?"
Checks against	Specification/design documents (internal consistency)	Actual user needs/requirements (external correctness)
Typical activities	Reviews, walkthroughs, static analysis, inspections	Actual testing against real usage/acceptance criteria

- **TRAP:** This exact pair of questions ("building the product right" vs. "building the right product") is one of the single most frequently tested SE facts across all exams — memorize which phrase maps to which term precisely; they are easy to swap.

8.2.3 Software Size & Complexity Metrics

- **LOC (Lines of Code):** simplest size metric; criticized for being language-dependent and not measuring actual functionality/complexity well.
- **Function Point (FP) Analysis:** measures functionality delivered, based on counting/weighting **External Inputs, External Outputs, External Inquiries, Internal Logical Files, External Interface Files** — language-independent, unlike LOC.
- **Cyclomatic Complexity (McCabe's metric):** $V(G) = E - N + 2P$ (E = edges, N = nodes, P = connected components in the control flow graph; for a single connected program, $P=1$, so $V(G) = E - N + 2$). Equivalently, $V(G)$ = number of decision points (predicates) + 1.
- **Cyclomatic complexity gives the minimum number of independent test paths** needed to achieve full branch/decision coverage of a program — directly usable to plan test-case counts.
- **TRAP:** The formula is **$E - N + 2P$** , not $E - N + P$ or $N - E + 2$ — a common numerical slip; also remember the simpler equivalent shortcut (decision points + 1) for quick calculation without drawing the full flow graph.

Questions

Q1. [Trap] "The system must respond to any query within 2 seconds" is an example of a:

A. Functional requirement B. Non-functional requirement (a performance quality attribute) C. Design constraint, not a requirement at all D. User interface requirement only

Answer: B. Why: Despite sounding concrete and specific, response-time targets describe *how well* the system performs (a quality attribute), which is the definition of non-functional — a classic classification trap.

Q2. [Trap] Which pair of questions correctly matches "Verification" and "Validation" respectively?

A. Verification = "are we building the right product?"; Validation = "are we building the product right?" B. Verification = "are we building the product right?"; Validation = "are we building the right product?" C. Both ask exactly the same question D. Neither relates to correctness at all

Answer: B. Why: Verification checks internal consistency against specifications ("right" as in "correctly, per spec"); validation checks against actual user needs ("right product" as in "the correct product to have built") — this exact phrase-to-term mapping is a top exam trap.

Q3. Function Point Analysis is preferred over LOC as a size metric mainly because it is:

A. Faster to compute automatically from source code B. Language-independent, measuring delivered functionality rather than raw code volume C. Only applicable to object-oriented languages D. A measure of code complexity, not size

Answer: B. Why: LOC varies wildly across languages/styles for the same functionality; FP Analysis instead counts functional elements (inputs, outputs, inquiries, files) independent of implementation language.

Q4. [Numerical] A control flow graph has 10 edges, 8 nodes, and is a single connected component. Its cyclomatic complexity is:

A. 2 B. 4 C. 18 D. 10

Answer: B. Why: $V(G) = E - N + 2P = 10 - 8 + 2(1) = 4$.

Q5. Cyclomatic complexity directly gives:

A. The exact number of bugs in a program B. The minimum number of independent paths needed for full branch/decision coverage in testing C. The number of lines of code D. The compilation time of the program

Answer: B. Why: This is cyclomatic complexity's most practically useful interpretation — it tells testers exactly how many independent test paths are needed for thorough decision-coverage testing.

Q6. A simpler, equivalent way to compute cyclomatic complexity without drawing the full flow graph is:

A. Number of variables + 1 B. Number of decision points (predicates) + 1 C. Number of lines of code divided by 10 D. Number of function calls

Answer: B. Why: Each decision point (if, while, for, case, etc.) adds one independent path; counting them and adding 1 gives the same result as the $E-N+2P$ formula, much faster to apply by inspection.

Q7. In the requirements engineering process, the activity of confirming with stakeholders that the documented requirements are correct and complete is called:

A. Elicitation B. Specification C. Validation D. Management

Answer: C. Why: Requirements validation specifically checks the documented requirements against actual stakeholder intent, distinct from elicitation (gathering) and specification (documenting).

Q8. Which of these is classified as a non-functional requirement?

A. "The system shall allow users to reset their password" B. "The system shall support at least 10,000 concurrent users" (scalability) C. "The system shall generate a monthly sales report" D. "The system shall allow an admin to delete a user account"

Answer: B. Why: Scalability, like performance and reliability, is a quality attribute (how well the system operates under load) rather than a specific feature/behavior — the other three options each describe a discrete function the system performs.

Q9. [Trap] LOC as a size metric is criticized primarily because:

- A. It cannot be measured automatically
- B. It is language-dependent — the same functionality can require vastly different line counts in different languages/coding styles
- C. It always overestimates project cost
- D. It cannot be used at all for procedural languages

Answer: B. Why: Two implementations of identical functionality in, say, assembly vs. a high-level language will have wildly different LOC, making direct comparison or estimation based on LOC alone unreliable across languages/styles.

Q10. An SRS (Software Requirements Specification) document is the primary output of which requirements engineering activity?

- A. Elicitation
- B. Specification
- C. Validation
- D. Management

Answer: B. Why: The Specification activity's deliverable is exactly this formal, documented statement of requirements, which then feeds into subsequent validation and management activities.

8.3 Project Estimation & Scheduling

8.3.1 COCOMO Model (Constructive Cost Model)

- Estimates effort (person-months) and duration based on estimated project **size in KLOC (thousands of lines of code)**.
- Basic COCOMO** classifies projects into 3 modes:

Mode	Project characteristics	Effort formula (E in person-months, a/b constants per mode)
Organic	Small team, familiar/in-house environment, well-understood requirements, relatively small size	$E = 2.4(KLOC)^{1.05}$
Semi-detached	Medium size/complexity, mixed team experience	$E = 3.0(KLOC)^{1.12}$
Embedded	Tight constraints (hardware, regulations, real-time), complex, high-risk	$E = 3.6(KLOC)^{1.20}$

- TRAP:** Both the **coefficient (a)** and, importantly, the **exponent (b)** increase from Organic → Semi-detached → Embedded — the exponent's growth means effort scales **super-linearly** with size, and increasingly so for riskier/more complex project types; a common mistake is assuming only the constant changes.
- Intermediate COCOMO** refines Basic COCOMO by additionally multiplying by an **Effort Adjustment Factor (EAF)**, derived from 15 cost driver attributes (e.g., required reliability, product complexity, personnel capability, use of tools) rated on a scale.
- Detailed COCOMO** further applies the model **phase-by-phase** (separate estimates for design, coding, testing, etc.) rather than treating the whole project as one estimate.

8.3.2 Project Scheduling: PERT & CPM

	PERT (Program Evaluation Review Technique)	CPM (Critical Path Method)
Time estimates	Probabilistic — uses 3 estimates per activity: Optimistic (O), Most likely (M), Pessimistic (P)	Deterministic — single, fixed time estimate per activity
Best suited for	Research/new projects with uncertain durations	Well-understood/repetitive projects (e.g., construction) with known durations

- Expected time (PERT):** $T_e = (O + 4M + P) / 6$ — this weighted-average formula gives 4× weight to the "most likely" estimate.
- Critical Path:** the **longest** path through the project network (in terms of total duration) — determines the **minimum possible project completion time**; any delay on a critical-path activity directly delays the whole project.
- Slack/Float:** the amount an activity can be delayed **without** delaying the overall project — by definition, **every activity on the critical path has zero slack**.
- TRAP:** The critical path is the **longest** path, not the shortest — a very commonly inverted fact (intuition might suggest "critical" = "shortest/fastest," but it's actually the bottleneck-determining longest path).

Questions

Q1. [Numerical] Using Basic COCOMO, an "Organic" mode project estimated at 10 KLOC requires approximately how much effort?

A. 2.4 person-months B. 24 person-months C. Approximately 26.9 person-months ($2.4 \times 10^{1.05}$) D. 240 person-months

Answer: C. **Why:** $E = 2.4 \times (10)^{1.05} \approx 2.4 \times 11.22 \approx 26.9$ person-months — always apply the exponent to KLOC before multiplying by the coefficient.

Q2. [Trap] Moving from Organic to Semi-detached to Embedded COCOMO modes, the effort formula's exponent (b):

A. Stays exactly the same across all three modes; only the coefficient (a) changes B. Increases, meaning effort scales increasingly super-linearly with project size for riskier/more complex modes C. Decreases as project risk increases D. Is irrelevant; only KLOC size matters

Answer: B. **Why:** The exponent grows from 1.05 (Organic) to 1.12 (Semi-detached) to 1.20 (Embedded) — larger, more complex/risky projects see effort grow faster than linearly with size, a detail often missed by assuming only the coefficient changes.

Q3. Intermediate COCOMO improves on Basic COCOMO by:

A. Ignoring project size (KLOC) entirely B. Multiplying the basic effort estimate by an Effort Adjustment Factor derived from 15 cost driver attributes C. Removing the organic/semi-detached/embedded classification D. Only applying to embedded-mode projects

Answer: B. **Why:** Intermediate COCOMO retains the basic size-based formula but refines it using additional qualitative project attributes (reliability requirements, team experience, tool usage, etc.) via the EAF multiplier.

Q4. PERT is generally preferred over CPM for projects that are:

A. Well-understood and repetitive, like standard construction projects B. Research-oriented or novel, with significant uncertainty in individual activity durations C. Guaranteed to have no risk of delay D. Extremely short in duration, under one day

Answer: B. **Why:** PERT's three-point (optimistic/most likely/pessimistic) probabilistic estimation is specifically designed to handle uncertain, hard-to-predict activity durations — CPM instead assumes durations are reliably known/fixed.

Q5. [Numerical] An activity has Optimistic = 4 days, Most Likely = 6 days, Pessimistic = 14 days. Its PERT expected time is:

A. 6 days B. 7 days C. 8 days D. 24 days

Answer: B. **Why:** $T_e = (O + 4M + P)/6 = (4 + 24 + 14)/6 = 42/6 = 7$ days.

Q6. [Trap] The critical path in a project network is defined as:

A. The shortest path from start to finish B. The longest path from start to finish, which determines the minimum overall project completion time C. Any path with zero-cost activities D. The path with the fewest number of activities

Answer: B. **Why:** Despite the name possibly suggesting "fastest," the critical path is the longest-duration path — it's the bottleneck that dictates the earliest the whole project can finish; delaying any activity on it delays the entire project.

Q7. An activity's "slack" or "float" represents:

A. The total duration of the entire project B. How much that activity can be delayed without delaying the overall project completion C. The number of resources assigned to that activity D. The probability the activity will be completed on time

Answer: B. **Why:** Slack quantifies scheduling flexibility for non-critical activities; by definition, critical-path activities have zero slack since any delay there directly delays the project.

Q8. Which COCOMO mode is appropriate for a small, in-house team working with well-understood, familiar requirements?

A. Embedded B. Semi-detached C. Organic D. Detailed

Answer: C. Why: Organic mode specifically describes low-risk, familiar, small-scale project conditions — the lowest coefficient/exponent pair of the three modes, reflecting comparatively lower effort growth.

Q9. [Numerical] A project's critical path has zero slack activities A, B, C with durations 5, 8, and 6 days respectively (sequential). A parallel, non-critical path has total duration 15 days. The project's minimum completion time is:

A. 15 days B. 19 days C. 6 days D. 29 days

Answer: B. Why: The critical path total is $5+8+6 = 19$ days, which exceeds the parallel path's 15 days — the critical (longest) path determines project duration, so it finishes in 19 days regardless of the shorter parallel path.

Q10. Detailed COCOMO, compared to Intermediate COCOMO, additionally:

A. Ignores cost driver attributes entirely B. Applies the effort/cost estimation phase-by-phase (e.g., separately for design, coding, testing) rather than as one whole-project estimate C. Only works for Organic-mode projects D. Removes the need for KLOC estimation

Answer: B. Why: Detailed COCOMO's refinement is granularity — breaking the estimate down across individual development phases for more precise tracking, building on top of Intermediate COCOMO's EAF-adjusted formula.

8.4 Software Design — Cohesion, Coupling & UML Basics

8.4.1 Cohesion (within a module) — want HIGH

Cohesion measures how strongly the elements **inside** one module relate to a single, well-defined purpose. Ranked **worst** → **best**:

1. **Coincidental** — elements grouped arbitrarily, no meaningful relationship (worst).
2. **Logical** — elements grouped because they're logically similar category-wise (e.g., "all input routines"), but perform different specific actions.
3. **Temporal** — elements grouped because they execute at the same **time** (e.g., an "initialization" module doing unrelated setup tasks).
4. **Procedural** — elements grouped because they follow a specific execution **order/sequence**.
5. **Communicational** — elements operate on the **same data**.
6. **Sequential** — output of one element feeds as input to the next.
7. **Functional** — every element contributes to a **single, well-defined task** (best).

- **TRAP: High cohesion is desirable**; students sometimes assume "more elements grouped together = more cohesive," but cohesion is about **relatedness of purpose**, not module size — a large module doing one focused thing (functional cohesion) beats a small module doing several unrelated things (coincidental cohesion).

8.4.2 Coupling (between modules) — want LOW

Coupling measures how strongly two **different** modules depend on each other. Ranked **worst** → **best** (most → least coupled):

1. **Content Coupling** — one module directly modifies/relies on another's **internal** data/code (worst — total loss of independence).
2. **Common Coupling** — modules share **global data**.
3. **Control Coupling** — one module passes a flag/parameter that explicitly **controls the internal logic/decisions** of another.
4. **Stamp Coupling** — modules share a composite data structure (e.g., a whole struct/record), but each only uses **part** of it.
5. **Data Coupling** — modules communicate using only **simple data parameters**, each element directly relevant to the called module's function (best — most independent).

- **TRAP: Low coupling + High cohesion is the universal design goal** — a very frequently tested combined fact; students sometimes reverse it as "high coupling, low cohesion" or mix up which one you want high vs. low.

8.4.3 UML Diagram Types (which one for which purpose)

Diagram	Type	Purpose
Use Case Diagram	Behavioral	Shows actors and the system functions (use cases) they interact with — captures functional requirements from a user perspective
Class Diagram	Structural	Shows classes, their attributes/methods, and relationships (association, aggregation, composition, inheritance) — the most common static structure diagram
Sequence Diagram	Behavioral	Shows time-ordered message exchanges between objects for a specific scenario
Activity Diagram	Behavioral	Shows workflow/control flow — similar to a flowchart, good for business process logic
State Chart (State Machine) Diagram	Behavioral	Shows the states an object can be in and transitions between them, triggered by events

- **TRAP: Aggregation vs. Composition** (both relationships in class diagrams) are a classic mix-up — **Aggregation** = "has-a," weak ownership, the part **can exist independently** of the whole (e.g., a Department has Professors, who exist independently). **Composition** = strong ownership, the part **cannot exist without** the whole (e.g., a House has Rooms — destroy the House, the Rooms cease to exist as part of it).

Questions

Q1. [Trap] Which combination represents the ideal software design goal?

A. High coupling, high cohesion B. Low coupling, low cohesion C. Low coupling, high cohesion D. High coupling, low cohesion

Answer: C. Why: Modules should be internally focused on one purpose (high cohesion) while depending on each other as little as possible (low coupling) — this combination maximizes maintainability and minimizes ripple effects from changes.

Q2. The strongest (best) form of cohesion is:

A. Coincidental B. Temporal C. Procedural D. Functional

Answer: D. Why: Functional cohesion means every element within the module contributes to one single, well-defined task — the gold standard, ranked highest on the cohesion scale.

Q3. The weakest (worst) form of coupling is:

A. Data coupling B. Content coupling C. Stamp coupling D. Control coupling

Answer: B. Why: Content coupling has one module reaching directly into another's internal implementation/data — the tightest, most fragile form of dependency, ranked worst.

Q4. [Trap] Two elements grouped into a module purely because they happen to execute at the same time (e.g., during initialization), despite being functionally unrelated, exhibit:

A. Functional cohesion B. Sequential cohesion C. Temporal cohesion D. Data coupling

Answer: C. Why: Temporal cohesion is specifically defined by "same time of execution" as the only unifying factor — a weak form of cohesion, better than coincidental/logical but well below functional.

Q5. Control coupling occurs when:

A. Modules share only simple, directly relevant data parameters B. One module passes a flag/parameter specifically intended to control the internal decision logic of the called module C. Modules share global variables D. One module directly edits another's internal code

Answer: B. Why: Passing a "mode" or "flag" that dictates what branch of logic the receiving module executes ties the two modules' internal behavior together more tightly than plain data exchange — worse than data/stamp coupling, but not as severe as common/content coupling.

Q6. [Trap] The key distinction between Aggregation and Composition in UML class diagrams is:

A. They are functionally identical; only the notation differs B. In Composition, the "part" cannot exist independently of the "whole" (strong ownership); in Aggregation, the part **CAN** exist independently (weak ownership) C. Aggregation is used only for inheritance relationships D. Composition never involves more than one object

Answer: B. Why: This lifecycle-dependency distinction (does the part survive the whole's destruction?) is exactly what separates the two — aggregation's classic Department–Professor example vs. composition's House–Room example.

Q7. A Use Case Diagram is primarily used to:

A. Show the time-ordered sequence of messages between specific objects B. Capture functional requirements by showing actors and their interactions with system

functions C. Show a class's attributes and methods D. Show the internal states an object can transition through

Answer: B. Why: Use case diagrams are a requirements-capture tool, focused on external actor-to-system-function interactions rather than internal structure or precise timing.

Q8. A Sequence Diagram's defining feature is that it shows:

A. Static class relationships B. Time-ordered message exchanges between objects for a particular scenario C. All possible states of a single object D. The overall project schedule (Gantt-chart style)

Answer: B. Why: Sequence diagrams are explicitly organized along a time axis, showing the order in which specific objects exchange messages during one particular interaction scenario.

Q9. Stamp coupling occurs when:

A. Two modules share global data directly B. Modules pass a composite/structured data item (e.g., a whole record), even though the receiving module only needs part of it C. One module modifies another's source code D. Modules communicate with no data exchange at all

Answer: B. Why: Passing an entire structure when only a subset of its fields is actually needed creates an unnecessary dependency on the structure's full shape — better than control/common/content coupling, but worse than plain data coupling (passing only what's needed).

Q10. A State Chart (State Machine) Diagram is best suited to model:

A. The static attributes of a class B. The various states an object can be in and the events that trigger transitions between them C. The overall project cost estimate D. The database schema of a system

Answer: B. Why: State diagrams focus specifically on an object's lifecycle — its distinct states and the triggering events/conditions for moving between them (e.g., an Order object moving from "Placed" → "Shipped" → "Delivered").

8.5 Software Testing — Levels, Techniques & Coverage

8.5.1 Levels of Testing (bottom-up order)

1. **Unit Testing:** tests the smallest testable component (a single function/class) in isolation — typically done by the developer.
2. **Integration Testing:** tests **interfaces/interactions** between already-unit-tested modules combined together. Common strategies: **Top-down** (test high-level modules first, using **stubs** for not-yet-integrated lower modules), **Bottom-up** (test low-level modules first, using **drivers** to simulate not-yet-integrated higher modules), **Big Bang** (integrate everything at once — hardest to isolate defects).
3. **System Testing:** tests the **complete, integrated system** against the full specified requirements.
4. **Acceptance Testing:** validates the system against **customer/business** needs — typically the final gate before delivery (may be **Alpha** testing, at the developer's site, or **Beta** testing, at real customer sites with real users).
 - **TRAP: Stubs** are dummy versions of **lower-level called modules** (used in top-down integration, where the caller exists but the callee doesn't yet); **Drivers** are dummy **calling routines** (used in bottom-up integration, where the callee exists but the caller doesn't yet) — this stub/driver, top-down/bottom-up pairing is very frequently swapped in exams.

8.5.2 White-Box vs. Black-Box Testing

	White-Box (Structural/Glass-box)	Black-Box (Functional)
Basis	Internal code structure/logic is known and used to design tests	Only external specification/requirements known; internal implementation is a "black box"
Techniques	Statement coverage, branch/decision coverage, condition coverage, path coverage	Equivalence Partitioning, Boundary Value Analysis, Decision Tables
Typically performed by	Developers (unit-level)	Testers/QA (often system/acceptance level)

- **Equivalence Partitioning:** divides input data into partitions where the program is expected to behave the **same way** — test **one representative value per partition** rather than every possible input.
- **Boundary Value Analysis (BVA):** tests values **at and immediately around the edges** of input ranges (min, min+1, max-1, max, and often just outside the valid range) — defects cluster disproportionately at boundaries, making this a very high-value testing technique.
- **TRAP:** BVA specifically targets **off-by-one style boundary defects** — a common exam question gives a valid range (e.g., 1 to 100) and asks which specific test values BVA would select (typically: 0, 1, 2, 99, 100, 101).

8.5.3 Code Coverage Criteria (white-box, strength ordering)

Statement Coverage $\subset$ **Branch/Decision Coverage** $\subset$ **Condition Coverage** $\subset$ **Path Coverage** (each is generally at least as strong a requirement as the previous, catching everything the weaker one catches, plus more):

- **Statement Coverage:** every statement executed at least once.
- **Branch (Decision) Coverage:** every branch (true AND false outcome of every decision) taken at least once — subsumes statement coverage.
- **Condition Coverage:** every individual boolean sub-condition within a compound decision evaluated to both true and false at least once (but doesn't guarantee the overall decision itself takes both outcomes, in rare cases).
- **Path Coverage:** every possible execution path through the program exercised — the strongest, but often infeasible for real programs (paths can be exponential or infinite with loops).
- **TRAP:** 100% statement coverage does **NOT** imply 100% branch coverage — e.g., an `if` with no `else` can have its single statement executed (covering the "true" branch) while the "false"/skip path is never separately exercised as a distinct test case.

Questions

Q1. [Trap] In top-down integration testing, dummy modules used to simulate not-yet-integrated LOWER-level modules are called:

A. Drivers B. Stubs C. Mocks (a term not used in this specific pairing) D. Harnesses

Answer: B. Why: Stubs stand in for callee modules that don't exist yet, allowing testing of the higher-level caller — the classic top-down pairing; drivers serve the opposite (bottom-up) role.

Q2. [Trap] Drivers, as used in bottom-up integration testing, are dummy:

A. Called (lower-level) modules B. Calling (higher-level) routines that simulate not-yet-integrated callers C. Test data generators only D. Compilers used during testing

Answer: B. Why: In bottom-up integration, the low-level modules already exist and are tested first, so drivers simulate the higher-level modules that would normally call them.

Q3. Acceptance testing performed by real customers/users at their own site is called:

A. Alpha testing B. Beta testing C. Unit testing D. Regression testing

Answer: B. Why: Beta testing occurs in the real customer's environment with real users; Alpha testing, by contrast, happens at the developer's own site (often with select internal or invited users).

Q4. Black-box testing is based on:

A. Detailed knowledge of the internal code structure B. Only the external specification/requirements, treating the implementation as unknown/opaque C. Cyclomatic complexity calculations D. Code coverage percentages exclusively

Answer: B. Why: Black-box testing deliberately ignores internal implementation, designing tests purely from what the system is specified to do — this is what enables techniques like equivalence partitioning and BVA, usable even without seeing the source code.

Q5. [Numerical] For a valid input range of 1 to 100, Boundary Value Analysis would typically test which set of values?

A. Only 1 and 100 B. 50 (the midpoint) only C. 0, 1, 2, 99, 100, 101 (just below, at, and just above each boundary) D. Every integer from 1 to 100

Answer: C. Why: BVA deliberately targets the edges and their immediate neighbors on both sides, since off-by-one defects concentrate exactly there — far more efficient than exhaustive testing while still catching the highest-risk defects.

Q6. Equivalence Partitioning reduces testing effort by:

A. Testing every single possible input value B. Testing only one representative value from each partition of inputs expected to be handled identically C. Only testing boundary values D. Skipping testing entirely for well-understood modules

Answer: B. Why: If the program is expected to behave the same way for a whole range/category of inputs, testing one representative from that partition is (in principle) as informative as testing them all, dramatically cutting the required test case count.

Q7. [Trap] 100% statement coverage in a test suite guarantees:

A. 100% branch coverage as well, automatically B. Every statement was executed at least once, but does NOT guarantee every branch/decision outcome was separately exercised C. Zero remaining defects in the program D. 100% path coverage as well

Answer: B. Why: An if-statement with no else can achieve full statement coverage by only ever taking the "true" path — the implicit "false"/skip path is a branch that statement coverage alone doesn't require testing.

Q8. The strength ordering (weakest to strongest, in terms of what each subsumes) of these coverage criteria is:

A. Path Coverage $\subset$ Condition Coverage $\subset$ Branch Coverage $\subset$ Statement Coverage B. Statement Coverage $\subset$ Branch Coverage $\subset$ Condition Coverage $\subset$ Path Coverage C. All four are exactly equally strong D. Condition Coverage $\subset$ Statement Coverage $\subset$ Path Coverage $\subset$ Branch Coverage

Answer: B. Why: Each successive criterion in this order generally demands strictly more test scenarios than the previous, with Path Coverage being the strongest (and often practically infeasible) criterion.

Q9. Integration testing specifically focuses on verifying:

A. A single function's internal logic in isolation B. The interfaces and interactions between already-unit-tested modules when combined C. The system's compliance with a legal contract D. Only the user interface's visual appearance

Answer: B. Why: By the time integration testing occurs, individual units are assumed already tested; the new risk being tested is specifically how correctly those units communicate/interact once combined.

Q10. [Trap] "Big Bang" integration testing refers to:

A. Testing modules one at a time in a carefully staged top-down order B. Integrating and testing all modules simultaneously, all at once, making it hardest to isolate the source of any defect found C. A performance/load testing technique only D. A synonym for acceptance testing

Answer: B. Why: Unlike the incremental top-down/bottom-up strategies (which integrate gradually, making defect isolation easier), Big Bang combines everything at once — any resulting failure is much harder to trace to its specific source module.

8.6 Software Quality, Maintenance & Risk Management

8.6.1 Software Maintenance Types

Type	Trigger	Example
Corrective	Fixing an actual defect/bug found after deployment	Fixing a crash on a specific input
Adaptive	Adjusting to a changed environment (new OS, new hardware, new regulation)	Updating an app for a new OS version
Perfective	Improving performance or maintainability , or adding minor enhancements, without fixing a defect or reacting to environment change	Refactoring code, improving response time
Preventive	Proactively fixing latent issues before they cause problems	Restructuring code to reduce future defect risk

- **TRAP:** Perfective maintenance is typically the **LARGEST** category by effort/cost in most real maintenance data (often cited as the majority of total maintenance effort) — a common wrong assumption is that bug-fixing (corrective) dominates maintenance cost; enhancement/improvement work (perfective) usually does.

8.6.2 Software Quality Models

- **ISO/IEC 9126 / ISO 25010 quality characteristics** (know the categories, not just "quality is good"): **Functionality, Reliability, Usability, Efficiency, Maintainability, Portability** (classic six from ISO 9126).
- **CMM / CMMI (Capability Maturity Model):** a 5-level framework for assessing/improving an organization's software process maturity:
 1. **Initial** — ad hoc, chaotic, unpredictable processes.
 2. **Repeatable** — basic project management processes established; past successes on similar projects can be repeated.
 3. **Defined** — processes documented, standardized, and integrated into a organization-wide standard process.
 4. **Managed** — process and product quality **quantitatively measured** and controlled.
 5. **Optimizing** — continuous process improvement, using quantitative feedback, is institutionalized.
- **TRAP:** CMM levels are strictly **sequential and cumulative** (you cannot skip a level) — a common numerical/ordering trap asks "what distinguishes level 4 from level 3," with the correct distinguishing answer being **quantitative measurement** (level 3 is merely "defined/standardized," not yet measured quantitatively).

8.6.3 Risk Management

- **Risk Identification → Risk Analysis** (assess probability × impact) → **Risk Prioritization → Risk Mitigation/Response Planning → Risk Monitoring** (ongoing, throughout the project).
- **Risk Exposure = Probability of the risk occurring × Impact/loss if it does occur** — a simple but frequently-numerically-tested formula for prioritizing which risks to address first.
- **Risk categories:** Project risks (schedule/budget/resource), Technical risks (design/implementation feasibility), Business risks (market/organizational).
- **TRAP: Risk Exposure is a product (multiplication), not a sum** — probability(0 to 1) × impact(some cost/loss unit) gives an expected-loss style metric used to rank risks by priority.

Questions

Q1. [Trap] Which type of software maintenance typically consumes the **LARGEST** share of total maintenance effort in most real-world data?

A. Corrective B. Adaptive C. Perfective D. Preventive

Answer: C. Why: Contrary to the common assumption that bug-fixing dominates, enhancement/improvement work (perfective maintenance) is usually the biggest single category of ongoing maintenance effort in practice.

Q2. Adaptive maintenance is performed specifically in response to:

A. A defect discovered in the deployed software B. A change in the external environment (new OS, hardware, regulations) that the software must now accommodate C. A desire to improve code readability with no functional change D. Proactive restructuring to reduce future defect risk

Answer: B. Why: Adaptive maintenance is environment-driven — the software itself may have no bugs, but the world around it (platform, regulation) has shifted and the software must adapt.

Q3. [Trap] Which CMM (Capability Maturity Model) level is specifically distinguished by processes being quantitatively measured and controlled?

A. Level 2 (Repeatable) B. Level 3 (Defined) C. Level 4 (Managed) D. Level 5 (Optimizing)

Answer: C. Why: Level 3 only standardizes/documents the process organization-wide; Level 4 adds the critical step of quantitative measurement and statistical control of process and product quality — a fine but frequently tested distinction.

Q4. The five CMM maturity levels, in order from least to most mature, are:

A. Initial, Defined, Repeatable, Managed, Optimizing B. Initial, Repeatable, Defined, Managed, Optimizing C. Repeatable, Initial, Managed, Defined, Optimizing D. Optimizing, Managed, Defined, Repeatable, Initial

Answer: B. Why: This precise sequential order (Initial → Repeatable → Defined → Managed → Optimizing) is a direct-recall fact, and levels cannot be skipped when an organization progresses.

Q5. Which of these is one of the six classic ISO/IEC 9126 software quality characteristics?

A. Profitability B. Maintainability C. Marketability D. Scalability (not one of the original six)

Answer: B. Why: The classic ISO 9126 set is Functionality, Reliability, Usability, Efficiency, Maintainability, and Portability — Maintainability is directly one of these six.

Q6. [Numerical] A risk has a 30% probability of occurring and would cause an estimated loss of ₹200,000 if it does. Its risk exposure is:

A. ₹200,000 B. ₹60,000 C. ₹30,000 D. ₹230,000

Answer: B. Why: Risk Exposure = Probability × Impact = 0.30 × ₹200,000 = ₹60,000 — a multiplicative, not additive, combination.

Q7. Preventive maintenance is characterized by:

A. Fixing a bug already reported by users B. Proactively addressing latent issues before they manifest as actual problems C. Reacting to a change in the operating system D. Adding a brand-new feature requested by a customer

Answer: B. Why: Preventive maintenance is forward-looking — restructuring or hardening code specifically to reduce the likelihood of future defects, rather than reacting to an already-occurred trigger.

Q8. The standard risk management process order is:

A. Mitigation → Identification → Analysis → Monitoring B. Identification → Analysis → Prioritization → Mitigation planning → Monitoring C. Monitoring → Identification → Mitigation → Analysis D. Analysis → Identification → Monitoring → Mitigation

Answer: B. Why: You must first identify risks, then analyze their probability/impact, prioritize which matter most, plan responses, and continuously monitor — skipping or reordering these steps undermines effective risk management.

Q9. A "Business risk" in software risk categorization refers to a risk related to:

A. Whether a proposed technical design is even feasible to implement B. Market/organizational factors, such as whether the product will actually be wanted or profitable once built C. Whether the schedule will be met D. Compiler or hardware compatibility issues

Answer: B. Why: Business risk concerns the product's viability in the market/organization (e.g., "will anyone want this once we build it"), distinct from technical risk (can we even build it) and project risk (schedule/budget/resource concerns).

Q10. [Trap] Corrective maintenance is triggered specifically by:

A. A change in government regulation B. An actual defect/bug discovered in the software after deployment C. A desire to improve performance with no bug present D. A proactive code restructuring effort

Answer: B. Why: Corrective maintenance is strictly reactive to a confirmed defect — this is the "bug fixing" category, distinct from adaptive (environment change), perfective (enhancement), and preventive (proactive) maintenance.

9. Part A — Quantitative Aptitude

Part B of the real ISRO paper (Aptitude/Ability) is 15 questions worth 20 marks — some carry 2 marks each (data interpretation, longer numericals), so this section is worth more per question than most of Part A. Quantitative Aptitude chapters below are pure arithmetic/number-crunching: master the fast mental-math shortcuts (fraction–percentage table, osculators, unit-digit tricks) — under exam time pressure these save the seconds that let you also attempt the harder reasoning and DI questions.

9.A1 Number System

A1.1 Classification of Numbers [Level: Basic]

- **Natural numbers (N):** 1, 2, 3, ... **Whole numbers (W):** 0, 1, 2, 3, ... **Integers (Z):** ..., -2, -1, 0, 1, 2, ...
- **Rational numbers:** expressible as p/q ($q \neq 0$); includes terminating and repeating decimals. **Irrational numbers:** non-terminating, non-repeating decimals ($\sqrt{2}$, π).
- **Prime number:** exactly 2 factors (1 and itself). **2 is the only even prime** — a classic recall trap.
- **Composite number:** more than 2 factors. **1 is neither prime nor composite.**
- **Co-prime (relatively prime) numbers:** pair with HCF = 1 (need not themselves be prime, e.g., 8 and 15 are co-prime).

A1.2 Factors, Multiples & Prime Factorization [Level: Basic]

- **Factor** of N: a number that divides N exactly (remainder 0). **Multiple** of N: $N \times k$ for integer k.
- **Prime factorization:** express N as a product of primes, e.g., $360 = 2^3 \times 3^2 \times 5^1$.

If $N = p_1^{a_1} \times p_2^{a_2} \times \dots \times p_k^{a_k}$ (prime factorization), then:

$$\begin{aligned} \text{Number of factors of } N &= (a_1+1)(a_2+1)\dots(a_k+1) \\ \text{Sum of factors of } N &= [(p_1^{a_1+1} - 1)/(p_1 - 1)] \times [(p_2^{a_2+1} - 1)/(p_2 - 1)] \times \dots \end{aligned}$$

- Worked example: $N = 360 = 2^3 \times 3^2 \times 5^1$. Number of factors = $(3+1)(2+1)(1+1) = 4 \times 3 \times 2 = \mathbf{24}$.
Sum of factors = $[(2^4-1)/(2-1)] \times [(3^3-1)/(3-1)] \times [(5^2-1)/(5-1)] = 15 \times 13 \times 6 = \mathbf{1170}$.

A1.3 Odd & Even Factors of N [Level: Basic-Medium]

- **TRICK:** To count only the **odd** factors of N, first strip out ALL powers of 2 from the factorization, then apply the factor-count formula to what remains.
- **Even factors = Total factors – Odd factors.**
- Worked example: $N = 360 = 2^3 \times 3^2 \times 5$. Odd part = $3^2 \times 5^1 \rightarrow$ odd factors = $(2+1)(1+1) = \mathbf{6}$. Even factors = $24 - 6 = \mathbf{18}$.
- **TRAP:** Students often try to directly formula-ize even factors instead of using the subtraction shortcut — the subtraction method is always faster and less error-prone.

A1.4 Factors of N Divisible by a Given Number Q [Level: Medium]

- **Ways to solve:**
 1. **Direct method:** List/verify manually for small N (slow, error-prone for exams).
 2. **Shortcut method:** If Q divides N, write $N = Q \times M$. Every factor of N divisible by Q has the form $Q \times (\text{factor of } M)$. So the count = number of factors of $M = N/Q$'s factorization factor-count.
- Worked example: Find factors of 360 divisible by 12. $M = 360/12 = 30 = 2 \times 3 \times 5$. Factors of M = $(1+1)(1+1)(1+1) = \mathbf{8}$. So 8 factors of 360 are divisible by 12.

A1.5 Concept of Remainders [Level: Medium]

$$\begin{aligned} \text{Remainder}(a + b, n) &= [\text{Remainder}(a, n) + \text{Remainder}(b, n)] \bmod n \\ \text{Remainder}(a \times b, n) &= [\text{Remainder}(a, n) \times \text{Remainder}(b, n)] \bmod n \\ \text{Remainder}(a^b, n) &= [\text{Remainder}(a, n)]^b \bmod n \quad (\text{reduce base first, then apply cyclicity/mod rules}) \end{aligned}$$

- **TRICK:** When dealing with a big product/power's remainder, ALWAYS reduce each individual number to its remainder mod n FIRST, then multiply/exponentiate — never compute the massive product first.
- **Negative remainder trick:** if a direct remainder calculation gives a value close to n, check whether expressing it as (remainder – n) [i.e., a negative/complementary remainder] simplifies a later multiplication — e.g., remainder of $49 \bmod 50 = 49 \equiv -1 \pmod{50}$ is far easier to raise to a power than 49.

A1.6 Successive Division — Short Trick [Level: Medium-Hard]

- **Successive division:** a number is divided by d_1 , the QUOTIENT of that division is divided by d_2 , that quotient by d_3 , and so on.
- **Reverse (short-trick) construction:** To build the SMALLEST number giving a chain of successive remainders r_1, r_2, r_3 for divisors d_1, d_2, d_3 : work backward — start from the last remainder, multiply up and add the next remainder at each stage.

Working backward: $x_2 = (d_3 \times r\text{-chain-so-far}) + r_2 \dots$ continue until the original dividend is rebuilt.

- Worked example: A number, when divided by 5, gives quotient Q_1 and remainder 3. Q_1 divided by 4 gives quotient Q_2 and remainder 2. Q_2 divided by 3 gives quotient 1 and remainder 0.
Build backward: $Q_2 = 3 \times 1 + 0 = 3$. $Q_1 = 4 \times 3 + 2 = 14$. Number = $5 \times 14 + 3 = \mathbf{73}$.

A1.7 Unit Digit / Last Digit of an Expression [Level: Medium-Hard]

- **Cyclicity of unit digits** — how many distinct unit digits a power of each base digit cycles through:

Cyclicity 1 (always same digit): 0, 1, 5, 6
Cyclicity 2 (2 alternating digits): 4 (4,6,4,6...), 9 (9,1,9,1...)
Cyclicity 4 (full 4-cycle): 2 (2,4,8,6), 3 (3,9,7,1), 7 (7,9,3,1), 8 (8,4,2,6)

- **Ways to solve a^b unit digit:**
 1. **Full-cycle method:** find unit digit's cyclicity length L, compute effective exponent = $b \bmod L$ (if remainder is 0, use L instead of 0), then read off that position in the cycle.
 2. **Shortcut for base ending in 4 or 9 only** (cyclicity 2): unit digit = (base's last digit)ⁿ where the check is simply whether the exponent is odd or even — $4^{\text{odd}} \rightarrow 4$, $4^{\text{even}} \rightarrow 6$; $9^{\text{odd}} \rightarrow 9$, $9^{\text{even}} \rightarrow 1$.
- Worked example: unit digit of 7^{94} . Cyclicity of 7 is 4 (7,9,3,1). $94 \bmod 4 = 2 \rightarrow$ 2nd position in the cycle (7,9,3,1) $\rightarrow$ unit digit = **9**.
- **TRAP:** for a SUM/product of several terms, find the unit digit of EACH term first via cyclicity, THEN add/multiply those unit digits together and take the unit digit of that result — do not average or misapply cyclicity to the combined expression directly.

A1.8 Perfect Squares & Perfect Cubes [Level: Hard]

- **Properties of perfect squares:** never end in 2, 3, 7, or 8 (only possible unit digits: 0,1,4,5,6,9). A perfect square has an **odd number of total factors** (TRAP: this is the reverse-recognizable test — if a number has an odd factor count, it MUST be a perfect square, since factors pair up (d, N/d) except when $d=\sqrt{N}$).
- **Finding factors of N that are perfect squares:** in the prime factorization of N, a factor is a perfect square only if EVERY prime's exponent used is even (0, 2, 4, ... up to that prime's max exponent in N). Count = product of $(\text{floor}(a_i/2)+1)$ over all primes.
- **Finding factors of N that are perfect cubes:** analogous — every exponent used must be a multiple of 3. Count = product of $(\text{floor}(a_i/3)+1)$.
- Worked example: $N = 2^4 \times 3^6$. Perfect-square factors: exponents of 2 usable = {0,2,4} → 3 choices; exponents of 3 usable = {0,2,4,6} → 4 choices → **12** perfect-square factors. Perfect-cube factors: 2's usable exponents {0,3} → 2 choices; 3's usable {0,3,6} → 3 choices → **6** perfect-cube factors.

Questions

Q1. [Easy] How many factors does 72 have?

A. 10 B. 12 C. 14 D. 16

Answer: B. Why: $72 = 2^3 \times 3^2$. Factor count = $(3+1)(2+1) = 12$.

Q2. [Easy] What is the unit digit of 3^{45} ?

A. 1 B. 3 C. 7 D. 9

Answer: B. Why: Cyclicity of 3 = 4 (3,9,7,1). $45 \bmod 4 = 1 \rightarrow$ 1st position = **3**.

Q3. [Medium] How many odd factors does 240 have?

A. 2 B. 4 C. 6 D. 8

Answer: B. Why: $240 = 2^4 \times 3 \times 5$. Odd part = $3 \times 5 \rightarrow$ factor count = $(1+1)(1+1) = 4$.

Q4. [Medium] [Trap] How many EVEN factors does 240 have?

A. 16 B. 18 C. 20 D. 12

Answer: A. Why: Total factors of 240 = $(4+1)(1+1)(1+1) = 20$. Odd factors = 4 (from Q3). Even = $20 - 4 = 16$.

Q5. [Medium] Find the number of factors of 900 that are divisible by 15.

A. 6 B. 9 C. 12 D. 15

Answer: C. Why: $900 = 2^2 \times 3^2 \times 5^2$. $M = 900/15 = 60 = 2^2 \times 3 \times 5$. Factor count of M = $(2+1)(1+1)(1+1) = 12$.

Q6. [Medium] [Trick] Find the unit digit of $17^{23} \times 13^{15}$.

A. 1 B. 3 C. 7 D. 9

Answer: B. Why: Unit digit of $17^{23} \rightarrow$ base unit 7, cyclicity 4 (7,9,3,1); $23 \bmod 4 = 3 \rightarrow$ **3**. Unit digit of $13^{15} \rightarrow$ base unit 3, cyclicity 4 (3,9,7,1); $15 \bmod 4 = 3 \rightarrow$ **7**. Product unit digit = unit digit of $3 \times 7 = 21 \rightarrow$ **1**. **Answer: A (1).**

Q7. [Hard] A number when divided successively by 4 and then 5 gives remainders 2 and 3 respectively, with a final quotient of 6. Find the number.

A. 126 B. 134 C. 146 D. 152

Answer: B. Why: Working backward: intermediate quotient = $5 \times 6 + 3 = 33$. Number = $4 \times 33 + 2 = 134$.

Q8. [Hard] How many perfect-square factors does $2^6 \times 3^4$ have?

A. 8 B. 10 C. 12 D. 15

Answer: C. Why: Usable exponents of 2: {0,2,4,6} → 4 choices. Usable exponents of 3: {0,2,4} → 3 choices. Total = $4 \times 3 = 12$.

Q9. [Hard] Find the smallest number which, when divided by 6, 15, and 20, always leaves remainder 5.

A. 55 B. 60 C. 65 D. 125

Answer: C. Why: $\text{LCM}(6,15,20) = 60$. Required number = $\text{LCM} + \text{remainder} = 60 + 5 = 65$.

Q10. [Hard] [Trap] A number has exactly 9 factors. What can definitely be said about it?

A. It is prime B. It is a perfect cube C. It is a perfect square D. It is even

Answer: C. Why: An odd total factor count always indicates a perfect square (factors pair up as (d, N/d) except the unpaired $\sqrt{N}$) — 9 is odd, so the number MUST be a perfect square (e.g., $36 = 2^2 \times 3^2$ has $(2+1)(2+1) = 9$ factors).

9.A2 HCF & LCM

A2.1 Definitions [Level: Basic]

- **HCF (Highest Common Factor / GCD):** the largest number that divides all given numbers exactly.
- **LCM (Lowest Common Multiple):** the smallest number that is divisible by all given numbers exactly.
- **Co-prime numbers:** HCF = 1 (need not be individually prime, e.g., 8 & 15).

A2.2 Ways to Find HCF [Level: Basic-Medium]

1. **By inspection:** for small numbers, list factors and pick the largest common one.
2. **By prime factorization:** HCF = product of common primes, each raised to the **lowest** power appearing in any of the numbers.
3. **By long division (Euclidean algorithm) — fastest for large numbers:** divide the larger number by the smaller; replace the larger with the smaller and the smaller with the remainder; repeat until remainder = 0. The last non-zero divisor is the HCF.

Euclid's algorithm example: HCF(1071, 462)

$$\begin{aligned} 1071 &= 462 \times 2 + 147 \\ 462 &= 147 \times 3 + 21 \\ 147 &= 21 \times 7 + 0 \end{aligned}$$

<- remainder 0, so HCF = 21

A2.3 Ways to Find LCM [Level: Basic-Medium]

1. **By inspection** (small numbers).
2. **By prime factorization:** LCM = product of ALL primes appearing, each raised to the **highest** power appearing in any of the numbers.
3. **By division method:** divide all numbers together by common prime factors repeatedly; LCM = product of all divisors used and the final leftover quotients.

A2.4 The HCF x LCM Property [Level: Medium]

For exactly 2 numbers: $\text{HCF}(a,b) \times \text{LCM}(a,b) = a \times b$

- **TRAP:** This property holds **only for exactly 2 numbers** — it does NOT generalize to 3 or more numbers (i.e., $\text{HCF}(a,b,c) \times \text{LCM}(a,b,c) \neq a \times b \times c$ in general). A very common exam mistake.
- **TRICK:** If two numbers are given to be co-prime, HCF = 1 automatically, so LCM = the plain product — skip all factorization.

A2.5 HCF & LCM of Fractions [Level: Medium]

HCF of fractions = $\text{HCF}(\text{numerators}) / \text{LCM}(\text{denominators})$
 LCM of fractions = $\text{LCM}(\text{numerators}) / \text{HCF}(\text{denominators})$

- **TRAP:** Students very commonly swap these two — remember "HCF needs LCM of denominators" and "LCM needs HCF of denominators" (each formula uses the OPPOSITE operation on the denominator).
- Worked example: Find HCF and LCM of $2/3, 8/9, 10/27$.
 Numerators: 2,8,10 $\rightarrow$ HCF=2, LCM=40. Denominators: 3,9,27 $\rightarrow$ HCF=3, LCM=27.
 HCF of fractions = $2/27$. LCM of fractions = $40/3$.

A2.6 HCF & LCM of Decimals [Level: Medium]

- **TRICK:** Make the number of decimal places equal in all numbers (padding with trailing zeros), remove the decimal point, find HCF/LCM as integers, then reinsert the decimal point at the same position in the answer.
- Worked example: HCF(1.2, 0.48). Equalize: 1.20 and 0.48 $\rightarrow$ treat as 120 and 48. $\text{HCF}(120,48)=24 \rightarrow$ reinsert 2 decimal places $\rightarrow$ **0.24**.

A2.7 HCF/LCM of More Than 2 Numbers [Level: Medium]

- Find HCF/LCM of the first two, then combine that result with the third number, and so on (pairwise reduction) — works for both prime-factorization and division methods.

A2.8 Remainder-Based Word Problems (High-Yield Patterns) [Level: Hard]

Pattern 1: Greatest number that divides x, y, z leaving the SAME remainder r in each case
 $\Rightarrow \text{Answer} = \text{HCF}(x-r, y-r, z-r)$

Pattern 2: Smallest number which, when divided by a, b, c leaves the SAME remainder r each time
 $\Rightarrow \text{Answer} = \text{LCM}(a,b,c) + r$

Pattern 3: Greatest number that divides x, y leaving DIFFERENT remainders r1, r2
 $\Rightarrow \text{Answer} = \text{HCF}(x-r1, y-r2)$

Pattern 4: Smallest number which, when divided by a,b,c leaves remainders that are each a fixed amount 'k' LESS than the divisor (i.e., remainder = divisor - k for every divisor)
 $\Rightarrow \text{Answer} = \text{LCM}(a,b,c) - k$

- TRICK to recognize Pattern 4 vs Pattern 2: check if (divisor – remainder) is the SAME constant k across all given divisor/remainder pairs — if so, use LCM – k instead of the standard LCM + r form.
- Worked example (Pattern 1): Find the greatest number that divides 62, 132, and 237 leaving remainder 2 in each case.
 $\text{HCF}(62-2, 132-2, 237-2) = \text{HCF}(60, 130, 235)$. $\text{HCF}(60,130)=10$; $\text{HCF}(10,235)=5$. **Answer: 5.**

Questions

Q1. [Easy] Find the HCF of 24 and 36.

A. 6 B. 8 C. 12 D. 18

Answer: C. Why: $24=2^3 \times 3$, $36=2^2 \times 3^2$. Common primes at lowest power: $2^2 \times 3=12$.

Q2. [Easy] Find the LCM of 12 and 18.

A. 24 B. 36 C. 48 D. 72

Answer: B. Why: $12=2^2 \times 3$, $18=2 \times 3^2$. $\text{LCM} = 2^2 \times 3^2 = 36$.

Q3. [Medium] The product of two numbers is 4032 and their HCF is 12. Find their LCM.

A. 288 B. 336 C. 360 D. 432

Answer: B. Why: $\text{HCF} \times \text{LCM} = \text{product}$ (2-number rule). $\text{LCM} = 4032/12 = 336$.

Q4. [Medium] [Trap] HCF and LCM of three numbers are 4 and 720 respectively. If two of the numbers are 24 and 48, is the product of all three necessarily $\text{HCF} \times \text{LCM}$?

A. Yes, always B. No — the $\text{HCF} \times \text{LCM} = \text{product}$ rule applies only to exactly 2 numbers C. Only if all three are co-prime D. Only if $\text{HCF}=1$

Answer: B. Why: The identity $\text{HCF} \times \text{LCM} = \text{product}$ strictly holds for 2 numbers only; for 3+ numbers there is no such direct product relationship.

Q5. [Medium] Find the HCF of $3/4$, $9/10$, and $15/16$.

A. $3/80$ B. $1/80$ C. $3/160$ D. $3/40$

Answer: A. Why: $\text{HCF}(\text{numerators } 3, 9, 15)=3$. $\text{LCM}(\text{denominators } 4, 10, 16)=80$. $\text{HCF of fractions} = 3/80$.

Q6. [Hard] Find the greatest number that divides 285 and 1249, leaving remainders 9 and 7 respectively.

A. 69 B. 92 C. 138 D. 276

Answer: C. Why: $\text{HCF}(285-9, 1249-7) = \text{HCF}(276, 1242)$. $1242 = 276 \times 4 + 138$; $276 = 138 \times 2 + 0 \rightarrow \text{HCF} = 138$.

Q7. [Hard] Find the smallest number which when divided by 6, 15, and 18 leaves remainder 5 in each case.

A. 90 B. 91 C. 95 D. 85

Answer: C. Why: $\text{LCM}(6, 15, 18)=90$. $\text{Answer} = \text{LCM} + r = 90+5 = 95$.

Q8. [Hard] Find the largest number which divides 70 and 125, leaving remainders 5 and 8 respectively.

A. 13 B. 15 C. 65 D. 117

Answer: A. Why: $\text{HCF}(70-5, 125-8) = \text{HCF}(65, 117)$. $117=65 \times 1+52$; $65=52 \times 1+13$; $52=13 \times 4+0 \rightarrow \text{HCF}=13$.

9.A3 Divisibility Rules

A3.1 Rules for 2 through 10 [Level: Basic]

Divisor	Rule
2	Last digit is even (0,2,4,6,8)
3	Sum of all digits divisible by 3
4	Last 2 digits (as a number) divisible by 4
5	Last digit is 0 or 5
6	Divisible by both 2 AND 3
7	See osculator method below (A3.3)
8	Last 3 digits (as a number) divisible by 8
9	Sum of all digits divisible by 9
10	Last digit is 0

A3.2 Rules for 11 through 20 [Level: Basic-Medium]

Divisor	Rule
11	(Sum of digits at odd places) – (Sum of digits at even places) = 0 or a multiple of 11
12	Divisible by both 3 AND 4
13	See osculator method below (A3.3)
14	Divisible by both 2 AND 7
15	Divisible by both 3 AND 5
16	Last 4 digits divisible by 16
17, 19	See osculator method (A3.3)
18	Divisible by both 2 AND 9
20	Last 2 digits divisible by 20 (equivalently, ends in 00,20,40,60,80)

A3.3 The Osculator Method — For 7, 13, 17, 19, and other "awkward" primes [Level: Medium-Hard]

- Every prime not dividing 10 has an **osculator** — a small multiplier used to repeatedly fold the last digit into the rest of the number until a small, checkable result remains.

General procedure (for divisor d with osculator value m):

1. Remove the last digit of the number; call it L . Call the remaining leading part R .
2. Compute $R (+ \text{ or } -) (m \times L)$, depending on whether d uses a "negative" or "positive" osculator.
3. Repeat on the new, shorter number until small enough to check divisibility directly.

- For 7:** osculator = 2, used as SUBTRACT (double the last digit, subtract from the rest).
Example: Is 1071 divisible by 7? $107 - (2 \times 1) = 105$. Repeat: $10 - (2 \times 5) = 0$. **0 is divisible by 7 → yes, 1071 is divisible by 7.**
- For 13:** osculator = 4, used as ADD (quadruple the last digit, add to the rest).
Example: Is 2145 divisible by 13? $214 + (4 \times 5) = 234$. Repeat: $23 + (4 \times 4) = 39$. $39 = 13 \times 3 \rightarrow$ **yes, divisible by 13.**
- For 17:** osculator = 5, used as SUBTRACT. **For 19:** osculator = 2, used as ADD.
- TRICK: You don't need to memorize which primes use "add" vs "subtract" osculators from theory — just try one iteration each way on a known multiple of that prime and see which one shrinks correctly; in practice 7 and 17 use subtract-type, 13 and 19 use add-type.

A3.4 Combining Divisibility Rules for Composite Divisors [Level: Hard]

- A number is divisible by a composite $d = a \times b$ **only if a and b are CO-PRIME** — then divisibility by $d \Leftrightarrow$ divisible by both a AND b .
- TRAP:** Divisibility by a and by b does **NOT** imply divisibility by $a \times b$ if a and b are **not** co-prime. Classic counter-example: 12 is divisible by both 2 and 6, but $2 \times 6 = 12$ — this accidentally works; a clearer counter-example: 36 is divisible by both 4 and 6 ($4 \times 6 = 24$), but 36 is NOT divisible by 24. Always check co-primality of the factor pair before combining.
- Correct composite breakdowns (co-prime factor pairs only): $6=2 \times 3$, $12=3 \times 4$, $14=2 \times 7$, $15=3 \times 5$, $18=2 \times 9$, $20=4 \times 5$, $21=3 \times 7$, $33=3 \times 11$, $45=9 \times 5$.

Questions

Q1. [Easy] Which of the following numbers is divisible by 4?

A. 1234 B. 1354 C. 1372 D. 1358

Answer: C. Why: Check last 2 digits: $72 \div 4 = 18$ exactly → divisible. (34, 54, 58 are not divisible by 4.)

Q2. [Easy] Is 78,912 divisible by 9?

A. Yes B. No

Answer: A. Why: Digit sum = $7+8+9+1+2 = 27$, and 27 is divisible by 9 → yes.

Q3. [Medium] Check whether 9,152 is divisible by 11.

A. Yes B. No

Answer: A. Why: Odd-place digits (from right): 2,1 → sum=3. Even-place digits: 5,9 → sum=14. Difference = $14-3 = 11$, a multiple of 11 → **yes, divisible**.

Q4. [Medium] Using the osculator method, check if 3,059 is divisible by 7.

A. Yes B. No

Answer: A. Why: $305 - (2 \times 9) = 305 - 18 = 287$. $28 - (2 \times 7) = 28 - 14 = 14$. 14 is divisible by 7 → **yes**.

Q5. [Medium] [Trap] A number is divisible by both 4 and 6. Is it necessarily divisible by 24?

A. Yes, always B. No — 4 and 6 are not co-prime, so this does not follow C. Only if the number is even D. Only if the number ends in 0

Answer: B. Why: 4 and 6 share a common factor of 2 (not co-prime), so divisibility by both does not guarantee divisibility by their product 24 — e.g., 12 is divisible by 4 and 6 but not by 24.

Q6. [Hard] Using the osculator method for 13, check whether 6,499 is divisible by 13.

A. Yes B. No

Answer: B. Why: $649 + (4 \times 9) = 685$. $68 + (4 \times 5) = 88$. $8 + (4 \times 8) = 40$. 40 is not a multiple of 13 ($13 \times 3 = 39$, remainder 1) → **not divisible**.

Q7. [Hard] Find the smallest digit that should replace * in 478 so that the number is divisible by 3.

A. 0 B. 2 C. 3 D. 4

Answer: B. Why: Digit sum so far = $4 + 7 + 8 = 19$. Need (19+) divisible by 3; since $19 \bmod 3 = 1$, we need $* \bmod 3 = 2$. The smallest digit satisfying this is **2** (0 and 1 don't work).

Q8. [Hard] A number is divisible by 15 and by 8. What is the smallest such 3-digit number?

A. 120 B. 240 C. 360 D. 480

Answer: A. Why: 15 and 8 are co-prime, so the number must be divisible by $\text{LCM}(15, 8) = 120$. Smallest 3-digit multiple of 120 is **120** itself.

9.A4 Percentage

(Foundational topic — nearly every other quant chapter (Profit & Loss, SI/CI, Data Interpretation) is "percentage in disguise." Master the shortcut conversions here first.)

A4.1 Basic Concept [Level: Basic]

- **Percentage** = "per hundred" — a fraction with denominator 100. $x\% = x/100$.
- Percentage is a **pure number** (no units) used to compare two quantities on a common 100-point scale.

To convert fraction $\rightarrow$ percentage: $(\text{fraction}) \times 100$
 To convert percentage $\rightarrow$ fraction: $(\text{percentage}) / 100$
 $x\% \text{ of } y = (x/100) \times y = y\% \text{ of } x$ <- TRICK: $x\% \text{ of } y$ always equals $y\% \text{ of } x$; pick whichever is easier to compute

- **TRAP:** "20% of 50" and "50% of 20" are NOT intuitively obviously equal to a beginner, but they both equal 10 — exploit this symmetry to simplify awkward-looking calculations by flipping the numbers.

A4.2 The Fraction–Percentage Conversion Table [Level: Basic — memorize this cold]

Speed in this chapter (and half of quant) comes from instant recall of these, not from long division.

Fraction	%	Fraction	%	Fraction	%
1/2	50%	1/6	16.67%	1/11	9.09%
1/3	33.33%	1/7	14.28%	1/12	8.33%
1/4	25%	1/8	12.5%	1/15	6.67%
1/5	20%	1/9	11.11%	1/16	6.25%
2/3	66.67%	1/10	10%	1/20	5%

- TRICK: 1/7 = 14.28% is worth memorizing to 2 decimals because multiples of 1/7 (2/7, 3/7...) just multiply 14.28 — e.g., 3/7 = 42.85%.
- Ways to find $x\%$ of a number N:
 1. **Direct method:** $(x \times N)/100$ — always works, sometimes slow.
 2. **Fraction-shortcut method:** convert $x\%$ to its fraction from the table above and multiply — e.g., 12.5% of 640 = $(1/8) \times 640 = 80$, instantly, no long multiplication.

A4.3 Percentage Increase / Decrease [Level: Basic-Medium]

New Value = Original $\times (1 + p/100)$ [increase by $p\%$]
 New Value = Original $\times (1 - p/100)$ [decrease by $p\%$]
 Percentage Change = $[(\text{New} - \text{Original})/\text{Original}] \times 100$

- **TRAP:** A $p\%$ increase followed by a $p\%$ decrease does **NOT** return you to the original value:

Net effect of $+p\%$ then $-p\%$ = $-(p^2)/100$ percent (always a NET DECREASE, never zero)

Example: $+10\%$ then $-10\% \rightarrow$ net change = $-(10 \times 10)/100 = -1\%$ (not 0%). This exact trap is one of the most repeated MCQ setups in this topic.

- **Successive percentage change formula** (for any two successive changes of $a\%$ and $b\%$, with correct signs — negative for decrease):

Net % change = $a + b + (a \times b)/100$

A4.4 Product-Constant Rule (Inverse Percentage Change) [Level: Medium]

- If a product $P = A \times B$ is to be kept CONSTANT and A increases by $x\%$, then B must change by:

Required % change in B = $[-x / (100 + x)] \times 100$ (a decrease, if $x > 0$)

- Classic application: "Price of sugar increases by 25%. By what % should consumption be reduced to keep expenditure unchanged?"
 $\rightarrow$ Required decrease = $[25/(100+25)] \times 100 = 2500/125 = \mathbf{20\%}$.
- TRICK (fraction shortcut): $+25\% = +1/4$, so new price = $5/4$ of old. To keep product same, new consumption must be $4/5$ of old $\rightarrow$ a decrease of $1/5 = \mathbf{20\%}$. (Flip the fraction — numerator and denominator swap.)

A4.5 Population / Repeated Percentage Change Over Time [Level: Medium-Hard]

Value after n years of $r\%$ compounding growth = $P \times (1 + r/100)^n$
 Value after n years of $r\%$ compounding decline = $P \times (1 - r/100)^n$
 Mixed (growth then decline in different years): apply each year's factor successively, e.g.
 $P \times (1+r1/100) \times (1-r2/100) \times (1+r3/100) \dots$

- **TRAP:** Do NOT simply add percentages across years when the base changes each year (e.g., "increases by 10% in year 1 and 20% in year 2" is NOT a flat $+30\%$; use successive-change formula: $10+20+(10 \times 20)/100 = \mathbf{32\%}$).

A4.6 Three Classic "Confusing Question" Patterns [Level: Hard]

These three patterns are explicitly the most-repeated trap format in this chapter:

1. "**A's income is $x\%$ more than B's**" vs "**B's income is $x\%$ less than A's**" — these are NOT the same percentage. If $A = 1.1x \times B$, then B is less than A by $[x/(100+x)] \times 100\%$, not by $x\%$. (Same product-constant logic as A4.4.)
2. "**Reduced by $x\%$** " applied to the wrong base — always re-anchor the base after each change; percentage change is only ever relative to the value immediately before that change, never the original value two steps back.

3. **Marks/Votes percentage problems** — "a candidate got x% of votes and lost by y votes" — the TOTAL votes is the unknown; set up $\text{Total} \times (\text{winner\%} - \text{loser\%}) = \text{margin votes}$ rather than working with one candidate's votes in isolation.

Questions

(ordered Easy → Hard; [Trick] tags a question specifically designed to test the shortcut, [Trap] tags a classic misconception)

Q1. [Easy] What is 12.5% of 480?

A. 40 B. 50 C. 60 D. 65

Answer: C. **Why:** $12.5\% = 1/8$ (fraction shortcut). $480/8 = 60$.

Q2. [Easy] Express $3/8$ as a percentage.

A. 32.5% B. 35% C. 37.5% D. 40%

Answer: C. **Why:** $1/8 = 12.5\%$, so $3/8 = 3 \times 12.5\% = 37.5\%$.

Q3. [Medium] [Trap] The price of an item is increased by 20% and then decreased by 20%. What is the net percentage change?

A. No change B. 4% increase C. 4% decrease D. 40% decrease

Answer: C. **Why:** Net change = $a+b+(ab)/100 = 20-20+(20 \times -20)/100 = 0 - 4 = -4\%$. A +p%/−p% pair always nets to a decrease of $p^2/100$ percent.

Q4. [Medium] [Trick] A number is increased by 25% and then decreased by 20%. What is the net effect?

A. No change B. 5% increase C. 5% decrease D. 10% increase

Answer: A. **Why:** Net = $25 - 20 + (25 \times -20)/100 = 5 - 5 = 0\%$. (Fraction check: $+25\% = \times 5/4$, then $-20\% = \times 4/5$; $(5/4) \times (4/5) = 1$ — no change.)

Q5. [Medium] The price of rice increases by 30%. By what percentage must a family reduce consumption to keep expenditure unchanged?

A. 20% B. 23.07% C. 30% D. 33.33%

Answer: B. **Why:** Required decrease = $[30/(100+30)] \times 100 = 3000/130 = 23.07\%$.

Q6. [Hard] [Trap] A's salary is 25% more than B's salary. By what percentage is B's salary less than A's?

A. 25% B. 20% C. 30% D. 33.33%

Answer: B. **Why:** A = 1.25B. Required decrease from A to B = $[25/(100+25)] \times 100 = 2500/125 = 20\%$ — NOT 25%. This is the single most repeated percentage trap.

Q7. [Hard] The population of a town increases by 10% in the first year and decreases by 10% in the second year. If the population after two years is 9,900, find the original population.

A. 9,800 B. 10,000 C. 10,100 D. 9,900

Answer: B. **Why:** Net two-year factor = $1.10 \times 0.90 = 0.99$ (i.e., net −1%, matching the +10%/−10% trap from A4.3). $\text{Original} \times 0.99 = 9900 \rightarrow \text{Original} = 9900/0.99 = 10,000$.

Q8. [Hard] In an election between two candidates, the winner got 60% of the total votes and won by 4,800 votes. Find the total number of votes polled.

A. 20,000 B. 22,000 C. 24,000 D. 25,000

Answer: C. **Why:** Winner=60%, Loser=40%, margin=20% of total = 4800 → Total = $4800/0.20 = 24,000$.

Q9. [Hard] [Trick] If 60% of A = $3/4$ of B, find A:B.

A. 3:4 B. 4:5 C. 5:4 D. 5:6

Answer: C. **Why:** $0.6A = 0.75B \rightarrow A/B = 0.75/0.6 = 5/4$, so **A:B = 5:4**.

9.A5 Ratio & Proportion

A5.1 Basic Concept [Level: Basic]

- **Ratio** $a:b = a/b$ — a comparison of two quantities of the **same kind/unit**. A ratio has no units itself.
- **Proportion** $a:b :: c:d$ (read "a is to b as c is to d") means $a/b = c/d$.

Product of Extremes = Product of Means
 $a:b :: c:d \Leftrightarrow a \times d = b \times c$

- **TRAP:** A ratio must compare like quantities (e.g., both in kg, or both in rupees) — "ratio of 5 kg to 200 g" must first be converted to the same unit (5000g:200g = 25:1), not written as 5:200.

A5.2 Types of Ratio [Level: Basic]

Type	Meaning	Example
Duplicate ratio	$a^2:b^2$	duplicate of 2:3 is 4:9
Sub-duplicate ratio	$\sqrt{a} : \sqrt{b}$	sub-duplicate of 4:9 is 2:3
TriPLICATE ratio	$a^3:b^3$	triplicate of 2:3 is 8:27
Sub-triplicate ratio	$a^{1/3} : b^{1/3}$	sub-triplicate of 8:27 is 2:3
Compounded ratio	$(a \times c):(b \times d)$, combining $a:b$ and $c:d$	2:3 compounded with 4:5 = 8:15
Inverse ratio	$b:a$	inverse of 2:3 is 3:2

A5.3 Ways to Divide a Quantity in a Given Ratio [Level: Basic]

To split Total in ratio $m:n$:
 First part = $\text{Total} \times m/(m+n)$
 Second part = $\text{Total} \times n/(m+n)$

For 3+ terms, ratio $m:n:p$ of Total:
 Each part = $\text{Total} \times (\text{its own term}) / (m+n+p)$

- **TRICK:** Never solve via simultaneous equations ($x = m \times k$, $y = n \times k$, $x+y=\text{Total}$) if you can directly plug into the fraction formula above — it's faster under time pressure.

A5.4 Properties of Proportion [Level: Basic-Medium]

(Given $a:b :: c:d$, i.e., $a/b = c/d$)

Property	Rule
Invertendo	$b:a :: d:c$
Alternendo	$a:c :: b:d$
Componendo	$(a+b):b :: (c+d):d$
Dividendo	$(a-b):b :: (c-d):d$
Componendo-Dividendo	$(a+b):(a-b) :: (c+d):(c-d)$
Addendo	if $a:b::c:d::e:f...$, then $a:b :: (a+c+e):(b+d+f)$

- **Componendo-Dividendo is the single most exam-useful property** — use it whenever a problem gives an equation of the form $(x+y)/(x-y) = p/q$.

Worked Example (Componendo-Dividendo) [Level: Medium]

If $(3x+2y)/(3x-2y) = 5/1$, find $x:y$.

By componendo-dividendo (reverse direction): $3x/2y = (5+1)/(5-1) = 6/4 = 3/2 \rightarrow 6x = 6y \rightarrow x/y = 1/1$.

Recheck directly: cross-multiply original: $3x+2y = 5(3x-2y) \rightarrow 3x+2y = 15x-10y \rightarrow 12y = 12x \rightarrow x:y = 1:1$. ✓ matches.

A5.5 Direct, Inverse & Joint Variation [Level: Medium]

Direct variation: $y = kx$ (y increases as x increases, ratio y/x constant)
 Inverse variation: $y = k/x$ (y decreases as x increases, product xy constant)
 Joint variation: $z = kxy$ (z depends jointly on 2+ variables)

- **TRAP:** "Time taken is proportional to work" = **direct** variation; "Time taken is proportional to speed" is **inverse** (more speed $\rightarrow$ less time) — always check the real-world direction of the relationship, don't assume "proportional to" always means direct variation in casual wording; read carefully whether it says "directly" or "inversely."

A5.6 Combining & Converting Ratios [Level: Medium-Hard]

- **Combining A:B and B:C into A:B:C:** make the common term (B) equal via LCM, then combine.

A:B = 2:3, B:C = 4:5
 LCM(3,4) = 12. Scale A:B by 4 → 8:12. Scale B:C by 3 → 12:15.
 ⇒ A:B:C = 8:12:15

- **Finding A:C given A:B and B:C** (chain-multiplication shortcut): $A:C = (A/B \times B/C)$ — multiply the two ratios directly after matching B, same result as above without writing the full A:B:C.
- **Finding A:B:C given mA = nB = pC:**

$mA = nB = pC \Rightarrow A:B:C = 1/m : 1/n : 1/p$ (then clear fractions by multiplying by $LCM(m,n,p)$)

- **Finding ratio of two numbers from their sum and difference:** if $sum=S$ and $difference=D$, the numbers are $(S+D)/2$ and $(S-D)/2$ — their ratio follows directly.
- **Comparing two ratios (which is bigger):** cross-multiply — $a:b$ vs $c:d$, compare $a \times d$ vs $b \times c$ (larger product's ratio is bigger). Faster than converting both to decimals.
- **Finding $(mX+nY):(pX+qY)$ given $x:y$:** substitute the given $x:y$ value directly into the expression and simplify — always faster than solving for actual X,Y values.

Questions

(ordered Easy → Hard)

Q1. [Easy] Divide 720 in the ratio 5:7.

A. 300, 420 B. 320, 400 C. 350, 370 D. 310, 410

Answer: A. **Why:** Parts = $720 \times 5/12 = 300$ and $720 \times 7/12 = 420$.

Q2. [Easy] Find the duplicate ratio of 3:5.

A. 6:10 B. 9:25 C. 3:25 D. 9:5

Answer: B. **Why:** Duplicate ratio = $a^2:b^2 = 9:25$.

Q3. [Medium] If $a:b = 2:3$ and $b:c = 4:5$, find $a:b:c$.

A. 8:12:15 B. 2:3:5 C. 8:15:12 D. 6:9:15

Answer: A. **Why:** $LCM(3,4)=12$. $a:b=2:3 \rightarrow 8:12$; $b:c=4:5 \rightarrow 12:15$. Combined $a:b:c = 8:12:15$.

Q4. [Medium] [Trap] The time taken to complete a journey is said to be "proportional to distance." If speed is constant, is this direct or inverse variation, and what if instead time is related to speed for a fixed distance?

A. Both are direct variation B. Time~distance (const. speed) is direct; time~speed (const. distance) is inverse C. Both are inverse variation D. Neither is a variation relationship

Answer: B. **Why:** At constant speed, more distance needs proportionally more time (direct, $T=D/\text{speed}$ with speed fixed acts as $T=kD$). At constant distance, more speed needs proportionally less time ($T=D/\text{speed}$, D fixed, so $T \propto 1/\text{speed}$ — inverse).

Q5. [Medium] Two numbers are in the ratio 3:4. If their sum is 84, find the numbers.

A. 36, 48 B. 30, 54 C. 32, 52 D. 40, 44

Answer: A. **Why:** $84 \times 3/7 = 36$, $84 \times 4/7 = 48$.

Q6. [Hard] If $(5x+3y):(5x-3y) = 4:1$, find $x:y$.

A. 1:1 B. 3:5 C. 5:3 D. 2:3

Answer: A. **Why:** By reverse componendo-dividendo, $5x:3y = (4+1):(4-1) = 5:3$, so $x:y = (5/5):(3/3) = 1:1$. Check: $5x+3y=4(5x-3y) \rightarrow 15y=15x \rightarrow x=y$. ✓

Q7. [Hard] Find A:B:C if $2A = 3B = 4C$.

A. 6:4:3 B. 2:3:4 C. 3:4:6 D. 4:3:2

Answer: A. **Why:** $A:B:C = 1/2:1/3:1/4$. $LCM(2,3,4)=12 \rightarrow$ multiply through: 6:4:3.

Q8. [Hard] The sum of two numbers is 8 times their difference. Find the ratio of the larger to the smaller number.

A. 9:7 B. 7:5 C. 5:3 D. 3:1

Answer: A. **Why:** Let numbers be x,y ($x>y$). $x+y = 8(x-y) \rightarrow x+y=8x-8y \rightarrow 9y=7x \rightarrow x/y=9/7 \rightarrow \text{larger:smaller} = 9:7$.

9.A6 Average

A6.1 Basic Concept [Level: Basic]

Average = Sum of all values / Number of values
Sum = Average x Number of values

- **Average of consecutive numbers (or any AP):** no need to add every term —

Average of an AP (equally spaced numbers) = (First term + Last term) / 2

- TRICK: For n consecutive integers starting at a, average = $a + (n-1)/2$.

A6.2 Change in Average — Join / Leave / Replace [Level: Basic-Medium]

Person REPLACES another:
Change in total = (New person's value - Old person's value)
Change in average = Change in total / (number of people, unchanged)

Person JOINS the group:
New Average = (Old Total + New person's value) / (Old Count + 1)

Person LEAVES the group:
New Average = (Old Total - Leaving person's value) / (Old Count - 1)

- TRICK (replacement shortcut): if average INCREASES by d when a person is replaced, then New person's value = Old person's value + d × (count).
- TRICK (join/leave shortcut): the new average shifts toward the joining/leaving value; if a new member's value equals the OLD average exactly, the average does NOT change at all.

Worked Example [Level: Medium]

The average age of 10 people increases by 2 years when one person aged 35 is replaced by a new person. Find the new person's age.
Change in total = $2 \times 10 = 20$. New person's age = $35 + 20 = 55$.

A6.3 Weighted Average [Level: Medium]

Weighted Average = $(n_1x_1 + n_2x_2 + \dots) / (n_1 + n_2 + \dots)$

- This is mathematically identical to the **Alligation** rule (see Mixture & Alligation chapter) — same formula, different visual presentation (alligation draws it as a cross/diagram).
- **TRAP:** Combined average of two groups is NOT the simple average of their two individual averages unless the group sizes are equal. You must weight by group size:

Combined Average = $(n_1 \times \text{avg}_1 + n_2 \times \text{avg}_2) / (n_1 + n_2)$

e.g., average of a class of 30 (avg 60) and a class of 20 (avg 80) is NOT 70 — it is $(30 \times 60 + 20 \times 80) / 50 = (1800 + 1600) / 50 = 68$.

A6.4 Practical Word-Problem Patterns [Level: Hard]

- **Average speed is NOT the simple average of speeds** for equal-distance trips — see Speed-Time-Distance chapter for the harmonic-mean formula $\frac{2xy}{(x+y)}$.
TRAP: only average two speeds directly (simple mean) if the TIME spent at each speed is equal, not the distance.
- **Cricket batting-average style problems:** "A batsman's average after 16 innings is A. After the 17th innings, scoring S runs, his average becomes A+d." Setup:

$16A + S = 17(A + d) \Rightarrow S = A + 17d$

Solve for the unknown directly from this single linear equation rather than guessing.

Questions

(ordered Easy → Hard)

Q1. [Easy] Find the average of the first 10 natural numbers.

A. 5 B. 5.5 C. 6 D. 10

Answer: B. **Why:** Average of AP $1..10 = (1+10)/2 = 5.5$.

Q2. [Easy] The average of 5 numbers is 20. If one number is excluded, the average becomes 18. Find the excluded number.

A. 24 B. 26 C. 28 D. 30

Answer: C. **Why:** Original total = 100. New total (4 numbers) = $18 \times 4 = 72$. Excluded number = $100 - 72 = 28$.

Q3. [Medium] The average weight of 8 people increases by 1.5 kg when a new person replaces one weighing 65 kg. Find the new person's weight.

A. 70 kg B. 74 kg C. 76 kg D. 77 kg

Answer: D. **Why:** Change in total = $1.5 \times 8 = 12$. New weight = $65 + 12 = 77$ kg.

Q4. [Medium] [Trap] Average marks of Section A (25 students) is 60 and Section B (15 students) is 80. Find the combined average.

A. 70 B. 67.5 C. 66 D. 68.5

Answer: B. **Why:** Combined = $(25 \times 60 + 15 \times 80) / 40 = (1500 + 1200) / 40 = 2700 / 40 = 67.5$ (NOT the simple average 70, since group sizes differ).

Q5. [Medium] A cricketer's average after 10 innings is 40. He scores 84 in the 11th innings. Find his new average.

A. 42 B. 43 C. 44 D. 45

Answer: C. **Why:** Old total = 400. New total = $400 + 84 = 484$. New average = $484 / 11 = 44$.

Q6. [Hard] The average age of a family of 5 members is 24 years today. The youngest member is 8 years old. Find the average age of the family at the time the youngest member was born (i.e., 8 years ago).

A. 18 years B. 19 years C. 20 years D. 21 years

Answer: C. Why: Present total age = $24 \times 5 = 120$. Excluding the youngest (not yet born 8 years ago), the other 4 members currently total $120 - 8 = 112$. Eight years ago each of those 4 was 8 years younger, so their total then = $112 - 4 \times 8 = 80$, over 4 people $\rightarrow$ average = $80/4 = 20$ years.

Q7. [Hard] A batsman's average after 16 innings is 36. He scores 104 in the 17th innings. By how much does his average increase?

A. 2 B. 3 C. 4 D. 5

Answer: C. Why: Using $16A + S = 17(A + d)$: $16(36) + 104 = 17(36 + d) \rightarrow 576 + 104 = 680 \rightarrow 612 + 17d = 680 \rightarrow 17d = 68 \rightarrow d = 4$.

9.A7 Profit and Loss

A7.1 Basic Concepts [Level: Basic]

- **CP** (Cost Price) = price at which an article is bought. **SP** (Selling Price) = price at which it is sold.
- **Profit** = $SP - CP$ (when $SP > CP$). **Loss** = $CP - SP$ (when $CP > SP$).

$$\begin{aligned}\text{Profit \%} &= (\text{Profit} / \text{CP}) \times 100 \\ \text{Loss \%} &= (\text{Loss} / \text{CP}) \times 100\end{aligned}$$

- **TRAP:** Profit%/Loss% is **ALWAYS calculated on CP**, never on SP, unless a question explicitly states otherwise. This is the single most common error in this chapter.

A7.2 Quick Transformation Shortcuts [Level: Basic-Medium]

$$\begin{aligned}SP &= CP \times (100 + \text{Profit\%}) / 100 & SP &= CP \times (100 - \text{Loss\%}) / 100 \\ CP &= SP \times 100 / (100 + \text{Profit\%}) & CP &= SP \times 100 / (100 - \text{Loss\%})\end{aligned}$$

- Ways to solve CP $\leftrightarrow$ SP conversions:
 1. **Direct formula method:** plug into the formulas above.
 2. **Fraction-ratio shortcut (faster):** convert Profit%/Loss% to a ratio SP:CP. E.g., 25% profit $\rightarrow$ SP:CP = 125:100 = **5:4**. 20% loss $\rightarrow$ SP:CP = 80:100 = **4:5**. Once you have the ratio, cross-multiply directly — no need to write the full percentage formula.
- **TRICK:** Memorize common ratio pairs: 10% $\rightarrow$ 11:10, 20% $\rightarrow$ 6:5, 25% $\rightarrow$ 5:4, 33 1/3% $\rightarrow$ 4:3, 50% $\rightarrow$ 3:2 (profit); 10% $\rightarrow$ 9:10, 20% $\rightarrow$ 4:5, 25% $\rightarrow$ 3:4 (loss).

A7.3 Marked Price & Discount [Level: Medium]

$$\begin{aligned}\text{Discount \% is calculated on Marked Price (MP), NOT on CP.} \\ SP &= MP \times (100 - \text{Discount\%}) / 100\end{aligned}$$

- **TRAP:** Do not confuse **Discount%** (always on MP) with **Profit/Loss%** (always on CP) — a mixed question may give both and expect you to chain: MP $\rightarrow$ (apply discount) $\rightarrow$ SP $\rightarrow$ (compare to CP) $\rightarrow$ Profit/Loss%.
- **Successive discounts:** two discounts of a% and b% applied one after another do NOT add up to (a+b)%. Use the successive-percentage-change formula:

$$\text{Net discount \%} = a + b - (a \times b)/100 \quad (\text{always LESS than the simple sum})$$

E.g., successive discounts of 10% and 10% give a net discount of $10+10-1 = 19\%$, not 20%.

A7.4 False Weight / Cheating Trader Problems [Level: Medium-Hard]

- A trader claims to sell at cost price (0% profit) but uses a **false weight** — e.g., uses a 900g "kg" instead of a true 1000g kg.

$$\begin{aligned}\text{Profit \%} &= [(\text{True Value} - \text{False Value}) / \text{False Value}] \times 100 \\ &= [\text{Error} / (\text{True weight used})] \times 100\end{aligned}$$

E.g., using 900g instead of 1000g while charging for 1000g: Profit% = $(1000-900)/900 \times 100 = 11.11\%$.

- **TRICK:** This is structurally identical to the Percentage chapter's product-constant rule — the trader effectively "increases price per gram" by shortchanging weight, so the same $[\text{error}/(\text{actual}-\text{error})] \times 100$ pattern applies.

A7.5 Overall / Combined Profit-Loss on Multiple Transactions [Level: Hard]

- **TRAP:** When profit% differs across multiple transactions, you **cannot average the percentages directly** — you must combine using total CP and total SP:

$$\text{Overall Profit/Loss \%} = [(\text{Total SP} - \text{Total CP}) / \text{Total CP}] \times 100$$

- **Classic trap — selling two items at the SAME SP, one at x% profit and the other at x% loss:** the result is **always a net loss**, regardless of x, given by:

$$\text{Net Loss \%} = x^2 / 100$$

E.g., two items sold at ₹100 each, one at 20% profit and other at 20% loss $\rightarrow$ net loss = $20^2/100 = 4\%$ loss overall — even though it looks like the profit and loss should "cancel out." This happens because the CPs of the two items are different (one is SP/1.2, the other is SP/0.8), so equal SP does not mean equal-magnitude profit/loss in absolute terms.

Questions

Q1. [Easy] A shopkeeper buys an article for ₹400 and sells it for ₹460. Find the profit percent.

A. 10% B. 12% C. 15% D. 20%

Answer: C. Why: Profit = $460 - 400 = 60$. Profit% = $60/400 \times 100 = 15\%$.

Q2. [Easy] [Trick] An article is sold at a profit of 25%. Find the ratio SP:CP.

A. 4:5 B. 5:4 C. 6:5 D. 3:2

Answer: B. Why: 25% profit $\rightarrow$ SP:CP = 125:100 = **5:4**.

Q3. [Medium] A shopkeeper marks an item 40% above cost price and gives a discount of 10%. Find his profit percent.

A. 20% B. 26% C. 30% D. 36%

Answer: B. Why: MP = $1.4 \times \text{CP}$. SP = $0.9 \times \text{MP} = 0.9 \times 1.4 \times \text{CP} = 1.26 \times \text{CP} \rightarrow$ **26% profit**.

Q4. [Medium] [Trap] A trader marks his goods 50% above cost price and allows a discount of 20%. What is his loss or gain percent?

A. 20% gain B. 30% gain C. 20% loss D. 30% loss

Answer: A. Why: MP = $1.5 \times \text{CP}$. SP = $0.8 \times 1.5 \times \text{CP} = 1.2 \times \text{CP} \rightarrow$ **20% gain** (not a loss — a common trap is assuming the discount always erases the markup gain).

Q5. [Medium] Successive discounts of 20% and 10% are given on an item marked at ₹500. Find the final selling price.

A. ₹350 B. ₹355 C. ₹360 D. ₹400

Answer: C. Why: Net discount = $20 + 10 - (20 \times 10)/100 = 28\%$. SP = $500 \times (1 - 0.28) = 500 \times 0.72 =$ **₹360**.

Q6. [Medium] A dishonest dealer professes to sell his goods at cost price but uses a weight of 900g for 1kg. Find his profit percent.

A. 9% B. 10% C. 11.11% D. 12%

Answer: C. Why: Profit% = $(1000-900)/900 \times 100 = 100/900 \times 100 = 11.11\%$.

Q7. [Hard] [Trap] A man sells two watches at ₹1000 each. On one he gains 20% and on the other he loses 20%. Find his overall gain or loss percent.

A. No profit no loss B. 4% loss C. 4% gain D. 2% loss

Answer: B. Why: Common equal-SP profit/loss trap: Net Loss% = $x^2/100 = 20^2/100 = 4\%$ **loss** overall — never zero despite the symmetric-looking percentages.

Q8. [Hard] A shopkeeper sold two articles, each for ₹2400. On the first he gained 20% and on the second he lost 20%. Find his total CP and the overall profit or loss in rupees.

A. Total CP=4900, Loss=₹100 B. Total CP=5000, Loss=₹200 C. Total CP=5000, Gain=₹200 D. Total CP=4800, no loss

Answer: B. Why: CP1 = $2400/1.2 = 2000$; CP2 = $2400/0.8 = 3000$. Total CP = 5000, Total SP = 4800 → Loss = ₹200 (matches 4% of 5000 = 200, consistent with Q7's formula).

Q9. [Hard] Find the single discount equivalent to three successive discounts of 10%, 20%, and 10%.

A. 35.2% B. 36.8% C. 38% D. 40%

Answer: A. Why: Combine first two: $10+20-(10 \times 20)/100 = 28\%$. Combine with third: $28+10-(28 \times 10)/100 = 38-2.8 = 35.2\%$.

Q10. [Hard] [Trap] By selling 45 lemons for ₹40, a woman loses 20%. How many lemons should she sell for ₹24 to gain 20% on the whole?

A. 12 B. 15 C. 18 D. 20

Answer: C. Why: SP of 45 lemons = 40 represents 80% of CP → CP of 45 lemons = $40/0.8 = 50$, so CP per lemon = $50/45 = 10/9$. To gain 20%, required SP per lemon = $(10/9) \times 1.2 = 4/3$. Number of lemons for ₹24 = $24 \div (4/3) = 18$.

9.A8 Simple Interest & Compound Interest

A8.1 Terminology & Simple Interest Basics [Level: Basic]

- **Principal (P):** the original sum borrowed/invested. **Rate (R):** interest rate per annum (%). **Time (T):** duration in years. **Interest (I):** the extra money paid/earned. **Amount (A):** $P + I$.

Simple Interest: $SI = (P \times R \times T) / 100$
Amount: $A = P + SI$

- Ways to solve SI problems: (1) **direct formula** substitution; (2) **unitary shortcut** — SI per year = $(P \times R)/100$, then multiply by T; useful when T changes across sub-questions but P,R stay fixed.

A8.2 Compound Interest Basics [Level: Basic-Medium]

Amount (compounded annually): $A = P \times (1 + R/100)^T$
Compound Interest: $CI = A - P$

- TRICK — 10-second CI for small T without a calculator: apply the successive-percentage-change formula (from Percentage chapter) repeatedly. E.g., $R=10\%$, $T=2$: net growth = $10+10+(10 \times 10)/100 = 21\% \rightarrow CI = 21\%$ of P (matches $(1.1)^2=1.21$ exactly). For $T=3$: apply the 2-year net (21%) with the 3rd year's 10% again: $21+10+(21 \times 10)/100 = 33.1\%$ — matches $(1.1)^3=1.331$.
- **TRAP:** Compounding is **half-yearly** or **quarterly** in many questions — you MUST adjust both R and T:

Half-yearly: use $R/2$ as the rate per period, and $2T$ as the number of periods.
Quarterly: use $R/4$ as the rate per period, and $4T$ as the number of periods.

Forgetting this adjustment (using annual R,T directly with half-yearly compounding) is one of the most common CI errors.

A8.3 CI vs SI Relationship (High-Yield Formulas) [Level: Medium]

For $T = 2$ years: $CI - SI = P \times (R/100)^2$
For $T = 3$ years: $CI - SI = P \times (R/100)^2 \times (3 + R/100)$

- These let you find P or R directly from a given (CI – SI) difference without computing CI and SI separately in full.
- TRICK: For $T=2$, the difference (CI–SI) is exactly "interest on the first year's interest" — conceptually, CI compounds the first year's SI into the second year's principal, while SI does not; this single year's "interest on interest" IS the entire CI–SI gap.

A8.4 Doubling / Multiplying Time Shortcuts (CI) [Level: Medium-Hard]

- If a sum **doubles** in T_1 years under CI, it becomes **4x** in $2 \times T_1$ years, **8x** in $3 \times T_1$ years, **2^n x** in $n \times T_1$ years — a direct geometric-growth shortcut (since each T_1 -year block simply reapplies the same $\times 2$ multiplier).
- **TRAP:** This clean doubling shortcut applies **only to CI**, not SI — under SI, if a sum doubles (i.e., earns 100% interest) in T_1 years, it becomes triple (not quadruple) in $2 \times T_1$ years, since SI grows linearly (each equal time block adds the SAME fixed interest, not a multiplying factor).

A8.5 Applications & Word-Problem Patterns [Level: Hard]

Extra interest earned due to a rate increase (SI-style, same P and T):
 $\text{Extra Interest} = P \times T \times (\text{Rate difference}) / 100$

Dividing a Principal into two parts such that SI on each becomes equal after different times T_1, T_2 :
 $P_1 \times T_1 = P_2 \times T_2$ (since equal SI at same rate R means equal $P \times T$ product)

- **Variable rate of interest** (different rates for different years, SI-style): total interest = $P \times (R_1 \times T_1 + R_2 \times T_2 + R_3 \times T_3 + \dots)/100$ — sum the rate-time products before dividing by 100, don't compute year-by-year separately then add (though both give the same answer, the combined approach is faster).
- **"Amount becomes x times the principal in T years" (SI):** since SI is linear, if amount becomes x times in T years, then $\text{Rate} = [(x-1) \times 100]/T$ (derived from $A = P + PRT/100 = Px \rightarrow RT = (x-1) \times 100$).

Questions

Q1. [Easy] Find the SI on ₹5000 at 8% per annum for 3 years.

A. ₹1000 B. ₹1100 C. ₹1200 D. ₹1500

Answer: C. **Why:** $SI = (5000 \times 8 \times 3)/100 = ₹1200$.

Q2. [Easy] Find the CI on ₹10,000 at 10% per annum for 2 years, compounded annually.

A. ₹2000 B. ₹2100 C. ₹2200 D. ₹2500

Answer: B. **Why:** Net 2-year growth = $10+10+(10 \times 10)/100 = 21\%$. CI = 21% of 10,000 = **₹2100**.

Q3. [Medium] [Trick] Find the difference between CI and SI on ₹8000 for 2 years at 5% per annum.

A. ₹15 B. ₹18 C. ₹20 D. ₹25

Answer: C. **Why:** $CI - SI = P \times (R/100)^2 = 8000 \times (0.05)^2 = 8000 \times 0.0025 = ₹20$.

Q4. [Medium] [Trap] Find the CI on ₹4000 at 20% per annum for 1 year, compounded half-yearly.

A. ₹800 B. ₹820 C. ₹840 D. ₹861

Answer: C. **Why:** Half-yearly: rate/period = 10%, periods = 2. $A = 4000 \times (1.1)^2 = 4000 \times 1.21 = 4840$. CI = $4840 - 4000 = ₹840$ (higher than the flat-20%-once value of ₹800 — a common trap is forgetting the half-yearly adjustment and answering ₹800).

Q5. [Medium] A sum of money doubles itself in 8 years at compound interest. In how many years will it become 8 times itself?

A. 16 B. 20 C. 24 D. 32

Answer: C. **Why:** Doubling shortcut: doubles in 8y $\rightarrow 4 \times$ in 16y $\rightarrow 8 \times$ in **24 years** ($3 \times T_1$, since $2^3=8$).

Q6. [Medium] The difference between CI and SI on a certain sum for 3 years at 10% per annum is ₹31. Find the sum.

A. ₹1000 B. ₹1500 C. ₹2000 D. ₹3100

Answer: A. Why: $CI - SI (T=3) = P \times (R/100)^2 \times (3 + R/100) = P \times 0.01 \times 3.1 = 0.031P$. Set $0.031P = 31 \rightarrow P = \text{₹}1000$.

Q7. [Hard] A sum of ₹12,000 becomes ₹15,600 in 3 years at simple interest. If the rate of interest is increased by 3%, what would the amount become?

A. ₹16,080 B. ₹16,410 C. ₹16,680 D. ₹15,900

Answer: C. Why: $SI = 15600 - 12000 = 3600$ over 3 years $\rightarrow$ Rate = $(3600 \times 100) / (12000 \times 3) = 10\%$. New rate = 13%. New SI = $(12000 \times 13 \times 3) / 100 = 4680$. New Amount = $12000 + 4680 = \text{₹}16,680$.

Q8. [Hard] A certain sum is divided into two parts such that the simple interest on the first part for 4 years at 5% per annum equals the simple interest on the second part for 5 years at 4% per annum. Find the ratio of the first part to the second.

A. 1:1 B. 4:5 C. 5:4 D. 16:25

Answer: A. Why: Equal-SI condition (same rate structure per part): $P_1 \times T_1 \times R_1 = P_2 \times T_2 \times R_2 \rightarrow P_1 \times 4 \times 5 = P_2 \times 5 \times 4 \rightarrow 20P_1 = 20P_2 \rightarrow P_1:P_2 = 1:1$.

Q9. [Hard] In what time will a sum of money become 4 times itself at 25% per annum simple interest?

A. 9 years B. 12 years C. 16 years D. 20 years

Answer: B. Why: Becomes $x=4$ times: Rate $\times$ Time = $(x-1) \times 100 \rightarrow 25 \times T = 300 \rightarrow T = 12$ years.

9.A9 Time & Work, Pipes & Cisterns

A9.1 Core Idea — Work as a Fixed Unit [Level: Basic]

- Treat "total work" as **1 unit**, or better, as the **LCM of all individual times given** in the question — this avoids fractions entirely and is the single biggest speed trick in this chapter.
- If A alone takes a days to finish a job: A's **one-day work** = $1/a$ (fraction method) — or, using the LCM trick, if Total Work = LCM($a, b, \dots$), A's one-day work (in units) = Total Work / a .

Ways to solve "A takes a days, B takes b days — find time together":

Method 1 (Fraction method): Combined 1-day work = $1/a + 1/b \Rightarrow$ Time = $1 / (1/a + 1/b) = ab/(a+b)$

Method 2 (LCM/Units method, faster for mental math):

Total Work = LCM(a, b)

A's efficiency = Total Work / a units/day

B's efficiency = Total Work / b units/day

Time together = Total Work / (A's efficiency + B's efficiency)

- TRICK: Method 2 turns every Time & Work problem into whole-number arithmetic — always prefer it over fractions.

A9.2 Efficiency Ratio [Level: Basic-Medium]

- Efficiency $\propto 1/\text{Time}$** — efficiency and time are INVERSELY related for a fixed amount of work.
- TRAP:** If A is twice as efficient as B, students often wrongly say "A takes twice the time." Correct: A is twice as efficient $\Rightarrow$ A takes **half** the time B takes.
- If efficiencies of A:B = $m:n$, then time taken by A:B = $n:m$ (the ratio flips).

A9.3 Work and Wages [Level: Medium]

- Total payment for a joint job must be split in the ratio of **work actually done**, which (for the same duration worked) equals the ratio of **efficiencies** — NOT the ratio of time taken alone, and NOT split equally by default.
- TRAP:** A common wrong approach is splitting wages by "days worked" when days worked are equal — if two people work the SAME number of days, wages split by efficiency ratio; only split by time/output actually contributed when durations differ too.

A9.4 Alternate-Days & Remaining-Work Problems [Level: Medium-Hard]

- When A and B work on alternate days, compute work done in **one full 2-day cycle** (A's one-day work + B's one-day work), find how many complete cycles are needed, then handle the leftover work in the next partial day separately.
- Person joining/leaving midway:** split the total timeline into phases (before the change, after the change) and add up the work done in each phase separately; the unknown (usually total time or one person's rate) is solved from the equation: Work done in Phase 1 + Work done in Phase 2 = Total Work.

A9.5 Pipes & Cisterns — Same Math, Just Signed Rates [Level: Medium]

- An **inlet pipe** (fills the tank) is a **positive** work rate; an **outlet/leak pipe** (empties the tank) is a **negative** work rate. Everything from A9.1–A9.4 (LCM trick, combined-rate addition, efficiency ratio) applies unchanged — just add rates algebraically with correct sign.
- TRAP:** A leak that "empties a full tank in x hours" must be SUBTRACTED from the inlet rate when computing the combined fill rate. If the combined rate comes out negative, the tank **never fills** (it net-empties) — always sanity-check the sign of your final combined rate before computing a time from it.
- TRICK: If an inlet fills in a hours and a leak empties in l hours, using Total Work = LCM(a, l): net rate = (TotalWork/ a) – (TotalWork/ l); Time to fill (with leak open) = TotalWork / net rate.

Questions

(Easy $\rightarrow$ Hard; [Trick] rewards the shortcut, [Trap] targets a common misconception)

Q1. [Easy] A can do a piece of work in 12 days and B in 6 days. In how many days will they finish it together?

A. 3 days B. 4 days C. 5 days D. 6 days

Answer: B. **Why:** Total work = LCM(12,6) = 12 units. A's rate = 1/day, B's rate = 2/day. Combined = 3/day. Time = $12/3 = 4$ days.

Q2. [Easy] A is twice as efficient as B. If B alone takes 18 days to finish a job, how long does A alone take?

A. 36 days B. 9 days C. 27 days D. 6 days

Answer: B. **Why:** Efficiency $\propto 1/\text{Time}$ (inverse). A is $2\times$ as efficient $\Rightarrow$ A takes half of B's time = $18/2 = 9$ days.

Q3. [Medium] [Trap] A, B and C do a job together and earn ₹1,800. A alone would take 6 days, B alone 8 days, and C alone 12 days for the same job. Find A's share.

A. ₹600 B. ₹800 C. ₹900 D. ₹400

Answer: B. **Why:** Total Work = LCM(6,8,12) = 24. Efficiencies: A=4, B=3, C=2/day (ratio 4:3:2, sum=9). A's share = $(4/9) \times 1800 = \text{₹}800$. (Wages split by efficiency, not equally — the classic trap here is splitting ₹600 each.)

Q4. [Medium] A can do a job in 20 days. After working alone for 4 days, B joins him and together they finish the remaining work in 8 days. In how many days can B alone do the job?

A. 15 days B. 16 days C. 20 days D. 24 days

Answer: C. **Why:** Total Work = 20 units. A's rate = 1 unit/day. In 4 days A completes 4 units, leaving 16 units. Together they finish these 16 units in 8 days $\Rightarrow$ combined rate = 2 units/day $\Rightarrow$ B's rate = $2 - 1 = 1$ unit/day $\Rightarrow$ B's time = $20/1 = 20$ days.

Q5. [Medium] [Trick] A tank has an inlet pipe that fills it in 6 hours and an outlet pipe that empties it in 10 hours. If both are opened together, in how many hours will the tank fill?

A. 12 hours B. 15 hours C. 8 hours D. 16 hours

Answer: B. **Why:** Total Work = LCM(6,10) = 30. Inlet rate = $+5/\text{hr}$, Outlet rate = $-3/\text{hr}$. Net rate = $+2/\text{hr}$. Time = $30/2 = 15$ hours.

Q6. [Hard] [Trap] Pipe A fills a tank in 4 hours, Pipe B fills it in 6 hours, and Pipe C (a leak) can empty the full tank in 12 hours. If all three are opened together, how long to fill the tank?

A. 3 hours B. 2.4 hours C. 4 hours D. 6 hours

Answer: A. **Why:** Total Work = LCM(4,6,12) = 12. Rates: A= $+3/\text{hr}$, B= $+2/\text{hr}$, C= $-1/\text{hr}$. Net rate = $3+2-1 = 4/\text{hr}$. Time = $12/4 = 3$ hours. (The trap: forgetting to subtract C's leak rate would wrongly give $12/5 = 2.4$ hours — option B.)

Q7. [Hard] A and B undertake a job for ₹720. Working alone, A would take 6 days and B would take 8 days. With the help of C, they finish it in 3 days. Find C's share.

A. ₹90 B. ₹180 C. ₹60 D. ₹120

Answer: A. Why: Total Work = LCM(6,8,3) = 24. A's rate=4, B's rate=3/day. Combined (A+B+C) rate needed = $24/3 = 8/\text{day}$. So C's rate = $8-4-3 = 1/\text{day}$. Wage split by efficiency ratio 4:3:1 (sum 8). C's share = $(1/8) \times 720 = \text{₹90}$.

Q8. [Hard] Working alone, A takes 6 days more than A and B together, and B alone takes 24 days more than A and B together. In how many days can A and B, working together, complete the job?

A. 8 days B. 12 days C. 10 days D. 6 days

Answer: B. Why: This is the classic pattern where, if T = time together, then $T^2 = (\text{extra time for A}) \times (\text{extra time for B})$ — here $T^2 = 6 \times 24 = 144$, so T = **12 days**. (Quick sanity check via the standard identity for this problem type: $T = \sqrt{xy}$ where x, y are the "extra days" each takes beyond the combined time.)

Q9. [Hard] [Trap] The efficiencies of A, B and C are in the ratio 5:4:3. Working together, they finish a job in 4 days. Find the number of days C alone would take.

A. 10 days B. 16 days C. 9 days D. 15 days

Answer: B. Why: Combined efficiency = $5+4+3 = 12 \text{ units/day}$. Total Work = $12 \times 4 = 48 \text{ units}$. C's rate = $3 \text{ units/day} \Rightarrow \text{C's time} = 48/3 = \text{16 days}$.

9.A10 Speed, Time & Distance (Trains, Boats & Streams)

A10.1 Core Relation & Unit Conversion [Level: Basic]

$$\text{Speed} = \text{Distance} / \text{Time} \quad \text{Distance} = \text{Speed} \times \text{Time} \quad \text{Time} = \text{Distance} / \text{Speed}$$

Unit conversion (TRICK – used constantly):

km/h → m/s : multiply by 5/18

m/s → km/h : multiply by 18/5

A10.2 Average Speed [Level: Basic-Medium]

$$\text{Average Speed} = \text{Total Distance} / \text{Total Time} \quad (\text{ALWAYS true – the only safe default formula})$$

- Ways to solve depending on what's constant across the journey's legs:
 - Equal DISTANCE legs** (e.g., going and returning over the same route at different speeds x and y): use the **harmonic mean** shortcut — $\text{Average Speed} = 2xy / (x + y)$.
 - Equal TIME legs** (e.g., first half of the total TIME at speed x , second half at speed y): the plain arithmetic mean is correct — $\text{Average Speed} = (x + y) / 2$.
 - Neither equal** (arbitrary distances/times per leg): fall back to the master formula — Total Distance / Total Time, computed leg by leg.
- TRAP:** Applying the simple average $(x+y)/2$ to an equal-DISTANCE journey is the single most common error in this entire chapter — it only works for equal TIME segments.
- Generalized equal-distance case (n legs of equal distance at speeds $v_1, v_2, \dots, v_n$): $\text{Average Speed} = n / (1/v_1 + 1/v_2 + \dots + 1/v_n)$ (harmonic mean of n terms).

Worked Examples (verified, increasing difficulty)

A bus takes students to college at 6 km/h; they miss the return bus and walk back the same route at 4 km/h. Average speed for the day?

Equal distance both ways → $\text{Avg} = 2 \times 6 \times 4 / (6 + 4) = 48/10 = \mathbf{4.8 \text{ km/h}}$. (Real placement-test question, TCS NQT 2018.)

A man commutes to office at 65 km/h and returns at 75 km/h. Average speed?

$\text{Avg} = 2 \times 65 \times 75 / (65 + 75) = 9750/140 = \mathbf{69.64 \text{ km/h}}$. (L&T Infotech 2016.)

A journey is covered 1/3 at 60 km/h, 1/3 at 75 km/h, and 1/3 at 45 km/h (equal DISTANCE thirds). Average speed?

Generalized harmonic mean: $\text{Avg} = 3 / (1/60 + 1/75 + 1/45) = 3 / 0.052222 = \mathbf{57.45 \text{ km/h}}$. (L&T Infotech.)

A bird flies 400 km: the first 100 km at 100 km/h, next 100 km at 200 km/h, next at 300 km/h, last at 400 km/h. Average speed?

Total time = $100/100 + 100/200 + 100/300 + 100/400 = 1 + 0.5 + 0.333 + 0.25 = 2.0833$ h. $\text{Average} = 400/2.0833 = \mathbf{192 \text{ km/h}}$. (GATE Electrical.) — Note this is NOT an equal-time case, so the master formula (Total Distance/Total Time) was applied directly rather than a mean shortcut.

A car travels 8 km in the 1st quarter-hour, 6 km in the 2nd quarter-hour, 16 km in the 3rd quarter-hour. Average speed over the whole journey?

These ARE equal-TIME legs (each a quarter hour) — but with different distances, so use the master formula directly: Total distance = $8 + 6 + 16 = 30$ km, Total time = $3 \times 1/4$ h = 0.75 h. $\text{Average} = 30/0.75 = \mathbf{40 \text{ km/h}}$. (GATE Electrical 2013.)

A man covers 10 km at 5 km/h, 30 km at 6 km/h, 20 km at 10 km/h. Average speed?

Not equal distance, not equal time — use master formula. Total distance = 60 km. Total time = $10/5 + 30/6 + 20/10 = 2 + 5 + 2 = 9$ h. $\text{Average} = 60/9 = \mathbf{6.67 \text{ km/h}}$. (Accenture 2019 — the answer is 6.67 km/h, not a mean-shortcut value, since neither the equal-distance nor equal-time condition holds.)

A10.3 Relative Speed [Level: Medium]

Same direction:	$\text{Relative Speed} = \text{Speed1} - \text{Speed2} $	(they close the gap slowly)
Opposite directions:	$\text{Relative Speed} = \text{Speed1} + \text{Speed2}$	(they close the gap fast)

This single idea underlies every train, boat, and "two people/vehicles" problem below.

A10.4 Train Problems [Level: Medium-Hard]

Case	Formula
Train crosses a stationary point/pole/man	$\text{Time} = \text{Train Length} / \text{Train Speed}$
Train crosses a platform/bridge/tunnel	$\text{Time} = (\text{Train Length} + \text{Platform Length}) / \text{Train Speed}$
Two trains cross each other, OPPOSITE directions	$\text{Time} = (L_1 + L_2) / (\text{Speed1} + \text{Speed2})$
Two trains cross each other, SAME direction	$\text{Time} = (L_1 + L_2) / \text{Speed1} - \text{Speed2} $
Train crosses a walking/moving person	Relative speed = Train speed $\pm$ person's speed (– if same direction, + if opposite)

- TRICK for stoppages: if a train's speed WITHOUT stoppage is S_w and WITH stoppage (i.e., including halt time) is S_w' , the stoppage time per hour = $[(S_w - S_w') / S_w] \times 60$ minutes.

Worked Examples (verified)

Two trains start from stations A and B toward each other at 54 km/h and 72 km/h. At the meeting point, one train has travelled 80 m more than the other. Find the distance AB.

Convert: 54 km/h = 15 m/s, 72 km/h = 20 m/s. In time t , distances are $15t$ and $20t$; difference = $5t = 80 \Rightarrow t = 16$ s. Distance AB = $15t + 20t = 35 \times 16 = \mathbf{560 \text{ m}}$.

Two trains of equal length cross a stationary person in 6 s and 8 s respectively. Find the time for them to cross each other (a) moving in opposite directions, (b) moving in the same direction.

Let length = L . Speed1 = $L/6$, Speed2 = $L/8$.

(a) Opposite: relative speed = $L/6 + L/8 = 7L/24$; combined length to clear = $2L$; time = $2L \div (7L/24) = 48/7$ s.

(b) Same direction: relative speed = $L/6 - L/8 = L/24$; time = $2L \div (L/24) = \mathbf{48 \text{ s}}$.

A10.5 Boats & Streams [Level: Medium]

Downstream Speed (D) = Boat speed in still water (x) + Stream speed (y)
 Upstream Speed (U) = Boat speed in still water (x) - Stream speed (y)

Recovering x and y from D and U:

$$\begin{aligned}\text{Boat speed in still water (x)} &= (D + U) / 2 \\ \text{Stream speed (y)} &= (D - U) / 2\end{aligned}$$

- TRAP:** This is a plain arithmetic mean/half-difference — NOT the harmonic-mean situation from A10.2. Downstream and upstream speeds are simple sums/differences of two underlying rates, not two different average speeds over the same distance.
- Round-trip average speed** (same distance covered downstream and upstream): THIS case IS the equal-distance harmonic-mean scenario from A10.2 — **Average speed for round trip = $2 \times D \times U / (D + U)$** .

Worked Examples

A boat rows downstream at 14 km/h and upstream at 5 km/h. Find the boat's speed in still water.

$$x = (14 + 5) / 2 = \mathbf{9.5 \text{ km/h.}}$$

A boat's speed in still water is 12 km/h; the stream flows at 5 km/h. It takes 8 hours for a round trip between two points. Find the distance between them.

Downstream = 17 km/h, Upstream = 7 km/h. Let distance = d: $d/17 + d/7 = 8 \Rightarrow d(7+17)/(119) = 8 \Rightarrow d \times 24/119 = 8 \Rightarrow d = 8 \times 119/24 = \mathbf{39.67 \text{ km.}}$

A boat covers 20 km upstream + 30 km downstream in 3 hours, and 10 km upstream + 20 km downstream in 5/3 hours. Find the sum of the boat's speed in still water and the stream's speed.

Let upstream speed = u, downstream = d. Equations: $20/u + 30/d = 3$ and $10/u + 20/d = 5/3$. Doubling the second: $20/u + 40/d = 10/3$. Subtracting the first: $10/d = 10/3 - 3 = 1/3 \Rightarrow d = 30$. Substituting back: $10/u + 20/30 = 5/3 \Rightarrow 10/u = 1 \Rightarrow u = 10$. Boat speed = $(10 + 30)/2 = 20$, stream speed = $(30 - 10)/2 = 10$. **Sum = 30 km/h.**

Questions

(Easy → Hard; [Trick] rewards the shortcut, [Trap] targets a common misconception)

Q1. [Easy] Convert 90 km/h to m/s.

A. 20 m/s B. 25 m/s C. 30 m/s D. 15 m/s

Answer: B. **Why:** $90 \times 5/18 = \mathbf{25 \text{ m/s.}}$

Q2. [Easy] A train 120 m long crosses a pole in 6 seconds. Find its speed in km/h.

A. 60 km/h B. 72 km/h C. 80 km/h D. 20 km/h

Answer: B. **Why:** Speed = $120/6 = 20 \text{ m/s} = 20 \times 18/5 = \mathbf{72 \text{ km/h.}}$

Q3. [Medium] [Trap] A man travels from A to B at 40 km/h and returns at 60 km/h. Find his average speed for the whole journey.

A. 50 km/h B. 48 km/h C. 52 km/h D. 45 km/h

Answer: B. **Why:** Equal distance both ways $\Rightarrow$ harmonic mean: $2 \times 40 \times 60 / (40 + 60) = 4800/100 = \mathbf{48 \text{ km/h}}$ — NOT the simple average 50 km/h, which is the classic trap here.

Q4. [Medium] A train 150 m long crosses a platform 250 m long in 20 seconds. Find the train's speed in km/h.

A. 54 km/h B. 60 km/h C. 72 km/h D. 45 km/h

Answer: C. **Why:** Total distance = $150 + 250 = 400 \text{ m}$ in $20 \text{ s} \Rightarrow 20 \text{ m/s} = \mathbf{72 \text{ km/h.}}$

Q5. [Medium] [Trick] Two trains 100 m and 150 m long run on parallel tracks in opposite directions at 30 km/h and 20 km/h. Find the time they take to cross each other.

A. 18 s B. 20 s C. 15 s D. 10 s

Answer: A. **Why:** Relative speed (opposite directions) = $30 + 20 = 50 \text{ km/h} = 50 \times 5/18 = 125/9 \text{ m/s}$. Total length = 250 m. Time = $250 \div (125/9) = 250 \times 9/125 = \mathbf{18 \text{ s.}}$

Q6. [Hard] A boat goes 24 km upstream in 6 hours and returns downstream in 4 hours. Find the speed of the boat in still water and the speed of the stream.

A. 5, 1 km/h B. 4, 1 km/h C. 5, 2 km/h D. 6, 1 km/h

Answer: A. **Why:** Upstream speed = $24/6 = 4 \text{ km/h}$. Downstream speed = $24/4 = 6 \text{ km/h}$. Boat speed = $(6 + 4)/2 = \mathbf{5 \text{ km/h}}$; stream speed = $(6 - 4)/2 = \mathbf{1 \text{ km/h.}}$

Q7. [Hard] [Trap] A car covers the first half of a journey at 40 km/h and the second half (by TIME, not distance) at 60 km/h. Find the average speed for the trip.

A. 48 km/h B. 50 km/h C. 52 km/h D. 46 km/h

Answer: B. **Why:** These are equal-TIME segments (not equal distance), so the plain arithmetic mean applies here: $(40 + 60)/2 = \mathbf{50 \text{ km/h}}$. Using the harmonic-mean formula here (as if distances were equal) would be the trap, giving the wrong value of 48.

Q8. [Hard] Two stations A and B are 300 km apart. A train starts from A at 7:00 AM at 50 km/h toward B; another starts from B at 8:00 AM at 100 km/h toward A. At what time do they meet?

A. 9:00 AM B. 9:40 AM C. 9:20 AM D. 10:00 AM

Answer: B. **Why:** By 8:00 AM, the first train has already covered 50 km, leaving a gap of $300 - 50 = 250 \text{ km}$, closing at a relative (opposite-direction) speed of $50 + 100 = 150 \text{ km/h}$. Time to close the gap = $250/150 = 5/3 \text{ h} = 1 \text{ h } 40 \text{ min}$. 8:00 AM + 1h40min = **9:40 AM**.

9.A11 Mixture & Alligation

A11.1 Weighted Average — The Idea Behind Alligation [Level: Basic]

- A **mixture** combines two or more ingredients (different prices/concentrations/speeds/rates). The mixture's overall value is a **weighted average**, pulled closer to whichever ingredient has the larger quantity.
- Alligation** is the shortcut rule that reverses this: given the mean (mixture) value and the two ingredient values, find the **RATIO** in which they were mixed — without setting up algebra from scratch every time.

A11.2 The Alligation Cross-Rule [Level: Basic — the single formula this whole chapter is built on]

Ingredient 1 value = d_1 Ingredient 2 value = d_2 ($d_1 < d_2$)
 Mean (mixture) value = m
 Quantity1 : Quantity2 = $(d_2 - m) : (m - d_1)$

- Ways to visualize it (the "cross diagram"):** write d_1 and d_2 on the left, m in the middle; draw diagonal lines from each to m ; the quantity ratio is read off the *far* differences: $(d_2 - m)$ pairs with ingredient 1, $(m - d_1)$ pairs with ingredient 2.
- TRICK: You never need to know the actual quantities to find the **RATIO** — alligation gives you the ratio directly from just the three values (d_1 , d_2 , m).
- TRAP:** The ratio $(d_2 - m):(m - d_1)$ attaches the "far" difference to the "near" ingredient — a very common slip is pairing d_1 with $(m - d_1)$ instead of with $(d_2 - m)$. If unsure, sanity-check: the **CHEAPER/LOWER**-value ingredient must always get the **LARGER** share of the ratio when the mean is closer to it, and vice versa.

Worked Example (Basic)

Item A: Rs. 450/kg, Item B: Rs. 510/kg. Mixture costs Rs. 475/kg. Find A:B.
 Ratio = $(510 - 475):(475 - 450) = 35:25 = \mathbf{7:5}$. (Real TCS-exam question — verified.)

A11.3 Applications Beyond Price Mixing [Level: Medium]

The SAME cross-rule applies wherever a weighted average is formed — recognize these as alligation in disguise:

Context	"d1, d2" are	"m" is
Price mixing	Price of ingredient 1, 2	Mixture price
Average weight/marks	Group 1 avg, Group 2 avg	Combined avg
Average speed (equal time)	Speed 1, Speed 2	Average speed
Simple interest	Rate 1, Rate 2	Overall/mean rate
Profit or Loss %	Profit%/Loss% of part 1, 2	Overall Profit/Loss %

Worked Examples (Medium)

Boys' average weight = 40 kg, girls' average weight = 30 kg. Combined average of 80 students = 33 kg. Find the number of boys.
 Ratio boys:girls = $(m - d_{\text{girls}}):(d_{\text{boys}} - m) = (33 - 30):(40 - 33) = 3:7 \rightarrow \text{boys} = 80 \times 3/10 = \mathbf{24}$, girls = 56.
 (TRAP worth flagging: some sources mislabel which count — always re-derive from the cross-rule rather than trusting a bare memorized number; direct-equation check: $40x + 30(80 - x) = 33 \times 80 \rightarrow 10x = 240 \rightarrow x = 24$. Confirmed.)
 Rs. 1000 lent in two parts at 6% and 8% simple interest p.a.; yearly income = Rs. 75. Find the sum lent at 8%.
 Mean rate = $75/1000 \times 100 = 7.5\%$. Ratio (at 6%):(at 8%) = $(8 - 7.5):(7.5 - 6) = 0.5:1.5 = 1:3$. Total parts = 4 $\rightarrow$ sum at 8% = $1000 \times 3/4 = \mathbf{Rs. 750}$.

A11.4 Profit/Loss-Based Alligation [Level: Medium-Hard]

- Treat the profit%/loss% figures themselves as the "values" being alligated (loss is a *negative* profit% for sign purposes).

Worked Example

A shopkeeper sells 91 kg rice, part at 7% profit and rest at 33% profit, overall gain 25%. Find quantity sold at 7% profit.
 Ratio (at 7%):(at 33%) = $(33 - 25):(25 - 7) = 8:18 = 4:9$. Total parts = 13 $\rightarrow$ qty at 7% = $91 \times 4/13 = \mathbf{28 \text{ kg}}$. (Verified — matches real Accenture/Tech Mahindra PYQ.)
 Two items sold for a combined Rs. 1500 total, one at 3% loss, other at 7% profit, overall no-profit-no-loss (mean = 0%).
 Ratio (loss-item SP-share):(profit-item SP-share) = $(7 - 0):(0 - (-3)) = 7:3$. Total parts = 10 $\rightarrow$ Loss item SP = $1500 \times 7/10 = \mathbf{Rs. 1050}$; Profit item SP = $\mathbf{Rs. 450}$. (Verified.)

A11.5 Water/Dilution & "Sold at CP, Still Profits" Problems [Level: Hard]

- Pure water has value/concentration **0** and (as an ingredient) costs nothing — **TRAP:** forgetting to use 0 (not a blank) as water's "d" value in the cross-rule is the most common error here.
- Classic pattern: "mixture sold at Cost Price (of the pure item) still yields x% profit" — because water dilutes the cost but the mixture is sold at the undiluted price, the "profit" IS the alligation ratio of water:pure-item.

Worked Example

In what ratio must water be mixed with milk to gain 16.66% by selling the mixture at cost price (of pure milk)?
 Treat pure milk's value = 100 (i.e., CP), water's value = 0, mean = mixture's "effective value" = $100/(1 + 16.66\%) \rightarrow$ equivalently, ratio water:milk = Profit% : $(100 + \text{Profit}\%) = 16.66:116.66 \approx \mathbf{1:7}$... more precisely $16.66\% = 1/6$, so ratio = $(1/6)/(1 + 1/6)$ rescaled = 1:6 directly by the standard shortcut **water:milk = Gain% expressed as fraction, numerator:denominator**. (Verified against real GATE-CS PYQ answer 1:6.)

A11.6 Repeated Replacement / Removal Problems [Level: Hard]

- When a fraction of a mixture is repeatedly removed and replaced by pure water (same fraction each time, n times):

$$\text{Final quantity of original ingredient} = \text{Initial} \times (1 - \text{replaced_fraction})^n$$

- TRICK: This is a **geometric decay**, not linear — do not multiply the fraction by n ; raise $(1 - \text{fraction})$ to the power n .

Worked Example

A container has 81 litres of pure milk. $1/3$ of it is removed and replaced with water, and this is repeated 3 times. Find the final quantity of milk.
 Final milk = $81 \times (1 - 1/3)^3 = 81 \times (2/3)^3 = 81 \times 8/27 = \mathbf{24 \text{ litres}}$.

A11.7 Adding a Pure Ingredient to Change Concentration [Level: Hard]

- Special alligation case: adding MORE of one pure ingredient (not water) to shift the mean — solve via direct proportion since the "before" quantity of the OTHER ingredient stays fixed.

Worked Example

80 litres of a milk-water mixture is 70% milk. How much pure milk must be added to raise it to 95% milk?
 Water quantity is fixed = $80 \times 30\% = 24 \text{ L}$ (this never changes). At the end, water must be 5% of the new total: $24 = 0.05 \times \text{New Total} \rightarrow \text{New Total} = 480 \text{ L}$.
 Milk to add = $480 - 80 = \mathbf{400 \text{ litres}}$. (Verified — matches real TCS PYQ.)

Questions

(Easy $\rightarrow$ Hard; [Trick] rewards the shortcut, [Trap] targets a common misconception)

Q1. [Easy] In what ratio should tea worth Rs. 60/kg be mixed with tea worth Rs. 65/kg so the mixture is worth Rs. 62/kg?

A. 2:3 B. 3:2 C. 1:1 D. 3:5

Answer: B. Why: Ratio = $(65-62):(62-60) = 3:2$.

Q2. [Easy] [Trick] 20 kg of rice at Rs.20/kg is mixed with rice at Rs.24/kg to get a mixture worth Rs.22/kg. Find quantity of the second rice.

A. 15 kg B. 20 kg C. 25 kg D. 30 kg

Answer: B. Why: Ratio = $(24-22):(22-20) = 2:2 = 1:1$, so equal quantities $\rightarrow 20 \text{ kg}$.

Q3. [Medium] A vessel has milk and water in ratio 5:1. How much of the mixture must be replaced with pure water to make the ratio 1:1? (Vessel capacity 60 L)

A. 20 L B. 24 L C. 30 L D. 25 L

Answer: B. Why: Milk=50L, water=10L. Removing $x \text{ L}$ of mixture ($5/6$ milk, $1/6$ water) and replacing with pure water leaves Milk = $50 - 5x/6$. For a 1:1 ratio in 60L total, Milk must equal 30: $50 - 5x/6 = 30 \rightarrow 5x/6 = 20 \rightarrow \mathbf{x = 24 \text{ L}}$.

Q4. [Medium] [Trap] The average score of boys is 60 and girls is 80. The combined average of the class is 68. If there are 30 girls, find the number of boys.

A. 20 B. 30 C. 45 D. 50

Answer: C. Why: Ratio boys:girls = $(80-68):(68-60) = 12:8 = 3:2$. Girls=30 corresponds to "2 parts" $\rightarrow 1 \text{ part}=15 \rightarrow \text{boys} = 3 \times 15 = \mathbf{45}$. (TRAP: don't accidentally swap and compute boys as the "2" side.)

Q5. [Medium] [Trick] 40 litres of a mixture is 90% acid. How much water must be added to reduce the acid concentration to 60%?

A. 15 L B. 18 L C. 20 L D. 25 L

Answer: C. Why: Pure acid = 36 L (fixed). At 60% concentration, $36 = 0.6 \times \text{NewTotal} \rightarrow \text{NewTotal} = 60 \text{ L}$. Water added = $60-40 = \mathbf{20 \text{ L}}$.

Q6. [Hard] A shopkeeper mixes rice worth Rs. 18/kg with rice worth Rs. 24/kg, and sells the mixture at Rs. 23/kg, gaining 15%. If he used 40 kg of the Rs. 24/kg rice, how much of the Rs. 18/kg rice did he use?

A. 40 kg B. 60 kg C. 80 kg D. 100 kg

Answer: C. Why: Mean CP = $SP/1.15 = 23/1.15 = 20$. Ratio Q(18-rice):Q(24-rice) = $(24-20):(20-18) = 4:2 = 2:1$. Since 24-rice = 40 kg (the "1" part), 18-rice = **80 kg** (the "2" part). Check: $(80 \times 18 + 40 \times 24)/120 = 2400/120 = 20 = \text{mean CP} \checkmark$.

Q7. [Hard] [Trap] 729 mL of a mixture contains milk and water in ratio 7:2. How much water must be added to make the ratio 7:3?

A. 81 mL B. 91 mL C. 100 mL D. 72 mL

Answer: A. Why: Milk = $729 \times 7/9 = 567 \text{ mL}$ (fixed), Water = 162 mL. Need Milk:NewWater = 7:3 $\rightarrow \text{NewWater} = 567 \times 3/7 = 243 \text{ mL}$. Water to add = $243-162 = \mathbf{81 \text{ mL}}$.

Q8. [Hard] A container has milk only. 30% of it is replaced by water, and this process is repeated once more (30% of the new mixture replaced by water). What % of the original milk remains?

A. 40% B. 49% C. 51% D. 60%

Answer: B. Why: Remaining fraction = $(1-0.3)^2 = 0.7^2 = 0.49 = \mathbf{49\%}$.

Q9. [Hard] In what ratio must water be mixed with milk costing Rs. 40/L so that after selling the mixture at Rs. 40/L (i.e., at cost price of pure milk), a profit of 25% is made?

A. 1:3 B. 1:4 C. 1:5 D. 3:1

Answer: B. Why: Using the CP-sale-profit shortcut: water:milk = Profit% fraction : 1 = $(1/4):1 = \mathbf{1:4}$ (since 25% = 1/4).

9.A12 Data Interpretation

(DI tests no new formulas — it applies Percentage/Ratio/Average skills to tabular or graphical data. The real skill is reading the data source correctly under time pressure.)

A12.1 Core Framing [Level: Basic]

- Every DI question is really a Percentage, Ratio, or Average question wearing a table/graph costume.
- TRAP:** The #1 real-exam error in DI is **misreading which row/column/axis represents which category**, not miscalculating — always spend 5 extra seconds confirming what each axis/row/legend means before computing.
- TRICK:** DI answer options are usually far apart — use **approximation** (round values) to estimate rather than compute exactly; exact arithmetic wastes time when option-gaps make estimation sufficient.

A12.2 Table Graphs [Level: Basic]

- Data given as rows × columns (e.g., Sales of 5 companies over 5 years).
- Standard question types: find % change year-on-year, find ratio between two entities, find the average across a row/column, find which entity had max/min growth.
- Ways to solve "find % change from year X to Y": (1) Direct formula $(\text{New} - \text{Old}) / \text{Old} \times 100$; (2) if the numbers are "nice," recognize the fraction directly (e.g., 120 → 150 is a jump of $30/120 = 1/4 = 25\%$, spot this without long division).

A12.3 Bar Graphs [Level: Basic-Medium]

- Single bar set (one value per category) vs grouped/stacked bars (multiple values per category, e.g., 2 years of data per company side-by-side, or one stacked bar showing composition).
- TRAP:** In a STACKED bar, an individual segment's value is the DIFFERENCE between its top and the segment below it, not the segment's own bar-height from the axis — a common misread.

A12.4 Pie Charts [Level: Medium]

- A pie chart divides a total into sectors, each labeled either in **degrees** (out of 360°) or **percentage** (out of 100%).

Sector's actual value = $(\text{Sector's degrees} / 360) \times \text{Total}$
 Sector's actual value = $(\text{Sector's percent} / 100) \times \text{Total}$

- TRICK:** To compare two sectors' relative size directly, you often don't need the "Total" at all — just compare their degree/percentage values directly (ratio of sectors = ratio of their degrees = ratio of their percentages).
- TRAP:** If a question gives a MIX of some sectors in degrees and others in percentage on the same chart, convert everything to ONE unit first (percentage is usually easier: divide degrees by 3.6).

A12.5 Line Graphs [Level: Medium-Hard]

- Shows a trend over a continuous variable (usually time). Slope = rate of change.
- TRAP:** A line graph labeled "**Sales in year X**" (period-wise) is read completely differently from one labeled "**Cumulative sales up to year X**" — for a cumulative graph, the actual value FOR a given year is the DIFFERENCE between consecutive points, not the point itself. Always check the graph's title/label for "cumulative" before reading period values off it directly.
- Steepest segment = highest rate of change (not necessarily the highest absolute value point).

A12.6 General DI Speed Strategy [Level: Hard]

- Always identify the **base** of a percentage question freshly for each sub-question in a DI set — DI sets often ask several questions off the same data where the "base" (denominator) silently shifts from one sub-question to the next (e.g., "% of Company A's sales" vs "% of total sales of all companies" in consecutive questions).
- For "find the average of these 5 values" type questions, use the AP-shortcut $(\text{first} + \text{last}) / 2$ if the values happen to be in arithmetic progression — check for this before summing all 5.

Questions

(Data given in words since diagrams can't be embedded; Easy → Hard)

Q1. [Easy] A pie chart shows a company's expenses: Salary 40%, Rent 25%, Marketing 20%, Misc 15%. If total expense is Rs. 20 lakh, find the Marketing expense.

A. Rs. 3 lakh B. Rs. 4 lakh C. Rs. 5 lakh D. Rs. 6 lakh

Answer: B. Why: 20% of 20 lakh = **Rs. 4 lakh**.

Q2. [Easy] In the same pie chart, find the ratio of Salary expense to Rent expense.

A. 8:5 B. 5:8 C. 4:5 D. 8:3

Answer: A. Why: Ratio of sectors = ratio of percentages = $40:25 = 8:5$.

Q3. [Medium] A table shows a company's revenue (in Rs. crore): 2020: 120, 2021: 150, 2022: 180, 2023: 210. Find the % growth from 2020 to 2021, and from 2022 to 2023.

A. 25% both years B. 25% then 16.67% C. 20% then 25% D. 30% then 20%

Answer: B. Why: 2020→21: $(150 - 120) / 120 = 30/120 = 25\%$. 2022→23: $(210 - 180) / 180 = 30/180 = 16.67\%$. (Same absolute increase of 30 crore each year, but % growth shrinks because the base grows — **TRAP:** don't assume equal absolute increases mean equal % growth.)

Q4. [Medium] [Trap] A line graph shows "Cumulative units sold" as: Jan=100, Feb=250, Mar=430. Find units sold in March alone.

A. 430 B. 330 C. 180 D. 150

Answer: C. Why: Since the graph is CUMULATIVE, March's own units = Cumulative(Mar) – Cumulative(Feb) = $430 - 250 = 180$.

Q5. [Hard] A stacked bar for "Total Students" shows Boys+Girls. For School X, the bar reaches 80 at the Boys/Girls boundary and 100 at the top (Girls stacked above Boys). Find the number of Girls.

A. 80 B. 20 C. 100 D. 180

Answer: B. Why: Girls = Top of stack – Boundary = $100 - 80 = 20$ (the segment's own height, not its top reading).

Q6. [Hard] Company revenues in a table (Rs. crore): A=200, B=250, C=150, D=400, Total of all 4 = 1000. A pie chart of the SAME data shows company D's sector at 144°. Verify if the pie chart is consistent with the table, and find D's percentage share.

A. 40%, consistent B. 36%, consistent C. 40%, inconsistent D. 36%, inconsistent

Answer: A. Why: D's actual value/total = $400/1000 = 40\%$. Degrees implied by table = $40\% \times 360^\circ = 144^\circ$, which **matches** the given pie chart — **consistent**, D's share = **40%**.

9.A13 Algebra (Simplification & Identities)

A13.1 Order of Operations [Level: Basic]

- BODMAS: Brackets -> Of -> Division -> Multiplication -> Addition -> Subtraction.
- TRAP:** "Of" is evaluated at the SAME priority as multiplication/division, left to right — NOT automatically before division. E.g., $10 - 2 \text{ of } 3 + 4/2 = 10 - 6 + 2 = 6$, not something else from misordering "of" ahead of everything.
- TRAP:** Division and multiplication have EQUAL precedence and are evaluated left-to-right (not "always divide before multiply") — $8/2 \times 2 = (8/2) \times 2 = 8$, a frequently miscalculated viral-style question.

A13.2 Core Algebraic Identities [Level: Basic]

$$\begin{aligned}(a+b)^2 &= a^2 + 2ab + b^2 \\ (a-b)^2 &= a^2 - 2ab + b^2 \\ a^2 - b^2 &= (a+b)(a-b) \\ (a+b)^3 &= a^3 + 3a^2b + 3ab^2 + b^3 = a^3 + b^3 + 3ab(a+b) \\ (a-b)^3 &= a^3 - 3a^2b + 3ab^2 - b^3 = a^3 - b^3 - 3ab(a-b) \\ a^3 + b^3 &= (a+b)(a^2 - ab + b^2) \\ a^3 - b^3 &= (a-b)(a^2 + ab + b^2)\end{aligned}$$

A13.3 The a+b+c=0 Shortcut [Level: Medium]

If $a + b + c = 0$, then: $a^3 + b^3 + c^3 = 3abc$

- TRICK: this single identity converts a messy cubic-sum question into one multiplication the instant you notice the three terms sum to zero — always check for this condition first when a question gives values that look designed to sum to 0.
- Worked example: $a=5, b=-3, c=-2$ (sum=0). $a^3+b^3+c^3 = 125-27-8 = 90$. Check: $3abc = 3 \times 5 \times (-3) \times (-2) = 90$. ✓

A13.4 Reciprocal Identities (find a^2+1/a^2 without finding a) [Level: Medium-Hard]

Given $a + 1/a = k$:

$$\begin{aligned}a^2 + 1/a^2 &= k^2 - 2 \\ a^3 + 1/a^3 &= k^3 - 3k\end{aligned}$$

Given $a - 1/a = k$:

$$\begin{aligned}a^2 + 1/a^2 &= k^2 + 2 \\ a^3 - 1/a^3 &= k^3 + 3k\end{aligned}$$

- TRICK: notice the sign flips between the two families — memorize by deriving once: $(a+1/a)^2 = a^2 + 2 + 1/a^2$, so $a^2 + 1/a^2 = (a+1/a)^2 - 2$.
- Worked example: If $a + 1/a = 5$, find $a^2 + 1/a^2$ and $a^3 + 1/a^3$.
 $a^2 + 1/a^2 = 5^2 - 2 = 23$.
 $a^3 + 1/a^3 = 5^3 - 3(5) = 125 - 15 = 110$.
- TRAP:** students often forget the $-3k$ term entirely and just cube k for the cubic case — always re-derive via $(a+1/a)(a^2+1/a^2-1) = a^3+1/a^3$ if in doubt.

A13.5 Linear Simultaneous Equations [Level: Medium]

- Ways to solve: (1) **Elimination** — multiply equations so a variable's coefficients match, subtract/add to cancel it; (2) **Substitution** — solve one variable in terms of the other from one equation, substitute into the second.
- TRICK for speed: when coefficients are large, check if the two equations can be added or subtracted directly to instantly cancel a variable, before doing any multiplication.

Questions

Q1. [Easy] Simplify: $12 + 6 \text{ of } 2 - 4/2$

A. 22 B. 8 C. 14 D. 10

Answer: A. Why: BODMAS: "of"=multiplication priority with division, left to right: $6 \text{ of } 2 = 12$; $4/2 = 2$. So $12 + 12 - 2 = 22$.

Q2. [Easy] If $a - b = 4$ and $a + b = 10$, find $a^2 - b^2$.

A. 40 B. 36 C. 44 D. 14

Answer: A. Why: $a^2 - b^2 = (a+b)(a-b) = 10 \times 4 = 40$.

Q3. [Medium] [Trick] If $x + y + z = 0$ and $x=3, y=-1$, find $x^3+y^3+z^3$.

A. 12 B. 18 C. -18 D. 6

Answer: B. Why: $z = -(x+y) = -2$. $x^3+y^3+z^3 = 3xyz = 3 \times 3 \times (-1) \times (-2) = 18$.

Q4. [Medium] If $a + 1/a = 4$, find $a^2 + 1/a^2$.

A. 14 B. 16 C. 18 D. 12

Answer: A. Why: $a^2 + 1/a^2 = 4^2 - 2 = 14$.

Q5. [Medium] [Trick] If $a - 1/a = 3$, find $a^2 + 1/a^2$.

A. 9 B. 7 C. 11 D. 13

Answer: C. Why: For $a - 1/a = k$, $a^2 + 1/a^2 = k^2 + 2 = 9 + 2 = 11$.

Q6. [Hard] If $a + 1/a = 6$, find $a^3 + 1/a^3$.

A. 216 B. 198 C. 180 D. 174

Answer: B. Why: $a^3 + 1/a^3 = k^3 - 3k = 216 - 18 = 198$.

Q7. [Hard] [Trap] Simplify: $24 \div 4 \times 2 + 3^2 - 1$

A. 19 B. 12 C. 20 D. 14

Answer: C. Why: Left-to-right for $\div$ and $\times$: $24 \div 4 = 6$, $6 \times 2 = 12$. Then $3^2 = 9$. Total: $12 + 9 - 1 = 20$.

Q8. [Hard] Solve for x, y: $2x + 3y = 13$, $3x - 2y = 0$.

A. $x=2, y=3$ B. $x=3, y=2$ C. $x=2, y=4.5$ D. $x=4, y=1.67$

Answer: A. Why: From eq2: $3x=2y \rightarrow y=1.5x$. Substitute into eq1: $2x+3(1.5x)=13 \rightarrow 2x+4.5x=13 \rightarrow 6.5x=13 \rightarrow x=2$, $y=1.5 \times 2=3$. Check eq2: $3(2)-2(3)=6-6=0 \checkmark$.

Q9. [Hard] [Trick] If $x^2 + 1/x^2 = 47$, find $x + 1/x$ (positive value).

A. 6 B. 7 C. 8 D. 9

Answer: B. Why: $(x+1/x)^2 = x^2 + 1/x^2 + 2 = 47+2 = 49 \rightarrow x+1/x = 7$ (taking positive root).

9.A14 Probability

A14.1 Basic Definitions [Level: Basic]

- **Experiment:** an action with an uncertain outcome. **Sample Space (S):** set of all possible outcomes. **Event:** a subset of the sample space.

$$\text{Probability } P(A) = (\text{Number of Favorable Outcomes}) / (\text{Total Number of Outcomes})$$

$$0 \leq P(A) \leq 1$$

- **Sure event:** $P = 1$ (certain to happen). **Impossible event:** $P = 0$ (cannot happen).

A14.2 Complementary Events — The Most Useful Shortcut [Level: Basic-Medium]

$$P(\text{not } A) = 1 - P(A)$$

- TRICK: for any "at least one" question, NEVER compute the "at least one" case directly (it usually has many sub-cases). Instead compute:

$$P(\text{at least one}) = 1 - P(\text{none})$$

- Worked example: 3 coins tossed. Find $P(\text{at least one head})$.
 $P(\text{no heads}) = P(\text{all tails}) = (1/2)^3 = 1/8$. $P(\text{at least one head}) = 1 - 1/8 = 7/8$.

A14.3 Mutually Exclusive, Exhaustive & Independent Events [Level: Medium]

Term	Meaning	Formula
Mutually Exclusive	Cannot both occur in one trial	$P(A \text{ or } B) = P(A) + P(B)$
Exhaustive	Together cover the entire sample space	Sum of P over all = 1
Independent	Outcome of one doesn't affect the other	$P(A \text{ and } B) = P(A) \times P(B)$

- **TRAP:** Mutually exclusive and independent are **NOT the same**, and in fact are almost opposites: if A and B are mutually exclusive with $P(A) > 0$ and $P(B) > 0$, then $P(A \text{ and } B) = 0$ (they can't both happen) — but independence requires $P(A \text{ and } B) = P(A) \times P(B)$, which is nonzero. So two mutually exclusive events with nonzero probability can **never** be independent.

A14.4 Odds vs Probability [Level: Medium]

Quantity	Definition
Odds in favor	Favorable : Unfavorable
Odds against	Unfavorable : Favorable
Probability	Favorable : Total (Favorable + Unfavorable)

- **TRAP:** "Odds of 3 to 2 in favor" means $P = 3/(3+2) = 3/5$, **not** $3/2$. Always convert odds (a ratio of two parts) into probability (a fraction of the total) by adding the two parts for the denominator.

A14.5 Classical Problem Types [Level: Medium-Hard]

- **Dice:** 2 dice $\rightarrow$ 36 total outcomes. Sum frequencies: sum=7 is most frequent (6 ways); sums 2 and 12 are rarest (1 way each); frequency rises 1,2,3,4,5,6 from sum=2 to sum=7, then falls symmetrically 5,4,3,2,1 from sum=8 to 12.
- **Cards:** standard deck = 52 cards = 4 suits $\times$ 13 ranks. Face cards = 12 (K,Q,J $\times$ 4 suits). Red cards = 26, Black cards = 26. Know these counts cold — most card questions are direct favorable/total plug-ins once you have them memorized.
- **Coins:** n coins $\rightarrow 2^n$ total outcomes.
- **"53 Sundays in a leap year":** a leap year has 366 days = 52 weeks + 2 extra days. Each of the 52 full weeks contributes exactly one of each weekday, guaranteeing 52 Sundays; the 2 "extra" days determine which day(s) occur a 53rd time.

$$P(53 \text{ Sundays in a leap year}) = P(\text{a specific day is among the 2 extra days}) = 2/7$$

(The 2 extra days are always 2 CONSECUTIVE weekdays, e.g. Mon-Tue, Tue-Wed, etc. — 7 possible consecutive pairs, of which exactly 2 contain Sunday.)

- **TRAP:** For a **non-leap year** (365 days = 52 weeks + 1 extra day), the question becomes "P(53 of a given day)" = $1/7$, not $2/7$ — don't reuse the leap-year formula for a normal year.

Questions

Q1. [Easy] A single die is rolled. Find $P(\text{getting a number greater than 4})$.

A. $1/6$ B. $1/3$ C. $1/2$ D. $2/3$

Answer: B. **Why:** Favorable = {5,6} = 2 outcomes. $P = 2/6 = 1/3$.

Q2. [Easy] Two coins are tossed. Find $P(\text{exactly one head})$.

A. $1/4$ B. $1/2$ C. $3/4$ D. 1

Answer: B. **Why:** Outcomes: HH, HT, TH, TT. Exactly one head: HT, TH = $2/4 = 1/2$.

Q3. [Medium] [Trick] A bag has 5 red and 4 blue balls. Two balls are drawn at random. Find $P(\text{at least one red})$.

A. $5/6$ B. $7/9$ C. $4/9$ D. $8/9$

Answer: A. **Why:** $P(\text{no red}) = P(\text{both blue}) = (4C2)/(9C2) = 6/36 = 1/6$. $P(\text{at least one red}) = 1 - 1/6 = 5/6$.

Q4. [Medium] Odds in favor of an event are 4:3. Find the probability of the event.

A. $4/3$ B. $3/7$ C. $4/7$ D. $3/4$

Answer: C. **Why:** $P = \text{favorable}/(\text{favorable} + \text{unfavorable}) = 4/(4+3) = 4/7$.

Q5. [Medium] A card is drawn from a standard deck. Find $P(\text{getting a face card})$.

A. $3/13$ B. $1/13$ C. $4/13$ D. $12/52$ simplified to $3/13$

Answer: A. **Why:** Face cards = 12. $P = 12/52 = 3/13$.

Q6. [Hard] What is the probability that a leap year selected at random will contain 53 Sundays?

A. $1/7$ B. $2/7$ C. $3/7$ D. $1/2$

Answer: B. **Why:** Leap year = 366 days = 52 weeks + 2 extra days; $P(\text{a specific day among the 2 extra}) = 2/7$.

Q7. [Hard] [Trap] Two dice are rolled. Find $P(\text{sum} = 7)$.

A. $1/6$ B. $1/12$ C. $5/36$ D. $1/9$

Answer: A. **Why:** Sum=7 has 6 favorable combinations out of 36 total: (1,6)(2,5)(3,4)(4,3)(5,2)(6,1). $P = 6/36 = 1/6$.

Q8. [Hard] From a group of 6 men and 4 women, a committee of 3 is formed at random. Find $P(\text{at least 2 women})$.

A. $1/6$ B. $3/10$ C. $1/3$ D. $2/5$

Answer: C. **Why:** Total ways = $10C3 = 120$. Exactly 2 women+1 man = $4C2 \times 6C1 = 6 \times 6 = 36$. Exactly 3 women = $4C3 = 4$. Favorable = $36+4=40$. $P=40/120=1/3$.

Q9. [Hard] [Trap] Events A and B are mutually exclusive with $P(A)=0.3$ and $P(B)=0.4$. Are A and B independent?

A. Yes, always B. No — mutually exclusive events with nonzero probability can never be independent C. Cannot be determined D. Only if $P(A)=P(B)$

Answer: B. **Why:** Mutually exclusive means $P(A \text{ and } B)=0$, but independence requires $P(A \text{ and } B)=P(A)P(B)=0.12 \neq 0$ — contradiction, so they cannot be independent.

9.A15 Permutation & Combination

A15.1 Fundamental Counting Principle [Level: Basic]

- If one task can be done in m ways and a second independent task in n ways, both together can be done in $m \times n$ ways.

A15.2 Permutation vs Combination [Level: Basic]

$nPr = n! / (n-r)!$ [ORDER MATTERS – arrangement]
 $nCr = n! / (r! (n-r)!)$ [ORDER DOESN'T MATTER – selection]
 $nCr = nC(n-r)$ (symmetry property)
 Sum of nCr for $r=0$ to $n = 2^n$ (total number of subsets of an n -element set)

- TRAP:** use **P** when the question involves arranging/ranking/ordering (seating, passwords, race positions); use **C** when it's just selecting/grouping (forming a committee, choosing a team) with no internal order.

A15.3 Permutations with Repetition [Level: Basic-Medium]

- Unlimited repetition allowed:** r positions, n choices each $\rightarrow n^r$.
- No repetition:** nPr .
- Repeated letters in a word:** arrangements of a word with letter-count repeats $p_1, p_2, \dots = n! / (p_1! \times p_2! \times \dots)$.
Worked example: arrangements of "MISSISSIPPI" (11 letters: M=1, I=4, S=4, P=2) = $11! / (1!4!4!2!) = 34,650$.

A15.4 Grouping Tricks (Vowels/Consonants Together) [Level: Medium]

- TRICK:** to keep a set of letters together, treat that whole set as ONE block. Arrange (blocks + remaining individual letters), THEN separately arrange the letters within the block, and multiply the two counts.
Worked example: arrange letters of "TABLE" so all vowels (A,E) are together. Treat (AE) as 1 block + T,B,L = 4 units $\rightarrow 4!$ arrangements of units $\times 2!$ arrangements within the block = $24 \times 2 = 48$.

A15.5 Circular Permutations [Level: Medium-Hard]

n distinct objects around a circle (rotations considered identical): $(n-1)!$
 If clockwise and anticlockwise arrangements are also considered identical (necklace/bracelet): $(n-1)!/2$

- TRAP:** a circular TABLE arrangement uses $(n-1)!$ (rotation-only symmetry); a NECKLACE or BRACELET arrangement uses $(n-1)!/2$ (rotation AND flip symmetry) — mixing these up is the single most common circular-permutation error.

A15.6 Rank of a Word [Level: Hard]

- Algorithm:** for each letter position (left to right), count how many of the REMAINING available letters are alphabetically SMALLER than the actual letter used there; multiply that count by (number of letters remaining after this position!); sum these products across all positions; add 1 at the end.
- Worked example: find the rank of the word "CAB" (all distinct letters, alphabetically: A,B,C).
Position 1 (C): smaller unused letters = {A,B} = $2 \rightarrow 2 \times 2! = 4$.
Position 2 (A): remaining letters {A,B}; smaller than A = none = $0 \rightarrow 0 \times 1! = 0$.
Position 3 (B): remaining letter {B}; smaller = $0 \rightarrow 0 \times 0! = 0$.
Rank = $4+0+0+1 = 5$.
- TRAP:** if the word has REPEATED letters, divide each position's contribution by the factorial of repeat-counts among the remaining letters at that step (same repeated-permutation adjustment as A15.3).

A15.7 Other Special Applied Problems [Level: Hard]

Handshakes / round-robin matches among n people = $nC2 = n(n-1)/2$
 Diagonals of an n -sided polygon = $nC2 - n$
 Derangements (no element in its original position):
 $D(n) = n! \times [1 - 1/1! + 1/2! - 1/3! + \dots + (-1)^n / n!]$
 Quick estimate for large n : $D(n) \sim n!/e$ ($e \sim 2.718$)

- TRICK:** $nC2 = n(n-1)/2$ is the single most reused sub-formula in this entire chapter — handshakes, matches, line-intersections, diagonals all reduce to it.

Questions

Q1. [Easy] In how many ways can 5 people be seated in a row?

A. 60 B. 120 C. 24 D. 720

Answer: B. **Why:** $5! = 120$.

Q2. [Easy] How many 3-letter codes can be formed from A-Z if repetition is allowed?

A. 15600 B. 17576 C. 2600 D. $26^3 = 17576$

Answer: B. **Why:** $26^3 = 17,576$.

Q3. [Medium] In how many ways can the letters of "APPLE" be arranged?

A. 120 B. 60 C. 20 D. 24

Answer: B. **Why:** 5 letters with P repeated twice: $5!/2! = 120/2 = 60$.

Q4. [Medium] [Trick] How many ways can the letters of "APPLE" be arranged so that both P's are together?

A. 24 B. 12 C. 48 D. 60

Answer: A. **Why:** Treat "PP" as one block: units = (PP),A,L,E = 4 units $\rightarrow 4! = 24$ (no internal arrangement needed since both letters in the block are identical).

Q5. [Medium] In how many ways can a committee of 3 be chosen from 7 people?

A. 210 B. 35 C. 21 D. 42

Answer: B. Why: $7C3 = 7!/(3!4!) = 35$.

Q6. [Medium] [Trap] 6 people are to be seated around a circular table. How many distinct arrangements are possible?

A. 720 B. 120 C. 360 D. 24

Answer: B. Why: Circular (rotation-only symmetry): $(n-1)! = 5! = 120$.

Q7. [Hard] [Trap] In how many ways can 6 identical-looking beads be arranged to form a necklace?

A. 120 B. 60 C. 720 D. 24

Answer: B. Why: Necklace (rotation AND flip symmetry): $(n-1)!/2 = 5!/2 = 120/2 = 60$.

Q8. [Hard] Find the rank of the word "RANK" if all letters are distinct and arranged alphabetically (A,K,N,R).

A. 18 B. 19 C. 20 D. 21

Answer: C. Why: Position1 'R': letters smaller than R among {A,K,N,R} = {A,K,N} = 3 $\rightarrow 3 \times 3! = 18$. Position2 'A': remaining {A,K,N}; smaller than A = 0 $\rightarrow 0 \times 2! = 0$. Position3 'N': remaining {K,N}; smaller than N = {K} = 1 $\rightarrow 1 \times 1! = 1$. Position4 'K': remaining {K}; smaller = 0 $\rightarrow 0$. Sum = $18+0+1+0 = 19$; Rank = $19+1 = 20$.

Q9. [Hard] In how many ways can 8 people shake hands with each other exactly once?

A. 56 B. 28 C. 64 D. 36

Answer: B. Why: $8C2 = 8 \times 7/2 = 28$.

Q10. [Hard] How many diagonals does a hexagon (6 sides) have?

A. 9 B. 12 C. 15 D. 6

Answer: A. Why: Diagonals = $nC2 - n = 15 - 6 = 9$.

Q11. [Hard] [Trick] Find the number of derangements of 4 distinct objects (D(4)).

A. 6 B. 9 C. 12 D. 3

Answer: B. Why: $D(4) = 4![1 - 1/1! + 1/2! - 1/3! + 1/4!] = 24[1 - 1 + 0.5 - 0.1667 + 0.0417] = 24 \times 0.375 = 9$.

Q12. [Hard] How many 4-digit numbers (no repetition of digits) can be formed using digits 1-9 that are divisible by 5?

A. 224 B. 336 C. 448 D. 280

Answer: B. Why: No zero is available among digits 1-9, so the last digit must be 5 (the only multiple of 5 in the set). The remaining 3 positions are filled from the remaining 8 digits: $8P3 = 8 \times 7 \times 6 = 336$.

Q13. [Hard] How many ways can 3 boys and 2 girls be arranged in a row such that the 2 girls are always together?

A. 24 B. 48 C. 12 D. 120

Answer: B. Why: Treat 2 girls as 1 block + 3 boys = 4 units $\rightarrow 4! = 24$ arrangements of units $\times 2!$ arrangement within the block = $24 \times 2 = 48$.

9.A16 Quant Miscellaneous: Venn Diagrams, Age Problems & Partnership

A16.1 Venn Diagrams (Set Theory) [Level: Basic → Hard]

- This is the **quantitative/counting** use of Venn diagrams (contrast with the *logical-relationship* Venn diagrams used for classifying word-categories like "Diwali, Eid, Festival" — those test category containment/overlap conceptually, not counting).

$$\begin{aligned} \text{2-set: } |A \cup B| &= |A| + |B| - |A \cap B| \\ \text{3-set: } |A \cup B \cup C| &= |A| + |B| + |C| - |A \cap B| - |B \cap C| - |C \cap A| + |A \cap B \cap C| \end{aligned}$$

- TRAP: "Only A"** (exclusively A, not B) = $|A| - |A \cap B|$ — very commonly confused with just $|A|$ alone.
- TRICK for 3-region "only" problems: when you're told the sizes of pairwise-overlap regions (some of which may already EXCLUDE the triple-overlap, i.e., are "exactly two" regions, not "at least two"), always clarify whether a given overlap number is "exactly 2 categories" or "at least 2 (could include all 3)" — this distinction drives which subtraction to use.

Worked Example (Medium) — verified real exam question

In a survey: 10% are students+parents only, 10% are students+teachers+parents (all three), 15% are teachers+parents only, 35% are students+teachers only. Find % of only-teachers, only-parents, only-students (each group's total membership is 100%... i.e., each of the three named categories individually sums to 100% of ITS OWN circle when combined with its overlaps).

Only teachers = $100 - (35 + 10 + 15) = 40\%$. Only parents = $100 - (10 + 10 + 15) = 65\%$. Only students = $100 - (35 + 10 + 10) = 45\%$.

Worked Example (Hard) — verified real exam question

In a group, 6 speak Tamil, 15 speak Hindi, 6 speak Gujarati; no one speaks any other language. 2 people speak exactly two languages, 1 person speaks all three. Find total group size.

Only Tamil = $6 - 2 - 1 = 3$ (subtracting the 2-language and 3-language overlaps attributed here), Only Hindi = $15 - 2 - 1 = 12$, Only Gujarati = $6 - 1 = 5$ (assumed the two "2-language" speakers are Tamil+Hindi, not involving Gujarati, per the standard version of this question). Total = $(3 + 12 + 5)$ [only-one] + 2 [exactly-two] + 1 [all-three] = **23**.

A16.2 Age Problems [Level: Basic → Hard]

- Universal method: assign variables to PRESENT ages, translate every past/future condition into an equation, solve the linear system.
- TRAP (sign confusion): "**X years ago**" → present_age - X. "**After X years / X years hence**" → present_age + X. Getting this backwards is the single most common error in this topic — always re-read which direction the shift goes before writing the equation.
- Ratio-based ages: if "ratio of ages is a:b," let the ages be ax and bx (NOT a and b directly) — this preserves the ratio while giving you one variable to solve for.

Ways to Solve — Worked Examples (Basic → Medium, all hand-verified)

(1) Twice son's age + father's age = 70; twice father's age + son's age = 95. Find both.

$2S + F = 70$, $2F + S = 95$. Solving: **Father=40, Son=15**.

(2) Sum of father & son's ages = 90. 15 years ago, father's age was 3× son's age.

$F + S = 90$, $F - 15 = 3(S - 15)$. Solving: **Father=60, Son=30**.

(3) Sum of ages = 35, product of ages = 150.

$F + S = 35$, $F \times S = 150$ → quadratic $x^2 - 35x + 150 = 0$ → $x = 30$ or 5. **Father=30, Son=5**.

(4) I am 3× my son's age. After 5 years I'll be 2.5× his age. Find both present ages.

$F = 3S$, $F + 5 = 2.5(S + 5)$. Solving: **Father=45, Son=15**.

Worked Examples (Hard) — ratio-based

(5) Ratio of A & B's ages 2 years ago was 5:3. After 2 years (from now), the ratio will be 7:5. Find present ages.

$(A-2):(B-2) = 5:3$ → $3A = 5B - 4$. $(A+2):(B+2) = 7:5$ → $5A = 7B + 4$. Solving simultaneously: **A=12, B=8**.

- TRAP:** Watch for ambiguous phrasing like "one year ago the ratio was X; **after 3 years** the ratio will be Y" — "after 3 years" can mean 3 years from now OR 3 years from that past reference point. A well-posed question should make this unambiguous (e.g., say "3 years from now" explicitly); if a question seems to give a non-integer/inconsistent answer under one reading, try the other reading before concluding an error.

A16.3 Partnership [Level: Basic → Hard]

$$\text{Profit-sharing ratio} = C_1 \times T_1 : C_2 \times T_2 : \dots$$

(Capital invested × Time invested, for EACH partner)

- TRAP:** If all partners invest for the SAME duration, ratio simplifies to just capital ratio — but the moment durations DIFFER, you **must** weight by time; splitting profit by capital alone (ignoring time) is the most common partnership error.
- Working partner:** may receive a fixed management fee/commission off the top BEFORE the remaining profit is split in the capital×time ratio among all partners — watch for this two-step split in "working vs sleeping partner" questions.
- Ways to solve capital-withdrawal/addition mid-year: split each partner's investment into time-weighted segments and sum them (e.g., invests 5000 for 4 months then withdraws to 3000 for remaining 8 months → effective $C \times T = 5000 \times 4 + 3000 \times 8$).

Worked Example (Medium)

A invests Rs. 6000 for the full year; B invests Rs. 9000 but only for 8 months. Find their profit-sharing ratio.

$C \times T$: A = $6000 \times 12 = 72,000$. B = $9000 \times 8 = 72,000$. Ratio = **1:1** (equal shares, despite B investing more capital, because A invested for longer).

Questions

(Easy → Hard)

Q1. [Easy] In a class, 60% play cricket, 50% play football, 30% play both. What % play at least one game?

A. 70% B. 80% C. 90% D. 100%

Answer: B. Why: $|CuF| = 60 + 50 - 30 = 80\%$.

Q2. [Easy] [Trap] In Q1's class, what % play ONLY cricket (not football)?

A. 60% B. 30% C. 20% D. 40%

Answer: B. Why: Only cricket = $60 - 30 = 30\%$ — not the full 60%.

Q3. [Medium] A father's age is 4 times his son's age now. 5 years ago, the father was 7 times as old as his son was then. Find their present ages.

A. Father 32, Son 8 B. Father 28, Son 7 C. Father 40, Son 10 D. Father 36, Son 9

Answer: C. Why: $F = 4S$, $F - 5 = 7(S - 5) \rightarrow 4S - 5 = 7S - 35 \rightarrow 30 = 3S \rightarrow S = 10$, $F = 40$. Check: $F - 5 = 35 = 7 \times (S - 5) = 7 \times 5 = 35$ ✓.

Q4. [Medium] Ram is 3 times as old as Shyam. After 15 years, Ram will be twice as old as Shyam. Find Shyam's present age.

A. 10 B. 12 C. 15 D. 20

Answer: C. Why: $R = 3S$, $R + 15 = 2(S + 15) \rightarrow 3S + 15 = 2S + 30 \rightarrow S = 15$.

Q5. [Hard] A invests Rs. 2000 for 6 months, B invests Rs. 3000 for 4 months, and C invests Rs. 6000 for 3 months. Find C's share of a total profit of Rs. 700.

A. Rs. 200 B. Rs. 250 C. Rs. 300 D. Rs. 350

Answer: C. Why: $C \times T$: $A = 2000 \times 6 = 12000$, $B = 3000 \times 4 = 12000$, $C = 6000 \times 3 = 18000$. Ratio = $12:12:18 = 2:2:3$ (7 parts). C's share = $700 \times 3/7 = \text{Rs. } 300$.

Q6. [Hard] In a group, 10 speak Spanish, 20 speak German, and 8 speak Italian; no one speaks any other language. 3 people speak exactly Spanish+German (and no other), and 2 people speak all three languages. Find the total group size.

A. 28 B. 29 C. 31 D. 33

Answer: C. Why: Only Spanish = $10 - 3 - 2 = 5$. Only German = $20 - 3 - 2 = 15$. Only Italian = $8 - 2 = 6$ (no separate two-language-with-Italian group given). Only-one-language total = $5 + 15 + 6 = 26$. Total = $26 + 3$ (exactly-two) + 2 (all-three) = **31**.

9.A17 Sequence & Series, Statistics, Surds & Indices

A17.1 Arithmetic & Geometric Progressions [Level: Basic]

Arithmetic Progression (AP): common difference d is constant
 n th term = $a + (n-1)d$
 Sum of n terms = $n/2 [2a + (n-1)d] = n/2 \times (\text{first term} + \text{last term})$

Geometric Progression (GP): common ratio r is constant
 n th term = $a \cdot r^{(n-1)}$
 Sum of n terms = $a(r^n - 1)/(r - 1)$ [$r \neq 1$]

- TRICK: for an AP, Sum = (number of terms) $\times$ (average of first and last term) — faster than the full formula when you can spot first/last quickly (e.g., sum of 1 to 100 = $100 \times (1+100)/2 = 5050$, no need to invoke " $n/2[2a+(n-1)d]$ " explicitly).

A17.2 Statistics: Mean, Median, Mode [Level: Basic-Medium]

- **Mean** (raw data) = Sum of observations / Number of observations.
- **Median** = middle value when data is sorted (average of two middle values if count is even).
- **Mode** = most frequently occurring value.

Grouped data — Assumed Mean (Step-Deviation) shortcut:
 Mean = $A + h \times [\sum(f_i \cdot u_i) / \sum(f_i)]$
 where A = assumed mean (any convenient class mark), h = class width, $u_i = (x_i - A)/h$

Median (grouped) = $L + [(N/2 - CF) / f] \times h$
 L = lower boundary of median class, CF = cumulative frequency before median class,
 f = frequency of median class, h = class width

Mode (grouped) = $L + [(f_1 - f_0) / (2f_1 - f_0 - f_2)] \times h$
 f_1 = modal class frequency, f_0 = frequency of preceding class, f_2 = frequency of following class

- TRICK: use the step-deviation method (assumed mean + u_i substitution) whenever raw class-mark values are large — it converts big numbers into small integers, making hand calculation fast.

A17.3 Laws of Indices (Exponents) [Level: Basic — must be automatic]

$a^m \times a^n = a^{(m+n)}$
 $a^m / a^n = a^{(m-n)}$
 $(a^m)^n = a^{(mn)}$
 $a^0 = 1$ (for $a \neq 0$)
 $a^{(-n)} = 1/a^n$
 $a^{(1/n)}$ = n -th root of a
 $(ab)^n = a^n \cdot b^n$

A17.4 Surds — Simplification & Rationalization [Level: Medium]

- A **surd** is an irrational root, e.g. $\sqrt{2}$, $\sqrt{3}$.
- **Rationalization**: remove a surd from the denominator by multiplying by its conjugate.

$$1/(\sqrt{a} + \sqrt{b}) \times (\sqrt{a} - \sqrt{b})/(\sqrt{a} - \sqrt{b}) = (\sqrt{a} - \sqrt{b})/(a - b)$$

- **Comparing surds of different orders** (e.g., is $\sqrt[3]{5}$ bigger or $\sqrt{3}$?): raise both to a common index (LCM of the root orders), then compare the resulting integer powers.
 Example: compare $\sqrt[3]{5}$ (cube root) and $\sqrt{3}$ (square root). LCM(2,3)=6. $\sqrt[3]{5} = 5^{1/3} = 5^{2/6} = (5^2)^{1/6} = 25^{1/6}$. $\sqrt{3} = 3^{1/2} = 3^{3/6} = (3^3)^{1/6} = 27^{1/6}$.
 Since $27 > 25$, $\sqrt{3} > \sqrt[3]{5}$.

A17.5 Comparing Exponent Towers [Level: Hard]

- TRICK: to compare a^m vs b^n , try to express both with a **common exponent** by rewriting the base as a power, then compare the resulting bases directly.
- Worked example: Which is bigger, 2^{300} or 3^{200} ?

$2^{300} = (2^3)^{100} = 8^{100}$
 $3^{200} = (3^2)^{100} = 9^{100}$
 Since $9 > 8$, $3^{200} > 2^{300}$.

A17.6 Basic Arithmetic Speed Tricks [Level: Medium]

- **Squaring a number ending in 5**: for $N = (10a+5)$, $N^2 = a(a+1)$ followed by "25". Example: $65^2 \rightarrow a=6$, $a(a+1)=6 \times 7=42 \rightarrow$ answer **4225**.
- **Multiplying near a round base**: for numbers close to 100 (or another power of 10), use $(100-x)(100-y) = 10000 - 100(x+y) + xy$ — faster than direct long multiplication.

Questions

Q1. [Easy] Find the sum of the first 20 natural numbers.
 A. 200 B. 210 C. 190 D. 220

Answer: B. Why: Sum = $n(n+1)/2 = 20 \times 21/2 = 210$.

Q2. [Easy] Simplify: $2^5 \times 2^{-2}$

A. 8 B. 2 C. 16 D. 4

Answer: A. Why: $2^5 \times 2^{-2} = 2^{(5-2)} = 2^3 = 8$.

Q3. [Medium] Rationalize: $1/(\sqrt{7} - \sqrt{5})$

A. $(\sqrt{7}+\sqrt{5})/2$ B. $(\sqrt{7}-\sqrt{5})/2$ C. $\sqrt{7}+\sqrt{5}$ D. $(\sqrt{7}+\sqrt{5})/12$

Answer: A. Why: Multiply by $(\sqrt{7}+\sqrt{5})/(\sqrt{7}+\sqrt{5})$: denominator becomes $7-5=2$, giving $(\sqrt{7}+\sqrt{5})/2$.

Q4. [Medium] Find 85^2 .

A. 7225 B. 7025 C. 7125 D. 7325

Answer: A. Why: $a=8$, $a(a+1)=8 \times 9=72$, append 25 $\rightarrow$ **7225**.

Q5. [Medium] [Trick] Which is greater: $\sqrt[3]{5}$ or $\sqrt{3}$?

A. $\sqrt[3]{5}$ B. $\sqrt{3}$ C. Equal D. Cannot be determined

Answer: B. Why: Raise both to power 6 (LCM of 3,2): $(\sqrt[3]{5})^6=5^2=25$, $(\sqrt{3})^6=3^3=27$. Since $27>25$, $\sqrt{3}$ is greater.

Q6. [Hard] [Trick] Which is greater: 2^{300} or 3^{200} ?

A. 2^{300} B. 3^{200} C. Equal D. Cannot be determined

Answer: B. Why: $2^{300}=(2^3)^{100}=8^{100}$; $3^{200}=(3^2)^{100}=9^{100}$. Since $9>8$, 3^{200} is greater.

Q7. [Hard] The mean of 10 observations is 24. If one observation of value 33 is removed, find the new mean.

A. 22 B. 23 C. 24 D. 25

Answer: B. Why: Original total = $24 \times 10 = 240$. New total = $240 - 33 = 207$, new count = 9. New mean = $207/9 = 23$.

Q8. [Hard] Find the sum: $3 + 6 + 9 + \dots + 300$.

A. 15150 B. 14850 C. 15000 D. 15300

Answer: A. Why: AP with $a=3, d=3$, last term=300 $\rightarrow n = (300-3)/3+1=100$. Sum = $n \times (\text{first} + \text{last})/2 = 100 \times 303/2 = 15150$.

9.A18 Mensuration & Geometry

A18.1 2D Mensuration Reference [Level: Basic]

Shape	Area	Perimeter
Square (side a)	a^2	$4a$
Rectangle (l,b)	$l \times b$	$2(l+b)$
Triangle (base b, height h)	$(1/2) \times b \times h$	sum of sides
Triangle (Heron's, sides a,b,c, $s=(a+b+c)/2$)	$\sqrt{s(s-a)(s-b)(s-c)}$	$a+b+c$
Equilateral triangle (side a)	$(\sqrt{3}/4) a^2$	$3a$
Circle (radius r)	πr^2	$2\pi r$
Parallelogram (base b, height h)	$b \times h$	$2(\text{sum of adjacent sides})$
Rhombus (diagonals d1,d2)	$(1/2) d_1 d_2$	$4 \times \text{side}$
Trapezium (parallel sides a,b, height h)	$(1/2)(a+b) h$	sum of sides

A18.2 3D Mensuration Reference [Level: Basic-Medium]

Solid	Volume	Curved/Lateral Surface Area	Total Surface Area
Cube (side a)	a^3	$4a^2$	$6a^2$
Cuboid (l,b,h)	lbh	$2h(l+b)$	$2(lb+bh+hl)$
Cylinder (r,h)	$\pi r^2 h$	$2\pi r h$	$2\pi r(h+r)$
Cone (r,h, slant $l=\sqrt{r^2+h^2}$)	$(1/3) \pi r^2 h$	$\pi r l$	$\pi r(l+r)$
Sphere (r)	$(4/3) \pi r^3$	—	$4\pi r^2$
Hemisphere (r)	$(2/3) \pi r^3$	$2\pi r^2$	$3\pi r^2$

- TRAP:** "Curved/Lateral Surface Area" excludes the flat base(s); "Total Surface Area" includes them. A question asking for the cost of painting only the curved surface of a cylindrical pillar needs the CSA, not TSA — misreading this is the #1 error in this chapter.

A18.3 Circle Theorems [Level: Medium]

- Angle in a semicircle = 90° (angle subtended by a diameter at any point on the circle).
- Angle subtended by an arc at the center = $2 \times$ angle subtended by the same arc at the circumference.

A18.4 Pythagorean Triples (Speed Trick) [Level: Medium]

- TRICK: memorize these common triples (and their multiples) to instantly solve right-triangle problems without computing square roots:

3-4-5 5-12-13 8-15-17 7-24-25
(and multiples: 6-8-10, 9-12-15, 10-24-26, etc.)

Questions

Q1. [Easy] Find the area of a rectangle with length 12 cm and breadth 7 cm.

A. 19 cm² B. 84 cm² C. 38 cm² D. 96 cm²

Answer: B. **Why:** Area = $l \times b = 12 \times 7 = 84$.

Q2. [Easy] [Trick] A right triangle has legs 9 cm and 12 cm. Find the hypotenuse.

A. 13 cm B. 15 cm C. 21 cm D. 18 cm

Answer: B. **Why:** 9-12-15 is $3 \times$ the 3-4-5 triple $\rightarrow$ hypotenuse = 15.

Q3. [Medium] Find the area of an equilateral triangle with side 8 cm.

A. $16\sqrt{3}$ cm² B. $32\sqrt{3}$ cm² C. $8\sqrt{3}$ cm² D. $24\sqrt{3}$ cm²

Answer: A. **Why:** Area = $(\sqrt{3}/4)a^2 = (\sqrt{3}/4)(64) = 16\sqrt{3}$.

Q4. [Medium] [Trap] Find the curved surface area of a cone with radius 7 cm and slant height 10 cm. (Use $\pi=22/7$)

A. 220 cm² B. 440 cm² C. 154 cm² D. 374 cm²

Answer: A. **Why:** CSA = $\pi r l = (22/7)(7)(10) = 220$ cm² (NOT total surface area, which would add $\pi r^2 = 154$ more).

Q5. [Medium] Find the volume of a cylinder with radius 7 cm and height 10 cm. (Use $\pi=22/7$)

A. 1540 cm³ B. 3080 cm³ C. 440 cm³ D. 770 cm³

Answer: A. **Why:** Volume = $\pi r^2 h = (22/7)(49)(10) = 1540$ cm³.

Q6. [Hard] Find the area of a triangle with sides 13, 14, 15 cm (use Heron's formula).

A. 84 cm² B. 91 cm² C. 75 cm² D. 98 cm²

Answer: A. **Why:** $s=(13+14+15)/2=21$. Area = $\sqrt{21(21-13)(21-14)(21-15)} = \sqrt{21 \times 8 \times 7 \times 6} = \sqrt{7056} = 84$.

Q7. [Hard] [Trap] A hemispherical bowl of radius 21 cm is to be painted only on the outer curved surface. Find the area to be painted. ($\pi=22/7$)

A. 1386 cm² B. 2772 cm² C. 4158 cm² D. 924 cm²

Answer: B. **Why:** Curved surface area of hemisphere = $2\pi r^2 = 2 \times (22/7) \times 441 = 2772$ cm² (total surface area $3\pi r^2 = 4158$ would incorrectly include the flat circular base).

Q8. [Hard] The diagonals of a rhombus are 16 cm and 12 cm. Find its area.

A. 96 cm² B. 192 cm² C. 48 cm² D. 144 cm²

Answer: A. **Why:** Area = $(1/2)(d_1)(d_2) = (1/2)(16)(12) = 96$.

Q9. [Hard] A ladder 25 m long is placed against a wall such that its foot is 7 m from the wall's base. How high up the wall does the ladder reach?
A. 24 m B. 18 m C. 20 m D. 22 m

Answer: A. **Why:** 7-24-25 is a Pythagorean triple $\rightarrow$ height = 24 m.

9.A19 Speed Mathematics (Fast Calculation Tricks)

A19.1 Square Root of a Perfect Square [Level: Basic]

- TRICK: (1) Pair digits from the right to estimate the digit-count of the root. (2) Use the unit digit of the square to narrow the possible unit digit of the root using this table:

Square ends in:	1	4	5	6	9	0
Root ends in:	1/9	2/8	5	4/6	3/7	0

(3) Bracket the leading digit(s) using nearby known squares (e.g., $20^2=400$, $30^2=900$) to pick between the two unit-digit candidates.

- Worked example: $\sqrt{7056}$. Pairs: "70" "56" $\rightarrow$ 2-digit root. Last digit of 7056 is 6 $\rightarrow$ root ends in 4 or 6. $80^2=6400$, $90^2=8100 \rightarrow$ root is between 80-90, closer to 84 (since 7056 is nearer 6400). Try 84: $84^2=7056$. $\sqrt{\text{Root}} = 84$.

A19.2 Square Root of an Imperfect Number (Approximation) [Level: Medium]

$$\sqrt{\text{N}} \sim R + (N - R^2) / (2R) \quad \text{where } R = \text{nearest lower perfect-square root}$$

- Worked example: $\sqrt{50}$. Nearest lower perfect square = 49 ($R=7$). $\sqrt{50} \approx 7 + (50-49)/(14) = 7 + 0.071 = \mathbf{7.071}$ (actual ≈ 7.0711 — very close).

A19.3 Fast Multiplication Near a Round Base [Level: Medium]

$$(\text{Base}+x)(\text{Base}+y) = \text{Base}^2 + \text{Base}(x+y) + xy$$

- Worked example: 104×107 (base=100, $x=4$, $y=7$): $100^2 + 100(11) + 28 = 10000 + 1100 + 28 = \mathbf{11128}$.
- Worked example (numbers below base): 97×96 (base=100, $x=-3$, $y=-4$): $10000 + 100(-7) + 12 = 10000 - 700 + 12 = \mathbf{9312}$.

A19.4 Clock Angle Formula [Level: Medium]

Angle between hour and minute hands = $|30H - 5.5M|$ degrees (H = hour 0-11, M = minutes)
If result > 180, subtract from 360 to get the smaller angle.

- Worked example: angle at 3:40. $|30(3) - 5.5(40)| = |90 - 220| = 130^\circ$.
- (See the dedicated Clocks chapter for the full theory behind this formula and additional clock problem types.)

A19.5 Counting Figures — General Approach [Level: Medium-Hard]

- TRICK: don't count triangles/squares by eye. Label every intersection point, systematically count how many ways base-lines/sub-triangles can combine using combinatorics (nCr -style counting per row/layer of the figure), and add contributions from each "size level" of shape separately.
- (See the dedicated Analytical Reasoning — Counting Figures chapter for the complete systematic method with worked examples.)

Questions

Q1. [Easy] Find $\sqrt{11025}$ using the digit-pattern method.

A. 105 B. 115 C. 95 D. 125

Answer: A. Why: Last digit 5 $\rightarrow$ root ends in 5. Pairs "1" "10" "25" $\rightarrow$ 3-digit root roughly, but 11025 is between $100^2=10000$ and $110^2=12100$, closer to $105^2=11025$. Check: $105^2=11025$ $\checkmark$.

Q2. [Easy] Multiply 98×97 using the base-100 method.

A. 9506 B. 9606 C. 9412 D. 9706

Answer: A. Why: Base=100, $x=-2$, $y=-3$: $10000 + 100(-5) + 6 = 10000 - 500 + 6 = 9506$.

Q3. [Medium] Approximate $\sqrt{65}$.

A. 7.8 B. 8.06 C. 8.5 D. 7.5

Answer: B. Why: Nearest lower perfect square = 64 ($R=8$). $\sqrt{65} \approx 8 + (65-64)/16 = 8 + 0.0625 = 8.06$.

Q4. [Medium] [Trick] Multiply 112×108 using the base method.

A. 12086 B. 12096 C. 12106 D. 12076

Answer: B. Why: Base=100, $x=12$, $y=8$: $10000 + 100(20) + 96 = 10000 + 2000 + 96 = 12096$.

Q5. [Medium] Find the angle between the hour and minute hands at 6:00.

A. 90° B. 180° C. 150° D. 120°

Answer: B. Why: $|30(6) - 5.5(0)| = |180 - 0| = 180^\circ$.

Q6. [Hard] Find the angle between the hour and minute hands at 7:20.

A. 90° B. 95° C. 100° D. 105°

Answer: C. Why: $|30(7) - 5.5(20)| = |210 - 110| = 100^\circ$.

Q7. [Hard] Find $\sqrt{9801}$ using the digit-pattern method.

A. 97 B. 99 C. 101 D. 93

Answer: B. Why: Last digit 1 $\rightarrow$ root ends in 1 or 9. 9801 is close to $100^2=10000$, so try 99: $99^2=9801$ $\checkmark$.

Q8. [Hard] Multiply 203×198 using the base-200 method.

A. 40114 B. 40214 C. 40194 D. 39994

Answer: C. Why: Base=200, $x=3$, $y=-2$: $200^2 + 200(1) + (3 \times -2) = 40000 + 200 - 6 = 40194$.

9. Part B — Engineering Mathematics

This block (linear algebra, calculus, trigonometry, vectors) is the mathematics ISRO expects from an engineering graduate rather than pure "aptitude" tricks — it overlaps with what you studied in first/second-year engineering maths. Questions here are usually direct formula application (matrix rank/eigenvalues, a standard limit, a basic integral) rather than puzzles, so they are high-value, low-ambiguity marks if you keep the core formulas fresh.

9.B1 Linear Algebra

B1.1 Matrix Basics & Special Matrix Types [Level: Basic]

- A **matrix** is a rectangular array of numbers arranged in rows and columns; an $m \times n$ matrix has m rows, n columns.
- Two matrices are **equal** only if they have the same order (dimensions) AND every corresponding entry matches.

Matrix Type	Defining Property
Row / Column matrix	Single row ($1 \times n$) / single column ($m \times 1$)
Square matrix	rows = columns ($n \times n$)
Diagonal matrix	Square, all off-diagonal entries = 0
Scalar matrix	Diagonal matrix with all diagonal entries equal
Identity matrix (I)	Scalar matrix with diagonal entries = 1
Zero / Null matrix	Every entry = 0
Symmetric matrix	$A^T = A$
Skew-symmetric matrix	$A^T = -A$ (diagonal entries must be 0)
Orthogonal matrix	$A^T A = I$ (so $A^{-1} = A^T$)
Idempotent matrix	$A^2 = A$
Involutory matrix	$A^2 = I$
Nilpotent matrix	$A^k = 0$ for some positive integer k

TRAP: Every diagonal element of a skew-symmetric matrix must be 0 (since $a_{ii} = -a_{ii}$ forces $a_{ii} = 0$) — a frequently tested one-liner.

B1.2 Matrix Addition, Multiplication & Transpose [Level: Basic-Medium]

Addition: $A + B$ defined only if A, B have the SAME order; add entry-wise.
 Multiplication: $(m \times n) \cdot (n \times p) = (m \times p)$ <- inner dimensions must match
 Matrix multiplication is NOT commutative in general: $AB \neq BA$

- TRAP:** AB may exist while BA doesn't (dimension mismatch), and even when both exist, $AB \neq BA$ in general.
- Transpose** A^T : rows and columns swapped. Key properties:

$(A^T)^T = A$
 $(A+B)^T = A^T + B^T$
 $(AB)^T = B^T \cdot A^T$ <- **TRAP:** order REVERSES, a classic trick question
 $(kA)^T = k \cdot A^T$

B1.3 Determinant [Level: Medium]

2x2: $\begin{vmatrix} a & b \\ c & d \end{vmatrix} = ad - bc$
 3x3 (expansion along row 1):

$$\begin{vmatrix} a & b & c \\ d & e & f \\ g & h & i \end{vmatrix} = a(ei - fh) - b(di - fg) + c(dh - eg)$$

- Properties: $\det(AB) = \det(A) \cdot \det(B)$; $\det(A^T) = \det(A)$; $\det(kA) = k^n \cdot \det(A)$ for an $n \times n$ matrix (**TRAP:** the k scales by k^n , NOT just k); $\det = 0 \Leftrightarrow$ matrix is **singular** (non-invertible, rows/columns linearly dependent); swapping any two rows/columns flips the sign of the determinant; if any row/column is all zeros or two rows/columns are identical, $\det = 0$.
- Ways to solve for a triangular matrix:** $\det =$ product of diagonal entries (huge shortcut — never expand a triangular matrix the long way).

B1.4 Cofactor, Adjoint & Inverse [Level: Medium]

- Cofactor** $C_{ij} = (-1)^{i+j} \times$ (minor obtained by deleting row i , column j).
- Adjoint** $\text{adj}(A)$ = transpose of the cofactor matrix.
- Inverse:**

$A^{-1} = \text{adj}(A) / \det(A)$ <- exists ONLY if $\det(A) \neq 0$
 Key identity: $A \cdot \text{adj}(A) = \det(A) \cdot I$

- TRAP:** A matrix with $\det(A) = 0$ has NO inverse at all — don't attempt $\text{adj}(A)/0$.

B1.5 Trace [Level: Medium]

$\text{trace}(A)$ = sum of diagonal entries
 $\text{trace}(A+B) = \text{trace}(A) + \text{trace}(B)$
 $\text{trace}(AB) = \text{trace}(BA)$ <- TRUE even though $AB \neq BA$ in general (non-obvious, worth memorizing)

B1.6 Rank of a Matrix [Level: Medium-Hard]

- **Rank** = number of linearly independent rows (equivalently, columns) = number of non-zero rows after reducing to row-echelon form via elementary row operations (Gauss elimination).
- $\text{rank}(A) \leq \min(\text{rows}, \text{columns})$. A square $n \times n$ matrix has **full rank n** $\Leftrightarrow$ it is invertible $\Leftrightarrow \det(A) \neq 0$.
- **Ways to solve:** (1) Row-reduce to echelon form and count non-zero rows [general method]. (2) For a square matrix, quickly check $\det(A) \neq 0$ to confirm full rank without row-reducing at all [shortcut].

B1.7 System of Linear Equations [Level: Hard]

For a system written as $Ax = b$, compare $\text{rank}(A)$ [coefficient matrix] with $\text{rank}([A|b])$ [augmented matrix]:

$\text{rank}(A)$ vs $\text{rank}([A b])$	vs number of unknowns n	Result
$\text{rank}(A) = \text{rank}([A b]) = n$	—	Unique solution (consistent)
$\text{rank}(A) = \text{rank}([A b]) < n$	—	Infinite solutions (consistent)
$\text{rank}(A) \neq \text{rank}([A b])$	—	No solution (inconsistent)

- **TRAP:** A **homogeneous** system ($Ax = 0$) is NEVER inconsistent — it always has at least the trivial solution $x=0$. It has a **non-trivial** solution $\Leftrightarrow \det(A) = 0$ ($\text{rank}(A) < n$).
- **Ways to solve:** (1) Gauss elimination — reduce to echelon form, back-substitute. (2) Cramer's Rule — $x_i = \det(A_i)/\det(A)$, only valid for a square system with $\det(A) \neq 0$ (replace column i of A with b to get A_i). (3) Matrix method — $x = A^{-1}b$, again only if $\det(A) \neq 0$.

B1.8 Eigenvalues & Eigenvectors [Level: Hard]

Definition: $A \cdot v = \lambda v$ ($v \neq 0$ is the eigenvector, λ the eigenvalue)
Characteristic equation: $\det(A - \lambda I) = 0$

- **Shortcuts (avoid solving the full characteristic polynomial for verification):**

Sum of eigenvalues = $\text{trace}(A)$
Product of eigenvalues = $\det(A)$
Eigenvalues of a triangular (or diagonal) matrix = its diagonal entries, directly

- TRICK: given a 2×2 matrix, you can find both eigenvalues instantly from trace and determinant by solving $\lambda^2 - \text{trace} \cdot \lambda + \det = 0$, without writing out $\det(A - \lambda I)$ longhand.

Questions

Q1. [Easy] For matrix $A = \begin{bmatrix} 2 & 3 \\ 1 & 4 \end{bmatrix}$, find $\det(A)$.

A. 5 B. 8 C. 11 D. -5

Answer: A. **Why:** $\det = (2 \times 4) - (3 \times 1) = 8 - 3 = 5$.

Q2. [Easy] Which of these matrices is always invertible?

A. A zero matrix B. A matrix with $\det(A) = 0$ C. A matrix with $\det(A) \neq 0$ D. A singular matrix

Answer: C. **Why:** Invertibility requires a non-zero determinant by definition.

Q3. [Medium] If A is a 3×3 matrix with $\det(A) = 5$, find $\det(2A)$.

A. 10 B. 20 C. 40 D. 5

Answer: C. **Why:** $\det(kA) = k^n \det(A)$ for $n \times n$ matrix; here $k=2, n=3 \rightarrow 2^3 \times 5 = 40$.

Q4. [Medium] [Trap] For matrices A (2×3) and B (3×2), which of the following is generally FALSE?

A. AB exists and is 2×2 B. BA exists and is 3×3 C. $AB = BA$ D. Both AB and BA can be computed

Answer: C. **Why:** Even when both AB and BA exist, they generally have different orders here (2×2 vs 3×3) and are never equal in general — matrix multiplication is not commutative.

Q5. [Medium] A matrix A satisfies $A^2 = A$. What type of matrix is A ?

A. Nilpotent B. Idempotent C. Involutory D. Orthogonal

Answer: B. **Why:** $A^2 = A$ is the defining property of an idempotent matrix.

Q6. [Medium] [Trick] If A is a 4×4 upper triangular matrix with diagonal entries 2, 3, 5, 7, find $\det(A)$.

A. 17 B. 210 C. 35 D. 12

Answer: B. **Why:** For a triangular matrix, $\det = \text{product of diagonal entries} = 2 \times 3 \times 5 \times 7 = 210$ — no cofactor expansion needed.

Q7. [Hard] A system $Ax=b$ has 3 equations, 3 unknowns, with $\text{rank}(A) = 2$ and $\text{rank}([A|b]) = 3$. What is true of the system?

A. Unique solution B. Infinite solutions C. No solution D. Only the trivial solution

Answer: C. **Why:** $\text{rank}(A) \neq \text{rank}([A|b]) \rightarrow$ inconsistent $\rightarrow$ no solution.

Q8. [Hard] A 3×3 matrix A has $\text{trace}(A) = 9$ and $\det(A) = 24$. If two of its eigenvalues are 2 and 3, find the third.

A. 4 B. 3 C. 6 D. 9

Answer: A. **Why:** Sum of eigenvalues = trace = 9 $\rightarrow 2+3+x=9 \rightarrow x=4$. (Check: product = $2 \times 3 \times 4 = 24 = \det(A)$ ✓, confirming consistency.)

Q9. [Hard] [Trap] For matrices A and B of compatible order, $(AB)^T$ equals:

A. $A^T B^T$ B. $B^T A^T$ C. $(BA)^T$ D. AB

Answer: B. **Why:** The transpose-of-a-product rule reverses order: $(AB)^T = B^T A^T$ — a very commonly reversed trap.

Q10. [Hard] A homogeneous system of 3 equations in 3 unknowns has a non-trivial solution. What must be true?

A. $\det(A) \neq 0$ B. $\det(A) = 0$ C. $\text{rank}(A) = 3$ D. The system is inconsistent

Answer: B. **Why:** A homogeneous system always has the trivial solution $x=0$; a non-trivial solution exists only when $\det(A)=0$ ($\text{rank}(A)<n$), making the coefficient matrix singular.

9.B2a Calculus — Limits, Continuity & Differentiability

B2a.1 Limits — Basics & Indeterminate Forms [Level: Basic]

- A **limit** describes the value a function *approaches* as the input approaches some point — the function need not actually be defined (or attain that value) at the point itself.
- 7 Indeterminate Forms** (direct substitution gives a form with no fixed value — must be resolved before you can conclude anything):

$0/0$	infinity/infinity	$\text{infinity} - \text{infinity}$
$0 \times \text{infinity}$	1^{infinity}	0^0 infinity^0

- TRAP:** A limit that is NOT one of these 7 forms is already resolved by direct substitution — do not apply L'Hopital's or factorization tricks unnecessarily.

B2a.2 Ways to Solve Limits [Level: Basic-Medium]

Five distinct methods — pick based on the form of the expression:

- Direct Substitution** — plug in the value; use when the result is NOT indeterminate.
- Factorization Method** — cancel a common factor causing 0/0 (typical for polynomial ratios), e.g. $\lim_{x \rightarrow 2} (x^2 - 4)/(x - 2) = \lim_{x \rightarrow 2} (x + 2) = 4$.
- Rationalization Method** — multiply by the conjugate to clear a surd causing 0/0.
- Standard/Notable Special Limits** — memorize this reference block cold, it resolves most "trick" limits instantly without any algebra:

$\lim_{x \rightarrow 0} \sin(x)/x = 1$	$\lim_{x \rightarrow 0} \tan(x)/x = 1$
$\lim_{x \rightarrow 0} (1 - \cos x)/x = 0$	$\lim_{x \rightarrow 0} (e^x - 1)/x = 1$
$\lim_{x \rightarrow 0} \ln(1+x)/x = 1$	$\lim_{n \rightarrow \text{infinity}} (1 + 1/n)^n = e$
$\lim_{x \rightarrow 0} (a^x - 1)/x = \ln(a)$	

- L'Hopital's Rule** — for 0/0 or infinity/infinity forms ONLY: differentiate numerator and denominator SEPARATELY (not as a quotient-rule derivative of the whole fraction), then re-evaluate; repeat if still indeterminate.
 - TRAP:** Applying L'Hopital's to a non-indeterminate expression gives a WRONG answer — always verify the form is 0/0 or ∞/∞ immediately before each application, including after each repeated round.

B2a.3 Continuity [Level: Medium]

- f is **continuous at $x=a$** iff all three agree:

Left-Hand Limit ($x \rightarrow a^-$) = Right-Hand Limit ($x \rightarrow a^+$) = $f(a)$

- Sum, product, and composition of continuous functions are continuous.
- Types of Discontinuity:**

Type	Cause	Example pattern
Removable	Limit exists but $\neq f(a)$, or $f(a)$ undefined ("a hole")	$f(x) = (x^2 - 1)/(x - 1)$ at $x = 1$
Jump	LHL $\neq$ RHL	A step function
Infinite/Essential	The limit itself is infinite	$f(x) = 1/x$ at $x = 0$

GATE CS 2015 (Set 2, 1 Mark) — worked

$f(x) = x^{-(1/3)}$. A = area bounded by $f(x)$ and the x -axis for x in $[-1, 1]$. Which statements are TRUE? (1) f is continuous in $[-1, 1]$. (2) f is not bounded in $[-1, 1]$. (3) A is nonzero and finite.

A. 2 only B. 3 only C. 2 and 3 only D. 1, 2 and 3

Answer: C. f is undefined/blows up at $x=0$ (a change of 1 in x can produce an unbounded change in $f(x)$) → **not continuous**, so (1) is false. $f \rightarrow \pm\infty$ near 0 → **unbounded**, (2) true. $A = 2 \int_0^1 x^{-(1/3)} dx = 2 \times [x^{(2/3)}/(2/3)]_0^1 = 2 \times (3/2) = 3$ (nonzero, finite) → (3) true.

GATE CS 2014 (Set 1, 2 Marks) — worked

f is continuous on $[0, 2]$, $f(0)=f(2)=-1$, $f(1)=1$. Which must be TRUE?

A. $\exists y \in (0, 1)$: $f(y)=f(y+1)$ B. $\forall y \in (0, 1)$: $f(y)=f(2-y)$ C. max value on $(0, 2)$ is 1 D. $\exists y \in (0, 1)$: $f(y)=-f(2-y)$

Answer: A. Define $g(y)=f(y)-f(y+1)$ on $[0, 1]$. $g(0)=f(0)-f(1)=-1-1=-2$ (negative). $g(1)=f(1)-f(2)=1-(-1)=2$ (positive). g is continuous (f is continuous), so by the **Intermediate Value Theorem** there must exist $y \in (0, 1)$ with $g(y)=0$, i.e. $f(y)=f(y+1)$. (B, C, D are not guaranteed by the given data alone.)

- TRICK: Whenever a GATE continuity question gives boundary values and asks "must be true," try constructing an auxiliary continuous function and applying the **Intermediate Value Theorem** — this is the standard technique behind this entire question family.

B2a.4 Differentiability [Level: Medium-Hard]

- Mathematical definition:** f is differentiable at $x=a$ if $\lim_{h \rightarrow 0} [f(a+h)-f(a)]/h$ exists (a finite value).
- Geometric definition:** a unique, well-defined tangent line exists at that point — no sharp corner, cusp, or vertical tangent.
- TRAP: Differentiability $\Rightarrow$ Continuity, but Continuity does NOT $\Rightarrow$ Differentiability.** Classic counter-example: $f(x)=|x|$ is continuous at $x=0$ but not differentiable there (sharp corner).
- Geometric red flags for non-differentiability: sharp corners/kinks, cusps, vertical tangents, and any point of discontinuity (which auto-disqualifies differentiability there).

GATE CS 2016 (Set 2, 1 Mark) — worked

$f(x)$ is a polynomial, $g(x)=f'(x)$. If degree of $(f(x)+f(-x))$ is 10, find degree of $(g(x)-g(-x))$.

Answer: 9. If f were odd, $f(x)+f(-x)=0$ (degree undefined/zero) — contradicts "degree 10." So **f must be even:** $f(x)=f(-x) \Rightarrow f'(x) = -f'(-x)$ (derivative of an even function is odd). With $g=f'$: $g(x)-g(-x) = f'(x)-(-f'(x)) = 2f'(x)$. Since f has even degree 10, f' has degree 9.

GATE CS 2014 (Set 1, 2 Marks) — worked

$f(x) = x \sin(x)$ satisfies $f'(x) + f(x) + t \cos(x) = 0$. Find t .

A. -1 B. -2 C. 1 D. 2

Answer: B. $f(x) = x \cos x + \sin x$. $f'(x) = -x \sin x + \cos x + \cos x = -x \sin x + 2 \cos x$. Substituting: $(-x \sin x + 2 \cos x) + (x \sin x) + t \cos x = 0 \rightarrow 2 \cos x + t \cos x = 0 \rightarrow \cos x(t+2)=0 \rightarrow t = -2$.

B2a.5 Rolle's Theorem & Mean Value Theorems [Level: Hard]

Rolle's Theorem: if f continuous on $[a,b]$, differentiable on (a,b) , and $f(a)=f(b)$, then there EXISTS c in (a,b) with $f'(c) = 0$.

Lagrange's Mean Value Theorem (LMVT): if f continuous on $[a,b]$, differentiable on (a,b) , then there EXISTS c in (a,b) with $f'(c) = [f(b)-f(a)] / (b-a)$.
(Geometrically: some point has a tangent PARALLEL to the secant line through the endpoints.)

Cauchy's Extended MVT: for two functions f, g satisfying the LMVT conditions on $[a,b]$ ($g'(x) \neq 0$), there exists c in (a,b) with $[f(b)-f(a)] / [g(b)-g(a)] = f'(c) / g'(c)$.

- TRAP:** Rolle's Theorem is just LMVT with $f(a)=f(b)$, forcing $f'(c)=0$ — recognize Rolle's as a **special case** of LMVT, not a separate unrelated result.
- Recognition pattern: any GATE question giving continuity+differentiability+boundary conditions and asking "must there exist a point where..." is almost always testing Rolle's/LMVT/IVT — construct the right auxiliary function.

Questions

(Easy → Hard; [Trick]/[Trap] tag shortcut/misconception-testing items; the 4 GATE PYQs above are reused here as Q9-Q12 with full verification)

Q1. [Easy] Evaluate $\lim_{x \rightarrow 0} \sin(3x)/x$.

A. 0 B. 1 C. 3 D. $1/3$

Answer: C. Why: Rewrite as $3 \times \sin(3x)/(3x)$; as $x \rightarrow 0$, $3x \rightarrow 0$ so $\sin(3x)/(3x) \rightarrow 1$, giving $3 \times 1 = 3$.

Q2. [Easy] Evaluate $\lim_{x \rightarrow 2} (x^2 - 4)/(x - 2)$.

A. 0 B. 2 C. 4 D. Does not exist

Answer: C. Why: Factor: $(x-2)(x+2)/(x-2) = x+2 \rightarrow$ at $x=2$, **4**.

Q3. [Medium] [Trick] Evaluate $\lim_{x \rightarrow 0} (e^{2x} - 1)/x$.

A. 0 B. 1 C. 2 D. e^2

Answer: C. Why: Using $\lim_{x \rightarrow 0} (e^{kx} - 1)/x = k$ (from the standard limit with substitution kx): here $k=2$, so answer = **2**.

Q4. [Medium] Evaluate $\lim_{x \rightarrow \infty} (3x^2 + 2x)/(x^2 + 5)$ using L'Hopital's Rule.

A. 0 B. 1 C. 3 D. ∞

Answer: C. Why: Form is ∞/∞ . Differentiate top & bottom: $(6x+2)/(2x) \rightarrow$ still ∞/∞ , differentiate again: $6/2 = 3$. (Faster: for equal-degree polynomials, the limit at ∞ is simply the ratio of leading coefficients, $3/1=3$.)

Q5. [Medium] [Trap] A student applies L'Hopital's Rule to $\lim_{x \rightarrow 0} (x+1)/(x+2)$. What is wrong?

A. Nothing, the answer is $1/2$ B. L'Hopital's cannot be applied since the form is NOT indeterminate (direct substitution gives $1/2$ immediately) C. The rule needs a trig function present D. L'Hopital's only works at $x \rightarrow \infty$

Answer: B. Why: Direct substitution gives $(0+1)/(0+2) = 1/2$ — a defined, non-indeterminate value; L'Hopital's Rule must never be invoked here (though coincidentally it would still give $1/2$, relying on it is conceptually wrong and fails on non-coincidental cases).

Q6. [Medium] Identify the type of discontinuity of $f(x) = (x^2 - 1)/(x - 1)$ at $x=1$ (with $f(1)$ separately defined as 5).

A. Jump B. Infinite C. Removable D. No discontinuity

Answer: C. Why: $\lim_{x \rightarrow 1} f(x) = \lim_{x \rightarrow 1} (x+1) = 2$, which EXISTS but $\neq f(1)=5$ — a "hole" that could be removed by redefining $f(1)=2$. This is the textbook **removable discontinuity**.

Q7. [Hard] [Trap] Is $f(x)=|x|$ differentiable at $x=0$?

A. Yes, since it is continuous there B. No — continuity does not guarantee differentiability; $|x|$ has a sharp corner at 0 C. Yes, its derivative is 1 D. No, because it is discontinuous at 0

Answer: B. Why: f is continuous at 0, but the left-hand derivative $(-1) \neq$ right-hand derivative $(+1)$ — a sharp corner, hence not differentiable, despite being continuous.

Q8. [Hard] A function f is continuous on $[1,5]$, differentiable on $(1,5)$, with $f(1)=f(5)=7$. By which theorem must some $c \in (1,5)$ have $f'(c)=0$?

A. Lagrange's MVT B. Rolle's Theorem C. Cauchy's Theorem D. L'Hopital's Rule

Answer: B. Why: Equal endpoint values ($f(1)=f(5)$) is exactly the extra condition that turns LMVT into **Rolle's Theorem**.

Q9. [Hard] $f(x) = x^{(-1/3)}$. A = area bounded by $f(x)$ and the x -axis for $x \in [-1,1]$. Which statements are TRUE? (1) f is continuous in $[-1,1]$. (2) f is not bounded in $[-1,1]$. (3) A is nonzero and finite. (GATE CS 2015, Set 2)

A. 2 only B. 3 only C. 2 and 3 only D. 1, 2 and 3

Answer: C. Why: f is undefined/unbounded at $x=0 \rightarrow$ not continuous (1 false), but unbounded (2 true); $A = 2 \int_0^1 x^{(-1/3)} dx = 2 \times (3/2) = 3$, finite and nonzero (3 true).

Q10. [Hard] f is continuous on $[0,2]$, $f(0)=f(2)=-1$, $f(1)=1$. Which must be TRUE? (GATE CS 2014, Set 1)

A. $\exists y \in (0,1): f(y)=f(y+1)$ B. $\forall y \in (0,1): f(y)=f(2-y)$ C. max value on $(0,2)$ is 1 D. $\exists y \in (0,1): f(y)=-f(2-y)$

Answer: A. Why: $g(y)=f(y)-f(y+1)$ is continuous, $g(0)=-2$, $g(1)=2 \rightarrow$ IVT guarantees a zero of g in $(0,1)$.

Q11. [Hard] f is a polynomial, $g=f$. If $\text{degree}(f(x)+f(-x))=10$, find $\text{degree}(g(x)-g(-x))$. (GATE CS 2016, Set 2)

A. 8 B. 9 C. 10 D. 11

Answer: B. Why: f must be even (else the sum would be 0), so $f'(x)=-f'(-x)$; $g(x)-g(-x)=2f'(x)$, degree = $10-1 = 9$.

Q12. [Hard] $f(x)=x \sin(x)$ satisfies $f''(x)+f(x)+t \cos(x)=0$. Find t . (GATE CS 2014, Set 1)

A. -1 B. -2 C. 1 D. 2

Answer: B. Why: $f'(x) = -x \sin x + 2 \cos x$; substituting and simplifying gives $x(t+2)=0$ for all x , so **$t=-2$** .

9.B2b Calculus — Maxima-Minima & Integration

B2b.1 Stationary/Critical Points [Level: Basic]

- A **stationary (critical) point** is where $f'(x) = 0$ (or f' is undefined) — a candidate for a local maximum, minimum, or an inflection/saddle point.
- Monotonic functions:** f is strictly increasing on an interval if $f'(x) > 0$ throughout; strictly decreasing if $f'(x) < 0$ throughout.

B2b.2 Ways to Classify a Critical Point [Level: Medium]

Two methods — use the First if the sign pattern is easy to read, the Second if you can differentiate again quickly:

1. **First Derivative Test:** examine the sign of f' just before and after the critical point.

f' changes $+$ $\rightarrow$ $-$ at $x=c$ $\Rightarrow$ local MAXIMUM at c
 f' changes $-$ $\rightarrow$ $+$ at $x=c$ $\Rightarrow$ local MINIMUM at c
 f' does not change sign $\Rightarrow$ NOT an extremum (inflection-type point)

2. **Second Derivative Test:** evaluate $f''(c)$.

$f''(c) < 0$ $\Rightarrow$ local MAXIMUM
 $f''(c) > 0$ $\Rightarrow$ local MINIMUM
 $f''(c) = 0$ $\Rightarrow$ INCONCLUSIVE — must fall back to the First Derivative Test (or a higher-order test)

- TRAP:** $f'(c)=0$ does **not** mean "no extremum" — it means the second-derivative test simply fails to decide; the point could still be a max, min, or inflection point (e.g., $f(x)=x^4$ at $x=0$ is a minimum despite $f'(0)=0$).

Worked Example

Find and classify the critical points of $f(x) = x^3 - 3x$.

$$f'(x) = 3x^2 - 3 = 0 \rightarrow x = \pm 1.$$

$$f''(x) = 6x. \quad f''(1) = 6 > 0 \rightarrow \text{local minimum at } x=1 \text{ (value } f(1) = 1-3 = -2).$$

$$f''(-1) = -6 < 0 \rightarrow \text{local maximum at } x=-1 \text{ (value } f(-1) = -1+3 = 2).$$

B2b.3 Integration — Basics [Level: Basic]

- Indefinite Integral:** the antiderivative family, $\int f(x)dx = F(x) + C$ (arbitrary constant C , since differentiation of a constant is 0).
- Definite Integral:** $\int$ from a to b of $f(x)dx = F(b) - F(a)$, a single NUMBER, no "+C".

Standard forms:

Integral of $x^n dx$	$= x^{(n+1)}/(n+1) + C$	($n \neq -1$)
Integral of $1/x dx$	$= \ln x + C$	
Integral of $e^x dx$	$= e^x + C$	
Integral of $\sin(x) dx$	$= -\cos(x) + C$	
Integral of $\cos(x) dx$	$= \sin(x) + C$	
Integral of $\sec^2(x) dx$	$= \tan(x) + C$	

- Properties of definite integrals** (very exam-useful):

Integral(a to b) $f(x)dx$ = - Integral(b to a) $f(x)dx$
 Integral(a to a) $f(x)dx$ = 0
 Integral(θ to a) $f(x)dx$ = Integral(θ to a) $f(a-x)dx$ <- powerful symmetry trick

- TRICK: The symmetry property above often converts a hard integral into one that, when ADDED to the original, cancels into something trivial — a classic GATE/JEE pattern: if $I = \int_0^a f(x)dx = \int_0^a f(a-x)dx$, then $2I = \int_0^a [f(x)+f(a-x)]dx$, which is frequently a constant.

B2b.4 Ways to Solve Integration Problems [Level: Medium-Hard]

Three principal techniques — choose based on the integrand's structure:

- Substitution Method (u-substitution):** for a composite function where one part's derivative appears elsewhere in the integrand. Set u = (inner function), rewrite entirely in terms of u , integrate, substitute back.
 - Example: $\int 2x \cos(x^2) dx \rightarrow$ let $u=x^2$, $du=2x dx \rightarrow \int \cos(u)du = \sin(u)+C = \sin(x^2)+C$.
- Partial Fractions Method:** for a rational function $P(x)/Q(x)$ — decompose the denominator's factors into simpler fractions with unknown constants, solve for those constants, then integrate each simple term (typically giving log or arctan forms).
 - Complication: a REPEATED linear factor $(x-a)^2$ needs TWO terms, $A/(x-a) + B/(x-a)^2$; an irreducible quadratic factor needs a linear numerator, $(Ax+B)/(x^2+bx+c)$ — forgetting these extra terms is the classic partial-fractions error.
- Integration by Parts:** $\int u \cdot dv = u \cdot v - \int v \cdot du$. Choose " u " (the part to differentiate) using the **LIATE** priority order — pick the first type present, top to bottom:

L - Logarithmic	($\ln x$, $\log x$)
I - Inverse Trig	($\sin^{-1} x$, $\tan^{-1} x$)
A - Algebraic	(x , x^2 , polynomials)
T - Trigonometric	($\sin x$, $\cos x$)
E - Exponential	(e^x , a^x)

- TRICK: LIATE tells you which factor to differentiate (u) — the higher item in the list becomes u , the rest becomes dv . Example: $\int x \ln(x) dx \rightarrow$ "L" beats "A", so $u=\ln(x)$ (differentiate), $dv=x \cdot dx$ (integrate).
- TRAP:** Forgetting "+C" on an INDEFINITE integral is marked wrong in most exams; conversely, writing "+C" on a DEFINITE integral (which evaluates to a plain number) is also wrong — know which one you're doing.

Worked Example (Integration by Parts)

Evaluate $\int x \cdot e^x dx$.

LIATE: "A" (x) beats "E" (e^x) $\rightarrow u=x$ ($du=dx$), $dv=e^x dx$ ($v=e^x$).
 $\int x \cdot e^x dx = x \cdot e^x - \int e^x dx = x \cdot e^x - e^x + C = e^x(x-1) + C$.

Questions

(Easy $\rightarrow$ Hard; [Trick]/[Trap] tag shortcut/misconception-testing items)

Q1. [Easy] Find the critical points of $f(x) = x^2 - 4x + 3$.

A. $x=1$ B. $x=2$ C. $x=3$ D. $x=4$

Answer: B. Why: $f'(x)=2x-4=0 \rightarrow x=2$.

Q2. [Easy] Classify the critical point of $f(x)=x^2-4x+3$ at $x=2$.

A. Maximum B. Minimum C. Inflection D. Cannot be determined

Answer: B. Why: $f''(x)=2 > 0$ (constant, always positive) $\rightarrow$ **minimum**.

Q3. [Medium] Evaluate $\int (3x^2 + 2x)dx$.

A. x^3+x^2+C B. $6x+2+C$ C. x^3+2x^2+C D. $3x^3+x^2+C$

Answer: A. Why: $\int 3x^2 dx = x^3$, $\int 2x dx = x^2 \rightarrow x^3 + x^2 + C$.

Q4. [Medium] [Trap] A student computes $\int_0^2 (3x^2)dx$ and writes the answer as " x^3+C evaluated" = $8+C$. What's wrong?

A. Nothing B. A definite integral yields a plain number — no "+C" belongs in the final answer, correct value is 8 C. The formula for x^2 integral is wrong D. Limits should be swapped

Answer: B. Why: A definite integral's arbitrary constant always cancels: $[x^3]_0^2 = 8 - 0 = 8$, with no C at all.

Q5. [Medium] Classify the critical point at $x=0$ for $f(x) = x^4$.

A. Maximum (via 2nd derivative test) B. Minimum, but the 2nd derivative test is inconclusive here ($f''(0)=0$) so the 1st derivative test is needed C. Not a critical point D. Inflection point

Answer: B. Why: $f'(x)=4x^3=0$ at $x=0$; $f''(x)=12x^2$, so $f''(0)=0$ — inconclusive by 2nd derivative test. First derivative test: f' is negative for $x<0$, positive for $x>0 \rightarrow$ sign change ($-$ to $+$) confirms a **minimum**.

Q6. [Medium] [Trick] Using LIATE, which part of $\int x^2 \cdot \sin(x) dx$ should be "u" (differentiated)?

A. $\sin(x)$ B. x^2 C. Either, it doesn't matter D. Neither, use substitution instead

Answer: B. Why: LIATE ranks Algebraic (x^2) above Trigonometric ($\sin x$), so $u=x^2$ is differentiated, $dv=\sin(x)dx$ is integrated.

Q7. [Hard] Evaluate $\int 2x \cdot e^{(x^2)} dx$ using substitution.

A. $e^{(x^2)}+C$ B. $2e^{(x^2)}+C$ C. $x^2 e^{(x^2)}+C$ D. $e^{(x^2)}/2 + C$

Answer: A. Why: Let $u=x^2$, $du=2x dx \rightarrow \int e^u du = e^u + C = e^{(x^2)} + C$.

Q8. [Hard] Decompose $1/[(x-1)(x-2)]$ using partial fractions.

A. $1/(x-1) + 1/(x-2)$ B. $1/(x-2) - 1/(x-1)$ C. $1/(x-1) - 1/(x-2)$ D. $-1/(x-1) - 1/(x-2)$

Answer: B. Why: $1/[(x-1)(x-2)] = A/(x-1) + B/(x-2)$. Clearing denominators: $1=A(x-2)+B(x-1)$. At $x=1$: $1=A(-1) \rightarrow A=-1$. At $x=2$: $1=B(1) \rightarrow B=1$. So the decomposition = $-1/(x-1) + 1/(x-2) = 1/(x-2) - 1/(x-1)$.

Q9. [Hard] Evaluate $\int x \cdot \ln(x) dx$ using integration by parts.

A. $(x^2/2)\ln(x) - x^2/4 + C$ B. $x \cdot \ln(x) - x + C$ C. $(x^2/2)\ln(x) + C$ D. $\ln(x)^2/2 + C$

Answer: A. Why: LIATE: $u=\ln(x)$ ($du=dx/x$), $dv=x dx$ ($v=x^2/2$). $\int x \ln(x) dx = (x^2/2)\ln(x) - \int (x^2/2)(1/x) dx = (x^2/2)\ln(x) - \int (x/2) dx = (x^2/2)\ln(x) - x^2/4 + C$.

Q10. [Hard] [Trick] Using the symmetry property, evaluate $I = \int_0^\pi x \cdot \sin(x)/(1+\cos^2 x) dx$, given that the same substitution ($x \rightarrow \pi-x$) transforms $\sin(x)/(1+\cos^2 x)$ into itself unchanged.

A. 0 B. $\pi^2/4$ C. π D. 2π

Answer: B. Why: Let $f(x)=\sin(x)/(1+\cos^2 x)$; $f(\pi-x)=f(x)$ (given). Then $I = \int_0^\pi x \cdot f(x) dx$ and also $I = \int_0^\pi \pi \cdot f(x) dx$ (by the symmetry property). Adding: $2I = \pi \int_0^\pi f(x) dx$. Since $\int_0^\pi \sin(x)/(1+\cos^2 x) dx = [-\arctan(\cos x)]_0^\pi = \arctan(1) - \arctan(-1) = \pi/4 - (-\pi/4) = \pi/2$, we get $2I = \pi \cdot (\pi/2) \rightarrow I = \pi^2/4$.

9.B2c Calculus — Numerical Methods

B2c.1 Foundations — Approximation, Error & Convergence [Level: Basic]

- Numerical methods give **approximate** answers — used when an exact solution is unavailable/difficult, only tabular/measured data exists, a computer must repeat calculations, or an answer is needed only to a stated accuracy (e.g., exact $\sqrt{2}$ vs numerical $\sqrt{2} \approx 1.4142$).
- Error sources: **Input** (data already approximate), **Round-off** (digits rounded during calculation), **Truncation** (an infinite process stopped after finite terms), **Discretisation** (continuous problem evaluated only at selected points), **Iteration** (process stopped before the true limiting value).

```
Let xt = true value, xa = approximate value.
True error:      Et = xt - xa
Absolute error:  |Et| = |xt - xa|
Relative error:  |xt - xa| / |xt|
Percentage error: Relative error x 100
When true value unknown, use consecutive iterates instead:
Approx. relative % error = |xn - xn-1| / |xn| x 100
```

- TRICK: Keep extra digits DURING calculation, round only the FINAL answer — early rounding is one of the most common sources of a "close but wrong" final answer in this whole chapter.
- Convergence & stopping criteria** (a safe algorithm uses an accuracy condition AND a max-iteration cap together): stop when $|x_n - x_{n-1}| < \text{tolerance}$, OR $|f(x_n)| < \text{tolerance}$, OR percentage error < permitted error, OR required decimal places stabilize, OR max iterations reached.

B2c.2 Root-Finding for Non-Linear Equations [Level: Basic-Medium]

- Linear equation: variable has power 1, no variables multiplied together (e.g., $2x+3=7$). Non-linear: power > 1 , or roots/exponentials/trig terms (e.g., $x^2-3x+1=0$, $e^x-x=0$).
- Bracketing (Intermediate Value) condition**: if f is continuous on $[a,b]$ and $f(a)f(b) < 0$, at least one root lies in (a,b) . **TRAP**: no sign change does NOT always mean no root exists — an even-multiplicity root can just touch the x -axis without crossing it.

Ways to solve $f(x)=0$ — four classic methods

1. Bisection Method (bracketing, most reliable, slowest)

```
Require f(a)f(b) < 0. Midpoint m=(a+b)/2.
If f(a)f(m)<0, new interval=[a,m]. If f(m)f(b)<0, new interval=[m,b]. If f(m)=0, m is exact root.
After n bisections: interval width = (b0-a0)/2^n ; max error of final midpoint = (b0-a0)/2^(n+1)
```

Use when reliability matters more than speed.

Worked example (GATE-style): $f(x)=x^3-x-2$, start $[1,2]$. Find the bracket after 4 iterations.

$f(1)=-2$, $f(2)=4$. $m_1=1.5000$, $f(m_1)<0 \rightarrow [1.5000, 2]$. $m_2=1.7500$, $f(m_2)>0 \rightarrow [1.5000, 1.7500]$. $m_3=1.6250$, $f(m_3)>0 \rightarrow [1.5000, 1.6250]$. $m_4=1.5625$, $f(m_4)>0 \rightarrow [1.5000, 1.5625]$ (approx root ≈ 1.53125).

2. Regula Falsi / False Position (bracketing, directed by function values instead of the plain midpoint)

```
Require f(a)f(b) < 0.
c = [a.f(b) - b.f(a)] / [f(b) - f(a)] (x-intercept of the line joining (a,f(a)), (b,f(b)))
Keep the half of [a,b] that still brackets the sign change.
```

Worked example: $x^2-2=0$, start $[1,2]$. $f(1)=-1$, $f(2)=2$. $c_1=[1(2)-2(1)]/[2-(-1)] = 4/3 = 1.3333$; $f(c_1) \approx -0.2222 \rightarrow$ new interval $[1.3333, 2]$. $c_2 \approx 1.4000$.

- Advantage over Bisection: converges faster on average since it's "directed," but can stagnate if one endpoint never updates.

3. Fixed-Point Iteration — rewrite $f(x)=0$ as $x=g(x)$; iterate $x_{n+1}=g(x_n)$ from a guess x_0 . Converges locally only if $|g'(x)| < 1$ near the root — a poor rearrangement of the SAME equation can diverge even though another rearrangement converges.

4. Newton-Raphson (open method, needs derivative, very fast near a simple root)

```
xn+1 = xn - f(xn)/f'(xn)
```

TRAP (failure cases): fails when $f'(x_n)=0$ (division by zero); a very small $f'(x_n)$ can cause a huge, unstable jump; a poor initial guess can converge to the WRONG root; oscillation/discontinuity can prevent convergence entirely.

GATE PYQ (Biotechnology 2025): $e^x-2=0$, $x_0=1$, find x_1 (round to 2 decimals). $f(x)=e^x-2$, $f'(x)=e^x$. $x_1 = 1-(e-2)/e = 2/e \approx 0.73576 \rightarrow 0.74$.

GATE PYQ (Chemical Engineering 2025): $f(x)=x^2-x-1=0$, $x^{(0)}=1$, find the SECOND iterate $x^{(2)}$. $f'(x)=2x-1$. $x^{(1)} = 1-(1-1-1)/(2-1) = 2$. $x^{(2)} = 2-(4-2-1)/(4-1) = 2-1/3 = 1.6667 \rightarrow 1.67$.

5. Secant Method (like Newton-Raphson, but the derivative is APPROXIMATED using two points — no derivative needed)

```
xn+1 = xn - [f(xn)(xn - xn-1)] / [f(xn) - f(xn-1)]
```

Worked example: $x^2-2=0$, $x_0=1$, $x_1=2$. $f(1)=-1$, $f(2)=2$. $x_2 = 2-[2(2-1)]/[2-(-1)] = 1.3333$. $f(x_2) \approx -0.2222$. $x_3 = 1.3333-[-0.2222](1.3333-2)/[-0.2222-2] \approx 1.4000$.

- Faster than Bisection/Regula Falsi on average, but NOT guaranteed to converge for every starting pair.

Root-finding method comparison

Method	Needs derivative?	Bracket required?	Guaranteed convergence?	Speed
Bisection	No	Yes	Yes	Slow
Regula Falsi	No	Yes	Usually (can stagnate)	Medium
Fixed-Point	No	No	Only if $ g'(x) < 1$	Depends on g
Newton-Raphson	Yes	No	Only near a good guess	Fast (quadratic)
Secant	No	No (2 estimates)	Not guaranteed	Fast

- Selection rule: reliability → Bisection; interpolation bracket → Regula Falsi; derivative + good guess → Newton; no derivative + two estimates → Secant.

B2c.3 Systems of Linear Equations [Level: Medium]

- Matrix form: $\mathbf{Ax} = \mathbf{b}$. For a square system, a UNIQUE solution exists only when $\det(\mathbf{A}) \neq 0$.
- Augmented matrix $[\mathbf{A}|\mathbf{b}]$; valid row operations: swap two rows, scale a row by $k \neq 0$, replace a row with itself minus a multiple of another row — every operation must be applied to the WHOLE row.
- Pivot** = leading nonzero element used to eliminate a column. A zero pivot forces a row swap; a very small pivot magnifies round-off error (fixed by **Partial Pivoting**: swap in the row with the largest absolute value in that column before eliminating).

Ways to solve $\mathbf{Ax}=\mathbf{b}$ — direct methods

Gaussian Elimination — Forward elimination to upper-triangular form, then back-substitution (solve last equation first, substitute upward).

Worked example: $x+y+z=6$, $2x-y+z=3$, $x+2y-z=2$. Forward elimination → $[1 \ 1 \ 1|6]$; $[0 \ -3 \ -1|-9]$; $[0 \ 1 \ -2|-4]$ → swap R_2, R_3 → R_3+3R_2 → $[0 \ 0 \ -7|-21]$. Back-substitute: $\mathbf{z=3, y=2, x=1}$.

Gauss-Jordan — continue elimination until the coefficient matrix becomes the identity, $[\mathbf{A}|\mathbf{b}] \rightarrow [\mathbf{I}|\mathbf{x}]$; solution read directly, no back-substitution needed — more total operations than Gaussian Elimination, but directly gives RREF/matrix inverse.

LU Decomposition — $\mathbf{A} = \mathbf{LU}$ ($\mathbf{L}$ =lower-triangular, $\mathbf{U}$ =upper-triangular). Solve $\mathbf{Lz}=\mathbf{b}$ (forward substitution), then $\mathbf{Ux}=\mathbf{z}$ (back substitution). Most useful when the SAME $\mathbf{A}$ must be solved for MANY different $\mathbf{b}$ vectors (decompose once, reuse for every right-hand side).

Worked example: $2x+y=5$, $4x+3y=11$. $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 4 & 3 \end{bmatrix} = \mathbf{LU}$ where $\mathbf{L} = \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix}$, $\mathbf{U} = \begin{bmatrix} 2 & 1 \\ 0 & 1 \end{bmatrix}$. $\mathbf{Lz}=\mathbf{b}$: $z_1=5$, $2(5)+z_2=11 \rightarrow z_2=1$. $\mathbf{Ux}=\mathbf{z}$: $y=1$, $2x+1=5 \rightarrow \mathbf{x=2, y=1}$.

- TRAP:** Gaussian Elimination/Gauss-Jordan/LU are **Direct methods** (exact after finite steps, best for small/medium dense systems). Jacobi/Gauss-Seidel below are **Iterative methods** (start from a guess, improve repeatedly — best for large, sparse systems).

Ways to solve $\mathbf{Ax}=\mathbf{b}$ — iterative methods

- Precondition: rearrange each equation to isolate one variable ($x=...$, $y=...$, $z=...$). **Diagonal Dominance** ($|a_{ii}| > \text{sum of other } |a_{ij}|$ in that row) is SUFFICIENT (not necessary) to guarantee convergence of both methods below.
- Jacobi Iteration:** every new value uses ONLY the PREVIOUS iteration's values (simultaneous update — parallelizable, but slower).
- Gauss-Seidel Iteration:** uses the NEWEST value immediately, as soon as it's computed (sequential update — usually faster, not parallelizable).
- TRAP:** "all old values" = Jacobi; "newest value immediately" = Gauss-Seidel — this recognition rule is the #1 way this pair is tested.

Shared worked example: $10x+y+z=12$; $2x+10y+z=13$; $2x+2y+10z=14$, all start at 0. Rearranged: $x=(12-y-z)/10$, $y=(13-2x-z)/10$, $z=(14-2x-2y)/10$.

Jacobi: Iteration 1 (all use old 0,0,0): $x^1=1.2$, $y^1=1.3$, $z^1=1.4$. Iteration 2 (all use iteration-1 values): $x^2=(12-1.3-1.4)/10=0.93$, $y^2=(13-2.4-1.4)/10=0.92$, $z^2=(14-2.4-2.6)/10=0.90$.

Gauss-Seidel: Iteration 1: $x^1=(12-0-0)/10=1.2$ (same start); y^1 uses the NEW x^1 immediately = $(13-2.4-0)/10=1.06$; z^1 uses NEW x^1 AND NEW y^1 = $(14-2.4-2.12)/10=0.948$. Gauss-Seidel's single first iteration (1.2, 1.06, 0.948) is already closer to the true (1,1,1) than Jacobi's first iteration (1.2,1.3,1.4) — illustrating its typically faster convergence.

- Residual** $\mathbf{r} = \mathbf{b} - \mathbf{A}\tilde{\mathbf{x}}$ (how well an approximate solution satisfies the equations) vs **Solution error** $\mathbf{e} = \mathbf{x} - \tilde{\mathbf{x}}$ (how far the approximate solution is from truth) — **TRAP:** a SMALL residual does NOT guarantee a small solution error, especially for **ill-conditioned** systems (small input changes cause large solution changes).

Linear-system method-selection guide

Need	Use
One system, direct/simple	Gaussian Elimination
Identity form / matrix inverse	Gauss-Jordan
Same A, many different b vectors	LU Decomposition
Large sparse system, simultaneous updates OK	Jacobi
Large sparse system, want faster practical convergence	Gauss-Seidel

B2c.4 Numerical Integration [Level: Medium]

$\mathbf{I} = \text{integral}(a \text{ to } b) \ f(x) \ dx$; Step size $\mathbf{h} = (b-a)/n$; Nodes $\mathbf{x_0=a, x_1=a+h, \dots, x_n=b}$; Ordinates $\mathbf{y_i=f(x_i)}$

- TRAP:** always verify equal spacing ($x_1-x_0 = x_2-x_1 = \dots$) before applying Simpson's rules — they require it; Trapezoidal can be applied interval-by-interval even with unequal widths.

Ways to solve — three composite rules

Trapezoidal Rule (straight-line/linear approximation per strip; ANY n works):

$\mathbf{I} \sim h/2 [y_0 + y_n + 2(y_1+y_2+\dots+y_{n-1})]$ weights: $\mathbf{1, 2, 2, \dots, 2, 1}$

GATE PYQ (Production & Industrial Engg 2024): $x=0,1,2,3,4$; $y=1,3,6,9,12$. $h=1$. $I \approx 1/2[1+12+2(3+6+9)] = 1/2[13+36] = \mathbf{24.5}$.

- Error order $\mathbf{O(h^2)}$; exact only for constant/linear functions; halving h cuts the leading error to $\sim 1/4$.

Simpson's One-Third Rule (quadratic/parabolic approximation; **n MUST be even**):

$$I \approx h/3 [y_0 + y_n + 4(y_1 + y_3 + \dots + y_{n-1}) + 2(y_2 + y_4 + \dots + y_{n-2})] \quad \text{weights: } 1, 4, 2, 4, 2, \dots, 4, 1$$

Worked example: $\int_0^2 (x^2+1)dx$, $n=2$, $h=1$. $y_0=1, y_1=2, y_2=5$. $I \approx 1/3[1+4(2)+5] = 14/3 \approx \mathbf{4.667}$ (this is EXACT here, since Simpson's 1/3 is exact up to degree-3 polynomials).

GATE PYQ (Chemical Engg 2023): $\int_{-1}^1 \sqrt{1-x^2}dx$, $h=0.5 \rightarrow$ nodes $-1, -0.5, 0, 0.5, 1$; $y_0=0, y_1=\sqrt{3}/2, y_2=1, y_3=\sqrt{3}/2, y_4=0$. $I \approx 1/6[4(\sqrt{3}/2 + \sqrt{3}/2) + 2] = 1/6(4\sqrt{3} + 2) = \mathbf{1/3 + 2/\sqrt{3}}$.

- Error order $O(h^4)$; exact for polynomials up to degree 3; halving h cuts leading error to $\sim 1/16$.

Simpson's Three-Eighth Rule (cubic approximation; **n MUST be divisible by 3**):

$$I \approx 3h/8 [y_0 + 3y_1 + 3y_2 + y_3] \quad (\text{basic form, } n=3) \quad \text{weights pattern: } 1, 3, 3, 2, 3, 3, \dots, 3, 1 \text{ (composite)}$$

Worked example: $\int_0^3 x^3 dx$, $n=3$, $h=1$. $y_0=0, y_1=1, y_2=8, y_3=27$. $I \approx 3/8[0+3(1)+3(8)+27] = 3/8(54) = \mathbf{20.25}$ (exact, matching $\int_0^3 x^3 dx = 81/4$).

- Error order also $O(h^4)$; exact up to degree 3.

Integration method comparison

Rule	Min. intervals	n condition	Weights	Exact up to degree	Error order
Trapezoidal	1	any	1, 2, 2, ..., 1	1	$O(h^2)$
Simpson 1/3	2	even	1, 4, 2, 4, ..., 1	3	$O(h^4)$
Simpson 3/8	3	divisible by 3	1, 3, 3, 2, ..., 1	3	$O(h^4)$

- TRICK: if you're given an ODD number of intervals not divisible by 3 in a way Simpson 1/3 alone can't handle, split — e.g., 5 intervals = apply Simpson 1/3 over the first 2 and Simpson 3/8 over the remaining 3, then add the two partial results.
- TRAP (common mistakes): confusing NODES ($n+1$) with INTERVALS (n); applying Simpson 1/3 when n is odd; applying Simpson 3/8 when n isn't divisible by 3; rounding intermediate values too early.

B2c.5 Numerical Solution of Differential Equations (IVPs) [Level: Medium-Hard]

Standard form: $dy/dx = f(x, y)$, with initial condition $y(x_0)=y_0$.
Grid points: $x_n = x_0 + n \cdot h$; Slope at current point = $f(x_n, y_n)$

Ways to solve — single-step methods (all self-starting, need only y_0)

Euler's Method (simplest, uses only the STARTING slope of the interval):

$$y_{n+1} = y_n + h \cdot f(x_n, y_n)$$

Local error $O(h^2)$, **global error $O(h)$** (first-order method) — only the beginning-of-interval slope is used, so it's inaccurate if the slope changes a lot within the step.

Worked example: $dy/dx=x-y$, $y(0)=1$, $h=0.2$, find $y(0.4)$. Step 1: $f(0,1)=-1 \rightarrow y_1=1+0.2(-1)=0.8$. Step 2: $f(0.2,0.8)=-0.6 \rightarrow y_2=0.8+0.2(-0.6)=\mathbf{0.68}$.

GATE PYQ (Civil Engg Set 1, 2025): $dy/dx=y+2x-x^2$, $y(0)=1$, $h=0.5$, find $y(1.0)$. $f(0,1)=1 \rightarrow y_1=1.5$. $f(0.5,1.5)=2.25 \rightarrow y_2=1.5+0.5(2.25)=2.625 \rightarrow \mathbf{2.6}$.

Modified Euler / Heun's Method (predictor-corrector; averages the START and predicted END slope):

$$\begin{aligned} \text{Predictor: } y_{n+1}(p) &= y_n + h \cdot f(x_n, y_n) \\ \text{Corrector: } y_{n+1}(c) &= y_n + h/2 [f(x_n, y_n) + f(x_{n+1}, y_{n+1}(p))] \end{aligned}$$

Global error $O(h^2)$.

Worked example: $dy/dx=x+y$, $y(0)=1$, $h=0.2$, find $y(0.2)$. Predictor: $y_1^p=1+0.2(1)=1.2$. Predicted end-slope $f(0.2,1.2)=1.4$. Corrector: $y_1^c=1+0.1(1+1.4)=\mathbf{1.24}$.

RK2 (Midpoint Method) — uses the slope AT the midpoint of the interval:

$$k_1 = h \cdot f(x_n, y_n); \quad k_2 = h \cdot f(x_n + h/2, y_n + k_1/2); \quad y_{n+1} = y_n + k_2$$

Global error $O(h^2)$, 2 function evaluations.

Worked example: $dy/dx=x+y^2$, $y(0)=1$, $h=0.1$, find $y(0.1)$. $k_1=0.1(0+1)=0.1$. Midpoint: $x=0.05, y=1.05$. $k_2=0.1(0.05+1.05^2)=0.1(1.1525)=0.11525$. $y_1=1+0.11525=\mathbf{1.11525}$.

RK4 (Classical Fourth-Order Runge-Kutta) — combines 4 slope estimates (beginning, 2 midpoints, end) with weights **1:2:2:1**:

$$\begin{aligned} k_1 &= h \cdot f(x_n, y_n) \\ k_2 &= h \cdot f(x_n + h/2, y_n + k_1/2) \\ k_3 &= h \cdot f(x_n + h/2, y_n + k_2/2) \\ k_4 &= h \cdot f(x_n + h, y_n + k_3) \\ y_{n+1} &= y_n + (k_1 + 2 \cdot k_2 + 2 \cdot k_3 + k_4)/6 \end{aligned}$$

Global error $O(h^4)$, 4 function evaluations — the standard workhorse for high single-step accuracy. **TRAP:** h is already baked into every k_i — do NOT multiply by h again in the final update.

GATE PYQ (Agricultural Engg 2023): $dy/dx=2x+y$, $y(0)=1$, $h=0.2$, find $y(0.2)$. $k_1=0.2(0+1)=0.2$. $k_2=0.2 \cdot f(0.1, 1.1)=0.26$. $k_3=0.2 \cdot f(0.1, 1.13)=0.266$. $k_4=0.2 \cdot f(0.2, 1.266)=0.3332$. $y_1=1+(0.2+0.52+0.532+0.3332)/6=1.2642 \rightarrow \mathbf{1.264}$.

Single-step methods comparison

Method	Evaluations/step	Local error	Global error	If h halved, error becomes
Euler	1	$O(h^2)$	$O(h)$	$\sim 1/2$
Modified Euler / RK2	2	$O(h^3)$	$O(h^2)$	$\sim 1/4$
RK4	4	$O(h^5)$	$O(h^4)$	$\sim 1/16$

- A higher-order ODE (e.g., $y''+y=0$) is converted to a first-order SYSTEM before applying any of the above: let $u_1=y$, $u_2=y' \rightarrow u_1'=u_2$, $u_2'=-u_1$ (a general $d^2y/dx^2=F(x,y,y')$ becomes $u_1'=u_2$, $u_2'=F(x,u_1,u_2)$).

B2c.6 Multi-Step Methods [Level: Hard]

- Unlike single-step methods, multi-step methods use SEVERAL previous points ($y_n, f_n, f_{n-1}, f_{n-2}, \dots$) to predict the next value — they are generally **NOT self-starting** (need extra starting values, typically generated by running RK4 for the first few steps) and require equal step size throughout.

Adams-Bashforth Predictor (4-step, explicit):
 $y_{n+1}(p) = y_n + h/24 [55.f_n - 59.f_{n-1} + 37.f_{n-2} - 9.f_{n-3}]$
 Adams-Moulton Corrector (4th order, implicit — needs the predicted f_{n+1}):
 $y_{n+1}(c) = y_n + h/24 [9.f_{n+1}(p) + 19.f_n - 5.f_{n-1} + f_{n-2}]$
 Milne Predictor: $y_{n+1}(p) = y_{n-3} + 4h/3 [2f_n - f_{n-1} + 2f_{n-2}]$
 Simpson Corrector: $y_{n+1}(c) = y_{n-1} + h/3 [f_{n-1} + 4f_n + f_{n+1}]$

- Predictor-Corrector workflow:** P (predict) $\rightarrow$ E (evaluate f at the predicted point) $\rightarrow$ C (correct) $\rightarrow$ E (re-evaluate if another correction round is needed) — stop once two consecutive corrected values agree to the required accuracy.

Worked example ($dy/dx=y$, so $f=y$ everywhere; $x: 0.0, 0.1, 0.2, 0.3$, $y: 1.00000, 1.10517, 1.22140, 1.34986$ — these are e^x values): find $y(0.4)$ using 4-step Adams-Bashforth then correct with Adams-Moulton.

Predictor: $y_4^p = 1.34986 + (0.1/24)[55(1.34986) - 59(1.22140) + 37(1.10517) - 9(1)] = 1.34986 + 0.14196 = \mathbf{1.49182}$.

Corrector: $y_4^c = 1.34986 + (0.1/24)[9(1.49182) + 19(1.34986) - 5(1.22140) + 1.10517] \approx \mathbf{1.491826}$.

- TRAP:** "self-starting" is a single-step-method property — Euler/RK2/RK4 ARE self-starting; Adams-Bashforth/Adams-Moulton/Milne are NOT.
- Consistency** (numerical equation $\rightarrow$ differential equation as $h \rightarrow 0$), **Stability** (small disturbances don't grow uncontrollably), and **Convergence** (numerical solution $\rightarrow$ exact solution as $h \rightarrow 0$) are three DIFFERENT properties — for linear multi-step methods, Consistency + Zero-stability TOGETHER imply Convergence. **TRAP:** a high-order method is not automatically useful — an unstable method can still magnify tiny errors without bound.

B2c.7 Complete Selection Map [Level: Reference]

Category	Use this when...
Non-linear root	Bisection (reliability); Regula Falsi (bracketed, directed); Fixed-Point ($x=g(x)$ form); Newton (derivative + good guess); Secant (no derivative, two estimates)
Linear system	Gaussian (one direct solve); Gauss-Jordan (need identity/inverse); LU (same A, many b's); Jacobi (parallel, all-old updates OK); Gauss-Seidel (sequential, want faster convergence)
Integration	Trapezoidal (general/simple); Simpson 1/3 (n even); Simpson 3/8 (n divisible by 3)
ODE	Euler (basic estimate); Modified Euler/RK2 (2-slope improvement); RK4 (high-accuracy single step); Multi-step (many equally-spaced points already available); Predictor-Corrector (refine a prediction)

Questions

(Easy $\rightarrow$ Hard; several are real official GATE PYQs or the source's own final mixed test, all independently verified)

Q1. [Easy] Bisection starts with an interval of width 3. What is the interval width after 5 bisections?

A. 3/8 B. 3/16 C. 3/32 D. 3/64

Answer: C. Why: Width after n bisections = original/ $2^n = 3/2^5 = 3/32$.

Q2. [Easy] Solve using Gaussian Elimination-style substitution: $2x+y=5$, $x-y=1$.

A. $x=1, y=3$ B. $x=2, y=1$ C. $x=3, y=-1$ D. $x=1, y=2$

Answer: B. Why: Adding both equations: $3x=6 \rightarrow x=2$, then $y=x-1=1$.

Q3. [Medium] Using Simpson's One-Third Rule with $h=1$, evaluate $\int_0^2 x^2 dx$.

A. 2 B. 7/3 C. 8/3 D. 3

Answer: C. Why: $y_0=0, y_1=1, y_2=4$. $I \approx 1/3[0+4(1)+4]=8/3$ (exact, matching the true integral).

Q4. [Medium] Using one Euler step for $dy/dx=x+y$, $y(0)=1$, $h=0.1$, find $y(0.1)$.

A. 1.00 B. 1.05 C. 1.10 D. 1.11

Answer: C. Why: $y_1 = 1 + 0.1 \cdot f(0,1) = 1 + 0.1(0+1) = 1.10$.

Q5. [Medium] [Trap] What is the global (accumulated) error order of the classical RK4 method?

A. $O(h)$ B. $O(h^2)$ C. $O(h^3)$ D. $O(h^4)$

Answer: D. Why: RK4's 4 weighted slope evaluations give 4th-order global accuracy — much better than Euler's $O(h)$ or RK2's $O(h^2)$.

Q6. [Medium] [Trap] Which of these methods is self-starting (needs only the initial condition, not extra prior points)?

A. Four-step Adams-Bashforth B. Milne Predictor C. RK4 D. Adams-Moulton Corrector

Answer: C. Why: RK4 is a single-step method needing only (x_0, y_0) ; the other three are multi-step methods requiring several previously computed points.

Q7. [Hard] [GATE PYQ] Using Newton-Raphson on $f(x)=e^x-2=0$ with $x_0=1$, find x_1 (round to 2 decimals).

A. 0.63 B. 0.69 C. 0.74 D. 0.82

Answer: C. Why: $x_1 = x_0 - f(x_0)/f'(x_0) = 1 - (e-2)/e = 2/e \approx 0.73576 \rightarrow 0.74$.

Q8. [Hard] [GATE PYQ] For $f(x)=x^2-x-1=0$ with $x^{(0)}=1$, find the SECOND Newton-Raphson iterate $x^{(2)}$ (round to 2 decimals).

A. 1.50 B. 1.60 C. 1.67 D. 1.75

Answer: C. Why: $x^{(1)} = 1 - (-1-1)/(2-1) = 2$. $x^{(2)} = 2 - (4-2-1)/(4-1) = 2 - 1/3 = 1.6667 \rightarrow 1.67$.

Q9. [Hard] [Trap] A system's coefficient matrix satisfies strict diagonal dominance. What does this guarantee?

A. The system has no solution B. Both Jacobi and Gauss-Seidel iterations are guaranteed to converge C. The determinant is zero D. Gaussian Elimination will fail

Answer: B. Why: Strict diagonal dominance is a SUFFICIENT (not necessary) condition for convergence of both Jacobi and Gauss-Seidel.

Q10. [Hard] [GATE PYQ] Using the Trapezoidal Rule on the table $x=0,1,2,3,4$ and $y=1,3,6,9,12$, estimate $\int_0^4 y(x)dx$.

A. 22.0 B. 23.5 C. 24.5 D. 26.0

Answer: C. **Why:** $h=1$. $I \approx 1/2[y_0+y_4+2(y_1+y_2+y_3)] = 1/2[1+12+2(18)] = 1/2(49) = 24.5$.

Q11. [Hard] [GATE PYQ] Using RK4 for $dy/dx=2x+y$, $y(0)=1$, $h=0.2$, find $y(0.2)$ (round to 3 decimals).

A. 1.240 B. 1.264 C. 1.280 D. 1.300

Answer: B. **Why:** $k_1=0.2$, $k_2=0.26$, $k_3=0.266$, $k_4=0.3332$. $y_1=1+(0.2+0.52+0.532+0.3332)/6=1.264$.

Q12. [Hard] [Trick] A 4-step Adams-Bashforth predictor gives $y_4^p=1.49182$ for $dy/dx=y$ with $y_3=1.34986$. Using the Adams-Moulton corrector

$y_4^c=y_3+h/24[9f_4^p+19f_3-5f_2+f_1]$ with $h=0.1$, $f_2=1.22140$, $f_1=1.10517$, find y_4^c .

A. 1.481826 B. 1.491826 C. 1.501826 D. 1.511826

Answer: B. **Why:** $y_4^c = 1.34986 + (0.1/24)[9(1.49182)+19(1.34986)-5(1.22140)+1.10517] \approx 1.491826$ — the corrector barely adjusts the predictor here since the function is smooth.

9.B3 Trigonometry

B3.1 Basic Ratios & Sign Rules [Level: Basic]

$\sin$, $\cos$, $\tan$ and their reciprocals $\operatorname{cosec} = 1/\sin$, $\sec = 1/\cos$, $\cot = 1/\tan$
 $\tan = \sin/\cos$, $\cot = \cos/\sin$

- **ASTC rule (sign by quadrant)** — mnemonic "**All Silver Tea Cups**":

Quadrant I (0-90): All functions positive
 Quadrant II (90-180): only Sin (and cosec) positive
 Quadrant III (180-270): only Tan (and cot) positive
 Quadrant IV (270-360): only Cos (and sec) positive

- **TRICK:** to find any trig ratio of an angle outside 0-90°, first pin down its quadrant (for the sign) then reduce it to the equivalent acute-angle ratio using the allied-angle rules below.

B3.2 Standard Angles & Allied Angles [Level: Basic-Medium]

Angle	0°	30°	45°	60°	90°
$\sin$	0	1/2	1/√2	√3/2	1
$\cos$	1	√3/2	1/√2	1/2	0
$\tan$	0	1/√3	1	√3	undefined

- **Allied angle rule:** for angles of the form $(n \cdot 90^\circ \pm \theta)$:

If n is ODD (90 +/- theta, 270 +/- theta): function NAME CHANGES ($\sin \leftrightarrow \cos$, $\tan \leftrightarrow \cot$, $\sec \leftrightarrow \operatorname{cosec}$)
 If n is EVEN (180 +/- theta, 360 +/- theta): function NAME STAYS THE SAME
 In both cases: the SIGN is whatever that function's sign would be in the resulting quadrant (use ASTC).

e.g., $\sin(90+\theta) = \cos\theta$ (name changes, and $90+\theta$ lands in QII where sin is positive); $\cos(180+\theta) = -\cos\theta$ (name same, QIII, cos is negative there).

B3.3 Compound Angle & Double Angle Formulas [Level: Medium]

$\sin(A+B) = \sin A \cos B + \cos A \sin B$ $\cos(A+B) = \cos A \cos B - \sin A \sin B$
 $\sin(A-B) = \sin A \cos B - \cos A \sin B$ $\cos(A-B) = \cos A \cos B + \sin A \sin B$
 $\tan(A+B) = (\tan A + \tan B) / (1 - \tan A \tan B)$

Double angle (set $B=A$):
 $\sin(2A) = 2 \sin A \cos A$
 $\cos(2A) = 1 - 2 \sin^2 A = 2 \cos^2 A - 1 = \cos^2 A - \sin^2 A$

B3.4 Core Identities [Level: Medium]

$\sin^2(\theta) + \cos^2(\theta) = 1$
 $1 + \tan^2(\theta) = \sec^2(\theta)$
 $1 + \cot^2(\theta) = \operatorname{cosec}^2(\theta)$

- **TRAP:** these three identities are just Pythagoras in disguise (divide $\sin^2 + \cos^2 = 1$ by $\cos^2$ or $\sin^2$ respectively to get the other two) — don't memorize them as unrelated facts.

B3.5 Properties of Triangles [Level: Hard]

Sine Rule: $a/\sin A = b/\sin B = c/\sin C = 2R$ (R = circumradius)
 Cosine Rule: $a^2 = b^2 + c^2 - 2bc \cos A$
 Projection Formula: $a = b \cos C + c \cos B$
 Area of triangle = $(1/2) \cdot a \cdot b \cdot \sin C$
 Area = $r \cdot s$ (r = inradius, s = semi-perimeter)
 Area = $(abc)/(4R)$ (R = circumradius)

- **TRICK:** given all 3 sides (no angles), find Area via Heron's formula first, then get $r = \text{Area}/s$ and $R = abc/(4 \times \text{Area})$ directly — no need to find any angle first.

B3.6 Heights & Distances [Level: Medium-Hard]

- **Angle of elevation:** angle from the horizontal UP to an object above eye level. **Angle of depression:** angle from the horizontal DOWN to an object below. Both are measured from the **horizontal**, not the vertical — the single most common student error here.

Core relation: $\tan(\text{angle}) = (\text{vertical height}) / (\text{horizontal distance})$

- **Ways to solve multi-triangle problems:** set up one right triangle per vantage point, express the unknown height/distance in terms of $\tan(\text{angle})$ for each, then equate or combine.

Worked Examples (verified)

A 20 m tall pole casts a shadow 10 m long. Find the angle of elevation of the sun.
 $\tan(\theta) = 20/10 = 2 \rightarrow \theta = \arctan(2) \approx \mathbf{63.43^\circ}$.

From a point 40 m from a tower's base, the angle of elevation to its top is 30° . Find the tower's height.

height = $40 \times \tan(30^\circ) = 40/\sqrt{3} = 40\sqrt{3}/3 \approx 23.09$ m.

Two towers of height 50 m and 80 m stand 100 m apart. Find the point on the line joining their bases from which the angles of elevation to both tops are equal.

Let the point be x m from the 50 m tower (so $100-x$ from the 80 m tower). Equal elevation angle $\Rightarrow$ equal height/distance ratio: $50/x = 80/(100-x) \rightarrow 50(100-x) = 80x \rightarrow 5000 = 130x \rightarrow x = 500/13 \approx 38.46$ m from the shorter tower.

B3.7 Inverse Trigonometric Functions (ITF) [Level: Hard]

Function	Principal value range
$\sin^{-1}x$	$[-\pi/2, \pi/2]$
$\cos^{-1}x$	$[0, \pi]$
$\tan^{-1}x$	$(-\pi/2, \pi/2)$
$\cot^{-1}x$	$(0, \pi)$

- **TRAP:** $\sin^{-1}$ and $\cos^{-1}$ do NOT share the same principal-value range — a very commonly tested mix-up.

B3.8 Trigonometric Equations — General Solutions [Level: Hard]

```

sin(theta) = sin(alpha) => theta = n.pi + (-1)^n . alpha      (n an integer)
cos(theta) = cos(alpha) => theta = 2n.pi +/- alpha
tan(theta) = tan(alpha) => theta = n.pi + alpha

```

Questions

Q1. [Easy] In which quadrant is $\tan(\theta)$ positive but $\sin(\theta)$ negative?

A. I B. II C. III D. IV

Answer: C. **Why:** By ASTC, Quadrant III has only Tan (and cot) positive; sin is negative there.

Q2. [Easy] Find the value of $\sin(30^\circ) + \cos(60^\circ)$.

A. 0.5 B. 1 C. 1.5 D. 2

Answer: B. **Why:** $\sin 30^\circ = 1/2$, $\cos 60^\circ = 1/2$, sum = 1.

Q3. [Medium] Evaluate $\sin(120^\circ)$.

A. $1/2$ B. $-1/2$ C. $\sqrt{3}/2$ D. $-\sqrt{3}/2$

Answer: C. **Why:** $120^\circ = 180^\circ - 60^\circ$; name stays "sin" (even multiple of 90 rule, since $180 = 2 \times 90$), and QII keeps sin positive $\rightarrow \sin(120^\circ) = \sin(60^\circ) = \sqrt{3}/2$.

Q4. [Medium] [Trap] Evaluate $\cos(90^\circ + \theta)$.

A. $\cos \theta$ B. $-\cos \theta$ C. $\sin \theta$ D. $-\sin \theta$

Answer: D. **Why:** 90 is an ODD multiple of $90^\circ \rightarrow$ function name CHANGES ($\cos \rightarrow \sin$); $90^\circ + \theta$ lies in QII where cos is negative $\rightarrow$ result is $-\sin \theta$.

Q5. [Medium] A ladder 10 m long rests against a wall making a 60° angle of elevation with the ground. How high up the wall does it reach?

A. 5 m B. $5\sqrt{3}$ m C. 8.66 m D. Both B and C

Answer: D. **Why:** height = $10 \times \sin(60^\circ) = 10 \times (\sqrt{3}/2) = 5\sqrt{3} \approx 8.66$ m — B and C are the same value in different forms.

Q6. [Hard] From a point 40 m from a tower's base, the angle of elevation to its top is 30° . Find the tower's height.

A. 20 m B. 23.09 m C. 40 m D. 69.28 m

Answer: B. **Why:** height = $40 \times \tan(30^\circ) = 40/\sqrt{3} \approx 23.09$ m.

Q7. [Hard] In a triangle, $a=7$, $b=8$, $c=9$. Using the cosine rule, find $\cos A$.

A. 0.5 B. 0.6 C. 0.7 D. 0.4

Answer: B. **Why:** $\cos A = (b^2 + c^2 - a^2)/(2bc) = (64 + 81 - 49)/(2 \times 8 \times 9) = 96/144 = 0.6$.

Q8. [Hard] [Trap] What is the principal value range of $\cos^{-1}x$?

A. $[-\pi/2, \pi/2]$ B. $[0, \pi]$ C. $(-\pi/2, \pi/2)$ D. $[-\pi, \pi]$

Answer: B. **Why:** $\cos^{-1}x$ has range $[0, \pi]$, distinct from $\sin^{-1}x$'s $[-\pi/2, \pi/2]$ — a frequently swapped pair.

Q9. [Hard] Solve for the general solution: $\sin(\theta) = 1/2$.

A. $\theta = n\pi + (-1)^n(\pi/6)$ B. $\theta = 2n\pi \pm (\pi/6)$ C. $\theta = n\pi + (\pi/6)$ D. $\theta = n\pi/6$

Answer: A. **Why:** For $\sin \theta = \sin \alpha$ ($\alpha = \pi/6$ here since $\sin(\pi/6) = 1/2$), the general solution form is $\theta = n\pi + (-1)^n \alpha$.

Q10. [Hard] Two towers of height 50 m and 80 m stand 100 m apart on level ground. Find the distance from the 50 m tower to the point between them where the angles of elevation to both tops are equal.

A. 30.77 m B. 38.46 m C. 50 m D. 61.54 m

Answer: B. **Why:** Equal-ratio condition $50/x = 80/(100-x)$ gives $130x = 5000 \rightarrow x = 500/13 \approx 38.46$ m.

9.B4 Vectors

B4.1 Basic Concepts & Types [Level: Basic]

- **Vector**: has both magnitude and direction (e.g., displacement, velocity, force). **Scalar**: magnitude only (e.g., mass, temperature, time).
- Component form: $\mathbf{a} = a_1\mathbf{i} + a_2\mathbf{j} + a_3\mathbf{k}$, magnitude $|\mathbf{a}| = \sqrt{a_1^2 + a_2^2 + a_3^2}$.
- Types: **Position vector** (from origin to a point), **Unit vector** (magnitude 1, direction-only), **Zero vector** (magnitude 0, undefined direction), **Coinitial vectors** (same starting point), **Collinear vectors** (parallel/along the same line), **Coplanar vectors** (lie in the same plane), **Free vector** (position doesn't matter, only magnitude+direction).

Unit vector along $\mathbf{a} = \mathbf{a} / |\mathbf{a}|$

B4.2 Vector Addition — Triangle & Parallelogram Law [Level: Basic]

- **Triangle Law**: if two vectors are the first two sides of a triangle (taken in order, tip-to-tail), their sum is the third side (tail of first to tip of second, closing the triangle).
- **Parallelogram Law**: if two vectors are adjacent sides of a parallelogram from a common point, their sum is the diagonal through that point.
- **TRAP**: both laws describe the SAME resultant — Triangle Law is just the Parallelogram Law's diagonal viewed as two triangles; don't treat them as giving different answers.

B4.3 Section Formula & Direction Cosines/Ratios [Level: Medium]

Position vector of point dividing AB in ratio $m:n$ (internally) $= (\mathbf{m} \cdot \mathbf{B} + \mathbf{n} \cdot \mathbf{A}) / (m+n)$

- **Direction Cosines (DCs)**: $l, m, n = \cos(\text{angles the vector makes with } x, y, z \text{ axes})$; always satisfy $l^2 + m^2 + n^2 = 1$.
- **Direction Ratios (DRs)**: any numbers proportional to the DCs (don't need to satisfy the sum-of-squares=1 condition themselves) — e.g., the components a_1, a_2, a_3 of a vector are one valid set of DRs.
- **Collinear/parallel vectors test**: $\mathbf{a}$ and $\mathbf{b}$ are parallel $\Leftrightarrow \mathbf{a} = \lambda \mathbf{b}$ for some scalar λ (Condition 1) $\Leftrightarrow$ their components are proportional: $a_1/b_1 = a_2/b_2 = a_3/b_3$ (Condition 2 — same test, ratio form). If $\lambda > 0$, same direction; if $\lambda < 0$, opposite direction.

B4.4 Dot (Scalar) Product [Level: Medium]

$\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}| |\mathbf{b}| \cdot \cos(\text{theta}) = a_1 \cdot b_1 + a_2 \cdot b_2 + a_3 \cdot b_3$ (result is a SCALAR)

- **Ways to use it**: (1) Find the **angle** between two vectors: $\cos(\theta) = (\mathbf{a} \cdot \mathbf{b}) / (|\mathbf{a}| |\mathbf{b}|)$. (2) Test **perpendicularity**: $\mathbf{a} \cdot \mathbf{b} = 0 \Leftrightarrow \mathbf{a} \perp \mathbf{b}$. (3) **Scalar projection** of $\mathbf{a}$ onto $\mathbf{b} = (\mathbf{a} \cdot \mathbf{b}) / |\mathbf{b}|$.
- **TRAP**: Dot product of a vector with itself gives its magnitude SQUARED: $\mathbf{a} \cdot \mathbf{a} = |\mathbf{a}|^2$.

B4.5 Cross (Vector) Product [Level: Medium-Hard]

$\mathbf{a} \times \mathbf{b} = |\mathbf{a}| |\mathbf{b}| \cdot \sin(\text{theta}) \cdot \mathbf{n}_{\text{hat}}$ (result is a VECTOR, perpendicular to BOTH $\mathbf{a}$ and $\mathbf{b}$)

- **TRAP**: Dot product $\rightarrow$ scalar (uses cos); Cross product $\rightarrow$ vector (uses sin) — the single most tested distinction in this chapter. Also $\mathbf{a} \times \mathbf{b} = -(\mathbf{b} \times \mathbf{a})$ (anti-commutative — order matters, unlike the dot product where $\mathbf{a} \cdot \mathbf{b} = \mathbf{b} \cdot \mathbf{a}$).
- **Ways to use it**: (1) Test **parallel vectors**: $\mathbf{a} \times \mathbf{b} = 0 \Leftrightarrow \mathbf{a} \parallel \mathbf{b}$. (2) **Area of parallelogram** formed by $\mathbf{a}, \mathbf{b} = |\mathbf{a} \times \mathbf{b}|$. (3) **Area of triangle** formed by $\mathbf{a}, \mathbf{b} = \frac{1}{2} |\mathbf{a} \times \mathbf{b}|$.

Questions

Q1. [Easy] Find the unit vector along $\mathbf{a} = 3\mathbf{i} + 4\mathbf{j}$.

A. $(3\mathbf{i}+4\mathbf{j})/5$ B. $(3\mathbf{i}+4\mathbf{j})/7$ C. $3\mathbf{i}+4\mathbf{j}$ D. $(4\mathbf{i}+3\mathbf{j})/5$

Answer: A. Why: $|\mathbf{a}| = \sqrt{9+16} = 5$, unit vector $= \mathbf{a}/|\mathbf{a}| = (3\mathbf{i}+4\mathbf{j})/5$.

Q2. [Easy] What is the result type of a dot product of two vectors?

A. Vector B. Scalar C. Matrix D. Unit vector

Answer: B. Why: Dot product always yields a scalar (magnitude only, no direction).

Q3. [Medium] Given $\mathbf{a} = 2\mathbf{i}+4\mathbf{j}+6\mathbf{k}$ and $\mathbf{b} = \mathbf{i}+2\mathbf{j}+3\mathbf{k}$, are they collinear?

A. Yes, since components are proportional (ratio 2:1 throughout) B. No C. Only if magnitudes are equal D. Cannot be determined

Answer: A. Why: $a_1/b_1=2, a_2/b_2=2, a_3/b_3=2$ — all ratios equal $\Rightarrow \mathbf{a} = 2\mathbf{b}$, so they're parallel/collinear.

Q4. [Medium] If $\mathbf{a} \cdot \mathbf{b} = 0$ for two non-zero vectors, what can you conclude?

A. They are parallel B. They are perpendicular C. They are equal D. One of them is a zero vector

Answer: B. Why: $\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}| |\mathbf{b}| \cos(\theta) = 0$ with both magnitudes non-zero forces $\cos(\theta)=0$, i.e., $\theta=90^\circ$.

Q5. [Medium] [Trap] If $\mathbf{a} \times \mathbf{b} = 0$ for two non-zero vectors, what can you conclude?

A. They are perpendicular B. They are parallel C. They have equal magnitude D. Their dot product is also 0

Answer: B. Why: $\mathbf{a} \times \mathbf{b} = |\mathbf{a}| |\mathbf{b}| \sin(\theta) \hat{n} = 0$ with non-zero magnitudes forces $\sin(\theta)=0$, i.e., $\theta=0^\circ$ or 180° — parallel vectors. (Contrast directly with Q4's dot-product case — the two products test opposite conditions.)

Q6. [Hard] Find the area of the triangle formed by vectors $\mathbf{a} = 2\mathbf{i}+3\mathbf{j}$ and $\mathbf{b} = \mathbf{i}+4\mathbf{j}$ (treating them as two sides from a common vertex).

A. 2.5 B. 5 C. 10 D. 11

Answer: A. Why: $\mathbf{a} \times \mathbf{b}$ (in 2D, the k-component) $= (2 \times 4 - 3 \times 1) = 8 - 3 = 5$, so $|\mathbf{a} \times \mathbf{b}| = 5$. Area of triangle $= \frac{1}{2} \times 5 = \mathbf{2.5}$.

Q7. [Hard] Find the angle between vectors $\mathbf{a} = \mathbf{i}+\mathbf{j}$ and $\mathbf{b} = \mathbf{i}-\mathbf{j}$.

A. 0° B. 45° C. 90° D. 180°

Answer: C. Why: $\mathbf{a} \cdot \mathbf{b} = (1)(1) + (1)(-1) = 0 \rightarrow$ perpendicular $\rightarrow 90^\circ$.

Q8. [Hard] [Trick] Point P divides segment AB in ratio 2:3 (internally), where $\mathbf{A}=(1,2)$ and $\mathbf{B}=(6,7)$ (as position vectors). Find P.

A. (3,4) B. (2,4) C. (4,5) D. (3,5)

Answer: A. Why: $\mathbf{P} = (\mathbf{m} \cdot \mathbf{B} + \mathbf{n} \cdot \mathbf{A}) / (\mathbf{m} + \mathbf{n}) = (2(6,7) + 3(1,2)) / 5 = ((12+3), (14+6)) / 5 = (15, 20) / 5 = \mathbf{(3,4)}$.

9. Part C — Verbal & Logical Reasoning

Word-based and statement-based reasoning: series, syllogisms, blood relations, coding-decoding, seating arrangements. **TRAP:** these questions are deliberately wordy — the trap is rarely the logic itself, it's misreading a single word ("not", "except", "some", "all", "either/or") in the question stem or in a syllogism's premise. Read the stem twice before touching the options.

9.C1 Analogy & Number/Letter Series

C1.1 Analogy — Basic Concept [Level: Basic]

- **Analogy** = a pair of items sharing a specific relationship; given a first pair, find the missing term in a second pair that shares the SAME relationship.
- TRICK: The master skill is to **name the relationship explicitly** before searching for the answer — once named, applying it is mechanical.

Common Relationship Types (reference list)

Relationship	Example pair
Synonym	Big : Large
Antonym	Hot : Cold
Part-Whole	Wheel : Car
Worker-Tool	Carpenter : Saw
Worker-Product	Baker : Bread
Object-Function	Knife : Cut
Cause-Effect	Fire : Burn
Degree/Intensity	Warm : Hot
Class-Member	Fruit : Mango
Male-Female	Lion : Lioness

- Letter Analogy: apply a fixed **positional shift** in the alphabet (e.g., B:D :: F:H, each letter +2).
- Number Analogy: apply a fixed **arithmetic rule** (e.g., 4:16 :: 5:25, rule = square; or 3:7 :: 5:11, rule = double+1).

C1.2 Number Series — Classification Checklist [Level: Basic-Medium]

TRICK: Before hunting for a pattern, run this checklist **in order** — this classification IS the shortcut:

1. Check constant DIFFERENCE (Arithmetic series)
2. Check constant RATIO (Geometric series)
3. Check difference-of-differences (Two-stage series)
4. Check odd/even position split (Alternating/combination series)
5. Check squares/cubes with an offset (Square/Cube-based series)
6. Check prime numbers
7. Check a mixed operation pattern (e.g., $x2+1$, $x2+2$, $x2+3...$)

Series Types with Examples

Type	Rule	Example
Arithmetic	constant d	3,7,11,15,19 (d=4)
Geometric	constant r	2,6,18,54 (r=3)
Two-stage	differences form a series	2,3,5,8,12 (diffs: 1,2,3,4)
Alternating/combination	two interleaved rules	1,10,3,8,5,6 (odd pos +2, even pos -2)
Square-based	$n^2 \pm \text{offset}$	3,6,11,18,27 (n^2+2 : 1+2,4+2,9+2...)
Mixed operation	changing operation per step	2,5,11,23,47 ($\times 2+1$ each time)

- **TRAP:** A **two-stage series** (differences forming their own series) is very commonly mistaken for a simple arithmetic series if only the first two gaps are checked — always verify at least 3-4 consecutive gaps before concluding the rule is constant.

C1.3 Letter / Alphabet Series [Level: Medium]

- Map letters to positions: A=1, B=2, ... Z=26 (and the reverse mapping Z=1...A=26 for "reverse alphabet" questions — always check which convention a question uses).
- Apply the same classification checklist (C1.2) to the position numbers instead of raw letters.
- Repeating letter-group patterns (e.g., AB, CD, EF, GH... — groups shift together) — treat each group as one "term" and find the group-level rule.

C1.4 Brute-Force Trick for Tough Series [Level: Hard]

TRICK: When no pattern is obvious quickly, compute **all simple differences AND all simple ratios** between consecutive terms simultaneously in a small table, and scan both rows for ANY consistency (constant value, or a second-level pattern) — this is faster than guessing randomly and catches two-stage/mixed-operation series quickly.

Questions

(Easy → Hard; [Trick] rewards the classification checklist, [Trap] tests common series-type misidentification)

Q1. [Easy] Doctor : Hospital :: Teacher : ?

A. Student B. School C. Book D. Chalk

Answer: B. **Why:** Relationship = workplace of the worker; a doctor works at a hospital, a teacher works at a school.

Q2. [Easy] Find the next term: 5, 10, 15, 20, ?

A. 22 B. 24 C. 25 D. 30

Answer: C. **Why:** Arithmetic series, constant difference +5.

Q3. [Easy] BD : FH :: JL : ?

A. NP B. MO C. NO D. OP

Answer: A. Why: Each letter shifts +4 positions (B→F→J→N, D→H→L→P): B,D → F,H → J,L → N,P.

Q4. [Medium] [Trap] Find the next term: 2, 6, 12, 20, 30, ?

A. 36 B. 40 C. 42 D. 44

Answer: C. Why: This LOOKS arithmetic-ish but differences are 4,6,8,10 (increasing by 2 each time — a two-stage series, not simple arithmetic). Next difference = 12, so next term = $30+12 = 42$. (Pattern is actually $n(n+1)$: $1 \times 2, 2 \times 3, 3 \times 4, 4 \times 5, 5 \times 6, 6 \times 7 = 42$.)

Q5. [Medium] Find the next term: 3, 9, 27, 81, ?

A. 162 B. 243 C. 324 D. 729

Answer: B. Why: Geometric series, constant ratio $\times 3$. $81 \times 3 = 243$.

Q6. [Medium] [Trick] Find the next term: 1, 4, 9, 18, 35, ?

A. 60 B. 64 C. 68 D. 70

Answer: C. Why: Differences: 3,5,9,17. Differences-of-differences: 2,4,8 — doubling each time. Next difference-of-difference = 16, so next difference = $17+16 = 33$, giving $35+33 = 68$. The brute-force diff table (checklist step 3) catches this in seconds.

Q7. [Medium] Find the missing letter series term: Z, W, T, Q, ?

A. N B. M C. O D. P

Answer: A. Why: Reverse alphabet positions 26,23,20,17 (constant difference -3) → next = $14 = N$.

Q8. [Hard] Find the next term: 4, 5, 9, 14, 23, 37, ?

A. 56 B. 60 C. 61 D. 65

Answer: B. Why: Fibonacci-style: each term = sum of previous two ($4+5=9$, $5+9=14$, $9+14=23$, $14+23=37$, $23+37=60$).

Q9. [Hard] [Trap] Find the odd one out: 2, 3, 5, 7, 9, 11

A. 2 B. 9 C. 11 D. 3

Answer: B. Why: All others are prime numbers; $9 = 3^2$ is composite — a prime-number-series check (checklist step 6) reveals the outlier.

Q10. [Hard] Complete the analogy: $3 : 28 :: 5 : ?$

A. 124 B. 126 C. 130 D. 150

Answer: B. Why: Rule: $n^3 + 1$. $3^3+1=28$ ✓. So $5^3+1 = 125+1 = 126$.

Q11. [Hard] Find the next term in the alternating series: 1, 20, 4, 17, 9, 14, 16, ?

A. 9 B. 11 C. 13 D. 15

Answer: B. Why: Odd positions (1st,3rd,5th,7th): 1,4,9,16 = squares ($1^2, 2^2, 3^2, 4^2$) → next odd-position term(9th) not asked. Even positions(2nd,4th,6th,8th): 20,17,14,? decreasing by 3 → next = 11.

Q12. [Hard] [Trick] Find the next term: 100, 96, 89, 79, 66, ?

A. 50 B. 52 C. 54 D. 56

Answer: A. Why: Differences: 4,7,10,13 (increasing by 3 — two-stage series). Next difference = 16, so $66-16 = 50$.

9.C2 Puzzle Solving

C2.1 The Master Strategy [Level: Basic]

Puzzles (seating/floor/ranking/box arrangements with multiple clues) are solved by **systematic deduction**, not by guessing. Follow this sequence every time:

1. Read ALL clues once before writing anything down.
2. Fill in DIRECT/absolute clues first (e.g., "X is on floor 3").
3. Use ELIMINATION for relative/comparative clues (e.g., "X is above Y").
4. Build a GRID (rows = entities, columns = attributes) – track possibilities on paper/in a table, never purely in your head.
5. After each new deduction, RE-SCAN all remaining clues – a fact you just fixed often unlocks a clue that seemed unusable before.

- TRICK: Always process the MOST restrictive/direct clue first (one that fixes an exact value), since it shrinks the possibility space fastest for every other clue.

C2.2 Common Puzzle Types [Level: Basic-Medium]

Type	Typical clue style
Linear/Floor arrangement	People/objects placed in a line or building floors
Circular/Rectangular seating	People seated around a table, facing in/out
Comparison/Ranking	"more than," "taller than," "just before/after"
Box/Stacking	Boxes stacked in an order, with colour/weight/size clues
Blood-relation-embedded	Puzzle clues also encode family relationships
Day/Date-embedded	Clues reference days of the week or dates

- **TRAP:** For **floor-based** puzzles, always re-read whether "floor 1" is stated as the ground/bottom floor or a specific reference floor — this convention is stated explicitly in the question and reverses the entire ordering if misread.
- **TRAP:** For **comparison clues**, "X is more than Y" and "X is less than Y" are opposite directions — misreading the direction is the single most common puzzle error. Underline the comparison word every time.

C2.3 Worked Example — Grid Method [Level: Medium]

Four friends P, Q, R, S live on 4 different floors (1 lowest to 4 highest) of a building.

Clues: (1) Q lives immediately above P. (2) S does not live on floor 1 or floor 4. (3) R lives on floor 4.

Step 1 (direct clue): R = floor 4 (fixed).

Step 2 (elimination): $S \neq 1$, $S \neq 4$ (R already has 4) $\rightarrow S = 2$ or 3.

Step 3 (relative clue): Q is immediately above P $\rightarrow$ (P,Q) is one of (1,2) or (2,3) or (3,4). Since R=4, (3,4) is impossible for Q. So (P,Q) = (1,2) or (2,3).

Step 4 (combine): If (P,Q)=(1,2), remaining floor 3 = S $\rightarrow$ but Step 2 said $S=2$ or 3; $S=3$ is fine here.

If (P,Q)=(2,3), remaining floor 1 = S $\rightarrow$ but Step 2 forbids $S=1$. Contradiction — reject this branch.

Conclusion: P=1, Q=2, S=3, R=4.

C2.4 Higher-Difficulty Puzzles [Level: Hard]

- Multi-attribute puzzles (e.g., floor AND colour AND profession simultaneously) — extend the grid to one column per attribute; solve attributes in the order that has the most direct clues first.
- Puzzles embedding blood relations or day/date facts require solving the "embedded" sub-puzzle (see Blood Relations / Calendar chapters) before it constrains the main arrangement.
- When two branches both seem consistent after direct clues, defer them explicitly (label as Case 1 / Case 2) and continue applying remaining clues to each branch separately until one is eliminated by a contradiction — don't try to force a single answer prematurely.

Questions

(Easy $\rightarrow$ Hard; [Trap] tests misreading a comparative/direction clue)

Q1. [Easy] Five boxes A, B, C, D, E are stacked with A at the top and E at the bottom in that fixed order. Which box is directly below C?

A. A B. B C. D D. E

Answer: C. Why: Order top $\rightarrow$ bottom is A,B,C,D,E, so D is directly below C.

Q2. [Easy] In a row of 5 people facing north, Ravi is 2nd from the left and 4th from the right. How many people are in the row?

A. 4 B. 5 C. 6 D. 7

Answer: B. Why: Position from left + position from right – 1 = total $\rightarrow 2+4-1 = 5$.

Q3. [Medium] [Trap] A is taller than B. C is shorter than B. Who is the shortest?

A. A B. B C. C D. Cannot be determined

Answer: C. Why: $A > B > C$, so C is shortest. (Trap: misreading "C is shorter than B" as "C is taller" reverses the whole order.)

Q4. [Hard] Four boxes W, X, Y, Z are stacked one on another (position 1 = topmost, 4 = bottommost). Z is at the bottom. W is directly above X. Y is not at the top. Find the order from top to bottom.

A. W, X, Y, Z B. X, W, Y, Z C. W, Y, X, Z D. Y, W, X, Z

Answer: A. Why: Z=4 (given). W directly above X means W's position = X's position–1; the only fit among remaining positions {1,2,3} is W=1, X=2 (the alternative W=2, X=3 would force Y=1, violating "Y is not at the top"). So Y=3. Order top $\rightarrow$ bottom: W, X, Y, Z.

Q5. [Hard] Among five friends, A is taller than B but shorter than C. D is taller than C. E is shorter than B. Who is the tallest and who is the shortest?

A. C tallest, B shortest B. D tallest, E shortest C. D tallest, B shortest D. A tallest, E shortest

Answer: B. Why: Chain the clues: $D > C > A > B > E$. Tallest = D, shortest = E.

9.C3 Syllogisms

C3.1 The Four Statement Types [Level: Basic]

Code	Name	Form	Example
A	Universal Affirmative	All A are B	All cats are animals
E	Universal Negative	No A is B	No cat is a dog
I	Particular Affirmative	Some A are B	Some cats are black
O	Particular Negative	Some A are not B	Some cats are not black

- A syllogism gives 2+ statements (premises) and asks which of several conclusions **logically follow** — using ONLY the given statements, disregarding real-world/commonly-known facts.

C3.2 The Venn Diagram Method — Core Principle [Level: Basic-Medium]

1. Draw the diagram(s) consistent with the given statements, using MINIMUM overlap first.
2. A conclusion is TRUE only if it holds in EVERY possible diagram consistent with the statements.
3. A conclusion is FALSE if even ONE valid diagram can be drawn where it fails.

- TRICK: This "must hold in ALL possible diagrams" rule is the single most important principle in this chapter — most wrong answers come from checking only the first (most obvious) diagram and stopping.

C3.3 Key Inference Rules [Level: Medium]

- Valid chain:** "All A are B" + "All B are C" $\Rightarrow$ "All A are C" (transitive, valid).
- TRAP:** "All A are B" + "All C are B" gives **NO relation** between A and C — two statements funnelling into the *same* B do not connect A and C. (e.g., "All cats are animals" + "All dogs are animals" tells you nothing about cats vs dogs.)
- "Some A are B" only guarantees its **conversion**: "Some B are A" — nothing stronger.
- TRAP:** "Some A are B" does **not** make "Some A are not B" false — both can be simultaneously true depending on the diagram (e.g., some cats are black AND some cats are not black, both true together).
- "No A is B" converts validly to "No B is A" (universal negatives convert freely).
- "All A are B" does NOT convert to "All B are A" — only to "Some B are A" (a universal affirmative only converts to a particular).

C3.4 Possibility-Type Questions [Level: Hard]

- Three distinct answer categories: **Definitely True**, **Definitely False**, **Possibly True** (true in at least one valid diagram, even if not all).
- TRICK: If a conclusion is not "Definitely True" by the all-diagrams rule, check whether it can be **Possibly True** before marking it false outright — many higher-difficulty questions specifically test this middle category.

Questions

(Easy $\rightarrow$ Hard; [Trap] targets the classic same-B fallacy and the Some-A-are-B scope trap)

Q1. [Easy] Statements: All pens are books. All books are tables.

Conclusions: (I) All pens are tables. (II) Some tables are pens.

A. Only I follows B. Only II follows C. Both I and II follow D. Neither follows

Answer: C. Why: All-All chain gives All pens are tables (I, valid transitive chain) which in turn implies Some tables are pens (II, valid conversion of I).

Q2. [Easy] Statements: No apple is a mango. All mangoes are fruits.

Conclusion: Some fruits are not apples.

A. Follows B. Does not follow C. Possibly true only D. Cannot be determined

Answer: A. Why: Since no apple is a mango, and all mangoes are fruits, the mango-fruits (which exist) are guaranteed to be non-apple fruits — so some fruits (the mangoes) are definitely not apples.

Q3. [Medium] [Trap] Statements: All doctors are engineers. All lawyers are engineers.

Conclusion: Some doctors are lawyers.

A. Follows B. Does not follow C. Possibly true D. Definitely false

Answer: B. Why: Both groups funnel into the SAME class (engineers) via separate "All...are engineers" statements — this gives NO relation between doctors and lawyers. Classic same-B fallacy.

Q4. [Medium] Statements: Some students are athletes. All athletes are disciplined.

Conclusion: Some students are disciplined.

A. Follows B. Does not follow C. Possibly true only D. Cannot be determined

Answer: A. Why: The athletes who are students (guaranteed to exist by "Some students are athletes") are also disciplined (by "All athletes are disciplined") — so some students are definitely disciplined.

Q5. [Medium] [Trap] Statement: Some flowers are red.

Conclusion: Some flowers are not red.

A. Follows B. Does not follow C. Both can be simultaneously true, but conclusion isn't forced D. Definitely false

Answer: C. Why: "Some A are B" does not rule out "Some A are not B" — both could hold, but the given statement alone does NOT force the conclusion to be true; it's merely not contradicted.

Q6. [Hard] Statements: All squares are rectangles. No rectangle is a triangle. Some triangles are polygons.

Conclusion: (I) No square is a triangle. (II) Some polygons are not squares.

A. Only I follows B. Only II follows C. Both follow D. Neither follows

Answer: C. Why: All squares are rectangles + No rectangle is a triangle $\Rightarrow$ No square is a triangle (I, valid — every square, being a rectangle, is excluded from triangles). This is equivalent to "No triangle is a square" (E-type statements convert freely). Combined with "Some triangles are polygons," those triangle-polygons are guaranteed non-square (since no triangle is a square) — so some polygons are definitely not squares (II, also valid).

Q7. [Hard] Statements: Some books are pens. Some pens are pencils. All pencils are erasers.

Conclusion: Some erasers are books.

A. Definitely follows B. Possibly true only C. Definitely false D. Cannot be determined

Answer: B. Why: A chain of two "Some" statements (books-pens, pens-pencils) does NOT guarantee the SAME pens are involved in both — the pens that are books might be entirely different from the pens that are pencils. So the eraser-book link is only possible, not definite.

Q8. [Hard] Statement: All coins are metals.

Which of these is a valid conclusion?

A. All metals are coins B. Some metals are coins C. No metal is a coin D. Some coins are not metals

Answer: B. Why: A universal affirmative (All A are B) converts validly only to a particular (Some B are A) — never to another universal (All B are A), which would be an unwarranted reversal.

Q9. [Hard] Statements: No teacher is lazy. Some lazy people are students.

Conclusion: Some students are not teachers.

A. Follows B. Does not follow C. Possibly true only D. Definitely false

Answer: A. Why: The lazy people who are students (guaranteed by "Some lazy people are students") cannot be teachers (since "No teacher is lazy" excludes all teachers from being lazy) — so those particular students are definitely not teachers.

9.C4 Statement-Based Reasoning (Conclusion, Argument, Assumption, Course of Action)

(One of the highest-yield logical reasoning topics — asked in almost every IT-placement and competitive exam. The four sub-types below are frequently mixed up; the table in C4.1 is the single most important thing to internalize.)

C4.1 The Four Types at a Glance [Level: Basic]

Type	What it asks	Judged by
Conclusion	What logically follows FROM the statement?	Strict derivability — no outside knowledge
Argument	Is a given "Yes/No, because..." reason strong or weak?	Relevance + practicality
Assumption	What is silently taken for granted BEHIND the statement?	Implicit necessity — statement fails without it
Course of Action	What practical step should follow?	Feasibility + direct remedy

- **TRAP:** **Conclusion** is derived *from* the statement (looks forward, what the statement implies); **Assumption** is presupposed *behind* the statement (looks backward, what the statement silently relies on to make sense). Students constantly swap these two directions.

C4.2 Statement & Conclusion [Level: Basic-Medium]

- A conclusion must follow **logically and directly** from the statement alone — never bring in outside/common-sense world knowledge, and never require an assumption to bridge the gap.
- **Five standard tips (from official test-prep convention):**
 1. **Do NOT assume** an unstated cause/reason. *Statement:* "Teacher scolded Ram in front of the class." → "The teacher disliked Ram" does **not** follow (many other causes are equally possible — late to class, incomplete homework, etc.).
 2. **Advice or a clearly-implied result is always true** if it is directly related to the statement. *Statement:* "Petrol may become rare due to rising consumption." → "Petroleum products should be used efficiently" **follows** (a direct, related piece of advice).
 3. **Avoid conclusions built on a pre-assumption.** *Statement:* "Many hospital admissions occurred; junk food is assumed poisonous." → "Junk food is ALWAYS poisonous" does **not** follow — it converts a hedge ("assumed") into an absolute, pre-assumed fact.
 4. **Ignore morality/ethics — judge only logical fit.** *Statement:* "Many medicines contain fish oil." → "Some vegetarians consume such medicines to cure diseases" **follows**, even though it may seem to contradict vegetarian ethics — logical relevance beats moral comfort.
 5. Watch for **absolute-scope words** (ALL, NO, EVERY, ONLY) introduced in a conclusion that the statement never licensed.

Worked Examples (from official practice sets)

- *Statement:* "Shyam is one of the probable students for securing 1st rank." *Conclusions:* (1) Shyam will secure 1st rank (2) Shyam will not secure 1st rank. → **Either 1 or 2 follows** — the statement restricts the outcome to exactly two mutually exclusive possibilities, so one of the two conclusions *must* be true even though neither is individually certain.
- *Statement:* "50% off on all Electronic Goods up till 31st January." *Conclusions:* (1) After 31 Jan, no discount will be provided (2) No sale of electronic goods after 31 Jan. → **Only conclusion 1 follows.** The offer's end-date implies the discount stops, but nothing says sales themselves stop — conclusion 2 overreaches.
- *Statement:* "Due to recession, Company X fired 200 employees." *Conclusions:* (1) Company X is well known for firing people (2) All competitors are also affected by the recession. → **Neither follows** — (1) wrongly generalizes a single event into a reputation/history claim; (2) wrongly extends the statement to unrelated companies.
- **TRAP:** If a statement is about ONE specific entity (a country, a company, a person), a conclusion generalizing to "all others" or "no others" almost never follows — this single pattern accounts for a large share of "neither follows" answers.

C4.3 Statement & Argument [Level: Medium]

- Judge whether each argument is **Strong** (directly relevant, practical, addresses the real crux of the issue) or **Weak** (irrelevant, superficial, an appeal to an unrelated example/country, or purely emotional with no practical backing).
- **Checklist for a strong argument:** (a) relevant to the exact issue raised, (b) based on a practical/factual consequence, not a vague generality, (c) not simply "it happens elsewhere" (what another country/company does is rarely, by itself, a strong reason for domestic policy).
- NOTE: Argument-strength questions are the most subjective sub-type in this topic — official answer keys sometimes differ between books. Apply the checklist consistently rather than chasing a single "always correct" rule.

Worked Examples

- *Should military training be compulsory?* Arg 1: "Yes, every child should go through it" (asserts the conclusion again in absolute terms — **weak**, no supporting reason given). Arg 2: "No, it is not compulsory in USA" (irrelevant — a foreign country's policy is not a reason for domestic policy — **weak**). → **Neither is strong.**
- *Should the Disarmament Policy be adopted?* Arg 1: "Yes, it promotes peace and encourages other nations to follow" (practical, addresses real consequence — **strong**). Arg 2: "No, it poses a security threat" (practical, addresses the core risk — **strong**). → **Both are strong** — this is the standard pattern when both sides cite a direct, practical consequence rather than a generality.
- *Should more big industries be started in urban areas?* Arg 1: "Yes, creates job opportunities" (**strong**, practical). Arg 2: "No, increases population and pollution" (**strong**, practical). → **Both strong.**

C4.4 Statement & Assumption [Level: Medium-Hard]

- An assumption is **implicit** — not stated, but *necessary* for the statement to make sense. Test: "If I deny this assumption, does the statement's logic collapse?" If yes, it is a valid (implicit) assumption.
- **Five official rules for spotting a FALSE assumption:**
 1. Words like **EVERY, EACH, NONE, ALL, ONLY** in the assumption → almost always **FALSE** (too absolute to be a safe hidden premise).
 2. An assumption **not tied to** the statement's actual content → **FALSE**.
 3. An assumption about the **past or future** (when the statement is about the present) → **FALSE**.
 4. An assumption that **contradicts** the statement → **FALSE**.
 5. If the statement is **general** but the assumption is **specific** (or vice versa) → **FALSE**.

Worked Examples (rules applied)

- *Statement*: "Liquid soap should be used instead of soap bars to wash hands." *Assumption*: "Soap bars are harsher to the skin than liquid soap." → **Valid/implicit** — this is exactly the unstated reason that makes the recommendation sensible; none of the 5 false-rules apply.
- *Statement*: "Electric Vehicles produce less air pollutants." *Assumption*: "EVs will be the ONLY means of transport in future." → **FALSE** (Rule 1: contains "ONLY"; also Rule 3: talks about the future).
- *Statement (warning)*: "Cellphone use is prohibited beyond this point." *Assumption*: "This warning is not necessary." → **FALSE** (Rule 4: directly contradicts the statement — a warning being posted presupposes it IS necessary).
- *Statement (warning)*: "Paper will get wet because of water." *Assumption*: "Water will NOT make the paper wet." → **FALSE** (Rule 4: a direct, explicit contradiction of the statement).
- *Statement*: "Seats allotted only for physically disabled people." *Assumption*: "ONLY females who are physically disabled can occupy such seats." → **FALSE** (Rule 1: contains "ONLY"; also Rule 5: the statement is gender-general, the assumption narrows it to a specific gender with no basis).
- **TRICK**: Scan first for ALL/EVERY/NONE/ONLY — a huge fraction of "false assumption" answers can be eliminated on this word-spotting check alone, before doing any deeper logical analysis.

C4.5 Statement & Course of Action [Level: Hard]

- A course of action is valid only if it is a **practical, feasible, and administratively reasonable step that directly follows** as a solution/response to the problem in the statement — not merely a related idea, and not something requiring resources/authority far beyond what's implied.
- **TRAP**: A course of action that is *morally desirable* but **impractical or too extreme/costly** for the stated problem is usually rejected — the standard is feasibility, not idealism.
- *Worked example* — *Statement*: "Absenteeism among factory workers has risen sharply in the last quarter." *Suggested actions*: (1) The factory should shut down immediately. (2) Management should investigate the cause of absenteeism and consider incentive schemes. → Action (1) is disproportionate/impractical (**does not follow**); Action (2) is a direct, feasible, practical response (**follows**).

Questions

(ordered Easy → Hard; [Trap] flags a classic misconception)

Q1. [Easy] *Statement*: "Ravi got the highest marks in his class this term." *Conclusion*: "Ravi is the most intelligent student in the class." Does the conclusion follow?

A. Yes, it follows directly B. No — marks reflect this term's performance only, not a general intelligence claim C. Cannot be determined D. It's a strong argument, not a conclusion question

Answer: B. Why: The statement is about one term's marks; "most intelligent" is a broader, unstated generalization — classic overreach (Tip 1: do not assume).

Q2. [Easy] *Statement*: "This medicine should be taken only after meals." Which is the implicit assumption?

A. The medicine is safe on an empty stomach B. Taking it on an empty stomach could cause harm/reduced effect C. All medicines should be taken after meals D. The medicine has no side effects

Answer: B. Why: The instruction only makes sense if there's a reason to avoid an empty stomach; option C uses "all" (Rule 1 — false), option A contradicts the statement's intent (Rule 4 — false).

Q3. [Medium] *Statement*: "Company Z's advertisement says, 'If you love adventure, buy our shoes.'" *Conclusion*: "People who buy the shoes love adventure." Does it follow?

A. Yes B. No — the statement is a conditional aimed at adventure-lovers, it does not claim only they will buy the shoes C. Cannot say D. It's an assumption, not a conclusion

Answer: B. Why: "If you love adventure, buy our shoes" does not logically imply the converse (that all buyers love adventure) — a classic if-then vs converse trap.

Q4. [Medium] [Trap] *Statement*: "The company has decided to promote employees based on performance, not seniority." *Assumption*: "Seniority alone does not guarantee good performance." Is this a valid assumption?

A. Valid — it's exactly the unstated premise behind the policy change B. Invalid — it uses the word "alone" so it's absolute C. Invalid — it talks about the past D. Cannot be determined

Answer: A. Why: The policy shift only makes sense if seniority isn't a reliable performance proxy — none of the 5 false-rules apply (no ALL/EVERY/ONLY, not past/future, not contradicting, not a scope mismatch).

Q5. [Medium] *Statement*: "Should smoking be banned in public places?" *Argument*: "Yes, because it protects non-smokers from passive smoking-related health risks." Is this argument strong or weak?

A. Strong — direct, practical, addresses the core issue B. Weak — too general C. Weak — it's an appeal to emotion D. Cannot be determined

Answer: A. Why: It cites a specific, practical, directly relevant consequence (health risk to non-smokers) — meets the strong-argument checklist fully.

Q6. [Medium] [Trap] *Statement*: "Should India increase its defence budget?" *Argument*: "No, because our neighboring country X has a much smaller defence budget." Is this strong or weak?

A. Strong B. Weak — comparison to another country's budget size is not, by itself, a relevant domestic reason C. Strong, since it's factual D. Cannot say

Answer: B. Why: Being factual isn't enough — the argument must be *relevant* to whether India's own security/practical needs justify the increase; a neighbor's smaller budget doesn't address that.

Q7. [Hard] *Statement*: "All employees earning above ₹10 lakh/year must pay a special surcharge." *Conclusion*: "No employee earning ₹10 lakh or below pays this surcharge." Does it follow?

A. Yes, it follows directly by contrast B. No — the statement doesn't state what happens below the threshold, only what happens above it C. Cannot be determined either way D. It's a course of action, not a conclusion

Answer: A. Why: This is a valid logical contrapositive-style conclusion: the statement fixes the surcharge condition to "above ₹10 lakh" as the entire triggering criterion, so anyone not meeting it is excluded — this is one of the few cases where the "opposite" DOES follow, because the original statement itself draws an exact, exclusive threshold (unlike vaguer statements in Q1/Q3 where such inferences overreach).

Q8. [Hard] *Statement*: "Road accidents have increased by 40% in the city over the past year." *Suggested courses of action*: (1) Immediately ban all private vehicles in the city. (2) Increase traffic police patrolling and install speed cameras at accident-prone junctions. Which follows?

A. Only 1 B. Only 2 — practical, feasible, and directly targets the stated problem C. Both D. Neither

Answer: B. Why: Action 1 is disproportionate and infeasible; Action 2 is a direct, practical, standard administrative response to rising accidents.

Q9. [Hard] [Trap] *Statement*: "The exam result of School A has been the best in the district for the third consecutive year." *Conclusion*: "School A will also top the district next year." Does it follow?

A. Yes — the trend guarantees it B. No — past performance across 3 years does not logically guarantee a future (4th year) outcome C. Cannot be determined — insufficient data on other schools D. Both B and C capture the flaw, but B is the core reasoning error

Answer: D. Why: This combines two errors: (1) any claim about the FUTURE from a purely past-tense statement is invalid (Rule 3 territory from the Assumption rules, applicable to conclusions too), and (2) the statement is not even a conclusion, it's a prediction. B correctly identifies why it fails, C is also technically true — D acknowledges both but flags B as the primary reasoning error tested.

Q10. [Hard] *Statement*: "Government warning on cigarette packets: 'Smoking is injurious to health.'" *Assumption*: "People read what is printed on cigarette packets." Is this a valid assumption?

A. Valid — the warning's entire purpose depends on people reading it B. Invalid — contains no absolute word C. Invalid — it's about the future D. Cannot be determined

Answer: A. Why: The warning is pointless unless we assume it will be read — a textbook example of a valid implicit assumption (none of the 5 false-rules trigger: no ALL/EVERY/ONLY, not past/future-only, doesn't contradict, matches scope).

9.C5 Calendar

(A pure-formula chapter — once the odd-days method is internalized, every question type below becomes mechanical. Source slides for this topic were image-only; all content below is standard, verified calendar arithmetic.)

C5.1 Odd Days — The Foundational Idea [Level: Basic]

- A week has 7 days. **Odd days** = days left over after removing complete weeks from a span.
- Ordinary year = 365 days = 52 weeks + **1 odd day**. Leap year = 366 days = 52 weeks + **2 odd days**.

Days in each month (ordinary year):
 Jan 31 Feb 28 Mar 31 Apr 30 May 31 Jun 30
 Jul 31 Aug 31 Sep 30 Oct 31 Nov 30 Dec 31
 (Leap year: Feb = 29)

- Odd days per month (ordinary year, mod 7): Jan=3, Feb=0, Mar=3, Apr=2, May=3, Jun=2, Jul=3, Aug=3, Sep=2, Oct=3, Nov=2, Dec=3.

C5.2 Leap Year Rule [Level: Basic]

A year is a leap year IF divisible by 4,
 EXCEPT a century year (divisible by 100),
 which is a leap year ONLY IF also divisible by 400.

- TRAP:** 1900 and 2100 are **NOT** leap years (divisible by 100, not by 400); 2000 and 2400 **ARE** leap years (divisible by 400). This century-exception is the single most tested leap-year fact.

C5.3 Odd Days Over Long Spans — Breaking a Year into Pieces [Level: Basic-Medium]

Odd days in 100 years = 5
 Odd days in 200 years = 3
 Odd days in 300 years = 1
 Odd days in 400 years = 0 (and repeats: 800, 1200... also = 0)

- TRICK: Because 400 years always contribute exactly 0 odd days, **the calendar exactly repeats every 400 years** — this single fact is *why* the century rule (divisible by 400) exists at all: it keeps the 400-year block's leap-day count constant.
- "Leap year calendar repeats after 28 years"** — true ONLY when no century year falls inside that 28-year span (a 28-year gap normally contains exactly 7 leap years including itself $\rightarrow 7 \times 2 + 21 \times 1 = 35$ odd days = exactly 0 mod 7). **TRAP:** if a non-leap century year (1900, 2100...) falls inside the 28-year window, this shortcut breaks and a longer/adjusted gap is needed instead.

C5.4 Finding the Day of the Week for Any Date [Level: Medium]

Weekday code: 0=Sunday 1=Monday 2=Tuesday 3=Wednesday 4=Thursday 5=Friday 6=Saturday

Total odd days = (odd days from all COMPLETE years before the given year)
 + (odd days from all COMPLETE months before the given month, in the given year)
 + (the date number itself)

Answer weekday = (Total odd days) mod 7 --> look up in the code table above

- Ways to compute "odd days from complete years":**
 - Direct/reference method:** count forward or backward from a known anchor date (e.g., "1 Jan 2000 = Saturday") using the per-century/leap counting rules above.
 - Formula method:** for Y complete years, odd days = $[Y + \text{floor}(Y/4) - \text{floor}(Y/100) + \text{floor}(Y/400)] \bmod 7$ (this single formula automatically encodes the century exception).

Worked Example (verified)

Find the day of the week on **15 August 1947**.
 Complete years before 1947 = 1946 = 1900 + 46.
 • 1900 years $\rightarrow$ split as 4×400 (0 odd days each) + 300 (1 odd day) = **1 odd day**.
 • Remaining 46 years (1901–1946): leap years among them = 1904, 1908, ..., 1944 = **11** leap years. Odd days = $46 + 11 = 57 \rightarrow 57 \bmod 7 = 1$.
 • Total for 1946 complete years = $1 + 1 = 2$ odd days.
 Complete months before August in 1947 (not a leap year): Jan(3)+Feb(0)+Mar(3)+Apr(2)+May(3)+Jun(2)+Jul(3) = 16 $\rightarrow 16 \bmod 7 = 2$.
 Add the date itself: **15** $\rightarrow 15 \bmod 7 = 1$.
 Total = $2 + 2 + 1 = 5 \rightarrow$ weekday code 5 = **Friday**. ✓ (Matches the well-known historical fact that India's independence fell on a Friday.)

C5.5 Date-to-Date Relationship Problems [Level: Medium-Hard]

A useful way to classify these problems by rising difficulty:

- Same year, no century/leap complication** — just count elapsed days between the two dates mod 7.
- Spanning a Feb 29** — **TRAP:** if the range crosses a leap year's Feb 29, add **one extra odd day** for that crossing; forgetting this is the most common error in this category.
- Spanning one or more centuries** — must invoke the 100/200/300/400-year odd-day table (C5.3) rather than just counting leap years by inspection.
- Multi-century + partial-year combination** — combine (1)+(3): break the total span into whole-century blocks plus a remainder of years plus a remainder of months/days, and sum all three odd-day contributions mod 7.

C5.6 Which Months Share the Same Calendar? [Level: Medium]

In a **non-leap** year, these month-pairs/groups always start on the same weekday (verified: elapsed days between each pair's 1st is a multiple of 7):

January <-> October (273 days apart = 39 weeks exactly)
 February <-> March <-> November
 April <-> July (91 days apart = 13 weeks exactly)
 September<-> December (91 days apart = 13 weeks exactly)

- **TRAP:** The **February↔March↔November** group holds only in a **non-leap** year — a leap year's extra day in February shifts March (and everything after) by one day, breaking this specific triple (April↔July and September↔December are unaffected either way, since they don't span Feb 29).

C5.7 Same Calendar in Different Years [Level: Hard]

Two years have an **identical full calendar** (every date falls on the same weekday) only if **both** conditions hold:

1. Total odd days between the two years is a multiple of 7, AND
2. Both years have the SAME leap-year status (both leap, or both non-leap)

- **TRAP:** Condition 2 is the one students forget — matching odd-day count alone is not sufficient; if one year is leap and the other isn't, the Feb 29 shift means the calendars cannot fully coincide even if the odd-day arithmetic superficially lines up.
- Non-leap years typically repeat every **6 or 11 years** later (varies with the specific leap years skipped in between); the **full 400-year cycle** is the only guarantee that works unconditionally for every date pairing.

Questions

(ordered Easy → Hard; all answers independently verified by odd-day computation)

Q1. [Easy] Which of these is a leap year?

A. 1900 B. 2000 C. 2100 D. 1800

Answer: B. **Why:** 2000 is divisible by 400 → leap. 1900, 2100, 1800 are divisible by 100 but not 400 → not leap.

Q2. [Easy] How many odd days are there in 400 years?

A. 5 B. 3 C. 1 D. 0

Answer: D. **Why:** The 400-year block always contains exactly 0 odd days — this is why the calendar repeats every 400 years.

Q3. [Medium] If 1 January of a certain non-leap year is a Monday, what day is 1 October of the same year?

A. Sunday B. Monday C. Tuesday D. Wednesday

Answer: B. **Why:** January and October are always the same weekday in a non-leap year (273 days = exactly 39 weeks apart).

Q4. [Medium] [Trick] If 1 February of a non-leap year is a Thursday, what day is 1 March of the same year?

A. Wednesday B. Thursday C. Friday D. Saturday

Answer: B. **Why:** February has 28 days in a non-leap year = exactly 4 weeks (0 odd days), so March 1 falls on the identical weekday as February 1.

Q5. [Medium] [Trap] A person says "the calendar of 2001 is identical to the calendar of 2002 since consecutive years usually differ by just 1 odd day, and I've heard some consecutive years match." Is this reasoning valid?

A. Yes B. No — 2001 has 1 odd day of its own, so 2002 starts one weekday later; they cannot be identical C. Yes, if both are non-leap D. Cannot be determined

Answer: B. **Why:** Any single non-leap year always contributes exactly 1 odd day, meaning the following year's calendar shifts by exactly one weekday — consecutive years can never have an identical calendar (a minimum 2-year gap, specifically 5, 6, or 11 years depending on leap placement, is required).

Q6. [Hard] What was the day of the week on 1 January 1901?

A. Monday B. Tuesday C. Wednesday D. Thursday

Answer: B. **Why:** Odd days for 1900 complete years = 1 (from C5.3: $1900 = 4 \times 400 + 300 \rightarrow 0 + 1 = 1$). 1 Jan 1901 is the day right after 31 Dec 1900, so total odd days = 1 (no months/dates elapsed yet in the new count since we want 1 Jan itself, i.e., 1900 complete years + 1 day) $\rightarrow 1 + 1 = 2 \rightarrow$ weekday code 2 = Tuesday.

Q7. [Hard] Find the day of the week on 26 January 1950 (Republic Day of India — a well-known historical anchor).

A. Wednesday B. Thursday C. Friday D. Saturday

Answer: B. **Why:** Complete years before 1950 = $1949 = 1900(1 \text{ odd day}) + 49 \text{ years}(1901-1949)$. Leap years in that 49-year span: $1904 \dots 1948 = 12$ leap years. Odd days = $49 + 12 = 61 \rightarrow 61 \bmod 7 = 5$. Total for 1949 years = $1 + 5 = 6$. Complete months before January = 0. Add date 26 $\rightarrow 26 \bmod 7 = 5$. Total = $6 + 0 + 5 = 11 \rightarrow 11 \bmod 7 = 4 \rightarrow$ weekday code 4 = Thursday.

Q8. [Hard] [Trap] Two dates are exactly 5 years apart (e.g., from 1 Jan Year X to 1 Jan Year X+5), and there are exactly 1 leap year in between. What is the total odd-day shift between the two 1-Januaries?

A. 5 B. 6 C. 7 (i.e., same weekday) D. 4

Answer: B. **Why:** 5 years contribute a base of 5 odd days, plus 1 extra for the single leap year crossed = 6 total odd days shift — NOT a full week, so the two dates fall on different weekdays ($6 \bmod 7 = 6$, not 0). This illustrates why "same calendar" requires a very specific odd-day-multiple-of-7 AND matching leap status (C5.7), not just "roughly similar" year gaps.

9.C6 Seating Arrangements

C6.1 Basic Concept [Level: Basic]

- A block of raw clues (positional + relative statements) must be converted into one definite arrangement of people/objects.
- Four standard formats: **Single Row**, **Double Row** (two rows facing each other), **Circular Table**, **Polygonal/Square Table** (mixed corner + edge seating).

C6.2 The Left-Right Facing Trap [Level: Basic — the single most important idea in this chapter]

Person facing NORTH: their Left/Right = same as an outside observer's Left/Right (as drawn on a map).
 Person facing SOUTH: their Left/Right = MIRRORED vs an outside observer (observer's left = person's right).

- TRAP:** When two people in a row face **opposite directions**, "immediate left" for one is "immediate right" for the other — always fix an orientation (usually North, matching map convention) and re-derive left/right for anyone facing the opposite way; never assume everyone's left/right agree.
- Circular table:** facing **inward** (toward center) → your right hand points **clockwise** (as drawn from above). Facing **outward** (away from center) → your right hand points **counter-clockwise** — the exact reverse of facing inward. Mixed inward/outward arrangements require checking each person's facing individually before applying "left/right."

C6.3 Master Method [Level: Basic-Medium]

Step 1: Count total people; draw a position template (numbered seats) as a "Seating Frame."
 Step 2: Place every ABSOLUTE/direct clue first (e.g., "X sits at the left end," "Y sits 2nd from right").
 Step 3: Use RELATIVE clues (immediate left/right, "between," "not adjacent to") to eliminate remaining slots.
 Step 4: Re-scan all clues after every placement — a new fact often unlocks another clue.
 Step 5: If 2 arrangements still satisfy every clue, check whether the question asks something true in BOTH (then it's still answerable) or need

C6.4 Fully Worked Example — Double Row [Level: Hard]

(Real puzzle, independently solved and verified below.)

Five boys (Ashwin, Dipesh, Eshan, Chetan, Bipin) and five girls (Parul, Komal, Radha, Savita, Vimla) sit in two rows facing each other — boys in one row, girls in the other. Clues: Eshan is immediate right of Bipin, opposite Parul, not at any end. Radha is immediate right of Komal, opposite Chetan, at one of the ends. Ashwin is opposite Komal, who is 3rd to the right of Savita. Dipesh and Vimla are NOT opposite each other.

Solving (positions 1–5 left→right, column i of one row directly opposite column i of the other):

```
Radha = immediate right of Komal AND at an end -> Radha must be at position 5 (rightmost);
  (position 1 is impossible: nothing sits "left of" position 1 for Komal to occupy)
=> Girls: pos5=Radha, pos4=Komal
Radha opposite Chetan => Boys: pos5=Chetan
Ashwin opposite Komal => Boys: pos4=Ashwin
Komal is 3rd right of Savita => Savita=pos1 (Girls)
Remaining boys {Bipin,Eshan,Dipesh} fill positions {1,2,3}; Eshan=Bipin+1, Eshan not at an end.
Try Bipin=1,Eshan=2,Dipesh=3 -> Parul(opposite Eshan)=pos2, Vimla=pos3
  -> Dipesh(pos3) opposite Vimla(pos3): VIOLATES "not opposite" clue. Reject.
Try Bipin=2,Eshan=3,Dipesh=1 -> Parul(opposite Eshan)=pos3, Vimla=pos2
  -> Dipesh(pos1) opposite Savita(pos1): no violation. ACCEPTED.

Final: Boys: 1-Dipesh 2-Bipin 3-Eshan 4-Ashwin 5-Chetan
        Girls: 1-Savita 2-Vimla 3-Parul 4-Komal 5-Radha
```

- Q: Who is in the middle of the boys' row? → position 3 = **Eshan** (a "None of these" answer if Eshan isn't listed as an option — classic trap where the obvious-looking named options are all wrong).
- Q: Who is immediately right of Parul (pos 3)? → pos 4 = **Komal**.
- Q: Who is opposite Dipesh (pos 1)? → pos 1 girls = **Savita**.
- Q: How many boys between Ashwin (pos 4) and Bipin (pos 2)? → only position 3 (Eshan) → **1**.

C6.5 Fully Worked Example — Circular, Facing Outward [Level: Hard]

Six friends A,B,C,D,E,F sit around a circular table **facing outward**. E is between B and C. B and F are exactly opposite each other. A is exactly to the right of B.

Solving (positions 1–6 clockwise; facing outward ⇒ "right" = counter-clockwise ⇒ right-neighbor of seat k is seat k–1):

```
Let B = seat 1. Opposite (3 apart on a 6-seat table) => F = seat 4.
A = right of B = counter-clockwise neighbor of seat1 = seat 6.
Remaining seats {2,3,5} for {C,D,E}. E is between B and C => E adjacent to B(seat1) => E=seat2.
E's other neighbor (seat3) must be C (E is "between B and C") => C = seat3. Remaining D = seat5.

Final (clockwise): 1-B 2-E 3-C 4-F 5-D 6-A
```

- Q: Who is between D(5) and B(1)? → going the short way 5→6→1 passes seat 6 = **A**.
- Q: Who is exactly to the right of D(5)? → right = counter-clockwise neighbor = seat 4 = **F**.
- Q: Neighbors of A(6)? → seats 5 and 1 = **D and B**.

C6.6 Double-Row "Opposite" Convention & Polygonal Tables [Level: Hard]

- **TRAP:** In a double-row facing-each-other setup, two people in the SAME numbered column (counted left-to-right by an outside observer for both rows) are "opposite" each other — do not re-flip left/right per row; only re-flip left/right when a question asks about a specific PERSON's own left/right, not about column position.
- **Polygonal/square tables** (people at corners facing center + people at edge-midpoints facing outward, or similar mixed setup): treat corner-facing-center and edge-facing-outward people with their OWN separate left/right rule (from C6.2) — the two groups' "right" directions can point opposite ways even though they share one table, so re-derive per person, never assume uniformly.

Questions

(All fully self-contained text puzzles, Easy → Hard; [Trap] marks the facing-direction reversal trap.)

Q1. [Easy] Five friends sit in a row facing North. If P is immediately to the left of Q, and Q is immediately to the left of R, what is P's position relative to R?

A. Immediately right of R B. Two places to the left of R C. Immediately left of R D. Cannot be determined

Answer: B. Why: P—Q—R left to right, so P is 2 places left of R (both facing North, so map-left = their own left; no reversal needed here).

Q2. [Medium] [Trap] Six people sit in a row. A faces North, B faces South. If A says "C is to my right," and B is sitting exactly where A is looking from the opposite side, is C also to B's right?

A. Yes, always B. No — since B faces the opposite direction, C would be to B's LEFT C. Cannot be determined without more information D. Only if C sits between A and B

Answer: B. Why: A and B face opposite directions, so their left/right senses are mirrored; whichever side is "right" for A is "left" for B (C6.2's core trap).

Q3. [Medium] Six friends P,Q,R,S,T,U sit in a row facing North. R is immediately right of Q. Q is 2nd from the left end. T is immediately left of Q. S is at the rightmost (6th) end. Who is at the leftmost end?

A. T B. P C. U D. Cannot be determined

Answer: A. Why: Q is fixed at position 2. T is immediately left of Q ⇒ T = position 1 = the leftmost end.

Q4. [Medium] Six friends sit around a circular table facing the CENTER. E is between B and D. A is to the immediate right of B. If facing outward instead (all else identical), would A still be to the immediate right of B?

A. Yes, right stays the same regardless of facing B. No — facing outward reverses the clockwise/counter-clockwise sense of "right," so A's seat relative to B would need to be re-derived C. Only if E also changes seats D. Cannot be determined

Answer: B. Why: "Right" flips between clockwise and counter-clockwise depending on whether the group faces inward or outward — this is C6.2's central rule.

Q5. [Hard] (Using the double-row puzzle solved in C6.4) Who is the immediate neighbor of Chetan on the boys' side?

A. Ashwin only B. Dipesh only C. Bipin and Ashwin D. Eshan and Ashwin

Answer: A. Why: Boys row = Dipesh(1)-Bipin(2)-Eshan(3)-Ashwin(4)-Chetan(5); Chetan (pos 5, an end) has only ONE neighbor: Ashwin (pos 4).

Q6. [Hard] (Using the circular facing-outward puzzle in C6.5) If B and D swap seats, who becomes the immediate right neighbor of F?

A. E B. C C. The new occupant of seat 5 D. A

Answer: B. Why: Original: 1-B 2-E 3-C 4-F 5-D 6-A. Swapping B(seat1)↔D(seat5) only changes seats 1 and 5: new layout is 1-D 2-E 3-C 4-F 5-B 6-A. Right-neighbor of F(seat4) = seat(4−1) = seat 3 = **C**, untouched by the swap.

Q7. [Hard] [Trap] In a double row facing each other, X (row 1) is exactly opposite Y (row 2). X says Y is sitting to X's left-front. Is Y necessarily sitting toward what an outside observer would call the "left" side of the table?

A. Yes, always B. Not necessarily — "left" from X's own facing direction may be the observer's right if X faces a direction where left/right is mirrored C. Only if both rows face North D. Cannot be determined ever

Answer: B. Why: As in C6.2/C6.6, a person's own stated left/right depends on their facing direction, not a fixed map direction — always convert to the observer's frame before matching against absolute clues like "the left end of the hall."

Q8. [Hard] Eight people sit at a square table: four at corners (facing center) and four at edge-midpoints (facing outward). P sits at a corner. Q sits at an edge-midpoint immediately clockwise (as drawn) of P. From Q's own perspective, is P to Q's left or right?

A. Left, because Q faces outward and P is clockwise of Q, which is Q's left B. Right, always C. Depends only on P's facing, not Q's D. Cannot be determined

Answer: A. Why: Q faces outward, so Q's "right" = counter-clockwise (per C6.2); since P is clockwise of Q, P must be on Q's LEFT — a direct application of the outward-facing reversal rule to a mixed polygonal table (C6.6).

Q9. [Hard] Using the double-row puzzle in C6.4, if the girls' row is now described from THEIR own left (i.e., mirrored versus the observer used in the solution), who is at "girls' own rightmost end"?

A. Savita B. Radha C. Komal D. Vimla

Answer: A. Why: The girls face the boys (opposite direction to the observer's implicit "left-to-right" numbering used for boys); the girls' own right end is the observer's LEFT end of the girls' row = position 1 = Savita — another direct application of the facing-reversal trap, now on a solved arrangement.

9.C7 Coding-Decoding

C7.1 Basic Concept [Level: Basic]

- A word/number/phrase is transformed by a **consistent hidden rule**. You are given one or more example transformations and must decode the rule, then apply it to a new input.
- Master method for every type below: **compare position-by-position** between the given input and its code; do not guess the rule from the whole word at once.

C7.2 Type 1 — Letter/Word-Based Coding [Level: Basic-Medium]

Alphabet position: A=1, B=2, C=3, ... Z=26
Reverse position: position + reverse-position = 27 (so Z=1st-from-end, A=26th-from-end, etc.)

- Fixed shift:** every letter moves forward/back by the SAME amount (e.g., CAT → FDW is a +3 shift on each letter).
- Variable shift:** the shift amount changes per letter position (e.g., +1, +2, +3... — check the shift for letter 1, then letter 2, etc., before assuming it's fixed).
- Reversal coding:** letters reversed in order, and/or each letter replaced by its "opposite" via the 27-minus rule ($A \leftrightarrow Z$, $B \leftrightarrow Y$, $C \leftrightarrow X$...).
- TRAP:** Never assume a fixed shift after checking only the first letter — always verify against at least 2 letters before locking in the rule (a variable shift looks identical to a fixed one on a single letter).

C7.3 Type 2 — Number-Based Coding [Level: Medium]

- Letters/words mapped to numbers, either via straightforward alphabet position, or via an **arbitrary assigned code** that must be cross-referenced across 2+ given example sentences to isolate which number belongs to which word (treat it like solving simultaneous equations: find a word common to two examples, its number must be common too — the differing words then map to the differing numbers).

C7.4 Type 3 — Mixed Alphabet/Word/Number Coding [Level: Medium-Hard]

- Combines Types 1 and 2 in the same question. Solve by isolating ONE part of the rule at a time (e.g., first determine how numbers are assigned, holding the letter-rule aside, then determine the letter-rule using the now-known number pattern).

C7.5 Type 4 — Symbol-Based Coding [Level: Medium]

- Letters/words replaced by arbitrary symbols. This is a pure substitution-cipher lookup once a mapping table is built from the given examples — no arithmetic trick, just careful bookkeeping of which symbol equals which letter/word.

C7.6 Type 5 — Language/"Chinese Coding" Substitution [Level: Hard]

- A fictional-language sentence maps to an English sentence; you're given 2+ paired example sentences and must decode a new sentence (or a specific word).
- Method (substitution-elimination, like solving simultaneous equations):**

Step 1: Find a fictional word that appears in TWO given example sentences.
Step 2: Find the English word common to both corresponding English sentences.
=> That fictional word means that common English word.
Step 3: Eliminate the now-known word from each sentence; the remaining fictional word(s) must map to the remaining English word(s) in that sentence.
Step 4: Repeat until every word is mapped, or until you've mapped the specific word(s) the question asks about.

- Worked example: "sim tam kol" = "roses are red" and "sim del pic" = "roses smell nice". Common fictional word = "sim"; common English word = "roses" ⇒ **sim = roses**. Then "tam kol" = "are red" and "del pic" = "smell nice" (no further overlap needed if the question only asks for "roses").

Questions

(Easy → Hard; [Trick] = reward for knowing a shortcut; [Trap] = common misconception)

Q1. [Easy] If CAT is coded as DBU, what is the coding rule?

A. Each letter -1 B. Each letter +1 C. Reverse the letters D. Replace with opposite letter

Answer: B. Why: C→D(+1), A→B(+1), T→U(+1) — consistent +1 shift on every letter.

Q2. [Easy] Using the same +1 rule from Q1, how is DOG coded?

A. EPH B. CNF C. EOG D. EPI

Answer: A. Why: D→E, O→P, G→H ⇒ EPH.

Q3. [Medium] [Trap] If BAT is coded as YZG (using $A \leftrightarrow Z$, $B \leftrightarrow Y$, ... 27-minus-position rule), what is CAT coded as?

A. XZG B. XZH C. ZYG D. YZH

Answer: A. Why: Position(B)=2→27-2=25=Y ✓; A=1→26=Z ✓; T=20→7=G ✓ (confirms the rule). For CAT: C=3→24=X; A=1→26=Z; T=20→7=G ⇒ **XZG**.

Q4. [Medium] In a code, ROSE is written as 6821 and CHAIR is written as 73456. What is the code for SEARCH?

A. 214673 B. 246173 C. 214763 D. 146273

Answer: A. Why: From ROSE: R=6, O=8, S=2, E=1. From CHAIR (R=6 consistent): C=7, H=3, A=4, I=5. SEARCH = S-E-A-R-C-H = 2-1-4-6-7-3 = **214673**.

Q5. [Medium] [Trick] If MONEY is coded as NQQID, what shift pattern is being used?

A. Fixed shift of +1 B. Fixed shift of +2 C. Variable shift, increasing by 1 for each successive letter D. Reversal coding

Answer: C. Why: M(+1)=N, O(+2)=Q, N(+3)=Q, E(+4)=I, Y(+5)=D (wrapping past Z: 25+5=30, 30-26=4=D) — the shift grows by 1 each letter, exactly the C7.2 trap of assuming a single fixed shift after checking only the first letter.

Q6. [Hard] In a symbol code, "@" = A, "#" = B, "\$" = C, and words are spelled out symbol-by-symbol in order. If "BAG" is coded as "#@%", what does "%" represent?

A. A B. B C. G D. Cannot be determined from other examples

Answer: C. Why: B→#, A→@ (both already known), so by elimination the remaining symbol % must represent the remaining letter G.

Q7. [Hard] Type 5: "pil mit sog" = "flowers are beautiful" and "rin mit kot" = "roses are fragrant". What does "mit" mean?

A. flowers B. are C. beautiful D. fragrant

Answer: B. Why: "mit" is common to both fictional sentences; "are" is the only word common to both English sentences ⇒ mit = are.

Q8. [Hard] Using Q7's data, what does "pil" most likely mean, given "pil" appears only in the first sentence alongside "sog"?

A. flowers B. are C. beautiful D. Cannot be determined from these two sentences alone

Answer: D. **Why:** With only 2 sentences and "pil"/"sog" both unique to sentence 1 (mapping to "flowers"/"beautiful" in some order), there is no third overlapping sentence to disambiguate which of "pil" or "sog" is "flowers" vs "beautiful" — a genuine coding-decoding question always needs enough overlapping examples to fully pin down every word; this is the classic trap of assuming a unique answer exists when the given data is actually insufficient.

Q9. [Hard] [Trap] A code assigns EACH letter of "STRONG" a number equal to its position in the alphabet MINUS its position in the word (1st letter, 2nd letter, etc.). What number codes the letter 'O' (4th letter of STRONG)?

A. 11 B. 15 C. 19 D. 9

Answer: A. **Why:** Position of O in alphabet = 15. Position in word STRONG: S(1)T(2)R(3)O(4)N(5)G(6) $\Rightarrow$ O is 4th. $15 - 4 = 11$.

9.C8 Blood Relations & Directions

C8.1 Blood Relations — Basic Concept [Level: Basic]

- Deduce a relationship between two people from a chain of stated relations (often through an unnamed intermediate person).
- Core reference list** (memorize — this list IS the shortcut):

Given relation	Resolves to
Mother's/Father's son	Brother (or self, if speaker is male — always check for self-reference)
Mother's/Father's daughter	Sister (or self, if speaker is female)
Father's son's son	Nephew (brother's son) — or own son, if "son" refers to the speaker
Mother's father	Grandfather
Father's sister	Paternal aunt
Mother's brother	Maternal uncle (mama)
Son's wife	Daughter-in-law
Husband's/Wife's sister	Sister-in-law
Husband's/Wife's brother	Brother-in-law
Sister's husband	Brother-in-law

C8.2 Method & the Self-Reference Trap [Level: Medium]

Step 1: Work from the OUTERMOST relation inward (start decoding from the last-stated link closest to the person named in the question), OR build a small generation-level family tree in text (Grandparents → Parents → Speaker's generation → Children).

Step 2: Track gender as "unknown" until a pronoun (he/she) or explicit word (brother/sister/son/daughter) fixes it.

Step 3: ALWAYS check whether a long relation chain loops back to the ORIGINAL speaker — e.g., "my father's son" is NOT always "my brother"; if the speaker has no other brother, "my father's son" IS the speaker himself.

- TRAP:** This self-reference loop is the single most common trap in the entire chapter — always ask "could this relation just mean the speaker?" before naming a new person.

C8.3 Coded Blood Relations [Level: Medium-Hard]

- Relations replaced by symbols/operators, e.g., " $A + B$ " = A is B's mother, " $A \times B$ " = A is B's brother, " $A - B$ " = A is B's father, " $A \div B$ " = A is B's sister (the exact symbol-meaning mapping is always GIVEN in the question — never assumed).
- Method: substitute the given meanings and trace the chain exactly as in C8.2, one link at a time, writing out the plain-language relation as you go rather than trying to hold the whole symbolic chain in your head.

C8.4 Directions — Basic Concept [Level: Basic]

Right-turn sequence (clockwise): $N \rightarrow E \rightarrow S \rightarrow W \rightarrow N$
 Left-turn sequence (counter-clockwise): $N \rightarrow W \rightarrow S \rightarrow E \rightarrow N$

- 4 main directions (N,E,S,W) + 4 intermediates (NE,SE,SW,NW) at 45° between them.

C8.5 Coordinate-Tracking Method [Level: Medium — the master technique]

Treat North/South as the Y-axis (+ for North, - for South).
 Treat East/West as the X-axis (+ for East, - for West).
 Add up every leg of the journey on its own axis; do NOT try to visualize the whole path — just accumulate (x, y) separately.
 Final straight-line/shortest distance from start = $\sqrt{x^2 + y^2}$ [Pythagoras]

- This coordinate method is far more reliable than mental path-tracing once a journey has 3+ turns.

C8.6 Shadow Direction [Level: Basic — a standalone fact]

- At **sunrise**, the sun is in the East, so a shadow falls toward the **West**.
- At **sunset**, the sun is in the West, so a shadow falls toward the **East**.
- TRAP:** Students frequently reverse these — anchor it by remembering "the shadow always falls AWAY from the sun."

Questions

(Easy → Hard; [Trap] tags the two classic error categories: self-referential blood relations, and sunrise/sunset shadow confusion.)

Q1. [Easy] Pointing to a man, a woman says, "His mother is my mother's only daughter." How is the man related to the woman?

A. Brother B. Son C. Father D. Uncle

Answer: B. **Why:** "My mother's only daughter" = the speaker herself (she has no sisters). So "his mother" = the speaker ⇒ the man is her **son**.

Q2. [Easy] A is B's father. B is C's brother. How is A related to C?

A. Father B. Grandfather C. Uncle D. Brother

Answer: A. Why: B and C are siblings, and A is B's father $\Rightarrow$ A is also C's **father** (same parent for both siblings, absent any statement to the contrary).

Q3. [Medium] Pointing to a photograph, a man says, "She is the daughter of my grandfather's only son." How is the woman in the photo related to the man?

A. Sister B. Wife C. Daughter D. Cannot be determined

Answer: A (or the speaker himself, if female — but since the speaker refers to "she," treat as distinct). **Why:** "My grandfather's only son" = the speaker's father (assuming the speaker's father is that only son) $\Rightarrow$ "daughter of my father" = the speaker's **sister**. [Trap check: could "my grandfather's only son" be the speaker himself? Only if the speaker is male and has no father alive in the frame — the standard reading takes it as the speaker's father since a woman is being described as a separate person "she".]

Q4. [Medium] [Trap] A man said to a woman, "Your mother's husband's sister is my aunt." How is the woman related to the man?

A. Sister B. Cousin C. Niece D. Mother

Answer: B. Why: "Your mother's husband" = the woman's father. "Father's sister" = the woman's paternal aunt. That same aunt is also the man's aunt $\Rightarrow$ the man and the woman share a grandparent through that aunt's sibling (the man's parent and the woman's father are siblings) $\Rightarrow$ they are **cousins**.

Q5. [Medium] In a family, P is the mother of Q. Q is the brother of R. R is the son of S. How is S related to P?

A. S is P's husband B. S is P's brother C. S is P's daughter D. Cannot be determined

Answer: A. Why: Q and R are siblings (same parents). P is their mother, so R's father must be P's husband. Since R is "the son of S," S is R's father $\Rightarrow$ **S is P's husband**.

Q6. [Hard] Coded relations: "A + B" means A is the mother of B; "A $\times$ B" means A is the brother of B; "A – B" means A is the father of B. If "P + Q $\times$ R – S" holds, how is S related to P?

A. Grandson B. Granddaughter C. Grandfather D. Son

Answer: A. Why: P+Q $\Rightarrow$ P is Q's mother. Q $\times$ R $\Rightarrow$ Q is R's brother (so Q and R are siblings, same mother P). R–S $\Rightarrow$ R is S's father. So S is R's child, R is P's child $\Rightarrow$ S is P's **grandson** (assuming S's gender is male per "grandson"; if ungendered, "grandchild" — question presumes male by the options given).

Q7. [Hard] Starting from point X, a man walks 5 km North, then turns right and walks 3 km, then turns right again and walks 5 km, then turns left and walks 4 km. How far is he from X, and in which direction?

A. 7 km, East B. 7 km, North C. 9 km, East D. 3 km, North

Answer: A. Why (coordinate method): Start (0,0). North 5 $\Rightarrow$ (0,5). Turn right (facing North $\rightarrow$ right = East), walk 3 $\Rightarrow$ (3,5). Turn right again (facing East $\rightarrow$ right = South), walk 5 $\Rightarrow$ (3,0). Turn left (facing South $\rightarrow$ left = East), walk 4 $\Rightarrow$ (7,0). Final position (7,0) $\Rightarrow$ **7 km East** of X.

Q8. [Hard] A man walks 8 km East, then 6 km South, then 8 km West. How far and in which direction is he from the start, and what is the straight-line distance?

A. 6 km South B. 6 km, direction South of start C. 10 km, South-West D. 14 km South

Answer: B. Why (coordinate method): (0,0) $\rightarrow$ East 8 $\rightarrow$ (8,0) $\rightarrow$ South 6 $\rightarrow$ (8,-6) $\rightarrow$ West 8 $\rightarrow$ (0,-6). Net displacement (0,-6) $\Rightarrow$ exactly **6 km due South** — the East and West legs (both 8 km) cancel out completely, leaving only the South component.

Q9. [Hard] [Trap] One morning, a man's shadow fell exactly to his LEFT as he stood still. Which direction was he facing?

A. North B. South C. East D. West

Answer: A. Why: Morning sun is in the East, so shadows fall toward the West (C8.6). Facing North, West is on your left (standard map orientation) — so the shadow-on-the-left condition places him **facing North**.

Q10. [Hard] [Trap] A tower's shadow is longer than the tower's height. Is it more likely to be near sunrise/sunset or near noon?

A. Near noon, when the sun is directly overhead B. Near sunrise or sunset, when the sun is low on the horizon, casting a long shadow C. Shadow length is unrelated to time of day D. Cannot be determined

Answer: B. Why: The lower the sun's angle above the horizon (as at sunrise/sunset), the longer the shadow cast by a fixed-height object; near noon the sun is high, giving the shortest shadow.

9.C9 Clocks

C9.1 Basic Mechanics [Level: Basic]

- Clock dial = circle = 360° , divided into 60 minute-spaces ($60 \times 6^\circ = 360^\circ$).
- Hour hand ("short hand") and Minute hand ("long hand") speeds:

Minute hand: $360 \text{ deg} / 60 \text{ min} = 6 \text{ deg/minute}$
 Hour hand: $360 \text{ deg} / (12 \times 60) \text{ min} = 0.5 \text{ deg/minute}$
 Relative speed (minute hand gains on hour hand) = $6 - 0.5 = 5.5 \text{ deg/minute}$

- TRICK: The minute hand is exactly **12 times faster** than the hour hand ($6 / 0.5 = 12$) — this single ratio underlies every clock formula below.

C9.2 Angle Between Hands [Level: Basic-Medium]

Angle = $|30H - 5.5M|$ degrees
 (H = hour shown, 0-11 form; M = minutes past that hour)
 If the result exceeds 180 degrees, subtract it from 360 to get the actual (reflex-avoided) angle.

- Derivation: Hour hand position from 12 = $30H + 0.5M$ degrees. Minute hand position from 12 = $6M$ degrees. Difference = $30H + 0.5M - 6M = 30H - 5.5M$.

Ways to solve "angle at exact o'clock" (M=0)

1. **Formula method:** $|30H - 0| = 30H$ (or $360 - 30H$ if > 180).
2. **Counting method:** each hour-mark = 30° apart; count the number of hour-marks between the hands and multiply by 30.
 - 5 o'clock: 5 marks $\times 30^\circ = 150^\circ$. 8 o'clock: 4 marks (shorter way round) $\times 30^\circ = 120^\circ$. 2 o'clock: 2 $\times 30^\circ = 60^\circ$.

Worked Examples (increasing difficulty)

At 5:30 $\rightarrow |30(5) - 5.5(30)| = |150 - 165| = 15^\circ$.
 At 8:40 $\rightarrow |30(8) - 5.5(40)| = |240 - 220| = 20^\circ$.
 At 2:13 $\rightarrow |30(2) - 5.5(13)| = |60 - 71.5| = 11.5^\circ$.

C9.3 Hands Coincide / Opposite / Perpendicular [Level: Medium — the highest-yield facts in this chapter]

Hands coincide (angle = 0°): occurs every $720/11 = 65 + 5/11$ minutes
 Hands opposite (angle = 180°): occurs every $720/11$ minutes (offset by half a cycle)
 Hands perpendicular (angle = 90°): occurs twice as often as coincidence

- **TRAP:** In any 12-hour span, hands coincide **11 times, not 12** (naive students assume once per hour = 12). Reason: between 11:00 and 1:00, they coincide only ONCE — at 12:00 — not twice. Same undercounting applies to "opposite" (11 times per 12h, 22 times/day) and "perpendicular" (22 times per 12h, 44 times/day).
- TRICK: List of coincidence times starting from 12:00 — 12:00, 1:05(5/11), 2:10(10/11), 3:16(4/11)... each **65+5/11 min** after the previous.
- To find WHEN hands coincide between H and H+1 o'clock: solve $30H - 5.5M = 0 \rightarrow M = 60H/11$.

Between 6 and 7: $M = 360/11 = 32 + 8/11$ minutes, i.e. 6:32 $\frac{8}{11}$.

C9.4 Mirror Image & Water Image of Clock Time [Level: Medium]

Mirror image time (reflect left-right, vertical mirror) = $12:00 - \text{given time}$ (borrow 12:00 if needed)
 Water image time (reflect top-bottom, horizontal mirror) = $6:00 - \text{given time}$ (borrow 12:00 if needed)

- **TRAP:** These use **different** reference points (12:00 for mirror, 6:00 for water) — do not swap them.

Mirror image of 8:40 $\rightarrow 12:00 - 8:40 = 3:20$.
 Mirror image of 4:12 $\rightarrow 12:00 - 4:12 = 7:48$.
 Water image of 10:10 $\rightarrow 6:00 - 10:10 \rightarrow \text{borrow } 12\text{h} \rightarrow 18:00 - 10:10 = 7:50$.

C9.5 Distance Traveled by Hands [Level: Medium]

- Each hand traces a circle of circumference $2\pi r$ (r = hand length) every time it completes a revolution. Minute hand: 1 revolution/hour. Hour hand: 1 revolution/12 hours.

A clock's long hand (minute hand) = 8 cm, short hand (hour hand) = 7 cm. Runs for 4 days (=96 hours). Total distance covered by both hands?
 Minute hand: 96 revolutions $\rightarrow 96 \times 2\pi(8) = 1536\pi$ cm.
 Hour hand: $96/12 = 8$ revolutions $\rightarrow 8 \times 2\pi(7) = 112\pi$ cm.
Total = $1536\pi + 112\pi = 1648\pi$ cm.

C9.6 Gaining / Losing (Faulty) Clocks [Level: Hard]

- Core idea: a faulty clock's **shown/displayed time** and **real elapsed time** run at different rates. Always convert via a proportion: (faulty minutes shown) : (real minutes elapsed).
- **TRAP:** The common WRONG shortcut is to just multiply "total hours elapsed" by the per-hour gain/loss and add/subtract directly — this silently assumes the elapsed-hours count is measured in *real* time, but if the elapsed hours are counted *on the faulty clock's face*, that approach is invalid. Always set up the full proportion instead.

Worked Example (classic multi-part problem)

A clock is set right at Monday 10:00 AM. It **loses 6 minutes every hour** (i.e., in 60 real minutes, the faulty clock advances only 54 minutes). Find the actual time when the faulty clock shows **Friday 3:00 PM**.
 Step 1 — elapsed time *shown on the faulty clock*: Monday 10 AM $\rightarrow$ Friday 3 PM = 4 days + 5 hours = **101 hours** = 6060 faulty-minutes.

Step 2 — convert to real minutes using the ratio 54 (faulty) : 60 (real): Real minutes = $6060 \times (60/54) = \mathbf{6733.33 \text{ min} = 112 \text{ h } 13 \text{ min } 20 \text{ s}}$.

Step 3 — add to the start time: Monday 10:00 AM + 112h13m20s = (112h = 4 days 16h) → Friday 10 AM + 16h = **Saturday 2:13:20 AM**.

TRAP: A popular wrong shortcut computes $101 \times 6 = 606 \text{ minutes}$ and simply adds this to Friday 3 PM, giving Saturday 1:06 AM — this is **WRONG** because it treats the 101 hours as real elapsed time rather than faulty-clock-displayed time; always use the Step-2 proportion.

More Worked Examples

A clock loses 1% of time in week 1, then gains 2% in week 2 (1 week = 168 hours). Set right at Sunday 12 noon. What does it show 14 days later?

Week 1: loses 1% of 168h = 1.68h. Week 2: gains 2% of 168h = 3.36h. Net after 2 weeks = $-1.68 + 3.36 = \mathbf{+1.68 \text{ h ahead}}$.

Real time 14 days later = Sunday 12 noon again. Clock shows 1.68h = 1h 40min 48s ahead → **1:40:48 PM**.

A watch is 2 minutes slow at noon Monday, and 4 min 48 sec fast at 2 PM the following Monday (uniform gain throughout). When was it exactly correct?

Total swing = from -2 min to $+4 \text{ min } 48 \text{ s}$ = 6.8 minutes gained, over (7 days 2 hours =) 170 hours → rate = $6.8/170 = 0.04 \text{ min/hour}$.

Watch is correct once it has gained back its initial 2-minute deficit: time needed = $2/0.04 = \mathbf{50 \text{ hours}}$ after noon Monday = 2 days 2 hours → **2:00 PM Wednesday**.

Questions

(Several of these are real, previously-asked exam questions — verified by hand.)

Q1. [Easy] Find the angle between hour and minute hand at 7 o'clock.

A. 150° B. 180° C. 210° D. 120°

Answer: A. Why: $[30 \times 7 - 5.5 \times 0] = 210^\circ$. Since the clock-angle convention always reports the smaller of the two angles formed ($\leq 180^\circ$), the answer is $360 - 210 = \mathbf{150^\circ}$. (Counting method check: 7 o'clock is 5 hour-marks apart the short way round → $5 \times 30^\circ = 150^\circ$.)

Q2. [Easy] What is the angle between the hands at 5:30?

A. 45° B. 30° C. 15° D. 60°

Answer: C. Why: $[30(5) - 5.5(30)] = [150 - 165] = \mathbf{15^\circ}$.

Q3. [Medium] Find the acute angle between the hour and minute hand at 2:13 PM.

A. 16.5° B. 18° C. 13.5° D. 11.5°

Answer: D. Why: $[30(2) - 5.5(13)] = [60 - 71.5] = \mathbf{11.5^\circ}$.

Q4. [Medium] [Trick] At what time between 6 and 7 o'clock do the hands of a clock coincide?

A. 6:13 B. 6:36 C. 6:32 + 8/11 min D. 6:35 + 8/11 min

Answer: C. Why: Solve $30(6) - 5.5M = 0 \rightarrow M = 180/5.5 = 32 + 8/11 \text{ minutes}$.

Q5. [Medium] [Trap] How many times do the hands of a clock coincide in 5 hours?

A. 4 B. 3 C. 2 D. 6

Answer: A. Why: Coincidences occur every $65 + 5/11 \approx 65.45 \text{ minutes}$ = 300 minutes available/ $65.45 \approx 4.58 \rightarrow \mathbf{4 \text{ complete coincidences}}$ in 5 hours (300 minutes).

Q6. [Medium] [Trap] Between 9 AM and 9 PM of the same day, how many times are the minute and hour hands together?

A. 9 B. 10 C. 11 D. 12

Answer: C. Why: They coincide once per hour on average, but between 11 and 1 o'clock the coincidence at 12:00 is shared/counted once — over any 12-hour span the count is **11**, not 12.

Q7. [Medium] [Trap] How many times in a day are the hands of a clock in a straight line but pointing in opposite directions?

A. 20 B. 22 C. 24 D. 48

Answer: B. Why: Opposite-direction alignment happens 11 times per 12 hours (between 5 and 7 o'clock it happens only once, at 6:00) → 22 times in a full day.

Q8. [Medium] [Trick] Find the mirror image of the time 8:40.

A. 3:20 B. 2:30 C. 8:20 D. 9:20

Answer: A. Why: Mirror image = $12:00 - 8:40 = \mathbf{3:20}$.

Q9. [Medium] If a clock shows 4 hours 12 minutes, what time does its mirror image show?

A. 8h 48m B. 9h 48m C. 10h 48m D. 7h 48m

Answer: D. Why: $12:00 - 4:12 = \mathbf{7:48}$.

Q10. [Hard] A clock's long hand (minute hand) is 8 cm and short hand (hour hand) is 7 cm. If the clock runs for 4 days, find the total distance covered by both hands.

A. $1824\pi \text{ cm}$ B. $1724\pi \text{ cm}$ C. $1648\pi \text{ cm}$ D. $2028\pi \text{ cm}$

Answer: C. Why: 4 days = 96 h. Minute hand: $96 \text{ rev} \times 2\pi(8) = 1536\pi$. Hour hand: $96/12 = 8 \text{ rev} \times 2\pi(7) = 112\pi$. Total = **$1648\pi \text{ cm}$** .

Q11. [Hard] [Trap] A church clock is set right at Monday 10:00 AM. It loses 6 minutes every hour. What is the actual time when the faulty clock shows 3:00 PM on Friday?

A. Saturday 1:06 AM B. Friday 4:54 AM C. Saturday 2:13:20 AM D. Friday 3:00 AM

Answer: C. Why: Faulty-clock-elapsed = 101 h = 6060 faulty-min. Real minutes = $6060 \times (60/54) = 6733.33 \text{ min} = 112 \text{ h } 13 \text{ m } 20 \text{ s}$. Monday 10 AM + 112h13m20s = **Saturday 2:13:20 AM**. (Option A results from the common WRONG shortcut of directly multiplying 101×6 minutes and adding — see the TRAP note in C9.6.)

Q12. [Hard] A clock loses 1% of time during the first week, then gains 2% during the next week. If set right at 12 noon Sunday, what time will it show exactly 14 days later?

A. 1:40:48 B. 1:36:48 C. 1:41:24 D. 10:19:12

Answer: A. Why: Net gain over 2 weeks = $-1.68 \text{ h} + 3.36 \text{ h} = +1.68 \text{ h} = 1 \text{ h } 40 \text{ min } 48 \text{ s}$ ahead of the real 12 noon → **1:40:48 PM**.

Q13. [Hard] A watch is 2 minutes slow at noon on Monday and 4 min 48 sec fast at 2 PM the following Monday, gaining uniformly throughout. When was it exactly correct?

A. 2 PM Tuesday B. 2 PM Wednesday C. 3 PM Thursday D. 1 PM Friday

Answer: B. Why: Total gain of 6.8 minutes over 170 hours → rate 0.04 min/h . Time to gain back the initial 2-min deficit = $2/0.04 = 50 \text{ hours} = 2 \text{ days } 2 \text{ hours}$ after noon Monday = **2 PM Wednesday**.

Q14. [Hard] [Trick] How many "palindrome" times (digit sequence reads the same forwards and backwards, e.g. 5:45 → "545") occur on a 12-hour clock face from 12:00 noon to the next midnight?

A. 43 B. 51 C. 58 D. 57

Answer: D. Why: For single-digit hours 1-9: format H:M1M2 is a palindrome whenever $M2 = H$ ($M1$ free, 0-5) → 6 times each $\times 9 \text{ hours} = 54$. For two-digit hours 10, 11, 12: exactly one palindrome each (10:01, 11:11, 12:21) = 3 more. Total = $54 + 3 = \mathbf{57}$.

9.C10 Ranking, Order & Sequence; Logical Sequence of Words

C10.1 Ranking — Core Formula [Level: Basic]

$$\text{Total people} = (\text{Rank from the front}) + (\text{Rank from the back}) - 1$$

- TRICK: this single formula solves the vast majority of ranking questions — memorize it as the anchor and derive whichever unknown is asked (total, or one of the two ranks) by rearranging it.
- Worked example: Rohan is 7th from the top and 12th from the bottom in a class. Total students = $7+12-1 = 18$.

C10.2 Change of Position (Someone Moves in a Row/Rank) [Level: Medium]

- If a person's rank from the front changes from a to b (moves back in rank, i.e., more people now ahead), the number of people who "overtaken" them = $b - a$. If asked how many are between two ranks, subtract and adjust by 1 depending on whether endpoints are included.
- TRAP:** "number of people between rank x and rank y " = $|x - y| - 1$ (exclude both endpoints), NOT $|x-y|$ — a very commonly missed off-by-one error.

C10.3 Ranking in Both Directions Simultaneously [Level: Medium]

- When ranks from left AND right (or top AND bottom) are both given for the SAME row, use the total-count formula (C10.1) directly.
- When comparing two DIFFERENT people's positions in the same row, set up each person's rank-from-front and rank-from-back as separate equations sharing the same total, then solve simultaneously.

C10.4 Distance-Direction-Ranking Combined Puzzles [Level: Hard]

- Some questions combine a physical arrangement (people standing in a line facing a direction) with ranking — draw the row as a simple left-to-right diagram with position numbers, mark which way each person faces, then answer rank/distance/direction sub-questions directly off the diagram rather than trying to reason abstractly.

C10.5 Logical Sequence of Words [Level: Medium]

- Given a set of words, arrange them in a logically meaningful order — common patterns:
 - Size/hierarchy order** (e.g., Village → Town → City → Metropolis).
 - Process/chronological order** (e.g., seed → sapling → tree → fruit, or the sequential steps of an activity like Application → Interview → Offer → Joining).
 - Dictionary/alphabetical order** — used when the question explicitly wants alphabetical arrangement of otherwise unrelated words.
 - Age/time order** (e.g., Infant → Child → Adolescent → Adult → Old age).
- TRICK: identify the CATEGORY of logical relationship first (size, time, process, hierarchy) before attempting to order the words — trying to order without first spotting the underlying dimension leads to guessing.
- TRAP:** some words may fit more than one plausible order (e.g., could be ordered by size OR by a different real-world criterion) — always pick the ordering criterion that uses ALL the given words consistently, discarding an ordering that leaves an odd one out unless the question explicitly allows it.

Questions

Q1. [Easy] In a row of students, Meena is 9th from the left and 15th from the right. How many students are in the row?

A. 24 B. 23 C. 25 D. 22

Answer: B. Why: Total = $9+15-1 = 23$.

Q2. [Easy] If there are 30 people in a queue and a person is 12th from the front, what is their rank from the back?

A. 18 B. 19 C. 20 D. 17

Answer: B. Why: Total = front + back - 1 → $30 = 12 + \text{back} - 1 \rightarrow \text{back} = 19$.

Q3. [Medium] [Trap] In a line of 20 people, how many people are standing between the 5th person from the left and the 15th person from the left?

A. 10 B. 9 C. 11 D. 8

Answer: B. Why: Between (exclusive of both endpoints) = $|15-5| - 1 = 10 - 1 = 9$.

Q4. [Medium] Arjun is ranked 10th from the top in a class. If there are 35 students in total, what is his rank from the bottom?

A. 25 B. 26 C. 24 D. 27

Answer: B. Why: Total = front+back-1 → $35 = 10+\text{back}-1 \rightarrow \text{back} = 26$.

Q5. [Medium] Arrange in logical (chronological/process) order: (1) Seed (2) Fruit (3) Flower (4) Plant

A. 1-4-3-2 B. 1-3-4-2 C. 4-1-3-2 D. 1-2-3-4

Answer: A. Why: Natural growth sequence: Seed → Plant → Flower → Fruit.

Q6. [Hard] Arrange in a logical size order: (1) Village (2) Country (3) Town (4) City (5) District

A. 1-3-4-5-2 B. 1-3-5-4-2 C. 3-1-4-5-2 D. 1-4-3-5-2

Answer: B. Why: Increasing size/administrative hierarchy: Village → Town → District → City → Country. (District is administratively above a City in this common ordering convention, sitting between Town and Country.)

Q7. [Hard] In a row of children facing North, Rakesh is 7th from the left and 4th from the right. A new child joins the row at the right end. What is the new total number of children?

A. 10 B. 11 C. 9 D. 12

Answer: B. Why: Original total = $7+4-1 = 10$; adding one more child at the end makes the new total 11.

Q8. [Hard] [Trap] In a class of 40, Priya's rank from the top improves from 15th to 10th after a re-test. How many students did she overtake?

A. 5 B. 10 C. 15 D. 25

Answer: A. Why: Rank improved from 15 to 10 → she moved past exactly $(15-10)=5$ students who were previously ahead of her.

9.C11 Inequality & Data Sufficiency

C11.1 Decoding Inequality Symbols [Level: Basic]

- Standard symbols: $>$ (greater than), $<$ (less than), $\geq$ (at least/greater-or-equal), $\leq$ (at most/less-or-equal), $=$ (equal).
- Many reasoning questions replace these with arbitrary CODED symbols and give you a key, e.g.:

A @ B means A $>$ B
 A # B means A $<$ B
 A \$ B means A = B
 A & B means A $\geq$ B
 A * B means A $\leq$ B

- TRICK: Always decode the ENTIRE statement chain into plain $>$ $<$ $=$ $\geq$ $\leq$ symbols FIRST, before doing any reasoning. Trying to reason directly in coded symbols is the #1 cause of careless errors.

C11.2 Chaining Inequalities [Level: Medium]

- A chain like A $>$ B $>$ C = D $\geq$ E lets you relate any two elements by combining the links between them.
- TRAP:** Only an unbroken chain of **strict** relations ($>$, $<$, $=$) gives a **strict** conclusion between two endpoints. The moment a $\geq$ or $\leq$ appears anywhere in the path between two elements, the *derived* relation between those two endpoints is also only "at least" / "at most" — **not** strict — even if every other link in that path is strict.
- TRICK: To test whether a given conclusion "X (relation) Y" *definitely* follows, trace the unbroken path of links from X to Y in the decoded chain and combine them:

All links strict ($>$, $<$, $=$) along the path $\rightarrow$ conclusion is STRICT
 Any single link in the path is $\geq$ or $\leq$ $\rightarrow$ conclusion is only "at least" / "at most"
 No unbroken path connects X and Y at all $\rightarrow$ NO definite relation can be concluded

Worked Example

Decoded chain: A $>$ B $>$ C = D $\geq$ E.

Conclusion I: A $>$ E — Path A $\rightarrow$ B $\rightarrow$ C $\rightarrow$ D $\rightarrow$ E has links $>$, $>$, $=$, $\geq$. The last link is $\geq$, so the combined relation is only A $\geq$ E, not strict. **Conclusion I does NOT definitely follow.**

Conclusion II: A $>$ D — Path A $\rightarrow$ B $\rightarrow$ C $\rightarrow$ D has links $>$, $>$, $=$, all strict. So A $>$ D **definitely follows.**

A Second Common Trap Pattern: Two Elements Both Related to a Common Third

Decoded: A $>$ B, C $>$ B (i.e., both A and C are greater than the same B), and C $\leq$ E.

Conclusion: A $>$ C? — A and C are both compared only to B, never to each other directly, and no chain connects A to C. **No definite relation between A and C can be concluded** — this is the single most common inequality trap: two elements sharing a common "anchor" element are NOT thereby related to each other.

Conclusion: C $\leq$ E? — Given directly. **Definitely true.**

C11.3 Data Sufficiency — The Framework [Level: Basic]

- Data Sufficiency does **not** ask you to solve the question — only to judge whether the given statement(s) provide **enough** information to answer it with a single, unique result.
- Standard 5-option framework (memorize this cold, it's identical across every exam that uses this format):

Option	Meaning
(A)	Statement I alone is sufficient, but Statement II alone is not
(B)	Statement II alone is sufficient, but Statement I alone is not
(C)	BOTH together are sufficient, but neither alone is
(D)	EACH statement alone is sufficient (either one works on its own)
(E)	Even both statements together are NOT sufficient

C11.4 The Solving Method — Order Matters [Level: Medium]

Step 1: Read the question. Identify exactly what needs a UNIQUE value/answer.
 Step 2: Judge Statement I COMPLETELY ALONE. Pretend Statement II does not exist.
 Step 3: Judge Statement II COMPLETELY ALONE. Pretend Statement I does not exist (and forget any partial progress from Step 2).
 Step 4: ONLY IF both are insufficient alone, combine them and judge together.

- TRAP:** Never let information from Statement II "leak" into your judgment of Statement I in isolation (or vice versa) — this is the most common process error, and it silently produces the wrong option even when the underlying math is done correctly.
- TRICK: "Sufficient" means the statement pins down **exactly one** possible value/answer. If a statement is consistent with **two or more** different possible answers, it is **not** sufficient — even if it narrows the possibilities down a lot.

C11.5 The Classic Quadratic Trap [Level: Hard]

- TRAP:** A statement that looks like it "gives you x" via an equation may actually yield **two** valid roots (e.g., from a quadratic), making it **insufficient** even though it appears to pin down a value.

Example: "What is x?" Statement: $x^2 = 9$
 This does NOT give a unique x — it gives $x = 3$ OR $x = -3$.
 $\Rightarrow$ Statement alone is INSUFFICIENT, despite looking like a complete equation.

Questions

(ordered Easy $\rightarrow$ Hard; [Trick] tests a shortcut, [Trap] tests a classic misconception)

Q1. [Easy] Given: $A > B$, $B > C$. Which is definitely true?

A. $A > C$ B. $A = C$ C. $A < C$ D. Cannot be determined

Answer: A. Why: Both links are strict $>$, so they chain directly: $A > B > C$ implies $A > C$.

Q2. [Easy] Decode: "P & Q" means $P \geq Q$, and "P # Q" means $P < Q$. Given P & Q and Q # R, what is the relation between P and R?

A. $P > R$ B. $P \geq R$ C. P could be less than, equal to, or greater than R D. $P < R$

Answer: C. Why: $P \geq Q$ and $Q < R$. Chaining "at least" with "less than" doesn't fix P relative to R: e.g., $P=Q=5, R=6$ gives PR. No definite relation exists.

Q3. [Medium] [Trap] Given: $A > B > C = D \geq E$. Does " $A > E$ " definitely follow?

A. Yes, always B. No — only $A \geq E$ is guaranteed C. No relation can be determined D. $A < E$ is guaranteed instead

Answer: B. Why: The path $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$ has links $(>, >, =, \geq)$. Since the final link is non-strict $(\geq)$, the combined result is only $A \geq E$, not strict $A > E$.

Q4. [Medium] Given: $X > Y$, $Z > Y$. Which conclusion definitely follows?

A. $X > Z$ B. $Z > X$ C. $X = Z$ D. No definite relation between X and Z

Answer: D. Why: X and Z are each compared only to the common element Y, never to each other — a classic "shared anchor" trap. Both being greater than Y says nothing about their relative order.

Q5. [Medium] "What is the value of x?" Statement I: $x + 5 = 12$. Statement II: $x^2 = 49$.

A. I alone sufficient, II not B. II alone sufficient, I not C. Both together needed D. Neither alone nor together sufficient

Answer: A. Why: Statement I: $x = 7$, unique — sufficient. Statement II: $x = 7$ or $x = -7$, two possible values — NOT sufficient (the classic quadratic trap).

Q6. [Medium] "Is n divisible by 3?" Statement I: n is divisible by 9. Statement II: n is divisible by 4.

A. I alone sufficient, II not B. II alone sufficient, I not C. Both together needed D. Each alone is sufficient

Answer: A. Why: Divisible by 9 ($=3 \times 3$) guarantees divisible by 3 — definite YES, sufficient alone. Divisible by 4 says nothing about divisibility by 3 (e.g., 12 is divisible by 4 and by 3, but 16 is divisible by 4 and NOT by 3) — insufficient alone.

Q7. [Hard] "What is the value of y?" Statement I: $x + y = 10$. Statement II: $x - y = 2$.

A. I alone sufficient B. II alone sufficient C. Both together sufficient, neither alone D. Not sufficient even together

Answer: C. Why: Each equation alone has infinitely many (x,y) solutions. Together: adding the equations gives $2x = 12$, $x = 6$, then $y = 4$ — a unique value. Neither alone works, but together they do.

Q8. [Hard] [Trap] "What is the value of x?" Statement I: $x^2 - 5x + 6 = 0$. Statement II: x is a positive number.

A. I alone sufficient B. II alone sufficient C. Both together sufficient D. Not sufficient even together

Answer: D. Why: Statement I factors to $(x-2)(x-3)=0$, so $x = 2$ or $x = 3$ — not unique. Statement II only says x is positive; both 2 and 3 are positive, so combining still leaves two possible values. Not sufficient even together — the quadratic trap survives combination.

Q9. [Hard] "Is $x > y$?" Statement I: $x > 0$. Statement II: $y < 0$.

A. I alone sufficient B. II alone sufficient C. Both together sufficient, neither alone D. Each alone is sufficient

Answer: C. Why: Statement I alone doesn't fix y's sign (y could be positive and larger than x). Statement II alone doesn't fix x's sign similarly. Together: x is positive and y is negative, so $x > 0 > y$ is guaranteed — a definite YES. Sufficient only together.

Q10. [Hard] Decoded chain: $A > B = C > D \geq E > F$. Which conclusion is definitely true?

A. $A > F$ B. $B > D$ C. $D > F$ D. $A = F$

Answer: B. Why: Path $B \rightarrow C \rightarrow D$ has links $(=, >)$, both strict/equality — so $B > D$ definitely follows. (Check A: path $A \rightarrow \dots \rightarrow F$ crosses the non-strict $D \geq E$ link, so only $A \geq F$ is guaranteed, not strict $A > F$ — eliminates option A.)

Q11. [Hard] "What is the perimeter of a rectangle?" Statement I: its length is 10 cm. Statement II: its area is 80 sq cm.

A. I alone sufficient B. II alone sufficient C. Both together sufficient, neither alone D. Not sufficient even together

Answer: C. Why: Length alone doesn't give width. Area alone allows many length-width pairs (e.g., 10×8 , 20×4 , each with different perimeter) — insufficient alone. Together: length=10 and area=80 gives width=8 uniquely, so perimeter $= 2(10+8)=36$ is fixed. Sufficient only together.

9. Part D — Non-Verbal, Spatial & Analytical Reasoning

Figure-based reasoning: mirror/water images, paper folding, series/classification of shapes, counting figures, cubes & dice. These cannot be "read" quickly in a written test the way a diagram-heavy PDF shows them — in the actual exam they appear as printed figures in the question booklet. Use the chapters here to internalize the *rules* (mirror-image = left-right flip along a vertical axis; water-image = top-bottom flip + upside-down; cube net folding rules; standard cube-counting formulas) so that when you see the real printed figures, you're pattern-matching against a known rule, not solving from scratch.

9.D1 Non-Verbal Reasoning — Image Analysis, Mirror & Water Images

D1.1 Mirror Images [Level: Basic]

- A **mirror image** is the reflection of a figure/letter/word across a **VERTICAL** line (mirror held upright, to the left or right of the object).
- Effect: **LEFT-RIGHT reversal** (horizontal flip). Top and bottom stay the same; what was on the left moves to the right and vice versa.
- For a **word**, both things happen together: (a) the **ORDER** of letters reverses (last letter becomes first), **AND** (b) each individual letter is itself mirrored.

Uppercase letters that look IDENTICAL after a mirror (vertical-flip) reflection:

A H I M O T U V W X Y

(all of these have a vertical line of symmetry)

- **TRICK:** Memorize this exact 11-letter list ("AHIMOTUVWXY" — spell it as a nonsense word to recall it fast). Any letter NOT in this list changes appearance when mirrored (e.g., B, C, D, E, F, G, J, K, L, N, P, Q, R, S, Z all look different or flip into a different/no valid letter).

D1.2 Water Images [Level: Basic]

- A **water image** is the reflection of a figure across a **HORIZONTAL** line (as if the object sits above a still water surface and you see its reflection below).
- Effect: **TOP-BOTTOM reversal** (vertical flip). Left and right stay the same; what was on top moves to the bottom.

Uppercase letters that look IDENTICAL after a water (horizontal-flip) reflection:

B C D E H I K O X

(all of these have a horizontal line of symmetry)

- **TRAP:** The mirror-invariant list (AHIMOTUVWXY) and the water-invariant list (BCDEHIKOX) are **DIFFERENT** — only H, I, O, X are common to both (they have BOTH a vertical and horizontal symmetry line). Do not assume a letter unchanged by a mirror is also unchanged by a water reflection, or vice versa — check each list separately.
- For a word's water image: the letters do NOT reorder (reading direction left-to-right is preserved), but each letter individually flips top-to-bottom, and the whole word appears "below" as if stacked upside-down vertically (the word is typically also shown upside-down/rotated 180° in many textbook conventions — always check what the specific question's convention states, but the core rule tested is the per-letter top-bottom flip).

D1.3 Mirror/Water Images of Digits [Level: Basic-Medium]

Digit mirror-image behavior (vertical flip, standard digital/seven-segment style font):

0, 1, 8 → look the same (vertical line of symmetry)

2, 3, 4, 5, 6, 7, 9 → look different / distorted

Note: exact appearance is FONT-DEPENDENT for digits — always go with what the specific question's font/convention shows rather than assuming a universal rule.

- **TRICK:** For digit-clock and digit-string mirror questions, 0, 1, 8 are the safe "always symmetric" anchors — use them to quickly check if a proposed mirror answer is even plausible.

D1.4 Mirror & Water Image of Clock Time [Level: Medium]

A very common combination question (bridges this chapter with the Clocks chapter):

Mirror image of a clock time = subtract the given time from 11:60 (i.e., 12:00)

Water image of a clock time = subtract the given time from 5:60 (i.e., 6:00)

[applies when the clock's 12-6 axis is vertical / 9-3 axis is horizontal, the standard clock face]

- Worked example: Time shown = 4:20.
- Mirror image = 11:60 – 4:20 = **7:40**.
- Water image = 5:60 – 4:20 = **1:40**.
- **TRAP:** Students often subtract from 12:00 for the water image too — remember mirror uses 12:00, water uses 6:00, because the mirror reflects across the vertical 12-6 axis (swapping left/right of the dial) while the water image reflects across the horizontal 9-3 axis (swapping top/bottom).

D1.5 Rotation of Figures [Level: Medium-Hard]

- Rotating a figure by 90°/180°/270° (clockwise or anticlockwise) about its center.
- Systematic method: pick 2-3 clearly identifiable reference points/corners of the figure (e.g., "the notch," "the longest edge"), mentally track where EACH one moves under the stated rotation, then redraw/compare.
- Shortcut equivalences: a 180° rotation (either direction) = flipping BOTH left-right AND top-bottom simultaneously (i.e., mirror + water image combined). A 90° clockwise rotation is the same end-state as a 270° (=90°×3) anticlockwise rotation — use whichever is easier to visualize.
- **TRAP:** Rotation is NOT the same as reflection — a rotated letter/figure and its mirror image usually look different (a rotation preserves "handedness"/chirality, a reflection reverses it). Do not substitute one for the other.

Questions

(ordered Easy → Hard; [Trick] tags the shortcut-list application, [Trap] tags mirror/water confusion)

Q1. [Easy] Which of these letters looks the SAME as its own mirror image (vertical flip)?

A. B B. E C. H D. S

Answer: C. **Why:** H is in the mirror-invariant list (AHIMOTUVWXY); B, E, S are not.

Q2. [Easy] Which of these letters looks the SAME as its own water image (horizontal flip)?

A. F B. D C. N D. Z

Answer: B. **Why:** D is in the water-invariant list (BCDEHIKOX); F, N, Z are not.

Q3. [Medium] [Trap] Which letter looks unchanged under BOTH a mirror image AND a water image?

A. A B. O C. T D. C

Answer: B. Why: O appears in both lists (AHIMOTUVVXY for mirror, BCDEHIKOX for water) — only H, I, O, X have this dual property; A, T are mirror-only, C is water-only.

Q4. [Medium] [Trick] What is the mirror image (letter order and letter shapes) of the word "TOOT"?

A. TOOT B. TUOT C. TOOT reversed as "TOOT" D. Cannot be determined

Answer: A. Why: Reverse the letter order: T-O-O-T reversed is still T-O-O-T (it's a palindrome), and every individual letter (T, O) is in the mirror-invariant list, so each letter also looks unchanged. Net result: identical to the original — "TOOT".

Q5. [Medium] A clock shows 3:50. What is its mirror image time?

A. 8:10 B. 7:10 C. 8:70 D. 9:10

Answer: A. Why: Mirror image = $11:60 - 3:50 = 8:10$.

Q6. [Medium] A clock shows 2:15. What is its water image time?

A. 4:45 B. 3:45 C. 3:15 D. 4:15

Answer: B. Why: Water image = $5:60 - 2:15 = 3:45$.

Q7. [Hard] [Trap] A clock shows 9:20. A student computes both its mirror and water image but mixes up the two base times. What are the CORRECT mirror and water images respectively?

A. Mirror = 2:40, Water = 8:40 B. Mirror = 2:40, Water = 3:40 C. Mirror = 3:40, Water = 2:40 D. Mirror = 8:40, Water = 2:40

Answer: A. Why: Mirror = $11:60 - 9:20 = 2:40$. Water = $5:60 - 9:20 \rightarrow$ since $9:20 > 5:60$, add 12 hours worth of minutes conceptually: treat as $(17:60 - 9:20) = 8:40$ (i.e., wrap using a 12-hour water-axis calculation: $5:60 + 12:00 - 9:20 = 8:40$). The key trap is using 12:00 for both instead of 12:00 (mirror) vs 6:00 (water).

Q8. [Hard] The word "FOX" is written and its water image is taken. Which letters (if any) change appearance, and does the letter order reverse?

A. Order reverses; F, O, X all change B. Order preserved; O and X unchanged, F changes C. Order preserved; all three unchanged D. Order reverses; only O unchanged

Answer: B. Why: Water image preserves letter ORDER (unlike a mirror image, which reverses order). Checking each letter against the water-invariant list (BCDEHIKOX): O and X are in the list (unchanged), F is NOT in the list (changes appearance).

Q9. [Hard] A figure is rotated 180° . Which single combined operation from this chapter produces the identical end result?

A. Mirror image only B. Water image only C. Mirror image followed by water image (or vice versa) D. No combination of mirror/water reproduces a 180° rotation

Answer: C. Why: A 180° rotation flips both left-right AND top-bottom simultaneously — exactly the combined effect of applying a mirror flip and a water flip together (in either order).

Q10. [Hard] [Trick] Without drawing, determine: does the letter "N" look the same after (a) a mirror image, (b) a water image, (c) a 180° rotation?

A. Same in all three B. Same only in (c) C. Different in all three D. Same in (a) and (b), different in (c)

Answer: B. Why: N is in neither the mirror list (AHIMOTUVVXY) nor the water list (BCDEHIKOX), so (a) and (b) both change its appearance — but N has 180° -rotational symmetry (point symmetry), so rotating it 180° reproduces the same N. This is a good example of a letter that is NOT mirror- or water-invariant but IS rotation-invariant, showing the three properties are independent.

9.D2 Non-Verbal Reasoning — Grouping, Shape Construction, Paper Folding & Cutting

D2.1 Grouping of Images / Classification by Shape Property [Level: Basic]

- Given a set of figures, find which ones share a common structural rule (and/or spot the one that breaks it).
- General method:** systematically check each figure against a fixed checklist of attributes — number of sides, number of enclosed/closed regions, number of line segments used, symmetry (does it have a line of symmetry?), open vs closed figure, curved vs straight edges — and identify which attribute is CONSTANT across most figures and which one differs for the odd one(s) out.
- TRICK: Work the checklist in the same order every time (sides → regions → symmetry → curve/straight) rather than randomly — this prevents missing the one attribute that actually varies.

D2.2 Shape Construction (Figure Assembly) [Level: Basic-Medium]

- Given several small piece-shapes, determine which combination assembles into a stated target shape (or which target shape CANNOT be formed from the given pieces).
- Method:** treat it as a jigsaw-matching problem — check that (a) edge LENGTHS of adjoining pieces match, (b) ANGLES at each join are compatible with the target's corners, and (c) the TOTAL AREA of the pieces equals the target's area (a quick elimination check: if total piece area $\neq$ target area, that combination is immediately wrong, without needing to check the geometry at all).
- TRICK: The area-matching check is the fastest elimination filter — compute total area of candidate pieces first before trying to mentally "assemble" them.

D2.3 Paper Folding [Level: Medium]

- A sheet of paper is folded one or more times (each fold is described, e.g., "folded in half vertically," "folded diagonally," "folded again in half"), then a hole is punched through all the folded layers at a stated location, and the paper is unfolded — find the resulting hole pattern.
- Core rule:** the punched hole is reproduced, as a MIRROR reflection, across every fold crease that was made. Each layer that was folded over the punch point gets its own reflected copy of the hole when unfolded.
- Systematic method (work backwards):**
 - Note the LAST fold made (the one closest to the punch).
 - Reflect the punch point across that fold's crease line to get a second hole.
 - Move to the SECOND-to-last fold; reflect BOTH holes found so far across that crease.
 - Repeat for every fold, working backwards to the very first fold, doubling the hole-count at each step (n folds → up to 2^n holes, fewer if some reflected positions coincide).
- TRAP:** A very common error is reflecting the punch point across only the LAST fold and stopping — you must reflect across **every** fold line, working backwards through the entire fold sequence, or you will under-count the holes (typically getting 2 holes instead of the correct 4, 8, etc.).

D2.4 Paper Cutting [Level: Medium]

- Identical logic to paper folding, but a SHAPE (not just a point) is cut from the folded edge. The cut shape reappears as a mirror-repeated pattern across each fold line, using the same backwards-reflection method as D2.3.
- Worked example: A square sheet is folded in half vertically (crease down the middle), then folded in half again horizontally (crease across the middle), giving a smaller square with 2 fold lines. A small triangular notch is cut from the corner that is diagonally opposite both folded edges (the only "free" corner). When unfolded:
- Reflecting the notch across the horizontal crease (last fold) gives 2 notches (one above, one below that crease).
- Reflecting both of those across the vertical crease (first fold) gives 4 notches total — one in each quadrant of the original square, all at mirror-symmetric positions relative to both crease lines.
- TRICK: With n folds each roughly halving the paper, the final unfolded sheet generally shows 2^n copies of the cut/hole (unless the punch/cut sits exactly ON a fold line, in which case some copies coincide and you get fewer than 2^n distinct marks).

D2.5 Dot Situation Problems [Level: Medium-Hard]

- A figure is described as several overlapping regions (e.g., a circle, a triangle, and a square, overlapping in various described ways). A dot is placed satisfying stated conditions (e.g., "inside the circle and the triangle, but outside the square"). Task: identify which of several alternative figure-descriptions is consistent with that same dot condition.
- Method:** this is a SET-MEMBERSHIP matching problem —
 - From the original figure's description, determine every overlapping region (which shapes overlap with which, and the region where all three overlap, if any).
 - Note precisely which regions the dot occupies (a set of "inside"/"outside" flags, one per shape).
 - For each alternative figure, check whether a region with that EXACT same set of "inside"/"outside" flags actually exists in that figure's overlap structure — only an alternative containing a genuinely matching region is correct.
- TRAP:** A region satisfying "inside A and B" is not the same as "inside A, B, and C" — always match the dot's condition on ALL stated shapes simultaneously, not just the ones explicitly mentioned as "inside" (an unmentioned shape may still need to be checked as "outside").

Questions

(ordered Easy → Hard; [Trick] tags a shortcut, [Trap] tags a common error)

Q1. [Easy] Five figures are shown: a pentagon, a hexagon, a heptagon, an octagon, and a triangle with one side replaced by a curve. Which is the odd one out?

A. Pentagon B. Hexagon C. The triangle-with-a-curved-side D. Octagon

Answer: C. **Why:** The other four are all closed figures made ENTIRELY of straight sides (polygons); the fifth includes a curved edge, breaking the "all straight sides" pattern shared by the rest.

Q2. [Easy] [Trick] Four piece-shapes have areas 4, 6, 9, and 5 square units. A target shape has area 20 square units. Can all four pieces combine to exactly form the target (assuming perfect edge-matching were otherwise possible)?

A. Yes, since $4+6+9+5=24$ which is more than enough B. No, since $4+6+9+5=24 \neq 20$ C. Cannot be determined D. Yes, since some pieces can overlap

Answer: B. **Why:** Total piece area = $4+6+9+5 = 24$, which does NOT equal the target's 20 — the fast area-matching elimination check immediately rules this out, no need to check edge-matching at all.

Q3. [Medium] A square sheet is folded in half once (one vertical crease) and a hole is punched away from the crease. How many holes appear when unfolded?

A. 1 B. 2 C. 4 D. Depends on hole size

Answer: B. Why: One fold $\rightarrow$ reflect the punch across that single crease $\rightarrow 2^n = 2^1 = 2$ holes.

Q4. [Medium] [Trap] A sheet is folded twice (two creases), and a student reflects the punch hole across only the second (last) fold, getting 2 holes, and stops. What is the error, and how many holes should actually appear?

A. No error; 2 is correct B. Must also reflect across the first fold; correct answer is 4 holes C. Must reflect 3 times; correct answer is 8 holes D. Folding twice always gives exactly 3 holes

Answer: B. Why: Every fold line must be reflected across, working backwards through the whole sequence — 2 folds $\rightarrow 2^2 = 4$ holes, not 2.

Q5. [Medium] A rectangular sheet is folded in half (vertical crease), then in half again (horizontal crease), then a hole is punched at the exact center of the resulting small rectangle — a point that lies ON both crease lines when unfolded conceptually would coincide. How many DISTINCT holes appear when fully unfolded?

A. 4 B. 2 C. 1 D. 8

Answer: C. Why: A punch exactly at the intersection of both fold lines reflects onto itself at every step (its reflected position across each crease is itself, since it lies exactly on both creases) — so all "copies" coincide at a single point, giving only 1 distinct hole, despite 2 folds normally suggesting $2^2=4$.

Q6. [Hard] A figure consists of a circle and a square overlapping partially, plus a triangle overlapping only the square (not the circle). A dot is placed inside both the circle and the square, but outside the triangle. Which region does the dot occupy?

A. Circle-only region B. Square-only region C. The circle-square overlap region, outside the triangle D. The triangle-square overlap region

Answer: C. Why: The dot must be inside BOTH circle and square simultaneously (their overlap region) while being outside the triangle — this uniquely identifies the circle-square (but not triangle) region.

Q7. [Hard] [Trap] Using the same figure as Q6, an alternative answer figure shows a region satisfying only "inside circle and square" without checking the triangle. Why is this alternative NOT necessarily a correct match?

A. It is always correct since circle and square are the shapes explicitly mentioned B. The triangle condition ("outside triangle") must also be verified, since the triangle overlaps the square — part of the circle-square region could also fall inside the triangle C. Triangles never affect circle-square regions D. Dot situation problems never involve three shapes

Answer: B. Why: Since the triangle overlaps the square (per the setup), the circle-square region may be split into a part that's also inside the triangle and a part that isn't — an alternative must match the FULL condition (inside circle, inside square, AND outside triangle), not just the two explicitly-named shapes.

Q8. [Hard] A square sheet is folded into quarters (2 perpendicular creases, giving 4 layers), and a triangular notch is cut from the single free corner of the folded quarter-square. How many notches appear on the fully unfolded sheet, and what is their arrangement?

A. 2 notches, diagonally opposite B. 4 notches, one per quadrant, each mirror-symmetric about both crease lines C. 8 notches D. 1 notch, since cutting is different from punching

Answer: B. Why: Two fold lines $\rightarrow 2^2 = 4$ reflected copies of the cut, one landing in each quadrant of the original square, each positioned as the mirror image of the original notch across both crease lines (same mechanism as paper folding, D2.3/D2.4 worked example).

9.D3 Non-Verbal Reasoning — Series, Classification, Embedded Figures, Figure Matrix & Pattern Completion

D3.1 The Universal Attribute Checklist [Level: Basic — apply this to every sub-topic below]

Every non-verbal reasoning problem in this chapter is really the same underlying skill: spot what changes vs what stays constant across a set of figures. Always scan attributes in this fixed order so nothing is missed:

1. Number of sides / number of elements (dots, lines, shapes)
2. Size (growing / shrinking)
3. Shading or color
4. Rotation / orientation
5. Position (of a sub-element within the figure)
6. Shape/type change (e.g., circle becomes square)

- TRICK: Go down this checklist top-to-bottom every single time, even if the answer seems obvious after step 1 — many "hard" questions have TWO simultaneously-changing attributes, and stopping early after spotting the first one is the most common error in this entire chapter.

D3.2 Non-Verbal Series / Pattern Recognition [Level: Basic-Medium]

- A sequence of figures changes step-by-step by one (or more) consistent rule(s): rotation by a fixed angle each step, a shape gaining/losing an element, a shading pattern cycling, or a size trend.
- **Method:** compare each CONSECUTIVE pair of figures (not just first-vs-last) to isolate the exact rule, confirm it holds for every consecutive pair in the given sequence, then apply it one more time to predict the next figure.
- TRICK: If the "obvious" rule from figures 1→2 doesn't also hold for 2→3, look for a rule that alternates every other step (a common way to make a series look harder than it is) — treat odd-numbered and even-numbered figures as two interleaved sub-series if a single simple rule doesn't fit all steps.

D3.3 Classification (Odd-One-Out) [Level: Basic-Medium]

- Same attribute-checklist method as D3.1/D2.1, but applied to find the ONE figure that breaks a rule shared by all the others.
- **TRAP:** The "odd one out" is sometimes odd in a SUBTLE way (e.g., 4 figures have an even number of sides and 1 has an odd number, even though all 5 otherwise look visually similar) — don't classify purely on "looks similar/different at a glance"; always run the full checklist.

D3.4 Embedded / Hidden Figures [Level: Medium]

- A simple target figure is hidden as a sub-part within one or more complex figures; find which complex figure(s) contain it.
- **Method:** count the target figure's exact vertex count and line-segment count first, then scan each candidate complex figure for a subset of lines matching that same vertex/segment count AND the same relative proportions/angles (not just "a similar-looking shape somewhere in there") — a genuine match must preserve the target's exact structure, not just a rough resemblance.
- TRICK: Trace the target shape's outline with a finger (or mentally, one continuous path) and try to find that exact continuous path within the complex figure — random shapes that only superficially resemble the target will fail this continuous-path test.

D3.5 Figure Matrix (Figure Analogy in Grid Form) [Level: Medium-Hard]

- A 2×2 or 3×3 grid of figures follows a rule ACROSS EACH ROW and (often independently) DOWN EACH COLUMN simultaneously. One cell (usually bottom-right) is missing; find it.
- **Method:** first determine the row-wise rule using the two complete rows, confirm the same TYPE of rule applies down the columns (rotation/addition/shading, etc. — the row-rule and column-rule need not be identical in kind, but each must be internally consistent), then apply BOTH the row-rule (using the two known cells in the target's row) and the column-rule (using the two known cells in the target's column) — a correct answer must satisfy both simultaneously.
- **TRAP:** Checking only the row (or only the column) and picking the first option that fits is a common shortcut that produces wrong answers when the two rules interact — always verify the candidate answer against BOTH the row and the column before finalizing.

D3.6 Rule Detection [Level: Medium]

- The general skill of identifying which single logical rule governs a set of figures — this is really D3.1's checklist applied as its own named category, often tested by asking "which rule best describes this set" as a standalone question rather than asking for a missing/odd figure directly.

D3.7 Pattern Completion / Figure Completion [Level: Medium-Hard]

- A single figure has a symmetric or repeating internal pattern, with one piece missing; find the piece that correctly completes it.
- **Method:** identify the figure's symmetry axis (or its repeat unit, if the pattern tiles/repeats rather than mirrors), then determine exactly what the missing piece must contain so the full figure satisfies that same symmetry/repetition when reassembled.
- TRICK: If the figure has rotational symmetry (not just mirror symmetry), the missing piece is found by rotating an existing piece by the pattern's rotation angle — don't default to assuming mirror symmetry when the pattern is actually rotational.

D3.8 Visual Reasoning — General Transformation Types [Level: Hard]

Spatial-apptitude questions across this entire chapter draw on a fixed set of named transformation types — recognizing WHICH transformation(s) a question is testing narrows down the method to use immediately:

Translation (shift position, shape/size/orientation unchanged)
 Scaling (grow/shrink, shape unchanged)
 Rotation (turn about a center point)
 Mirroring (reflect across an axis)
 Assembling (combine pieces into a whole – D2.2)
 Grouping (classify by shared property – D2.1/D3.3)
 Paper Cutting / Paper Folding (D2.3/D2.4)
 Pattern in 2D/3D (repeating/symmetric structure – D3.7)

- TRICK: Before attempting to solve, first name which ONE (or two) of these 8 transformation types the question is actually testing — this instantly tells you which method from this chapter to apply, rather than re-deriving an approach from scratch each time.

Questions

(ordered Easy → Hard; [Trick] tags the checklist/method shortcut, [Trap] tags a common error)

Q1. [Easy] A series of figures: a triangle, then a triangle with 1 dot inside, then a triangle with 2 dots inside, then a triangle with 3 dots inside. What comes next?

A. A square with 4 dots B. A triangle with 4 dots inside C. A triangle with 3 dots (repeat) D. A triangle with 0 dots

Answer: B. Why: The shape (triangle) stays constant; only the dot-count increases by 1 each step (0,1,2,3,...) — next is 4 dots, same triangle.

Q2. [Easy] Five shapes: square, rectangle, rhombus, trapezium, circle. Which is the odd one out?

A. Square B. Rectangle C. Circle D. Rhombus

Answer: C. Why: The first four are all quadrilaterals (4 straight sides); the circle has no straight sides or vertices at all.

Q3. [Medium] A figure rotates 90° clockwise at each step across 4 given figures, and also gains one additional shaded corner each step. A student identifies only the rotation and picks an answer that rotates correctly but has the wrong number of shaded corners. What error did they make?

A. No error, rotation is the only rule that matters B. They stopped at step 1 of the attribute checklist and missed the simultaneously-changing shading attribute C. Rotation and shading cannot both change in a valid series D. The answer should have MORE shaded corners than predicted

Answer: B. Why: This is exactly the D3.1 trap — two attributes (rotation AND shading count) change simultaneously; verifying only the first one found is insufficient.

Q4. [Medium] [Trick] In a number/letter analogy-style embedded-figure question, a target shape has 5 vertices and 5 line segments (a pentagon outline). A candidate complex figure contains a 5-sided region, but tracing it shows its 5 sides are not all connected in one continuous path (one "side" is actually two separate disconnected segments). Does this candidate contain the embedded target?

A. Yes, since it has 5 sides B. No — the continuous-path test fails, so it is not a genuine embedding of the pentagon outline C. Cannot be determined without more information D. Yes, since vertex count alone is sufficient

Answer: B. Why: A genuine embedded figure must trace as ONE continuous path matching the target's structure exactly — a region that merely has the right side-count without a continuous connected outline is not a true match (D3.4 method).

Q5. [Medium] A 2×2 figure matrix: top-left = 1 circle, top-right = 2 circles, bottom-left = 1 square, bottom-right = ?. The row rule is "add one more of the same shape going left to right." What is the missing cell?

A. 1 square B. 2 circles C. 2 squares D. 1 circle

Answer: C. Why: Applying the row rule (established from the top row: left→right adds one more of the same shape) to the bottom row (1 square → 2 squares) gives 2 squares.

Q6. [Hard] [Trap] In a 3×3 figure matrix, a student finds an option that correctly continues the row pattern and immediately selects it without checking the column. The grid's actual rule requires BOTH row and column consistency. What is the risk?

A. None — row-consistency alone always guarantees correctness B. The chosen option might satisfy the row rule but violate the independent column rule, making it wrong despite "looking right" for the row C. Columns are never tested in figure matrices D. Only the middle column ever matters

Answer: B. Why: This is the D3.5 trap explicitly — a matrix answer must be verified against BOTH the row rule and the column rule; satisfying only one is not sufficient.

Q7. [Hard] A pattern-completion figure is a circle divided into 6 equal sectors, alternately shaded and unshaded going around (shaded, unshaded, shaded, unshaded, shaded, unshaded), with one sector's shading state hidden/missing. The missing sector is the 4th one going clockwise from a fixed start. What should it be?

A. Shaded B. Unshaded C. Half-shaded D. Cannot be determined

Answer: B. Why: The pattern alternates strictly: sectors 1,3,5 = shaded and 2,4,6 = unshaded (or vice versa depending on the start) — sector 4 falls in the "unshaded" parity group, so it must be unshaded to preserve the alternating (rotationally-patterned) rule.

Q8. [Hard] A question describes a shape moving to a new position on a grid without changing its size, orientation, or shading. Which single transformation type (from the D3.8 list) does this test?

A. Rotation B. Scaling C. Translation D. Mirroring

Answer: C. Why: A pure position-shift with no change to size/orientation/shading is, by definition, a Translation — naming the transformation type immediately tells you no angle or size calculation is needed, only position-tracking.

Q9. [Hard] [Trick] A figure series shows a shape's size shrinking by half at each step while ALSO rotating 45° clockwise each step. Naming the D3.8 transformation types involved, which TWO are simultaneously at play, and what is the correct approach?

A. Only Rotation; size is irrelevant B. Only Scaling; rotation is irrelevant C. Both Scaling and Rotation are simultaneously active — track each independently (halve the size AND add 45° rotation) before comparing to the answer options D. This combination is impossible in a valid series

Answer: C. Why: Naming both active transformation types (per D3.8's method) makes clear that BOTH the size-halving rule and the 45°-rotation rule must be applied together to the last given figure to predict the next one — treating them independently avoids missing either change (same principle as the D3.1/Q3 checklist trap).

9.D4 Analytical Reasoning — Counting Figures (Squares, Rectangles, Triangles, Lines)

(No images are reproducible from the source for this topic — every method below is taught as a fully systematic, describable procedure so it can be applied to any figure described in words.)

D4.1 Counting Squares in a Grid [Level: Basic-Medium]

- A grid of size $n \times n$ (unit cells) contains squares of every size from 1×1 up to $n \times n$.

Number of $(n-k+1) \times (n-k+1) \dots$ simpler form:

Count of 1×1 squares = n^2

Count of 2×2 squares = $(n-1)^2$

Count of 3×3 squares = $(n-2)^2$

...

Count of $n \times n$ squares = 1^2

Total squares = $n^2 + (n-1)^2 + (n-2)^2 + \dots + 1^2 = \frac{n(n+1)(2n+1)}{6}$

- TRICK: You don't need to derive the sum-of-squares formula each time — just recall $\frac{n(n+1)(2n+1)}{6}$.

Worked Example

A 4×4 grid of unit squares. Total squares?

Sizes: $1 \times 1 \rightarrow 16$, $2 \times 2 \rightarrow 9$, $3 \times 3 \rightarrow 4$, $4 \times 4 \rightarrow 1$. Sum = $16+9+4+1 = 30$.

Check via formula: $\frac{4(5)(9)}{6} = 180/6 = 30$. ✓

- TRAP:** This formula only applies to a **square** grid ($n \times n$). For a rectangular grid ($m \times n$, $m \neq n$), you must count square-sizes up to $\min(m, n)$ individually: size $k \times k$ count = $(m-k+1)(n-k+1)$, summed for $k=1$ to $\min(m, n)$.

D4.2 Counting Rectangles in a Grid [Level: Medium]

- Core insight: any rectangle in a grid is uniquely determined by choosing **2 horizontal lines** and **2 vertical lines** from the grid's line network.

For an $m \times n$ grid of unit cells:

Horizontal lines available = $m + 1$

Vertical lines available = $n + 1$

Total rectangles = $C(m+1, 2) \times C(n+1, 2)$

- TRICK: This "choose 2 lines from each direction" insight also re-derives the square-counting formula as a special case (restricting to equal horizontal/vertical spacing), but it's easier to just use the two formulas separately.

Worked Example

A grid with 3 rows and 4 columns of unit cells ($m=3$, $n=4$). Total rectangles (including squares)?

Horizontal lines = 4, Vertical lines = 5. Total = $C(4, 2) \times C(5, 2) = 6 \times 10 = 60$.

- TRAP:** "Rectangles" **includes** squares (a square is a special rectangle) — if a question asks for "rectangles that are NOT squares," subtract the square-count (from D4.1) from the total rectangle-count.

D4.3 Counting Triangles [Level: Medium-Hard]

- No single universal formula exists (unlike squares/rectangles) — instead use this **systematic strategy**:

- Count the smallest ("basic") triangles first — the ones not made up of any smaller triangle.
- Count triangles formed by combining exactly 2 adjacent basic regions.
- Continue with combinations of 3, 4, ... basic regions, until no more distinct triangles can be formed.
- Sum all counts across every size category.

- TRICK for the common "triangle divided by cevians from vertices" figure (e.g., a triangle with lines drawn from each vertex to a point on the opposite side, or a triangle with its 3 medians/altitudes drawn): systematically list, for each pair/triple of drawn lines, whether their intersection region forms a valid triangle — this is tedious but mechanical if done size-by-size as above rather than trying to "see" all triangles at once.
- TRAP:** The most common error is **double-counting** a triangle that looks different when traced in two directions, or **missing** a large triangle formed by outer boundary lines plus one internal line — always do a final pass checking the largest possible triangle (using only the figure's outer boundary) separately.

D4.4 Counting Straight Lines / Intersecting Lines [Level: Basic-Medium]

If n straight lines are drawn such that no 2 are parallel and no 3 meet at a single point:

Number of intersection points = $C(n, 2) = \frac{n(n-1)}{2}$

- Adjustment for parallel lines: each pair of PARALLEL lines contributes **zero** intersection points (they never meet) — subtract $C(k, 2)$ from the total for each group of k mutually parallel lines.
- Adjustment for concurrent lines: if 3 or more lines all pass through one common point, that point is counted only ONCE, not once per pair — subtract the "extra" duplicate counts: if k lines meet at one point, they would naively contribute $C(k, 2)$ intersection points but actually contribute only 1, so subtract $[C(k, 2) - 1]$.

Worked Example

6 straight lines are drawn, no two parallel, no three concurrent. How many intersection points?

$C(6, 2) = \frac{6 \times 5}{2} = 15$.

Same 6 lines, but now 3 of them all pass through a common point (and no other coincidences). How many distinct intersection points?

Naive total (all $C(6,2)=15$) – extra duplicates from the concurrent triple: those 3 lines would contribute $C(3,2)=3$ pairs but only 1 actual point, so subtract $(3-1)=2$.

Distinct points = $15 - 2 = 13$.

Questions

Q1. [Easy] How many total squares are there in a 3×3 grid of unit squares?

A. 9 B. 12 C. 14 D. 16

Answer: C. **Why:** 1×1 : 9, 2×2 : 4, 3×3 : 1. Total = $9+4+1 = 14$. (Check: $3(4)(7)/6=14$.)

Q2. [Easy] How many total squares (of all sizes) are in a 5×5 grid?

A. 40 B. 55 C. 30 D. 25

Answer: B. **Why:** Formula $n(n+1)(2n+1)/6 = 5(6)(11)/6 = 55$.

Q3. [Medium] A grid has 2 rows and 3 columns of unit cells. How many total rectangles (including squares) are there?

A. 12 B. 15 C. 18 D. 24

Answer: C. **Why:** Horizontal lines = 3, vertical lines = 4. Total = $C(3,2) \times C(4,2) = 3 \times 6 = 18$.

Q4. [Medium] [Trap] Using the grid from Q3 (2×3 unit cells), how many rectangles are NOT squares?

A. 8 B. 10 C. 14 D. 18

Answer: B. **Why:** Squares in a 2×3 grid: $1 \times 1 \rightarrow 6$, $2 \times 2 \rightarrow (2-1)(3-1)=2$. Total squares = 8. Non-square rectangles = 18 (from Q3) – $8 = 10$.

Q5. [Medium] 5 straight lines are drawn on a plane such that no two are parallel and no three meet at a point. How many points of intersection are formed?

A. 5 B. 10 C. 15 D. 20

Answer: B. **Why:** $C(5,2) = 5 \times 4 / 2 = 10$.

Q6. [Medium] [Trap] Out of 7 lines drawn (no coincidences), 3 of them are mutually parallel. How many intersection points result?

A. 21 B. 18 C. 15 D. 12

Answer: B. **Why:** Naive total $C(7,2)=21$. The 3 parallel lines contribute 0 intersections among themselves instead of $C(3,2)=3$, so subtract 3: $21-3=18$.

Q7. [Hard] A rectangular grid has 4 rows and 6 columns of unit cells. Find the total number of squares of all sizes.

A. 24 B. 40 C. 50 D. 55

Answer: C. **Why:** $\min(m,n)=4$. Sizes: 1×1 : $4 \times 6=24$; 2×2 : $3 \times 5=15$; 3×3 : $2 \times 4=8$; 4×4 : $1 \times 3=3$. Total = $24+15+8+3 = 50$.

Q8. [Hard] In triangle ABC, a cevian is drawn from vertex A to a point D on side BC, and a second cevian is drawn from vertex B to a point E on side AC, with AD and BE crossing at an interior point F. How many total triangles are visible in the resulting figure?

A. 4 B. 6 C. 8 D. 10

Answer: C. **Why:** Systematically list every 3-sided region using only the 5 drawn lines (AB, BC, CA, AD, BE): the 3 basic regions **ABF**, **BFD**, **AFE** (the 4th basic region, quadrilateral FDCE, is not a triangle); the 2 half-splits from each single cevian, **ABD**, **ACD** (from AD) and **ABE**, **CBE** (from BE) — but ABD is a triangle formed by AB+BD(part of BC)+DA, so counting only NEW ones beyond the halves already listed: ABD, ACD, ABE, CBE (4 more); plus the outer triangle **ABC** itself (1 more). Total = 3 (basic) + 4 (single-cevian halves) + 1 (outer) = **8**. **TRAP:** it's easy to stop after counting only the "obvious" pieces (the 3 basic regions + outer = 4) and miss that each cevian ALSO independently creates 2 more valid half-triangles using the opposite side — always check triangles formed by just ONE cevian at a time, not only the fully-subdivided basic pieces.

Q9. [Hard] 8 straight lines are drawn such that no two are parallel, but there is one point where exactly 4 of them are concurrent (and no other coincidences). How many distinct intersection points are there?

A. 22 B. 25 C. 28 D. 23

Answer: D. **Why:** Naive total = $C(8,2) = 28$. The 4 concurrent lines would naively contribute $C(4,2)=6$ points but actually give only 1, so subtract $(6-1)=5$. Distinct points = $28-5 = 23$.

9.D5 Cubes & Dice

D5.1 Standard Dice Basics [Level: Basic]

- A standard (ordinary) six-faced die has faces numbered 1–6.
- Default assumption unless stated otherwise:** opposite faces always sum to **7** — so the three opposite pairs are **(1,6), (2,5), (3,4)**.
- A cube always has exactly **3 pairs of opposite faces** (commonly called Top-Bottom, Front-Back, Left-Right) — no matter how it's numbered or oriented.
- TRAP:** This "sum to 7" rule applies only to a **standard** die. Many reasoning questions use a **non-standard** die (arbitrary numbers) and expect you to *derive* the opposite pairs from given views — never assume sum-to-7 unless the question says "standard dice."

D5.2 Finding Opposite Faces from Multiple Views [Level: Medium]

- Core rule: any two faces shown **simultaneously visible** in the same view can **never** be opposite each other (you can never see two opposite faces of a cube at the same time).
- Ways to solve** — two complementary techniques:
 - Direct elimination:** for each view, rule out every pair of simultaneously-visible faces as "not opposite."
 - Elimination by remainder:** since there are exactly 3 opposite pairs among 6 numbers, once enough pairs are ruled out, the numbers left unpaired must be opposite each other **by process of elimination** — this is usually what actually solves the question, not the direct rule alone.

Worked Example

Two views of the same (non-standard) die are shown:

View 1: Top = 2, Front = 3, Right = 4

View 2: Top = 2, Front = 4, Right = 6

Find the face opposite to 3.

From View 1: {2,3,4} are mutually visible → none of these pairs are opposite: $2 \neq \text{opp}(3)$, $2 \neq \text{opp}(4)$, $3 \neq \text{opp}(4)$.

From View 2: {2,4,6} are mutually visible → $2 \neq \text{opp}(4)$ [already known], $2 \neq \text{opp}(6)$, $4 \neq \text{opp}(6)$.

So: $\text{opp}(2)$ must be from the untested remainder {1,5}. Likewise $\text{opp}(4)$ must be from {1,5} too (since 4 is ruled out against 2,3,6).

Since 2 and 4 cannot share the *same* opposite number, one of {2,4} pairs with 1 and the other with 5 — using up all four numbers {1,2,4,5} in two pairs.

That leaves exactly **3 and 6** unpaired → **by elimination, 3 is opposite 6**.

D5.3 Rolling / Rotating a Die [Level: Medium]

- Track all six faces (Top/Bottom/North/South/East/West) and update them after each roll:

Roll to the RIGHT: Top → East → Bottom → West → Top (North/South faces stay fixed)
 Roll FORWARD: Top → North → Bottom → South → Top (East/West faces stay fixed)

- TRICK: Only the two faces perpendicular to the rolling axis stay fixed each time — track just the other four.

D5.4 Dice Nets (Open Dice / Unfolding) [Level: Medium-Hard]

- A net is a flat, unfolded pattern of 6 squares that folds into a cube.
- TRICK (master rule):** in the common "cross" or "T" shaped net, two faces that lie in the **same row or column, exactly 2 squares apart (with exactly one square between them)** become **opposite** faces when folded.

Worked Example

Net layout (row and column of a cross-shaped net):

```

      [2]
[3] [1] [4] [6]
      [5]
```

Row (3,1,4,6): squares 2 apart are (3,4) and (1,6) → **3 & 4 are opposite; 1 & 6 are opposite**.

Column (2,1,5): squares 2 apart are (2,5) → **2 & 5 are opposite**.

(Sanity check: this is exactly the standard 1-6/2-5/3-4 pairing — nets built around a standard die will always reproduce it.)

D5.5 Cubes: The Painted-Cube Formula Set [Level: Basic-Hard]

- Setup: a large cube made of $n \times n \times n$ unit cubes has its entire outer surface painted, then is disassembled into unit cubes.
- This single reference block is the highest-yield content in the whole chapter — memorize it cold:**

```

Unit cubes with 3 faces painted (corners)      = 8                (always 8, for any n >= 2)
Unit cubes with 2 faces painted (edges)        = 12 x (n-2)
Unit cubes with 1 face painted (face centers)   = 6 x (n-2)^2
Unit cubes with 0 faces painted (fully inside) = (n-2)^3
```

Check: $8 + 12(n-2) + 6(n-2)^2 + (n-2)^3 = n^3$ (always — verify for $n=4$: $8+24+24+8=64=4^3$ (checked))

- TRAP:** These formulas require $n \geq 2$. For $n=1$ (a single unit cube), the entire cube is one piece with all 6 faces painted — none of the four categories above apply.
- TRICK for "at least" / "at most" questions: compute each category separately from the reference block, then **add** the relevant ones — e.g., "at least 2 faces painted" = (3-face count) + (2-face count).
- Variant: if different **pairs of opposite faces** are painted with different colors, the position-counting logic (corner/edge/face/internal) is identical — only the color labels on each category change, not the counts.

Questions

(ordered Easy → Hard; [Trick] tests a shortcut, [Trap] tests a classic misconception)

Q1. [Easy] On a standard die, if 1 is on top, what number is at the bottom?

A. 2 B. 5 C. 6 D. 4

Answer: C. **Why:** Standard opposite pairs sum to 7: opposite of 1 is 6.

Q2. [Easy] [Numerical] A cube of side 3 units ($n=3$) is painted on all faces and cut into 27 unit cubes. How many unit cubes have exactly 2 faces painted?

A. 8 B. 12 C. 6 D. 1

Answer: B. **Why:** Edge count = $12(n-2) = 12 \times 1 = 12$.

Q3. [Medium] [Numerical] A cube of side 5 units is painted and cut into 125 unit cubes. How many have exactly 1 face painted?

A. 24 B. 36 C. 54 D. 64

Answer: C. **Why:** Face count = $6(n-2)^2 = 6 \times (3)^2 = 6 \times 9 = 54$.

Q4. [Medium] [Numerical] A cube of side 6 units is painted and cut into unit cubes. How many have NO paint at all?

A. 27 B. 64 C. 16 D. 8

Answer: B. **Why:** Internal count = $(n-2)^3 = 4^3 = 64$.

Q5. [Medium] [Trap] A cube of side 4 units is painted and cut into 64 unit cubes. How many have AT LEAST one face painted?

A. 8 B. 24 C. 32 D. 56

Answer: D. **Why:** "At least 1" = total – (0-face count) = $64 - (4-2)^3 = 64 - 8 = 56$. (Or directly: corners+edges+faces = $8 + 12(2) + 6(2)^2 = 8 + 24 + 24 = 56$.)

Q6. [Medium] [Numerical] A cube of side 5 units is painted and cut into unit cubes. How many have AT MOST 1 face painted (i.e., 0 or 1 face)?

A. 54 B. 27 C. 81 D. 62

Answer: C. **Why:** 0-face = $(3)^3 = 27$; 1-face = $6 \times (3)^2 = 54$. Sum = $27 + 54 = 81$.

Q7. [Hard] Two views of a non-standard die are shown: View 1: Top=2, Front=3, Right=4. View 2: Top=2, Front=4, Right=6. What number is opposite to 3?

A. 1 B. 5 C. 6 D. Cannot be determined

Answer: C. **Why:** {2,3,4} mutually visible in View 1 and {2,4,6} in View 2 rule out all pairings among 2,3,4,6. Numbers 2 and 4 must each pair with one of the remaining {1,5}, leaving 3 and 6 — the only unpaired numbers — opposite each other by elimination.

Q8. [Hard] [Trick] A cube of side 10 units is painted on all faces and cut into 1000 unit cubes. How many unit cubes have exactly 2 faces painted?

A. 8 B. 96 C. 480 D. 64

Answer: B. **Why:** Edge count = $12(n-2) = 12 \times 8 = 96$.

Q9. [Hard] A cube's net (unfolded pattern) is laid out with squares labeled, in a row, as P-Q-R-S (four in a straight line), and squares T and U attached above and below Q respectively. Which faces become opposite when folded?

A. P & R, Q & S, T & U B. P & S, Q & U, R & T C. P & Q, R & S, T & U D. P & R only; T,U,Q,S indeterminate

Answer: A. **Why:** In the row P-Q-R-S, the pairs exactly 2 apart are (P,R) and (Q,S). In the column T-Q-U, the pair 2 apart is (T,U). All three opposite pairs are determined by the "2-apart in a straight line" net rule.

Q10. [Hard] [Trap] A cube of side 2 units is painted on all faces and cut into 8 unit cubes. How many have exactly 1 face painted?

A. 0 B. 6 C. 8 D. 4

Answer: A. **Why:** For $n=2$, $(n-2)=0$, so the 1-face formula $6(n-2)^2 = 6 \times 0 = 0$. In fact for $n=2$, every unit cube is a corner cube (all 8 have exactly 3 faces painted) — a boundary case worth checking formulas against directly rather than assuming they always give a "typical" spread.

Q11. [Hard] A large cube is painted and cut into 216 unit cubes. What was the value of n , and how many unit cubes have exactly 2 faces painted?

A. $n=6$; 96 B. $n=6$; 48 C. $n=5$; 54 D. $n=6$; 60

Answer: B. **Why:** $216 = 6^3$, so $n=6$. Edge count = $12(n-2) = 12 \times 4 = 48$.

Rapid Revision Round

A dense, module-by-module digest of the highest-yield facts, formulas, and trap points from this entire book — meant for a fast final sweep in the last day or two before the exam, not a first read. If a line here doesn't immediately make sense, that's a signal to go back and re-read that chapter, not to memorize it blind.

Module 1 — Digital Logic & Computer Organization

- 2's complement, n bits: range $-2^{(n-1)}$ to $+2^{(n-1)-1}$; overflow = same-sign operands → opposite-sign result.
- Gray code: exactly one bit changes between adjacent values (not "fewer bits").
- NAND and NOR are the only single-gate-type universal gates. Duality swaps operators/constants only, never variables.
- K-map cell order is Gray-coded so adjacency = 1-bit difference; EPI = covers a minterm no other PI covers.
- Ripple carry = slow but cheap; carry look-ahead = fast but more hardware. MUX = many→one (select lines pick an input); DEMUX = one→many.
- SR flip-flop's forbidden state is $S=R=1$; JK fixes it by making $S=R=1$ mean "toggle." Latch = level-triggered; flip-flop = edge-triggered.
- Ring counter (n flip-flops) → n states; Johnson counter (n flip-flops) → 2n states.
- PC = address of NEXT instruction; IR = CURRENT instruction being executed (commonly swapped).
- Indirect addressing = 1 extra memory access vs. direct addressing. RISC = Load/Store architecture (ALU never touches memory directly) — the real defining feature, not just "fewer instructions."
- Address bus width → max addressable memory (2^n); data bus width → transfer size per cycle, NOT memory size.
- Cache: direct-mapped (1 line only, most conflict misses) vs. fully associative (any line, no conflict misses, costly hardware) vs. set-associative (compromise). Only write-back needs a dirty bit.
- DMA: CPU sets up the transfer and is interrupted at completion — not involved per-word, but not fully absent either.
- Pipelining boosts throughput, not single-instruction latency. RAW = true dependency (occurs even in-order); WAR/WAW = false dependencies (only matter out-of-order). Load-use hazard needs ≥ 1 stall cycle even with perfect forwarding.

Module 2 — Programming, Data Structures & Algorithms

- Complete binary tree edge/node counts, BST traversal reconstruction (need in-order + one of pre/post-order to uniquely rebuild a tree), and recurrence-solving (Master theorem cases) are recurring numerical favorites — re-derive them by hand, don't just recognize the shape.
- Best/Average/Worst case complexity must always be matched to what the question actually asks (quicksort worst $O(n^2)$ vs average $O(n \log n)$ is the single most common complexity mix-up).
- Graph algorithms: Dijkstra fails with negative edge weights (use Bellman-Ford); Prim's and Kruskal's both give a valid MST but may differ in which one if edge weights tie.
- Stack = LIFO (recursion, expression evaluation); Queue = FIFO. Hashing: collision resolution method (chaining vs. open addressing) changes worst-case lookup behavior significantly.

Module 3 — Theory of Computation & Automata

- DFA and NFA are EQUALLY powerful (both = regular languages); nondeterminism doesn't add expressive power, only construction convenience. NFA→DFA subset construction: worst case 2^n states.
- Pumping lemma (regular): used ONLY to disprove regularity, never to prove it; $|xy| \leq p$ constrains y to the first p characters.
- Pumping lemma (CFL): pumps v and y TOGETHER, by the same count — not independently.
- $\{a^n b^n\}$ IS context-free; $\{a^n b^n c^n\}$ is NOT (one stack can enforce one count-match, not two simultaneously).
- Regular languages closed under intersection/complement; CFLs are NOT (except CFL $\cap$ Regular, which IS always context-free).
- Chomsky hierarchy: Type 0 = MOST powerful (unrestricted), Type 3 = LEAST powerful (regular) — numbering is backwards from intuition.
- NP = "Nondeterministic Polynomial" (fast verification), NOT "Not Polynomial." $P \subseteq NP$ is proven; $P=NP$ is open. NP-Hard need not be in NP; NP-Complete must be.
- Halting Problem: undecidable overall, but many SPECIFIC instances are perfectly decidable — undecidability is about no universal algorithm, not "unknowable in every case."

Module 4 — Compiler Design

- Lexical analysis catches only token-formation errors, never grammar (syntax) or meaning (semantic) errors. Lexical = regular-language level; parsing = context-free level.
- Left recursion must be eliminated before top-down/LL parsing (infinite recursion otherwise); left-recursive or ambiguous grammars can never be LL(1).
- Parser power ordering: $LR(0) \subset SLR(1) \subset LALR(1) \subset CLR(1)$ — LALR(1) has SLR(1)'s table size but CLR(1)-level per-item lookahead precision (why YACC/Bison default to it).
- Bottom-up parsing traces a rightmost derivation in REVERSE.
- Triples (position-referenced results) are harder to reorder than quadruples (named results) during optimization.
- CSE (reuse a repeated computed expression) $\neq$ constant propagation (substitute a known compile-time value) — different techniques, often confused.
- Control/dynamic link → caller at runtime; Access/static link → lexically enclosing scope — they diverge exactly in recursion/nested-function cases, which is where exams target this.
- Static allocation cannot support recursion (same fixed address reused, clobbering recursive calls); stack allocation naturally supports it.

Module 5 — Operating Systems

- Multitasking $\approx$ Time-sharing (same concept), not unrelated categories. Microkernel = more reliable/extensible but slower; Monolithic = faster but less modular.
- A system call uses a trap (mode bit changes); an ordinary function call never changes the mode bit.
- Max 1 process Running at a time on a single CPU (P CPUs → max P Running), regardless of total process count n.
- Long-term scheduler controls degree of multiprogramming and process mix balance; short-term (CPU) scheduler must be very fast.
- Context switch = pure overhead, no useful work done. Preemptive scheduling can cause race conditions that non-preemptive avoids (but at throughput/responsiveness cost).
- Deadlock needs all 4 conditions simultaneously: mutual exclusion, hold-and-wait, no preemption, circular wait — breaking even ONE prevents deadlock.
- Banker's algorithm needs advance knowledge of maximum resource claims — a common "why can't we just always use Banker's" trap.
- Belady's anomaly: more page frames can INCREASE page faults under FIFO (not LRU/Optimal) — a classic counter-intuitive numerical trap.

- Internal fragmentation = wasted space WITHIN an allocated block (paging); external fragmentation = wasted space BETWEEN allocated blocks (segmentation/variable partitioning).

Module 6 — Database Management Systems

- 1NF: atomic values only. 2NF: no partial dependency on part of a composite key. 3NF: no transitive dependency. BCNF: every determinant is a candidate key (stricter than 3NF) — boundary cases between 3NF and BCNF are a favorite trap.
- Lossless-join decomposition requires the common attribute to be a superkey of at least one resulting relation — verify this explicitly, don't assume any decomposition is safe.
- Serializability $\neq$ Serial execution: serializable means EQUIVALENT to some serial order, not that it literally runs one transaction at a time.
- 2-Phase Locking (2PL) guarantees conflict-serializability, but doesn't prevent deadlock by itself (Strict 2PL additionally prevents cascading rollback).
- Primary index vs. secondary index: primary requires the file to be physically sorted on that key; secondary does not.
- B+ trees: all data pointers at leaf level, internal nodes hold only keys for navigation — this is why range queries are efficient (leaf-level linked list traversal).

Module 7 — Computer Networks & Cybersecurity

- OSI 7 layers vs. TCP/IP 4-5 layers — know exactly which OSI layers merge into the TCP/IP model. Layer-to-protocol mapping (e.g., ARP at layer 2/3 boundary) is a recurring trap.
- TCP = connection-oriented, reliable, ordered (3-way handshake); UDP = connectionless, unreliable, no ordering guarantee, but low overhead — the "why use UDP at all" answer is always low latency/overhead for loss-tolerant use cases (DNS, streaming, VoIP).
- Subnetting numerals: always double-check whether the question wants usable hosts ($2^n - 2$ for network+broadcast reserved) or total addresses (2^n).
- Symmetric encryption (single shared key, fast, used for bulk data) vs. Asymmetric/public-key (key pair, slower, used for key exchange/signatures) — HTTPS uses BOTH (asymmetric for handshake/key exchange, symmetric for the actual session data).
- A digital signature provides authentication + integrity + non-repudiation; encryption alone provides only confidentiality — don't conflate the two guarantees.
- Distance Vector routing (e.g., RIP) suffers from the count-to-infinity problem; Link State (e.g., OSPF) does not, at the cost of more overhead/complexity.

Module 8 — Software Engineering

- Verification = "are we building the product right?" (internal consistency); Validation = "are we building the right product?" (matches actual need) — the single most swapped SE fact on any exam.
- Waterfall has no built-in feedback loop to earlier phases; defects caught later cost exponentially more to fix.
- Scrum Master = facilitator/coach, NOT a task-assigning manager; Product Owner owns the backlog.
- Cyclomatic complexity $V(G) = E - N + 2P = (\text{decision points}) + 1$ — gives the minimum number of independent test paths needed.
- COCOMO: both the coefficient AND the exponent increase from Organic $\rightarrow$ Semi-detached $\rightarrow$ Embedded (effort grows super-linearly, increasingly so).
- Critical path = LONGEST path through the project network (not shortest) — determines minimum project duration; critical-path activities have zero slack.
- High cohesion + Low coupling = design goal. Aggregation = part can outlive the whole; Composition = part cannot.
- Stub = dummy CALLED module (top-down integration); Driver = dummy CALLING module (bottom-up integration) — commonly swapped.
- 100% statement coverage does NOT guarantee 100% branch coverage (an if with no else is the classic counterexample).
- Perfective maintenance (enhancements), not corrective (bug-fixing), is usually the LARGEST maintenance cost category in real data.

Module 9 — General Aptitude & Reasoning

- Negative-marking math (see front matter): a pure blind guess is roughly break-even (+0.0025 EV); eliminating even one option makes attempting clearly worthwhile (+0.11 EV) — don't over-skip out of fear of negative marking.
- Percentage/profit-loss/SI-CI: convert to the fraction-percentage table mentally rather than long division — speed matters more than method elegance here.
- Data Interpretation (2-mark questions): read the whole table/graph once, extract every number you'll plausibly need, THEN answer all sub-questions on it — don't re-read the table per sub-question.
- Syllogisms/statement reasoning: the trap is almost always a misread quantifier ("all," "some," "no," "not") — re-read the premise before touching the options.
- Blood relations/seating arrangements: draw the diagram/table; don't try to hold the relationships purely in your head under time pressure.
- Non-verbal: mirror image = left-right flip about a vertical axis; water image = upside-down + left-right flip (top-bottom flip) — these are frequently confused with each other.

End of the Master Question Bank. Good luck — go back to any module above for the full concept notes and complete MCQ sets whenever a line above doesn't immediately click.